

A/B Testing: Statistical Foundations and Practical Implementation

Diogo Ribeiro
ESMAD - Instituto Politécnico do Porto

November 20, 2025

Contents

List of Abbreviations	xxi
------------------------------	------------

Preface	xxvii
----------------	--------------

1 Introduction to A/B Testing	1
1.1 Learning Objectives	1
1.2 What is A/B Testing?	1
1.2.1 Definition and Core Concepts	1
1.2.2 From Agricultural Fields to Digital Platforms	2
1.2.3 A/B Testing vs. Other Methodologies	3
1.3 Why A/B Testing Matters	5
1.3.1 The Business Case for Experimentation	5
1.3.2 Evidence-Based Culture	6
1.3.3 Examples from Industry Leaders	6
1.4 Basic Terminology and Concepts	7
1.4.1 Experimental Units and Assignment	7
1.4.2 Control and Treatment Groups	7
1.4.3 Metrics and Key Performance Indicators	8
1.4.4 Statistical Significance	8
1.4.5 Effect Size	9
1.5 Types of A/B Tests	9
1.5.1 Simple A/B Tests	9
1.5.2 A/B/n Tests (Multiple Variants)	9
1.5.3 Multivariate Tests (Factorial Designs)	10
1.5.4 Sequential Testing	10
1.5.5 Multi-Armed Bandits	11
1.6 Real-World Examples	11
1.6.1 Website Conversion Optimization	11
1.6.2 Email Marketing Campaigns	13
1.6.3 Product Feature Testing	15
1.6.4 Pricing Experiments	16
1.7 When A/B Testing Works (and When It Doesn't)	18
1.7.1 Ideal Scenarios for A/B Testing	18
1.7.2 When A/B Testing Struggles	18
1.8 Summary	19
1.9 Further Reading	19
1.10 Exercises	19
1.10.1 Conceptual Questions	19

1.10.2	Quantitative Problems	20
1.10.3	Programming Exercises	20
1.10.4	Critical Thinking	21
2	Statistical Foundations	23
2.1	Learning Objectives	23
2.2	Introduction	23
2.3	Probability Theory Fundamentals	24
2.3.1	Probability Spaces and Random Variables	24
2.3.2	Expected Value and Variance	24
2.3.3	The Central Limit Theorem	27
2.3.4	Law of Large Numbers	30
2.4	Key Probability Distributions for A/B Testing	31
2.4.1	Bernoulli and Binomial Distributions	31
2.4.2	Normal (Gaussian) Distribution	32
2.4.3	Student's t-Distribution	32
2.4.4	Beta Distribution	33
2.5	Statistical Inference	36
2.5.1	Population vs. Sample	36
2.5.2	Sampling Distributions	36
2.5.3	Standard Error	37
2.5.4	Confidence Intervals	37
2.6	Hypothesis Testing Framework	42
2.6.1	The Logic of Hypothesis Testing	42
2.6.2	Test Statistics and p-values	43
2.6.3	Significance Level and Decision Rule	43
2.6.4	Type I and Type II Errors	44
2.6.5	Two-Sample Z-Test for Proportions	44
2.7	Effect Size Measures	50
2.7.1	Why Effect Size Matters	50
2.7.2	Absolute vs. Relative Differences	50
2.7.3	Cohen's h for Proportions	51
2.7.4	Odds Ratio	51
2.7.5	Cohen's d for Continuous Metrics	52
2.7.6	Practical Guidelines for Effect Size Reporting	56
2.8	Frequentist vs. Bayesian Inference	57
2.8.1	Frequentist Framework	57
2.8.2	Bayesian Framework	58
2.8.3	When to Use Each Approach	59
2.9	Summary	59
2.10	Further Reading	60
2.11	Exercises	60
2.11.1	Conceptual Questions	60
2.11.2	Probability and Distributions	60
2.11.3	Hypothesis Testing	61
2.11.4	Confidence Intervals	61
2.11.5	Effect Sizes	62
2.11.6	Programming Projects	62

3	Experimental Design Principles	65
3.1	Learning Objectives	65
3.2	Introduction	65
3.3	Fundamental Principles of Experimental Design	66
3.3.1	Randomization	66
3.3.2	Control Groups	67
3.3.3	Blocking and Stratification	67
3.3.4	Blinding	68
3.4	Randomization Techniques	69
3.4.1	Simple Randomization	69
3.4.2	Block Randomization	73
3.4.3	Stratified Randomization	75
3.4.4	Cluster Randomization	80
3.5	Confounding Variables and Bias	86
3.5.1	Selection Bias	86
3.5.2	Survivorship Bias	87
3.5.3	Simpson's Paradox	87
3.5.4	Network Effects	92
3.6	Validity Considerations	92
3.6.1	Internal Validity	93
3.6.2	External Validity	93
3.6.3	Construct Validity	94
3.6.4	Statistical Conclusion Validity	95
3.7	Practical Design Decisions	95
3.7.1	Unit of Randomization	95
3.7.2	Temporal Considerations	96
3.7.3	Geographic Considerations	96
3.7.4	Platform-Specific Issues	97
3.8	Summary	97
3.9	Further Reading	97
3.10	Exercises	98
3.10.1	Conceptual Questions	98
3.10.2	Design Exercises	98
3.10.3	Implementation Exercises	99
3.10.4	Analysis Exercises	99
3.10.5	Simulation Projects	100
4	Sample Size Calculation and Statistical Power	101
4.1	Learning Objectives	101
4.2	Introduction	101
4.3	Power Analysis Fundamentals	101
4.3.1	Definition of Statistical Power	102
4.3.2	Factors Affecting Power	102
4.3.3	The Type I and Type II Error Tradeoff	103
4.3.4	Power Curves and Interpretation	103
4.4	Sample Size Formulas	107
4.4.1	Sample Size for Proportions (Conversion Rates)	107
4.4.2	Sample Size for Means (Continuous Metrics)	111

4.4.3	Sample Size for Survival Data	115
4.4.4	Sample Size for Ratio Metrics	115
4.5	Practical Considerations	116
4.5.1	Minimum Detectable Effect (MDE)	116
4.5.2	Business Significance vs. Statistical Significance	119
4.5.3	Cost-Benefit Analysis of Sample Sizes	120
4.5.4	Duration Estimation	121
4.6	Advanced Topics	122
4.6.1	Unequal Sample Sizes	122
4.6.2	Sequential Testing Implications	122
4.6.3	Multiple Metrics Considerations	123
4.7	Implementation Tools	123
4.7.1	Python Implementation	123
4.7.2	R Implementation	126
4.8	Summary	128
4.9	Further Reading	129
4.10	Exercises	129
4.10.1	Conceptual Questions	129
4.10.2	Calculation Exercises	129
4.10.3	Programming Exercises	130
4.10.4	Applied Exercises	131
4.10.5	Advanced Projects	132
5	Hypothesis Testing for A/B Tests	133
5.1	Learning Objectives	133
5.2	Classical Hypothesis Testing Framework	133
5.2.1	Structure of a Hypothesis Test	133
5.2.2	One-Sided vs. Two-Sided Tests	134
5.3	Tests for Proportions	134
5.3.1	Two-Sample Z-Test for Proportions	134
5.3.2	Confidence Interval for Difference in Proportions	135
5.3.3	Assumptions for Z-Test	139
5.4	Tests for Means	140
5.4.1	Two-Sample T-Test	140
5.5	Chi-Squared Test for Categorical Data	146
5.5.1	Pearson's Chi-Squared Test	146
5.5.2	Fisher's Exact Test	147
5.6	Understanding P-Values	151
5.6.1	Correct Interpretation	151
5.6.2	Common Misinterpretations	152
5.6.3	Statistical Significance vs. Practical Significance	152
5.7	Assumptions and Diagnostics	152
5.7.1	Normality Testing	153
5.7.2	Variance Homogeneity	153
5.7.3	Independence Assumptions	154
5.7.4	Outlier Detection and Treatment	154
5.8	Non-Parametric Alternatives	161
5.8.1	Mann-Whitney U Test	162

5.8.2	Permutation Tests	162
5.8.3	Bootstrap Methods	163
5.8.4	When to Use Non-Parametric Tests	171
5.9	Test Selection Framework	171
5.10	Common Pitfalls and How to Avoid Them	175
5.10.1	Pitfall 1: Peeking at Results	175
5.10.2	Pitfall 2: Multiple Testing Without Correction	175
5.10.3	Pitfall 3: Misinterpreting P-Values	176
5.10.4	Pitfall 4: Ignoring Assumptions	176
5.10.5	Pitfall 5: Conflating Statistical and Practical Significance	176
5.11	Summary	176
5.12	Further Reading	176
5.13	Exercises	177
5.13.1	Conceptual Questions	177
5.13.2	Calculation Exercises	177
5.13.3	Programming Exercises	177
5.13.4	Applied Exercises	178
5.13.5	Advanced Projects	178
6	Multiple Testing and False Discovery Rate	181
6.1	The Multiple Testing Problem	181
6.1.1	The Birthday Paradox of Hypothesis Testing	181
6.1.2	Family-Wise Error Rate (FWER)	182
6.1.3	False Discovery Rate (FDR)	182
6.1.4	Examples of Multiple Testing in A/B Contexts	183
6.2	Correction Methods	183
6.2.1	Bonferroni Correction	183
6.2.2	Holm-Bonferroni Method	184
6.2.3	Benjamini-Hochberg Procedure	185
6.2.4	Benjamini-Yekutieli Method	186
6.2.5	Comparison of Basic Correction Methods	186
6.3	Advanced Approaches	186
6.3.1	Closed Testing Procedures	186
6.3.2	Gatekeeping Strategies	187
6.3.3	Adaptive Procedures	191
6.3.4	Bayesian Multiple Testing	193
6.4	Practical Applications	196
6.4.1	Multiple Metrics Testing	196
6.4.2	Subgroup Analysis	197
6.4.3	Sequential Testing Considerations	200
6.4.4	Platform-Wide Experiment Programs	201
6.5	Implementation Strategies	201
6.5.1	Pre-Planned vs. Exploratory Analysis	201
6.5.2	Hierarchical Testing	207
6.5.3	Primary vs. Secondary Endpoints	207
6.6	Guidelines for Experiment Programs	209
6.6.1	Organizational Best Practices	209
6.6.2	Decision-Making Framework	210

6.7	Summary	212
6.8	Further Reading	212
6.9	Exercises	213
6.9.1	Conceptual Questions	213
6.9.2	Calculation Exercises	213
6.9.3	Programming Exercises	214
6.9.4	Applied Exercises	214
6.9.5	Advanced Exercises	215
7	Bayesian Approaches to A/B Testing	217
7.1	Learning Objectives	217
7.2	Introduction: Why Bayesian A/B Testing?	217
7.3	Bayesian Fundamentals	218
7.3.1	Bayes' Theorem and Its Applications	218
7.3.2	Prior Distributions and Their Selection	219
7.3.3	Posterior Distributions	221
7.4	Bayesian A/B Testing Framework	223
7.4.1	Beta-Binomial Model for Conversion Rates	223
7.4.2	Normal-Normal Model for Continuous Metrics	228
7.4.3	Comparing Two Variants: Bayesian A/B Testing	231
7.4.4	Expected Loss and Risk-Based Decisions	234
7.4.5	Conjugate Priors and Analytical Solutions	238
7.4.6	Non-Conjugate Priors and MCMC	239
7.4.7	Implementation with PyMC3	241
7.4.8	Implementation with Stan	244
7.5	Practical Advantages of Bayesian A/B Testing	246
7.5.1	Intuitive Probability Statements	246
7.5.2	Incorporation of Prior Knowledge	246
7.5.3	Early Stopping Decisions	247
7.5.4	Risk-Based Decision Making	249
7.5.5	Full Uncertainty Quantification	250
7.6	Comparison with Frequentist Methods	250
7.6.1	Philosophical Differences	250
7.6.2	Practical Implications	250
7.6.3	When to Use Each Approach	251
7.6.4	Empirical Comparison Study	252
7.6.5	Computational Considerations	254
7.7	Summary	255
7.8	Further Reading	256
7.9	Exercises	256
8	Advanced Experimental Designs	259
8.1	Learning Objectives	259
8.2	Introduction: Beyond Traditional A/B Testing	259
8.3	Factorial Designs	260
8.3.1	Full Factorial Designs	260
8.3.2	Statistical Model for Factorial Experiments	260
8.3.3	Fractional Factorial Designs	267

8.4	Multi-Armed Bandits	267
8.4.1	The Exploration-Exploitation Tradeoff	268
8.4.2	ϵ -Greedy Algorithm	268
8.4.3	Upper Confidence Bound (UCB) Algorithm	269
8.4.4	Thompson Sampling (Bayesian Bandit)	269
8.4.5	Contextual Bandits	270
8.4.6	When to Use Bandits vs. Traditional A/B Tests	274
8.5	Sequential Testing	274
8.5.1	The Peeking Problem	275
8.5.2	Sequential Probability Ratio Test (SPRT)	275
8.5.3	Group Sequential Designs	279
8.5.4	Practical Implementation Challenges	282
8.6	Quasi-Experimental Designs	282
8.6.1	When Randomization Fails	283
8.6.2	Difference-in-Differences (DID)	283
8.6.3	Regression Discontinuity Design (RDD)	287
8.6.4	Instrumental Variables (IV)	287
8.7	Platform-Specific Considerations	288
8.7.1	Network Effects and Interference	288
8.7.2	Two-Sided Marketplaces	289
8.7.3	Mobile App Testing	289
8.7.4	Cross-Platform Experiments	290
8.8	Summary	290
8.9	Further Reading	291
8.10	Exercises	291
8.10.1	Factorial Designs	291
8.10.2	Multi-Armed Bandits	291
8.10.3	Sequential Testing	292
8.10.4	Quasi-Experimental Designs	292
8.10.5	Applied Projects	292
9	Technical Implementation	293
9.1	Learning Objectives	293
9.2	Introduction: From Theory to Production	293
9.3	Experiment Infrastructure	294
9.3.1	Feature Flags and Configuration Management	294
9.3.2	Random Assignment Systems	300
9.3.3	Data Collection Architecture	303
9.3.4	Real-Time Monitoring	306
9.4	Data Engineering Considerations	310
9.4.1	ETL Pipelines for Experiment Data	310
9.4.2	Data Quality Checks	318
9.4.3	Storage and Retrieval Systems	322
9.4.4	Privacy and Compliance	322
9.5	Statistical Computing	324
9.5.1	Python Libraries Ecosystem	324
9.5.2	R Packages for Experimentation	330
9.5.3	Big Data Considerations with Spark	331

9.6	Automation and Scaling	334
9.6.1	Automated Statistical Tests	334
9.6.2	Alert Systems	337
9.6.3	Experiment Management API	339
9.7	Code Architecture	342
9.7.1	Object-Oriented Design for Experiments	342
9.7.2	Testing and Validation Frameworks	345
9.7.3	Documentation and Reproducibility	347
9.8	Summary	349
9.9	Further Reading	350
9.10	Exercises	350
10	Data Analysis and Result Interpretation	353
10.1	Learning Objectives	353
10.2	Introduction: From Data to Decisions	353
10.3	Data Preprocessing	354
10.3.1	Data Quality Assessment	354
10.3.2	Outlier Detection and Treatment	362
10.3.3	Missing Data Handling	370
10.3.4	Data Transformation Techniques	376
10.3.5	Quality Assurance Checks	379
10.4	Exploratory Data Analysis	382
10.4.1	Descriptive Statistics	382
10.4.2	Visualization Techniques	385
10.4.3	Distribution Analysis	390
10.4.4	Trend and Seasonality Detection	393
10.5	Statistical Analysis Pipeline	397
10.5.1	Assumption Checking	397
10.5.2	Test Execution	400
10.5.3	Effect Size Calculation	406
10.5.4	Confidence Interval Estimation	409
10.6	Result Interpretation	413
10.6.1	Statistical vs. Practical Significance	413
10.6.2	Segment Analysis	416
10.6.3	Long-term Effects Consideration	419
10.7	Reporting and Communication	420
10.7.1	Stakeholder Communication Strategies	420
10.7.2	Visualization Best Practices	420
10.7.3	Executive Summary Template	424
10.8	Summary	428
10.9	Further Reading	428
10.10	Exercises	429
11	Case Studies and Real-World Applications	431
11.1	Learning Objectives	431
11.2	Introduction: From Theory to Practice	431
11.3	Case Study 1: Google Search Results and Page Load Time	432
11.3.1	Context and Business Problem	432

11.3.2	Experimental Design	432
11.3.3	Results and Analysis	433
11.3.4	Deep Dive: Causal Mechanism	433
11.3.5	Statistical Analysis Framework	433
11.3.6	Lessons Learned	437
11.4	Case Study 2: E-commerce Checkout Optimization	437
11.4.1	Context	437
11.4.2	Experimental Design	437
11.4.3	Overall Results	438
11.4.4	Segmentation Analysis	438
11.4.5	Decision Framework	442
11.4.6	Post-Launch Monitoring	443
11.4.7	Lessons Learned	443
11.5	Case Study 3: Netflix Thumbnail Personalization	443
11.5.1	Context	443
11.5.2	Technical Challenge	443
11.5.3	Multi-Stage Experimental Approach	443
11.5.4	Experimental Design Details (Stage 2)	444
11.5.5	Results	445
11.5.6	Implementation Considerations	445
11.5.7	Lessons Learned	445
11.6	Case Study 4: Failed Experiment—Subscription Paywall	445
11.6.1	Context	446
11.6.2	Design	446
11.6.3	Results: The Good and The Bad	446
11.6.4	What Went Wrong	446
11.6.5	Correct Analysis	447
11.6.6	Lessons Learned	450
11.7	Digital Marketing	450
11.7.1	Case Study 5: Email Subject Line A/B Test	450
11.7.2	Case Study 6: Landing Page Redesign	451
11.8	Product Development	452
11.8.1	Case Study 7: Feature Onboarding Flow	452
11.8.2	Case Study 8: Mobile App Performance Optimization	453
11.9	Content and Media	453
11.9.1	Case Study 9: Video Recommendation Algorithm	454
11.9.2	Case Study 10: News Feed Ranking (Partial Failure)	455
11.10	Lessons Learned: Meta-Insights Across Cases	455
11.10.1	Successes: What Worked	455
11.10.2	Failures: What to Avoid	456
11.10.3	Organizational Challenges	456
11.10.4	Scaling Experiment Programs	457
11.10.5	Industry-Specific Adaptations	457
11.11	Common Anti-Patterns and How to Avoid Them	458
11.11.1	Anti-Pattern 1: P-Hacking	458
11.11.2	Anti-Pattern 2: HiPPO (Highest Paid Person’s Opinion)	458
11.11.3	Anti-Pattern 3: Analysis Paralysis	459
11.11.4	Anti-Pattern 4: Novelty Effect Ignorance	459

11.12	Industry-Specific Considerations	459
11.12.1	E-commerce	459
11.12.2	Social Media	460
11.12.3	SaaS/B2B	460
11.12.4	Healthcare/EdTech	461
11.13	Building an Experimentation Culture: Meta-Lessons	461
11.13.1	Infrastructure and Tooling	461
11.13.2	Process and Governance	461
11.13.3	Culture and Education	462
11.14	Summary	462
11.15	Further Reading	462
11.16	Exercises	463
11.16.1	Case Study Analysis	463
11.16.2	Business Impact Calculation	463
11.16.3	Anti-Pattern Detection	463
11.16.4	Industry Application	463
11.16.5	Meta-Analysis Project	464
12	Ethics and Best Practices	465
12.1	Learning Objectives	465
12.2	Ethical Foundations	465
12.2.1	Core Ethical Principles	465
12.2.2	The Belmont Report	465
12.3	Informed Consent	466
12.3.1	Consent in Digital Experiments	466
12.3.2	When Explicit Consent is Required	466
12.3.3	Balancing Innovation and Consent	466
12.4	Ethical Guardrails	466
12.4.1	Minimal Risk Standard	466
12.4.2	Experimentation Review Board	467
12.4.3	Automated Guardrails	467
12.5	Privacy and Data Protection	469
12.5.1	Data Minimization	469
12.5.2	Regulatory Compliance	469
12.5.3	Internal Data Governance	470
12.6	Regulatory Compliance	470
12.6.1	GDPR and Data Protection Laws	470
12.6.2	HIPAA (Healthcare)	473
12.6.3	COPPA (Children's Privacy)	474
12.6.4	Financial Services Regulations	474
12.6.5	Institutional Review Boards (IRBs)	477
12.6.6	Industry-Specific Compliance Checklist	478
12.7	Transparency and Communication	478
12.7.1	Internal Transparency	478
12.7.2	External Transparency	479
12.8	Experimental Integrity	479
12.8.1	Pre-registration and Documentation	479
12.8.2	Avoiding P-Hacking and Data Dredging	482

12.8.3	Reproducibility Standards	485
12.8.4	Conflict of Interest Management	487
12.9	Building Experimentation Culture	487
12.9.1	Cultural Principles	487
12.9.2	Organizational Structure	488
12.9.3	Training and Education	488
12.10	Organizational Best Practices	488
12.10.1	Experiment Governance Framework	489
12.10.2	Team Structure and Responsibilities	491
12.10.3	Knowledge Management System	492
12.10.4	Knowledge Sharing Mechanisms	494
12.10.5	Continuous Learning Culture	495
12.11	Quality Assurance	495
12.11.1	Peer Review Process	495
12.11.2	Statistical Consulting	498
12.11.3	Automated Validation	498
12.11.4	External Auditing	500
12.12	Best Practices Checklist	501
12.12.1	Design Phase	501
12.12.2	Implementation Phase	501
12.12.3	Analysis Phase	502
12.12.4	Decision Phase	502
12.13	Common Pitfalls and How to Avoid Them	502
12.14	Summary	502
12.15	Further Reading	503
12.16	Exercises	503

Mathematical Proofs 505

Mathematical Proofs and Derivations 507

.1	Sample Size Formulas	507
.1.1	Sample Size for Two-Sample t-Test	507
.1.2	Sample Size for Comparing Two Proportions	509
.1.3	Minimum Detectable Effect	510
.1.4	Unequal Sample Sizes	510
.2	Central Limit Theorem	511
.2.1	Classical Central Limit Theorem	511
.2.2	Berry-Esseen Theorem	512
.2.3	Delta Method	512
.3	Power Function Derivations	513
.3.1	Power Function for Two-Sample t-Test	513
.3.2	Power Function for Proportion Test	514
.3.3	Non-Centrality Parameter	515
.4	Bayesian Posterior Calculations	515
.4.1	Beta-Binomial Model	515
.4.2	Posterior Predictive Distribution	516
.4.3	Probability of Superiority	516
.4.4	Normal-Normal Conjugacy	517

.5	Sequential Testing Boundaries	517
.5.1	Sequential Probability Ratio Test (SPRT)	518
.5.2	Group Sequential Boundaries	518
.5.3	Pocock Boundaries	519
.5.4	Alpha Spending Functions	519
.5.5	Information-Based Monitoring	520
.6	Additional Results	520
.6.1	Variance of Sample Proportion	520
.6.2	CUPED Variance Reduction	521
.6.3	Bonferroni and Šidák Corrections	521
.7	Summary	522
Code Examples		523
Code Examples and Implementation		525
.1	Python Implementation	525
.1.1	Complete ABTest Framework	525
.2	R Implementation	540
.2.1	Statistical Analysis Functions	540
.3	SQL Queries for Data Extraction	550
.3.1	Experiment Data Extraction	550
.4	Configuration Files	559
.4.1	Experiment Configuration (YAML)	559
.4.2	Analysis Pipeline Configuration (JSON)	563
.5	API Implementation	567
.5.1	RESTful API for Experiment Management	567
.5.2	API Client Library (Python)	577
.6	Summary	585
Statistical Tables		587
Statistical Tables and Reference Values		589
.1	Critical Values for Common Tests	589
.1.1	Standard Normal Distribution (Z)	589
.1.2	Student's t-Distribution	589
.1.3	Chi-Squared Distribution	589
.1.4	F-Distribution	590
.2	Sample Size Tables	591
.2.1	Sample Size for Two-Proportion Test	591
.2.2	Sample Size for Different Power Levels	591
.2.3	Sample Size for Continuous Metrics	591
.3	Power Tables	592
.3.1	Power for Two-Proportion Tests	592
.3.2	Power vs. Effect Size	592
.4	Minimum Detectable Effect (MDE)	593
.4.1	MDE for Two-Proportion Tests	593
.4.2	MDE in Relative Terms	593
.5	Multiple Testing Corrections	594
.5.1	Bonferroni Correction	594

.5.2	Šidák Correction	594
.5.3	Benjamini-Hochberg FDR Thresholds	594
.5.4	Sample Size Inflation for Multiple Testing	595
.6	Sequential Testing Boundaries	596
.6.1	O'Brien-Fleming Boundaries	596
.6.2	Pocock Boundaries	596
.6.3	Alpha Spending Function Values	597
.7	Distribution Parameters and Properties	598
.7.1	Common Distributions for A/B Testing	598
.7.2	Beta Distribution Quantiles	598
.7.3	Normal Distribution Probabilities	599
.7.4	Variance Formulas for Common Metrics	599
.8	Finite Population Corrections	600
.8.1	Finite Population Correction Factor	600
.8.2	Design Effect for Cluster Randomization	600
.9	Effect Size Benchmarks	601
.9.1	Cohen's d (Standardized Mean Difference)	601
.9.2	Cohen's h (Standardized Proportion Difference)	601
.9.3	Practical Significance Thresholds	601
.10	Quick Reference Guide	602
.10.1	Test Selection Decision Tree	602
.10.2	Common Pitfalls and Corrections	602
.10.3	Recommended Parameters by Scenario	602
.11	Summary	602
Notation and Symbols		607
.12	Introduction	607
.13	General Mathematical Notation	607
.14	Probability and Statistics	608
.14.1	Random Variables and Distributions	608
.14.2	Common Distributions	608
.15	Hypothesis Testing	609
.16	Experimental Design	609
.16.1	Treatment and Assignment	609
.16.2	Sample Size and Power	610
.17	A/B Testing Specific Notation	610
.17.1	Conversion and Proportions	610
.17.2	Metrics and Performance	611
.18	Multiple Testing	611
.19	Bayesian Methods	611
.20	Sequential Testing	612
.21	Variance Reduction	612
.22	Network Effects and Interference	612
.23	Causal Inference	613
.24	Bandit Algorithms	613
.25	Subscripts and Superscripts	613
.25.1	Common Subscripts	613
.25.2	Common Superscripts	614

.26	Operators and Functions	614
.27	Usage Notes	614
.27.1	Notation Conventions	614
.27.2	Distribution Notation	615
.27.3	Asymptotic Notation	615
.27.4	Context-Dependent Notation	615

List of Figures

List of Tables

2.1	Type I and Type II errors in hypothesis testing	44
4.1	Type I and Type II errors in hypothesis testing	103
7.1	Prior selection guidelines for common A/B testing scenarios	220
7.2	Common conjugate prior families for A/B testing	238
7.3	Philosophical differences between frequentist and Bayesian frameworks	251
7.4	Recommended approach by scenario (checkmarks indicate suitability)	252
8.1	Bandit vs. traditional A/B test selection guide	274
9.1	Data storage strategy by type	323
1	Standard Normal Critical Values ($z_{\alpha/2}$)	589
2	t-Distribution Critical Values for Two-Sided Tests	590
3	Chi-Squared Distribution Critical Values	591
4	F-Distribution Critical Values ($\alpha = 0.05$)	592
5	Sample Size per Variant (Two-Proportion Z-Test, $\alpha = 0.05$, Power = 0.80)	592
6	Sample Size vs. Power (Baseline 10%, $\alpha = 0.05$)	593
7	Sample Size for t-Test (Effect Size = Cohen's d)	593
8	Statistical Power vs. Sample Size (Baseline 10%)	594
9	Power for Fixed Sample Size ($n = 5,000$ per variant, $\alpha = 0.05$)	594
10	Minimum Detectable Effect (Percentage Points, $\alpha = 0.05$, Power = 0.80)	595
11	Minimum Detectable Effect (Relative %, $\alpha = 0.05$, Power = 0.80)	595
12	Bonferroni Corrected Significance Levels	596
13	Šidák vs. Bonferroni Corrected Alpha	596
14	Benjamini-Hochberg Critical Values (FDR = 0.05)	597
15	Sample Size Inflation Factor (to maintain 80% power after correction)	597
16	O'Brien-Fleming Boundaries (Overall $\alpha = 0.05$, two-sided)	598
17	Pocock Constant Boundaries (Overall $\alpha = 0.05$, two-sided)	598
18	O'Brien-Fleming Alpha Spending (Total $\alpha = 0.05$)	599
19	Distribution Parameters and Properties	600
20	Beta Distribution Quantiles	601
21	Standard Normal Cumulative Probabilities	601
22	Variance Formulas for A/B Test Metrics	602
23	Finite Population Correction: $\sqrt{\frac{N-n}{N-1}}$	602
24	Design Effect: $DEFF = 1 + (m - 1)\rho$	603
25	Cohen's d Effect Size Classification	603
26	Cohen's h: $h = 2[\arcsin(\sqrt{p_2}) - \arcsin(\sqrt{p_1})]$	604

27	Practical Significance Guidelines by Industry	604
28	Statistical Test Selection Guide	605
29	Common Issues in A/B Testing	605
30	Standard Parameter Choices	606

List of Abbreviations

This section provides definitions for abbreviations and acronyms used throughout this book, organized alphabetically.

A/A Test	Test where both groups receive the same treatment, used to validate randomization	CDF	Cumulative Distribution Function
		CI	Confidence Interval
A/B Test	Randomized experiment comparing two variants (A and B)	CLT	Central Limit Theorem
		CR	Conversion Rate
		CTR	Click-Through Rate
A/B/n Test	Experiment with more than two variants	CUPED	Controlled-experiment Using Pre-Experiment Data
ANOVA	Analysis of Variance	CVR	Conversion Rate (alternative)
AOV	Average Order Value	DE	Design Effect
API	Application Programming Interface	DID	Difference-in-Differences
ARPU	Average Revenue Per User	df	Degrees of Freedom
ASA	American Statistical Association	ETL	Extract, Transform, Load
		EXP3	Exponential-weight algorithm for Exploration and Exploitation
ATE	Average Treatment Effect		
ATT	Average Treatment Effect on the Treated	FDR	False Discovery Rate
		FPC	Finite Population Correction
BF	Bayes Factor		
BH	Benjamini-Hochberg (FDR control procedure)	FWER	Family-Wise Error Rate
		GDPR	General Data Protection Regulation
CATE	Conditional Average Treatment Effect	HTE	Heterogeneous Treatment Effects
CCPA	California Consumer Privacy Act	HPD	Highest Posterior Density

HTML	HyperText Markup Language	OEC	Overall Evaluation Criterion
HTTP	HyperText Transfer Protocol	PDF	Probability Density Function
i.i.d.	Independent and Identically Distributed	PMF	Probability Mass Function
ICC	Intraclass Correlation Coefficient	QA	Quality Assurance
IQR	Interquartile Range	QQ Plot	Quantile-Quantile Plot
ITT	Intention To Treat	RDBMS	Relational Database Management System
IV	Instrumental Variable	RDD	Regression Discontinuity Design
JSON	JavaScript Object Notation	REST	Representational State Transfer
KL	Kullback-Leibler (divergence)	RMSE	Root Mean Squared Error
KPI	Key Performance Indicator	ROI	Return on Investment
LATE	Local Average Treatment Effect	RPU	Revenue Per User
LTV	Lifetime Value	SaaS	Software as a Service
MAB	Multi-Armed Bandit	SDK	Software Development Kit
MAP	Maximum A Posteriori	SE	Standard Error
MCMC	Markov Chain Monte Carlo	SQL	Structured Query Language
MDE	Minimum Detectable Effect	SRM	Sample Ratio Mismatch
MLE	Maximum Likelihood Estimation/Estimator	SUTVA	Stable Unit Treatment Value Assumption
MSE	Mean Squared Error	TS	Thompson Sampling
MVT	Multivariate Testing	UCB	Upper Confidence Bound
NHST	Null Hypothesis Significance Testing	UI	User Interface
NPS	Net Promoter Score	URL	Uniform Resource Locator
OBF	O'Brien-Fleming (sequential boundary)	UX	User Experience
		VR	Variance Reduction
		VWO	Visual Website Optimizer
		WWW	World Wide Web
		YAML	YAML Ain't Markup Language

Statistical Test Abbreviations

t-test	Student's t-test for comparing means
Welch's t-test	t-test for unequal variances
z-test	Normal approximation test for large samples
χ^2 test	Chi-squared test for categorical data
F-test	Test for comparing variances or ANOVA
Mann-Whitney U	Non-parametric test for comparing distributions
Wilcoxon	Non-parametric paired test
Kolmogorov-Smirnov	Non-parametric test for comparing distributions
Fisher's exact	Exact test for 2×2 contingency tables
Kruskal-Wallis	Non-parametric alternative to one-way ANOVA

Common Distribution Abbreviations

Bernoulli	Binary outcome distribution (success/failure)
Binomial	Sum of independent Bernoulli trials
Poisson	Count distribution for rare events
Normal/Gaussian	Bell-shaped continuous distribution
Uniform	Equal probability distribution
Exponential	Continuous distribution for time between events
Beta	Continuous distribution on $[0, 1]$, conjugate prior for Bernoulli
Gamma	Continuous distribution, conjugate prior for Poisson rate
t-distribution	Student's t-distribution for small samples
Chi-squared	Distribution of sum of squared normal variables
F-distribution	Ratio of chi-squared variables
Multinomial	Generalization of binomial to multiple categories
Dirichlet	Multivariate generalization of beta distribution
Negative Binomial	Distribution of trials until r successes
Geometric	Distribution of trials until first success
Hypergeometric	Sampling without replacement

Software and Platform Abbreviations

Python	High-level programming language
R	Statistical programming language
SQL	Query language for databases
NumPy	Numerical computing library for Python
SciPy	Scientific computing library for Python
pandas	Data analysis library for Python
statsmodels	Statistical modeling library for Python
scikit-learn	Machine learning library for Python
matplotlib	Plotting library for Python
seaborn	Statistical visualization library for Python
PyMC	Probabilistic programming in Python
Stan	Probabilistic programming language
JAGS	Just Another Gibbs Sampler
ggplot2	Data visualization package in R
dplyr	Data manipulation package in R
tidyverse	Collection of R packages for data science
PostgreSQL	Open-source relational database
MySQL	Open-source relational database
MongoDB	NoSQL document database
Redis	In-memory data structure store
Kafka	Distributed event streaming platform
Spark	Distributed computing framework
Hadoop	Distributed storage and processing framework
TensorFlow	Machine learning framework
PyTorch	Machine learning framework

Company/Platform-Specific Terms

Google Analytics	Web analytics service by Google
Google Optimize	A/B testing platform by Google
Optimizely	Commercial experimentation platform
VWO	Visual Website Optimizer, A/B testing platform
Adobe Target	Personalization and testing platform
Mixpanel	Product analytics platform
Amplitude	Product analytics platform
Heap	Digital insights platform
Segment	Customer data platform
LaunchDarkly	Feature flag management platform
Split.io	Feature delivery platform
GrowthBook	Open-source feature flagging and A/B testing
Unleash	Open-source feature toggle system

Notes on Usage

- **Context Matters:** Some abbreviations have different meanings in different contexts (e.g., "SE" can mean Standard Error or Software Engineering).
- **Capitalization:** While abbreviations are typically capitalized, their expansions may not be (e.g., "PDF" stands for "probability density function," not "Probability Density Function").
- **Plural Forms:** When pluralizing abbreviations, add lowercase "s" without an apostrophe (e.g., "KPIs," "PDFs").
- **Regional Variations:** Some terms have different names in different regions or disciplines (e.g., "conversion rate" vs. "CVR").
- **Domain-Specific:** Many abbreviations are specific to online experimentation and may have different meanings in other fields.

For detailed definitions and mathematical formulations, please refer to Appendix D: Notation and Symbols.

Preface

The idea for this book crystallized during a late-night debugging session at Mysense.ai, where I serve as Lead Data Scientist. We were investigating why an A/B test that should have been straightforward—comparing two checkout button designs—had produced confusing results. The statistical significance was marginal, the effect size was tiny, yet business stakeholders were pressing for a decision. As I walked through the analysis with the team, I realized that the real challenge wasn't the mathematics or the code—it was the gap between statistical rigor and practical decision-making under uncertainty.

This experience, repeated countless times across industry and academia, revealed a fundamental problem: most practitioners learn A/B testing from one of two extremes. Either they study rigorous statistical theory without adequate grounding in real-world constraints, or they learn practical "recipes" without understanding the underlying assumptions and limitations. Both approaches leave critical gaps. The theorist may design a perfectly valid experiment that's impossible to implement in production. The practitioner may run hundreds of tests without recognizing systematic biases that invalidate their conclusions.

This book attempts to bridge that divide.

Why This Book, Why Now

The practice of experimentation has undergone a quiet revolution over the past two decades. What began as a specialized technique used primarily in pharmaceutical trials and agricultural research has become the cornerstone of decision-making at technology companies, e-commerce platforms, government agencies, and organizations of all types. Google runs tens of thousands of experiments annually. Amazon tests nearly every product change. Netflix uses experimentation to optimize everything from recommendation algorithms to thumbnail images. Even traditional industries—retail, finance, health-care—increasingly embrace data-driven experimentation.

Yet this democratization of A/B testing has created new challenges. The barrier to running an experiment has dropped dramatically—modern platforms make it trivially easy to split traffic and measure outcomes. But the barrier to running a *good* experiment, one that produces trustworthy insights and leads to sound decisions, remains high. Statistical literacy has not kept pace with experimental practice.

The consequences manifest in multiple ways. I've seen teams celebrate "winning" variants that were merely lucky noise. I've watched organizations make million-dollar decisions based on p-values they didn't understand. I've encountered experiments that violated basic statistical assumptions yet were treated as gospel. Most concerning, I've observed a growing cynicism about experimentation itself when poorly designed tests produce conflicting or irreproducible results.

These experiences, accumulated through my dual roles—as Lead Data Scientist at Mysense.ai, where I design and analyze experiments for real products and real customers, and as Researcher and Instructor at ESMAD (Instituto Politécnico do Porto), where I teach the statistical foundations to the next generation of data scientists—convinced me that a different kind of book was needed.

Who This Book Is For

I wrote this book with several overlapping audiences in mind, all united by a need to understand both the "how" and the "why" of A/B testing.

Data Scientists and Analysts working in industry will find practical guidance grounded in statistical rigor. You face daily pressures to move fast, to test everything, to "let the data decide." This book will help you move quickly *and* correctly, understanding when standard methods apply and when you need something more sophisticated. You'll learn to spot the subtle implementation issues that invalidate experiments, to communicate uncertainty clearly to stakeholders, and to push back on unreasonable demands ("Can we just peek at the results?") with solid statistical reasoning.

Graduate Students in statistics, data science, computer science, or related fields will find a modern treatment of experimental design that connects classical theory to contemporary practice. The mathematical development is rigorous—I don't shy away from proofs when they illuminate understanding—but always motivated by practical questions. You'll see how the Central Limit Theorem justifies real-world approximations, why Bayesian methods handle certain problems elegantly, and where theory meets the messiness of production systems.

Researchers in academia or industry who design experiments will discover how classical experimental design principles translate to digital contexts. If you're familiar with randomized controlled trials in clinical or social science settings, you'll learn how online experimentation differs—both the new opportunities (instant results, massive sample sizes) and new challenges (network effects, novelty effects, implementation bugs).

Product Managers and Technical Leaders who make decisions based on experimental evidence need to understand what the numbers really mean. This book will help you ask the right questions: Is this sample size adequate? Are we measuring the right thing? What assumptions underlie this analysis? You'll develop intuition for when to trust experimental results and when to dig deeper.

Practitioners Transitioning from Other Fields who have statistical training but are new to online experimentation will find familiar concepts (hypothesis testing, confidence intervals, power analysis) applied in novel contexts. You'll learn the domain-specific challenges—how to handle sample ratio mismatches, why sequential testing requires special care, when to use variance reduction techniques.

What unites all these audiences is a need for both depth and breadth—statistical foundations solid enough to support critical decisions, practical guidance specific enough to implement tomorrow, and judgment sophisticated enough to navigate the grey areas where theory doesn't provide clear answers.

What Makes This Book Different

Several principles guided the development of this book:

Theory Motivated by Practice. Every statistical concept is introduced in response to a real problem. Why do we need the Central Limit Theorem? Because we want to make probability statements about sample means. Why study multiple testing corrections? Because without them, we fool ourselves with false discoveries. The mathematics serves the practice, not the other way around.

Practice Grounded in Theory. Conversely, every practical technique is connected to its theoretical foundation. When I present a Python function for calculating sample size, I also derive the formula and explain the assumptions. When we implement Bayesian A/B testing, we understand why conjugate priors work and when they're appropriate. You won't just learn what to do—you'll understand why it works and when it doesn't.

Code as First-Class Citizen. This is not a book where code appears as an afterthought. The implementations in Python and R are production-quality, documented, and ready to adapt. I've run every code example, debugged the edge cases, and provided not just the happy path but also error handling and validation. Because I've been the person at 2 AM trying to debug a broken experiment, I know that implementation details matter.

Honest About Limitations. Statistics education often presents methods as if they solve problems cleanly. Real experimentation is messier. This book acknowledges the gaps: where assumptions are likely violated, when approximations break down, which questions statistics cannot answer. I share failures alongside successes, because learning what doesn't work is as valuable as learning what does.

Modern Methodologies. While grounded in classical statistics, this book embraces contemporary approaches. We cover Bayesian methods seriously, not as a curiosity but as a practical alternative with real advantages in certain contexts. We explore multi-armed bandits, sequential testing, and variance reduction techniques that have emerged from modern practice. We address challenges—network effects, novelty effects, metric dilution—that classical texts don't consider.

How to Use This Book

The book is designed for multiple modes of engagement:

As a Course Text: Graduate courses can proceed through chapters sequentially, using exercises to reinforce learning. I've structured the material to build progressively, with each chapter assuming mastery of previous content. Problem sets range from mathematical derivations to simulation studies to analysis of real datasets.

As a Reference: Practitioners can dive directly into specific chapters as needs arise. Comprehensive cross-referencing, a detailed index, and self-contained chapters enable targeted learning. Need to understand multiple testing corrections? Chapter 6 provides everything required. Want to implement Bayesian A/B testing? Chapter 7 is your guide.

As Professional Development: Working data scientists can deepen their understanding systematically. Perhaps you've been running A/B tests for years using standard tools but want to understand the statistical foundations more rigorously. Or you're comfortable with frequentist methods but curious about Bayesian approaches. The book supports both deepening and broadening your expertise.

Each chapter includes learning objectives, worked examples with complete solutions, production-ready code, exercises at varying difficulty levels, and suggestions for further reading. The appendices provide mathematical proofs for those seeking rigor, compre-

hensive code libraries for those seeking implementation, and statistical tables for quick reference.

A Personal Note

Writing this book has been a journey of synthesis—bringing together years of experience across industry and academia, reconciling practical constraints with theoretical ideals, and distilling complex ideas into accessible explanations. My work at Mysense.ai has provided invaluable real-world context: the pressure to move fast, the complexity of production systems, the challenge of communicating uncertainty to stakeholders who want certainty. My teaching and research at ESMAD has forced me to articulate foundations clearly, to answer "why" questions rigorously, and to stay current with methodological developments.

This dual perspective—practitioner and educator, industry and academia—has shaped every page. I've tried to write the book I wish had existed when I began my career, one that would have saved me from numerous mistakes and accelerated my understanding of this powerful methodology.

Experimentation, at its best, represents a commitment to learning from reality rather than assuming we already know. It's an acknowledgment that our intuitions, however informed, are fallible—and that careful measurement can surprise us. This intellectual humility, coupled with methodological rigor, enables genuine discovery.

My hope is that this book helps you design better experiments, analyze results more critically, make sounder decisions, and ultimately contribute to the growing culture of evidence-based decision making. Whether you're optimizing a website, evaluating a policy intervention, or conducting scientific research, the principles of rigorous experimentation apply.

The field of A/B testing continues to evolve. New methods emerge, new challenges arise, new applications appear. But the fundamentals—randomization, statistical inference, careful interpretation—endure. Master these foundations, and you'll be equipped not just for today's problems but for tomorrow's challenges.

Diogo Ribeiro
Lead Data Scientist, Mysense.ai
Researcher and Instructor, ESMAD
Instituto Politécnico do Porto
Porto, Portugal
November 20, 2025

Chapter 1

Introduction to A/B Testing

1.1 Learning Objectives

After completing this chapter, readers will be able to:

- Define A/B testing and explain its fundamental principles
- Trace the historical evolution from agricultural experiments to digital applications
- Distinguish A/B testing from other experimental methodologies
- Understand core terminology and concepts used throughout the book
- Recognize different types of A/B tests and their appropriate applications
- Implement basic A/B tests using Python and statistical libraries
- Identify real-world scenarios where A/B testing adds value

1.2 What is A/B Testing?

1.2.1 Definition and Core Concepts

Definition 1.1 (A/B Test). An A/B test is a randomized controlled experiment in which a population is randomly divided into two or more groups to compare the effects of different treatments, interventions, or variations on a measured outcome. The fundamental structure includes:

- A **control group** (A) receiving the existing or baseline treatment
- One or more **treatment groups** (B, C, etc.) receiving new variations
- **Random assignment** ensuring unbiased allocation
- **Quantifiable outcomes** measured consistently across groups

At its core, A/B testing operationalizes the scientific method for decision-making under uncertainty. Rather than relying on intuition, expert opinion, or observational

data—all valuable but potentially misleading—we directly test hypotheses through controlled experimentation. This approach answers a deceptively simple question: *Does this change actually work?*

The elegance of A/B testing lies in its simplicity. By randomly assigning subjects to groups and measuring outcomes, we isolate the causal effect of our intervention from confounding variables. If users assigned to the new checkout flow convert at a higher rate than those in the control group, and this difference exceeds what we’d expect from random variation, we have evidence that the change caused the improvement.

1.2.2 From Agricultural Fields to Digital Platforms

The intellectual foundations of A/B testing trace back over a century, evolving through several distinct phases that reveal both continuity and transformation in experimental thinking.

Early Pioneers: Fisher and Agricultural Science

In the 1920s, Sir Ronald Fisher revolutionized experimental design while working at Rothamsted Experimental Station in England. Fisher confronted a practical problem: how to determine which fertilizers, crop varieties, and farming techniques produced the best yields when countless variables—soil quality, weather, pest pressure—varied unpredictably across fields and seasons.

Fisher’s solution introduced three principles that remain foundational today [Fisher, 1935]:

1. **Randomization:** Rather than systematically assigning treatments (which might confound treatment effects with field characteristics), Fisher advocated random assignment to eliminate systematic bias.
2. **Replication:** Testing each treatment on multiple plots enabled statistical estimation of variability and increased confidence in results.
3. **Blocking:** Grouping similar experimental units (e.g., adjacent plots) and randomizing within blocks reduced noise and improved precision.

These principles solved the agricultural scientist’s dilemma: how to draw valid causal inferences in the face of uncontrolled variation. They work equally well for digital experimentation, though the scale and speed have changed dramatically.

Clinical Trials and Medical Research

The mid-20th century saw experimental design principles migrate to medical research. The 1948 streptomycin tuberculosis trial is often cited as the first modern randomized controlled trial (RCT) [Lind, 1953]. Medical researchers recognized that comparing treatments without randomization introduced selection bias—sicker patients might systematically receive more aggressive treatments, confounding causal inference.

Clinical trials formalized concepts still central to A/B testing:

- **Blinding:** Keeping subjects (and often researchers) unaware of treatment assignment to prevent expectation effects

- **Intention-to-treat analysis:** Analyzing all randomized subjects regardless of compliance
- **Power analysis:** Calculating required sample sizes before conducting experiments
- **Interim monitoring:** Checking results during trials for ethical reasons, requiring special statistical methods

The Digital Revolution

The internet transformed experimentation. What took months or years in agricultural or medical research could now happen in hours. Early online experiments at Amazon, Google, and Microsoft in the late 1990s and early 2000s demonstrated that digital platforms offered unprecedented advantages:

- **Massive scale:** Millions of subjects available
- **Low cost:** Marginal cost of adding one more subject approaches zero
- **Rapid iteration:** Tests complete in days or weeks, not months or years
- **Precise measurement:** Every click, view, and conversion tracked automatically
- **Perfect randomization:** Computers execute random assignment flawlessly

However, digital experimentation also introduced new challenges that Fisher never contemplated:

- **Implementation bugs:** Code errors can invalidate experiments
- **Network effects:** One user's treatment may affect others
- **Novelty effects:** Users respond differently to new features initially
- **Metric dilution:** Measuring too many outcomes increases false positives
- **Continuous monitoring:** "Peeking" at results invalidates classical statistics

Modern A/B testing inherits Fisher's foundational principles while adapting them to digital contexts. This synthesis—classical rigor meets contemporary scale—defines the field today.

1.2.3 A/B Testing vs. Other Methodologies

Understanding A/B testing requires distinguishing it from alternative approaches to learning and decision-making.

Observational Studies

Observational studies analyze existing data without intervention. For example, examining whether users who engage with a particular feature have higher retention. While valuable for hypothesis generation, observational studies suffer from confounding: perhaps engaged users were already more loyal, and feature usage is a symptom rather than a cause.

Example 1.1 (Simpson’s Paradox). An e-commerce platform observes that customers who use the mobile app spend 50% more than website-only customers. However, randomizing users to receive an app download prompt shows no effect on spending. Why?

The observational data confounds app usage with customer type. High-value customers independently discovered and adopted the app. The app didn’t cause higher spending; higher-spending customers chose to use it. Randomization eliminates this confounding.

Multi-Armed Bandits

Multi-armed bandit (MAB) algorithms dynamically allocate more traffic to better-performing variants, optimizing cumulative performance during the test. Unlike A/B tests with fixed allocation, bandits adapt.

Consider testing K variants with unknown success rates $\theta_1, \dots, \theta_K$. The regret after T trials is:

$$R(T) = T \cdot \max_k \theta_k - \sum_{t=1}^T \mathbb{E}[r_t] \quad (1.1)$$

where r_t is the reward at trial t .

MAB algorithms like Thompson Sampling or Upper Confidence Bound (UCB) minimize regret by balancing exploration (testing uncertain variants) and exploitation (favoring likely winners).

Trade-offs:

- A/B tests provide clean statistical inference but sacrifice performance during testing
- MABs optimize during testing but complicate inference (non-uniform assignment probabilities)
- A/B tests suit scenarios where learning is the goal
- MABs suit scenarios where optimization during testing matters (e.g., ad selection)

We explore bandits in depth in Chapter 8.

Quasi-Experimental Methods

When randomization is infeasible, quasi-experimental designs attempt causal inference from observational data under assumptions. Methods include:

- **Difference-in-differences:** Comparing trends before/after intervention in treatment vs. control groups
- **Regression discontinuity:** Exploiting arbitrary cutoffs (e.g., test score thresholds) for assignment

- **Instrumental variables:** Using variables that affect treatment but not outcomes directly
- **Propensity score matching:** Creating matched pairs with similar characteristics

These methods complement A/B testing when experimentation is impractical but carry stronger assumptions.

1.3 Why A/B Testing Matters

1.3.1 The Business Case for Experimentation

A/B testing delivers measurable value through better decision-making, risk mitigation, and organizational learning.

Quantifiable Return on Investment

Consider a concrete example. An e-commerce company with 1 million monthly visitors and 3% conversion rate generates 30,000 orders monthly. An A/B test finds that a new checkout flow increases conversion to 3.3%—a 10% relative lift.

Annual impact:

$$\text{Additional conversions} = 1,000,000 \times 12 \times (0.033 - 0.030) = 36,000 \quad (1.2)$$

$$\text{Revenue (at \$50 AOV)} = 36,000 \times \$50 = \$1,800,000 \quad (1.3)$$

If the test cost \$10,000 to design, implement, and analyze, the ROI is 180x. Even conservative estimates assuming 5% lift still yield \$900,000 annually.

De-Risking Product Changes

Not all changes improve outcomes. Experimentation prevents costly mistakes:

Example 1.2 (Microsoft Bing). Microsoft tested displaying ads more prominently, hypothesizing increased revenue. The test showed short-term revenue gains but significant user satisfaction declines and reduced search quality metrics. Without testing, full deployment would have damaged the product and brand [Kohavi et al., 2009].

Failures are valuable. Learning that an idea doesn't work before full rollout saves development resources and protects user experience.

Compound Returns from Iteration

Individual experiments often yield small improvements—2-5% lifts. The power comes from compounding many improvements over time.

If a team runs one experiment weekly, each improving a key metric by 2%, annual compound growth is:

$$(1.02)^{52} - 1 \approx 1.83 = 183\% \quad (1.4)$$

This calculation oversimplifies (not all tests succeed, metrics may not be independent), but illustrates how systematic experimentation drives dramatic cumulative gains.

1.3.2 Evidence-Based Culture

Beyond individual tests, A/B testing transforms organizational decision-making.

Objective Evaluation of Ideas

Experimentation democratizes decision-making. Good ideas can come from anywhere—junior engineers, customer support, users themselves—and all receive objective evaluation. This meritocracy of ideas contrasts with authority-based or seniority-based decision-making.

Learning from Surprises

The most valuable experiments challenge assumptions. When results contradict expert opinion or conventional wisdom, we learn something fundamental about user behavior, market dynamics, or product mechanics.

Example 1.3 (Netflix Personalization). Netflix hypothesized that showing personalized movie thumbnails would increase engagement. Testing revealed the effect was far larger than anticipated—personalized images increased viewing by 20-30% for some content. This insight led to broader personalization initiatives worth billions in subscriber value [Kohavi et al., 2020].

1.3.3 Examples from Industry Leaders

Google: Search Result Optimization

Google runs over 10,000 experiments annually on Search alone. A famous 2009 experiment tested displaying 10, 20, or 30 search results per page. The hypothesis: more results would reduce pagination and improve user experience.

Results showed that increasing results to 30 caused a 20% increase in page load time (from 0.4s to 0.9s). This seemingly small delay significantly reduced searches per user—users abandoned searches rather than wait. The test demonstrated that even half-second delays meaningfully impact behavior at scale.

Amazon: Everything is Testable

Amazon's culture of experimentation is legendary. The company tests nearly every product change before deployment. Examples include:

- Testing "Customers who bought this also bought" recommendation algorithms
- Optimizing product page layouts
- Testing checkout flows and payment options
- Evaluating search result ranking algorithms

Amazon founder Jeff Bezos famously said, "Our success at Amazon is a function of how many experiments we do per year, per month, per week, per day."

Facebook: Continuous Optimization

Facebook treats its platform as a continuous experiment. The company tests features on small user subsets before gradual rollout. Notable examples:

- News Feed ranking algorithms optimizing for engagement
- Ad formats and placement testing
- Like button variations and reactions
- Video autoplay behavior

However, Facebook also illustrates ethical challenges—a 2014 "emotional contagion" study manipulating News Feed content sparked controversy about informed consent in experimentation.

1.4 Basic Terminology and Concepts

Precise language enables clear thinking about experiments. This section establishes terminology used throughout the book.

1.4.1 Experimental Units and Assignment

Definition 1.2 (Experimental Unit). The experimental unit is the entity randomized to treatment. Common units include:

- **Users:** Individual people (most common in digital experiments)
- **Sessions:** Individual visits or browsing sessions
- **Page views:** Individual page loads
- **Clusters:** Groups (e.g., geographic regions, schools, stores)

The choice of experimental unit affects both implementation and analysis. User-level randomization ensures consistency across sessions but requires persistent identifiers. Session-level randomization is simpler but creates inconsistent experiences for repeat visitors.

1.4.2 Control and Treatment Groups

Definition 1.3 (Control Group). The control group (often called variant A) receives the existing or baseline experience. It serves as the comparison benchmark for measuring treatment effects.

Definition 1.4 (Treatment Group). Treatment groups (B, C, etc.) receive new variations being tested. In simple A/B tests, there is one treatment group; multivariate tests have multiple.

Why maintain a control? Even if we believe a change will improve outcomes, we need the counterfactual—what would have happened without the change? External factors (seasonality, marketing campaigns, news events) affect all users. Comparing treatment to control isolates the causal effect of our intervention.

1.4.3 Metrics and Key Performance Indicators

Definition 1.5 (Metric). A metric is a quantifiable measure of user behavior or business outcome. Metrics can be:

- **Binary:** Conversion (yes/no), click (yes/no)
- **Continuous:** Revenue per user, time on site, pages viewed
- **Count:** Number of purchases, search queries, shares

Effective metrics share several properties:

1. **Measurable:** Can be tracked reliably and consistently
2. **Attributable:** Can be linked to experimental units
3. **Timely:** Observable within reasonable experimental duration
4. **Sensitive:** Responds to interventions we care about
5. **Aligned:** Correlates with long-term business goals

Metric Hierarchies

Mature experimentation programs organize metrics hierarchically:

- **Primary metric:** The main outcome for decision-making (e.g., revenue per user)
- **Secondary metrics:** Supporting measures providing context (e.g., conversion rate, average order value)
- **Guardrail metrics:** Measures we monitor but don't want to degrade (e.g., page load time, error rate)

This hierarchy prevents "metric shopping"—testing many metrics until finding significance by chance.

1.4.4 Statistical Significance

Definition 1.6 (Statistical Significance). A result is statistically significant if the observed difference between groups is unlikely to have occurred by chance alone under the null hypothesis of no effect. Conventionally, we use significance level $\alpha = 0.05$, meaning we tolerate a 5% chance of false positives.

The p-value quantifies this: $p = P(\text{observe data this extreme} \mid H_0 \text{ true})$. If $p < \alpha$, we reject the null hypothesis and conclude a statistically significant effect exists.

Critical distinction: Statistical significance differs from practical significance. A tiny effect might be statistically significant with enough data but too small to matter. Conversely, a meaningful effect might not reach significance with limited data.

1.4.5 Effect Size

Definition 1.7 (Effect Size). Effect size quantifies the magnitude of difference between groups, independent of sample size. Common measures include:

- **Absolute difference:** $\hat{p}_B - \hat{p}_A$ (e.g., 10.5% - 10.0% = 0.5 percentage points)
- **Relative lift:** $\frac{\hat{p}_B - \hat{p}_A}{\hat{p}_A}$ (e.g., 0.5% / 10.0% = 5% relative increase)
- **Cohen's d:** $d = \frac{\bar{x}_B - \bar{x}_A}{s_{\text{pooled}}}$ (standardized difference for continuous outcomes)

Effect sizes communicate practical importance and facilitate comparison across experiments. A 5% relative lift in conversion rate from 10% baseline has different business impact than the same lift from 1% baseline, even if both are statistically significant.

1.5 Types of A/B Tests

Experimentation encompasses various designs suited to different objectives and constraints.

1.5.1 Simple A/B Tests

The canonical design compares one treatment against a control:

- 50% of users see control (A)
- 50% see treatment (B)
- Measure and compare a primary metric

When to use:

- Testing a single change (button color, headline, algorithm)
- Clear hypothesis about improvement
- Want clean statistical inference
- First test of a new idea

1.5.2 A/B/n Tests (Multiple Variants)

Testing multiple variations simultaneously:

- Control (A): 50%
- Treatment 1 (B): 25%
- Treatment 2 (C): 25%

Trade-offs:

- **Efficiency:** Test multiple ideas with same traffic

- **Complexity:** More comparisons increase multiple testing burden
- **Power:** Splitting traffic reduces power for each comparison

Proper multiple testing corrections (Chapter 6) are essential to maintain valid error rates.

1.5.3 Multivariate Tests (Factorial Designs)

Factorial designs test multiple factors simultaneously. A 2^k factorial tests k binary factors:

Example 1.4 (2×2 Factorial Design). Testing email subject line (A/B) and send time (morning/evening):

	Morning	Evening
Subject A	25%	25%
Subject B	25%	25%

This design estimates:

- Main effect of subject line
- Main effect of send time
- Interaction: does subject line effect depend on time?

Advantages:

- Test multiple factors with one experiment
- Detect interactions between factors
- More efficient than separate tests

Challenges:

- Complexity grows exponentially (2^k combinations)
- Interpretation harder with many factors
- Requires larger samples for interaction detection

Chapter 8 covers factorial designs comprehensively.

1.5.4 Sequential Testing

Traditional A/B tests fix sample size in advance and analyze once at the end. Sequential testing allows monitoring during the experiment with appropriate statistical controls.

Group sequential methods: Pre-specify analysis times (e.g., 25%, 50%, 75%, 100% of planned sample) and use spending functions to control Type I error.

Advantages:

- Stop early if strong evidence emerges

- Ethical in some contexts (e.g., medical trials)
- Reduce time to decision

Challenges:

- Requires special statistical methods
- "Peeking" at results invalidates standard p-values
- More complex implementation

1.5.5 Multi-Armed Bandits

Rather than fixed allocation, bandit algorithms dynamically shift traffic toward better performers:

Algorithm 1 Simple ϵ -Greedy Bandit

```

Initialize:  $N_i = 0$ ,  $\hat{\mu}_i = 0$  for variants  $i = 1, \dots, K$ 
for each user  $t = 1, 2, \dots$  do
  With probability  $\epsilon$ : select random variant (explore)
  With probability  $1 - \epsilon$ : select  $\arg \max_i \hat{\mu}_i$  (exploit)
  Observe outcome  $r_t$ 
  Update estimates for selected variant
end for

```

When bandits outperform A/B tests:

- Many variants to test (e.g., ad creative)
- Performance during testing matters (opportunity cost high)
- Priors available from similar tests
- Willing to trade inference clarity for optimization

1.6 Real-World Examples

1.6.1 Website Conversion Optimization

E-commerce Checkout Flow

An online retailer redesigns the checkout process, reducing steps from 5 to 3 by combining shipping and billing information.

Hypothesis: Fewer steps will reduce abandonment and increase completion rate.

Metrics:

- Primary: Checkout completion rate (conversions / checkout initiations)
- Secondary: Revenue per session, average order value
- Guardrails: Form errors, customer support contacts

Results:

- Completion rate increased from 68% to 71.5% ($p < 0.01$)
- Revenue per session up 6.2%
- No increase in form errors or support contacts

Business impact: With 100,000 monthly checkout initiations, 3.5 percentage point improvement yields 3,500 additional monthly conversions. At \$80 average order value: \$280,000 monthly revenue increase, or \$3.36M annually.

Listing 1.1: Basic A/B Test Analysis for Conversion Rates

```

1 import numpy as np
2 from scipy import stats
3 import pandas as pd
4
5 def analyze_conversion_test(control_conversions, control_total,
6                             treatment_conversions, treatment_total,
7                             alpha=0.05):
8     """
9     Analyze A/B test for conversion rates.
10
11     Parameters:
12     -----
13     control_conversions: int
14         Number of conversions in control
15     control_total: int
16         Total users in control
17     treatment_conversions: int
18         Number of conversions in treatment
19     treatment_total: int
20         Total users in treatment
21     alpha: float
22         Significance level (default 0.05)
23
24     Returns:
25     -----
26     dict: Test results including statistics and interpretation
27     """
28     # Calculate proportions
29     p_control = control_conversions / control_total
30     p_treatment = treatment_conversions / treatment_total
31
32     # Pooled proportion for null hypothesis
33     p_pooled = (control_conversions + treatment_conversions) / \
34                (control_total + treatment_total)
35
36     # Standard error under null
37     se_null = np.sqrt(p_pooled * (1 - p_pooled) *
38                       (1/control_total + 1/treatment_total))
39
40     # Z-statistic
41     z_stat = (p_treatment - p_control) / se_null
42
43     # Two-sided p-value
44     p_value = 2 * (1 - stats.norm.cdf(abs(z_stat)))
45

```

```

46     # Confidence interval (using unpooled variance)
47     se_unpooled = np.sqrt(p_control*(1-p_control)/control_total +
48                             p_treatment*(1-p_treatment)/treatment_total)
49     z_crit = stats.norm.ppf(1 - alpha/2)
50     ci_lower = (p_treatment - p_control) - z_crit * se_unpooled
51     ci_upper = (p_treatment - p_control) + z_crit * se_unpooled
52
53     # Calculate effect sizes
54     absolute_lift = p_treatment - p_control
55     relative_lift = absolute_lift / p_control if p_control > 0 else np.
        nan
56
57     # Determine statistical significance
58     is_significant = p_value < alpha
59
60     results = {
61         'control_rate': p_control,
62         'treatment_rate': p_treatment,
63         'absolute_lift': absolute_lift,
64         'relative_lift': relative_lift,
65         'z_statistic': z_stat,
66         'p_value': p_value,
67         'ci_lower': ci_lower,
68         'ci_upper': ci_upper,
69         'is_significant': is_significant,
70         'alpha': alpha
71     }
72
73     return results
74
75 # Example: Checkout flow test
76 results = analyze_conversion_test(
77     control_conversions=6800,
78     control_total=10000,
79     treatment_conversions=7150,
80     treatment_total=10000
81 )
82
83 print("A/B Test Results")
84 print("=" * 50)
85 print(f"Control conversion rate: {results['control_rate']:.1%}")
86 print(f"Treatment conversion rate: {results['treatment_rate']:.1%}")
87 print(f"\nAbsolute lift: {results['absolute_lift']:.1%}")
88 print(f"Relative lift: {results['relative_lift']:.1%}")
89 print(f"\nZ-statistic: {results['z_statistic']:.3f}")
90 print(f"P-value: {results['p_value']:.4f}")
91 print(f"95% CI: [{results['ci_lower']:.1%}, {results['ci_upper']:.1%}]")
92 print(f"\nStatistically significant: {results['is_significant']}")

```

1.6.2 Email Marketing Campaigns

Subject Line Testing

An e-commerce company tests three email subject lines for a promotional campaign:

- A (Control): "Weekend Sale - 20% Off"

- B: "Flash Sale: Save 20% This Weekend Only"
- C: "Your Exclusive 20% Weekend Discount"

Experimental design:

- Randomly assign 100,000 subscribers equally to three variants
- Send simultaneously (Saturday 9 AM)
- Measure open rate, click-through rate, conversion rate

Listing 1.2: Multi-Variant Email Test Analysis

```

1 def analyze_email_test(variants_data, metric='open_rate', alpha=0.05):
2     """
3     Analyze multi-variant email test with Bonferroni correction.
4
5     Parameters:
6     -----
7     variants_data: dict
8         Dictionary mapping variant names to (successes, total) tuples
9     metric: str
10        Metric name for reporting
11    alpha: float
12        Family-wise error rate
13
14    Returns:
15    -----
16    DataFrame: Comparison of all variants with adjusted p-values
17    """
18    variant_names = list(variants_data.keys())
19    n_variants = len(variant_names)
20    n_comparisons = n_variants * (n_variants - 1) // 2
21
22    # Bonferroni correction
23    alpha_adjusted = alpha / n_comparisons
24
25    results = []
26
27    # Pairwise comparisons
28    for i in range(n_variants):
29        for j in range(i + 1, n_variants):
30            var1, var2 = variant_names[i], variant_names[j]
31
32            successes1, total1 = variants_data[var1]
33            successes2, total2 = variants_data[var2]
34
35            # Perform test
36            test_results = analyze_conversion_test(
37                successes1, total1, successes2, total2, alpha_adjusted
38            )
39
40            results.append({
41                'comparison': f'{var2}_vs_{var1}',
42                f'{var1}_{metric}': test_results['control_rate'],
43                f'{var2}_{metric}': test_results['treatment_rate'],
44                'absolute_diff': test_results['absolute_lift'],

```

```

45         'relative_lift': test_results['relative_lift'],
46         'p_value': test_results['p_value'],
47         'significant': test_results['is_significant']
48     })
49
50     df = pd.DataFrame(results)
51
52     print(f"Email_Test_Results_{metric}")
53     print("=" * 70)
54     print(f"Number_of_comparisons: {n_comparisons}")
55     print(f"Bonferroni-adjusted_alpha: {alpha_adjusted:.4f}")
56     print()
57
58     return df
59
60 # Example data
61 email_data = {
62     'A_Control': (4200, 33333),      # Open rate ~12.6%
63     'B_Flash': (4500, 33333),      # Open rate ~13.5%
64     'C_Exclusive': (4800, 33334)   # Open rate ~14.4%
65 }
66
67 results_df = analyze_email_test(email_data, metric='open_rate')
68 print(results_df.to_string(index=False))

```

1.6.3 Product Feature Testing

Recommendation Algorithm

A streaming platform tests a new content recommendation algorithm against the existing system.

Hypothesis: Machine learning-based personalized recommendations will increase viewing time compared to popularity-based recommendations.

Experimental design:

- User-level randomization (consistent experience across sessions)
- 50/50 split: 5 million users per variant
- Duration: 4 weeks (capture weekly patterns, reduce novelty effects)

Metrics:

- Primary: Hours watched per user per week
- Secondary: Content diversity (unique titles watched), completion rate
- Guardrails: Churn rate, customer satisfaction scores

Results demonstrate heterogeneous treatment effects:

User Segment	Control	Treatment	Lift
New users (< 1 month)	8.2 hrs	10.1 hrs	+23%
Established (1-6 months)	12.5 hrs	13.8 hrs	+10%
Long-term (> 6 months)	15.3 hrs	15.1 hrs	-1%
Overall	12.4 hrs	13.2 hrs	+6.5%

New algorithm benefits new and moderately engaged users but provides no value (or slight harm) to long-term users who have strong preferences. This insight suggests opportunities for segment-specific optimization.

1.6.4 Pricing Experiments

Pricing is perhaps the most sensitive area for experimentation, requiring careful ethical and business considerations.

Subscription Tier Testing

A SaaS company tests two pricing strategies:

- Control: \$29/month, \$290/year (17% annual discount)
- Treatment: \$29/month, \$249/year (28% annual discount)

Hypothesis: Larger annual discount will increase annual plan adoption without significantly reducing revenue.

Key considerations:

- **Ethics:** All users see both options; no one charged different prices for same product
- **Segment:** Test on new signups only (existing customers unaffected)
- **Duration:** 8 weeks to capture full subscription cycle

Listing 1.3: Revenue Analysis for Pricing Test

```

1 def analyze_pricing_test(control_monthly, control_annual,
2                           treatment_monthly, treatment_annual,
3                           avg_months_subscribed=12):
4     """
5     Analyze pricing experiment considering customer lifetime value.
6
7     Parameters:
8     -----
9     control_monthly, treatment_monthly: tuple
10    count, price for monthly subscribers
11    control_annual, treatment_annual: tuple
12    count, price for annual subscribers
13    avg_months_subscribed: int
14    Average subscription duration in months
15
16    Returns:
17    -----
18    dict: Revenue metrics and comparison
19    """
20    # Unpack data
21    c_monthly_count, c_monthly_price = control_monthly
22    c_annual_count, c_annual_price = control_annual
23    t_monthly_count, t_monthly_price = treatment_monthly
24    t_annual_count, t_annual_price = treatment_annual
25
26    # Calculate annual plan adoption rate
27    control_total = c_monthly_count + c_annual_count

```

```

28     treatment_total = t_monthly_count + t_annual_count
29
30     c_annual_rate = c_annual_count / control_total
31     t_annual_rate = t_annual_count / treatment_total
32
33     # Calculate average revenue per customer (annualized)
34     c_avg_revenue = (
35         (c_monthly_count * c_monthly_price * avg_months_subscribed +
36          c_annual_count * c_annual_price) / control_total
37     )
38
39     t_avg_revenue = (
40         (t_monthly_count * t_monthly_price * avg_months_subscribed +
41          t_annual_count * t_annual_price) / treatment_total
42     )
43
44     # Statistical test on annual adoption rate
45     from scipy.stats import chi2_contingency
46
47     contingency_table = [
48         [c_annual_count, c_monthly_count],
49         [t_annual_count, t_monthly_count]
50     ]
51     chi2, p_value, dof, expected = chi2_contingency(contingency_table)
52
53     results = {
54         'control_annual_rate': c_annual_rate,
55         'treatment_annual_rate': t_annual_rate,
56         'annual_adoption_lift': (t_annual_rate - c_annual_rate) /
57             c_annual_rate,
58         'control_avg_revenue': c_avg_revenue,
59         'treatment_avg_revenue': t_avg_revenue,
60         'revenue_diff': t_avg_revenue - c_avg_revenue,
61         'revenue_diff_pct': (t_avg_revenue - c_avg_revenue) /
62             c_avg_revenue,
63         'chi2_statistic': chi2,
64         'p_value': p_value,
65         'significant': p_value < 0.05
66     }
67
68     return results
69
70 # Example: Pricing test data
71 pricing_results = analyze_pricing_test(
72     control_monthly=(2800, 29),      # 2800 monthly at $29
73     control_annual=(1200, 290),      # 1200 annual at $290
74     treatment_monthly=(2400, 29),    # 2400 monthly at $29
75     treatment_annual=(1600, 249)     # 1600 annual at $249
76 )
77
78 print("Pricing Test Results")
79 print("=" * 60)
80 print(f"Control annual adoption: {pricing_results['control_annual_rate']:.1%}")
81 print(f"Treatment annual adoption: {pricing_results['treatment_annual_rate']:.1%}")
82 print(f"Lift in annual adoption: {pricing_results['annual_adoption_lift']:.1%}")

```

```

81 print()
82 print(f"Control_avg_revenue:_{pricing_results['control_avg_revenue']:.2f}")
83 print(f"Treatment_avg_revenue:_{pricing_results['treatment_avg_revenue']:.2f}")
84 print(f"Revenue_difference:_{pricing_results['revenue_diff']:+.2f}_{f"({pricing_results['revenue_diff_pct']:+.1%})")")
85
86 print()
87 print(f"Chi-squared:_{pricing_results['chi2_statistic']:.2f},_{f"p-value:_{pricing_results['p_value']:.4f}")")
88
89 print(f"Statistically_significant:_{pricing_results['significant']}")

```

Results interpretation: Treatment increased annual plan adoption from 30% to 40%, but the lower price point resulted in slight decrease in average revenue per customer (from \$319.20 to \$314.10 annually). However, higher adoption of annual plans improves retention and reduces churn, making this a strategic win despite short-term revenue impact.

1.7 When A/B Testing Works (and When It Doesn't)

1.7.1 Ideal Scenarios for A/B Testing

A/B testing excels when:

1. **Sufficient traffic:** Adequate users/events to detect meaningful effects
2. **Clear metrics:** Quantifiable outcomes observable in reasonable timeframe
3. **Reversible changes:** Can easily roll back if needed
4. **Limited network effects:** Users' experiences are independent
5. **Ethical feasibility:** Randomization and experimentation are appropriate

1.7.2 When A/B Testing Struggles

Insufficient Sample Size

If your website receives 100 visitors weekly and conversion rate is 2%, detecting a 25% relative improvement (from 2% to 2.5%) requires approximately 6,300 users per variant—over a year of testing.

Long-Term Effects

Some interventions show different short-term and long-term effects. A/B tests typically run weeks to months. Evaluating impacts on 5-year customer lifetime value requires different approaches.

Rare Events

Testing impacts on events occurring once per 10,000 users requires enormous samples. Consider proxy metrics or specialized methods.

Fundamental Redesigns

Radical changes create such different experiences that incremental testing may not apply. Qualitative research, gradual rollouts, and careful monitoring complement experimentation.

Network Effects

Social networks, marketplaces, and communication platforms violate the "no interference" assumption. One user's treatment affects others. Specialized designs (Chapter 8) address this.

1.8 Summary

A/B testing provides a rigorous framework for learning from data and making evidence-based decisions. Its core principles—randomization, controlled comparison, statistical inference—enable causal conclusions that observational studies cannot support.

Key takeaways:

- A/B testing evolved from Fisher's agricultural experiments to become central to digital product development
- Randomization eliminates confounding and enables causal inference
- Proper terminology (control/treatment, metrics, significance, effect size) enables precise thinking
- Multiple test types (simple A/B, multivariate, sequential, bandits) suit different objectives
- Real-world applications span e-commerce, marketing, product features, and pricing
- Understanding both capabilities and limitations guides appropriate application

The subsequent chapters develop the statistical theory, implementation practices, and analytical techniques necessary for rigorous experimentation. We begin in Chapter 2 with probability and statistical foundations.

1.9 Further Reading

- Fisher [1935]: Original exposition of experimental design principles
- Kohavi et al. [2020]: Comprehensive modern treatment from industry leaders
- Box et al. [2005]: Classical text bridging theory and practice
- Imbens and Rubin [2015]: Rigorous causal inference framework

1.10 Exercises

1.10.1 Conceptual Questions

1. Understanding Randomization

- (a) Explain in your own words why randomization is essential for causal inference.
- (b) Describe a scenario where observational data would be misleading but a randomized experiment would reveal the truth.

- (c) What could go wrong if we assigned users to variants based on the first letter of their username instead of random assignment?

2. Metrics and Business Impact

- (a) A test shows 10% increase in clicks but no change in revenue. What might explain this? Would you ship the change?
- (b) Propose a complete metric hierarchy (primary, secondary, guardrails) for testing a new mobile app checkout flow.
- (c) Explain the difference between statistical significance and practical significance with a concrete example.

3. Experimental Design

- (a) When would you choose user-level over session-level randomization? Provide three scenarios for each.
- (b) A social network wants to test a new feature but worries about network effects. How would this concern affect their experimental design?
- (c) Describe the trade-offs between A/B testing and multi-armed bandits for selecting email subject lines.

1.10.2 Quantitative Problems

1. Sample Size Intuition

- (a) Your website has 10,000 daily visitors. How long must a test run to accumulate 20,000 users per variant?
- (b) If detecting a 5% effect requires 10,000 users per variant, approximately how many users are needed to detect a 2.5% effect (half as large)?
- (c) Explain why detecting small effects requires disproportionately more data.

2. Effect Size Calculations

- (a) Control: 8.2% conversion. Treatment: 9.0% conversion. Calculate absolute and relative lift.
- (b) A test increases average order value from \$45 to \$48. Express as absolute difference, relative lift, and Cohen's d (assume SD = \$15).
- (c) Why might the same 10% relative improvement have different business value at different baseline rates?

1.10.3 Programming Exercises

1. Basic Analysis

- (a) Use the provided code to analyze: Control (450 conversions / 5000 users), Treatment (520 conversions / 5000 users). Interpret all outputs.
- (b) Modify the code to calculate 90% confidence intervals instead of 95%. How does this change affect the width?

- (c) Implement a function that determines if an observed difference could plausibly be due to chance (p-value interpretation).

2. Simulation Study

- (a) Simulate 10,000 A/B tests where the null hypothesis is true (no effect). What proportion show $p < 0.05$? Why?
- (b) Simulate tests with 10% true relative lift. What proportion correctly detect this effect (statistical power)?
- (c) Create a visualization showing how power changes with sample size for a fixed effect size.

3. Real Data Analysis

- (a) Download a public A/B test dataset (e.g., from Kaggle). Perform complete analysis including data validation, statistical testing, and business recommendation.
- (b) Implement pre-analysis checks: sample ratio mismatch detection, temporal patterns, outlier identification.
- (c) Create a reusable analysis template you could apply to future experiments.

1.10.4 Critical Thinking

1. Case Analysis

- (a) Microsoft Bing's experiment (mentioned earlier) showed short-term revenue gains but long-term user satisfaction declines. Design an experimental protocol that would have detected this issue earlier.
- (b) A company runs 100 experiments simultaneously. Several show statistically significant improvements. What concerns should you have? How would you address them?
- (c) Critics argue that A/B testing leads to incremental optimization rather than breakthrough innovation. How would you respond? When is this criticism valid?

2. Ethical Considerations

- (a) A dating app wants to test whether showing fewer high-quality matches increases engagement (by creating scarcity). Is this ethical? What principles guide your answer?
- (b) When, if ever, should informed consent be required for A/B tests in commercial products?
- (c) Design ethical guidelines for experimentation in your domain of interest.

Chapter 2

Statistical Foundations

2.1 Learning Objectives

After completing this chapter, readers will be able to:

- Apply probability theory fundamentals to experimental design and analysis
- Work with probability distributions essential for A/B testing
- Understand and apply the Central Limit Theorem to justify normal approximations
- Construct and interpret confidence intervals for common metrics
- Distinguish between frequentist and Bayesian inference frameworks
- Formulate and test statistical hypotheses rigorously
- Calculate and interpret effect size measures for practical significance
- Implement statistical analyses using Python and R

2.2 Introduction

A/B testing rests on a foundation of probability theory and statistical inference. While modern software makes running experiments trivially easy, understanding the underlying mathematics is essential for:

1. **Correct Implementation:** Knowing when assumptions are violated
2. **Proper Interpretation:** Understanding what results actually mean
3. **Sound Decisions:** Distinguishing statistical significance from practical importance
4. **Effective Communication:** Explaining uncertainty to stakeholders

This chapter develops these foundations systematically, connecting abstract theory to concrete experimental practice. We emphasize not just the mechanics of calculation but the reasoning that justifies each method.

2.3 Probability Theory Fundamentals

Probability provides the language for reasoning about uncertainty—the fundamental challenge in A/B testing where we observe samples but want to make inferences about populations.

2.3.1 Probability Spaces and Random Variables

Definition 2.1 (Probability Space). A probability space is a triple $(\Omega, \mathcal{F}, P)$ where:

- Ω is the sample space (set of all possible outcomes)
- $\mathcal{F}$ is a σ -algebra of events (subsets of Ω)
- $P : \mathcal{F} \rightarrow [0, 1]$ is a probability measure satisfying:
 1. $P(\Omega) = 1$ (normalization)
 2. For disjoint events $A_1, A_2, \dots$: $P(\bigcup_{i=1}^{\infty} A_i) = \sum_{i=1}^{\infty} P(A_i)$ (countable additivity)

Example 2.1 (A/B Test Sample Space). In a simple A/B test where we observe whether a user converts:

- $\Omega = \{\text{convert}, \text{no convert}\}$
- $\mathcal{F} = \{\emptyset, \{\text{convert}\}, \{\text{no convert}\}, \Omega\}$
- $P(\{\text{convert}\}) = p$, $P(\{\text{no convert}\}) = 1 - p$

For n users, Ω becomes the set of all possible sequences of conversions and non-conversions, containing 2^n elements.

Definition 2.2 (Random Variable). A random variable X is a measurable function $X : \Omega \rightarrow \mathbb{R}$ that maps outcomes to real numbers. We distinguish:

- **Discrete random variables:** Taking countable values with probability mass function (PMF) $p(x) = P(X = x)$
- **Continuous random variables:** Taking uncountable values with probability density function (PDF) $f(x)$ where $P(a \leq X \leq b) = \int_a^b f(x)dx$

2.3.2 Expected Value and Variance

The expected value and variance are the most important summary statistics of a distribution, characterizing location and spread respectively.

Definition 2.3 (Expected Value). For discrete X :

$$\mathbb{E}[X] = \sum_x x \cdot p(x) \quad (2.1)$$

For continuous X :

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f(x)dx \quad (2.2)$$

The expected value represents the long-run average if we could repeat the experiment infinitely many times.

Definition 2.4 (Variance and Standard Deviation).

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \quad (2.3)$$

The standard deviation is $\sigma = \sqrt{\text{Var}(X)}$.

Variance measures the expected squared deviation from the mean—how spread out the distribution is. Standard deviation has the same units as X , making it more interpretable.

Essential Properties for A/B Testing

Proposition 2.1 (Linearity of Expectation). For random variables $X_1, \dots, X_n$ and constants $a_1, \dots, a_n, b$:

$$\mathbb{E} \left[b + \sum_{i=1}^n a_i X_i \right] = b + \sum_{i=1}^n a_i \mathbb{E}[X_i] \quad (2.4)$$

This holds **regardless of dependence** between variables—a remarkable fact that greatly simplifies calculations.

Proposition 2.2 (Variance of Linear Combinations). For independent random variables $X_1, \dots, X_n$ and constants $a_1, \dots, a_n, b$:

$$\text{Var} \left(b + \sum_{i=1}^n a_i X_i \right) = \sum_{i=1}^n a_i^2 \text{Var}(X_i) \quad (2.5)$$

Note: The constant b vanishes (shifting doesn't change spread), and independence is crucial.

Corollary 2.1 (Variance of Sample Mean). For $X_1, \dots, X_n$ independent with $\mathbb{E}[X_i] = \mu$ and $\text{Var}(X_i) = \sigma^2$:

$$\text{Var}(\bar{X}) = \text{Var} \left(\frac{1}{n} \sum_{i=1}^n X_i \right) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{\sigma^2}{n} \quad (2.6)$$

This fundamental result shows uncertainty decreases with $\sqrt{n}$, not n . Quadrupling sample size only halves standard error.

Example 2.2 (Computing Means and Variances). Consider an A/B test where users spend time on site. Let X_i represent time spent by user i , with individual $\mathbb{E}[X_i] = 5$ minutes and $\text{Var}(X_i) = 4$ minutes².

For $n = 100$ users:

$$\mathbb{E}[\bar{X}] = \mathbb{E} \left[\frac{1}{100} \sum_{i=1}^{100} X_i \right] = \frac{1}{100} \sum_{i=1}^{100} \mathbb{E}[X_i] = 5 \text{ minutes} \quad (2.7)$$

$$\text{Var}(\bar{X}) = \frac{4}{100} = 0.04 \text{ minutes}^2 \quad (2.8)$$

$$\text{SD}(\bar{X}) = \sqrt{0.04} = 0.2 \text{ minutes} \quad (2.9)$$

The sample mean has the same expected value as individual observations, but much less variability.

Listing 2.1: Simulating Expected Value and Variance

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from scipy import stats
5
6 sns.set_style("whitegrid")
7 np.random.seed(42)
8
9 # Simulate time spent on site (exponential distribution)
10 true_mean = 5 # minutes
11 n_users = 100
12 n_simulations = 10000
13
14 # Individual observations
15 individual_times = np.random.exponential(true_mean, size=(n_simulations
16     , n_users))
17
18 # Sample means
19 sample_means = individual_times.mean(axis=1)
20
21 # Theoretical values
22 theoretical_var_individual = true_mean**2 # For exponential
23 theoretical_var_sample_mean = theoretical_var_individual / n_users
24 theoretical_sd_sample_mean = np.sqrt(theoretical_var_sample_mean)
25
26 # Empirical values
27 empirical_mean = sample_means.mean()
28 empirical_sd = sample_means.std()
29
30 # Visualization
31 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
32
33 # Individual observations
34 axes[0].hist(individual_times[0, :], bins=50, density=True, alpha=0.7,
35     label='Sample data', color='steelblue')
36 x = np.linspace(0, 25, 1000)
37 axes[0].plot(x, stats.expon.pdf(x, scale=true_mean), 'r-', linewidth=2,
38     label=f'Theoretical PDF')
39 axes[0].axvline(true_mean, color='green', linestyle='--', linewidth=2,
40     label=f'True mean = {true_mean}')
41 axes[0].set_xlabel('Time Spent (minutes)', fontsize=12)
42 axes[0].set_ylabel('Density', fontsize=12)
43 axes[0].set_title('Individual User Times', fontsize=14, fontweight='
44     bold')
45 axes[0].legend()
46 axes[0].grid(alpha=0.3)
47
48 # Sample means
49 axes[1].hist(sample_means, bins=50, density=True, alpha=0.7,
50     label=f'Simulated sample means (n={n_simulations:,})',
51     color='steelblue')
52 axes[1].axvline(empirical_mean, color='red', linestyle='-', linewidth
53     =2,
54     label=f'Empirical mean = {empirical_mean:.3f}')
55 axes[1].axvline(true_mean, color='green', linestyle='--', linewidth=2,
56     label=f'True mean = {true_mean}')
57 axes[1].set_xlabel('Sample Mean (minutes)', fontsize=12)

```



```

54 axes[1].set_ylabel('Density', fontsize=12)
55 axes[1].set_title(f'Distribution of Sample Means (n={n_users})',
56                  fontsize=14, fontweight='bold')
57 axes[1].legend()
58 axes[1].grid(alpha=0.3)
59
60 plt.tight_layout()
61 plt.savefig('expectation_variance_demo.pdf', dpi=300, bbox_inches='
    tight')
62 plt.show()
63
64 # Print comparison
65 print(f"Theoretical SD of sample mean: {theoretical_sd_sample_mean:.4f}
    ")
66 print(f"Empirical SD of sample mean: {empirical_sd:.4f}")
67 print(f"Ratio (should be ~1): {empirical_sd /
    theoretical_sd_sample_mean:.4f}")

```

2.3.3 The Central Limit Theorem

The Central Limit Theorem (CLT) is arguably the most important result in statistics. It justifies using normal approximations in A/B testing, enabling inference even when individual observations are far from normally distributed.

Theorem 2.1 (Central Limit Theorem). Let $X_1, X_2, \dots, X_n$ be independent, identically distributed random variables with $\mathbb{E}[X_i] = \mu$ and $\text{Var}(X_i) = \sigma^2 < \infty$. Define the sample mean:

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i \quad (2.10)$$

Then as $n \rightarrow \infty$:

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1) \quad (2.11)$$

Equivalently, for large n :

$$\bar{X}_n \approx \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right) \quad (2.12)$$

Interpretation: The sample mean of *any* distribution (with finite variance) becomes approximately normally distributed for large sample sizes. The approximation quality improves as n increases.

Implications for A/B Testing

The CLT has profound practical consequences:

1. **Universality:** Whether testing conversions (Bernoulli), revenue (right-skewed), or engagement time (exponential), sample means are approximately normal for sufficient n .
2. **Quantifiable Uncertainty:** The standard error $\text{SE}(\bar{X}) = \sigma/\sqrt{n}$ precisely quantifies estimation uncertainty.

3. **Confidence Intervals:** Normal approximation enables constructing confidence intervals: $\bar{X} \pm z_{\alpha/2} \cdot \text{SE}(\bar{X})$.
4. **Hypothesis Testing:** Test statistics become approximately normal, enabling p-value calculation.

Example 2.3 (CLT for Conversion Rates). Users convert with probability $p = 0.05$. Individual conversions $X_i \sim \text{Bernoulli}(0.05)$ take only values 0 and 1—decidedly non-normal! The PMF is:

$$P(X_i = 0) = 0.95, \quad P(X_i = 1) = 0.05 \quad (2.13)$$

However, with $n = 1000$ users, the sample conversion rate $\hat{p} = \bar{X}_n$ is approximately:

$$\hat{p} \sim \mathcal{N}\left(0.05, \frac{0.05 \times 0.95}{1000}\right) = \mathcal{N}(0.05, 0.0000475) \quad (2.14)$$

Standard error: $\text{SE}(\hat{p}) = \sqrt{0.0000475} \approx 0.0069$ or about 0.69 percentage points.

A 95% confidence interval:

$$0.05 \pm 1.96 \times 0.0069 = [0.0365, 0.0635] \quad (2.15)$$

How Large is "Large Enough"?

A practical question: when is the normal approximation accurate? Rules of thumb:

- **For proportions:** $np \geq 5$ and $n(1 - p) \geq 5$
- **For symmetric distributions:** $n \geq 30$ often sufficient
- **For highly skewed distributions:** May need $n \geq 100$ or more

These are guidelines, not hard rules. Simulation can verify adequacy for specific cases.

Listing 2.2: Demonstrating the Central Limit Theorem

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from scipy.stats import norm, bernoulli, expon, kstest
5
6 sns.set_style("whitegrid")
7 np.random.seed(42)
8
9 def demonstrate_clt(distribution, params, sample_sizes, n_simulations
10                     =10000):
11     """
12     Demonstrate CLT for different distributions and sample sizes.
13
14     Parameters:
15     -----
16     distribution: str
17     'bernoulli', 'exponential', or 'uniform'
18     params: dict
19     Distribution parameters
20     sample_sizes: list

```

```

20 #####List of sample sizes to demonstrate
21 #####n_simulations: int
22 #####Number of simulations per sample size
23 #####"""
24     n_plots = len(sample_sizes)
25     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
26     axes = axes.flatten()
27
28     # Generate data based on distribution
29     if distribution == 'bernoulli':
30         p = params['p']
31         true_mean = p
32         true_std = np.sqrt(p * (1-p))
33         generate_sample = lambda size: bernoulli.rvs(p, size=size)
34         title_prefix = f"Bernoulli(p={p})"
35     elif distribution == 'exponential':
36         scale = params['scale']
37         true_mean = scale
38         true_std = scale
39         generate_sample = lambda size: expon.rvs(scale=scale, size=size
40         )
41         title_prefix = f"Exponential(lambda=1/{scale})"
42     elif distribution == 'uniform':
43         a, b = params['a'], params['b']
44         true_mean = (a + b) / 2
45         true_std = np.sqrt((b - a)**2 / 12)
46         generate_sample = lambda size: np.random.uniform(a, b, size)
47         title_prefix = f"Uniform({a}, {b})"
48
49     ks_results = []
50
51     for idx, n in enumerate(sample_sizes):
52         # Simulate sample means
53         sample_means = np.array([
54             generate_sample(n).mean()
55             for _ in range(n_simulations)
56         ])
57
58         # Theoretical normal distribution for sample mean
59         theoretical_mean = true_mean
60         theoretical_std = true_std / np.sqrt(n)
61
62         # Standardize for normality test
63         standardized = (sample_means - theoretical_mean) /
64             theoretical_std
65         ks_stat, p_value = kstest(standardized, 'norm')
66         ks_results.append((n, ks_stat, p_value))
67
68         # Create theoretical normal curve
69         x_range = np.linspace(
70             theoretical_mean - 4*theoretical_std,
71             theoretical_mean + 4*theoretical_std,
72             1000
73         )
74         theoretical_pdf = norm.pdf(x_range, theoretical_mean,
75             theoretical_std)
76
77         # Plot

```

```

75     ax = axes[idx]
76     ax.hist(sample_means, bins=50, density=True, alpha=0.7,
77            color='steelblue', label='Simulated_sample_means',
78            edgecolor='black')
79     ax.plot(x_range, theoretical_pdf, 'r-', linewidth=2.5,
80            label='Theoretical_Normal', alpha=0.9)
81     ax.axvline(true_mean, color='green', linestyle='--', linewidth
82               =2,
83               label=f'True_mean={true_mean:.3f}')
84
85     ax.set_title(f'{title_prefix}, n={n}\nSE={theoretical_std
86               :.4f}, ' +
87               f'KS_p-value={p_value:.4f}',
88               fontsize=11, fontweight='bold')
89     ax.set_xlabel('Sample_Mean', fontsize=10)
90     ax.set_ylabel('Density', fontsize=10)
91     ax.legend(fontsize=9)
92     ax.grid(alpha=0.3)
93
94     plt.tight_layout()
95     plt.savefig(f'clt_demonstration_{distribution}.pdf', dpi=300,
96               bbox_inches='tight')
97     plt.show()
98
99     # Print normality test results
100    print(f"\nKolmogorov-Smirnov test for normality ({title_prefix}):")
101    print(f"{n':>6} {KS_statistic':>15} {p_value':>10} {Assessment
102          ':>20}")
103    print("-" * 55)
104    for n, ks_stat, p_val in ks_results:
105        assessment = "Excellent_fit" if p_val > 0.05 else "Poor_fit"
106        print(f"{n:>6} {ks_stat:>15.4f} {p_val:>10.4f} {assessment:>20}
107              ")
108
109    # Demonstrate for different distributions
110    print("=" * 70)
111    print("CLT Demonstration: Bernoulli Distribution (conversion_rates)")
112    print("=" * 70)
113    demonstrate_clt('bernoulli', {'p': 0.05}, [10, 50, 200, 1000])
114
115    print("\n" + "=" * 70)
116    print("CLT Demonstration: Exponential Distribution (session_times)")
117    print("=" * 70)
118    demonstrate_clt('exponential', {'scale': 5}, [10, 50, 200, 1000])
119
120    print("\n" + "=" * 70)
121    print("CLT Demonstration: Uniform Distribution")
122    print("=" * 70)
123    demonstrate_clt('uniform', {'a': 0, 'b': 10}, [10, 50, 200, 1000])

```

2.3.4 Law of Large Numbers

Closely related to the CLT is the Law of Large Numbers, which formalizes the intuition that sample averages converge to population means.

Theorem 2.2 (Weak Law of Large Numbers). Let $X_1, X_2, \dots$ be independent, identically

distributed with $\mathbb{E}[X_i] = \mu$. Then for any $\epsilon > 0$:

$$P(|\bar{X}_n - \mu| > \epsilon) \rightarrow 0 \text{ as } n \rightarrow \infty \quad (2.16)$$

In words: the sample mean converges in probability to the population mean.

Distinction from CLT: The Law of Large Numbers tells us sample means get close to the truth. The CLT tells us how the sample mean is distributed around the truth, enabling quantification of uncertainty.

2.4 Key Probability Distributions for A/B Testing

Several distributions arise repeatedly in experimental analysis. Understanding their properties and relationships is essential for correct application.

2.4.1 Bernoulli and Binomial Distributions

Definition 2.5 (Bernoulli Distribution). A random variable $X \sim \text{Bernoulli}(p)$ represents a single binary trial, taking value 1 (success) with probability p and 0 (failure) with probability $1 - p$.

$$P(X = 1) = p, \quad P(X = 0) = 1 - p \quad (2.17)$$

$$\mathbb{E}[X] = p \quad (2.18)$$

$$\text{Var}(X) = p(1 - p) \quad (2.19)$$

Note that variance is maximized at $p = 0.5$ (maximum uncertainty) and approaches 0 as $p \rightarrow 0$ or $p \rightarrow 1$ (near certainty).

Definition 2.6 (Binomial Distribution). If $X_1, \dots, X_n$ are independent $\text{Bernoulli}(p)$ variables, their sum $Y = \sum_{i=1}^n X_i$ follows $\text{Binomial}(n, p)$, representing the number of successes in n trials.

$$P(Y = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, \dots, n \quad (2.20)$$

$$\mathbb{E}[Y] = np \quad (2.21)$$

$$\text{Var}(Y) = np(1 - p) \quad (2.22)$$

Practical Application: In A/B testing:

- X_i = whether user i converts (Bernoulli)
- Y = total number of conversions (Binomial)
- $\hat{p} = Y/n$ = observed conversion rate

2.4.2 Normal (Gaussian) Distribution

The normal distribution is ubiquitous in A/B testing, justified by the CLT.

Definition 2.7 (Normal Distribution). A random variable $X \sim \mathcal{N}(\mu, \sigma^2)$ has probability density function:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R} \quad (2.23)$$

Properties:

- $\mathbb{E}[X] = \mu$ (location parameter)
- $\text{Var}(X) = \sigma^2$ (scale parameter)
- Symmetric around μ
- $\approx 68\%$ of mass within $\mu \pm \sigma$, 95% within $\mu \pm 2\sigma$, 99.7% within $\mu \pm 3\sigma$

The **standard normal distribution** $Z \sim \mathcal{N}(0, 1)$ serves as reference. Any normal variable can be standardized:

$$Z = \frac{X - \mu}{\sigma} \sim \mathcal{N}(0, 1) \quad (2.24)$$

Proposition 2.3 (Properties of Normal Distribution). 1. **Linear transformation:** If $X \sim \mathcal{N}(\mu, \sigma^2)$ and $Y = aX + b$, then:

$$Y \sim \mathcal{N}(a\mu + b, a^2\sigma^2) \quad (2.25)$$

2. **Sum of independent normals:** If $X_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$ independently, then:

$$\sum_{i=1}^n X_i \sim \mathcal{N}\left(\sum_{i=1}^n \mu_i, \sum_{i=1}^n \sigma_i^2\right) \quad (2.26)$$

3. **Difference of independent normals:** Crucial for A/B testing:

$$X_1 - X_2 \sim \mathcal{N}(\mu_1 - \mu_2, \sigma_1^2 + \sigma_2^2) \quad (2.27)$$

2.4.3 Student's t-Distribution

When population variance is unknown (the usual case) and must be estimated from data, the t-distribution provides more accurate inference than the normal distribution, especially for small samples.

Definition 2.8 (Student's t-Distribution). If $Z \sim \mathcal{N}(0, 1)$ and $V \sim \chi_\nu^2$ independently, then:

$$T = \frac{Z}{\sqrt{V/\nu}} \sim t_\nu \quad (2.28)$$

where ν is the degrees of freedom parameter.

Key properties:

- Symmetric around 0, like the normal distribution

- Heavier tails than normal (more probability in extremes)
- As $\nu \rightarrow \infty$, $t_\nu \rightarrow \mathcal{N}(0, 1)$
- For $\nu \geq 30$, practically indistinguishable from normal

When to use:

- Comparing means when variance is estimated from data
- Small to moderate sample sizes ($n < 30$ per group)
- Constructing confidence intervals for means
- Two-sample t-tests in A/B experiments

Example 2.4 (t-Distribution in Practice). Comparing average order value between variants with $n_1 = 25$ and $n_2 = 25$:

- Degrees of freedom: $\nu = n_1 + n_2 - 2 = 48$
- Use t_{48} critical values, not $\mathcal{N}(0, 1)$
- For 95% CI: $t_{0.025, 48} = 2.011$ vs. $z_{0.025} = 1.96$

The difference is small but matters for proper uncertainty quantification.

With $n_1 = n_2 = 1000$, we'd have $\nu = 1998$, and $t_{0.025, 1998} \approx 1.961$ — essentially identical to the normal quantile.

2.4.4 Beta Distribution

The Beta distribution is the conjugate prior for the Bernoulli/Binomial likelihood in Bayesian A/B testing (Chapter 7).

Definition 2.9 (Beta Distribution). A random variable $X \sim \text{Beta}(\alpha, \beta)$ has support on $[0, 1]$ with PDF:

$$f(x) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1} (1-x)^{\beta-1}, \quad 0 \leq x \leq 1 \quad (2.29)$$

Properties:

$$\mathbb{E}[X] = \frac{\alpha}{\alpha + \beta} \quad (2.30)$$

$$\text{Var}(X) = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)} \quad (2.31)$$

$$\text{Mode} = \frac{\alpha - 1}{\alpha + \beta - 2} \quad (\text{for } \alpha, \beta > 1) \quad (2.32)$$

The Beta distribution is extremely flexible:

- $\text{Beta}(1, 1) = \text{Uniform}(0, 1)$ (non-informative prior)
- $\alpha = \beta$ gives symmetric distributions
- $\alpha < \beta$ skews left, $\alpha > \beta$ skews right

- Large $\alpha + \beta$ gives narrow distributions (strong prior beliefs)

Listing 2.3: Visualizing Key Distributions

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from scipy.stats import binom, norm, t, beta
5
6 sns.set_style("whitegrid")
7 np.random.seed(42)
8
9 fig = plt.figure(figsize=(16, 10))
10
11 # 1. Binomial distributions
12 ax1 = plt.subplot(2, 3, 1)
13 n = 100
14 for p in [0.1, 0.3, 0.5]:
15     x = np.arange(0, n+1)
16     pmf = binom.pmf(x, n, p)
17     ax1.plot(x, pmf, marker='o', markersize=2, label=f'p={p}', alpha
18             =0.7)
19 ax1.set_xlabel('Number of Successes', fontsize=11)
20 ax1.set_ylabel('Probability', fontsize=11)
21 ax1.set_title('Binomial(n=100, p) Distributions', fontsize=12,
22             fontweight='bold')
23 ax1.legend()
24 ax1.grid(alpha=0.3)
25
26 # 2. Normal distributions
27 ax2 = plt.subplot(2, 3, 2)
28 x = np.linspace(-8, 8, 1000)
29 for mu, sigma in [(0, 1), (0, 2), (2, 1)]:
30     pdf = norm.pdf(x, mu, sigma)
31     ax2.plot(x, pdf, linewidth=2.5,
32             label=f'mu={mu}, sigma={sigma}', alpha=0.7)
33 ax2.set_xlabel('x', fontsize=11)
34 ax2.set_ylabel('Density', fontsize=11)
35 ax2.set_title('Normal Distributions', fontsize=12, fontweight='bold')
36 ax2.legend()
37 ax2.grid(alpha=0.3)
38
39 # 3. t-distributions
40 ax3 = plt.subplot(2, 3, 3)
41 x = np.linspace(-4, 4, 1000)
42 ax3.plot(x, norm.pdf(x, 0, 1), linewidth=2.5,
43         label='Normal(0,1)', color='black', linestyle='--')
44 for df in [1, 3, 10, 30]:
45     pdf = t.pdf(x, df)
46     ax3.plot(x, pdf, linewidth=2, label=f'df={df}', alpha=0.7)
47 ax3.set_xlabel('x', fontsize=11)
48 ax3.set_ylabel('Density', fontsize=11)
49 ax3.set_title("Student's t-Distributions", fontsize=12, fontweight='
50             bold')
51 ax3.legend()
52 ax3.grid(alpha=0.3)
53
54 # 4. Beta distributions

```



```

52 ax4 = plt.subplot(2, 3, 4)
53 x = np.linspace(0, 1, 1000)
54 for alpha, beta_param in [(1, 1), (2, 5), (5, 2), (10, 10)]:
55     pdf = beta.pdf(x, alpha, beta_param)
56     ax4.plot(x, pdf, linewidth=2.5,
57             label=f'alpha={alpha}, beta={beta_param}', alpha=0.7)
58 ax4.set_xlabel('x', fontsize=11)
59 ax4.set_ylabel('Density', fontsize=11)
60 ax4.set_title('Beta Distributions', fontsize=12, fontweight='bold')
61 ax4.legend()
62 ax4.grid(alpha=0.3)
63
64 # 5. Normal approximation to Binomial
65 ax5 = plt.subplot(2, 3, 5)
66 n, p = 100, 0.3
67 x_binom = np.arange(0, n+1)
68 pmf = binom.pmf(x_binom, n, p)
69 ax5.bar(x_binom, pmf, alpha=0.6, label='Binomial(100, 0.3)', color='
    steelblue')
70
71 mu = n * p
72 sigma = np.sqrt(n * p * (1-p))
73 x_norm = np.linspace(0, n, 1000)
74 pdf_norm = norm.pdf(x_norm, mu, sigma)
75 ax5.plot(x_norm, pdf_norm, 'r-', linewidth=2.5,
76         label=f'Normal({mu:.0f}, {sigma:.2f}^2)', alpha=0.9)
77 ax5.set_xlabel('Number of Successes', fontsize=11)
78 ax5.set_ylabel('Probability/Density', fontsize=11)
79 ax5.set_title('Normal Approximation to Binomial', fontsize=12,
80             fontweight='bold')
81 ax5.legend()
82 ax5.grid(alpha=0.3)
83
84 # 6. Comparison of confidence intervals: z vs t
85 ax6 = plt.subplot(2, 3, 6)
86 sample_sizes = np.arange(5, 101, 5)
87 z_critical = norm.ppf(0.975)
88 t_critical = [t.ppf(0.975, n-1) for n in sample_sizes]
89
90 ax6.plot(sample_sizes, t_critical, 'b-', linewidth=2.5,
91         label='t_critical value', marker='o', markersize=4)
92 ax6.axhline(z_critical, color='red', linestyle='--', linewidth=2.5,
93         label=f'z_critical value={z_critical:.3f}')
94 ax6.set_xlabel('Sample Size', fontsize=11)
95 ax6.set_ylabel('Critical Value (two-tailed, α=0.05)', fontsize=11)
96 ax6.set_title('t vs Normal Critical Values', fontsize=12, fontweight='
    bold')
97 ax6.legend()
98 ax6.grid(alpha=0.3)
99
100 plt.tight_layout()
101 plt.savefig('distributions_visualization.pdf', dpi=300, bbox_inches='
    tight')
102 plt.show()
103
104 # Print practical insights
105 print("\n" + "="*70)
106 print("Practical Insights on Distribution Selection")

```

```

106 print("="*70)
107 print(f"\nt-distribution vs Normal:")
108 print(f"Sample size t-critical (95% CI) z-critical Difference")
109 print("-" * 70)
110 for n in [5, 10, 20, 30, 50, 100]:
111     t_crit = t.ppf(0.975, n-1)
112     z_crit = norm.ppf(0.975)
113     diff = (t_crit - z_crit) / z_crit * 100
114     print(f"n={n:3d} {t_crit:.4f} {z_crit:.4f} {diff:+.2f}%")

```

2.5 Statistical Inference

Statistical inference is the process of drawing conclusions about populations from samples. The fundamental challenge: we observe sample data but want to make statements about unobserved population parameters.

2.5.1 Population vs. Sample

- **Population:** The complete set of all individuals/units of interest
 - Parameters (fixed, unknown): μ, σ, p
 - What we want to know about
- **Sample:** A subset of the population that we actually observe
 - Statistics (random, observed): $\bar{x}, s, \hat{p}$
 - What we use to learn about parameters

Example 2.5 (Population vs. Sample in A/B Testing). • **Population:** All potential users who could ever see our website

- **Parameter:** True conversion rate p for each variant
- **Sample:** Users in our experiment (e.g., 10,000 users)
- **Statistic:** Observed conversion rate $\hat{p} = 0.078$
- **Goal:** Use $\hat{p}$ to infer p and compare variants

2.5.2 Sampling Distributions

A sampling distribution is the probability distribution of a statistic over repeated samples from the same population.

Definition 2.10 (Sampling Distribution). If we:

1. Draw a random sample of size n from a population
2. Calculate a statistic (e.g., sample mean $\bar{X}$)

3. Repeat this process infinitely many times

The distribution of all those statistics is the sampling distribution.

Key insight: Even though we observe only one sample, we can deduce properties of the sampling distribution using probability theory (especially the CLT). This enables quantifying uncertainty.

2.5.3 Standard Error

The standard error (SE) is the standard deviation of a sampling distribution—it quantifies how much a statistic varies across repeated samples.

Definition 2.11 (Standard Error). For a statistic $\hat{\theta}$ with sampling distribution variance $\text{Var}(\hat{\theta})$:

$$\text{SE}(\hat{\theta}) = \sqrt{\text{Var}(\hat{\theta})} \quad (2.33)$$

Common standard errors:

- **Sample mean:** $\text{SE}(\bar{X}) = \sigma/\sqrt{n}$
- **Sample proportion:** $\text{SE}(\hat{p}) = \sqrt{p(1-p)/n}$
- **Difference in proportions:** $\text{SE}(\hat{p}_1 - \hat{p}_2) = \sqrt{\frac{p_1(1-p_1)}{n_1} + \frac{p_2(1-p_2)}{n_2}}$

In practice, we estimate SE by plugging in sample values for unknown parameters:

$$\widehat{\text{SE}}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}} \quad (2.34)$$

2.5.4 Confidence Intervals

Confidence intervals provide a range of plausible values for a parameter, quantifying estimation uncertainty.

Definition 2.12 (Confidence Interval). A $(1-\alpha) \times 100\%$ confidence interval for parameter θ is a random interval $[L, U]$ constructed from sample data such that:

$$P(L \leq \theta \leq U) = 1 - \alpha \quad (2.35)$$

before we observe the data. Common choices: $\alpha = 0.05$ (95% CI) or $\alpha = 0.01$ (99% CI).

Correct interpretation: "If we repeated this experiment many times and constructed a 95% CI each time, approximately 95% of those intervals would contain the true parameter value."

Common misinterpretation: "There is a 95% probability the true parameter is in this specific interval." (This is a Bayesian interpretation, not frequentist!)

Confidence Interval for a Proportion

For large n , by the CLT:

$$\hat{p} \sim \mathcal{N}\left(p, \frac{p(1-p)}{n}\right) \quad (2.36)$$

A $(1 - \alpha)$ confidence interval (Wald interval):

$$\hat{p} \pm z_{\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{n}} \quad (2.37)$$

where $z_{\alpha/2} = \Phi^{-1}(1 - \alpha/2)$ is the standard normal quantile. For $\alpha = 0.05$: $z_{0.025} = 1.96$.

Example 2.6 (CI for Conversion Rate). Experiment results: 85 conversions from 1,000 users.

$$\hat{p} = 85/1000 = 0.085 \quad (2.38)$$

$$\widehat{\text{SE}}(\hat{p}) = \sqrt{\frac{0.085 \times 0.915}{1000}} = 0.00882 \quad (2.39)$$

95% CI:

$$0.085 \pm 1.96 \times 0.00882 = [0.0677, 0.1023] \quad (2.40)$$

Interpretation: We are 95% confident the true conversion rate is between 6.77% and 10.23%.

Confidence Interval for Difference in Proportions

For comparing two independent proportions (the core of A/B testing):

$$(\hat{p}_2 - \hat{p}_1) \pm z_{\alpha/2} \sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{n_1} + \frac{\hat{p}_2(1-\hat{p}_2)}{n_2}} \quad (2.41)$$

If the CI contains 0, we cannot conclude a significant difference at level α . If the CI is entirely positive (or negative), we have evidence of a difference.

Listing 2.4: Computing Confidence Intervals

```

1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 sns.set_style("whitegrid")
7
8 def proportion_ci(conversions, n, confidence=0.95, method='wald'):
9     """
10     Calculate confidence interval for a proportion.
11
12     Parameters:
13     -----
14     conversions: int
15     Number of successes

```

```

16  #####n: int
17  #####Total number of trials
18  #####confidence: float
19  #####Confidence level (default 0.95)
20  #####method: str
21  #####'wald' (default), 'wilson', or 'agresti-coull'
22
23  #####Returns:
24  #####-----
25  #####dict with 'estimate', 'ci_lower', 'ci_upper', 'se'
26  #####"""
27      p_hat = conversions / n
28      alpha = 1 - confidence
29      z = stats.norm.ppf(1 - alpha/2)
30
31      if method == 'wald':
32          # Standard Wald interval
33          se = np.sqrt(p_hat * (1 - p_hat) / n)
34          ci_lower = p_hat - z * se
35          ci_upper = p_hat + z * se
36
37      elif method == 'wilson':
38          # Wilson score interval (better for small p or n)
39          denominator = 1 + z**2/n
40          center = (p_hat + z**2/(2*n)) / denominator
41          margin = z * np.sqrt(p_hat*(1-p_hat)/n + z**2/(4*n**2)) /
42                  denominator
43          ci_lower = center - margin
44          ci_upper = center + margin
45          se = np.sqrt(p_hat * (1 - p_hat) / n)
46          p_hat = center # Wilson estimate
47
48      elif method == 'agresti-coull':
49          # Agresti-Coull interval (add 2 successes and 2 failures)
50          n_tilde = n + z**2
51          p_tilde = (conversions + z**2/2) / n_tilde
52          se = np.sqrt(p_tilde * (1 - p_tilde) / n_tilde)
53          ci_lower = p_tilde - z * se
54          ci_upper = p_tilde + z * se
55          p_hat = p_tilde
56
57      return {
58          'estimate': p_hat,
59          'ci_lower': max(0, ci_lower), # Ensure in [0,1]
60          'ci_upper': min(1, ci_upper),
61          'se': se
62      }
63
64  def proportion_diff_ci(conv1, n1, conv2, n2, confidence=0.95):
65      """
66      #####Calculate confidence interval for difference in proportions.
67
68      #####Parameters:
69      #####-----
70      #####conv1, conv2: int
71      #####Number of conversions in each group
72      #####n1, n2: int

```

```

73 #####Sample_sizes
74 #####confidence_:float
75 #####Confidence_level
76
77 #####Returns:
78 #####-----
79 #####dict_with_difference_estimates_and_CI
80 #####"""
81     p1_hat = conv1 / n1
82     p2_hat = conv2 / n2
83     diff = p2_hat - p1_hat
84
85     se = np.sqrt(p1_hat * (1 - p1_hat) / n1 +
86                 p2_hat * (1 - p2_hat) / n2)
87
88     alpha = 1 - confidence
89     z = stats.norm.ppf(1 - alpha/2)
90
91     ci_lower = diff - z * se
92     ci_upper = diff + z * se
93
94     # Relative lift
95     relative_lift = (p2_hat - p1_hat) / p1_hat if p1_hat > 0 else np.
96         nan
97
98     return {
99         'p1': p1_hat,
100         'p2': p2_hat,
101         'difference': diff,
102         'relative_lift': relative_lift,
103         'ci_lower': ci_lower,
104         'ci_upper': ci_upper,
105         'se': se,
106         'significant': not (ci_lower <= 0 <= ci_upper)
107     }
108
109 # Example: Compare CI methods
110 print("="*70)
111 print("Comparing_Confidence_Interval_Methods")
112 print("="*70)
113
114 scenarios = [
115     (50, 500, "Moderate_p,_large_n"),
116     (5, 50, "Small_p,_small_n"),
117     (250, 500, "p_near_0.5,_large_n"),
118     (2, 100, "Very_small_p")
119 ]
120
121 for conversions, n, description in scenarios:
122     print(f"\n{description}:_{conversions}/{n}_conversions")
123     print("-" * 70)
124
125     for method in ['wald', 'wilson', 'agresti-coull']:
126         result = proportion_ci(conversions, n, method=method)
127         width = result['ci_upper'] - result['ci_lower']
128         print(f"{method:15s}:_{[{result['ci_lower']:.4f},_{result['ci_upper']:.4f}]}_ +

```

```

129         f"width_{width:.4f}")
130
131 # Example: A/B test comparison
132 print("\n" + "="*70)
133 print("A/B Test: Confidence Interval for Difference")
134 print("="*70)
135
136 control_conv, control_n = 80, 1000
137 treatment_conv, treatment_n = 95, 1000
138
139 result = proportion_diff_ci(control_conv, control_n,
140                             treatment_conv, treatment_n)
141
142 print(f"\nControl: {result['p1']:.2%} conversion rate ({control_conv} / {control_n})")
143 print(f"Treatment: {result['p2']:.2%} conversion rate ({treatment_conv} / {treatment_n})")
144 print(f"\nAbsolute difference: {result['difference']:.4f} ({result['difference']*100:.2f} pp)")
145 print(f"Relative lift: {result['relative_lift']:.2%}")
146 print(f"\n95% CI for difference: [{result['ci_lower']:.4f}, {result['ci_upper']:.4f}]")
147 print(f"Significant at  $\alpha=0.05$ : {result['significant']}")
148
149 # Visualize confidence intervals
150 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
151
152 # Plot 1: CI comparison across methods
153 ax1 = axes[0]
154 methods = ['Wald', 'Wilson', 'Agresti-Coull']
155 estimates = []
156 lowers = []
157 uppers = []
158
159 for method in ['wald', 'wilson', 'agresti-coull']:
160     ci = proportion_ci(50, 500, method=method)
161     estimates.append(ci['estimate'])
162     lowers.append(ci['ci_lower'])
163     uppers.append(ci['ci_upper'])
164
165 y_pos = np.arange(len(methods))
166 errors = np.array([[est - low for est, low in zip(estimates, lowers)],
167                   [up - est for up, est in zip(uppers, estimates)]])
168
169 ax1.errorbar(estimates, y_pos, xerr=errors, fmt='o', markersize=8,
170              capsize=5, capthick=2)
171 ax1.set_yticks(y_pos)
172 ax1.set_yticklabels(methods)
173 ax1.set_xlabel('Conversion Rate', fontsize=12)
174 ax1.set_title('95% CI Methods Comparison (50/500 conversions)',
175               fontsize=12, fontweight='bold')
176 ax1.grid(axis='x', alpha=0.3)
177
178 # Plot 2: A/B test visualization
179 ax2 = axes[1]
180 groups = ['Control', 'Treatment']
181 conversion_rates = [result['p1'], result['p2']]
182 ci_control = proportion_ci(control_conv, control_n)

```

```

182 ci_treatment = proportion_ci(treatment_conv, treatment_n)
183
184 errors = np.array([
185     [ci_control['estimate'] - ci_control['ci_lower'],
186      ci_treatment['estimate'] - ci_treatment['ci_lower']],
187     [ci_control['ci_upper'] - ci_control['estimate'],
188      ci_treatment['ci_upper'] - ci_treatment['estimate']]
189 ])
190
191 x_pos = np.arange(len(groups))
192 ax2.bar(x_pos, conversion_rates, alpha=0.7, color=['steelblue', 'coral',
193 ])
194 ax2.errorbar(x_pos, conversion_rates, yerr=errors, fmt='none',
195              ecolord='black', capsize=10, capthick=2)
196 ax2.set_xticks(x_pos)
197 ax2.set_xticklabels(groups, fontsize=12)
198 ax2.set_ylabel('Conversion_Rate', fontsize=12)
199 ax2.set_title('A/B Test Comparison with 95% CIs', fontsize=12,
200               fontweight='bold')
201 ax2.grid(axis='y', alpha=0.3)
202
203 # Add significance indicator
204 if result['significant']:
205     ax2.text(0.5, max(conversion_rates)*1.1, 'Significant difference (p
206                 <0.05)',
207             ha='center', fontsize=11, bbox=dict(boxstyle='round',
208                 facecolor='lightgreen'))
209 else:
210     ax2.text(0.5, max(conversion_rates)*1.1, 'No significant difference
211                 (p>0.05)',
212             ha='center', fontsize=11, bbox=dict(boxstyle='round',
213                 facecolor='lightyellow'))
214
215 plt.tight_layout()
216 plt.savefig('confidence_intervals_comparison.pdf', dpi=300, bbox_inches
217             = 'tight')
218 plt.show()

```

2.6 Hypothesis Testing Framework

Hypothesis testing provides a formal framework for making decisions under uncertainty. It's the workhorse methodology for A/B testing analysis.

2.6.1 The Logic of Hypothesis Testing

Hypothesis testing uses proof by contradiction:

1. Assume the null hypothesis (typically "no effect") is true
2. Calculate: "If null were true, how surprising is our observed data?"
3. If data is very surprising under the null, reject the null
4. Otherwise, fail to reject (we don't "accept" the null)

Definition 2.13 (Null and Alternative Hypotheses). • **Null hypothesis (H_0):** The default assumption, typically "no effect" or "no difference"

• **Alternative hypothesis (H_1 or H_A):** What we suspect or want to demonstrate
In A/B testing:

$$H_0 : p_{\text{treatment}} = p_{\text{control}} \quad (\text{no difference}) \quad (2.42)$$

$$H_1 : p_{\text{treatment}} \neq p_{\text{control}} \quad (\text{two-sided}) \quad (2.43)$$

$$\text{or } H_1 : p_{\text{treatment}} > p_{\text{control}} \quad (\text{one-sided}) \quad (2.44)$$

2.6.2 Test Statistics and p-values

Definition 2.14 (Test Statistic). A test statistic is a function of sample data used to test hypotheses. It quantifies how far the data deviates from what we'd expect under H_0 .

For comparing proportions:

$$z = \frac{(\hat{p}_2 - \hat{p}_1) - 0}{\text{SE}_0(\hat{p}_2 - \hat{p}_1)} \quad (2.45)$$

where SE_0 is computed under the null hypothesis of equal proportions.

Definition 2.15 (p-value). The p-value is the probability of observing a test statistic at least as extreme as what we observed, assuming H_0 is true:

$$p\text{-value} = P(\text{test statistic} \geq \text{observed value} \mid H_0 \text{ true}) \quad (2.46)$$

Interpretation:

- Small p-value (< 0.05): Data is surprising under H_0 , reject H_0
- Large p-value (≥ 0.05): Data is consistent with H_0 , fail to reject

Critical point: The p-value is NOT the probability that H_0 is true. It's the probability of seeing data this extreme if H_0 were true—a subtle but crucial distinction.

2.6.3 Significance Level and Decision Rule

Definition 2.16 (Significance Level). The significance level α is the threshold for rejecting H_0 . Common choices:

- $\alpha = 0.05$ (5%): Standard in most applications
- $\alpha = 0.01$ (1%): More conservative, reduces false positives
- $\alpha = 0.10$ (10%): More liberal, increases power but more false positives

Decision rule: Reject H_0 if $p\text{-value} < \alpha$

Reality	Fail to Reject H_0	Reject H_0
H_0 true	Correct decision $(1-\alpha)$	Type I Error (α)
H_1 true	Type II Error (β)	Correct decision $(1-\beta)$

Table 2.1: Type I and Type II errors in hypothesis testing

2.6.4 Type I and Type II Errors

Hypothesis testing involves two types of errors:

Definition 2.17 (Type I Error (False Positive)). Rejecting H_0 when it's actually true—declaring a difference when none exists.

$$P(\text{Type I Error}) = \alpha \quad (2.47)$$

In A/B testing: Launching a "winning" variant that's actually no better (or worse) than control.

Definition 2.18 (Type II Error (False Negative)). Failing to reject H_0 when H_1 is true—missing a real effect.

$$P(\text{Type II Error}) = \beta \quad (2.48)$$

In A/B testing: Abandoning a variant that would actually improve metrics.

Definition 2.19 (Statistical Power). Power is the probability of correctly rejecting H_0 when H_1 is true:

$$\text{Power} = 1 - \beta = P(\text{reject } H_0 \mid H_1 \text{ true}) \quad (2.49)$$

Desired power is typically 0.80 (80%) or 0.90 (90%). Chapter 4 covers power analysis in depth.

Tradeoff: Decreasing α (being more conservative about declaring significance) increases β (reduces power) for fixed sample size. The only way to decrease both is to increase sample size.

2.6.5 Two-Sample Z-Test for Proportions

The workhorse test for A/B testing with binary outcomes.

Setup:

- Control: n_1 trials, x_1 successes, $\hat{p}_1 = x_1/n_1$
- Treatment: n_2 trials, x_2 successes, $\hat{p}_2 = x_2/n_2$

Hypotheses:

$$H_0 : p_1 = p_2 \quad (2.50)$$

$$H_1 : p_1 \neq p_2 \quad (\text{two-sided}) \quad (2.51)$$

Under H_0 : Pool data to estimate common proportion:

$$\hat{p} = \frac{x_1 + x_2}{n_1 + n_2} \quad (2.52)$$

Test statistic:

$$z = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{\hat{p}(1 - \hat{p}) \left(\frac{1}{n_1} + \frac{1}{n_2} \right)}} \quad (2.53)$$

Under H_0 , $z \sim \mathcal{N}(0, 1)$ approximately (by CLT).

p-value (two-sided):

$$p = 2 \cdot P(Z \geq |z|) = 2 \cdot (1 - \Phi(|z|)) \quad (2.54)$$

where Φ is the standard normal CDF.

Example 2.7 (Z-Test for A/B Experiment). • Control: 1,200 users, 96 conversions, $\hat{p}_1 = 0.08$

• Treatment: 1,200 users, 120 conversions, $\hat{p}_2 = 0.10$

Pooled proportion:

$$\hat{p} = \frac{96 + 120}{1200 + 1200} = \frac{216}{2400} = 0.09 \quad (2.55)$$

Test statistic:

$$z = \frac{0.10 - 0.08}{\sqrt{0.09 \times 0.91 \times (1/1200 + 1/1200)}} = \frac{0.02}{0.01168} = 1.712 \quad (2.56)$$

p-value:

$$p = 2 \cdot P(Z \geq 1.712) = 2 \cdot (1 - 0.9566) = 0.0868 \quad (2.57)$$

Conclusion: With $p = 0.0868 > 0.05$, we fail to reject H_0 at the 5% level. Insufficient evidence to conclude treatment differs from control, despite the 2 percentage point observed difference.

Listing 2.5: Implementing Hypothesis Tests

```

1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 sns.set_style("whitegrid")
7
8 def z_test_proportions(conv1, n1, conv2, n2, alpha=0.05, alternative='
9     two-sided'):
10     """
11     Two-sample z-test for proportions.
12
13     Parameters:
14     -----
15     conv1, conv2: int
16         Number of conversions in each group
17     n1, n2: int
18         Sample sizes
19     alpha: float
20         Significance level
21     alternative: str
22         'two-sided', 'greater', or 'less'

```

```

23  """Returns:
24  """-----
25  """dict with test results
26  """
27  p1_hat = conv1 / n1
28  p2_hat = conv2 / n2
29
30  # Pooled proportion under H0
31  p_pool = (conv1 + conv2) / (n1 + n2)
32
33  # Standard error under H0
34  se_null = np.sqrt(p_pool * (1 - p_pool) * (1/n1 + 1/n2))
35
36  # Test statistic
37  z_stat = (p2_hat - p1_hat) / se_null
38
39  # p-value
40  if alternative == 'two-sided':
41      p_value = 2 * (1 - stats.norm.cdf(abs(z_stat)))
42  elif alternative == 'greater':
43      p_value = 1 - stats.norm.cdf(z_stat)
44  elif alternative == 'less':
45      p_value = stats.norm.cdf(z_stat)
46
47  # Critical value
48  if alternative == 'two-sided':
49      z_critical = stats.norm.ppf(1 - alpha/2)
50  else:
51      z_critical = stats.norm.ppf(1 - alpha)
52
53  # Decision
54  reject_null = p_value < alpha
55
56  # Effect size (relative lift)
57  relative_lift = (p2_hat - p1_hat) / p1_hat if p1_hat > 0 else np.
    nan
58
59  return {
60      'p1': p1_hat,
61      'p2': p2_hat,
62      'p_pool': p_pool,
63      'difference': p2_hat - p1_hat,
64      'relative_lift': relative_lift,
65      'z_statistic': z_stat,
66      'p_value': p_value,
67      'alpha': alpha,
68      'z_critical': z_critical,
69      'reject_null': reject_null,
70      'significant': reject_null
71  }
72
73
74  def visualize_hypothesis_test(result):
75      """Visualize the hypothesis test result."""
76      fig, axes = plt.subplots(1, 2, figsize=(14, 5))
77
78      # Plot 1: Conversion rates with error bars
79      ax1 = axes[0]

```

```

80 groups = ['Control', 'Treatment']
81 rates = [result['p1'], result['p2']]
82 colors = ['steelblue', 'coral']
83
84 bars = ax1.bar(groups, rates, color=colors, alpha=0.7, edgecolor='
      black', linewidth=1.5)
85 ax1.set_ylabel('Conversion_Rate', fontsize=12)
86 ax1.set_title('Conversion_Rates_Comparison', fontsize=13,
      fontweight='bold')
87 ax1.grid(axis='y', alpha=0.3)
88
89 # Add value labels
90 for i, (bar, rate) in enumerate(zip(bars, rates)):
91     height = bar.get_height()
92     ax1.text(bar.get_x() + bar.get_width()/2., height,
93             f'{rate:.2%}',
94             ha='center', va='bottom', fontsize=11, fontweight='bold'
95             ')
96
97 # Add significance indicator
98 y_max = max(rates) * 1.15
99 if result['significant']:
100     ax1.plot([0, 1], [y_max, y_max], 'k-', linewidth=2)
101     ax1.text(0.5, y_max*1.02, f'p={result["p_value"]:.4f}* ',
102             ha='center', fontsize=11, fontweight='bold',
103             bbox=dict(boxstyle='round', facecolor='lightgreen',
104                     alpha=0.7))
105 else:
106     ax1.text(0.5, y_max, f'p={result["p_value"]:.4f}\n(not
107             significant)',
108             ha='center', fontsize=11,
109             bbox=dict(boxstyle='round', facecolor='lightyellow',
110                     alpha=0.7))
111
112 # Plot 2: Standard normal distribution with test statistic
113 ax2 = axes[1]
114 x = np.linspace(-4, 4, 1000)
115 y = stats.norm.pdf(x, 0, 1)
116
117 ax2.plot(x, y, 'b-', linewidth=2, label='Null_distribution_N(0,1)')
118 ax2.fill_between(x, 0, y, where=(x <= -result['z_critical']) | (x
119     >= result['z_critical']),
120                 alpha=0.3, color='red', label=f'Rejection_region_
121                 alpha={result["alpha"]}')
122
123 # Mark observed z-statistic
124 ax2.axvline(result['z_statistic'], color='darkred', linestyle='--',
125             linewidth=2.5, label=f'Observed_z={result["z_statistic"]:.3f}')
126
127 ax2.set_xlabel('z-value', fontsize=12)
128 ax2.set_ylabel('Density', fontsize=12)
129 ax2.set_title('Hypothesis_Test_Visualization', fontsize=13,
130             fontweight='bold')
131 ax2.legend(fontsize=10)
132 ax2.grid(alpha=0.3)
133
134 plt.tight_layout()

```

```

128     return fig
129
130
131 # Example: Run multiple tests
132 print("="*80)
133 print("Hypothesis Testing Examples")
134 print("="*80)
135
136 test_scenarios = [
137     {
138         'name': 'Significant_effect_(large_sample)',
139         'conv1': 80, 'n1': 1000,
140         'conv2': 120, 'n2': 1000
141     },
142     {
143         'name': 'Non-significant_effect_(large_sample)',
144         'conv1': 80, 'n1': 1000,
145         'conv2': 95, 'n2': 1000
146     },
147     {
148         'name': 'Significant_effect_(small_sample)',
149         'conv1': 8, 'n1': 50,
150         'conv2': 20, 'n2': 50
151     },
152     {
153         'name': 'Non-significant_effect_(small_sample)',
154         'conv1': 4, 'n1': 40,
155         'conv2': 8, 'n2': 40
156     }
157 ]
158
159 for i, scenario in enumerate(test_scenarios, 1):
160     print(f"\n{'='*80}")
161     print(f"Scenario_{i}: {scenario['name']}")
162     print('='*80)
163
164     result = z_test_proportions(
165         scenario['conv1'], scenario['n1'],
166         scenario['conv2'], scenario['n2']
167     )
168
169     print(f"\nControl: {result['p1']:.2%} ({scenario['conv1']}/{scenario['n1']} conversions)")
170     print(f"Treatment: {result['p2']:.2%} ({scenario['conv2']}/{scenario['n2']} conversions)")
171     print(f"\nAbsolute difference: {result['difference']:.4f} ({result['difference']*100:.2f}pp)")
172     print(f"Relative lift: {result['relative_lift']:.2%}")
173     print(f"\nPooled proportion: {result['p_pool']:.4f}")
174     print(f"z-statistic: {result['z_statistic']:.4f}")
175     print(f"p-value: {result['p_value']:.6f}")
176     print(f"Critical value (alpha=0.05): +/-{result['z_critical']:.4f}")
177     print(f"\nDecision: {'REJECT_H0 - Significant difference' if result['significant'] else 'FAIL TO REJECT_H0 - No significant difference'}")
178
179 # Visualize first scenario

```

```

180     if i == 1:
181         fig = visualize_hypothesis_test(result)
182         plt.savefig(f'hypothesis_test_scenario_{i}.pdf', dpi=300,
183                     bbox_inches='tight')
184         plt.show()
185
186 # Demonstrate power of test via simulation
187 print("\n" + "="*80)
188 print("Statistical Power Demonstration via Simulation")
189 print("="*80)
190
191 def simulate_power(p1, p2, n, alpha=0.05, n_sims=10000):
192     """
193     Simulate statistical power for given parameters.
194     """
195     significant_count = 0
196
197     for _ in range(n_sims):
198         # Generate data under H1 (p2 != p1)
199         conv1 = np.random.binomial(n, p1)
200         conv2 = np.random.binomial(n, p2)
201
202         # Perform test
203         result = z_test_proportions(conv1, n, conv2, n, alpha=alpha)
204         if result['significant']:
205             significant_count += 1
206
207     return significant_count / n_sims
208
209 # Test different sample sizes
210 p_control = 0.08
211 p_treatment = 0.10 # 25% relative lift
212 sample_sizes = [100, 300, 500, 1000, 2000, 5000]
213
214 print(f"\nTrue conversion rates: Control={p_control:.1%}, Treatment=
215       {p_treatment:.1%}")
216 print(f"True relative lift: {(p_treatment-p_control)/p_control:.1%}")
217 print(f"\n{'Sample size (per group)':>25}{'Empirical power':>20}")
218 print("-" * 50)
219
220 powers = []
221 for n in sample_sizes:
222     power = simulate_power(p_control, p_treatment, n)
223     powers.append(power)
224     print(f"{n:>25}{power:>20.3f}")
225
226 # Plot power curve
227 fig, ax = plt.subplots(figsize=(10, 6))
228 ax.plot(sample_sizes, powers, 'bo-', linewidth=2, markersize=8)
229 ax.axhline(0.8, color='red', linestyle='--', linewidth=2, label='Target
230       power=0.80')
231 ax.set_xlabel('Sample Size (per group)', fontsize=12)
232 ax.set_ylabel('Statistical Power', fontsize=12)
233 ax.set_title(f'Power Curve (p1={p_control:.1%}, p2={p_treatment:.1%}, alpha
234             =0.05)',
235             fontsize=13, fontweight='bold')
236 ax.grid(alpha=0.3)
237 ax.legend(fontsize=11)

```

```

234 plt.tight_layout()
235 plt.savefig('power_curve_simulation.pdf', dpi=300, bbox_inches='tight')
236 plt.show()

```

2.7 Effect Size Measures

Statistical significance tells us whether an effect exists; effect size tells us whether it matters. This distinction is crucial for A/B testing—a statistically significant result may be practically irrelevant, especially with large samples.

2.7.1 Why Effect Size Matters

Consider two scenarios:

1. $n = 100,000$ per variant: 5.00% vs. 5.05% conversion, $p < 0.001$ (highly significant)
2. $n = 500$ per variant: 5.0% vs. 7.5% conversion, $p = 0.08$ (not significant)

In Scenario 1, the difference is statistically significant but tiny (0.05 percentage points). Implementation costs may exceed benefits.

In Scenario 2, the difference is large (2.5 percentage points, 50% relative lift) but not statistically significant due to small sample. More data might be worthwhile.

Key principle: Always report both statistical significance (p-value) and effect size. Make decisions considering both along with practical constraints.

2.7.2 Absolute vs. Relative Differences

Definition 2.20 (Absolute Difference).

$$\text{Absolute difference} = p_2 - p_1 \quad (2.58)$$

Measured in the same units as the metric (e.g., percentage points for conversion rates).

Definition 2.21 (Relative Difference (Relative Lift)).

$$\text{Relative lift} = \frac{p_2 - p_1}{p_1} = \frac{p_2}{p_1} - 1 \quad (2.59)$$

Expressed as a percentage or decimal. Answers: "By what fraction did the metric change?"

Example 2.8 (Absolute vs. Relative). 1. Control: 2%, Treatment: 3%

- Absolute difference: $3\% - 2\% = 1$ percentage point
- Relative lift: $(3\% - 2\%)/2\% = 0.50 = 50\%$

2. Control: 20%, Treatment: 21%

- Absolute difference: $21\% - 20\% = 1$ percentage point
- Relative lift: $(21\% - 20\%)/20\% = 0.05 = 5\%$

Same absolute difference, vastly different relative lifts. Which matters more depends on context.

2.7.3 Cohen's h for Proportions

Cohen's h standardizes the difference between proportions, enabling comparison across experiments with different baseline rates.

Definition 2.22 (Cohen's h). For proportions p_1 and p_2 :

$$h = 2(\arcsin \sqrt{p_2} - \arcsin \sqrt{p_1}) \quad (2.60)$$

where $\arcsin$ is the inverse sine function (arcsine).

Interpretation guidelines (Cohen, 1988):

- $|h| < 0.2$: Small effect
- $0.2 \leq |h| < 0.5$: Medium effect
- $|h| \geq 0.5$: Large effect

Advantage: Unlike relative lift, Cohen's h behaves well for proportions near 0 or 1.

2.7.4 Odds Ratio

The odds ratio is commonly used in medical research and logistic regression contexts.

Definition 2.23 (Odds and Odds Ratio). The **odds** of an event with probability p is:

$$\text{Odds} = \frac{p}{1-p} \quad (2.61)$$

The **odds ratio** comparing two proportions:

$$\text{OR} = \frac{p_2/(1-p_2)}{p_1/(1-p_1)} = \frac{p_2(1-p_1)}{p_1(1-p_2)} \quad (2.62)$$

Interpretation:

- $\text{OR} = 1$: Equal odds (no effect)
- $\text{OR} > 1$: Higher odds in treatment
- $\text{OR} < 1$: Lower odds in treatment

Example 2.9 (Odds Ratio Calculation). Control: 10% conversion, Treatment: 15% conversion

$$\text{Odds}_{\text{control}} = 0.10/0.90 = 0.111 \quad (2.63)$$

$$\text{Odds}_{\text{treatment}} = 0.15/0.85 = 0.176 \quad (2.64)$$

$$\text{OR} = 0.176/0.111 = 1.59 \quad (2.65)$$

Interpretation: The odds of conversion are 1.59 times higher in treatment—or 59% higher odds.

Log odds ratio: Often $\log(\text{OR})$ is reported because it's symmetric around 0 and unbounded.

2.7.5 Cohen's d for Continuous Metrics

For continuous metrics (e.g., revenue, session duration), Cohen's d measures effect size in standard deviation units.

Definition 2.24 (Cohen's d).

$$d = \frac{\bar{x}_2 - \bar{x}_1}{s_{\text{pooled}}} \quad (2.66)$$

where the pooled standard deviation is:

$$s_{\text{pooled}} = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}} \quad (2.67)$$

Interpretation guidelines:

- $|d| < 0.2$: Small effect
- $0.2 \leq |d| < 0.5$: Small to medium effect
- $0.5 \leq |d| < 0.8$: Medium to large effect
- $|d| \geq 0.8$: Large effect

Example 2.10 (Cohen's d for Revenue). Testing pricing change:

- Control: $\bar{x}_1 = \$45$, $s_1 = \$20$, $n_1 = 500$
- Treatment: $\bar{x}_2 = \$52$, $s_2 = \$22$, $n_2 = 500$

Pooled standard deviation:

$$s_{\text{pooled}} = \sqrt{\frac{499 \times 20^2 + 499 \times 22^2}{998}} = \sqrt{440.2} = 20.98 \quad (2.68)$$

Cohen's d:

$$d = \frac{52 - 45}{20.98} = \frac{7}{20.98} = 0.334 \quad (2.69)$$

This is a small to medium effect (between 0.2 and 0.5).

Listing 2.6: Computing Effect Sizes

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from scipy import stats
5
6 sns.set_style("whitegrid")
7
8 def compute_effect_sizes(conv1, n1, conv2, n2):
9     """
10     Compute multiple effect size measures for proportion comparison.
11
12     Returns:
13     -----
14     dict with various effect size measures
15     """

```

```

16     p1 = conv1 / n1
17     p2 = conv2 / n2
18
19     # Absolute difference
20     abs_diff = p2 - p1
21
22     # Relative lift
23     rel_lift = (p2 - p1) / p1 if p1 > 0 else np.nan
24
25     # Cohen's h
26     cohens_h = 2 * (np.arcsin(np.sqrt(p2)) - np.arcsin(np.sqrt(p1)))
27
28     # Odds ratio
29     odds1 = p1 / (1 - p1) if p1 < 1 else np.inf
30     odds2 = p2 / (1 - p2) if p2 < 1 else np.inf
31     odds_ratio = odds2 / odds1 if odds1 > 0 and np.isfinite(odds1) else
        np.nan
32     log_odds_ratio = np.log(odds_ratio) if odds_ratio > 0 and np.
        isfinite(odds_ratio) else np.nan
33
34     # Number needed to treat (NNT) - for proportions
35     nnt = 1 / abs_diff if abs_diff != 0 else np.inf
36
37     return {
38         'p1': p1,
39         'p2': p2,
40         'absolute_diff': abs_diff,
41         'relative_lift': rel_lift,
42         'cohens_h': cohens_h,
43         'odds_ratio': odds_ratio,
44         'log_odds_ratio': log_odds_ratio,
45         'nnt': nnt
46     }
47
48
49 def cohens_d_continuous(mean1, std1, n1, mean2, std2, n2):
50     """
51     Compute Cohen's d for continuous metrics.
52     """
53     # Pooled standard deviation
54     pooled_std = np.sqrt(((n1-1)*std1**2 + (n2-1)*std2**2) / (n1 + n2 -
        2))
55
56     # Cohen's d
57     d = (mean2 - mean1) / pooled_std
58
59     return {
60         'mean1': mean1,
61         'mean2': mean2,
62         'pooled_std': pooled_std,
63         'cohens_d': d,
64         'interpretation': interpret_cohens_d(abs(d))
65     }
66
67
68 def interpret_cohens_h(h):
69     """Interpret Cohen's h effect size."""
70     h_abs = abs(h)

```

```

71     if h_abs < 0.2:
72         return "Small"
73     elif h_abs < 0.5:
74         return "Medium"
75     else:
76         return "Large"
77
78
79 def interpret_cohens_d(d):
80     """Interpret Cohen's d effect size."""
81     d_abs = abs(d)
82     if d_abs < 0.2:
83         return "Negligible"
84     elif d_abs < 0.5:
85         return "Small"
86     elif d_abs < 0.8:
87         return "Medium"
88     else:
89         return "Large"
90
91
92 # Example calculations
93 print("="*80)
94 print("Effect Size Examples for Proportions")
95 print("="*80)
96
97 scenarios = [
98     {
99         'name': 'Low baseline, small absolute diff',
100         'conv1': 20, 'n1': 1000,
101         'conv2': 30, 'n2': 1000
102     },
103     {
104         'name': 'Low baseline, large absolute diff',
105         'conv1': 20, 'n1': 1000,
106         'conv2': 60, 'n2': 1000
107     },
108     {
109         'name': 'High baseline, small absolute diff',
110         'conv1': 400, 'n1': 1000,
111         'conv2': 410, 'n2': 1000
112     },
113     {
114         'name': 'High baseline, large absolute diff',
115         'conv1': 400, 'n1': 1000,
116         'conv2': 500, 'n2': 1000
117     }
118 ]
119
120 for scenario in scenarios:
121     print(f"\n{'-'*80}")
122     print(f"{scenario['name']}")
123     print('-'*80)
124
125     es = compute_effect_sizes(
126         scenario['conv1'], scenario['n1'],
127         scenario['conv2'], scenario['n2']
128     )

```

```

129
130     print(f"Control: {es['p1']:.2%} ({scenario['conv1']}/{scenario['n1']})")
131     print(f"Treatment: {es['p2']:.2%} ({scenario['conv2']}/{scenario['n2']})")
132     print(f"\nEffect sizes:")
133     print(f"Absolute difference: {es['absolute_diff']:.4f} ({es['absolute_diff']*100:.2f}pp)")
134     print(f"Relative lift: {es['relative_lift']:.2%}")
135     print(f"Cohen's h: {es['cohens_h']:.4f} ({interpret_cohens_h(es['cohens_h'])})")
136     print(f"Odds ratio: {es['odds_ratio']:.4f}")
137     print(f"Log odds ratio: {es['log_odds_ratio']:.4f}")
138     print(f"NNT: {es['nnt']:.1f}")
139
140 # Example for continuous metrics
141 print("\n" + "="*80)
142 print("Effect Size Examples for Continuous Metrics")
143 print("="*80)
144
145 continuous_scenarios = [
146     {
147         'name': 'Revenue_per_user',
148         'mean1': 45, 'std1': 20, 'n1': 500,
149         'mean2': 52, 'std2': 22, 'n2': 500,
150         'unit': '$'
151     },
152     {
153         'name': 'Session_duration',
154         'mean1': 180, 'std1': 120, 'n1': 1000,
155         'mean2': 200, 'std2': 125, 'n2': 1000,
156         'unit': 'seconds'
157     }
158 ]
159
160 for scenario in continuous_scenarios:
161     print(f"\n{'-'*80}")
162     print(f"{scenario['name']}")
163     print(f{'-'*80})
164
165     result = cohens_d_continuous(
166         scenario['mean1'], scenario['std1'], scenario['n1'],
167         scenario['mean2'], scenario['std2'], scenario['n2']
168     )
169
170     print(f"Control: {scenario['unit']}{result['mean1']:.2f} (SD={scenario['std1']:.2f}, n={scenario['n1']})")
171     print(f"Treatment: {scenario['unit']}{result['mean2']:.2f} (SD={scenario['std2']:.2f}, n={scenario['n2']})")
172     print(f"\nPooled SD: {scenario['unit']}{result['pooled_std']:.2f}")
173     print(f"Cohen's d: {result['cohens_d']:.4f} ({result['interpretation']} effect)")
174
175 # Visualize effect size landscape
176 print("\n" + "="*80)
177 print("Visualizing Effect Size Landscape")
178 print("="*80)
179

```

```

180 # Create grid of p1 and p2 values
181 p1_values = np.linspace(0.01, 0.5, 50)
182 p2_values = np.linspace(0.01, 0.5, 50)
183 P1, P2 = np.meshgrid(p1_values, p2_values)
184
185 # Compute relative lift and Cohen's h for each combination
186 relative_lift = (P2 - P1) / P1
187 cohens_h_grid = 2 * (np.arcsin(np.sqrt(P2)) - np.arcsin(np.sqrt(P1)))
188
189 fig, axes = plt.subplots(1, 2, figsize=(16, 6))
190
191 # Plot 1: Relative lift heatmap
192 im1 = axes[0].contourf(P1, P2, relative_lift, levels=20, cmap='RdYlGn')
193 axes[0].plot([0, 0.5], [0, 0.5], 'k--', linewidth=2, label='No effect line')
194 axes[0].set_xlabel('Control Conversion Rate', fontsize=12)
195 axes[0].set_ylabel('Treatment Conversion Rate', fontsize=12)
196 axes[0].set_title('Relative Lift Landscape', fontsize=13, fontweight='bold')
197 plt.colorbar(im1, ax=axes[0], label='Relative Lift')
198 axes[0].legend()
199
200 # Plot 2: Cohen's h heatmap
201 im2 = axes[1].contourf(P1, P2, cohens_h_grid, levels=20, cmap='RdYlGn')
202 axes[1].plot([0, 0.5], [0, 0.5], 'k--', linewidth=2, label='No effect line')
203
204 # Add contour lines for small/medium/large thresholds
205 axes[1].contour(P1, P2, np.abs(cohens_h_grid), levels=[0.2, 0.5],
206                 colors='black', linewidths=2, linestyles=[':', '--'])
207 axes[1].set_xlabel('Control Conversion Rate', fontsize=12)
208 axes[1].set_ylabel('Treatment Conversion Rate', fontsize=12)
209 axes[1].set_title("Cohen's sh Landscape", fontsize=13, fontweight='bold')
210
211 plt.colorbar(im2, ax=axes[1], label="Cohen's sh")
212 axes[1].legend()
213
214 plt.tight_layout()
215 plt.savefig('effect_size_landscape.pdf', dpi=300, bbox_inches='tight')
216 plt.show()

```

2.7.6 Practical Guidelines for Effect Size Reporting

When reporting A/B test results, always include:

1. **Sample sizes:** $n_1 = 1,000$, $n_2 = 1,000$
2. **Observed metrics:** Control = 8.0%, Treatment = 10.0%
3. **Absolute difference:** 2.0 percentage points [95% CI: 0.5, 3.5 pp]
4. **Relative lift:** 25.0% [95% CI: 6.3%, 43.8%]
5. **Statistical significance:** $p = 0.009$ (significant at $\alpha = 0.05$)
6. **Effect size:** Cohen's $h = 0.15$ (small effect)

7. **Business impact:** Expected +200 conversions/day, \$10,000 additional revenue/-month

This comprehensive reporting enables stakeholders to judge both statistical and practical significance.

2.8 Frequentist vs. Bayesian Inference

Two philosophical frameworks underpin statistical inference. Understanding both perspectives enriches A/B testing practice.

2.8.1 Frequentist Framework

Core philosophy:

- Parameters are fixed but unknown constants
- Probability describes long-run frequencies of events
- Inference focuses on procedures with good repeated sampling properties
- Uncertainty is quantified through confidence intervals and p-values

Key concepts:

- **Confidence intervals:** "95% of such intervals contain the true parameter"
- **p-values:** "Probability of data this extreme if null is true"
- **Hypothesis testing:** Reject or fail to reject H_0 based on α

Strengths:

- Objective—no prior beliefs required
- Well-established theory and widespread acceptance
- Clear Type I and Type II error control
- Standard in regulatory contexts (FDA, etc.)

Limitations:

- Cannot make probability statements about parameters ("probability that $p > 0.05$ ")
- P-values are frequently misinterpreted
- Fixed sample size typically required for valid inference
- Doesn't naturally incorporate prior information

2.8.2 Bayesian Framework

Core philosophy:

- Parameters are random variables with probability distributions
- Probability describes degree of belief about unknowns
- Prior beliefs are updated with data via Bayes' theorem
- Inference produces posterior distributions for parameters

Theorem 2.3 (Bayes' Theorem). For parameter θ and data D :

$$\underbrace{p(\theta|D)}_{\text{Posterior}} = \frac{\overbrace{p(D|\theta)}^{\text{Likelihood}} \times \overbrace{p(\theta)}^{\text{Prior}}}{\underbrace{p(D)}_{\text{Evidence}}} \quad (2.70)$$

where $p(D) = \int p(D|\theta)p(\theta)d\theta$ normalizes the posterior.

In words:

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior} \quad (2.71)$$

Key concepts:

- **Prior distribution:** Encodes initial beliefs before seeing data
- **Likelihood:** How probable the data is under different parameter values
- **Posterior distribution:** Updated beliefs after observing data
- **Credible intervals:** "95% probability parameter is in this interval"

Strengths:

- Direct probability statements about parameters
- Natural incorporation of prior information
- Coherent framework for sequential testing
- Intuitive interpretation for decision-making

Limitations:

- Requires specifying prior distributions (though this can be an advantage)
- Computational complexity for complex models (mitigated by modern MCMC)
- Results depend on prior choice
- Less standard in regulatory contexts

2.8.3 When to Use Each Approach

Prefer frequentist when:

- Regulatory requirements demand it
- You want objectivity without prior specification
- You have fixed sample size and single analysis
- Standard null hypothesis testing suffices

Prefer Bayesian when:

- You have meaningful prior information to incorporate
- You want direct probability statements about parameters
- Sequential testing or early stopping is needed
- Decision-making requires posterior distributions

In practice: Many modern A/B testing platforms use Bayesian methods because they handle sequential testing naturally and provide intuitive outputs for business stakeholders. Chapter 7 covers Bayesian A/B testing comprehensively.

2.9 Summary

This chapter established the probabilistic and statistical foundations essential for rigorous A/B testing:

- **Probability theory** provides the mathematical framework for quantifying uncertainty. Key results include the Central Limit Theorem, which justifies normal approximations for sample means regardless of underlying distributions.
- **Key distributions**—Bernoulli, binomial, normal, t, beta, and chi-squared—model different aspects of experiments. Understanding their properties enables correct application.
- **Statistical inference** uses sample data to draw conclusions about populations. Point estimates provide best guesses; confidence intervals quantify uncertainty.
- **Hypothesis testing** offers a formal framework for decision-making. Understanding null hypotheses, p-values, Type I and Type II errors, and statistical power is essential for correct interpretation.
- **Effect sizes** measure practical significance, complementing statistical significance. Always report both—a large p-value with large effect size may warrant more data; a small p-value with tiny effect size may be practically irrelevant.
- **Frequentist and Bayesian frameworks** offer complementary perspectives. Frequentist methods dominate traditional practice; Bayesian methods offer advantages for sequential testing and direct probability statements.

These foundations enable the hypothesis testing procedures, sample size calculations, multiple testing corrections, and Bayesian methods developed in subsequent chapters. Mastery of these concepts distinguishes practitioners who merely run tests from those who design sound experiments and interpret results correctly.

2.10 Further Reading

- Casella and Berger [2002]: Comprehensive treatment of statistical inference with mathematical rigor
- Wasserman [2004]: Modern theoretical statistics with clear exposition
- Rice [2006]: Mathematical statistics emphasizing practical application
- Gelman et al. [2013]: Definitive text on Bayesian data analysis
- Cohen [1988]: Classic reference on statistical power and effect sizes
- Good [2013]: Permutation tests and resampling methods
- Wasserstein and Lazar [2016]: ASA statement on p -values and their interpretation

2.11 Exercises

2.11.1 Conceptual Questions

1. Explain the distinction between a parameter and a statistic. Why does this matter for A/B testing?
2. A colleague says: "We ran the experiment and got $p = 0.03$, so there's only a 3% chance the null hypothesis is true." What is wrong with this interpretation? Provide the correct interpretation.
3. Why does the Central Limit Theorem matter for A/B testing? What would be different if it didn't hold?
4. Explain the difference between a 95% confidence interval (frequentist) and a 95% credible interval (Bayesian). When would each be appropriate?
5. An experiment shows a statistically significant difference ($p < 0.001$) with Cohen's $h = 0.05$. What does this tell you? Should you implement the change?
6. Why might Type II error be more costly than Type I error in A/B testing? Conversely, when might Type I error be more concerning?

2.11.2 Probability and Distributions

1. Prove that for $X \sim \text{Bernoulli}(p)$: $\text{Var}(X) = p(1 - p)$.
2. Show that the binomial distribution $\text{Binomial}(n, p)$ has maximum variance at $p = 0.5$ for fixed n .

3. For $X \sim \mathcal{N}(\mu, \sigma^2)$, derive the distribution of:
 - (a) $Y = aX + b$
 - (b) $Z = (X - \mu)/\sigma$
4. Simulate the CLT for an exponential distribution with $\lambda = 0.2$. Show convergence to normality for $n = 5, 20, 50, 200$. Use K-S test to assess normality.
5. A website has average session duration of 4 minutes, exponentially distributed. With 100 users, what is the approximate distribution of average session duration? Calculate a 95% confidence interval.

2.11.3 Hypothesis Testing

1. Conduct a two-sample z-test by hand (show all steps):
 - Control: 1,500 users, 120 conversions
 - Treatment: 1,500 users, 150 conversions
 - $\alpha = 0.05$

Include: hypotheses, test statistic, p-value, decision, interpretation.

2. For the scenario above, construct a 95% confidence interval for the difference in proportions. Is it consistent with your hypothesis test conclusion?
3. Verify by simulation that Type I error rate is controlled at $\alpha = 0.05$ when testing $H_0 : p_1 = p_2 = 0.1$ with $n_1 = n_2 = 500$. Run 10,000 simulations.
4. Simulate power for detecting a difference from $p_1 = 0.08$ to $p_2 = 0.10$ with $\alpha = 0.05$. Plot power vs. sample size for $n = 100, 200, \dots, 2000$ (per group).
5. Investigate the relationship between α , β , effect size, and sample size via simulation. Fix three and vary the fourth.

2.11.4 Confidence Intervals

1. Construct 90%, 95%, and 99% confidence intervals for conversion rate given 75 conversions from 1,000 users. How does interval width change with confidence level?
2. Implement and compare three CI methods for proportions: Wald, Wilson score, and Agresti-Coull. For which scenarios (n , p combinations) do they differ substantially?
3. Verify by simulation that 95% confidence intervals contain the true parameter approximately 95% of the time. Test for $p = 0.05, 0.1, 0.3, 0.5$ and $n = 50, 200, 1000$.
4. Two variants show: A ($145/2000 = 7.25\%$), B ($170/2000 = 8.50\%$). Construct:
 - (a) 95% CI for each proportion
 - (b) 95% CI for difference in proportions
 - (c) 95% CI for relative lift

2.11.5 Effect Sizes

1. Calculate all effect size measures (absolute difference, relative lift, Cohen's h , odds ratio) for:

(a) $p_1 = 0.05, p_2 = 0.07$

(b) $p_1 = 0.20, p_2 = 0.22$

(c) $p_1 = 0.40, p_2 = 0.50$

Compare and interpret the measures across scenarios.

2. For continuous metrics, compute Cohen's d :

- Control: $\bar{x}_1 = 100, s_1 = 30, n_1 = 250$

- Treatment: $\bar{x}_2 = 110, s_2 = 32, n_2 = 250$

Interpret the effect size. How many additional observations would be needed to detect this effect with 80% power at $\alpha = 0.05$?

3. Create a visualization showing the "landscape" of effect sizes: For $p_1 \in [0.01, 0.5]$ and $p_2 \in [0.01, 0.5]$, plot heatmaps of (a) relative lift, (b) Cohen's h , and (c) log odds ratio. Identify regions of small/medium/large effects.
4. A company runs 100 A/B tests per year. Each test has a true effect size of Cohen's $h = 0.1$ (small). With $n = 1000$ per variant and $\alpha = 0.05$, what fraction of tests will:
 - (a) Correctly detect the effect (power)?
 - (b) Fail to detect the effect (Type II error)?
 - (c) Show a "significant" effect in the wrong direction?

Verify via simulation.

2.11.6 Programming Projects

1. **A/B Test Analysis Package:** Implement a comprehensive Python class `ABTest` that:

- Accepts control and treatment data
- Computes all relevant statistics (proportions, SEs, CIs)
- Performs hypothesis tests (z-test, chi-squared test, permutation test)
- Calculates effect sizes
- Generates publication-quality visualizations
- Outputs formatted report with interpretation

Include docstrings, type hints, unit tests, and example usage.

2. **Interactive Dashboard:** Create a web dashboard (using Streamlit, Dash, or Shiny) that:

- Allows users to input A/B test data
- Computes and displays all statistics
- Visualizes confidence intervals, power curves
- Provides interpretation and recommendations
- Includes sample size calculator

3. **Simulation Study:** Design and execute a comprehensive simulation investigating:

- Coverage of different CI methods across parameter space
- Power as a function of effect size, sample size, baseline rate
- Impact of unequal sample sizes on test properties
- Consequences of peeking (sequential testing without correction)

Write a report summarizing findings with visualizations.

4. **Bayesian vs. Frequentist Comparison:** Implement both frameworks for A/B testing:

- Frequentist: Confidence intervals and hypothesis tests
- Bayesian: Posterior distributions with Beta-Binomial conjugacy

Compare results across scenarios varying sample size, effect size, and prior specification. When do they agree/disagree?

Chapter 3

Experimental Design Principles

3.1 Learning Objectives

After completing this chapter, readers will be able to:

- Understand and apply the fundamental principles of experimental design
- Implement various randomization techniques for different experimental contexts
- Identify and mitigate common sources of bias and confounding
- Assess experimental validity across multiple dimensions
- Make informed design decisions for online A/B tests
- Implement randomization schemes and bias detection in Python

3.2 Introduction

Experimental design is the architecture underlying A/B testing. Poor design invalidates even the most sophisticated statistical analysis—garbage in, garbage out. Conversely, thoughtful design can dramatically increase the efficiency and reliability of experiments, enabling faster learning and better decisions.

This chapter explores the principles that separate rigorous experiments from flawed ones. We'll see how randomization protects against bias, how stratification increases power, how confounding variables mislead, and how validity considerations constrain generalization. These aren't merely theoretical concerns—they directly impact whether your experiment provides trustworthy guidance or leads you astray.

The history of experimental design offers sobering lessons. Fisher's agricultural experiments in the 1920s established randomization as essential. Medical trials in the 1940s demonstrated the necessity of control groups and blinding. The digital era has introduced new challenges: network effects, novelty effects, and implementation bugs that invalidate traditional assumptions. Modern A/B testing requires both classical wisdom and contemporary adaptations.

3.3 Fundamental Principles of Experimental Design

Four core principles underpin sound experimental design. Violate any of them, and your experiment's validity is threatened.

3.3.1 Randomization

Randomization is the cornerstone of causal inference. By randomly assigning units to treatment and control, we ensure that—in expectation—groups differ only in the treatment received, not in other characteristics that might affect outcomes.

Definition 3.1 (Randomization). Random assignment is the process of allocating experimental units to treatment conditions using a random mechanism, ensuring each unit has a known, non-zero probability of assignment to each condition.

Why randomization matters:

1. **Eliminates selection bias:** Without randomization, systematic differences between groups confound treatment effects. Users who self-select into groups differ in unmeasured ways.
2. **Balances confounders:** Randomization balances both measured and *unmeasured* confounding variables across groups. This is crucial—we can't control for variables we don't know about or can't measure.
3. **Justifies statistical inference:** Probability theory underlying hypothesis tests and confidence intervals assumes random sampling or random assignment. Without randomization, these tools lack foundation.
4. **Enables causal interpretation:** Observational studies can establish association; randomized experiments establish causation. The difference is randomization.

Example 3.1 (Randomization vs. Self-Selection). Consider testing a new premium subscription tier:

Flawed approach (self-selection): Offer the new tier to all users, compare adopters vs. non-adopters. Problem: Adopters likely differ systematically—higher engagement, more resources, different needs. Observed differences confound tier effect with user characteristics.

Correct approach (randomization): Randomly assign users to see new tier vs. old tier. Now groups are balanced in expectation. Observed differences are attributable to tier design, not pre-existing user differences.

Common randomization mistakes:

- **Alternating assignment:** "Every other user gets treatment" is NOT random. Predictability enables gaming; time-based patterns confound results.
- **Assignment by characteristics:** "Users in California get treatment" introduces geographic confounding.
- **Insufficient randomness:** Using low-quality random number generators or seeding incorrectly can introduce patterns.
- **Post-hoc selection:** Assigning users randomly but then filtering or selecting subsets for analysis reintroduces bias.

3.3.2 Control Groups

A control group provides the counterfactual: what would have happened without treatment? Without controls, we can't distinguish treatment effects from natural variation, seasonal trends, or external factors.

Definition 3.2 (Control Group). The control group receives no treatment (or the current standard treatment), providing a baseline for comparison. The treatment group receives the intervention being tested.

Example 3.2 (The Necessity of Controls). A company launches a new website design and observes a 10% increase in conversions over the next week. Success? Not necessarily:

- Was there a marketing campaign that week?
- Is it a seasonal peak (e.g., holiday shopping)?
- Did competitors change their offerings?
- Was there a temporary bug in the old design that was fixed?

Without a control group maintaining the old design, these confounds are inseparable from the design effect. With proper randomization and controls, we isolate the causal impact of the design change.

Types of controls:

- **No-treatment control:** Users receive no intervention. Appropriate when testing entirely new features.
- **Placebo control:** Users receive a neutral intervention that mimics treatment exposure without active ingredients. Less common in digital contexts but relevant for psychological effects.
- **Active control:** Users receive current standard treatment. Essential when testing improvements to existing features.
- **Multiple controls:** Sometimes multiple baseline conditions are needed to isolate specific mechanisms.

3.3.3 Blocking and Stratification

While randomization ensures balance in expectation, chance can produce imbalanced groups in any single experiment. Blocking and stratification improve balance on known covariates, increasing precision.

Definition 3.3 (Blocking). Blocking divides experimental units into homogeneous groups (blocks) based on covariates, then randomizes separately within each block. This ensures balance on the blocking variables.

Definition 3.4 (Stratification). Stratification is similar to blocking but typically refers to analysis rather than design. Units are divided into strata post-randomization, and stratum-specific effects are estimated.

When to use blocking:

- **Known important covariates:** If user type, geographic region, or device strongly affects outcomes, blocking ensures balance and increases power.
- **Small sample sizes:** With few units, chance imbalance is more likely. Blocking provides insurance.
- **Heterogeneous treatment effects:** If effects vary across subgroups, blocking enables subgroup-specific analysis.

Example 3.3 (Blocking in Practice). Testing a mobile app redesign where behavior differs dramatically between iOS and Android:

Simple randomization: 50% iOS users to treatment, 50% Android users to treatment. By chance, treatment might get 52% iOS, control 48% iOS—introducing imbalance.

Blocked randomization: Within iOS users, randomize 50% to each group. Within Android users, randomize 50% to each group. Now platform is perfectly balanced.

If iOS users have 15% conversion rate and Android users 10%, blocking eliminates this source of variance, increasing power to detect the treatment effect.

Practical considerations:

- **Number of blocks:** Too many small blocks reduce efficiency. Aim for substantial units per block.
- **Blocking variables:** Choose strong predictors of outcomes. Blocking on weak predictors adds complexity without benefit.
- **Crossover:** Can block on multiple variables (e.g., platform $\times$ country), but beware exponentially growing strata.

3.3.4 Blinding

Blinding conceals treatment assignment from participants, experimenters, or both, preventing knowledge of assignment from influencing behavior or assessment.

Definition 3.5 (Blinding). • **Single-blind:** Participants don't know their treatment assignment

- **Double-blind:** Neither participants nor experimenters know assignment
- **Triple-blind:** Participants, experimenters, and analysts all remain blind

Why blinding matters:

1. **Placebo effects:** Participants' knowledge of treatment can affect outcomes independent of treatment's direct effects.
2. **Experimenter bias:** Researchers who know assignment may unconsciously influence participants or assess outcomes differently.
3. **Demand characteristics:** Participants may alter behavior to confirm experimenters' hypotheses.

4. **Differential attrition:** If users realize they're in control, they may drop out at different rates than treatment.

Example 3.4 (Blinding in Digital Experiments). In A/B testing online, blinding is often implicit:

Naturally blinded: Users typically don't know they're in an experiment or which group they're in. A new button color appears to some users; they don't realize others see something different.

Not blinded: If treatment is a new feature prominently labeled "BETA" or "NEW!", users know they're in treatment. Their behavior may reflect curiosity, novelty-seeking, or frustration with bugs rather than the feature's inherent value.

Experimenter blinding: Data analysts should ideally remain blind to which variant is A vs. B until analysis completion, preventing p-hacking or selective reporting.

Limitations in online testing:

- Complete blinding is often impossible—users see different interfaces
- Product teams must know which variant is which to iterate
- The goal: minimize systematic bias from knowledge of assignment

3.4 Randomization Techniques

Different contexts demand different randomization approaches. We examine four primary techniques with implementation details.

3.4.1 Simple Randomization

Simple randomization assigns each unit independently to treatment or control with fixed probability, typically 50%.

Advantages:

- Conceptually simple
- Easy to implement
- Provably unbiased
- No memory or coordination needed

Disadvantages:

- Can produce imbalanced group sizes
- No guarantee of balance on covariates
- Variance in group sizes reduces power

When to use: Large experiments (thousands of units) where chance imbalance is negligible.

Listing 3.1: Simple Randomization Implementation

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from typing import List, Dict, Tuple
6
7 sns.set_style("whitegrid")
8 np.random.seed(42)
9
10 def simple_randomization(n_units: int, p_treatment: float = 0.5,
11                          seed: int = None) -> np.ndarray:
12     """
13     Simple randomization: each unit independently assigned.
14
15     Parameters:
16     -----
17     n_units: int
18         Number of units to randomize
19     p_treatment: float
20         Probability of treatment assignment (default 0.5)
21     seed: int
22         Random seed for reproducibility
23
24     Returns:
25     -----
26     np.ndarray: Binary array (0=control, 1=treatment)
27     """
28     if seed is not None:
29         np.random.seed(seed)
30
31     assignments = np.random.binomial(1, p_treatment, n_units)
32
33     return assignments
34
35
36 def evaluate_randomization(assignments: np.ndarray,
37                            covariates: pd.DataFrame = None,
38                            covariate_names: List[str] = None) -> Dict:
39     """
40     Evaluate quality of randomization.
41
42     Parameters:
43     -----
44     assignments: np.ndarray
45         Treatment assignments (0/1)
46     covariates: pd.DataFrame, optional
47         Covariates to check balance on
48     covariate_names: List[str], optional
49         Names of covariates to check
50
51     Returns:
52     -----
53     dict: Evaluation metrics
54     """
55     n_total = len(assignments)
56     n_treatment = assignments.sum()
57     n_control = n_total - n_treatment

```

```

58
59     results = {
60         'n_total': n_total,
61         'n_treatment': n_treatment,
62         'n_control': n_control,
63         'prop_treatment': n_treatment / n_total,
64         'balance_score': abs(n_treatment - n_control) / n_total
65     }
66
67     # Check covariate balance if provided
68     if covariates is not None:
69         from scipy import stats as sp_stats
70         balance_tests = {}
71
72         for col in covariate_names or covariates.columns:
73             treatment_vals = covariates.loc[assignments == 1, col]
74             control_vals = covariates.loc[assignments == 0, col]
75
76             # t-test for continuous, chi-square for categorical
77             if covariates[col].dtype in [np.float64, np.int64]:
78                 t_stat, p_val = sp_stats.ttest_ind(treatment_vals,
79                                                    control_vals)
79                 balance_tests[col] = {
80                     'mean_treatment': treatment_vals.mean(),
81                     'mean_control': control_vals.mean(),
82                     'diff': treatment_vals.mean() - control_vals.mean()
83                     ,
84                     'p_value': p_val
85                 }
86
87             results['covariate_balance'] = balance_tests
88
89     return results
90
91 # Example: Simple randomization with evaluation
92 print("="*80)
93 print("Simple Randomization Example")
94 print("="*80)
95
96 n_users = 1000
97 assignments = simple_randomization(n_users, p_treatment=0.5, seed=42)
98
99 # Create synthetic covariates
100 np.random.seed(42)
101 covariates = pd.DataFrame({
102     'age': np.random.normal(35, 10, n_users),
103     'prior_purchases': np.random.poisson(3, n_users),
104     'engagement_score': np.random.normal(50, 15, n_users)
105 })
106
107 # Evaluate
108 results = evaluate_randomization(assignments, covariates)
109
110 print(f"\nTotal units: {results['n_total']}")
111 print(f"Treatment: {results['n_treatment']} ({results['prop_treatment']:.2%})")
112 print(f"Control: {results['n_control']} ({1-results['prop_treatment']:.2%})")

```

```

113     ']:.2%})")
114 print(f"Balance_score:_{results['balance_score']:.4f}")
115 if 'covariate_balance' in results:
116     print("\nCovariate_Balance:")
117     print("-" * 80)
118     print(f"{'Covariate':<20}_{'Treatment_Mean':<15}_{'Control_Mean':<15}_{'Difference':<12}_{'p-value':<10}")
119     print("-" * 80)
120     for cov, stats in results['covariate_balance'].items():
121         print(f"{cov:<20}_{stats['mean_treatment']:<15.2f}_{stats['mean_control']:<15.2f}_{stats['diff']:<12.2f}_{stats['p_value']:<10.4f}")
122
123
124 # Simulate multiple randomizations to show variability
125 n_sims = 1000
126 treatment_props = []
127 for i in range(n_sims):
128     sim_assign = simple_randomization(1000, p_treatment=0.5, seed=i)
129     treatment_props.append(sim_assign.sum() / 1000)
130
131 # Visualize
132 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
133
134 # Distribution of treatment proportions
135 axes[0].hist(treatment_props, bins=30, edgecolor='black', alpha=0.7, color='steelblue')
136 axes[0].axvline(0.5, color='red', linestyle='--', linewidth=2, label='Target_(50%)')
137 axes[0].axvline(np.mean(treatment_props), color='green', linestyle='-', linewidth=2, label=f'Mean_{(np.mean(treatment_props):.3f)}')
138
139 axes[0].set_xlabel('Treatment_Proportion', fontsize=12)
140 axes[0].set_ylabel('Frequency', fontsize=12)
141 axes[0].set_title(f'Distribution_of_Treatment_Proportions\n({n_sims} simulations, n=1000)',
142                  fontsize=12, fontweight='bold')
143 axes[0].legend()
144 axes[0].grid(alpha=0.3)
145
146 # Covariate balance check for one randomization
147 treatment_mask = assignments == 1
148 axes[1].scatter(covariates.loc[treatment_mask, 'age'],
149                covariates.loc[treatment_mask, 'engagement_score'],
150                alpha=0.5, label='Treatment', color='coral', s=20)
151 axes[1].scatter(covariates.loc[~treatment_mask, 'age'],
152                covariates.loc[~treatment_mask, 'engagement_score'],
153                alpha=0.5, label='Control', color='steelblue', s=20)
154 axes[1].set_xlabel('Age', fontsize=12)
155 axes[1].set_ylabel('Engagement_Score', fontsize=12)
156 axes[1].set_title('Covariate_Balance: Age vs Engagement', fontsize=12, fontweight='bold')
157 axes[1].legend()
158 axes[1].grid(alpha=0.3)
159
160 plt.tight_layout()
161 plt.savefig('simple_randomization.pdf', dpi=300, bbox_inches='tight')
162 plt.show()

```

3.4.2 Block Randomization

Block randomization ensures exactly balanced group sizes by randomizing in small blocks rather than one unit at a time.

Mechanism:

1. Define block size (e.g., 4 or 6)
2. Within each block, assign equal numbers to treatment and control
3. Randomize the order within block

Advantages:

- Guarantees balanced group sizes
- Protects against temporal trends
- Reduces variance in treatment effect estimates

Disadvantages:

- If block size is known and assignments observable, late assignments in a block are predictable
- Requires coordination/memory across units
- Slightly more complex implementation

Listing 3.2: Block Randomization Implementation

```

1 def block_randomization(n_units: int, block_size: int = 4,
2                           p_treatment: float = 0.5, seed: int = None) ->
3                               np.ndarray:
4     """
5     Block randomization: guarantees balance within blocks.
6
7     Parameters:
8     -----
9     n_units: int
10            Number of units to randomize
11     block_size: int
12            Size of each block (should be even for 50-50 split)
13     p_treatment: float
14            Proportion assigned to treatment within each block
15     seed: int
16            Random seed for reproducibility
17
18     Returns:
19     -----
20     np.ndarray: Binary array (0=control, 1=treatment)
21     """
22     if seed is not None:
23         np.random.seed(seed)
24
25     # Calculate number of treatment assignments per block
26     n_treatment_per_block = int(block_size * p_treatment)

```

```

27 # Create full blocks
28 n_full_blocks = n_units // block_size
29 assignments = []
30
31 for _ in range(n_full_blocks):
32     # Create block with correct proportions
33     block = np.array([1] * n_treatment_per_block +
34                     [0] * (block_size - n_treatment_per_block))
35     # Shuffle within block
36     np.random.shuffle(block)
37     assignments.extend(block)
38
39 # Handle remaining units
40 remainder = n_units % block_size
41 if remainder > 0:
42     n_treatment_remainder = int(remainder * p_treatment)
43     remainder_block = np.array([1] * n_treatment_remainder +
44                               [0] * (remainder -
45                                       n_treatment_remainder))
46     np.random.shuffle(remainder_block)
47     assignments.extend(remainder_block)
48
49 return np.array(assignments)
50
51 # Example: Compare simple vs block randomization
52 print("\n" + "="*80)
53 print("Block Randomization vs Simple Randomization")
54 print("="*80)
55
56 n_users = 1000
57 n_sims = 1000
58
59 simple_balances = []
60 block_balances = []
61
62 for i in range(n_sims):
63     # Simple randomization
64     simple_assign = simple_randomization(n_users, seed=i)
65     simple_balances.append(abs(simple_assign.sum() - n_users/2))
66
67     # Block randomization
68     block_assign = block_randomization(n_users, block_size=10, seed=i)
69     block_balances.append(abs(block_assign.sum() - n_users/2))
70
71 print(f"\nBalance (deviation from 50-50 split):")
72 print(f"{'Method':<20}{'Mean':<12}{'Std Dev':<12}{'Max':<12}")
73 print("-" * 60)
74 print(f"{'Simple':<20}{np.mean(simple_balances):<12.2f}"
75       f"{np.std(simple_balances):<12.2f}{np.max(simple_balances):<12.0f}")
76 print(f"{'Block (size=10)':<20}{np.mean(block_balances):<12.2f}"
77       f"{np.std(block_balances):<12.2f}{np.max(block_balances):<12.0f}")
78
79 # Visualize comparison
80 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
81

```



```

82 axes[0].hist(simple_balances, bins=30, alpha=0.6, label='Simple',
83             color='steelblue', edgecolor='black')
84 axes[0].hist(block_balances, bins=30, alpha=0.6, label='Block_(size=10)
85             ',
86             color='coral', edgecolor='black')
87 axes[0].set_xlabel('Absolute_Deviation_from_Perfect_Balance', fontsize
88                   =12)
89 axes[0].set_ylabel('Frequency', fontsize=12)
90 axes[0].set_title('Randomization_Balance_Comparison', fontsize=12,
91                   fontweight='bold')
92 axes[0].legend()
93 axes[0].grid(alpha=0.3)
94
95 # Cumulative assignments over time
96 np.random.seed(42)
97 simple_cumsum = np.cumsum(simple_randomization(1000, seed=42))
98 block_cumsum = np.cumsum(block_randomization(1000, block_size=10, seed
99                   =42))
100 perfect = np.arange(1, 1001) * 0.5
101
102 axes[1].plot(simple_cumsum, label='Simple_randomization', linewidth=2,
103             alpha=0.7)
104 axes[1].plot(block_cumsum, label='Block_randomization', linewidth=2,
105             alpha=0.7)
106 axes[1].plot(perfect, 'k--', label='Perfect_balance', linewidth=2)
107 axes[1].set_xlabel('User_Number', fontsize=12)
108 axes[1].set_ylabel('Cumulative_Treatment_Assignments', fontsize=12)
109 axes[1].set_title('Balance_Over_Time', fontsize=12, fontweight='bold')
110 axes[1].legend()
111 axes[1].grid(alpha=0.3)
112
113 plt.tight_layout()
114 plt.savefig('block_randomization_comparison.pdf', dpi=300, bbox_inches=
115             'tight')
116 plt.show()

```

3.4.3 Stratified Randomization

Stratified randomization divides units into strata based on important covariates, then randomizes separately within each stratum.

Difference from blocking: Blocking typically refers to small, homogeneous groups. Stratification involves larger groups defined by key characteristics (e.g., country, user segment).

Advantages:

- Guarantees balance on stratifying variables
- Increases power by reducing between-stratum variance
- Enables stratified analysis
- Protects against chance imbalances on important covariates

Disadvantages:

- Requires knowing stratifying variables before randomization


```

51         strata: np.ndarray) -> Dict:
52     """
53     """Analyze stratified experiment with proper variance estimation.
54
55     Parameters:
56     -----
57     outcomes: np.ndarray
58     Outcome values
59     assignments: np.ndarray
60     Treatment assignments (0/1)
61     strata: np.ndarray
62     Stratum indicators
63
64     Returns:
65     -----
66     dict: Analysis results including stratum-specific effects
67     """
68     from scipy import stats as sp_stats
69
70     results = {
71         'strata_effects': {},
72         'overall_effect': None
73     }
74
75     unique_strata = np.unique(strata)
76     n_strata = len(unique_strata)
77
78     stratum_effects = []
79     stratum_vars = []
80     stratum_weights = []
81
82     for stratum in unique_strata:
83         stratum_mask = strata == stratum
84         stratum_outcomes = outcomes[stratum_mask]
85         stratum_assignments = assignments[stratum_mask]
86
87         # Calculate stratum-specific effect
88         treatment_outcomes = stratum_outcomes[stratum_assignments == 1]
89         control_outcomes = stratum_outcomes[stratum_assignments == 0]
90
91         if len(treatment_outcomes) > 0 and len(control_outcomes) > 0:
92             effect = treatment_outcomes.mean() - control_outcomes.mean()
93
94             # Variance of difference
95             var_treatment = treatment_outcomes.var() / len(
96                 treatment_outcomes)
97             var_control = control_outcomes.var() / len(control_outcomes)
98             var_effect = var_treatment + var_control
99
100             stratum_effects.append(effect)
101             stratum_vars.append(var_effect)
102             stratum_weights.append(stratum_mask.sum())
103
104             results['strata_effects'][stratum] = {
105                 'effect': effect,
106                 'se': np.sqrt(var_effect),

```

```

106         'n': stratum_mask.sum(),
107         'n_treatment': (stratum_assignments == 1).sum(),
108         'n_control': (stratum_assignments == 0).sum(),
109         'mean_treatment': treatment_outcomes.mean(),
110         'mean_control': control_outcomes.mean()
111     }
112
113     # Calculate overall weighted effect
114     stratum_effects = np.array(stratum_effects)
115     stratum_vars = np.array(stratum_vars)
116     stratum_weights = np.array(stratum_weights) / sum(stratum_weights)
117
118     overall_effect = np.sum(stratum_weights * stratum_effects)
119     overall_var = np.sum(stratum_weights**2 * stratum_vars)
120     overall_se = np.sqrt(overall_var)
121
122     # Test statistic
123     z_stat = overall_effect / overall_se
124     p_value = 2 * (1 - sp_stats.norm.cdf(abs(z_stat)))
125
126     results['overall_effect'] = {
127         'effect': overall_effect,
128         'se': overall_se,
129         'z_statistic': z_stat,
130         'p_value': p_value,
131         'ci_lower': overall_effect - 1.96 * overall_se,
132         'ci_upper': overall_effect + 1.96 * overall_se
133     }
134
135     return results
136
137
138     # Example: Stratified randomization with heterogeneous effects
139     print("\n" + "="*80)
140     print("Stratified Randomization Example")
141     print("="*80)
142
143     n_users = 2000
144     np.random.seed(42)
145
146     # Create strata (e.g., user segments)
147     segments = np.random.choice(['New', 'Casual', 'Power'], n_users,
148                                 p=[0.3, 0.5, 0.2])
149
150     # Stratified randomization
151     assignments = stratified_randomization(segments, p_treatment=0.5, seed
152                                           =42)
153
154     # Simulate outcomes with heterogeneous treatment effects
155     baseline_conversions = {
156         'New': 0.05,
157         'Casual': 0.10,
158         'Power': 0.20
159     }
160
161     treatment_effects = {
162         'New': 0.02,      # 40% relative lift for new users
163         'Casual': 0.01,   # 10% relative lift for casual
164         'Power': 0.01     # 5% relative lift for power users

```

```

163 }
164
165 outcomes = np.zeros(n_users)
166 for i in range(n_users):
167     segment = segments[i]
168     baseline = baseline_conversions[segment]
169     effect = treatment_effects[segment] if assignments[i] == 1 else 0
170     outcomes[i] = np.random.binomial(1, baseline + effect)
171
172 # Analyze
173 results = analyze_stratified_experiment(outcomes, assignments, segments
174 )
175
176 print("\nStratum-Specific Effects:")
177 print("-" * 90)
178 print(f"{'Stratum':<12}{'Treatment_Mean':<15}{'Control_Mean':<15}{'Effect':<12}{'SE':<10}{'n':<8}")
179 print("-" * 90)
180 for stratum, stats in results['strata_effects'].items():
181     print(f"{stratum:<12}{stats['mean_treatment']:<15.4f}{stats['mean_control']:<15.4f}"
182           f"{stats['effect']:<12.4f}{stats['se']:<10.4f}{stats['n']:<8}")
183
184 print("\nOverall Effect (Weighted):")
185 print("-" * 90)
186 overall = results['overall_effect']
187 print(f"Effect: {overall['effect']:.4f}")
188 print(f"SE: {overall['se']:.4f}")
189 print(f"95% CI: [{overall['ci_lower']:.4f}, {overall['ci_upper']:.4f}]")
190 print(f"z-statistic: {overall['z_statistic']:.4f}")
191 print(f"p-value: {overall['p_value']:.6f}")
192
193 # Visualize stratum-specific effects
194 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
195
196 # Effect sizes by stratum
197 strata_names = list(results['strata_effects'].keys())
198 effects = [results['strata_effects'][s]['effect'] for s in strata_names]
199 ses = [results['strata_effects'][s]['se'] for s in strata_names]
200
201 axes[0].barh(strata_names, effects, xerr=[1.96*se for se in ses],
202             alpha=0.7, color='steelblue', capsize=5)
203 axes[0].axvline(0, color='red', linestyle='--', linewidth=2)
204 axes[0].axvline(overall['effect'], color='green', linestyle='-', linewidth=2,
205                label=f"Overall: {overall['effect']:.4f}")
206 axes[0].set_xlabel('Treatment Effect', fontsize=12)
207 axes[0].set_ylabel('Stratum', fontsize=12)
208 axes[0].set_title('Stratum-Specific Treatment Effects', fontsize=12, fontweight='bold')
209 axes[0].legend()
210 axes[0].grid(alpha=0.3)
211
212 # Conversion rates by group and stratum
213 x = np.arange(len(strata_names))

```

```

213 width = 0.35
214 control_means = [results['strata_effects'][s]['mean_control'] for s in
    strata_names]
215 treatment_means = [results['strata_effects'][s]['mean_treatment'] for s
    in strata_names]
216
217 axes[1].bar(x - width/2, control_means, width, label='Control',
218             alpha=0.7, color='steelblue')
219 axes[1].bar(x + width/2, treatment_means, width, label='Treatment',
220             alpha=0.7, color='coral')
221 axes[1].set_xlabel('Stratum', fontsize=12)
222 axes[1].set_ylabel('Conversion_Rate', fontsize=12)
223 axes[1].set_title('Conversion_Rates_by_Stratum_and_Group', fontsize=12,
224                   fontweight='bold')
225 axes[1].set_xticks(x)
226 axes[1].set_xticklabels(strata_names)
227 axes[1].legend()
228 axes[1].grid(alpha=0.3, axis='y')
229
230 plt.tight_layout()
231 plt.savefig('stratified_randomization.pdf', dpi=300, bbox_inches='tight')
232 plt.show()

```

3.4.4 Cluster Randomization

Cluster randomization assigns groups of units rather than individual units. Essential when treatment naturally applies to groups or when individual-level randomization is infeasible.

When to use:

- Treatment inherently group-level (e.g., teacher training affects entire classrooms)
- Individual contamination likely (users in same household interact)
- Technical constraints prevent individual randomization
- Network effects mean individual outcomes are not independent

Key challenge: Outcomes within clusters are correlated, inflating standard errors. Effective sample size is reduced.

Definition 3.6 (Intraclass Correlation (ICC)). The ICC measures correlation of outcomes within clusters:

$$\rho = \frac{\sigma_{\text{between}}^2}{\sigma_{\text{between}}^2 + \sigma_{\text{within}}^2} \quad (3.1)$$

where $\sigma_{\text{between}}^2$ is variance between cluster means and σ_{within}^2 is variance within clusters.

Design effect: Cluster randomization increases required sample size by the design effect:

$$\text{DE} = 1 + (m - 1)\rho \quad (3.2)$$

where m is average cluster size and ρ is ICC. Higher ICC or larger clusters require proportionally more total units.

Example 3.5 (Cluster Randomization Penalty). Testing a feature where users in same household interact:

- ICC = 0.2 (moderate correlation within households)
- Average household size = 3
- Design effect: $1 + (3 - 1) \times 0.2 = 1.4$

Must collect 40% more households (or equivalently, more total users) compared to individual randomization to achieve same power.

Listing 3.4: Cluster Randomization Implementation

```

1 def cluster_randomization(cluster_ids: np.ndarray, p_treatment: float =
    0.5,
2                               seed: int = None) -> np.ndarray:
3     """
4     Cluster randomization: randomize at cluster level.
5
6     Parameters:
7     -----
8     cluster_ids: np.ndarray
9         Array indicating cluster membership for each unit
10    p_treatment: float
11        Proportion of clusters assigned to treatment
12    seed: int
13        Random seed for reproducibility
14
15    Returns:
16    -----
17    np.ndarray: Binary array (0=control, 1=treatment)
18    """
19    if seed is not None:
20        np.random.seed(seed)
21
22    # Get unique clusters
23    unique_clusters = np.unique(cluster_ids)
24    n_clusters = len(unique_clusters)
25
26    # Randomize clusters
27    n_treatment_clusters = int(n_clusters * p_treatment)
28    treatment_clusters = np.random.choice(unique_clusters,
29                                           n_treatment_clusters,
30                                           replace=False)
31
32    # Assign all units in treatment clusters
33    assignments = np.isin(cluster_ids, treatment_clusters).astype(int)
34
35    return assignments
36
37 def calculate_icc(outcomes: np.ndarray, cluster_ids: np.ndarray) ->
    float:
38     """
39     Calculate intraclass correlation coefficient.
40
41     Parameters:

```

```

42  """-----
43  """outcomes_: np.ndarray
44  """Outcome values
45  """cluster_ids_: np.ndarray
46  """Cluster identifiers
47
48  Returns:
49  """-----
50  """float_: ICC
51  """
52      unique_clusters = np.unique(cluster_ids)
53      n_clusters = len(unique_clusters)
54
55      # Grand mean
56      grand_mean = outcomes.mean()
57
58      # Calculate between and within variance
59      cluster_means = []
60      cluster_sizes = []
61      within_ss = 0
62
63      for cluster in unique_clusters:
64          cluster_mask = cluster_ids == cluster
65          cluster_outcomes = outcomes[cluster_mask]
66          cluster_mean = cluster_outcomes.mean()
67          cluster_size = len(cluster_outcomes)
68
69          cluster_means.append(cluster_mean)
70          cluster_sizes.append(cluster_size)
71
72          # Within-cluster sum of squares
73          within_ss += np.sum((cluster_outcomes - cluster_mean)**2)
74
75      # Between-cluster sum of squares
76      cluster_means = np.array(cluster_means)
77      cluster_sizes = np.array(cluster_sizes)
78      between_ss = np.sum(cluster_sizes * (cluster_means - grand_mean)
79                          **2)
80
81      # Mean squares
82      df_between = n_clusters - 1
83      df_within = len(outcomes) - n_clusters
84      ms_between = between_ss / df_between
85      ms_within = within_ss / df_within
86
87      # ICC calculation
88      mean_cluster_size = np.mean(cluster_sizes)
89      icc = (ms_between - ms_within) / (ms_between + (mean_cluster_size -
90          1) * ms_within)
91      icc = max(0, icc) # ICC should be non-negative
92
93      return icc
94
95  def analyze_cluster_experiment(outcomes: np.ndarray,
96                                assignments: np.ndarray,
97                                cluster_ids: np.ndarray) -> Dict:
98      """

```



```

98 Analyze_cluster-randomized_experiment_with_cluster-robust_SEs.
99
100 Parameters:
101 -----
102 outcomes: np.ndarray
103 Outcome values
104 assignments: np.ndarray
105 Treatment assignments (0/1)
106 cluster_ids: np.ndarray
107 Cluster identifiers
108
109 Returns:
110 -----
111 dict: Analysis results accounting for clustering
112 ""
113
114     from scipy import stats as sp_stats
115
116     # Calculate ICC
117     icc = calculate_icc(outcomes, cluster_ids)
118
119     # Get cluster-level summaries
120     unique_clusters = np.unique(cluster_ids)
121     cluster_means = []
122     cluster_assignments = []
123     cluster_sizes = []
124
125     for cluster in unique_clusters:
126         cluster_mask = cluster_ids == cluster
127         cluster_means.append(outcomes[cluster_mask].mean())
128         cluster_assignments.append(assignments[cluster_mask][0]) # All
129         same
130         cluster_sizes.append(cluster_mask.sum())
131
132     cluster_means = np.array(cluster_means)
133     cluster_assignments = np.array(cluster_assignments)
134     cluster_sizes = np.array(cluster_sizes)
135
136     # Cluster-level analysis
137     treatment_cluster_means = cluster_means[cluster_assignments == 1]
138     control_cluster_means = cluster_means[cluster_assignments == 0]
139
140     effect = treatment_cluster_means.mean() - control_cluster_means.
141     mean()
142
143     # Cluster-robust standard error
144     var_treatment = treatment_cluster_means.var() / len(
145     treatment_cluster_means)
146     var_control = control_cluster_means.var() / len(
147     control_cluster_means)
148     se_cluster = np.sqrt(var_treatment + var_control)
149
150     # Naive individual-level SE (for comparison)
151     treatment_outcomes = outcomes[assignments == 1]
152     control_outcomes = outcomes[assignments == 0]
153     var_treatment_naive = treatment_outcomes.var() / len(
154     treatment_outcomes)
155     var_control_naive = control_outcomes.var() / len(control_outcomes)
156     se_naive = np.sqrt(var_treatment_naive + var_control_naive)

```

```

151
152     # Design effect
153     mean_cluster_size = np.mean(cluster_sizes)
154     design_effect = 1 + (mean_cluster_size - 1) * icc
155
156     # Test statistic
157     z_stat = effect / se_cluster
158     p_value = 2 * (1 - sp_stats.norm.cdf(abs(z_stat)))
159
160     return {
161         'effect': effect,
162         'se_cluster_robust': se_cluster,
163         'se_naive': se_naive,
164         'inflation_factor': se_cluster / se_naive,
165         'icc': icc,
166         'design_effect': design_effect,
167         'mean_cluster_size': mean_cluster_size,
168         'n_clusters': len(unique_clusters),
169         'n_treatment_clusters': (cluster_assignments == 1).sum(),
170         'n_control_clusters': (cluster_assignments == 0).sum(),
171         'z_statistic': z_stat,
172         'p_value': p_value,
173         'ci_lower': effect - 1.96 * se_cluster,
174         'ci_upper': effect + 1.96 * se_cluster
175     }
176
177
178     # Example: Cluster randomization
179     print("\n" + "="*80)
180     print("Cluster Randomization Example")
181     print("="*80)
182
183     n_clusters = 200
184     cluster_sizes = np.random.poisson(10, n_clusters) + 1 # Vary cluster
185     sizes
186     n_users = cluster_sizes.sum()
187
188     # Create cluster IDs
189     cluster_ids = np.repeat(np.arange(n_clusters), cluster_sizes)
190
191     # Randomize at cluster level
192     np.random.seed(42)
193     assignments = cluster_randomization(cluster_ids, p_treatment=0.5, seed
194     =42)
195
196     # Simulate outcomes with ICC
197     icc_true = 0.3
198     treatment_effect = 0.05
199
200     # Generate cluster-level random effects
201     cluster_effects = np.random.normal(0, np.sqrt(icc_true), n_clusters)
202
203     # Generate outcomes
204     baseline_prob = 0.10
205     outcomes = np.zeros(n_users)
206
207     for i in range(n_users):
208         cluster_id = cluster_ids[i]

```

```

207     cluster_effect = cluster_effects[cluster_id]
208     treatment_contrib = treatment_effect if assignments[i] == 1 else 0
209
210     # Probability for this user
211     prob = baseline_prob + cluster_effect + treatment_contrib
212     prob = np.clip(prob, 0, 1)
213     outcomes[i] = np.random.binomial(1, prob)
214
215 # Analyze
216 results = analyze_cluster_experiment(outcomes, assignments, cluster_ids
217 )
218
219 print(f"\nCluster_Design:")
220 print(f"    Number_of_clusters: {results['n_clusters']}")
221 print(f"    Treatment_clusters: {results['n_treatment_clusters']}")
222 print(f"    Control_clusters: {results['n_control_clusters']}")
223 print(f"    Mean_cluster_size: {results['mean_cluster_size']:.1f}")
224 print(f"    Total_units: {n_users}")
225
226 print(f"\nIntraclass_Correlation:")
227 print(f"    ICC: {results['icc']:.4f} (true: {icc_true:.4f})")
228 print(f"    Design_effect: {results['design_effect']:.4f}")
229
230 print(f"\nTreatment_Effect:")
231 print(f"    Effect: {results['effect']:.4f} (true: {treatment_effect:.4f})")
232
233 print(f"    SE (cluster-robust): {results['se_cluster_robust']:.4f}")
234 print(f"    SE (naive): {results['se_naive']:.4f}")
235 print(f"    Inflation_factor: {results['inflation_factor']:.4f}")
236 print(f"    95% CI: [{results['ci_lower']:.4f}, {results['ci_upper']:.4f}]")
237
238 print(f"    z-statistic: {results['z_statistic']:.4f}")
239 print(f"    p-value: {results['p_value']:.6f}")
240
241 # Visualize clustering effect
242 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
243
244 # Distribution of cluster means
245 cluster_means_by_group = {
246     'Control': [],
247     'Treatment': []
248 }
249
250 for cluster in np.unique(cluster_ids):
251     cluster_mask = cluster_ids == cluster
252     cluster_mean = outcomes[cluster_mask].mean()
253     cluster_assignment = assignments[cluster_mask][0]
254
255     if cluster_assignment == 1:
256         cluster_means_by_group['Treatment'].append(cluster_mean)
257     else:
258         cluster_means_by_group['Control'].append(cluster_mean)
259
260 axes[0].hist(cluster_means_by_group['Control'], bins=20, alpha=0.6,
261             label='Control_clusters', color='steelblue', edgecolor='black')
262 axes[0].hist(cluster_means_by_group['Treatment'], bins=20, alpha=0.6,
263             label='Treatment_clusters', color='coral', edgecolor='black')

```

```

260 axes[0].set_xlabel('Cluster_Mean_Outcome', fontsize=12)
261 axes[0].set_ylabel('Frequency', fontsize=12)
262 axes[0].set_title('Distribution_of_Cluster_Means', fontsize=12,
    fontweight='bold')
263 axes[0].legend()
264 axes[0].grid(alpha=0.3)
265
266 # SE inflation as function of ICC
267 icc_range = np.linspace(0, 0.5, 50)
268 cluster_size = results['mean_cluster_size']
269 inflation = np.sqrt(1 + (cluster_size - 1) * icc_range)
270
271 axes[1].plot(icc_range, inflation, linewidth=2.5, color='steelblue')
272 axes[1].axvline(results['icc'], color='red', linestyle='--', linewidth
    =2,
273               label=f"Observed_ICC={results['icc']:.3f}")
274 axes[1].axhline(results['inflation_factor'], color='green', linestyle='
    --', linewidth=2,
275               label=f"Inflation={results['inflation_factor']:.3f}")
276 axes[1].set_xlabel('Intracluster_Correlation_(ICC)', fontsize=12)
277 axes[1].set_ylabel('SE_Inflation_Factor', fontsize=12)
278 axes[1].set_title(f'SE_Inflation_(cluster_size={cluster_size:.0f})',
279               fontsize=12, fontweight='bold')
280 axes[1].legend()
281 axes[1].grid(alpha=0.3)
282
283 plt.tight_layout()
284 plt.savefig('cluster_randomization.pdf', dpi=300, bbox_inches='tight')
285 plt.show()

```

3.5 Confounding Variables and Bias

Even with randomization, various biases can threaten experimental validity. Recognizing and addressing them is crucial.

3.5.1 Selection Bias

Selection bias occurs when treatment and control groups differ systematically due to the selection mechanism.

Common sources:

1. **Self-selection:** Users choose their group rather than being randomly assigned
2. **Non-random attrition:** Differential dropout between groups
3. **Survivorship bias:** Analyzing only users who complete the experiment
4. **Sample restriction:** Post-randomization filtering creates imbalance

Example 3.6 (Selection Bias in Practice). Testing a premium feature:

Biased approach: "Offer to all users, compare those who adopt vs. don't adopt"
 Adopters likely have:

- Higher income / willingness to pay

- Different needs
- More engagement
- Different demographics

Observed differences confound adoption decision with feature effect.

Correct approach: Randomly offer feature to 50%, don't offer to other 50% (regardless of interest). Compare outcomes between groups.

3.5.2 Survivorship Bias

Survivorship bias arises when analysis excludes units that drop out, especially if dropout is differential across groups.

Example 3.7 (Survivorship Bias). Testing a complex new interface:

- Treatment (new interface): 30% of users abandon immediately due to confusion
- Control (old interface): 10% abandon
- Analysis of remaining users shows treatment has higher engagement

Conclusion: New interface is better?

Problem: You're comparing the 70% who persevered through treatment (highly engaged, patient users) with 90% of control users (more representative). The selection creates bias.

Solution: Analyze all randomized users using intention-to-treat principle, not just those who completed.

3.5.3 Simpson's Paradox

Simpson's paradox occurs when a trend appears in different groups but reverses when groups are combined—or vice versa.

Example 3.8 (Simpson's Paradox in A/B Testing). Testing checkout redesign across mobile and desktop:

Platform	Control Conv.	Treatment Conv.	Winner
Mobile	5% (100/2000)	6% (120/2000)	Treatment
Desktop	15% (1500/10000)	16% (1600/10000)	Treatment
Combined	13.3% (1600/12000)	14.3% (1720/12000)	Treatment

Treatment wins on both platforms and overall—straightforward.

Now consider a different allocation:

Platform	Control Conv.	Treatment Conv.	Winner
Mobile	5% (50/1000)	6% (180/3000)	Treatment
Desktop	15% (1650/11000)	14% (1260/9000)	Control
Combined	14.2% (1700/12000)	12.0% (1440/12000)	Control

Treatment wins on mobile but loses on desktop and overall! The reversal occurs because:

- Treatment got more mobile users (low conversion)
- Control got more desktop users (high conversion)
- Platform composition confounds treatment effect

Resolution: Proper randomization prevents such confounding by balancing group composition. Post-stratification or regression adjustment can account for it in analysis.

Listing 3.5: Detecting Simpson's Paradox

```

1 def detect_simpsons_paradox(outcomes: np.ndarray,
2                             assignments: np.ndarray,
3                             strata: np.ndarray) -> Dict:
4     """
5     Detect Simpson's paradox in stratified data.
6
7     Parameters:
8     -----
9     outcomes: np.ndarray
10            Binary outcomes
11     assignments: np.ndarray
12            Treatment assignments (0/1)
13     strata: np.ndarray
14            Stratum indicators
15
16     Returns:
17     -----
18     dict: Analysis showing overall and stratum-specific effects
19     """
20     results = {
21         'overall': {},
22         'strata': {},
23         'paradox_detected': False
24     }
25
26     # Overall effect
27     overall_treatment_rate = outcomes[assignments == 1].mean()
28     overall_control_rate = outcomes[assignments == 0].mean()
29     overall_effect = overall_treatment_rate - overall_control_rate
30
31     results['overall'] = {
32         'treatment_rate': overall_treatment_rate,
33         'control_rate': overall_control_rate,
34         'effect': overall_effect,
35         'treatment_wins': overall_effect > 0
36     }
37
38     # Stratum-specific effects
39     unique_strata = np.unique(strata)
40     stratum_treatment_wins = []
41
42     for stratum in unique_strata:
43         stratum_mask = strata == stratum
44         stratum_outcomes = outcomes[stratum_mask]
45         stratum_assignments = assignments[stratum_mask]
46

```

```

47     treatment_rate = stratum_outcomes[stratum_assignments == 1].
        mean()
48     control_rate = stratum_outcomes[stratum_assignments == 0].mean
        ()
49     effect = treatment_rate - control_rate
50
51     results['strata'][stratum] = {
52         'treatment_rate': treatment_rate,
53         'control_rate': control_rate,
54         'effect': effect,
55         'treatment_wins': effect > 0,
56         'n_treatment': (stratum_assignments == 1).sum(),
57         'n_control': (stratum_assignments == 0).sum()
58     }
59
60     stratum_treatment_wins.append(effect > 0)
61
62     # Check for paradox
63     all_strata_agree = len(set(stratum_treatment_wins)) == 1
64     overall_agrees = results['overall']['treatment_wins'] ==
        stratum_treatment_wins[0]
65
66     if all_strata_agree and not overall_agrees:
67         results['paradox_detected'] = True
68         results['paradox_type'] = 'reversal'
69     elif not all_strata_agree:
70         results['paradox_detected'] = True
71         results['paradox_type'] = 'inconsistent'
72
73     return results
74
75
76     # Example: Simpson's Paradox simulation
77     print("\n" + "="*80)
78     print("Simpson's Paradox Detection")
79     print("="*80)
80
81     # Scenario 1: No paradox
82     np.random.seed(42)
83     n_mobile = 4000
84     n_desktop = 20000
85
86     platform = np.array(['Mobile'] * n_mobile + ['Desktop'] * n_desktop)
87     assignments_balanced = simple_randomization(len(platform), seed=42)
88
89     # Generate outcomes (treatment helps on both platforms)
90     outcomes_no_paradox = np.zeros(len(platform))
91     for i in range(len(platform)):
92         if platform[i] == 'Mobile':
93             base_rate = 0.05
94             treatment_lift = 0.01
95         else:
96             base_rate = 0.15
97             treatment_lift = 0.01
98
99     prob = base_rate + (treatment_lift if assignments_balanced[i] == 1
        else 0)
100    outcomes_no_paradox[i] = np.random.binomial(1, prob)

```

```

101 results_no_paradox = detect_simpsons_paradox(outcomes_no_paradox,
102                                             assignments_balanced,
103                                             platform)
104
105 print("\nScenario 1: Balanced Randomization (No Paradox)")
106 print("-" * 80)
107 print(f"Overall: Treatment={results_no_paradox['overall']['treatment_rate']:.4f},
108       f"Control={results_no_paradox['overall']['control_rate']:.4f},
109       f"Winner={ 'Treatment' if results_no_paradox['overall']['treatment_wins'] else 'Control' }")
110
111 for stratum, stats in results_no_paradox['strata'].items():
112     print(f"{stratum}: Treatment={stats['treatment_rate']:.4f},
113           f"Control={stats['control_rate']:.4f},
114           f"Winner={ 'Treatment' if stats['treatment_wins'] else 'Control' }")
115
116 print(f"Simpson's Paradox: {results_no_paradox['paradox_detected']}")
117
118 # Scenario 2: Simpson's Paradox due to imbalanced allocation
119 # Artificially create imbalance
120 assignments_imbalanced = np.zeros(len(platform), dtype=int)
121 # More treatment in mobile (low conversion)
122 assignments_imbalanced[platform == 'Mobile'] = np.random.binomial(
123     1, 0.75, n_mobile
124 )
125 # Less treatment in desktop (high conversion)
126 assignments_imbalanced[platform == 'Desktop'] = np.random.binomial(
127     1, 0.25, n_desktop
128 )
129
130 outcomes_paradox = outcomes_no_paradox.copy() # Same true effects
131
132 results_paradox = detect_simpsons_paradox(outcomes_paradox,
133                                           assignments_imbalanced,
134                                           platform)
135
136 print("\nScenario 2: Imbalanced Allocation (Potential Paradox)")
137 print("-" * 80)
138 print(f"Overall: Treatment={results_paradox['overall']['treatment_rate']:.4f},
139       f"Control={results_paradox['overall']['control_rate']:.4f},
140       f"Winner={ 'Treatment' if results_paradox['overall']['treatment_wins'] else 'Control' }")
141
142 for stratum, stats in results_paradox['strata'].items():
143     print(f"{stratum}: Treatment={stats['treatment_rate']:.4f},
144           f"Control={stats['control_rate']:.4f},
145           f"Winner={ 'Treatment' if stats['treatment_wins'] else 'Control' },
146           f"n_treat={stats['n_treatment']}, n_ctrl={stats['n_control']}")
147
148 print(f"Simpson's Paradox: {results_paradox['paradox_detected']}")
149 if results_paradox['paradox_detected']:

```



```

149     print(f"Type: {results_paradox['paradox_type']}")
150
151 # Visualize
152 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
153
154 # Scenario 1
155 strata_names = list(results_no_paradox['strata'].keys())
156 x = np.arange(len(strata_names) + 1)
157 treatment_rates = [results_no_paradox['overall']['treatment_rate']] + \
158     [results_no_paradox['strata'][s]['treatment_rate']
159     for s in strata_names]
160 control_rates = [results_no_paradox['overall']['control_rate']] + \
161     [results_no_paradox['strata'][s]['control_rate'] for s
162     in strata_names]
163 labels = ['Overall'] + strata_names
164
165 width = 0.35
166 axes[0].bar(x - width/2, control_rates, width, label='Control',
167             alpha=0.7, color='steelblue')
168 axes[0].bar(x + width/2, treatment_rates, width, label='Treatment',
169             alpha=0.7, color='coral')
170 axes[0].set_ylabel('Conversion_Rate', fontsize=12)
171 axes[0].set_title('Balanced_Allocation_(No_Paradox)', fontsize=12,
172                  fontweight='bold')
173 axes[0].set_xticks(x)
174 axes[0].set_xticklabels(labels, rotation=15)
175 axes[0].legend()
176 axes[0].grid(alpha=0.3, axis='y')
177
178 # Scenario 2
179 treatment_rates_p = [results_paradox['overall']['treatment_rate']] + \
180     [results_paradox['strata'][s]['treatment_rate'] for
181     s in strata_names]
182 control_rates_p = [results_paradox['overall']['control_rate']] + \
183     [results_paradox['strata'][s]['control_rate'] for s
184     in strata_names]
185
186 axes[1].bar(x - width/2, control_rates_p, width, label='Control',
187             alpha=0.7, color='steelblue')
188 axes[1].bar(x + width/2, treatment_rates_p, width, label='Treatment',
189             alpha=0.7, color='coral')
190 axes[1].set_ylabel('Conversion_Rate', fontsize=12)
191 axes[1].set_title("Imbalanced_Allocation_(Simpson's_Paradox)",
192                  fontsize=12, fontweight='bold')
193 axes[1].set_xticks(x)
194 axes[1].set_xticklabels(labels, rotation=15)
195 axes[1].legend()
196 axes[1].grid(alpha=0.3, axis='y')
197
198 plt.tight_layout()
199 plt.savefig('simpsons_paradox.pdf', dpi=300, bbox_inches='tight')
200 plt.show()

```

3.5.4 Network Effects

Network effects occur when one user's treatment affects another user's outcomes, violating the Stable Unit Treatment Value Assumption (SUTVA).

Definition 3.7 (SUTVA). The Stable Unit Treatment Value Assumption states that:

1. Each unit has only one potential outcome under each treatment (no hidden variations)
2. One unit's treatment does not affect another's outcome (no interference)

Common network effects in digital experiments:

- **Direct interaction:** Social features where treated users interact with control users (messaging, sharing, tagging)
- **Marketplace effects:** Two-sided platforms where treatment affecting buyers impacts sellers, and vice versa
- **Capacity constraints:** Limited resources (e.g., customer support) where treatment increasing demand affects all users
- **Competitive dynamics:** Treatment improving one seller's visibility reduces others' (zero-sum outcomes)

Consequences of network effects:

- Biased treatment effect estimates (usually toward zero)
- Invalid confidence intervals
- Spillover effects confound results
- May underestimate global impact

Solutions:

1. **Cluster randomization:** Randomize social networks, households, or geographic regions rather than individuals
2. **Geo-based experiments:** Randomize by city or region when network effects are geographically bounded
3. **Temporal rollouts:** Full rollout with before/after comparison (weaker causal inference but avoids contamination)
4. **Model-based adjustment:** Estimate and adjust for spillover effects (advanced, requires modeling assumptions)

3.6 Validity Considerations

Experimental validity assesses whether an experiment tests what it purports to test and whether conclusions generalize appropriately.

3.6.1 Internal Validity

Internal validity asks: Does the experiment actually measure the causal effect of treatment on outcomes, or are results confounded?

Threats to internal validity:

1. **Selection bias:** Non-random assignment or differential attrition
2. **History effects:** External events occurring during experiment affecting one group more than another
3. **Maturation effects:** Natural changes over time (learning, fatigue) affecting groups differently
4. **Instrumentation:** Measurement changes during experiment (e.g., bug fixes, metric definition changes)
5. **Testing effects:** Awareness of being tested affects behavior (Hawthorne effect)
6. **Regression to the mean:** Extreme values naturally regress toward average on retesting

Ensuring internal validity:

- Proper randomization
- Adequate sample size
- Blinding where feasible
- Consistent measurement
- Intention-to-treat analysis
- Pre-registration of hypotheses

3.6.2 External Validity

External validity asks: Do results generalize beyond the specific experimental context to other populations, settings, and times?

Threats to external validity:

1. **Sample non-representativeness:** Experimental sample differs from target population
2. **Setting artificiality:** Experimental conditions differ from real-world deployment
3. **Temporal specificity:** Results hold only at specific times (seasonality, trends)
4. **Treatment variation:** Experimental treatment differs from how it would be deployed at scale

Example 3.9 (External Validity Challenges). Testing a new mobile app design:

- **Sample:** Only iOS users, English language, US residents
- **Setting:** Short 1-week test during holiday season
- **Treatment:** Prototype with potential bugs, incomplete features

Questions:

- Will effects hold for Android users? Non-English speakers? Other countries?
- Are holiday patterns representative of typical usage?
- Will polished version perform differently than prototype?

These questions concern external validity—whether specific experimental findings generalize.

Improving external validity:

- Representative sampling
- Realistic treatment implementation
- Sufficient duration to capture normal patterns
- Multiple experiments across contexts
- Replication studies

3.6.3 Construct Validity

Construct validity asks: Do we measure the theoretical construct we intend to measure?

Common construct validity issues:

1. **Metric doesn't capture intent:** Using clicks as proxy for user satisfaction
2. **Metric captures unintended constructs:** Revenue increase might reflect price increase, not value increase
3. **Missing important dimensions:** Focusing on engagement while ignoring user well-being
4. **Confounded metrics:** Single metric conflates multiple constructs

Example 3.10 (Construct Validity). Goal: Improve user satisfaction with search results

Weak construct validity:

- Metric: Number of search result clicks
- Problem: More clicks might indicate *worse* results (users can't find answer, keep clicking)

Better construct validity:

- Metrics: Time to first click (faster is better), probability of return to search (lower is better), explicit satisfaction ratings
- These more directly measure the intended construct (satisfaction)

3.6.4 Statistical Conclusion Validity

Statistical conclusion validity asks: Are statistical inferences correct? Do we properly quantify uncertainty?

Threats to statistical conclusion validity:

1. **Low statistical power:** Insufficient sample size to detect real effects
2. **Violated assumptions:** Using tests that assume normality, independence, etc. when assumptions don't hold
3. **Fishing and p-hacking:** Multiple testing, selective reporting, post-hoc hypotheses
4. **Improper error estimation:** Ignoring clustering, using wrong standard errors
5. **Unreliable measurement:** High measurement error inflates variance, reduces power

3.7 Practical Design Decisions

Translating experimental design principles into actual A/B tests requires numerous practical decisions.

3.7.1 Unit of Randomization

The unit of randomization is the entity assigned to treatment or control. Common choices:

- **User:** Most common; tracks individual across sessions
- **Session:** Each visit randomized independently (rarely appropriate)
- **Device:** When user identity is unavailable or unreliable
- **Household/Account:** When multiple users share accounts
- **Geographic unit:** City, region, or country for market-level interventions
- **Temporal unit:** Time periods (week, day) for platform-wide changes

Considerations:

1. **Treatment application level:** Unit must be at least as coarse as treatment application
2. **Outcome measurement level:** Ideally aligned with measurement
3. **Network effects:** Cluster at level that contains interference
4. **Sample size:** More granular units provide larger samples but risk contamination

3.7.2 Temporal Considerations

Time affects experiments in multiple ways:

1. **Duration:**

- Too short: Miss delayed effects, affected by day-of-week patterns
- Too long: External events confound results, delayed decisions
- Typical: 1-4 weeks, capturing full weekly cycles

2. **Seasonality:**

- Holiday periods atypical
- Back-to-school, tax season, etc. for relevant domains
- Consider running in representative periods

3. **Novelty effects:**

- Initial curiosity inflates treatment metrics
- May fade after days/weeks
- Distinguish short-term from long-term effects

4. **Learning effects:**

- Users adapt to new interfaces
- Performance may improve over time
- Consider learning curves in duration decisions

3.7.3 Geographic Considerations

Geographic variation affects experimental design:

1. **Regulatory constraints:** Some features unavailable in certain regions
2. **Market maturity:** Effects may differ between mature and emerging markets
3. **Cultural differences:** UI conventions, color meanings, interaction patterns vary
4. **Infrastructure:** Network speed, device types affect experience
5. **Time zones:** Usage patterns differ; ensure balanced sampling across times

Approaches:

- **Stratify by region:** Ensure balance across key markets
- **Run region-specific tests:** Estimate heterogeneous effects
- **Stage rollouts geographically:** Test in one market before expanding

Platform	Issues	Considerations
Web	Session continuity, cookie deletion	Use server-side randomization
Mobile apps	App store delays, version fragmentation	Staged rollouts, feature flags
Desktop software	Infrequent updates, offline usage	Long experiment duration
APIs	Rate limiting, caching	Cache invalidation, proper logging
Email	Spam filters, client rendering	Check deliverability, test across clients

3.7.4 Platform-Specific Issues

Different platforms present unique challenges:

3.8 Summary

Experimental design principles form the foundation for trustworthy A/B testing:

- **Randomization** eliminates selection bias and enables causal inference. It's non-negotiable for valid experiments.
- **Control groups** provide the counterfactual, distinguishing treatment effects from background variation.
- **Blocking and stratification** improve precision by ensuring balance on important covariates.
- **Different randomization techniques**—simple, block, stratified, cluster—suit different contexts. Choose based on sample size, covariate importance, and interference patterns.
- **Biases**—selection, survivorship, confounding—threaten validity even with randomization. Recognize and mitigate them through design and analysis choices.
- **Validity** encompasses internal (causal correctness), external (generalizability), construct (measurement appropriateness), and statistical conclusion (inference correctness) dimensions. Strong experiments excel on all four.
- **Practical decisions**—unit of randomization, duration, geographic scope, platform specifics—require balancing statistical ideals with operational constraints.

Sound experimental design precedes and enables sound statistical analysis. Invest time in design; it pays dividends in reliable insights and confident decisions.

3.9 Further Reading

- Fisher [1935]: Fisher's foundational work on experimental design and randomization
- Box et al. [2005]: Comprehensive modern treatment of experimental design
- Imbens and Rubin [2015]: Causal inference framework for experimental and observational studies
- Kohavi et al. [2020]: Practical guide to online controlled experiments

- Hayes and Moulton [2017]: Detailed treatment of cluster randomized trials
- Eckles et al. [2016]: Network interference in experiments

3.10 Exercises

3.10.1 Conceptual Questions

1. Explain why randomization is essential for causal inference. What assumptions does randomization satisfy that observational studies typically violate?
2. A colleague proposes assigning even-numbered user IDs to treatment and odd-numbered to control. What's wrong with this approach?
3. When would you prefer stratified randomization over simple randomization? What are the costs and benefits?
4. Describe three scenarios where cluster randomization is necessary despite its statistical efficiency loss.
5. Explain Simpson's paradox in your own words. Provide an original example from a domain you're familiar with.
6. Distinguish between internal and external validity. Can an experiment have high internal validity but low external validity? Provide an example.

3.10.2 Design Exercises

1. Design an A/B test for a two-sided marketplace (e.g., Airbnb, Uber) where treatment affects one side but you care about both sides' outcomes. What design challenges arise? How would you address them?
2. You're testing a social feature where users can tag friends. Describe the network effects you'd expect. How would you design the randomization to account for these?
3. Design a long-duration experiment (6+ months) for a subscription service. What temporal factors would you consider? How would you handle seasonality?
4. Compare designs for testing the same feature on web, iOS app, and Android app. What platform-specific challenges arise? Would you run separate experiments or one combined experiment?
5. You want to test a pricing change but worry about fairness (charging different prices to different users). Design an experiment that balances statistical validity with ethical concerns.

3.10.3 Implementation Exercises

1. Implement a hash-based randomization function that:
 - Takes user ID and experiment name as input
 - Returns deterministic assignment (same user always gets same variant)
 - Supports arbitrary split ratios
 - Allows easy "salting" for independent experiments
2. Create a stratified randomization system that:
 - Accepts multiple stratifying variables
 - Handles continuous and categorical variables
 - Ensures balance within strata
 - Provides diagnostic plots and balance tests
3. Write a bias detection tool that:
 - Checks for selection bias via covariate balance tests
 - Detects potential Simpson's paradox across strata
 - Identifies temporal trends suggesting history effects
 - Generates report with visualizations and recommendations
4. Implement cluster randomization with ICC estimation:
 - Randomize at cluster level
 - Estimate ICC from pilot data
 - Calculate design effect and required sample size
 - Perform cluster-robust inference

3.10.4 Analysis Exercises

1. Given experimental data with known selection bias (provided dataset), use multiple methods to detect and quantify the bias. How would you correct for it in analysis?
2. Simulate a scenario with network effects. Compare results from individual randomization (ignoring interference) vs. cluster randomization. Quantify the bias introduced by ignoring network effects.
3. Create synthetic data exhibiting Simpson's paradox. Show reversal of treatment effect sign when aggregating vs. stratifying. Explain which analysis is correct and why.
4. Analyze a multi-platform experiment (web + mobile) where treatment was randomly assigned but platform composition differs between groups. Demonstrate stratified analysis accounting for this imbalance.
5. Given survival data (time-to-conversion) with differential attrition, compare intention-to-treat analysis vs. per-protocol analysis vs. survivor analysis. Discuss when each is appropriate.

3.10.5 Simulation Projects

1. **Randomization comparison:** Simulate 10,000 experiments comparing simple, block, and stratified randomization. Compare:

- Group size balance
- Covariate balance
- Power to detect effects
- Type I error rates

Write report with visualizations and recommendations.

2. **Network effects impact:** Simulate social network with varying levels of connectivity. Compare:

- Individual randomization (ignoring network)
- Cluster randomization by network communities
- Bias as function of network density and treatment effect strength

3. **Validity threats:** Implement experiments with various validity threats:

- Selection bias (non-random attrition)
- History effects (external shock during experiment)
- Instrumentation (metric definition change mid-experiment)
- Regression to mean (selecting extreme performers)

Quantify how each threat affects conclusions. Demonstrate mitigation strategies.

4. **Optimal stratification:** Simulate experiments with multiple potential stratifying variables. Investigate:

- How many strata provide optimal power?
- Which variables are worth stratifying on?
- Trade-off between stratification benefit and complexity

Chapter 4

Sample Size Calculation and Statistical Power

4.1 Learning Objectives

After completing this chapter, readers will be able to:

- Calculate required sample sizes for different types of A/B tests
- Understand the relationship between sample size, effect size, and statistical power
- Apply power analysis to determine experiment feasibility
- Create and interpret power curves
- Account for practical constraints in sample size determination
- Implement sample size calculators in Python and R
- Make informed decisions about minimum detectable effects

4.2 Introduction

"How many users do we need?" is perhaps the most common question in A/B testing. Too few, and we waste time running inconclusive experiments. Too many, and we delay decisions unnecessarily or expose users to potentially inferior variants longer than needed.

Sample size calculation transforms this practical question into a rigorous statistical framework. By specifying what we want to detect and how confident we need to be, we determine exactly how much data is required. This chapter develops the theory and practice of sample size determination, from fundamental formulas to real-world constraints that complicate the textbook approach.

4.3 Power Analysis Fundamentals

Statistical power is the probability of detecting an effect when it truly exists. Understanding power is essential for designing effective experiments.

4.3.1 Definition of Statistical Power

Definition 4.1 (Statistical Power). Statistical power is the probability of correctly rejecting the null hypothesis when the alternative hypothesis is true:

$$\text{Power} = 1 - \beta = P(\text{Reject } H_0 \mid H_1 \text{ true}) \quad (4.1)$$

where β is the probability of a Type II error (false negative).

Power answers the question: "If there really is an effect of size δ , what's the probability our experiment will detect it?"

Example 4.1 (Interpreting Power). Power = 0.80 means:

- If the true treatment effect is δ , we have an 80% chance of obtaining $p < \alpha$
- Equivalently, 20% chance of missing the effect (Type II error)
- If we ran 100 such experiments where an effect exists, 80 would correctly declare significance

4.3.2 Factors Affecting Power

Four interrelated factors determine statistical power:

1. **Effect Size (δ)**: Larger effects are easier to detect
 - Power increases with δ
 - Detecting 10% lift requires less data than 1% lift
2. **Sample Size (n)**: More data increases power
 - Power increases with $\sqrt{n}$
 - Doubling power requires roughly 4× sample size
3. **Significance Level (α)**: Higher α increases power
 - Relaxing from $\alpha = 0.01$ to 0.05 increases power
 - Tradeoff: more false positives
4. **Variance (σ^2)**: Lower variance increases power
 - Variance reduction techniques boost power without more data
 - Covered in Chapter 8

Theorem 4.1 (Power Relationships). For comparing two proportions with equal sample sizes:

$$\text{Power} = \Phi \left(\frac{\delta\sqrt{n}}{2\sqrt{p(1-p)}} - z_{1-\alpha/2} \right) \quad (4.2)$$

where $p = (p_1 + p_2)/2$, $\delta = p_2 - p_1$, and Φ is the standard normal CDF.
Key insights:

- Power increases with effect size δ
- Power increases with $\sqrt{n}$ (not linearly!)
- Power decreases as α becomes more stringent
- Power depends on baseline rate p (highest at $p = 0.5$)

4.3.3 The Type I and Type II Error Tradeoff

Sample size determination involves balancing two types of errors:

Reality	Fail to Reject H_0	Reject H_0
H_0 true	Correct decision ($1-\alpha$)	Type I Error (α)
H_1 true	Type II Error (β)	Correct decision ($1-\beta$)

Table 4.1: Type I and Type II errors in hypothesis testing

Conventional choices:

- $\alpha = 0.05$: Willing to accept 5% false positive rate
- Power = 0.80: Willing to accept 20% false negative rate

Why power = 0.80 is standard: This represents a 4:1 tradeoff between Type II and Type I errors ($\beta/\alpha = 0.20/0.05 = 4$). In most contexts, missing a real effect is considered 4× worse than claiming a spurious one.

When to use higher power:

- High implementation costs (difficult to change later)
- Regulatory requirements
- Mission-critical features
- One-shot opportunities

4.3.4 Power Curves and Interpretation

Power curves visualize how power depends on various parameters, providing intuition about experimental design tradeoffs.

Listing 4.1: Power Curves for Proportions

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 from scipy import stats
5 from typing import Tuple, List
6
7 sns.set_style("whitegrid")
8
9 def calculate_power_proportions(p1: float, p2: float, n: int,
10                                alpha: float = 0.05,
11                                alternative: str = 'two-sided') ->
                                float:

```

```

12     """
13     Calculate statistical power for two-sample proportion test.
14
15     Parameters:
16     -----
17     p1: float
18         Control proportion
19     p2: float
20         Treatment proportion
21     n: int
22         Sample size per group
23     alpha: float
24         Significance level
25     alternative: str
26         'two-sided' or 'one-sided'
27
28     Returns:
29     -----
30     float: Statistical power
31     """
32     # Effect size
33     delta = p2 - p1
34
35     # Pooled proportion under H0
36     p_bar = (p1 + p2) / 2
37
38     # Standard error under H0
39     se_null = np.sqrt(2 * p_bar * (1 - p_bar) / n)
40
41     # Standard error under H1
42     se_alt = np.sqrt((p1 * (1 - p1) + p2 * (1 - p2)) / n)
43
44     # Critical value
45     if alternative == 'two-sided':
46         z_crit = stats.norm.ppf(1 - alpha/2)
47     else:
48         z_crit = stats.norm.ppf(1 - alpha)
49
50     # Non-centrality parameter
51     ncp = delta / se_alt
52
53     # Power calculation
54     if alternative == 'two-sided':
55         power = 1 - stats.norm.cdf(z_crit - ncp) + stats.norm.cdf(-
56             z_crit - ncp)
57     else:
58         power = 1 - stats.norm.cdf(z_crit - ncp)
59
60     return power
61
62 def plot_power_curves(p1: float = 0.10, alpha: float = 0.05,
63                       save_path: str = None):
64     """
65     Generate comprehensive power curve visualizations.
66
67     Parameters:
68     -----

```

```

69 p1: float
70 Baseline_conversion_rate
71 alpha: float
72 Significance_level
73 save_path: str
74 Optional_path_to_save_figure
75 """
76 fig, axes = plt.subplots(2, 2, figsize=(16, 12))
77
78 # 1. Power vs Sample Size for different effect sizes
79 ax1 = axes[0, 0]
80 sample_sizes = np.arange(100, 10001, 100)
81 relative_lifts = [0.05, 0.10, 0.20, 0.30]
82
83 for lift in relative_lifts:
84     p2 = p1 * (1 + lift)
85     powers = [calculate_power_proportions(p1, p2, n, alpha) for n
86               in sample_sizes]
87     ax1.plot(sample_sizes, powers, linewidth=2.5,
88             label=f'{lift:.0%}_relative_lift', alpha=0.8)
89
90 ax1.axhline(0.8, color='red', linestyle='--', linewidth=2,
91            label='Target_power=0.80')
92 ax1.set_xlabel('Sample_Size(per_group)', fontsize=12)
93 ax1.set_ylabel('Statistical_Power', fontsize=12)
94 ax1.set_title(f'Power vs Sample Size (baseline={p1:.1%},  $\alpha$ = $\alpha$ {alpha})',
95             fontsize=13, fontweight='bold')
96 ax1.legend(fontsize=10)
97 ax1.grid(alpha=0.3)
98 ax1.set_ylim([0, 1])
99
100 # 2. Power vs Effect Size for different sample sizes
101 ax2 = axes[0, 1]
102 relative_lifts = np.linspace(0.01, 0.50, 100)
103 sample_sizes_to_plot = [500, 1000, 2000, 5000]
104
105 for n in sample_sizes_to_plot:
106     powers = []
107     for lift in relative_lifts:
108         p2 = p1 * (1 + lift)
109         power = calculate_power_proportions(p1, p2, n, alpha)
110         powers.append(power)
111     ax2.plot(relative_lifts * 100, powers, linewidth=2.5,
112            label=f'n={n:,}', alpha=0.8)
113
114 ax2.axhline(0.8, color='red', linestyle='--', linewidth=2,
115            label='Target_power=0.80')
116 ax2.set_xlabel('Relative_Lift(%)', fontsize=12)
117 ax2.set_ylabel('Statistical_Power', fontsize=12)
118 ax2.set_title(f'Power vs Effect Size (baseline={p1:.1%},  $\alpha$ = $\alpha$ {alpha})',
119             fontsize=13, fontweight='bold')
120 ax2.legend(fontsize=10)
121 ax2.grid(alpha=0.3)
122 ax2.set_ylim([0, 1])
123
124 # 3. Power vs Baseline Rate for fixed relative lift

```

```

124 ax3 = axes[1, 0]
125 baselines = np.linspace(0.01, 0.50, 100)
126 relative_lift = 0.10
127 sample_sizes_to_plot = [1000, 2000, 5000, 10000]
128
129 for n in sample_sizes_to_plot:
130     powers = []
131     for p1_var in baselines:
132         p2 = p1_var * (1 + relative_lift)
133         power = calculate_power_proportions(p1_var, p2, n, alpha)
134         powers.append(power)
135     ax3.plot(baselines * 100, powers, linewidth=2.5,
136             label=f'N={n}', alpha=0.8)
137
138 ax3.axhline(0.8, color='red', linestyle='--', linewidth=2,
139            label='Target power=0.80')
140 ax3.set_xlabel('Baseline Conversion Rate (%)', fontsize=12)
141 ax3.set_ylabel('Statistical Power', fontsize=12)
142 ax3.set_title(f'Power vs Baseline Rate ({relative_lift:.0%} \u03b1={alpha})',
143             fontsize=13, fontweight='bold')
144 ax3.legend(fontsize=10)
145 ax3.grid(alpha=0.3)
146 ax3.set_ylim([0, 1])
147
148 # 4. Required Sample Size vs Effect Size
149 ax4 = axes[1, 1]
150 relative_lifts = np.linspace(0.05, 0.50, 50)
151 target_powers = [0.70, 0.80, 0.90]
152
153 for target_power in target_powers:
154     required_n = []
155     for lift in relative_lifts:
156         p2 = p1 * (1 + lift)
157         # Binary search for required n
158         n_low, n_high = 10, 100000
159         while n_high - n_low > 1:
160             n_mid = (n_low + n_high) // 2
161             power = calculate_power_proportions(p1, p2, n_mid,
162                                                alpha)
163             if power < target_power:
164                 n_low = n_mid
165             else:
166                 n_high = n_mid
167             required_n.append(n_high)
168
169 ax4.plot(relative_lifts * 100, required_n, linewidth=2.5,
170         label=f'Power={target_power:.0%}', alpha=0.8)
171
172 ax4.set_xlabel('Relative Lift (%)', fontsize=12)
173 ax4.set_ylabel('Required Sample Size (per group)', fontsize=12)
174 ax4.set_title(f'Sample Size Requirements (baseline={p1:.1%}, \u03b1={alpha})',
175             fontsize=13, fontweight='bold')
176 ax4.legend(fontsize=10)
177 ax4.grid(alpha=0.3)
178 ax4.set_yscale('log')

```



```

179 plt.tight_layout()
180
181 if save_path:
182     plt.savefig(save_path, dpi=300, bbox_inches='tight')
183 plt.show()
184
185 # Print key insights
186 print("="*80)
187 print("Power Analysis Insights")
188 print("="*80)
189
190 # Calculate some specific examples
191 print(f"\nBaseline conversion rate: {p1:.1%}")
192 print(f"Significance level:  $\alpha$ ={alpha}")
193 print("\nSample size required for 80% power:\n")
194
195 for lift in [0.05, 0.10, 0.20]:
196     p2 = p1 * (1 + lift)
197     # Binary search
198     n_low, n_high = 10, 100000
199     while n_high - n_low > 1:
200         n_mid = (n_low + n_high) // 2
201         power = calculate_power_proportions(p1, p2, n_mid, alpha)
202         if power < 0.80:
203             n_low = n_mid
204         else:
205             n_high = n_mid
206
207     print(f"_{lift:>5.0%}_ lift_({p1:.1%}->_{p2:.1%}):_{n_high}
208           :>6,}users/group")
209
210 # Generate power curves
211 print("Generating power curve visualizations...\n")
212 plot_power_curves(p1=0.10, alpha=0.05, save_path='power_curves.pdf')

```

4.4 Sample Size Formulas

Sample size formulas translate desired power, significance level, and effect size into required data volume. We derive formulas for the most common metrics.

4.4.1 Sample Size for Proportions (Conversion Rates)

The most common A/B test compares conversion rates between two groups.

Two-Sample Test for Proportions

Test: $H_0 : p_1 = p_2$ vs. $H_1 : p_1 \neq p_2$

Assumptions:

- Two independent samples
- Equal sample sizes $n_1 = n_2 = n$
- Two-sided test

Theorem 4.2 (Sample Size for Proportions). The required sample size per group is:

$$n = \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 \cdot [p_1(1-p_1) + p_2(1-p_2)]}{(p_2 - p_1)^2} \quad (4.3)$$

where:

- $z_{1-\alpha/2}$ is the $(1 - \alpha/2)$ quantile of standard normal (e.g., 1.96 for $\alpha = 0.05$)
- $z_{1-\beta}$ is the $(1 - \beta)$ quantile of standard normal (e.g., 0.84 for power = 0.80)
- p_1, p_2 are the control and treatment proportions

Derivation (sketch). Under H_0 , the test statistic follows:

$$Z_0 = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{\bar{p}(1-\bar{p})(1/n + 1/n)}} \sim \mathcal{N}(0, 1) \quad (4.4)$$

Under H_1 with true difference $\delta = p_2 - p_1$:

$$Z_1 = \frac{\hat{p}_2 - \hat{p}_1 - \delta}{\sqrt{p_1(1-p_1)/n + p_2(1-p_2)/n}} \sim \mathcal{N}(0, 1) \quad (4.5)$$

For power $1 - \beta$, we require:

$$P(|Z_0| > z_{1-\alpha/2} \mid H_1) = 1 - \beta \quad (4.6)$$

This occurs when:

$$\frac{\delta}{\sqrt{(p_1(1-p_1) + p_2(1-p_2))/n}} = z_{1-\alpha/2} + z_{1-\beta} \quad (4.7)$$

Solving for n yields the formula above. $\square$

Example 4.2 (Sample Size Calculation). An e-commerce site wants to test a new check-out flow:

- Current conversion rate: $p_1 = 0.10$ (10%)
- Minimum detectable effect: 10% relative lift $\rightarrow p_2 = 0.11$ (11%)
- Significance level: $\alpha = 0.05$
- Desired power: $1 - \beta = 0.80$

$$n = \frac{(1.96 + 0.84)^2 \cdot [0.10(0.90) + 0.11(0.89)]}{(0.11 - 0.10)^2} \quad (4.8)$$

$$= \frac{7.84 \cdot [0.09 + 0.0979]}{0.0001} \quad (4.9)$$

$$= \frac{7.84 \cdot 0.1879}{0.0001} \quad (4.10)$$

$$= 14,732 \quad (4.11)$$

Required: 14,732 users per group (29,464 total).

At 1,000 users/day, this experiment takes 30 days.

Listing 4.2: Sample Size Calculator for Proportions

```

1 def sample_size_proportions(p1: float, p2: float,
2                             alpha: float = 0.05,
3                             power: float = 0.80,
4                             ratio: float = 1.0) -> dict:
5     """
6     Calculate required sample size for two-proportion test.
7
8     Parameters:
9     -----
10    p1: float
11        Control proportion
12    p2: float
13        Treatment proportion
14    alpha: float
15        Significance level (default 0.05)
16    power: float
17        Statistical power (default 0.80)
18    ratio: float
19        Ratio n2/n1 (default 1.0 for equal allocation)
20
21    Returns:
22    -----
23    dict: Sample sizes and related statistics
24    """
25    from scipy import stats
26
27    # Critical values
28    z_alpha = stats.norm.ppf(1 - alpha/2) # Two-sided
29    z_beta = stats.norm.ppf(power)
30
31    # Effect size
32    delta = p2 - p1
33    if delta == 0:
34        raise ValueError("p1 and p2 must be different")
35
36    # Sample size calculation
37    if ratio == 1.0:
38        # Equal allocation (more efficient)
39        numerator = (z_alpha + z_beta)**2 * (p1*(1-p1) + p2*(1-p2))
40        denominator = delta**2
41        n1 = numerator / denominator
42        n2 = n1
43    else:
44        # Unequal allocation
45        p_avg = (p1 + ratio * p2) / (1 + ratio)
46        numerator = (z_alpha * np.sqrt((1+ratio)*p_avg*(1-p_avg)) +
47                    z_beta * np.sqrt(p1*(1-p1) + ratio*p2*(1-p2)))**2
48        denominator = ratio * delta**2
49        n1 = numerator / denominator
50        n2 = ratio * n1
51
52    # Round up
53    n1 = int(np.ceil(n1))
54    n2 = int(np.ceil(n2))
55
56    # Calculate actual power with these sample sizes
57    se = np.sqrt(p1*(1-p1)/n1 + p2*(1-p2)/n2)

```

```

58     ncp = delta / se
59     actual_power = 1 - stats.norm.cdf(z_alpha - ncp) + stats.norm.cdf(-
        z_alpha - ncp)
60
61     return {
62         'n1': n1,
63         'n2': n2,
64         'n_total': n1 + n2,
65         'p1': p1,
66         'p2': p2,
67         'absolute_difference': delta,
68         'relative_lift': delta / p1,
69         'alpha': alpha,
70         'requested_power': power,
71         'actual_power': actual_power,
72         'z_alpha': z_alpha,
73         'z_beta': z_beta
74     }
75
76
77 def sample_size_table(p1: float, relative_lifts: List[float],
78                       alpha: float = 0.05, power: float = 0.80):
79     """
80     Generate table of sample sizes for different effect sizes.
81
82     Parameters:
83     -----
84     p1: float
85         Baseline conversion rate
86     relative_lifts: List[float]
87         List of relative lifts to evaluate
88     alpha: float
89         Significance level
90     power: float
91         Desired power
92     """
93     print("="*90)
94     print(f"Sample Size Requirements (baseline={p1:.1%}, α={alpha},
95           power={power:.0%})")
96     print("="*90)
97     print(f"{'Relative Lift':<15} {'Absolute':<12} {'p2':<10}"
98           f"{'n_per_group':<15} {'Total_n':<12} {'Duration(days)':>15}"
99           ")")
100     print("-"*90)
101
102     for lift in relative_lifts:
103         p2 = p1 * (1 + lift)
104         result = sample_size_proportions(p1, p2, alpha, power)
105
106         # Assume 1000 users per day per group
107         duration = np.ceil(result['n1'] / 1000)
108
109         print(f"{{lift:>14.1%}} {{result['absolute_difference']:>11.4f}} {{
110               f"{{p2:>9.3f}} {{result['n1']:>14,}} {{result['n_total']:>11,}}
111               }}
112               f"{{duration:>14.0f}}")
113
114     print("="*90)

```

```

112     print("Assumes 1,000 users per day per group")
113     print()
114
115
116 # Example usage
117 print("\n" + "="*80)
118 print("Sample Size Calculations for A/B Testing")
119 print("="*80 + "\n")
120
121 # Example 1: E-commerce checkout
122 print("Example 1: E-commerce Checkout Optimization")
123 print("-"*80)
124 result = sample_size_proportions(p1=0.10, p2=0.11, alpha=0.05, power
    =0.80)
125 print(f"Baseline conversion: {result['p1']:.1%}")
126 print(f"Target conversion: {result['p2']:.1%}")
127 print(f"Relative lift: {result['relative_lift']:.1%}")
128 print(f"Required n per group: {result['n1']:,}")
129 print(f"Total required n: {result['n_total']:,}")
130 print(f"Actual power: {result['actual_power']:.3f}")
131 print()
132
133 # Example 2: Generate table
134 print("Example 2: Sample Size Table for Different Effect Sizes")
135 print("-"*80)
136 sample_size_table(p1=0.10,
137                   relative_lifts=[0.05, 0.10, 0.15, 0.20, 0.25, 0.30],
138                   alpha=0.05,
139                   power=0.80)
140
141 # Example 3: Compare different power levels
142 print("Example 3: Impact of Power Requirements")
143 print("-"*80)
144 p1, p2 = 0.10, 0.12
145 print(f"Baseline: {p1:.1%}, Target: {p2:.1%} ({(p2-p1)/p1:.0%} lift)\n"
    )
146 print(f"{'Power':<10} {'n per group':<15} {'Total n':<12}")
147 print("-"*40)
148 for power in [0.70, 0.75, 0.80, 0.85, 0.90]:
149     result = sample_size_proportions(p1, p2, power=power)
150     print(f"{'power':<10.0%} {result['n1']:<14,} {result['n_total']:<11,}"
    )

```

4.4.2 Sample Size for Means (Continuous Metrics)

When testing continuous outcomes like revenue, session duration, or engagement scores, we use formulas for comparing means.

Theorem 4.3 (Sample Size for Means). For testing $H_0 : \mu_1 = \mu_2$ vs. $H_1 : \mu_1 \neq \mu_2$ with equal sample sizes:

$$n = \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{(\mu_2 - \mu_1)^2} \quad (4.12)$$

where:

- σ^2 is the common variance (assumed equal in both groups)

- μ_1, μ_2 are the control and treatment means

For unequal variances (Welch's t-test):

$$n_1 = \frac{(z_{1-\alpha/2} + z_{1-\beta})^2(\sigma_1^2 + \sigma_2^2/r)}{(\mu_2 - \mu_1)^2} \quad (4.13)$$

where $r = n_2/n_1$ is the allocation ratio.

Cohen's d effect size:

Often we specify effect size in standardized units:

$$d = \frac{\mu_2 - \mu_1}{\sigma} \quad (4.14)$$

Then:

$$n = \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2}{d^2} \quad (4.15)$$

Cohen's conventions:

- Small effect: $d = 0.2$
- Medium effect: $d = 0.5$
- Large effect: $d = 0.8$

Example 4.3 (Sample Size for Revenue). Testing pricing change:

- Current average revenue per user: \$50
- Standard deviation: \$30
- Minimum detectable increase: \$5 (10%)
- $\alpha = 0.05$, power = 0.80

$$n = \frac{2(1.96 + 0.84)^2 \cdot 30^2}{5^2} \quad (4.16)$$

$$= \frac{2 \cdot 7.84 \cdot 900}{25} \quad (4.17)$$

$$= \frac{14,112}{25} \quad (4.18)$$

$$= 565 \quad (4.19)$$

Required: 565 users per group (1,130 total).

Cohen's d: $d = 5/30 = 0.167$ (small effect).

Listing 4.3: Sample Size Calculator for Means

```

1 def sample_size_means(mu1: float, mu2: float, sigma: float,
2     alpha: float = 0.05,
3     power: float = 0.80,
4     ratio: float = 1.0) -> dict:
5     """

```

```

6  """Calculate required sample size for two-sample t-test.
7
8  """Parameters:
9  """-----
10 """mu1: float
11 """Control mean
12 """mu2: float
13 """Treatment mean
14 """sigma: float
15 """Common standard deviation
16 """alpha: float
17 """Significance level
18 """power: float
19 """Statistical power
20 """ratio: float
21 """Ratio n2/n1
22
23 """Returns:
24 """-----
25 """dict: Sample sizes and effect size measures
26 """
27 from scipy import stats
28
29 # Critical values
30 z_alpha = stats.norm.ppf(1 - alpha/2)
31 z_beta = stats.norm.ppf(power)
32
33 # Effect size
34 delta = mu2 - mu1
35 if delta == 0:
36     raise ValueError("mu1 and mu2 must be different")
37
38 cohens_d = delta / sigma
39
40 # Sample size
41 if ratio == 1.0:
42     n1 = 2 * (z_alpha + z_beta)**2 * sigma**2 / delta**2
43     n2 = n1
44 else:
45     n1 = (1 + 1/ratio) * (z_alpha + z_beta)**2 * sigma**2 / delta
46         **2
47     n2 = ratio * n1
48
49 n1 = int(np.ceil(n1))
50 n2 = int(np.ceil(n2))
51
52 # Actual power
53 se = sigma * np.sqrt(1/n1 + 1/n2)
54 ncp = delta / se
55 actual_power = 1 - stats.norm.cdf(z_alpha - ncp) + stats.norm.cdf(-
56     z_alpha - ncp)
57
58 return {
59     'n1': n1,
60     'n2': n2,
61     'n_total': n1 + n2,
62     'mu1': mu1,
63     'mu2': mu2,

```

```

62     'sigma': sigma,
63     'absolute_difference': delta,
64     'relative_difference': delta / mu1,
65     'cohens_d': cohens_d,
66     'effect_size_interpretation': interpret_cohens_d(abs(cohens_d))
67     ,
68     'alpha': alpha,
69     'requested_power': power,
70     'actual_power': actual_power
71 }
72
73 def interpret_cohens_d(d: float) -> str:
74     """Interpret Cohen's d effect size."""
75     if d < 0.2:
76         return "Negligible"
77     elif d < 0.5:
78         return "Small"
79     elif d < 0.8:
80         return "Medium"
81     else:
82         return "Large"
83
84
85 # Example usage
86 print("\n" + "="*80)
87 print("Sample Size for Continuous Metrics")
88 print("="*80 + "\n")
89
90 # Example 1: Revenue per user
91 print("Example 1: Revenue Per User")
92 print("-"*80)
93 result = sample_size_means(mu1=50, mu2=55, sigma=30, alpha=0.05, power
94                             =0.80)
95 print(f"Control mean: {result['mu1']:.2f}")
96 print(f"Treatment mean: {result['mu2']:.2f}")
97 print(f"Standard dev: {result['sigma']:.2f}")
98 print(f"Absolute diff: {result['absolute_difference']:.2f}")
99 print(f"Relative diff: {result['relative_difference']:.1%}")
100 print(f"Cohen's d: {result['cohens_d']:.3f} ({result['effect_size_interpretation']})")
101 print(f"Required n/group: {result['n1']:,}")
102 print(f"Total required n: {result['n_total']:,}")
103 print(f"Actual power: {result['actual_power']:.3f}")
104 print()
105
106 # Example 2: Session duration
107 print("Example 2: Session Duration (seconds)")
108 print("-"*80)
109 result = sample_size_means(mu1=180, mu2=200, sigma=120, alpha=0.05,
110                             power=0.80)
111 print(f"Control mean: {result['mu1']:.0f}s")
112 print(f"Treatment mean: {result['mu2']:.0f}s")
113 print(f"Increase: {result['absolute_difference']:.0f}s ({result['relative_difference']:.1%})")
114 print(f"Cohen's d: {result['cohens_d']:.3f} ({result['effect_size_interpretation']})")
115 print(f"Required n/group: {result['n1']:,}")

```



```

114 print()
115
116 # Example 3: Effect size table
117 print("Example 3: Sample Size by Effect Size (sigma=30)")
118 print("-"*80)
119 print(f"{'Cohen\'s d':<12}{'Interpretation':<15}{'Delta mu':<10}"
120       f"{'n per group':<15}{'Total n':<12}")
121 print("-"*70)
122
123 for d in [0.1, 0.2, 0.3, 0.5, 0.8]:
124     delta = d * 30 # sigma = 30
125     result = sample_size_means(mu1=100, mu2=100+delta, sigma=30)
126     print(f"{'d':<12.1f}{'result['effect_size_interpretation']':<15}"
127           f"{'delta':<10.1f}{'result['n1']':<14,}{'result['n_total']':<11,}"
128           ")

```

4.4.3 Sample Size for Survival Data

For time-to-event outcomes (e.g., time to purchase, churn time), sample size depends on the expected number of events rather than just sample size.

Theorem 4.4 (Sample Size for Survival Analysis). For comparing two survival curves using the log-rank test:

$$n = \frac{(z_{1-\alpha/2} + z_{1-\beta})^2}{p_1 p_2 (\log(HR))^2} \quad (4.20)$$

where:

- HR is the hazard ratio (treatment/control)
- p_1, p_2 are proportions in each group
- The formula gives total number of *events* needed

Total sample size depends on event rate:

$$N = \frac{n}{\text{event rate}} \quad (4.21)$$

4.4.4 Sample Size for Ratio Metrics

Ratio metrics (e.g., revenue per user, clicks per session) require special handling due to their non-normal distributions.

Delta method approximation:

For ratio $R = X/Y$ where X and Y are correlated:

$$\text{Var}(R) \approx \frac{1}{\mu_Y^2} [\text{Var}(X) + R^2 \text{Var}(Y) - 2R \text{Cov}(X, Y)] \quad (4.22)$$

Sample size calculation uses this variance in the standard means formula, but often requires pilot data to estimate covariances.

Practical approach: Bootstrap or simulate from pilot data to estimate variance, then use means formula.

4.5 Practical Considerations

Textbook formulas provide starting points, but real-world experiments require additional considerations.

4.5.1 Minimum Detectable Effect (MDE)

The minimum detectable effect is the smallest effect size your experiment can reliably detect given sample size, power, and significance level constraints.

Definition 4.2 (Minimum Detectable Effect). Given fixed n , α , and power, the MDE is the effect size δ satisfying:

$$\text{Power}(\delta, n, \alpha) = 1 - \beta \quad (4.23)$$

Why MDE matters:

- Experiments can only detect effects $\geq$ MDE reliably
- Smaller effects may exist but will appear non-significant
- MDE should align with minimum practically important difference

Example 4.4 (MDE Calculation). Given:

- Available sample: 5,000 users per group
- Baseline conversion: 10%
- $\alpha = 0.05$, power = 0.80

Solve for MDE by inverting sample size formula:

$$\delta = \sqrt{\frac{(z_{1-\alpha/2} + z_{1-\beta})^2 [p_1(1-p_1) + p_2(1-p_2)]}{n}} \quad (4.24)$$

With $p_2 = p_1 + \delta$ and $n = 5000$, we solve iteratively to find MDE ≈ 0.0165 (1.65 percentage points), corresponding to 16.5% relative lift.

Interpretation: This experiment can reliably detect effects of 16.5% or larger, but will likely miss smaller effects.

Listing 4.4: MDE Calculator

```

1 def calculate_mde(p1: float, n: int,
2                   alpha: float = 0.05,
3                   power: float = 0.80) -> dict:
4     """
5     Calculate minimum detectable effect given sample size.
6
7     Parameters:
8     -----
9     p1: float
10    Baseline conversion rate
11    n: int
12    Sample size per group
13    alpha: float

```

```

14  """Significance_level
15  """power: float
16  """Desired_power
17
18  Returns:
19  """-----
20  """dict: MDE_statistics
21  """
22      from scipy import stats
23
24      z_alpha = stats.norm.ppf(1 - alpha/2)
25      z_beta = stats.norm.ppf(power)
26
27      # Binary search for MDE
28      delta_low, delta_high = 0.0001, 1.0
29      epsilon = 0.0001
30
31      while delta_high - delta_low > epsilon:
32          delta_mid = (delta_low + delta_high) / 2
33          p2 = p1 + delta_mid
34
35          # Calculate power for this effect size
36          actual_power = calculate_power_proportions(p1, p2, n, alpha)
37
38          if actual_power < power:
39              delta_low = delta_mid
40          else:
41              delta_high = delta_mid
42
43      mde_absolute = delta_high
44      mde_relative = mde_absolute / p1
45
46      return {
47          'p1': p1,
48          'n_per_group': n,
49          'mde_absolute': mde_absolute,
50          'mde_relative': mde_relative,
51          'p2': p1 + mde_absolute,
52          'alpha': alpha,
53          'power': power
54      }
55
56
57  def mde_sensitivity_analysis(p1: float, sample_sizes: List[int],
58                              alpha: float = 0.05, power: float = 0.80):
59      """
60      Analyze how MDE changes with sample size.
61      """
62      print("="*80)
63      print(f"MDE Sensitivity Analysis (baseline={p1:.1%}, α={alpha},
64            power={power:.0%})")
65      print("="*80)
66      print(f"{'n_per_group':<15}{'MDE(abs)':<12}{'MDE(rel)':<12}"
67            f"{'p2':<10}{'Feasibility':<20}")
68      print("-"*80)
69
70      for n in sample_sizes:
71          result = calculate_mde(p1, n, alpha, power)

```

```

71     feasibility = assess_mde_feasibility(result['mde_relative'])
72
73     print(f"{n:<14,}\t{result['mde_absolute']:<11.4f}\t"
74           f"{result['mde_relative']:<11.1%}\t{result['p2']:<9.3f}\t{feasibility:<20}")
75
76     print("="*80 + "\n")
77
78
79 def assess_mde_feasibility(mde_relative: float) -> str:
80     """Assess whether MDE is practically feasible."""
81     if mde_relative < 0.05:
82         return "Very ambitious"
83     elif mde_relative < 0.10:
84         return "Ambitious"
85     elif mde_relative < 0.20:
86         return "Moderate"
87     else:
88         return "Conservative"
89
90
91 # Example usage
92 print("\n" + "="*80)
93 print("Minimum Detectable Effect (MDE) Analysis")
94 print("="*80 + "\n")
95
96 # Example 1: What can we detect with available traffic?
97 print("Example 1: MDE for Given Sample Size")
98 print("-"*80)
99 result = calculate_mde(p1=0.10, n=5000, alpha=0.05, power=0.80)
100 print(f"Baseline conversion: {result['p1']:.1%}")
101 print(f"Sample size per group: {result['n_per_group']:,}")
102 print(f"MDE (absolute): {result['mde_absolute']:.4f} ({result['mde_absolute']*100:.2f}pp)")
103 print(f"MDE (relative): {result['mde_relative']:.1%}")
104 print(f"Detectable p2: {result['p2']:.3f}")
105 print()
106
107 # Example 2: Sensitivity analysis
108 print("Example 2: How MDE Changes with Sample Size")
109 print("-"*80)
110 mde_sensitivity_analysis(
111     p1=0.10,
112     sample_sizes=[1000, 2000, 5000, 10000, 20000, 50000],
113     alpha=0.05,
114     power=0.80
115 )
116
117 # Visualization
118 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
119
120 # Plot 1: MDE vs sample size
121 sample_sizes = np.logspace(2.5, 5, 50)
122 mdes_abs = []
123 mdes_rel = []
124
125 for n in sample_sizes:
126     n_int = int(n)

```

```

127     result = calculate_mde(p1=0.10, n=n_int, alpha=0.05, power=0.80)
128     mdes_abs.append(result['mde_absolute'])
129     mdes_rel.append(result['mde_relative'])
130
131 axes[0].plot(sample_sizes, np.array(mdes_abs)*100, linewidth=2.5, color
132             ='steelblue')
133 axes[0].set_xlabel('Sample_Size(per_group)', fontsize=12)
134 axes[0].set_ylabel('MDE_(percentage_points)', fontsize=12)
135 axes[0].set_title('Minimum_Detectable_Effect_vs_Sample_Size\n(baseline_
136                   =_10%,_alpha=0.05,_power=0.80)',
137                   fontsize=12, fontweight='bold')
138 axes[0].set_xscale('log')
139 axes[0].grid(alpha=0.3)
140
141 axes[1].plot(sample_sizes, np.array(mdes_rel)*100, linewidth=2.5, color
142             ='coral')
143 axes[1].set_xlabel('Sample_Size(per_group)', fontsize=12)
144 axes[1].set_ylabel('MDE_(relative_lift_%)', fontsize=12)
145 axes[1].set_title('Relative_MDE_vs_Sample_Size\n(baseline=_10%,_alpha
146                   =0.05,_power=0.80)',
147                   fontsize=12, fontweight='bold')
148 axes[1].set_xscale('log')
149 axes[1].grid(alpha=0.3)
150
151 plt.tight_layout()
152 plt.savefig('mde_analysis.pdf', dpi=300, bbox_inches='tight')
153 plt.show()

```

4.5.2 Business Significance vs. Statistical Significance

Statistical significance ($p < 0.05$) doesn't imply business significance. Sample size planning must consider minimum *practically important* differences.

Framework for determining MDE:

1. **Business value:** What's the smallest improvement worth implementing?
 - Consider implementation costs
 - Account for risks
 - Factor in opportunity costs
2. **Statistical feasibility:** Can we detect that difference with available traffic?
 - Calculate required sample size
 - Estimate experiment duration
 - Assess if timeline is acceptable
3. **Reconciliation:** If desired MDE requires impractical sample size:
 - Increase traffic allocation
 - Run experiment longer
 - Accept lower power
 - Reconsider if test is worthwhile

Example 4.5 (Business vs. Statistical Significance). E-commerce company testing checkout redesign:

Business analysis:

- Current conversion: 10%
- Average order value: \$100
- Monthly users: 100,000
- Implementation cost: \$50,000

Break-even calculation:

- Need $\$50,000 / (\$100 \times 100,000/\text{month}) = 5\%$ conversion lift to break even in 1 month
- Absolute lift: 0.5 percentage points
- This becomes minimum practically important difference

Sample size for 0.5pp lift:

- $n = 14,732$ per group (from earlier calculation)
- At 50,000 users/month per group: 9 days
- Feasible!

If implementation cost were \$500,000, break-even requires 50% lift—likely infeasible.

4.5.3 Cost-Benefit Analysis of Sample Sizes

Larger samples increase power but have costs: longer experiments, more users exposed to potentially inferior variants, delayed decisions.

Cost factors:

- **Opportunity cost:** Delayed rollout of winning variant
- **Exposure cost:** Users experiencing losing variant
- **Resource cost:** Engineering time, infrastructure

Benefit factors:

- **Reduced Type II error:** Lower risk of missing beneficial changes
- **More precise estimates:** Better understanding of effect magnitude
- **Subgroup analysis:** Sufficient power for heterogeneous effects

Optimal sample size balances these factors. Standard power = 0.80 represents conventional wisdom, but context-specific optimization may justify different choices.

4.5.4 Duration Estimation

Sample size determines experiment duration given traffic volume.

$$\text{Duration (days)} = \frac{n_{\text{total}}}{\text{users per day} \times \text{allocation fraction}} \quad (4.25)$$

Considerations:

- **Day-of-week effects:** Run full weekly cycles
- **Seasonality:** Avoid atypical periods
- **Minimum duration:** At least 1 week to capture patterns
- **Maximum duration:** Product changes, external events limit feasibility

Example 4.6 (Duration Calculation). Required sample: 20,000 per group (40,000 total)
Scenario 1: High traffic site

- Daily users: 50,000
- 50/50 split: 25,000 per group per day
- Duration: $20,000 / 25,000 = 0.8$ days
- **Decision:** Run for 1 week to capture weekly patterns

Scenario 2: Medium traffic site

- Daily users: 2,000
- 50/50 split: 1,000 per group per day
- Duration: $20,000 / 1,000 = 20$ days
- **Decision:** Acceptable, run for 3 weeks (21 days)

Scenario 3: Low traffic site

- Daily users: 200
- 50/50 split: 100 per group per day
- Duration: $20,000 / 100 = 200$ days
- **Decision:** Infeasible! Consider:
 - Accept larger MDE (smaller sample)
 - Lower power requirement
 - Don't run experiment (too uncertain)

4.6 Advanced Topics

4.6.1 Unequal Sample Sizes

Sometimes unequal allocation is optimal or necessary.

Reasons for unequal allocation:

- Limited traffic to treatment variant
- Risk mitigation (keep most users on proven control)
- Cost asymmetry (treatment is expensive)

Theorem 4.5 (Optimal Allocation Ratio). For fixed total sample size $N = n_1 + n_2$, power is maximized when:

$$\frac{n_2}{n_1} = \frac{\sigma_2}{\sigma_1} \quad (4.26)$$

For proportions with similar rates, equal allocation (1:1) is nearly optimal.

For highly unequal allocation, efficiency loss is:

$$\text{Efficiency} = \frac{4r}{(1+r)^2} \quad (4.27)$$

where $r = n_2/n_1$.

Example 4.7 (Allocation Efficiency). • 1:1 (50/50): Efficiency = 1.00 (optimal)

- 2:1 (67/33): Efficiency = 0.89 (11% loss)
- 3:1 (75/25): Efficiency = 0.75 (25% loss)
- 9:1 (90/10): Efficiency = 0.36 (64% loss!)

Practical recommendation: Avoid allocation more extreme than 2:1 unless necessary. Efficiency loss outweighs most benefits.

4.6.2 Sequential Testing Implications

Classical sample size formulas assume:

- Fixed sample size determined in advance
- Single analysis at experiment conclusion

Sequential testing (peeking at results, stopping early) inflates Type I error rate. Naïve peeking can increase α from 0.05 to 0.30!

Solutions:

- **Alpha spending:** Adjust significance thresholds for multiple looks
- **Sequential probability ratio test (SPRT):** Optimal sequential procedure
- **Group sequential design:** Pre-specified analysis points with adjusted α

Sample sizes for sequential testing are **not** determined by classical formulas. Chapter 8 covers sequential methods.

4.6.3 Multiple Metrics Considerations

Testing multiple metrics simultaneously requires adjustment for multiple comparisons.

Approaches:

1. **Primary metric only:** Design for power on single most important metric
 - Simple, clear decision rule
 - Risk: Miss effects on other metrics
2. **Family-wise error rate (FWER):** Control probability of any false positive
 - Bonferroni: Increase sample size by factor of $\sqrt{m}$ for m metrics
 - Very conservative
3. **False discovery rate (FDR):** Control expected proportion of false discoveries
 - More powerful than FWER
 - Appropriate when some false positives acceptable
4. **Guardrail metrics:** Primary metric at $\alpha = 0.05$, secondary metrics at $\alpha = 0.10$ or 0.20
 - Practical compromise
 - Flags potential issues without requiring huge samples

Practical recommendation: Design for adequate power on primary metric. Use secondary metrics as diagnostics with less stringent thresholds. See Chapter 6 for comprehensive treatment.

4.7 Implementation Tools

4.7.1 Python Implementation

We've developed comprehensive functions. Here's a unified calculator:

Listing 4.5: Comprehensive Sample Size Calculator

```

1 class ABTestCalculator:
2     """
3     Comprehensive A/B test sample size and power calculator.
4     """
5
6     def __init__(self, alpha: float = 0.05, power: float = 0.80):
7         """
8         Initialize calculator with default parameters.
9
10        Parameters:
11        -----
12        alpha: float
13        Significance level (default 0.05)
14        power: float
15        Statistical power (default 0.80)
16        """

```

```

17         self.alpha = alpha
18         self.power = power
19
20     def sample_size_proportions(self, p1: float, p2: float,
21                                ratio: float = 1.0) -> dict:
22         """Calculate sample size for proportion test."""
23         return sample_size_proportions(p1, p2, self.alpha, self.power,
24                                         ratio)
25
26     def sample_size_means(self, mu1: float, mu2: float, sigma: float,
27                           ratio: float = 1.0) -> dict:
28         """Calculate sample size for means test."""
29         return sample_size_means(mu1, mu2, sigma, self.alpha, self.
30                                 power, ratio)
31
32     def calculate_power(self, p1: float, p2: float, n: int) -> float:
33         """Calculate power for given parameters."""
34         return calculate_power_proportions(p1, p2, n, self.alpha)
35
36     def calculate_mde(self, p1: float, n: int) -> dict:
37         """Calculate minimum detectable effect."""
38         return calculate_mde(p1, n, self.alpha, self.power)
39
40     def recommend_sample_size(self, p1: float, desired_lift: float,
41                              traffic_per_day: int,
42                              max_duration_days: int = 30) -> dict:
43         """
44         Provide sample size recommendation with practical constraints.
45
46         Parameters:
47         -----
48         p1: float
49             Baseline conversion rate
50         desired_lift: float
51             Desired relative lift to detect
52         traffic_per_day: int
53             Available users per day
54         max_duration_days: int
55             Maximum acceptable experiment duration
56
57         Returns:
58         -----
59         dict: Recommendation with feasibility assessment
60         """
61         p2 = p1 * (1 + desired_lift)
62
63         # Calculate required sample size
64         result = self.sample_size_proportions(p1, p2)
65
66         # Calculate duration
67         users_per_group_per_day = traffic_per_day / 2
68         duration_days = np.ceil(result['n1'] / users_per_group_per_day)
69
70         # Feasibility assessment
71         feasible = duration_days <= max_duration_days
72
73         # If not feasible, calculate achievable MDE
74         if not feasible:

```

```

73         max_n = int(users_per_group_per_day * max_duration_days)
74         achievable_mde = self.calculate_mde(p1, max_n)
75     else:
76         achievable_mde = None
77
78     return {
79         'required_n_per_group': result['n1'],
80         'required_total_n': result['n_total'],
81         'duration_days': int(duration_days),
82         'duration_weeks': int(np.ceil(duration_days / 7)),
83         'feasible': feasible,
84         'achievable_mde': achievable_mde,
85         'recommendation': self._generate_recommendation(
86             feasible, duration_days, max_duration_days,
87             achievable_mde
88         ),
89         'details': result
90     }
91
92     def _generate_recommendation(self, feasible: bool, duration: float,
93                                 max_duration: float, achievable_mde:
94                                     dict) -> str:
95         """Generate human-readable recommendation."""
96         if feasible:
97             return (f"Experiment is FEASIBLE. Run for {int(np.ceil(
98                 duration/7))} weeks"
99                 f"({int(duration)} days) to achieve desired power.")
100         else:
101             if achievable_mde:
102                 return (f"Experiment NOT FEASIBLE within {max_duration}
103                     days."
104                     f"Can only detect {achievable_mde['mde_relative']:.1%} lift"
105                     f"with available traffic. Consider: (1) Accepting larger MDE,"
106                     f"(2) Running longer, (3) Increasing traffic allocation,"
107                     f"(4) Not running experiment.")
108             else:
109                 return "Experiment NOT FEASIBLE. Insufficient traffic."
110
111 # Example: Comprehensive analysis
112 print("\n" + "="*80)
113 print("Comprehensive A/B Test Sample Size Analysis")
114 print("="*80 + "\n")
115
116 # Initialize calculator
117 calc = ABTestCalculator(alpha=0.05, power=0.80)
118
119 # Scenario: E-commerce checkout optimization
120 print("Scenario: E-commerce Checkout Redesign")
121 print("-"*80)
122 print("Current conversion rate: 10%")
123 print("Desired lift to detect: 10% (relative)")
124 print("Daily traffic: 10,000 users")
125 print("Maximum acceptable duration: 30 days")
126 print()

```

```

124
125 recommendation = calc.recommend_sample_size(
126     p1=0.10,
127     desired_lift=0.10,
128     traffic_per_day=10000,
129     max_duration_days=30
130 )
131
132 print("ANALYSIS RESULTS:")
133 print("-"*80)
134 print(f"Required sample size: {recommendation['required_n_per_group']:,} per group")
135 print(f"Total required: {recommendation['required_total_n']:,} users")
136 print(f"Estimated duration: {recommendation['duration_days']} days ({recommendation['duration_weeks']} weeks)")
137 print(f"Feasibility: {'✓ FEASIBLE' if recommendation['feasible'] else '✗ NOT FEASIBLE'}")
138 print()
139 print("RECOMMENDATION:")
140 print("-"*80)
141 print(recommendation['recommendation'])
142 print()
143
144 # Additional scenarios
145 print("\nComparing Different Traffic Levels:")
146 print("-"*80)
147
148 for daily_traffic in [1000, 5000, 10000, 50000]:
149     rec = calc.recommend_sample_size(
150         p1=0.10,
151         desired_lift=0.10,
152         traffic_per_day=daily_traffic,
153         max_duration_days=30
154     )
155
156     print(f"\nDaily traffic: {daily_traffic:,}")
157     print(f"Duration: {rec['duration_days']} days")
158     print(f"Feasible: {'Yes' if rec['feasible'] else 'No'}")
159     if not rec['feasible'] and rec['achievable_mde']:
160         print(f"Achievable MDE: {rec['achievable_mde']['mde_relative']:.1%}")

```

4.7.2 R Implementation

R provides excellent power analysis tools through the `pwr` package.

Listing 4.6: Sample Size Calculation in R

```

1 # Install and load required packages
2 install.packages("pwr")
3 library(pwr)
4
5 # 1. Sample size for two proportions
6 # Testing 10% vs 11% conversion
7 p1 <- 0.10
8 p2 <- 0.11
9

```

```

10 # Calculate effect size (Cohen's h)
11 h <- ES.h(p1, p2)
12 cat(sprintf("Cohen's h: %.4f\n", h))
13
14 # Calculate required sample size
15 power_result <- pwr.2p.test(
16   h = h,
17   sig.level = 0.05,
18   power = 0.80,
19   alternative = "two.sided"
20 )
21
22 print(power_result)
23 cat(sprintf("\nRequired n per group: %.0f\n", ceiling(power_result$n)))
24 cat(sprintf("Total n: %.0f\n", ceiling(power_result$n * 2)))
25
26 # 2. Sample size for means
27 # Testing $50 vs $55 with SD = $30
28 mu1 <- 50
29 mu2 <- 55
30 sigma <- 30
31
32 # Cohen's d
33 d <- (mu2 - mu1) / sigma
34 cat(sprintf("\nCohen's d: %.4f\n", d))
35
36 # Calculate required sample size
37 power_result_means <- pwr.t.test(
38   d = d,
39   sig.level = 0.05,
40   power = 0.80,
41   type = "two.sample",
42   alternative = "two.sided"
43 )
44
45 print(power_result_means)
46
47 # 3. Power curve
48 p1_vals <- seq(0.05, 0.30, 0.05)
49 lift <- 0.10
50 sample_sizes <- c(500, 1000, 2000, 5000)
51
52 power_matrix <- matrix(NA, nrow = length(p1_vals), ncol = length(sample
53   _sizes))
54 colnames(power_matrix) <- paste0("n=", sample_sizes)
55 rownames(power_matrix) <- paste0("p1=", p1_vals)
56
57 for (i in seq_along(p1_vals)) {
58   p1 <- p1_vals[i]
59   p2 <- p1 * (1 + lift)
60   h <- ES.h(p1, p2)
61
62   for (j in seq_along(sample_sizes)) {
63     n <- sample_sizes[j]
64     result <- pwr.2p.test(
65       h = h,
66       n = n,
67       sig.level = 0.05,

```

```

67     alternative = "two.sided"
68   )
69   power_matrix[i, j] <- result$power
70 }
71 }
72
73 print(round(power_matrix, 3))
74
75 # 4. Plot power curve
76 plot_power_curve <- function(p1, lift_range, n, alpha = 0.05) {
77   powers <- numeric(length(lift_range))
78
79   for (i in seq_along(lift_range)) {
80     p2 <- p1 * (1 + lift_range[i])
81     h <- ES.h(p1, p2)
82     result <- pwr.2p.test(h = h, n = n, sig.level = alpha)
83     powers[i] <- result$power
84   }
85
86   plot(lift_range * 100, powers,
87        type = "l", lwd = 2,
88        xlab = "Relative Lift (%)",
89        ylab = "Statistical Power",
90        main = sprintf("Power Curve (baseline = %.1f%%, n = %d)", p1 *
91                        100, n))
92   abline(h = 0.80, col = "red", lty = 2)
93   grid()
94 }
95
96 # Generate power curve
97 lift_range <- seq(0.05, 0.50, 0.01)
98 plot_power_curve(p1 = 0.10, lift_range = lift_range, n = 5000)

```

4.8 Summary

Sample size determination transforms experimental planning from guesswork to rigorous calculation:

- **Power analysis** connects sample size, effect size, significance level, and power. Specifying any three determines the fourth.
- **Standard conventions** ($\alpha = 0.05$, power = 0.80) represent reasonable defaults but should be adjusted for context.
- **Sample size formulas** differ by metric type. Proportions and means have closed-form solutions; other metrics require simulation or approximation.
- **Minimum detectable effect** (MDE) defines the smallest effect reliably detectable. MDE must be both statistically feasible and business-relevant.
- **Practical constraints**—traffic volume, acceptable duration, implementation costs—often dominate pure statistical considerations.
- **Advanced topics**—unequal allocation, sequential testing, multiple metrics—require specialized methods beyond classical formulas.

- **Implementation tools** in Python and R enable rapid calculation and sensitivity analysis.

Proper sample size calculation ensures experiments are neither underpowered (wasting time and resources) nor overpowered (unnecessarily delaying decisions). Master these techniques to design efficient, informative experiments.

4.9 Further Reading

- Cohen [1988]: Definitive reference on statistical power analysis
- Chow et al. [2007]: Sample size methods for clinical research
- Kohavi et al. [2020]: Practical guidance for online experiments
- Champely [2020]: Documentation for R's pwr package

4.10 Exercises

4.10.1 Conceptual Questions

1. Explain the relationship between Type I error, Type II error, power, and significance level. How are they connected?
2. Why is $\text{power} = 0.80$ a common standard? Under what circumstances would you use $\text{power} = 0.90$? $\text{Power} = 0.70$?
3. A colleague says: "Let's just run the test for a month and see what happens." What's wrong with this approach?
4. Explain why doubling sample size doesn't double statistical power. What's the actual relationship?
5. Describe three scenarios where unequal allocation (e.g., 75/25 split) might be justified despite efficiency loss.
6. What's the difference between minimum detectable effect and minimum practically important difference? Why do both matter?

4.10.2 Calculation Exercises

1. Calculate required sample size for testing:
 - Baseline conversion: 5%
 - Target conversion: 6% (20% relative lift)
 - $\alpha = 0.05$, $\text{power} = 0.80$

Show all work. How long would this experiment take with 10,000 users/day?

2. Repeat previous calculation for $\text{power} = 0.90$. By what factor does required sample size increase?

3. Given 3,000 users per group available, baseline conversion 15%, calculate the minimum detectable effect (MDE) for $\alpha = 0.05$, power = 0.80.
4. Calculate required sample size for continuous metric:
 - Control mean: \$75, SD: \$40
 - Target mean: \$85 (13.3% increase)
 - $\alpha = 0.05$, power = 0.80
5. Compare sample size requirements for detecting 10% relative lift at baseline rates of 2%, 10%, 30%, and 50%. What pattern emerges?

4.10.3 Programming Exercises

1. Implement a comprehensive power analysis function that:
 - Accepts multiple input formats (absolute difference, relative lift, Cohen's h)
 - Supports both one-sided and two-sided tests
 - Handles unequal allocation ratios
 - Provides clear warnings about assumptions
 - Includes input validation
2. Create an interactive sample size calculator using Streamlit or Shiny that:
 - Takes baseline metrics and desired lift as input
 - Shows power curves dynamically
 - Estimates experiment duration given traffic
 - Provides feasibility recommendations
 - Exports results as PDF report
3. Write a function to perform sensitivity analysis:
 - How does power change if true effect is 50% smaller/larger than assumed?
 - How does power change if variance is 20% higher than expected?
 - Generate visualization showing sensitivity regions
4. Implement simulation-based power calculation for:
 - Ratio metrics (revenue per user)
 - Count metrics (purchases per customer)
 - Mixture distributions (users who convert vs. don't convert)

Compare simulation results to closed-form approximations.

4.10.4 Applied Exercises

1. Design experiments for these scenarios with full justification:

Scenario A: SaaS company testing onboarding flow

- Current trial-to-paid conversion: 8%
- 500 new trials per week
- Implementation cost: \$100,000
- Average customer lifetime value: \$5,000

Scenario B: News website testing article layout

- Current average time on page: 2.5 minutes (SD: 3.0 minutes)
- 1 million pageviews per day
- Easy to implement and reverse

Scenario C: Mobile app testing premium feature

- Current premium conversion: 2%
- 10,000 daily active users
- Major engineering effort (\$500K)

For each: Specify desired lift, calculate sample size, estimate duration, assess feasibility, provide recommendation.

2. A stakeholder insists on detecting 2% relative lift with 95% power. Your analysis shows this requires 100,000 users per group but only 50,000 total are available. Prepare a memo explaining:
 - Why their requirement is infeasible
 - What can be achieved with available sample
 - Alternative approaches (sequential testing, focus on larger effects, etc.)
 - Recommended path forward
3. Review a past A/B test that failed to reach significance. Using actual baseline metrics and observed difference:
 - Calculate what power the experiment actually had
 - Determine what sample size would have been needed for 80% power
 - Assess whether observed effect was below MDE
 - Recommend whether to rerun with larger sample or abandon

4.10.5 Advanced Projects

1. **Sequential testing sample size:** Implement group sequential design with:
 - O'Brien-Fleming spending function
 - 3-5 planned interim analyses
 - Calculate required sample size accounting for multiple looks
 - Compare to fixed sample size
2. **Multiple metrics:** Design experiment testing 1 primary and 4 secondary metrics:
 - Determine sample size for each metric individually
 - Apply Bonferroni, Holm, and FDR corrections
 - Compare required sample sizes
 - Recommend practical approach
3. **Variance reduction impact:** Simulate experiments with and without CUPED variance reduction:
 - Generate correlated pre/post data
 - Compare power with/without covariate adjustment
 - Quantify sample size savings
 - Visualize power gains
4. **Bayesian sample size determination:** Implement Bayesian approach:
 - Specify prior distribution for effect size
 - Calculate probability of success for various sample sizes
 - Compare to frequentist power analysis
 - Discuss advantages and limitations

Chapter 5

Hypothesis Testing for A/B Tests

5.1 Learning Objectives

After completing this chapter, readers will be able to:

- Apply classical hypothesis testing frameworks to A/B tests
- Select appropriate statistical tests for different metric types
- Check assumptions and diagnose violations
- Implement non-parametric alternatives when assumptions fail
- Interpret test results correctly and avoid common pitfalls
- Use decision frameworks for systematic test selection

5.2 Classical Hypothesis Testing Framework

Hypothesis testing provides a formal procedure for making decisions under uncertainty. While introduced earlier, this chapter provides rigorous treatment and practical implementation.

5.2.1 Structure of a Hypothesis Test

1. **State hypotheses:**

- Null hypothesis H_0 : No effect or no difference
- Alternative hypothesis H_1 : An effect exists

2. **Choose significance level:** α (typically 0.05 or 0.01)

3. **Calculate test statistic:** Function of data measuring evidence against H_0

4. **Determine p-value:** Probability of observing data this extreme under H_0

5. **Make decision:**

- Reject H_0 if p-value $< \alpha$

- Fail to reject H_0 otherwise

6. **Interpret in context:** Translate statistical conclusion to practical meaning

5.2.2 One-Sided vs. Two-Sided Tests

Two-sided test:

$$H_0 : \theta_1 = \theta_2 \quad \text{vs.} \quad H_1 : \theta_1 \neq \theta_2 \quad (5.1)$$

Tests for any difference (positive or negative). Most common in A/B testing.

One-sided test:

$$H_0 : \theta_1 \geq \theta_2 \quad \text{vs.} \quad H_1 : \theta_1 < \theta_2 \quad (5.2)$$

Tests specifically for improvement (or degradation). More powerful but only detects effects in specified direction.

Theorem 5.1 (Relationship Between One-Sided and Two-Sided Tests). For the same data and test statistic, the one-sided p-value equals half the two-sided p-value (when the effect is in the hypothesized direction).

If $p_{\text{two-sided}} = 2P(|Z| > |z_{\text{obs}}|)$, then:

$$p_{\text{one-sided}} = P(Z > z_{\text{obs}}) = \frac{p_{\text{two-sided}}}{2} \quad (5.3)$$

When to use one-sided tests:

- Only care about improvements (would not implement if treatment is worse)
- Regulatory or business constraints dictate directionality
- Pre-registered directional hypothesis
- Strong theoretical reason to expect one direction only

Caution: One-sided tests are often misused. Default to two-sided unless strong justification exists.

5.3 Tests for Proportions

Proportion tests are fundamental for A/B testing of conversion rates, click-through rates, and other binary outcomes.

5.3.1 Two-Sample Z-Test for Proportions

For control proportion $\hat{p}_1 = x_1/n_1$ and treatment proportion $\hat{p}_2 = x_2/n_2$:

$$H_0 : p_1 = p_2 \quad \text{vs.} \quad H_1 : p_1 \neq p_2 \quad (5.4)$$

Under H_0 , we use the pooled proportion:

$$\hat{p}_{\text{pooled}} = \frac{x_1 + x_2}{n_1 + n_2} \quad (5.5)$$

Test statistic:

$$Z = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{\hat{p}_{\text{pooled}}(1 - \hat{p}_{\text{pooled}})(1/n_1 + 1/n_2)}} \quad (5.6)$$

Under H_0 : $Z \sim \mathcal{N}(0, 1)$ approximately (by CLT for large samples).

P-value (two-sided):

$$p = 2(1 - \Phi(|Z|)) \quad (5.7)$$

where Φ is the standard normal CDF.

Decision rule: Reject H_0 if $|Z| > z_{\alpha/2}$ (equivalently, if $p < \alpha$).

5.3.2 Confidence Interval for Difference in Proportions

For confidence interval, use unpooled variance:

$$\text{SE}_{\text{unpooled}} = \sqrt{\frac{\hat{p}_1(1 - \hat{p}_1)}{n_1} + \frac{\hat{p}_2(1 - \hat{p}_2)}{n_2}} \quad (5.8)$$

$(1 - \alpha)$ confidence interval:

$$(\hat{p}_2 - \hat{p}_1) \pm z_{\alpha/2} \cdot \text{SE}_{\text{unpooled}} \quad (5.9)$$

Example 5.1 (Conversion Rate Test).

Control: $n_1 = 2000, x_1 = 160, \hat{p}_1 = 0.080$

Treatment: $n_2 = 2000, x_2 = 190, \hat{p}_2 = 0.095$

Pooled proportion:

$$\hat{p}_{\text{pooled}} = \frac{160 + 190}{2000 + 2000} = \frac{350}{4000} = 0.0875 \quad (5.10)$$

Standard error:

$$\text{SE} = \sqrt{0.0875 \times 0.9125 \times (1/2000 + 1/2000)} = 0.00894 \quad (5.11)$$

Test statistic:

$$Z = \frac{0.095 - 0.080}{0.00894} = 1.68 \quad (5.12)$$

P-value: $p = 2(1 - \Phi(1.68)) = 2(1 - 0.9535) = 0.093$

At $\alpha = 0.05$, we fail to reject H_0 . The observed difference is not statistically significant.

Listing 5.1: Comprehensive Z-Test Implementation for Proportions

```

1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4 from typing import Dict, Tuple
5 import pandas as pd
6
7 def z_test_proportions(
8     x1: int,
9     n1: int,
```

```

10     x2: int,
11     n2: int,
12     alpha: float = 0.05,
13     alternative: str = 'two-sided'
14 ) -> Dict:
15     """
16     Perform z-test for comparing two proportions.
17
18     Parameters:
19     -----
20     x1, x2: int
21     Number of successes in each group
22     n1, n2: int
23     Sample sizes
24     alpha: float
25     Significance level
26     alternative: str
27     'two-sided', 'greater', or 'less'
28
29     Returns:
30     -----
31     dict: Test results including statistic, p-value, CI, decision
32     """
33     # Sample proportions
34     p1_hat = x1 / n1
35     p2_hat = x2 / n2
36     diff = p2_hat - p1_hat
37
38     # Pooled proportion under null hypothesis
39     p_pooled = (x1 + x2) / (n1 + n2)
40
41     # Standard error under null (pooled)
42     se_pooled = np.sqrt(p_pooled * (1 - p_pooled) * (1/n1 + 1/n2))
43
44     # Test statistic
45     z_stat = diff / se_pooled
46
47     # P-value
48     if alternative == 'two-sided':
49         p_value = 2 * (1 - stats.norm.cdf(abs(z_stat)))
50     elif alternative == 'greater':
51         p_value = 1 - stats.norm.cdf(z_stat)
52     elif alternative == 'less':
53         p_value = stats.norm.cdf(z_stat)
54     else:
55         raise ValueError("alternative must be 'two-sided', 'greater', or 'less'")
56
57     # Decision
58     reject_null = p_value < alpha
59
60     # Confidence interval (using unpooled variance)
61     se_unpooled = np.sqrt(p1_hat*(1-p1_hat)/n1 + p2_hat*(1-p2_hat)/n2)
62
63     if alternative == 'two-sided':
64         z_crit = stats.norm.ppf(1 - alpha/2)
65         ci_lower = diff - z_crit * se_unpooled
66         ci_upper = diff + z_crit * se_unpooled

```

```

67 elif alternative == 'greater':
68     z_crit = stats.norm.ppf(1 - alpha)
69     ci_lower = diff - z_crit * se_unpooled
70     ci_upper = np.inf
71 else: # less
72     z_crit = stats.norm.ppf(1 - alpha)
73     ci_lower = -np.inf
74     ci_upper = diff + z_crit * se_unpooled
75
76 # Effect size (relative lift)
77 relative_lift = diff / p1_hat if p1_hat > 0 else np.nan
78
79 return {
80     'test': 'Z-test_for_proportions',
81     'p1_hat': p1_hat,
82     'p2_hat': p2_hat,
83     'difference': diff,
84     'relative_lift': relative_lift,
85     'z_statistic': z_stat,
86     'p_value': p_value,
87     'alpha': alpha,
88     'reject_null': reject_null,
89     'ci_lower': ci_lower,
90     'ci_upper': ci_upper,
91     'se_pooled': se_pooled,
92     'se_unpooled': se_unpooled,
93     'alternative': alternative,
94     'decision': 'Reject_H0' if reject_null else 'Fail_to_reject_H0'
95 }
96
97 def visualize_proportion_test(result: Dict, x1: int, n1: int, x2: int,
98 n2: int):
99     """Visualize_proportion_test_results."""
100     fig, axes = plt.subplots(1, 3, figsize=(15, 5))
101
102     # 1. Proportions with CI
103     ax = axes[0]
104     props = [result['p1_hat'], result['p2_hat']]
105     labels = ['Control', 'Treatment']
106     colors = ['C0', 'C1']
107
108     # Standard errors for individual proportions
109     se1 = np.sqrt(result['p1_hat'] * (1 - result['p1_hat']) / n1)
110     se2 = np.sqrt(result['p2_hat'] * (1 - result['p2_hat']) / n2)
111     errors = [1.96 * se1, 1.96 * se2]
112
113     x_pos = np.arange(2)
114     bars = ax.bar(x_pos, props, yerr=errors, capsize=5, alpha=0.7,
115 color=colors)
116     ax.set_xticks(x_pos)
117     ax.set_xticklabels(labels)
118     ax.set_ylabel('Conversion_Rate')
119     ax.set_title('Conversion_Rates_with_95%_CI')
120     ax.grid(axis='y', alpha=0.3)
121
122     # Add values on bars
123     for i, (bar, prop) in enumerate(zip(bars, props)):
124         height = bar.get_height()

```

```

123     ax.text(bar.get_x() + bar.get_width()/2., height + errors[i] +
124             0.01,
125             f'{prop:.3f}', ha='center', va='bottom', fontweight='
126             bold')
127
128 # 2. Difference with CI
129 ax = axes[1]
130 diff = result['difference']
131 ci_lower = result['ci_lower']
132 ci_upper = result['ci_upper']
133
134 ax.barh([0], [diff], xerr=[[diff - ci_lower], [ci_upper - diff]],
135         capsize=5, alpha=0.7, color='green' if diff > 0 else 'red')
136 ax.axvline(x=0, color='black', linestyle='--', linewidth=1)
137 ax.set_yticks([])
138 ax.set_xlabel('Difference in Proportions')
139 ax.set_title(f'Difference: {diff:.4f} [{ci_lower:.4f}, {ci_upper:.4f}]')
140 ax.grid(axis='x', alpha=0.3)
141
142 # Add significance marker
143 if result['reject_null']:
144     ax.text(diff, 0.3, 'Significant', ha='center', fontsize=12,
145             fontweight='bold', color='darkgreen')
146 else:
147     ax.text(diff, 0.3, 'Not Significant', ha='center', fontsize=12,
148             fontweight='bold', color='gray')
149
150 # 3. Null distribution with test statistic
151 ax = axes[2]
152 x = np.linspace(-4, 4, 1000)
153 y = stats.norm.pdf(x, 0, 1)
154 ax.plot(x, y, 'b-', linewidth=2, label='Null distribution N(0,1)')
155 ax.axvline(x=result['z_statistic'], color='red', linestyle='--',
156           linewidth=2, label=f'Observed z = {result["z_statistic"]:.2f}')
157
158 # Shade rejection region
159 if result['alternative'] == 'two-sided':
160     z_crit = stats.norm.ppf(1 - result['alpha']/2)
161     x_left = x[x <= -z_crit]
162     x_right = x[x >= z_crit]
163     ax.fill_between(x_left, 0, stats.norm.pdf(x_left, 0, 1),
164                    alpha=0.3, color='red')
165     ax.fill_between(x_right, 0, stats.norm.pdf(x_right, 0, 1),
166                    alpha=0.3, color='red', label=f'Rejection
167                    region (α={result["alpha"]})')
168     ax.axvline(x=-z_crit, color='red', linestyle=':', alpha=0.5)
169     ax.axvline(x=z_crit, color='red', linestyle=':', alpha=0.5)
170
171 ax.set_xlabel('Z-statistic')
172 ax.set_ylabel('Density')
173 ax.set_title(f'Null Distribution (p-value = {result["p_value"]:.4f})')
174 ax.legend()
175 ax.grid(alpha=0.3)
176
177 plt.tight_layout()

```



```

175     return fig
176
177 # Example usage
178 print("=== Z-Test for Proportions ===\n")
179
180 # Example 1: Non-significant result
181 result1 = z_test_proportions(x1=160, n1=2000, x2=190, n2=2000)
182
183 print("Example 1: Marginal effect")
184 print(f"Control: {result1['p1_hat']:.3f} ({160}/{2000})")
185 print(f"Treatment: {result1['p2_hat']:.3f} ({190}/{2000})")
186 print(f"Difference: {result1['difference']:.4f}")
187 print(f"Relative lift: {result1['relative_lift']*100:.1f}%")
188 print(f"Z-statistic: {result1['z_statistic']:.3f}")
189 print(f"P-value: {result1['p_value']:.4f}")
190 print(f"95% CI: [{result1['ci_lower']:.4f}, {result1['ci_upper']:.4f}] ")
191 )
192 print(f"Decision: {result1['decision']}\n")
193
194 # Example 2: Significant result
195 result2 = z_test_proportions(x1=200, n1=2000, x2=260, n2=2000)
196
197 print("Example 2: Strong effect")
198 print(f"Control: {result2['p1_hat']:.3f} ({200}/{2000})")
199 print(f"Treatment: {result2['p2_hat']:.3f} ({260}/{2000})")
200 print(f"Difference: {result2['difference']:.4f}")
201 print(f"Relative lift: {result2['relative_lift']*100:.1f}%")
202 print(f"Z-statistic: {result2['z_statistic']:.3f}")
203 print(f"P-value: {result2['p_value']:.4e}")
204 print(f"95% CI: [{result2['ci_lower']:.4f}, {result2['ci_upper']:.4f}] ")
205 )
206 print(f"Decision: {result2['decision']}\n")
207
208 # Example 3: One-sided test
209 result3 = z_test_proportions(x1=200, n1=2000, x2=260, n2=2000,
210                             alternative='greater')
211
212 print("Example 3: One-sided test (greater)")
213 print(f"Z-statistic: {result3['z_statistic']:.3f}")
214 print(f"P-value (one-sided): {result3['p_value']:.4e}")
215 print(f"P-value (two-sided): {result2['p_value']:.4e}")
216 print(f"Ratio: {result2['p_value']/result3['p_value']:.1f}\n")
217
218 # Visualize
219 fig1 = visualize_proportion_test(result1, 160, 2000, 190, 2000)
220 plt.savefig('proportion_test_marginal.pdf')
221
222 fig2 = visualize_proportion_test(result2, 200, 2000, 260, 2000)
223 plt.savefig('proportion_test_strong.pdf')
224
225 plt.show()

```

5.3.3 Assumptions for Z-Test

The z-test for proportions relies on:

1. **Independence:** Observations are independent within and between groups

2. **Random sampling:** Groups formed by randomization

3. **Large samples:** Normal approximation valid

Rule of thumb: $n_i p_i > 5$ and $n_i(1 - p_i) > 5$ for $i = 1, 2$.

More conservative: $n_i p_i > 10$ and $n_i(1 - p_i) > 10$.

When assumptions are violated, consider:

- Fisher's exact test for small samples
- Permutation tests (robust to distributional assumptions)
- Adjustments for clustering or correlation

5.4 Tests for Means

When comparing continuous outcomes (revenue, time on site, engagement metrics), we use t-tests.

5.4.1 Two-Sample T-Test

For means $\bar{X}_1$ and $\bar{X}_2$ from groups with variances s_1^2 and s_2^2 :

$$H_0 : \mu_1 = \mu_2 \quad \text{vs.} \quad H_1 : \mu_1 \neq \mu_2 \quad (5.13)$$

Pooled (Student's) t-test (assumes equal variances):

Pooled variance:

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2} \quad (5.14)$$

Test statistic:

$$t = \frac{\bar{X}_2 - \bar{X}_1}{s_p \sqrt{1/n_1 + 1/n_2}} \quad (5.15)$$

Distribution under H_0 : $t \sim t_{n_1+n_2-2}$

Welch's t-test (unequal variances):

Test statistic:

$$t = \frac{\bar{X}_2 - \bar{X}_1}{\sqrt{s_1^2/n_1 + s_2^2/n_2}} \quad (5.16)$$

Degrees of freedom (Welch-Satterthwaite approximation):

$$\nu = \frac{(s_1^2/n_1 + s_2^2/n_2)^2}{\frac{(s_1^2/n_1)^2}{n_1-1} + \frac{(s_2^2/n_2)^2}{n_2-1}} \quad (5.17)$$

Theorem 5.2 (Robustness of Welch's t-test). Welch's t-test maintains correct Type I error rate even when:

- Variances are unequal
- Sample sizes are unequal
- Both conditions hold simultaneously

The pooled t-test can have inflated Type I error when $\sigma_1 \neq \sigma_2$ and $n_1 \neq n_2$.

Practical recommendation: Default to Welch's t-test. It performs nearly as well as pooled t-test when variances are equal, and much better when they're not.

Listing 5.2: Comprehensive T-Test Implementation

```

1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4 from typing import Dict
5 import pandas as pd
6
7 def t_test_means(
8     data1: np.ndarray,
9     data2: np.ndarray,
10    alpha: float = 0.05,
11    equal_var: bool = False,
12    alternative: str = 'two-sided'
13 ) -> Dict:
14     """
15     Perform t-test for comparing two means.
16
17     Parameters:
18     -----
19     data1, data2: np.ndarray
20         Data for each group
21     alpha: float
22         Significance level
23     equal_var: bool
24         If True, use pooled t-test. If False, use Welch's t-test
25     alternative: str
26         'two-sided', 'greater', or 'less'
27
28     Returns:
29     -----
30     dict: Test results
31     """
32     n1, n2 = len(data1), len(data2)
33     mean1, mean2 = np.mean(data1), np.mean(data2)
34     var1, var2 = np.var(data1, ddof=1), np.var(data2, ddof=1)
35     std1, std2 = np.std(data1, ddof=1), np.std(data2, ddof=1)
36
37     diff = mean2 - mean1
38
39     if equal_var:
40         # Pooled t-test
41         pooled_var = ((n1 - 1) * var1 + (n2 - 1) * var2) / (n1 + n2 - 2)
42         se = np.sqrt(pooled_var * (1/n1 + 1/n2))
43         df = n1 + n2 - 2
44         test_type = "Pooled t-test"
45     else:
46         # Welch's t-test
47         se = np.sqrt(var1/n1 + var2/n2)
48         # Welch-Satterthwaite degrees of freedom
49         df = (var1/n1 + var2/n2)**2 / (
50             (var1/n1)**2/(n1-1) + (var2/n2)**2/(n2-1)
51         )

```

```

52     test_type = "Welch's_t-test"
53
54     # Test statistic
55     t_stat = diff / se
56
57     # P-value
58     if alternative == 'two-sided':
59         p_value = 2 * (1 - stats.t.cdf(abs(t_stat), df))
60     elif alternative == 'greater':
61         p_value = 1 - stats.t.cdf(t_stat, df)
62     elif alternative == 'less':
63         p_value = stats.t.cdf(t_stat, df)
64     else:
65         raise ValueError("alternative must be 'two-sided', 'greater', or 'less'")
66
67     # Decision
68     reject_null = p_value < alpha
69
70     # Confidence interval
71     if alternative == 'two-sided':
72         t_crit = stats.t.ppf(1 - alpha/2, df)
73         ci_lower = diff - t_crit * se
74         ci_upper = diff + t_crit * se
75     elif alternative == 'greater':
76         t_crit = stats.t.ppf(1 - alpha, df)
77         ci_lower = diff - t_crit * se
78         ci_upper = np.inf
79     else: # less
80         t_crit = stats.t.ppf(1 - alpha, df)
81         ci_lower = -np.inf
82         ci_upper = diff + t_crit * se
83
84     # Effect size (Cohen's d)
85     if equal_var:
86         pooled_std = np.sqrt(pooled_var)
87     else:
88         # Use pooled std even for Welch's when calculating Cohen's d
89         pooled_std = np.sqrt(((n1-1)*var1 + (n2-1)*var2) / (n1 + n2 - 2))
90
91     cohens_d = diff / pooled_std
92
93     # Relative effect (if mean1 > 0)
94     relative_diff = diff / mean1 if mean1 != 0 else np.nan
95
96     return {
97         'test': test_type,
98         'n1': n1,
99         'n2': n2,
100         'mean1': mean1,
101         'mean2': mean2,
102         'std1': std1,
103         'std2': std2,
104         'difference': diff,
105         'relative_diff': relative_diff,
106         't_statistic': t_stat,
107         'df': df,

```

```

108     'p_value': p_value,
109     'alpha': alpha,
110     'reject_null': reject_null,
111     'ci_lower': ci_lower,
112     'ci_upper': ci_upper,
113     'se': se,
114     'cohens_d': cohens_d,
115     'alternative': alternative,
116     'decision': 'Reject_H0' if reject_null else 'Fail_to_reject_H0'
117 }
118
119 def compare_t_test_variants(data1: np.ndarray, data2: np.ndarray, alpha
: float = 0.05):
120     """Compare_pooled_vs._Welch's_t-test."""
121
122     pooled = t_test_means(data1, data2, alpha=alpha, equal_var=True)
123     welch = t_test_means(data1, data2, alpha=alpha, equal_var=False)
124
125     print("===_Comparison:_Pooled_vs._Welch's_t-test_===\n")
126
127     print(f"Sample_sizes:_n1={len(data1)},_n2={len(data2)}")
128     print(f"Means:_{np.mean(data1):.2f},_{np.mean(data2):.2f}")
129     print(f"Std_devs:_{np.std(data1,_ddof=1):.2f},_{np.std(data2,_ddof
=1):.2f}")
130     print(f"Variance_ratio:_{np.var(data2,_ddof=1)}_/_{np.var(data1,_ddof
=1):.2f}\n")
131
132     comparison = pd.DataFrame({
133         'Pooled_t-test': [
134             pooled['t_statistic'],
135             pooled['df'],
136             pooled['p_value'],
137             pooled['ci_lower'],
138             pooled['ci_upper'],
139             pooled['decision']
140         ],
141         'Welch\'s_t-test': [
142             welch['t_statistic'],
143             welch['df'],
144             welch['p_value'],
145             welch['ci_lower'],
146             welch['ci_upper'],
147             welch['decision']
148         ]
149     }, index=['t-statistic', 'df', 'p-value', 'CI_lower', 'CI_upper', '
Decision'])
150
151     print(comparison)
152     print()
153
154     return pooled, welch
155
156 def visualize_t_test(result: Dict, data1: np.ndarray, data2: np.ndarray
):
157     """Visualize_t-test_results."""
158     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
159
160     # 1. Distributions

```

```

161 ax = axes[0, 0]
162 ax.hist(data1, bins=30, alpha=0.6, label='Control', density=True,
163         color='C0')
164 ax.hist(data2, bins=30, alpha=0.6, label='Treatment', density=True,
165         color='C1')
166 ax.axvline(result['mean1'], color='C0', linestyle='--', linewidth
167           =2,
168           label=f'Control_mean={result["mean1"]:.2f}')
169 ax.axvline(result['mean2'], color='C1', linestyle='--', linewidth
170           =2,
171           label=f'Treatment_mean={result["mean2"]:.2f}')
172 ax.set_xlabel('Value')
173 ax.set_ylabel('Density')
174 ax.set_title('Data_Distributions')
175 ax.legend()
176 ax.grid(alpha=0.3)
177
178 # 2. Means with CI
179 ax = axes[0, 1]
180 means = [result['mean1'], result['mean2']]
181 sems = [result['std1']/np.sqrt(result['n1']),
182         result['std2']/np.sqrt(result['n2'])]
183 labels = ['Control', 'Treatment']
184
185 x_pos = np.arange(2)
186 bars = ax.bar(x_pos, means, yerr=[1.96*s for s in sems],
187             capsize=5, alpha=0.7, color=['C0', 'C1'])
188 ax.set_xticks(x_pos)
189 ax.set_xticklabels(labels)
190 ax.set_ylabel('Mean_Value')
191 ax.set_title('Means_with_95%_CI')
192 ax.grid(axis='y', alpha=0.3)
193
194 # Add values
195 for i, (bar, mean) in enumerate(zip(bars, means)):
196     height = bar.get_height()
197     ax.text(bar.get_x() + bar.get_width()/2., height + sems[i]*2,
198           f'{mean:.2f}', ha='center', va='bottom', fontweight='
199           bold')
200
201 # 3. Difference with CI
202 ax = axes[1, 0]
203 diff = result['difference']
204 ci_lower = result['ci_lower']
205 ci_upper = result['ci_upper']
206
207 ax.barh([0], [diff], xerr=[[diff - ci_lower], [ci_upper - diff]],
208         capsize=5, alpha=0.7, color='green' if diff > 0 else 'red')
209 ax.axvline(x=0, color='black', linestyle='--', linewidth=1)
210 ax.set_yticks([])
211 ax.set_xlabel('Difference_in_Means')
212 ax.set_title(f'{result["test"]}\nDifference: {diff:.2f}
213             f'[{ci_lower:.2f}, {ci_upper:.2f}]')
214 ax.grid(axis='x', alpha=0.3)
215
216 # Add significance and effect size
217 if result['reject_null']:
218     sig_text = 'Significant'

```

```

214     color = 'darkgreen'
215 else:
216     sig_text = 'Not_Significant'
217     color = 'gray'
218
219 ax.text(diff, 0.3, f'{sig_text}\nCohen\'s_d={result["cohens_d"]:.3f}',
220         ha='center', fontsize=10, fontweight='bold', color=color)
221
222 # 4. T-distribution with test statistic
223 ax = axes[1, 1]
224 df = result['df']
225 x = np.linspace(-5, 5, 1000)
226 y = stats.t.pdf(x, df)
227 ax.plot(x, y, 'b-', linewidth=2, label=f't-distribution_(df={df:.1f})')
228 ax.axvline(x=result['t_statistic'], color='red', linestyle='--',
229            linewidth=2, label=f'Observed_t={result["t_statistic"]:.2f}')
230
231 # Shade rejection region for two-sided
232 if result['alternative'] == 'two-sided':
233     t_crit = stats.t.ppf(1 - result['alpha']/2, df)
234     x_left = x[x <= -t_crit]
235     x_right = x[x >= t_crit]
236     ax.fill_between(x_left, 0, stats.t.pdf(x_left, df),
237                    alpha=0.3, color='red')
238     ax.fill_between(x_right, 0, stats.t.pdf(x_right, df),
239                    alpha=0.3, color='red',
240                    label=f'Rejection_region_(α={result["alpha"]})')
241     ax.axvline(x=-t_crit, color='red', linestyle=':', alpha=0.5)
242     ax.axvline(x=t_crit, color='red', linestyle=':', alpha=0.5)
243
244 ax.set_xlabel('t-statistic')
245 ax.set_ylabel('Density')
246 ax.set_title(f't-Distribution_(p-value={result["p_value"]:.4f})')
247 ax.legend()
248 ax.grid(alpha=0.3)
249
250 plt.tight_layout()
251 return fig
252
253 # Example usage
254 print("===T-Test_for_Means===\n")
255
256 # Generate data
257 np.random.seed(42)
258
259 # Example 1: Equal variances
260 data1_eq = np.random.normal(45, 10, 500)
261 data2_eq = np.random.normal(48, 10, 500)
262
263 pooled_eq, welch_eq = compare_t_test_variants(data1_eq, data2_eq)
264
265 # Example 2: Unequal variances
266 data1_uneq = np.random.normal(45, 10, 500)
267 data2_uneq = np.random.normal(48, 20, 500)

```

```

268
269 print("\n=== Example 2: Unequal Variances ===\n")
270 pooled_uneq, welch_uneq = compare_t_test_variants(data1_uneq, data2_
    uneq)
271
272 # Visualize
273 fig1 = visualize_t_test(welch_eq, data1_eq, data2_eq)
274 plt.savefig('t_test_equal_var.pdf')
275
276 fig2 = visualize_t_test(welch_uneq, data1_uneq, data2_uneq)
277 plt.savefig('t_test_unequal_var.pdf')
278
279 plt.show()

```

5.5 Chi-Squared Test for Categorical Data

The chi-squared test assesses association between categorical variables, useful for multi-category outcomes.

5.5.1 Pearson's Chi-Squared Test

For $r \times c$ contingency table with observed counts O_{ij} :

$$H_0 : \text{Row and column variables are independent} \quad (5.18)$$

Expected counts under independence:

$$E_{ij} = \frac{(\text{row } i \text{ total}) \times (\text{column } j \text{ total})}{n} \quad (5.19)$$

Test statistic:

$$\chi^2 = \sum_{i=1}^r \sum_{j=1}^c \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \quad (5.20)$$

Under H_0 : $\chi^2 \sim \chi_{(r-1)(c-1)}^2$ approximately.

Proposition 5.1 (Equivalence for 2×2 Tables). For 2×2 contingency tables, $\chi^2 = Z^2$ where Z is the z-test statistic for proportions. The tests yield identical p-values for two-sided alternatives.

Assumptions:

- Independent observations
- Expected cell counts $E_{ij} \geq 5$ for all cells
- If some $E_{ij} < 5$: use Fisher's exact test or combine categories

5.5.2 Fisher's Exact Test

For small samples or when chi-squared assumptions fail, Fisher's exact test calculates exact p-values.

Particularly useful for 2×2 tables with small expected counts.

Listing 5.3: Chi-Squared and Fisher's Exact Test Implementation

```

1 from scipy.stats import chi2_contingency, fisher_exact
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 from typing import Dict, Tuple
6
7 def chi_square_test(
8     table: np.ndarray,
9     alpha: float = 0.05
10 ) -> Dict:
11     """
12     Perform chi-squared test of independence.
13
14     Parameters:
15     -----
16     table: np.ndarray
17         Contingency table (rows x columns)
18     alpha: float
19         Significance level
20
21     Returns:
22     -----
23     dict: Test results
24     """
25     chi2_stat, p_value, dof, expected = chi2_contingency(table)
26
27     reject_null = p_value < alpha
28
29     # Check assumption: expected counts >= 5
30     min_expected = expected.min()
31     assumption_met = min_expected >= 5
32
33     # Cramer's V effect size
34     n = table.sum()
35     min_dim = min(table.shape[0] - 1, table.shape[1] - 1)
36     cramers_v = np.sqrt(chi2_stat / (n * min_dim))
37
38     return {
39         'test': 'Chi-squared test',
40         'chi2_statistic': chi2_stat,
41         'p_value': p_value,
42         'df': dof,
43         'expected_counts': expected,
44         'min_expected': min_expected,
45         'assumption_met': assumption_met,
46         'cramers_v': cramers_v,
47         'alpha': alpha,
48         'reject_null': reject_null,
49         'decision': 'Reject H0' if reject_null else 'Fail to reject H0'
50     }
51

```

```

52 def fishers_test_2x2(
53     table: np.ndarray,
54     alpha: float = 0.05,
55     alternative: str = 'two-sided'
56 ) -> Dict:
57     """
58     Perform Fisher's exact test for 2x2 table.
59
60     Parameters:
61     -----
62     table: np.ndarray
63             2x2 contingency table
64     alpha: float
65             Significance level
66     alternative: str
67             'two-sided', 'greater', or 'less'
68
69     Returns:
70     -----
71     dict: Test results
72     """
73     if table.shape != (2, 2):
74         raise ValueError("Fisher's exact test requires a 2x2 table")
75
76     oddsratio, p_value = fisher_exact(table, alternative=alternative)
77
78     reject_null = p_value < alpha
79
80     return {
81         'test': 'Fisher\'s exact test',
82         'odds_ratio': oddsratio,
83         'p_value': p_value,
84         'alpha': alpha,
85         'reject_null': reject_null,
86         'alternative': alternative,
87         'decision': 'Reject H0' if reject_null else 'Fail to reject H0'
88     }
89
90 def compare_chi_square_fisher(table: np.ndarray, alpha: float = 0.05):
91     """Compare chi-squared test with Fisher's exact test."""
92
93     print("=== Comparison: Chi-Squared vs. Fisher's Exact Test ===\n")
94
95     print("Contingency table:")
96     print(table)
97     print()
98
99     # Chi-squared test
100     chi2_result = chi_square_test(table, alpha)
101
102     print("Chi-Squared Test:")
103     print(f"Chi-squared statistic: {chi2_result['chi2_statistic']:.3f}")
104     print(f"Degrees of freedom: {chi2_result['df']}")
105     print(f"P-value: {chi2_result['p_value']:.4f}")
106     print(f"Cram\'er\'s V: {chi2_result['cramers_v']:.3f}")
107     print(f"Min expected count: {chi2_result['min_expected']:.2f}")
108     print(f"Assumption met (E >= 5): {chi2_result['assumption_met']}")

```

```

109     print(f"Decision: {chi2_result['decision']}\n")
110
111     print("Expected frequencies:")
112     print(chi2_result['expected_counts'].round(2))
113     print()
114
115     # Fisher's exact test (if 2x2)
116     if table.shape == (2, 2):
117         fisher_result = fishers_test_2x2(table, alpha)
118
119         print("Fisher's Exact Test:")
120         print(f"Odds ratio: {fisher_result['odds_ratio']:.3f}")
121         print(f"P-value: {fisher_result['p_value']:.4f}")
122         print(f"Decision: {fisher_result['decision']}\n")
123
124         # Compare p-values
125         print(f"P-value difference: {abs(chi2_result['p_value'] -
126             fisher_result['p_value']):.4f}")
127
128         if not chi2_result['assumption_met']:
129             print("\nNote: Expected counts < 5. Fisher's exact test is
130                 more reliable.")
131
132         return chi2_result, fisher_result
133     else:
134         return chi2_result, None
135
136 def visualize_contingency_table(table: np.ndarray, result: Dict,
137     row_labels=None, col_labels=None):
138     """Visualize contingency table analysis."""
139     fig, axes = plt.subplots(1, 3, figsize=(15, 5))
140
141     if row_labels is None:
142         row_labels = [f'Row {i}' for i in range(table.shape[0])]
143     if col_labels is None:
144         col_labels = [f'Col {j}' for j in range(table.shape[1])]
145
146     # 1. Observed counts heatmap
147     ax = axes[0]
148     im1 = ax.imshow(table, cmap='YlOrRd', aspect='auto')
149     ax.set_xticks(range(table.shape[1]))
150     ax.set_yticks(range(table.shape[0]))
151     ax.set_xticklabels(col_labels)
152     ax.set_yticklabels(row_labels)
153     ax.set_title('Observed Counts')
154
155     # Add text annotations
156     for i in range(table.shape[0]):
157         for j in range(table.shape[1]):
158             text = ax.text(j, i, int(table[i, j]),
159                 ha="center", va="center", color="black",
160                 fontweight='bold', fontsize=12)
161
162     plt.colorbar(im1, ax=ax)
163
164     # 2. Expected counts heatmap
165     ax = axes[1]
166     expected = result['expected_counts']

```

```

165     im2 = ax.imshow(expected, cmap='Blues', aspect='auto')
166     ax.set_xticks(range(expected.shape[1]))
167     ax.set_yticks(range(expected.shape[0]))
168     ax.set_xticklabels(col_labels)
169     ax.set_yticklabels(row_labels)
170     ax.set_title('Expected Counts (under H0)')
171
172     # Add text annotations
173     for i in range(expected.shape[0]):
174         for j in range(expected.shape[1]):
175             text = ax.text(j, i, f'{expected[i,j]:.1f}',
176                           ha="center", va="center", color="black",
177                           fontweight='bold', fontsize=12)
178
179     plt.colorbar(im2, ax=ax)
180
181     # 3. Standardized residuals
182     ax = axes[2]
183     residuals = (table - expected) / np.sqrt(expected)
184     max_abs = np.max(np.abs(residuals))
185     im3 = ax.imshow(residuals, cmap='RdBu_r', aspect='auto',
186                   vmin=-max_abs, vmax=max_abs)
187     ax.set_xticks(range(residuals.shape[1]))
188     ax.set_yticks(range(residuals.shape[0]))
189     ax.set_xticklabels(col_labels)
190     ax.set_yticklabels(row_labels)
191     ax.set_title('Standardized Residuals\n(|residual| > 2 suggests\ndeparture)')
192
193     # Add text annotations
194     for i in range(residuals.shape[0]):
195         for j in range(residuals.shape[1]):
196             color = 'white' if abs(residuals[i, j]) > 1.5 else 'black'
197             text = ax.text(j, i, f'{residuals[i,j]:.2f}',
198                           ha="center", va="center", color=color,
199                           fontweight='bold', fontsize=10)
200
201     plt.colorbar(im3, ax=ax)
202
203     plt.suptitle(f'Chi-Squared Test:  $\chi^2 = \{result["chi2\_statistic"]:.2f\}$ ,
204                 f' $p = \{result["p\_value"]:.4f\}$ ,
205                 fontsize=14, fontweight='bold')
206     plt.tight_layout()
207     return fig
208
209     # Example usage
210     print("=== Chi-Squared Test Example ===\n")
211
212     # Example 1: Standard 2x2 table
213     table1 = np.array([[160, 1840],
214                       [190, 1810]])
215
216     chi2_1, fisher_1 = compare_chi_square_fisher(table1, alpha=0.05)
217
218     # Verify equivalence with z-test
219     x1, n1 = table1[0, 0], table1[0].sum()
220     x2, n2 = table1[1, 0], table1[1].sum()

```

```

221
222 p1 = x1 / n1
223 p2 = x2 / n2
224 p_pooled = (x1 + x2) / (n1 + n2)
225
226 z = (p2 - p1) / np.sqrt(p_pooled * (1 - p_pooled) * (1/n1 + 1/n2))
227 p_z = 2 * (1 - stats.norm.cdf(abs(z)))
228
229 print("=== Equivalence Verification (2x2 table) ===")
230 print(f"Z-test p-value: {p_z:.4f}")
231 print(f"Chi-squared p-value: {chi2_1['p_value']:.4f}")
232 print(f"z^2 = {z**2:.3f}, chi^2 = {chi2_1['chi2_statistic']:.3f}")
233 print(f"Difference: {abs(z**2 - chi2_1['chi2_statistic']):.6f}\n")
234
235 # Example 2: Small expected counts (Fisher's better)
236 table2 = np.array([[8, 12],
237                    [2, 18]])
238
239 print("=== Example 2: Small Expected Counts ===\n")
240 chi2_2, fisher_2 = compare_chi_square_fisher(table2, alpha=0.05)
241
242 # Example 3: Larger table (3x3)
243 table3 = np.array([[30, 20, 10],
244                  [25, 30, 15],
245                  [15, 25, 30]])
246
247 print("\n=== Example 3: 3x3 Table ===\n")
248 chi2_3, _ = compare_chi_square_fisher(table3, alpha=0.05)
249
250 # Visualize
251 row_labels = ['Control', 'Treatment']
252 col_labels = ['Converted', 'Not Converted']
253
254 fig1 = visualize_contingency_table(table1, chi2_1, row_labels, col_labels)
255 plt.savefig('chi_square_2x2.pdf')
256
257 category_labels = ['Category A', 'Category B', 'Category C']
258 fig3 = visualize_contingency_table(table3, chi2_3,
259                                   category_labels, category_labels)
260 plt.savefig('chi_square_3x3.pdf')
261
262 plt.show()

```

5.6 Understanding P-Values

Definition 5.1 (P-Value). The p-value is the probability, under the null hypothesis, of observing a test statistic at least as extreme as the one actually observed.

Formally: $p = P(T \geq t_{\text{obs}} | H_0)$ for one-sided, or $p = P(|T| \geq |t_{\text{obs}}| | H_0)$ for two-sided.

5.6.1 Correct Interpretation

A p-value of 0.03 means:

- ✓ If H_0 were true, we would observe data this extreme in 3% of experiments

- ✓ The data provide evidence against H_0 (but don't prove H_1)
- ✓ At $\alpha = 0.05$, we reject H_0

5.6.2 Common Misinterpretations

1. **WRONG:** "The probability that H_0 is true is 0.03"

P-values are *not* probabilities of hypotheses. They assume H_0 is true and calculate probability of data.

Correct: $P(\text{data}|H_0)$, not $P(H_0|\text{data})$

2. **WRONG:** "The probability that the result occurred by chance is 0.03"

Everything has some probability. This confuses conditional probabilities and event probabilities.

3. **WRONG:** "A p-value of 0.049 is meaningfully different from 0.051"

The $\alpha = 0.05$ threshold is arbitrary. P-values are continuous measures of evidence against H_0 , not binary outcomes.

4. **WRONG:** "A non-significant result means no effect exists"

Absence of evidence is not evidence of absence. Failure to reject H_0 may indicate:

- True null effect
- Insufficient power to detect existing effect
- High variance masking signal

5. **WRONG:** "A significant result means the effect is large or important"

Statistical significance depends on sample size. With $n=1,000,000$, tiny meaningless effects become "significant."

5.6.3 Statistical Significance vs. Practical Significance

Statistical significance ($p < \alpha$) does not imply practical importance.

Example 5.2 (Large Sample Paradox). With $n = 100,000$ per variant:

Conversion rate: 10.00% $\rightarrow$ 10.05% (0.05 percentage points)

This is highly statistically significant ($p < 0.001$) but may be economically negligible if implementation cost exceeds benefit.

Conversely, with $n = 100$:

Conversion rate: 10% $\rightarrow$ 15% (5 percentage points)

This may not reach significance ($p = 0.12$) despite clear practical importance.

Recommendation: Report effect sizes, confidence intervals, and business impact alongside p-values. Make decisions based on practical significance informed by statistical uncertainty.

5.7 Assumptions and Diagnostics

All parametric tests make assumptions. Violating assumptions can invalidate conclusions.

5.7.1 Normality Testing

Many tests (t-test, ANOVA) assume data are approximately normally distributed.

Visual diagnostics:

- Histograms
- Q-Q plots (quantile-quantile)
- Kernel density estimates

Formal tests:

- Shapiro-Wilk test (most powerful for small-medium samples)
- Kolmogorov-Smirnov test
- Anderson-Darling test

Important caveat: With large samples, normality tests often reject H_0 for trivial departures. Visual inspection often more useful.

Robustness: T-tests are robust to moderate departures from normality, especially with:

- Large samples ($n > 30$ per group)
- Symmetric distributions
- Equal sample sizes

5.7.2 Variance Homogeneity

Pooled t-test and ANOVA assume equal variances across groups.

Visual diagnostics:

- Side-by-side boxplots
- Residual plots
- Spread-location plots

Formal tests:

- Levene's test (robust to non-normality)
- Bartlett's test (sensitive to non-normality)
- F-test (assumes normality)

Rule of thumb: If largest variance / smallest variance > 4 , consider variances unequal.

Solution: Use Welch's t-test (doesn't assume equal variances).

5.7.3 Independence Assumptions

Most critical assumption, hardest to test.

Violations:

- Repeated measures (same user appears multiple times)
- Clustering (users within groups more similar)
- Time series (temporal autocorrelation)
- Spatial correlation
- Network effects

Detection:

- Check data collection process
- Autocorrelation plots (time series)
- Intraclass correlation coefficient (clustering)

Solutions:

- Mixed effects models
- Cluster-robust standard errors
- Time series methods
- Aggregation to independent units

5.7.4 Outlier Detection and Treatment

Outliers can severely affect means and standard deviations, distorting test results.

Detection methods:

- Boxplot (IQR method): values $< Q_1 - 1.5 \times IQR$ or $> Q_3 + 1.5 \times IQR$
- Z-score: $|z| > 3$ (or 2.5 for stricter criterion)
- Modified Z-score using median absolute deviation (robust)
- Visual inspection (always!)

Treatment options:

1. **Investigate:** Are outliers errors or valid extreme values?
2. **Cap/winsorize:** Replace extreme values with threshold (e.g., 99th percentile)
3. **Transform:** Log transform to reduce impact of extremes
4. **Remove:** Only if clear data errors, document clearly

5. **Robust methods:** Use median instead of mean, trimmed means, non-parametric tests
 6. **Separate analysis:** Analyze with and without outliers, report both
- Never:** Selectively remove outliers to achieve desired p-value.

Listing 5.4: Comprehensive Diagnostic Functions

```

1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from typing import Dict, Tuple
6 import pandas as pd
7
8 def check_normality(
9     data: np.ndarray,
10     name: str = "Data",
11     alpha: float = 0.05
12 ) -> Dict:
13     """
14     Check normality assumption using multiple methods.
15
16     Parameters:
17     -----
18     data: np.ndarray
19         Data to test
20     name: str
21         Name for labeling
22     alpha: float
23         Significance level for tests
24
25     Returns:
26     -----
27     dict: Normality test results
28     """
29     # Shapiro-Wilk test
30     shapiro_stat, shapiro_p = stats.shapiro(data)
31
32     # Kolmogorov-Smirnov test
33     ks_stat, ks_p = stats.kstest(data, 'norm',
34                                 args=(np.mean(data), np.std(data)))
35
36     # Anderson-Darling test
37     anderson_result = stats.anderson(data, dist='norm')
38
39     # Skewness and kurtosis
40     skewness = stats.skew(data)
41     kurtosis = stats.kurtosis(data)
42
43     return {
44         'name': name,
45         'n': len(data),
46         'shapiro_statistic': shapiro_stat,
47         'shapiro_pvalue': shapiro_p,
48         'shapiro_normal': shapiro_p > alpha,
49         'ks_statistic': ks_stat,
50         'ks_pvalue': ks_p,

```

```

51         'ks_normal': ks_p > alpha,
52         'anderson_statistic': anderson_result.statistic,
53         'anderson_critical_values': anderson_result.critical_values,
54         'skewness': skewness,
55         'kurtosis': kurtosis
56     }
57
58 def check_variance_equality(
59     data1: np.ndarray,
60     data2: np.ndarray,
61     alpha: float = 0.05
62 ) -> Dict:
63     """
64     Check equality of variances.
65
66     Parameters:
67     -----
68     data1, data2: np.ndarray
69         Data for each group
70     alpha: float
71         Significance level
72
73     Returns:
74     -----
75     dict: Variance equality test results
76     """
77     var1, var2 = np.var(data1, ddof=1), np.var(data2, ddof=1)
78     std1, std2 = np.std(data1, ddof=1), np.std(data2, ddof=1)
79
80     # Variance ratio
81     var_ratio = max(var1, var2) / min(var1, var2)
82
83     # Levene's test (robust to non-normality)
84     levene_stat, levene_p = stats.levene(data1, data2)
85
86     # Bartlett's test (assumes normality)
87     bartlett_stat, bartlett_p = stats.bartlett(data1, data2)
88
89     # F-test (assumes normality)
90     if var1 >= var2:
91         f_stat = var1 / var2
92         df1, df2 = len(data1) - 1, len(data2) - 1
93     else:
94         f_stat = var2 / var1
95         df2, df1 = len(data1) - 1, len(data2) - 1
96
97     f_p = 2 * min(stats.f.cdf(f_stat, df1, df2),
98                  1 - stats.f.cdf(f_stat, df1, df2))
99
100     return {
101         'var1': var1,
102         'var2': var2,
103         'std1': std1,
104         'std2': std2,
105         'variance_ratio': var_ratio,
106         'levene_statistic': levene_stat,
107         'levene_pvalue': levene_p,
108         'levene_equal': levene_p > alpha,

```

```

109         'bartlett_statistic': bartlett_stat,
110         'bartlett_pvalue': bartlett_p,
111         'bartlett_equal': bartlett_p > alpha,
112         'f_statistic': f_stat,
113         'f_pvalue': f_p,
114         'f_equal': f_p > alpha
115     }
116
117 def detect_outliers(
118     data: np.ndarray,
119     method: str = 'iqr'
120 ) -> Dict:
121     """
122     Detect outliers using various methods.
123
124     Parameters:
125     -----
126     data: np.ndarray
127         Data to analyze
128     method: str
129         'iqr', 'zscore', or 'modified_zscore'
130
131     Returns:
132     -----
133     dict: Outlier detection results
134     """
135     if method == 'iqr':
136         q1 = np.percentile(data, 25)
137         q3 = np.percentile(data, 75)
138         iqr = q3 - q1
139         lower_bound = q1 - 1.5 * iqr
140         upper_bound = q3 + 1.5 * iqr
141         outliers = (data < lower_bound) | (data > upper_bound)
142
143     elif method == 'zscore':
144         z_scores = np.abs(stats.zscore(data))
145         outliers = z_scores > 3
146
147     elif method == 'modified_zscore':
148         median = np.median(data)
149         mad = np.median(np.abs(data - median))
150         modified_z_scores = 0.6745 * (data - median) / mad
151         outliers = np.abs(modified_z_scores) > 3.5
152
153     else:
154         raise ValueError("method must be 'iqr', 'zscore', or 'modified_zscore'")
155
156     outlier_indices = np.where(outliers)[0]
157     outlier_values = data[outliers]
158
159     return {
160         'method': method,
161         'n_outliers': np.sum(outliers),
162         'outlier_proportion': np.mean(outliers),
163         'outlier_indices': outlier_indices,
164         'outlier_values': outlier_values,
165         'outlier_mask': outliers

```

```

166     }
167
168 def visualize_diagnostics(
169     data1: np.ndarray,
170     data2: np.ndarray,
171     name1: str = "Control",
172     name2: str = "Treatment"
173 ):
174     """Create comprehensive diagnostic plots."""
175     fig = plt.figure(figsize=(16, 12))
176     gs = fig.add_gridspec(3, 3, hspace=0.3, wspace=0.3)
177
178     # 1. Histograms with normal overlay
179     ax1 = fig.add_subplot(gs[0, 0])
180     ax1.hist(data1, bins=30, alpha=0.6, density=True, label=name1,
181             color='C0')
182     mu1, std1 = np.mean(data1), np.std(data1)
183     x1 = np.linspace(data1.min(), data1.max(), 100)
184     ax1.plot(x1, stats.norm.pdf(x1, mu1, std1), 'C0--', linewidth=2,
185             label='Normal_fit')
186     ax1.set_xlabel('Value')
187     ax1.set_ylabel('Density')
188     ax1.set_title(f'{name1}_Distribution')
189     ax1.legend()
190     ax1.grid(alpha=0.3)
191
192     ax2 = fig.add_subplot(gs[0, 1])
193     ax2.hist(data2, bins=30, alpha=0.6, density=True, label=name2,
194             color='C1')
195     mu2, std2 = np.mean(data2), np.std(data2)
196     x2 = np.linspace(data2.min(), data2.max(), 100)
197     ax2.plot(x2, stats.norm.pdf(x2, mu2, std2), 'C1--', linewidth=2,
198             label='Normal_fit')
199     ax2.set_xlabel('Value')
200     ax2.set_ylabel('Density')
201     ax2.set_title(f'{name2}_Distribution')
202     ax2.legend()
203     ax2.grid(alpha=0.3)
204
205     # 2. Q-Q plots
206     ax3 = fig.add_subplot(gs[0, 2])
207     stats.probplot(data1, dist="norm", plot=ax3)
208     ax3.set_title(f'{name1}_Q-Q_Plot')
209     ax3.grid(alpha=0.3)
210
211     ax4 = fig.add_subplot(gs[1, 0])
212     stats.probplot(data2, dist="norm", plot=ax4)
213     ax4.set_title(f'{name2}_Q-Q_Plot')
214     ax4.grid(alpha=0.3)
215
216     # 3. Box plots
217     ax5 = fig.add_subplot(gs[1, 1])
218     box_data = [data1, data2]
219     bp = ax5.boxplot(box_data, labels=[name1, name2],
220                     patch_artist=True, widths=0.6)
221     for patch, color in zip(bp['boxes'], ['C0', 'C1']):
222         patch.set_facecolor(color)
223         patch.set_alpha(0.6)

```

```

222 ax5.set_ylabel('Value')
223 ax5.set_title('Box Plots (outlier detection)')
224 ax5.grid(axis='y', alpha=0.3)
225
226 # Add variance ratio annotation
227 var_ratio = max(np.var(data1), np.var(data2)) / min(np.var(data1),
228 np.var(data2))
229 ax5.text(0.5, 0.95, f'Variance ratio: {var_ratio:.2f}',
230         transform=ax5.transAxes, ha='center', va='top',
231         bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
232
233 # 4. Violin plots with individual points
234 ax6 = fig.add_subplot(gs[1, 2])
235 parts = ax6.violinplot([data1, data2], positions=[1, 2],
236                        showmeans=True, showmedians=True)
237 ax6.set_xticks([1, 2])
238 ax6.set_xticklabels([name1, name2])
239 ax6.set_ylabel('Value')
240 ax6.set_title('Violin Plots')
241 ax6.grid(axis='y', alpha=0.3)
242
243 # 5. ECDF comparison
244 ax7 = fig.add_subplot(gs[2, 0])
245 sorted_data1 = np.sort(data1)
246 sorted_data2 = np.sort(data2)
247 ecdf1 = np.arange(1, len(sorted_data1) + 1) / len(sorted_data1)
248 ecdf2 = np.arange(1, len(sorted_data2) + 1) / len(sorted_data2)
249 ax7.plot(sorted_data1, ecdf1, label=name1, linewidth=2, color='C0')
250 ax7.plot(sorted_data2, ecdf2, label=name2, linewidth=2, color='C1')
251 ax7.set_xlabel('Value')
252 ax7.set_ylabel('ECDF')
253 ax7.set_title('Empirical Cumulative Distribution')
254 ax7.legend()
255 ax7.grid(alpha=0.3)
256
257 # 6. Variance comparison
258 ax8 = fig.add_subplot(gs[2, 1])
259 variances = [np.var(data1, ddof=1), np.var(data2, ddof=1)]
260 std_devs = [np.std(data1, ddof=1), np.std(data2, ddof=1)]
261
262 x_pos = np.arange(2)
263 bars = ax8.bar(x_pos, std_devs, alpha=0.7, color=['C0', 'C1'])
264 ax8.set_xticks(x_pos)
265 ax8.set_xticklabels([name1, name2])
266 ax8.set_ylabel('Standard Deviation')
267 ax8.set_title('Standard Deviations Comparison')
268 ax8.grid(axis='y', alpha=0.3)
269
270 for i, (bar, std) in enumerate(zip(bars, std_devs)):
271     height = bar.get_height()
272     ax8.text(bar.get_x() + bar.get_width()/2., height,
273             f'{std:.2f}', ha='center', va='bottom', fontweight='
274
275 # 7. Summary statistics table
276 ax9 = fig.add_subplot(gs[2, 2])
277 ax9.axis('off')

```

```

278     summary_data = [
279         ['Statistic', name1, name2],
280         ['n', f'{len(data1)}', f'{len(data2)}'],
281         ['Mean', f'{np.mean(data1):.2f}', f'{np.mean(data2):.2f}'],
282         ['Median', f'{np.median(data1):.2f}', f'{np.median(data2):.2f}'],
283         ['Std_Dev', f'{np.std(data1, ddof=1):.2f}', f'{np.std(data2, ddof=1):.2f}'],
284         ['Min', f'{data1.min():.2f}', f'{data2.min():.2f}'],
285         ['Max', f'{data1.max():.2f}', f'{data2.max():.2f}'],
286         ['Q1', f'{np.percentile(data1, 25):.2f}', f'{np.percentile(data2, 25):.2f}'],
287         ['Q3', f'{np.percentile(data1, 75):.2f}', f'{np.percentile(data2, 75):.2f}'],
288         ['Skewness', f'{stats.skew(data1):.2f}', f'{stats.skew(data2):.2f}'],
289         ['Kurtosis', f'{stats.kurtosis(data1):.2f}', f'{stats.kurtosis(data2):.2f}']
290     ]
291
292     table = ax9.table(cellText=summary_data, cellLoc='center',
293                      loc='center', bbox=[0, 0, 1, 1])
294     table.auto_set_font_size(False)
295     table.set_fontsize(9)
296     table.scale(1, 1.8)
297
298     # Style header row
299     for i in range(3):
300         table[(0, i)].set_facecolor('#4CAF50')
301         table[(0, i)].set_text_props(weight='bold', color='white')
302
303     plt.suptitle('Comprehensive_Diagnostic_Plots', fontsize=16,
304                 fontweight='bold')
305
306     return fig
307
308 # Example usage
309 print("=== Assumption_Diagnostics ===\n")
310
311 np.random.seed(42)
312
313 # Generate test data
314 data1_normal = np.random.normal(50, 10, 500)
315 data2_normal = np.random.normal(53, 10, 500)
316
317 data1_skewed = np.random.gamma(2, 5, 500)
318 data2_skewed = np.random.gamma(2.5, 5, 500)
319
320 data1_unequal_var = np.random.normal(50, 10, 500)
321 data2_unequal_var = np.random.normal(53, 20, 500)
322
323 # Check normality
324 print("=== Normality_Tests (Normal_Data) ===")
325 norm_result1 = check_normality(data1_normal, "Control_Normal")
326 print(f"{norm_result1['name']} (n={norm_result1['n']}):")
327 print(f"Shapiro-Wilk: W={norm_result1['shapiro_statistic']:.4f}, p={norm_result1['shapiro_pvalue']:.4f}, Normal: {norm_result1['shapiro_normal']}")
328

```

```

329 print(f"Skewness: {norm_result1['skewness']:.3f}")
330 print(f"Kurtosis: {norm_result1['kurtosis']:.3f}\n")
331
332 print("=== Normality Tests (Skewed Data) ===")
333 norm_result2 = check_normality(data1_skewed, "Control (Skewed)")
334 print(f"{norm_result2['name']} (n={norm_result2['n']})")
335 print(f"Shapiro-Wilk: W={norm_result2['shapiro_statistic']:.4f}, "
336       f"p={norm_result2['shapiro_pvalue']:.4e}, "
337       f"Normal: {norm_result2['shapiro_normal']}")
338 print(f"Skewness: {norm_result2['skewness']:.3f}")
339 print(f"Kurtosis: {norm_result2['kurtosis']:.3f}\n")
340
341 # Check variance equality
342 print("=== Variance Equality Tests ===")
343 var_result = check_variance_equality(data1_unequal_var, data2_unequal_
    var)
344 print(f"Variance 1: {var_result['var1']:.2f}, Variance 2: {var_result['
    var2']:.2f}")
345 print(f"Variance ratio: {var_result['variance_ratio']:.2f}")
346 print(f"Levene's test: F={var_result['levене_statistic']:.4f}, "
347       f"p={var_result['levене_pvalue']:.4f}, "
348       f"Equal: {var_result['levене_equal']}\n")
349
350 # Detect outliers
351 print("=== Outlier Detection ===")
352 # Add some outliers
353 data_with_outliers = np.concatenate([data1_normal, [100, 105, -20]])
354 outlier_result = detect_outliers(data_with_outliers, method='iqr')
355 print(f"Method: {outlier_result['method']}")
356 print(f"Number of outliers: {outlier_result['n_outliers']}")
357 print(f"Outlier proportion: {outlier_result['outlier_proportion']:.3f}"
    )
358 print(f"Outlier values: {outlier_result['outlier_values']}\n")
359
360 # Visualize
361 fig1 = visualize_diagnostics(data1_normal, data2_normal)
362 plt.savefig('diagnostics_normal.pdf')
363
364 fig2 = visualize_diagnostics(data1_skewed, data2_skewed,
    "Control (Skewed)", "Treatment (Skewed)")
365 plt.savefig('diagnostics_skewed.pdf')
366
367 fig3 = visualize_diagnostics(data1_unequal_var, data2_unequal_var,
    "Control ( $\sigma=10$ )", "Treatment ( $\sigma=20$ )")
368 plt.savefig('diagnostics_unequal_var.pdf')
369
370
371
372 plt.show()

```

5.8 Non-Parametric Alternatives

When parametric assumptions fail, non-parametric tests provide robust alternatives.

5.8.1 Mann-Whitney U Test

The Mann-Whitney U test (also called Wilcoxon rank-sum test) tests whether two samples come from populations with the same distribution.

Null hypothesis: The distributions of the two groups are identical (specifically, $P(X_1 > X_2) = 0.5$).

Procedure:

1. Rank all observations from both groups combined
2. Calculate sum of ranks for each group: R_1, R_2
3. Calculate U statistic:

$$U_1 = R_1 - \frac{n_1(n_1 + 1)}{2}, \quad U_2 = R_2 - \frac{n_2(n_2 + 1)}{2} \quad (5.21)$$

4. Use $U = \min(U_1, U_2)$ as test statistic

Advantages:

- No normality assumption
- Robust to outliers
- Works for ordinal data
- More powerful than t-test for heavy-tailed distributions

Disadvantages:

- Less powerful than t-test when data are normal
- Tests distribution similarity, not just location
- Harder to interpret (medians, not means)

5.8.2 Permutation Tests

Permutation tests provide exact p-values without distributional assumptions.

Principle: Under H_0 of no treatment effect, treatment labels are exchangeable.

Procedure:

1. Calculate observed test statistic T_{obs}
2. Randomly permute treatment labels many times (e.g., 10,000)
3. For each permutation, calculate test statistic
4. P-value = proportion of permutations with $|T| \geq |T_{\text{obs}}|$

Advantages:

- Exact finite-sample validity (no approximations)
- No distributional assumptions

- Works for any test statistic (means, medians, quantiles, etc.)
- Intuitive interpretation

Disadvantages:

- Computationally intensive
- Requires exchangeability (randomization)
- Less familiar to stakeholders

5.8.3 Bootstrap Methods

Bootstrap resampling estimates sampling distributions and constructs confidence intervals.

Procedure for confidence interval:

1. Resample with replacement from each group
2. Calculate statistic of interest (e.g., difference in means)
3. Repeat B times (e.g., B=10,000)
4. Use percentiles of bootstrap distribution for CI

Percentile CI:

$$CI_{1-\alpha} = [\text{Percentile}_{\alpha/2}, \text{Percentile}_{1-\alpha/2}] \quad (5.22)$$

Bias-corrected and accelerated (BCa) CI: Adjusts for bias and skewness in bootstrap distribution.

Listing 5.5: Non-Parametric Tests Implementation

```

1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4 from typing import Dict
5 import pandas as pd
6
7 def mann_whitney_test(
8     data1: np.ndarray,
9     data2: np.ndarray,
10    alpha: float = 0.05,
11    alternative: str = 'two-sided'
12 ) -> Dict:
13     """
14     Perform Mann-Whitney U test.
15
16     Parameters:
17     -----
18     data1, data2: np.ndarray
19         Data for each group
20     alpha: float
21         Significance level
22     alternative: str
23         'two-sided', 'greater', or 'less'

```

```

24
25 """Returns:
26 """-----
27 """dict_: Test results
28 """
29     u_stat, p_value = stats.mannwhitneyu(
30         data1, data2, alternative=alternative
31     )
32
33     reject_null = p_value < alpha
34
35     # Effect size: rank-biserial correlation
36     n1, n2 = len(data1), len(data2)
37     rank_biserial = 1 - (2*u_stat) / (n1 * n2)
38
39     # Median difference (approximate)
40     median1, median2 = np.median(data1), np.median(data2)
41     median_diff = median2 - median1
42
43     return {
44         'test': 'Mann-Whitney_U_test',
45         'u_statistic': u_stat,
46         'p_value': p_value,
47         'alpha': alpha,
48         'reject_null': reject_null,
49         'alternative': alternative,
50         'median1': median1,
51         'median2': median2,
52         'median_diff': median_diff,
53         'rank_biserial': rank_biserial,
54         'n1': n1,
55         'n2': n2,
56         'decision': 'Reject_H0' if reject_null else 'Fail_to_reject_H0'
57     }
58
59 def permutation_test(
60     data1: np.ndarray,
61     data2: np.ndarray,
62     n_permutations: int = 10000,
63     statistic: str = 'mean_diff',
64     alternative: str = 'two-sided',
65     seed: int = None
66 ) -> Dict:
67     """
68     """Perform permutation test.
69
70     Parameters:
71     """-----
72     data1, data2: np.ndarray
73     Data for each group
74     n_permutations: int
75     Number of permutations
76     statistic: str
77     Test statistic: 'mean_diff', 'median_diff', 't_stat', or custom
78     function
79     alternative: str
80     'two-sided', 'greater', or 'less'
81     seed: int

```

```

81  """Random seed
82
83  """Returns:
84  """-----
85  """dict_: Test results
86  """
87      if seed is not None:
88          np.random.seed(seed)
89
90      combined = np.concatenate([data1, data2])
91      n1 = len(data1)
92      n_total = len(combined)
93
94      # Define test statistic function
95      def calc_statistic(d1, d2):
96          if statistic == 'mean_diff':
97              return np.mean(d2) - np.mean(d1)
98          elif statistic == 'median_diff':
99              return np.median(d2) - np.median(d1)
100         elif statistic == 't_stat':
101             # Welch's t-statistic
102             m1, m2 = np.mean(d1), np.mean(d2)
103             v1, v2 = np.var(d1, ddof=1), np.var(d2, ddof=1)
104             n1, n2 = len(d1), len(d2)
105             return (m2 - m1) / np.sqrt(v1/n1 + v2/n2)
106         elif callable(statistic):
107             return statistic(d1, d2)
108         else:
109             raise ValueError("Invalid statistic")
110
111     # Observed test statistic
112     obs_stat = calc_statistic(data1, data2)
113
114     # Permutation distribution
115     perm_stats = np.zeros(n_permutations)
116
117     for i in range(n_permutations):
118         # Randomly permute
119         permuted = np.random.permutation(combined)
120         perm_data1 = permuted[:n1]
121         perm_data2 = permuted[n1:]
122
123         perm_stats[i] = calc_statistic(perm_data1, perm_data2)
124
125     # P-value
126     if alternative == 'two-sided':
127         p_value = np.mean(np.abs(perm_stats) >= np.abs(obs_stat))
128     elif alternative == 'greater':
129         p_value = np.mean(perm_stats >= obs_stat)
130     elif alternative == 'less':
131         p_value = np.mean(perm_stats <= obs_stat)
132     else:
133         raise ValueError("alternative must be 'two-sided', 'greater', or 'less'")
134
135     reject_null = p_value < 0.05
136
137     return {

```

```

138         'test': 'Permutation_test',
139         'observed_statistic': obs_stat,
140         'permutation_statistics': perm_stats,
141         'p_value': p_value,
142         'n_permutations': n_permutations,
143         'alternative': alternative,
144         'reject_null': reject_null,
145         'statistic_type': statistic if isinstance(statistic, str) else
            'custom',
146         'decision': 'Reject_H0' if reject_null else 'Fail_to_reject_H0'
147     }
148
149 def bootstrap_ci(
150     data1: np.ndarray,
151     data2: np.ndarray,
152     statistic: str = 'mean_diff',
153     n_bootstrap: int = 10000,
154     alpha: float = 0.05,
155     method: str = 'percentile',
156     seed: int = None
157 ) -> Dict:
158     """
159     Bootstrapped confidence interval for difference.
160
161     Parameters:
162     -----
163     data1, data2: np.ndarray
164         Data for each group
165     statistic: str
166         'mean_diff', 'median_diff', or custom function
167     n_bootstrap: int
168         Number of bootstrap samples
169     alpha: float
170         Significance level
171     method: str
172         'percentile' or 'bca' (bias-corrected accelerated)
173     seed: int
174         Random seed
175
176     Returns:
177     -----
178     dict: Bootstrapped results
179     """
180     if seed is not None:
181         np.random.seed(seed)
182
183     def calc_statistic(d1, d2):
184         if statistic == 'mean_diff':
185             return np.mean(d2) - np.mean(d1)
186         elif statistic == 'median_diff':
187             return np.median(d2) - np.median(d1)
188         elif callable(statistic):
189             return statistic(d1, d2)
190         else:
191             raise ValueError("Invalid statistic")
192
193     # Observed statistic
194     obs_stat = calc_statistic(data1, data2)

```

```

195
196 # Bootstrap distribution
197 bootstrap_stats = np.zeros(n_bootstrap)
198
199 for i in range(n_bootstrap):
200     # Resample with replacement from each group
201     boot_data1 = np.random.choice(data1, size=len(data1), replace=
202                                     True)
203     boot_data2 = np.random.choice(data2, size=len(data2), replace=
204                                     True)
205
206     bootstrap_stats[i] = calc_statistic(boot_data1, boot_data2)
207
208 if method == 'percentile':
209     ci_lower = np.percentile(bootstrap_stats, 100 * alpha/2)
210     ci_upper = np.percentile(bootstrap_stats, 100 * (1 - alpha/2))
211
212 elif method == 'bca':
213     # Bias correction
214     z0 = stats.norm.ppf(np.mean(bootstrap_stats < obs_stat))
215
216     # Acceleration (jackknife)
217     n1, n2 = len(data1), len(data2)
218     jack_stats = []
219
220     for i in range(n1):
221         jack_data1 = np.delete(data1, i)
222         jack_stats.append(calc_statistic(jack_data1, data2))
223
224     jack_mean = np.mean(jack_stats)
225     numerator = np.sum((jack_mean - jack_stats)**3)
226     denominator = 6 * (np.sum((jack_mean - jack_stats)**2)**(3/2))
227     a = numerator / denominator if denominator != 0 else 0
228
229     # Adjusted percentiles
230     z_alpha_2 = stats.norm.ppf(alpha/2)
231     z_1_alpha_2 = stats.norm.ppf(1 - alpha/2)
232
233     alpha_1 = stats.norm.cdf(z0 + (z0 + z_alpha_2)/(1 - a*(z0 + z_
234     alpha_2)))
235     alpha_2 = stats.norm.cdf(z0 + (z0 + z_1_alpha_2)/(1 - a*(z0 + z
236     _1_alpha_2)))
237
238     ci_lower = np.percentile(bootstrap_stats, 100 * alpha_1)
239     ci_upper = np.percentile(bootstrap_stats, 100 * alpha_2)
240
241 else:
242     raise ValueError("method must be 'percentile' or 'bca'")
243
244 # Standard error
245 bootstrap_se = np.std(bootstrap_stats)
246
247 return {
248     'method': f'Bootstrap_{method}_CI',
249     'observed_statistic': obs_stat,
250     'bootstrap_statistics': bootstrap_stats,
251     'bootstrap_se': bootstrap_se,
252     'ci_lower': ci_lower,

```

```

249         'ci_upper': ci_upper,
250         'alpha': alpha,
251         'n_bootstrap': n_bootstrap,
252         'statistic_type': statistic if isinstance(statistic, str) else
            'custom'
253     }
254
255 def visualize_nonparametric_tests(
256     data1: np.ndarray,
257     data2: np.ndarray,
258     perm_result: Dict,
259     boot_result: Dict,
260     name1: str = "Control",
261     name2: str = "Treatment"
262 ):
263     """Visualize non-parametric test results."""
264     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
265
266     # 1. Permutation distribution
267     ax = axes[0, 0]
268     ax.hist(perm_result['permutation_statistics'], bins=50, density=
        True,
269             alpha=0.7, label='Permutation distribution', color='skyblue'
        )
270     ax.axvline(perm_result['observed_statistic'], color='red',
271               linestyle='--', linewidth=2,
272               label=f'Observed={perm_result["observed_statistic"]:.3f}
        ')
273
274     # Add p-value region
275     if perm_result['alternative'] == 'two-sided':
276         obs_abs = abs(perm_result['observed_statistic'])
277         ax.axvline(-obs_abs, color='red', linestyle='--',
278                   linewidth=2, alpha=0.5)
279
280     ax.set_xlabel('Test Statistic')
281     ax.set_ylabel('Density')
282     ax.set_title(f'Permutation Test\n'
283                 f'p-value={perm_result["p_value"]:.4f}')
284     ax.legend()
285     ax.grid(alpha=0.3)
286
287     # 2. Bootstrap distribution
288     ax = axes[0, 1]
289     ax.hist(boot_result['bootstrap_statistics'], bins=50, density=True,
290            alpha=0.7, label='Bootstrap distribution', color='lightgreen'
291            )
292     ax.axvline(boot_result['observed_statistic'], color='red',
293               linestyle='--', linewidth=2,
294               label=f'Observed={boot_result["observed_statistic"]:.3f}
        ')
295     ax.axvline(boot_result['ci_lower'], color='orange',
296               linestyle=':', linewidth=2,
297               label=f'CI: [{boot_result["ci_lower"]:.3f}, {boot_result
298               ["ci_upper"]:.3f}]')
299     ax.axvline(boot_result['ci_upper'], color='orange',
300               linestyle=':', linewidth=2)

```

```

300 ax.set_xlabel('Difference')
301 ax.set_ylabel('Density')
302 ax.set_title(f'Bootstrap Distribution\n'
303             f'SE={boot_result["bootstrap_se"]:.3f}')
304 ax.legend()
305 ax.grid(alpha=0.3)
306
307 # 3. ECDF comparison (Mann-Whitney concept)
308 ax = axes[1, 0]
309 sorted_data1 = np.sort(data1)
310 sorted_data2 = np.sort(data2)
311 ecdf1 = np.arange(1, len(sorted_data1) + 1) / len(sorted_data1)
312 ecdf2 = np.arange(1, len(sorted_data2) + 1) / len(sorted_data2)
313 ax.plot(sorted_data1, ecdf1, label=name1, linewidth=2, color='C0')
314 ax.plot(sorted_data2, ecdf2, label=name2, linewidth=2, color='C1')
315
316 # Add medians
317 ax.axvline(np.median(data1), color='C0', linestyle='--', alpha=0.5)
318 ax.axvline(np.median(data2), color='C1', linestyle='--', alpha=0.5)
319
320 ax.set_xlabel('Value')
321 ax.set_ylabel('ECDF')
322 ax.set_title('Empirical CDFs (Mann-Whitney Concept)')
323 ax.legend()
324 ax.grid(alpha=0.3)
325
326 # 4. Comparison table
327 ax = axes[1, 1]
328 ax.axis('off')
329
330 # Run all tests for comparison
331 t_result = t_test_means(data1, data2)
332 mw_result = mann_whitney_test(data1, data2)
333
334 comparison_data = [
335     ['Test', 'Statistic', 'P-value', 'Decision'],
336     ['T-test', f'{t_result["t_statistic"]:.3f}',
337      f'{t_result["p_value"]:.4f}', t_result['decision']],
338     ['Mann-Whitney', f'{mw_result["u_statistic"]:.1f}',
339      f'{mw_result["p_value"]:.4f}', mw_result['decision']],
340     ['Permutation', f'{perm_result["observed_statistic"]:.3f}',
341      f'{perm_result["p_value"]:.4f}', perm_result['decision']],
342     ['Bootstrap CI', f'[{boot_result["ci_lower"]:.3f}, {boot_result["ci_upper"]:.3f}]',
343      '--', 'CI includes 0' if boot_result['ci_lower'] < 0 < boot_
344          result['ci_upper'] else 'CI excludes 0']
345 ]
346
347 table = ax.table(cellText=comparison_data, cellLoc='left',
348                 loc='center', bbox=[0, 0, 1, 1])
349 table.auto_set_font_size(False)
350 table.set_fontsize(9)
351 table.scale(1, 2.5)
352
353 # Style header row
354 for i in range(4):
355     table[(0, i)].set_facecolor('#4CAF50')
356     table[(0, i)].set_text_props(weight='bold', color='white')

```

```

356     plt.suptitle('Non-Parametric Test Comparison', fontsize=14,
357                 fontweight='bold')
358     plt.tight_layout()
359
360     return fig
361
362 # Example usage
363 print("=== Non-Parametric Tests ===\n")
364
365 np.random.seed(42)
366
367 # Example 1: Normal data (t-test and non-parametric should agree)
368 data1_normal = np.random.normal(50, 10, 200)
369 data2_normal = np.random.normal(53, 10, 200)
370
371 # Example 2: Skewed data (non-parametric more appropriate)
372 data1_skewed = np.random.gamma(2, 5, 200)
373 data2_skewed = np.random.gamma(2.5, 5, 200)
374
375 print("=== Example 1: Normal Data ===\n")
376
377 # T-test
378 t_result = t_test_means(data1_normal, data2_normal)
379 print(f"T-test: t={t_result['t_statistic']:.3f}, p={t_result['p_value']:.4f}\n")
380
381 # Mann-Whitney
382 mw_result = mann_whitney_test(data1_normal, data2_normal)
383 print(f"Mann-Whitney: U={mw_result['u_statistic']:.1f}, p={mw_result['p_value']:.4f}")
384 print(f"Median difference: {mw_result['median_diff']:.3f}")
385 print(f"Rank-biserial correlation: {mw_result['rank_biserial']:.3f}\n")
386
387 # Permutation test
388 perm_result = permutation_test(data1_normal, data2_normal,
389                               n_permutations=10000, seed=42)
390 print(f"Permutation test: p={perm_result['p_value']:.4f}\n")
391
392 # Bootstrap CI
393 boot_result = bootstrap_ci(data1_normal, data2_normal,
394                            n_bootstrap=10000, seed=42)
395 print(f"Bootstrap 95% CI: [{boot_result['ci_lower']:.3f}, {boot_result['ci_upper']:.3f}]")
396 print(f"Bootstrap SE: {boot_result['bootstrap_se']:.3f}\n")
397
398 print("=== Example 2: Skewed Data ===\n")
399
400 t_result_skewed = t_test_means(data1_skewed, data2_skewed)
401 mw_result_skewed = mann_whitney_test(data1_skewed, data2_skewed)
402 perm_result_skewed = permutation_test(data1_skewed, data2_skewed,
403                                       n_permutations=10000, seed=42)
404 boot_result_skewed = bootstrap_ci(data1_skewed, data2_skewed,
405                                   n_bootstrap=10000, seed=42)
406
407 print(f"T-test: t={t_result_skewed['t_statistic']:.3f}, p="
408       f"p={t_result_skewed['p_value']:.4f}")
409 print(f"Mann-Whitney: U={mw_result_skewed['u_statistic']:.1f}, p="

```



```

410     f"p={mw_result_skewed['p_value']:.4f}")
411 print(f"Permutation test: p={perm_result_skewed['p_value']:.4f}")
412 print(f"Bootstrap 95% CI: [{boot_result_skewed['ci_lower']:.3f}, "
413       f"{boot_result_skewed['ci_upper']:.3f}]\n")
414
415 # Visualize
416 fig1 = visualize_nonparametric_tests(data1_normal, data2_normal,
417                                     perm_result, boot_result)
418 plt.savefig('nonparametric_normal.pdf')
419
420 fig2 = visualize_nonparametric_tests(data1_skewed, data2_skewed,
421                                     perm_result_skewed, boot_result_
422                                     skewed,
423                                     "Control (Skewed)", "Treatment (
424                                     Skewed)")
425 plt.savefig('nonparametric_skewed.pdf')
426 plt.show()

```

5.8.4 When to Use Non-Parametric Tests

Use non-parametric tests when:

- Data are heavily skewed or have extreme outliers
- Sample sizes are small ($n < 30$ per group)
- Distributions are clearly non-normal (confirmed visually)
- Ordinal data (rankings, Likert scales)
- Variances are very unequal and sample sizes differ

Use parametric tests when:

- Data are approximately normal
- Sample sizes are large ($n > 30$)
- More statistical power needed
- Easier interpretation desired (means vs. medians)

Consider both: For borderline cases, run both parametric and non-parametric tests. If conclusions agree, confidence increases. If they differ, investigate further.

5.9 Test Selection Framework

Listing 5.6: Test Selection Decision Tree

```

1 def select_statistical_test(
2     data1: np.ndarray,
3     data2: np.ndarray,
4     metric_type: str,
5     check_assumptions: bool = True,

```

```

6     alpha: float = 0.05
7 ) -> Dict:
8     """
9     Recommend appropriate statistical test based on data
    characteristics.
10
11     Parameters:
12     -----
13     data1, data2: np.ndarray
14     Data for each group
15     metric_type: str
16     'proportion', 'mean', 'count', or 'ordinal'
17     check_assumptions: bool
18     Whether to check parametric assumptions
19     alpha: float
20     Significance level
21
22     Returns:
23     -----
24     dict: Test recommendation and results
25     """
26     n1, n2 = len(data1), len(data2)
27
28     recommendations = []
29     tests_run = {}
30
31     # Proportion test
32     if metric_type == 'proportion':
33         # Check sample size requirements
34         if isinstance(data1[0], (int, np.integer)):
35             # Binary data (0/1)
36             x1, x2 = data1.sum(), data2.sum()
37             p1, p2 = x1/n1, x2/n2
38
39             min_count = min(x1, n1-x1, x2, n2-x2)
40
41             if min_count >= 10:
42                 # Z-test appropriate
43                 result = z_test_proportions(int(x1), n1, int(x2), n2,
44                                             alpha)
45                 recommendations.append("Use Z-test (normal
46                                     approximation valid)")
47                 tests_run['z_test'] = result
48             elif min_count >= 5:
49                 # Z-test with continuity correction or Fisher's
50                 result_z = z_test_proportions(int(x1), n1, int(x2), n2,
51                                             alpha)
52                 table = np.array([[x1, n1-x1], [x2, n2-x2]])
53                 result_fisher = fishers_test_2x2(table, alpha)
54
55                 recommendations.append("Consider both Z-test (with
56                                     caution) and Fisher's exact")
57                 tests_run['z_test'] = result_z
58                 tests_run['fisher'] = result_fisher
59             else:
60                 # Fisher's exact test
61                 table = np.array([[x1, n1-x1], [x2, n2-x2]])
62                 result = fishers_test_2x2(table, alpha)

```

```

59         recommendations.append("Use_Fisher's_exact_test_(small_
60             expected_counts)")
61         tests_run['fisher'] = result
62
63     # Mean test
64     elif metric_type == 'mean':
65         if check_assumptions:
66             # Check normality
67             norm1 = check_normality(data1)
68             norm2 = check_normality(data2)
69
70             # Check variance equality
71             var_eq = check_variance_equality(data1, data2, alpha)
72
73             # Detect outliers
74             outliers1 = detect_outliers(data1, method='iqr')
75             outliers2 = detect_outliers(data2, method='iqr')
76
77             normal1 = norm1['shapiro_normal']
78             normal2 = norm2['shapiro_normal']
79             equal_var = var_eq['levene_equal']
80             has_outliers = (outliers1['n_outliers'] > 0 or
81                             outliers2['n_outliers'] > 0)
82
83             tests_run['normality_check'] = {
84                 'group1_normal': normal1,
85                 'group2_normal': normal2,
86                 'variance_equal': equal_var,
87                 'has_outliers': has_outliers
88             }
89
90         if normal1 and normal2:
91             if equal_var:
92                 result = t_test_means(data1, data2, alpha, equal_
93                     var=True)
94                 recommendations.append("Use_pooled_t-test_(normal_
95                     data, equal variances)")
96             else:
97                 result = t_test_means(data1, data2, alpha, equal_
98                     var=False)
99                 recommendations.append("Use_Welch's_t-test_(normal_
100                     data, unequal variances)")
101             tests_run['t_test'] = result
102         else:
103             # Non-normal data
104             if n1 >= 30 and n2 >= 30:
105                 result_t = t_test_means(data1, data2, alpha, equal_
106                     var=False)
107                 result_mw = mann_whitney_test(data1, data2, alpha)
108
109                 recommendations.append("Large_samples:_t-test_
110                     likely_robust_(CLT),_"
111                     "but_consider_Mann-Whitney_if_
112                     heavily_skewed")
113                 tests_run['t_test'] = result_t
114                 tests_run['mann_whitney'] = result_mw
115             else:
116                 result_mw = mann_whitney_test(data1, data2, alpha)

```

```

109         result_perm = permutation_test(data1, data2,
110                                         n_permutations
111                                         =10000,
112                                         seed=42)
113
114         recommendations.append("Use_Mann-Whitney_U_test_or_
115                                permutation_test_
116                                "(non-normal,small_samples)")
117         tests_run['mann_whitney'] = result_mw
118         tests_run['permutation'] = result_perm
119     else:
120         # Default: Welch's t-test (generally robust)
121         result = t_test_means(data1, data2, alpha, equal_var=False)
122         recommendations.append("Use_Welch's_t-test_(default,robust
123                                choice)")
124         tests_run['t_test'] = result
125
126     # Ordinal or rank data
127     elif metric_type == 'ordinal':
128         result = mann_whitney_test(data1, data2, alpha)
129         recommendations.append("Use_Mann-Whitney_U_test_(ordinal_data)"
130                                )
131         tests_run['mann_whitney'] = result
132
133     # Count data
134     elif metric_type == 'count':
135         # Poisson-based tests or transformations
136         recommendations.append("Consider_(1)_Square-root_transform_+_t
137                                -test,_"
138                                "(2)_Log_transform_+_t-test,_(3)_Mann-
139                                Whitney,_"
140                                "(4)_Generalized_linear_model")
141
142         # Try multiple approaches
143         result_t = t_test_means(data1, data2, alpha, equal_var=False)
144         result_mw = mann_whitney_test(data1, data2, alpha)
145
146         tests_run['t_test'] = result_t
147         tests_run['mann_whitney'] = result_mw
148
149     return {
150         'metric_type': metric_type,
151         'recommendations': recommendations,
152         'tests_performed': list(tests_run.keys()),
153         'results': tests_run,
154         'sample_sizes': (n1, n2)
155     }
156
157 # Example usage
158 print("===_Test_Selection_Framework_===\n")
159
160 np.random.seed(42)
161
162 # Example 1: Binary outcome (conversion rate)
163 conversions_control = np.random.binomial(1, 0.10, 2000)
164 conversions_treatment = np.random.binomial(1, 0.12, 2000)
165
166 print("Example_1:_Conversion_Rate")

```

```

161 result1 = select_statistical_test(conversions_control, conversions_
    treatment,
162                                     metric_type='proportion')
163 print(f"Metric_type: {result1['metric_type']}")
164 print(f"Recommendations:")
165 for rec in result1['recommendations']:
166     print(f"__-{rec}")
167 print(f"Tests_performed: {', '.join(result1['tests_performed'])}\n")
168
169 # Example 2: Continuous metric (normal data)
170 revenue_control = np.random.normal(50, 10, 500)
171 revenue_treatment = np.random.normal(53, 10, 500)
172
173 print("Example_2: Revenue (Normal Distribution)")
174 result2 = select_statistical_test(revenue_control, revenue_treatment,
175                                   metric_type='mean', check_assumptions
                                   =True)
176 print(f"Recommendations:")
177 for rec in result2['recommendations']:
178     print(f"__-{rec}")
179 if 'normality_check' in result2['results']:
180     checks = result2['results']['normality_check']
181     print(f"Assumption_checks:")
182     print(f"__Group_1_normal: {checks['group1_normal']}")
183     print(f"__Group_2_normal: {checks['group2_normal']}")
184     print(f"__Variances_equal: {checks['variance_equal']}")
185 print()
186
187 # Example 3: Skewed data
188 pageviews_control = np.random.gamma(2, 5, 500)
189 pageviews_treatment = np.random.gamma(2.5, 5, 500)
190
191 print("Example_3: Page_Views (Skewed Distribution)")
192 result3 = select_statistical_test(pageviews_control, pageviews_
    treatment,
193                                   metric_type='mean', check_assumptions
                                   =True)
194 print(f"Recommendations:")
195 for rec in result3['recommendations']:
196     print(f"__-{rec}")

```

5.10 Common Pitfalls and How to Avoid Them

5.10.1 Pitfall 1: Peeking at Results

Problem: Continuously checking p-values and stopping when significant inflates Type I error.

Solution: Pre-specify sample size and duration. If monitoring needed, use group sequential testing methods (Chapter 8).

5.10.2 Pitfall 2: Multiple Testing Without Correction

Problem: Testing many metrics without adjustment increases false positives.

Solution: Apply Bonferroni or FDR correction (Chapter 6). Pre-specify primary metric.

5.10.3 Pitfall 3: Misinterpreting P-Values

Problem: Confusing $P(\text{data}|H_0)$ with $P(H_0|\text{data})$.

Solution: Report effect sizes and confidence intervals. Focus on practical significance.

5.10.4 Pitfall 4: Ignoring Assumptions

Problem: Using t-tests on heavily skewed data with small samples.

Solution: Check assumptions visually and statistically. Use non-parametric alternatives when needed.

5.10.5 Pitfall 5: Conflating Statistical and Practical Significance

Problem: Declaring tiny effects "significant" with huge samples, or dismissing important effects as "non-significant" with small samples.

Solution: Always report effect sizes. Use minimum detectable effect (MDE) framework from Chapter 4.

5.11 Summary

Hypothesis testing provides a rigorous framework for A/B test analysis:

- **Proportions:** Z-test for large samples, Fisher's exact for small
- **Means:** Welch's t-test as robust default
- **Categorical:** Chi-squared test (expected counts ≥ 5)
- **Assumptions matter:** Check normality, variance equality, independence
- **Non-parametric alternatives:** Mann-Whitney, permutation, bootstrap when assumptions fail
- **P-values are not enough:** Report effect sizes, confidence intervals, practical significance

Proper test selection and interpretation enable sound statistical inference and business decisions.

5.12 Further Reading

- Casella and Berger [2002]: Rigorous theoretical foundations
- Good [2013]: Comprehensive treatment of permutation and bootstrap methods
- Wasserstein and Lazar [2016]: ASA statement on p-values and statistical significance

5.13 Exercises

5.13.1 Conceptual Questions

1. Explain why p-value is $P(\text{data}|H_0)$ and not $P(H_0|\text{data})$. What additional information would be needed to calculate $P(H_0|\text{data})$?
2. When should you use Welch's t-test instead of pooled t-test? Why is Welch's often recommended as the default?
3. Describe a scenario where statistical significance ($p < 0.05$) exists but practical significance does not, and vice versa.
4. Why does "peeking" at A/B test results inflate Type I error? Sketch how error rate changes with number of peeks.

5.13.2 Calculation Exercises

1. Given: Control (n=1000, conversions=80), Treatment (n=1000, conversions=95)
 - (a) Calculate z-statistic and p-value for two-sided test
 - (b) Construct 95
 - (c) Calculate relative lift
 - (d) Verify $\chi^2 = Z^2$ using contingency table
2. Given: Control (n=500, mean=\$45.20, sd=\$28.30), Treatment (n=500, mean=\$48.70, sd=\$31.10)
 - (a) Perform Welch's t-test
 - (b) Calculate Cohen's d
 - (c) Construct 95
 - (d) Calculate variance ratio and assess assumption

5.13.3 Programming Exercises

1. Implement comprehensive test comparison framework:
 - Generate data from various distributions (normal, skewed, heavy-tailed)
 - Run t-test, Mann-Whitney, permutation test
 - Compare p-values and power across methods
 - Visualize results
2. Create assumption diagnostic dashboard:
 - Automated normality checks
 - Variance equality tests
 - Outlier detection
 - Visual diagnostics

- Test recommendations
3. Bootstrap confidence interval comparison:
 - Implement percentile and BCa methods
 - Compare to parametric CIs
 - Evaluate coverage rates via simulation
 - Test with various distributions

5.13.4 Applied Exercises

1. **Real data analysis:** Obtain an A/B test dataset (public or from your organization)
 - Perform comprehensive assumption checks
 - Run appropriate tests
 - Compare parametric and non-parametric methods
 - Report results with effect sizes and CIs
 - Make business recommendation
2. **Simulation study:** Investigate robustness of t-test
 - Generate data from increasingly non-normal distributions
 - Compare Type I error rates for t-test vs. Mann-Whitney
 - Vary sample sizes
 - Identify boundary where t-test breaks down
3. **Decision framework:** Create flowchart for test selection
 - Decision points based on metric type, sample size, assumptions
 - Implement as interactive tool
 - Test with various scenarios
 - Document edge cases

5.13.5 Advanced Projects

1. **Meta-analysis:** Analyze 100+ A/B tests from an organization
 - What proportion use appropriate tests?
 - How often are assumptions checked?
 - Do parametric and non-parametric yield different conclusions?
 - Recommend process improvements
2. **Power simulation:** Compare power across test types
 - Fix true effect size

-
- Vary distribution shape
 - Calculate empirical power for each test
 - Create power curves
 - Identify optimal test for each scenario

Chapter 6

Multiple Testing and False Discovery Rate

Learning Objectives

By the end of this chapter, you will be able to:

- Understand the multiple testing problem and its impact on false discovery rates
- Calculate and control family-wise error rate (FWER) using various methods
- Apply false discovery rate (FDR) control procedures appropriately
- Implement advanced approaches including closed testing and gatekeeping
- Apply Bayesian methods to multiple testing problems
- Design hierarchical testing strategies for complex experiments
- Manage platform-wide experimentation programs with multiple testing control
- Choose appropriate correction methods based on experimental context

6.1 The Multiple Testing Problem

In practice, A/B tests rarely involve testing a single hypothesis. Product teams typically monitor multiple metrics simultaneously—conversion rate, revenue per user, engagement time, bounce rate, and more. Similarly, multivariate tests compare multiple variants against a control. Each comparison represents a hypothesis test, and the more tests we conduct, the greater our risk of false discoveries.

6.1.1 The Birthday Paradox of Hypothesis Testing

Consider a seemingly simple scenario: you run an A/B test and measure 10 different metrics at $\alpha = 0.05$. If there are truly no differences between variants (all null hypotheses are true), what's the probability of finding at least one "statistically significant" result purely by chance?

Many practitioners incorrectly assume the answer is 5%, reasoning that each test has a 5% false positive rate. But this reasoning is flawed. Let's calculate the actual probability.

If each test has a 5% chance of a Type I error, then each test has a 95% chance of correctly not rejecting a true null hypothesis. The probability that all 10 tests correctly fail to reject is:

$$P(\text{no false positives}) = (0.95)^{10} \approx 0.599 \quad (6.1)$$

Therefore, the probability of at least one false positive is:

$$P(\text{at least one false positive}) = 1 - 0.599 = 0.401 \quad (6.2)$$

With 10 independent tests, we have a 40% chance of finding at least one spurious "significant" result! This phenomenon—analogue to the birthday paradox—is the multiple testing problem.

6.1.2 Family-Wise Error Rate (FWER)

Definition 6.1 (Family-Wise Error Rate). The family-wise error rate (FWER) is the probability of making at least one Type I error (false positive) among a family of m hypothesis tests:

$$\text{FWER} = P(\text{reject at least one true null hypothesis}) \quad (6.3)$$

A procedure that controls FWER ensures:

$$\text{FWER} \leq \alpha \quad (6.4)$$

for a specified significance level α .

FWER control is strict: it bounds the probability of making *any* false discovery. This is appropriate when false positives are particularly costly, but it can be conservative when testing many hypotheses.

6.1.3 False Discovery Rate (FDR)

Definition 6.2 (False Discovery Rate). Let V denote the number of false rejections (Type I errors) and R denote the total number of rejections. The false discovery rate is:

$$\text{FDR} = \mathbb{E} \left[\frac{V}{R} \right] \quad (6.5)$$

where by convention $V/R = 0$ if $R = 0$.

The FDR is the expected proportion of false discoveries among all discoveries.

Key Distinction:

- FWER controls the probability of *any* false positive: $P(V \geq 1) \leq \alpha$
- FDR controls the expected proportion of false positives: $\mathbb{E}[V/R] \leq \alpha$

FDR is less stringent than FWER, allowing more power to detect true effects at the cost of tolerating a small proportion of false discoveries.

6.1.4 Examples of Multiple Testing in A/B Contexts

1. **Multiple Metrics:** Testing conversion rate, revenue per user, engagement time, and bounce rate simultaneously
2. **Multiple Variants:** A/B/C/D test comparing 4 treatments against a control (5 comparisons)
3. **Sequential Testing:** Repeatedly checking test results (peeking) constitutes multiple testing across time
4. **Subgroup Analysis:** Testing effects across mobile vs. desktop, new vs. returning users, geographic regions
5. **Platform-Wide Programs:** Running hundreds of concurrent experiments on the same platform
6. **Exploratory Analysis:** Post-hoc investigation of unexpected patterns across multiple dimensions

Example 6.1 (E-commerce Multiple Testing Scenario). An e-commerce platform tests a new recommendation algorithm. They monitor:

- Primary: Conversion rate
- Secondary: Revenue per user, average order value, items per cart
- Guardrails: Page load time, error rate, returns

They also analyze effects separately for:

- New vs. returning users
- Mobile vs. desktop
- 3 geographic regions

Total comparisons: 7 metrics $\times$ 6 segments = 42 hypothesis tests!

Without correction, the probability of at least one false positive (if all nulls are true) is:

$$P(\text{at least one FP}) = 1 - (0.95)^{42} \approx 0.899 \quad (6.6)$$

Nearly 90% chance of finding a spurious "significant" result!

6.2 Correction Methods

6.2.1 Bonferroni Correction

The Bonferroni correction is the simplest and most conservative FWER control method.

Theorem 6.1 (Bonferroni Correction). To control FWER at level α when conducting m hypothesis tests, reject the i -th null hypothesis if its p-value satisfies:

$$p_i \leq \frac{\alpha}{m} \quad (6.7)$$

This procedure guarantees $\text{FWER} \leq \alpha$ regardless of the dependence structure among tests.

Proof. Let H_i denote the i -th null hypothesis, and let $\mathcal{H}_0$ be the set of true null hypotheses. By Boole's inequality (union bound):

$$\text{FWER} = P\left(\bigcup_{i \in \mathcal{H}_0} \{\text{reject } H_i\}\right) \quad (6.8)$$

$$\leq \sum_{i \in \mathcal{H}_0} P(\text{reject } H_i) \quad (6.9)$$

$$= \sum_{i \in \mathcal{H}_0} P(p_i \leq \alpha/m) \quad (6.10)$$

$$\leq \sum_{i \in \mathcal{H}_0} \frac{\alpha}{m} \quad (6.11)$$

$$\leq m \cdot \frac{\alpha}{m} = \alpha \quad (6.12)$$

□

Properties:

- **Conservative:** Often overly strict, especially for large m , leading to reduced power
- **Simple:** Easy to compute and explain
- **Universal:** Makes no assumptions about dependence between tests

6.2.2 Holm-Bonferroni Method

Holm's method improves upon Bonferroni by using a sequential testing procedure that is uniformly more powerful while still controlling FWER.

Theorem 6.2 (Holm's Step-Down Procedure). To control FWER at level α for m tests:

1. Order p-values: $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$
2. For $i = 1, 2, \dots, m$, reject hypothesis (i) if:

$$p_{(i)} \leq \frac{\alpha}{m - i + 1} \quad (6.13)$$

3. Stop at the first $p_{(i)}$ that fails this criterion; fail to reject all remaining hypotheses

This procedure controls $\text{FWER} \leq \alpha$ under any dependence structure.

The key insight is that once we fail to reject a hypothesis, we stop testing. This sequential approach is more powerful than Bonferroni because later tests use less stringent thresholds.

Example 6.2 (Holm's Procedure Application). Consider $m = 5$ tests at $\alpha = 0.05$ with ordered p-values:

i	$p_{(i)}$	$m - i + 1$	Threshold	Decision	Bonferroni
1	0.003	5	$0.05/5 = 0.010$	Reject	Reject
2	0.008	4	$0.05/4 = 0.0125$	Reject	Reject
3	0.015	3	$0.05/3 = 0.0167$	Reject	Fail to reject
4	0.042	2	$0.05/2 = 0.025$	Fail to reject	Fail to reject
5	0.091	1	$0.05/1 = 0.050$	Fail to reject	Fail to reject

Holm's procedure rejects 3 hypotheses, while Bonferroni rejects only 2.

Properties:

- **Uniformly more powerful:** Always rejects at least as many hypotheses as Bonferroni
- **Simple:** Easy to implement with sorted p-values
- **Robust:** Controls FWER under any dependence structure

6.2.3 Benjamini-Hochberg Procedure

The Benjamini-Hochberg (BH) procedure is the most widely used FDR control method.

Theorem 6.3 (Benjamini-Hochberg Procedure). To control FDR at level α for m tests (under independence or positive dependence):

1. Order p-values: $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$
2. Find the largest i such that:

$$p_{(i)} \leq \frac{i}{m}\alpha \quad (6.14)$$

3. Reject hypotheses $(1), (2), \dots, (i)$

Under independence or positive dependence, this procedure controls $\text{FDR} \leq \alpha$.

The key difference from FWER methods is that the threshold for hypothesis (i) is $i\alpha/m$ rather than $\alpha/(m - i + 1)$. This makes the procedure more liberal, allowing more discoveries.

Example 6.3 (Benjamini-Hochberg Application). Consider $m = 20$ tests at $\alpha = 0.05$. The BH-adjusted thresholds increase linearly:

i	$p_{(i)}$	BH Threshold	Bonferroni	Decision
1	0.001	0.0025	0.0025	Reject (BH & Bonf)
2	0.003	0.0050	0.0025	Reject (BH), Fail (Bonf)
3	0.004	0.0075	0.0025	Reject (BH), Fail (Bonf)
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
10	0.023	0.0250	0.0025	Reject (BH), Fail (Bonf)
11	0.031	0.0275	0.0025	Fail to reject (both)

BH rejects 10 hypotheses, while Bonferroni rejects only 1. Among the 10 BH rejections, we expect fewer than $0.05 \times 10 = 0.5$ false discoveries on average.

6.2.4 Benjamini-Yekutieli Method

The Benjamini-Yekutieli (BY) procedure extends FDR control to arbitrary dependence structures.

Theorem 6.4 (Benjamini-Yekutieli Procedure). To control FDR at level α for m tests under arbitrary dependence:

1. Order p-values: $p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(m)}$

2. Find the largest i such that:

$$p_{(i)} \leq \frac{i}{m \cdot c(m)} \alpha \quad (6.15)$$

where $c(m) = \sum_{j=1}^m \frac{1}{j} \approx \ln(m) + 0.577$ (harmonic sum)

3. Reject hypotheses $(1), (2), \dots, (i)$

This procedure controls $\text{FDR} \leq \alpha$ under arbitrary dependence.

The BY procedure is more conservative than BH due to the $c(m)$ correction, but it requires no assumptions about dependence structure.

6.2.5 Comparison of Basic Correction Methods

Method	Error Control	Assumptions	Power	Recommendation
Bonferroni	FWER	None	Low	Conservative baseline
Holm	FWER	None	Moderate-High	Default FWER method
BH	FDR	Independence	High	Default FDR method
BY	FDR	None	Moderate	Dependent tests

General Recommendation: Use Holm's procedure for FWER control and Benjamini-Hochberg for FDR control. These provide good power while maintaining appropriate error control.

6.3 Advanced Approaches

6.3.1 Closed Testing Procedures

Closed testing is a powerful framework that allows testing multiple hypotheses while controlling FWER, often with greater power than standard methods.

Definition 6.3 (Closed Testing Principle). For a family of hypotheses $\mathcal{H} = \{H_1, \dots, H_m\}$, a closed testing procedure tests each hypothesis H_i by:

1. Testing all intersection hypotheses that include H_i
2. Rejecting H_i only if all intersection hypotheses containing H_i are rejected at level α

This procedure controls FWER at level α regardless of dependence among tests.

Theorem 6.5 (Closed Testing FWER Control). Any closed testing procedure controls FWER at level α .

Sketch. Under the global null hypothesis (all individual nulls true), the closed testing procedure first tests the complete intersection hypothesis $H_1 \cap H_2 \cap \dots \cap H_m$. This is true under the global null, so it will be rejected with probability at most α . Therefore, $\text{FWER} \leq \alpha$. $\square$

Practical Implementation:

Closed testing provides a framework rather than a specific procedure. Common implementations include:

1. **Hommel's Procedure:** More powerful than Holm for singly ordered hypotheses
2. **Fallback Procedures:** Sequential testing with power transferred from rejected hypotheses
3. **Graphical Approaches:** Visual representation of testing strategies (Bretz et al., 2009)

Example 6.4 (Closed Testing for 3 Hypotheses). Testing H_1, H_2, H_3 at $\alpha = 0.05$:

Intersection hypotheses to test:

- H_{123} : Test $H_1 \cap H_2 \cap H_3$ at level 0.05
- H_{12} : Test $H_1 \cap H_2$ at level 0.05
- H_{13} : Test $H_1 \cap H_3$ at level 0.05
- H_{23} : Test $H_2 \cap H_3$ at level 0.05
- H_1, H_2, H_3 : Individual hypotheses at level 0.05

Reject H_1 only if all of $H_{123}, H_{12}, H_{13}, H_1$ are rejected.

This can be more powerful than Bonferroni because we can use the full α for each intersection test.

6.3.2 Gatekeeping Strategies

Gatekeeping procedures allow hierarchical testing where some hypotheses must be rejected before others can be tested, controlling FWER across the entire family.

Definition 6.4 (Gatekeeping Procedure). A gatekeeping procedure organizes hypotheses into ordered families $F_1, F_2, \dots, F_K$ where:

- Family F_k can only be tested if certain conditions in families $F_1, \dots, F_{k-1}$ are met
- FWER is controlled at level α across all families

Types of Gatekeeping:

1. **Serial Gatekeeping:** All hypotheses in family F_k must be rejected to proceed to F_{k+1}

2. **Parallel Gatekeeping:** At least one hypothesis in F_k must be rejected to proceed
3. **Tree-Structured Gatekeeping:** Complex hierarchical structures with multiple branches

Example 6.5 (Serial Gatekeeping for Clinical Trial). A pharmaceutical trial tests:

- Family 1 (Primary): Efficacy on primary endpoint
- Family 2 (Secondary): Efficacy on secondary endpoints
- Family 3 (Exploratory): Safety and quality of life

Gatekeeping rules:

1. Test Family 1 at $\alpha = 0.05$
2. If Family 1 is rejected, test Family 2 at $\alpha = 0.05$ using Holm's procedure
3. If at least one hypothesis in Family 2 is rejected, test Family 3 at $\alpha = 0.05$ using BH

This controls overall FWER at 0.05 while allowing exploration of secondary outcomes.

Application to A/B Testing:

Listing 6.1: Gatekeeping Procedure Implementation

```

1 import numpy as np
2 from scipy import stats
3 from typing import Dict, List, Tuple
4
5 def serial_gatekeeping(
6     families: List[np.ndarray],
7     family_tests: List[str],
8     alpha: float = 0.05
9 ) -> Dict:
10     """
11     Implement serial gatekeeping procedure.
12
13     Parameters:
14     -----
15     families: List of np.ndarray
16     family_tests: List of p-value arrays, one per family
17     family_tests: List of str
18     Testing procedure for each family: 'bonferroni', 'holm', 'bh'
19     alpha: float
20     Overall FWER level
21
22     Returns:
23     -----
24     dict: Results for each family
25     """
26     results = {}
27     proceed = True
28
29     for i, (pvalues, method) in enumerate(zip(families, family_tests)):
30         family_name = f'Family_{i+1}'

```

```

31
32     if not proceed:
33         # Cannot test this family
34         results[family_name] = {
35             'tested': False,
36             'reject': np.zeros(len(pvalues), dtype=bool),
37             'message': 'Gatekeeper_not_satisfied'
38         }
39         continue
40
41     # Test this family
42     if method == 'bonferroni':
43         reject = pvalues <= alpha / len(pvalues)
44     elif method == 'holm':
45         from statsmodels.stats.multitest import multipletests
46         reject, pvals_adj, _, _ = multipletests(
47             pvalues, alpha=alpha, method='holm'
48         )
49     elif method == 'bh':
50         from statsmodels.stats.multitest import multipletests
51         reject, pvals_adj, _, _ = multipletests(
52             pvalues, alpha=alpha, method='fdr_bh'
53         )
54     else:
55         raise ValueError(f"Unknown_method:_{method}")
56
57     results[family_name] = {
58         'tested': True,
59         'reject': reject,
60         'n_rejected': np.sum(reject),
61         'method': method
62     }
63
64     # Check if we can proceed to next family (serial gatekeeping)
65     # Require all hypotheses in current family to be rejected
66     if not np.all(reject):
67         proceed = False
68
69     return results
70
71
72 def parallel_gatekeeping(
73     families: List[np.ndarray],
74     family_tests: List[str],
75     alpha: float = 0.05,
76     min_rejections: int = 1
77 ) -> Dict:
78     """
79     Implement parallel gatekeeping procedure.
80
81     Parameters:
82     -----
83     families: list of np.ndarray
84             List of p-value arrays, one per family
85     family_tests: list of str
86             Testing procedure for each family
87     alpha: float
88             Overall FWER level

```

```

89  """min_rejections: int
90  """Minimum rejections needed to open gate
91
92  Returns:
93  """-----
94  dict: Results for each family
95  """
96      results = {}
97      proceed = True
98
99      for i, (pvalues, method) in enumerate(zip(families, family_tests)):
100          family_name = f'Family_{i+1}'
101
102          if not proceed:
103              results[family_name] = {
104                  'tested': False,
105                  'reject': np.zeros(len(pvalues), dtype=bool),
106                  'message': 'Gatekeeper not satisfied'
107              }
108              continue
109
110          # Test this family
111          if method == 'bonferroni':
112              reject = pvalues <= alpha / len(pvalues)
113          elif method == 'holm':
114              from statsmodels.stats.multitest import multipletests
115              reject, pvals_adj, _, _ = multipletests(
116                  pvalues, alpha=alpha, method='holm'
117              )
118          elif method == 'bh':
119              from statsmodels.stats.multitest import multipletests
120              reject, pvals_adj, _, _ = multipletests(
121                  pvalues, alpha=alpha, method='fdr_bh'
122              )
123
124          results[family_name] = {
125              'tested': True,
126              'reject': reject,
127              'n_rejected': np.sum(reject),
128              'method': method
129          }
130
131          # Check if we can proceed (parallel gatekeeping)
132          # Require at least min_rejections
133          if np.sum(reject) < min_rejections:
134              proceed = False
135
136      return results
137
138
139  # Example usage
140  np.random.seed(42)
141
142  # Simulate three families of hypotheses
143  family1 = np.array([0.001, 0.003]) # Primary endpoints
144  family2 = np.array([0.015, 0.025, 0.045]) # Secondary endpoints
145  family3 = np.array([0.060, 0.080, 0.120, 0.150]) # Exploratory
146

```

```

147 families = [family1, family2, family3]
148 methods = ['holm', 'holm', 'bh']
149
150 # Serial gatekeeping
151 results_serial = serial_gatekeeping(families, methods, alpha=0.05)
152
153 print("Serial_Gatekeeping_Results:")
154 for family, res in results_serial.items():
155     if res['tested']:
156         print(f"{family}: {res['n_rejected']} rejections")
157     else:
158         print(f"{family}: {res['message']}")
159
160 # Parallel gatekeeping
161 results_parallel = parallel_gatekeeping(
162     families, methods, alpha=0.05, min_rejections=1
163 )
164
165 print("\nParallel_Gatekeeping_Results:")
166 for family, res in results_parallel.items():
167     if res['tested']:
168         print(f"{family}: {res['n_rejected']} rejections")
169     else:
170         print(f"{family}: {res['message']}")

```

6.3.3 Adaptive Procedures

Adaptive procedures estimate the proportion of true null hypotheses and adjust the rejection threshold accordingly, potentially gaining power.

Definition 6.5 (Adaptive Benjamini-Hochberg). Let $\hat{\pi}_0$ denote an estimate of the proportion of true null hypotheses. The adaptive BH procedure:

1. Estimate π_0 from the p-value distribution
2. Find the largest i such that:

$$p_{(i)} \leq \frac{i}{m \cdot \hat{\pi}_0} \alpha \quad (6.16)$$

3. Reject hypotheses $(1), (2), \dots, (i)$

Under appropriate conditions, this controls FDR at level $\alpha \cdot \pi_0 \leq \alpha$.

Estimating π_0 :

Several estimators exist:

1. **Storey's Estimator:**

$$\hat{\pi}_0(\lambda) = \frac{\#\{p_i > \lambda\}}{m(1 - \lambda)} \quad (6.17)$$

for some $\lambda \in [0, 1)$ (typically 0.5)

2. **Least Slope Estimator:** Based on the slope of the p-value distribution
3. **Histogram-Based:** Estimate from the flat portion of the p-value histogram

Listing 6.2: Adaptive FDR Control

```

1 def estimate_pi0(pvalues: np.ndarray, lambda_val: float = 0.5) -> float
2     :
3     """
4     Estimate proportion of true nulls using Storey's method.
5
6     Parameters:
7     -----
8     pvalues: np.ndarray
9         Array of p-values
10    lambda_val: float
11        Threshold for estimation (default 0.5)
12
13    Returns:
14    -----
15    float: Estimated proportion of true nulls
16    """
17    m = len(pvalues)
18    num_above = np.sum(pvalues > lambda_val)
19    pi0_hat = num_above / (m * (1 - lambda_val))
20
21    # Ensure estimate is in [0, 1]
22    return min(pi0_hat, 1.0)
23
24 def adaptive_bh(
25     pvalues: np.ndarray,
26     alpha: float = 0.05,
27     lambda_val: float = 0.5
28 ) -> Dict:
29     """
30     Adaptive Benjamini-Hochberg procedure.
31
32     Parameters:
33     -----
34     pvalues: np.ndarray
35         Array of p-values
36     alpha: float
37         FDR level
38     lambda_val: float
39         Threshold for pi0 estimation
40
41    Returns:
42    -----
43    dict: Results including pi0 estimate
44    """
45    m = len(pvalues)
46
47    # Estimate pi0
48    pi0_hat = estimate_pi0(pvalues, lambda_val)
49
50    # Sort p-values
51    sort_idx = np.argsort(pvalues)
52    pvalues_sorted = pvalues[sort_idx]
53
54    # Adaptive BH threshold
55    reject_sorted = np.zeros(m, dtype=bool)
56    rejection_idx = -1

```

```

57
58     for i in range(m - 1, -1, -1):
59         threshold = (i + 1) / (m * pi0_hat) * alpha
60         if pvalues_sorted[i] <= threshold:
61             rejection_idx = i
62             break
63
64     if rejection_idx >= 0:
65         reject_sorted[:rejection_idx + 1] = True
66
67     # Unsort
68     reject = np.zeros(m, dtype=bool)
69     reject[sort_idx] = reject_sorted
70
71     return {
72         'method': 'Adaptive_BH',
73         'pi0_estimate': pi0_hat,
74         'alpha': alpha,
75         'reject': reject,
76         'n_reject': np.sum(reject),
77         'rejected_indices': np.where(reject)[0].tolist()
78     }
79
80
81 # Example: Compare standard BH vs adaptive BH
82 np.random.seed(123)
83
84 # Simulate p-values: 80% true nulls, 20% alternatives
85 m = 100
86 n_true_nulls = 80
87 n_alternatives = 20
88
89 true_null_pvals = np.random.uniform(0, 1, n_true_nulls)
90 alternative_pvals = np.random.beta(0.5, 5, n_alternatives) # Enriched
91                     for small p-values
92
93 pvalues = np.concatenate([true_null_pvals, alternative_pvals])
94 np.random.shuffle(pvalues)
95
96 # Standard BH
97 from statsmodels.stats.multitest import multipletests
98 reject_bh, _, _, _ = multipletests(pvalues, alpha=0.05, method='fdr_bh',
99                                   )
100
101 # Adaptive BH
102 results_adaptive = adaptive_bh(pvalues, alpha=0.05, lambda_val=0.5)
103
104 print(f"True pi0: 0.80")
105 print(f"Estimated pi0: {results_adaptive['pi0_estimate']:.3f}")
106 print(f"Standard BH rejections: {np.sum(reject_bh)}")
107 print(f"Adaptive BH rejections: {results_adaptive['n_reject']}")

```

6.3.4 Bayesian Multiple Testing

Bayesian approaches provide a principled alternative to frequentist multiple testing, particularly useful when prior information is available.

Definition 6.6 (Bayesian FDR). The Bayesian false discovery rate for a set of rejected hypotheses $\mathcal{R}$ is:

$$\text{FDR}_{\text{Bayes}}(\mathcal{R}) = \mathbb{E} \left[\frac{|\mathcal{R} \cap \mathcal{H}_0|}{|\mathcal{R}|} \mid \text{data} \right] \quad (6.18)$$

where $\mathcal{H}_0$ is the set of true null hypotheses.

Local FDR:

The local false discovery rate for hypothesis i is the posterior probability that the null is true:

$$\text{lfdr}_i = P(H_i \text{ is true} \mid p_i) \quad (6.19)$$

Decision Rules:

Reject hypothesis i if:

$$\text{lfdr}_i \leq \tau \quad (6.20)$$

for some threshold τ chosen to control FDR.

Listing 6.3: Bayesian Multiple Testing

```

1 from scipy.stats import beta, norm
2
3 def compute_local_fdr(
4     pvalues: np.ndarray,
5     pi0: float = 0.5,
6     alternative_params: Tuple[float, float] = (0.5, 5.0)
7 ) -> np.ndarray:
8     """
9     Compute local false discovery rate.
10
11     Parameters:
12     -----
13     pvalues: np.ndarray
14             Array of p-values
15     pi0: float
16             Prior probability of null hypothesis
17     alternative_params: tuple
18             Beta distribution parameters for alternative
19
20     Returns:
21     -----
22     np.ndarray: Local FDR for each hypothesis
23     """
24     m = len(pvalues)
25
26     # Likelihood under null: uniform(0, 1)
27     f0 = np.ones(m)
28
29     # Likelihood under alternative: beta distribution
30     a, b = alternative_params
31     f1 = beta.pdf(pvalues, a, b)
32
33     # Marginal likelihood
34     f_marginal = pi0 * f0 + (1 - pi0) * f1
35
36     # Local FDR via Bayes' rule
37     lfdr = (pi0 * f0) / f_marginal
38

```



```

39     return lfdr
40
41
42 def bayesian_fdr_control(
43     pvalues: np.ndarray,
44     alpha: float = 0.05,
45     pi0: float = None,
46     alternative_params: Tuple[float, float] = (0.5, 5.0)
47 ) -> Dict:
48     """
49     Bayesians FDR control using local FDR.
50
51     Parameters:
52     -----
53     pvalues: np.ndarray
54             Array of p-values
55     alpha: float
56             FDR level
57     pi0: float
58             Prior probability of null (estimated if None)
59     alternative_params: tuple
60             Beta parameters for alternative distribution
61
62     Returns:
63     -----
64     dict: Results including local FDR values
65     """
66     m = len(pvalues)
67
68     # Estimate pi0 if not provided
69     if pi0 is None:
70         pi0 = estimate_pi0(pvalues, lambda_val=0.5)
71
72     # Compute local FDR
73     lfdr = compute_local_fdr(pvalues, pi0, alternative_params)
74
75     # Sort by local FDR
76     sort_idx = np.argsort(lfdr)
77     lfdr_sorted = lfdr[sort_idx]
78
79     # Find rejection threshold
80     # Reject if cumulative average lfdr <= alpha
81     reject_sorted = np.zeros(m, dtype=bool)
82     rejection_idx = -1
83
84     for i in range(m):
85         avg_lfdr = np.mean(lfdr_sorted[:i+1])
86         if avg_lfdr <= alpha:
87             rejection_idx = i
88         else:
89             break
90
91     if rejection_idx >= 0:
92         reject_sorted[:rejection_idx + 1] = True
93
94     # Unsort
95     reject = np.zeros(m, dtype=bool)
96     reject[sort_idx] = reject_sorted

```

```

97     return {
98         'method': 'Bayesian_FDR',
99         'pi0': pi0,
100         'alpha': alpha,
101         'local_fdr': lfdr,
102         'reject': reject,
103         'n_reject': np.sum(reject),
104         'rejected_indices': np.where(reject)[0].tolist(),
105         'avg_lfdr': np.mean(lfdr[reject]) if np.sum(reject) > 0 else
106             0.0
107     }
108
109
110 # Example: Bayesian FDR control
111 np.random.seed(456)
112
113 # Simulate p-values
114 m = 50
115 n_true_nulls = 40
116 n_alternatives = 10
117
118 true_null_pvals = np.random.uniform(0, 1, n_true_nulls)
119 alternative_pvals = np.random.beta(0.5, 5, n_alternatives)
120
121 pvalues = np.concatenate([true_null_pvals, alternative_pvals])
122 np.random.shuffle(pvalues)
123
124 # Bayesian FDR control
125 results_bayes = bayesian_fdr_control(
126     pvalues, alpha=0.05, pi0=None, alternative_params=(0.5, 5.0)
127 )
128
129 print(f"Estimated pi0: {results_bayes['pi0']:.3f}")
130 print(f"Rejections: {results_bayes['n_reject']}")
131 print(f"Average local FDR of rejections: {results_bayes['avg_lfdr']:.3f}")
132
133 # Compare with standard BH
134 reject_bh, _, _, _ = multipletests(pvalues, alpha=0.05, method='fdr_bh')
135 print(f"Standard BH rejections: {np.sum(reject_bh)}")

```

6.4 Practical Applications

6.4.1 Multiple Metrics Testing

Most A/B tests monitor multiple metrics simultaneously. Proper handling requires thoughtful strategy.

Metric Hierarchy:

1. **Primary Metrics (1-2):** Decision-making metrics. Test at $\alpha = 0.05$ without correction.

2. **Secondary Metrics (3-5):** Supporting evidence. Apply FDR control (BH) at $\alpha = 0.05$.
3. **Guardrail Metrics (5-10):** Monitor for harm. Use conservative FWER control (Holm).
4. **Exploratory Metrics (unlimited):** Hypothesis generation only. Report descriptively.

Example 6.6 (Multi-Metric A/B Test Strategy). E-commerce checkout redesign:

Primary:

- Conversion rate ($p = 0.012$)

Decision: $p < 0.05$, so primary is significant. Proceed with secondary analysis.

Secondary:

- Revenue per user ($p = 0.045$)
- Items per cart ($p = 0.123$)
- Average order value ($p = 0.038$)

BH correction at $\alpha = 0.05$:

- Ordered p-values: 0.038, 0.045, 0.123
- Thresholds: $1/3 \times 0.05 = 0.0167$, $2/3 \times 0.05 = 0.0333$, $3/3 \times 0.05 = 0.05$
- No rejections (all p-values exceed thresholds)

Guardrails:

- Page load time ($p = 0.234$)
- Error rate ($p = 0.456$)
- Return rate ($p = 0.089$)

Holm correction: All p-values $> 0.05/3 = 0.0167$, so no harm detected.

Decision: Launch redesign based on significant primary metric. Secondary metrics show directionally positive but non-significant effects. No harm on guardrails.

6.4.2 Subgroup Analysis

Testing effects across multiple subgroups requires multiple testing correction to avoid false subgroup discoveries.

Pre-Planned Subgroups:

When subgroups are specified a priori, apply appropriate corrections:

1. **Few Subgroups (2-4):** Use Holm's procedure for FWER control
2. **Many Subgroups (5+):** Use BH procedure for FDR control

3. **Hierarchical Subgroups:** Use gatekeeping (e.g., test overall effect, then subgroups)

Post-Hoc Subgroups:

Exploratory subgroup analysis after seeing results is particularly prone to false discoveries. Strategies:

1. **Report Descriptively:** Don't make claims without correction
2. **Strict Correction:** Use Bonferroni or BY for strong FWER/FDR control
3. **Validation:** Plan follow-up experiment targeting identified subgroups

Listing 6.4: Subgroup Analysis with Multiple Testing

```

1 def subgroup_analysis(
2     data: np.ndarray,
3     treatment: np.ndarray,
4     subgroups: np.ndarray,
5     correction: str = 'holm',
6     alpha: float = 0.05
7 ) -> Dict:
8     """
9     Perform subgroup analysis with multiple testing correction.
10
11     Parameters:
12     -----
13     data: np.ndarray
14         Outcome data
15     treatment: np.ndarray
16         Treatment assignment (0=control, 1=treatment)
17     subgroups: np.ndarray
18         Subgroup labels
19     correction: str
20         Correction method: 'holm', 'bh', or 'bonferroni'
21     alpha: float
22         Significance level
23
24     Returns:
25     -----
26     dict: Results for each subgroup
27     """
28     from scipy.stats import ttest_ind
29
30     unique_subgroups = np.unique(subgroups)
31     n_subgroups = len(unique_subgroups)
32     pvalues = np.zeros(n_subgroups)
33     effects = np.zeros(n_subgroups)
34
35     # Test each subgroup
36     for i, subgroup in enumerate(unique_subgroups):
37         mask = subgroups == subgroup
38         control_data = data[mask & (treatment == 0)]
39         treatment_data = data[mask & (treatment == 1)]
40
41         # T-test for difference
42         t_stat, p_val = ttest_ind(treatment_data, control_data)

```

```

43     pvalues[i] = p_val
44     effects[i] = np.mean(treatment_data) - np.mean(control_data)
45
46     # Apply correction
47     if correction == 'bonferroni':
48         reject = pvalues <= alpha / n_subgroups
49         pvals_adj = np.minimum(pvalues * n_subgroups, 1.0)
50     elif correction == 'holm':
51         from statsmodels.stats.multitest import multipletests
52         reject, pvals_adj, _, _ = multipletests(
53             pvalues, alpha=alpha, method='holm'
54         )
55     elif correction == 'bh':
56         from statsmodels.stats.multitest import multipletests
57         reject, pvals_adj, _, _ = multipletests(
58             pvalues, alpha=alpha, method='fdr_bh'
59         )
60     else:
61         raise ValueError(f"Unknown correction: {correction}")
62
63     # Compile results
64     results = {}
65     for i, subgroup in enumerate(unique_subgroups):
66         results[str(subgroup)] = {
67             'effect': effects[i],
68             'pvalue': pvalues[i],
69             'pvalue_adj': pvals_adj[i],
70             'significant': reject[i]
71         }
72
73     return {
74         'subgroup_results': results,
75         'n_subgroups': n_subgroups,
76         'n_significant': np.sum(reject),
77         'correction': correction,
78         'alpha': alpha
79     }
80
81
82 # Example: Mobile vs Desktop analysis
83 np.random.seed(789)
84
85 # Simulate data: 3 subgroups (mobile, desktop, tablet)
86 n_per_group = 500
87 n_subgroups = 3
88
89 data_list = []
90 treatment_list = []
91 subgroup_list = []
92
93 for subgroup_id in range(n_subgroups):
94     # Control group
95     control = np.random.normal(10, 2, n_per_group)
96     # Treatment group (slight effect in subgroup 0 only)
97     if subgroup_id == 0:
98         treatment = np.random.normal(10.5, 2, n_per_group) # Real
99         effect
100     else:

```

```

100     treatment = np.random.normal(10, 2, n_per_group) # No effect
101
102     data_list.extend(list(control) + list(treatment))
103     treatment_list.extend([0] * n_per_group + [1] * n_per_group)
104     subgroup_list.extend([subgroup_id] * (2 * n_per_group))
105
106 data = np.array(data_list)
107 treatment = np.array(treatment_list)
108 subgroups = np.array(subgroup_list)
109
110 # Analyze with different corrections
111 for method in ['bonferroni', 'holm', 'bh']:
112     results = subgroup_analysis(
113         data, treatment, subgroups, correction=method, alpha=0.05
114     )
115     print(f"\n{method.upper()} Correction:")
116     print(f"Significant subgroups: {results['n_significant']}/{
117         results['n_subgroups']}")
118     for subgroup, res in results['subgroup_results'].items():
119         if res['significant']:
120             print(f"Subgroup {subgroup}: effect={res['effect']:.3f},
121                 p={res['pvalue']:.4f}")

```

6.4.3 Sequential Testing Considerations

A special case of multiple testing arises when experimenters repeatedly check test results—the "peeking" problem.

Why Peeking Inflates Type I Error:

Each peek constitutes a hypothesis test. With k peeks, the overall Type I error can be approximated by:

$$\text{FWER}_{\text{peeks}} \approx 1 - (1 - \alpha)^k \quad (6.21)$$

Example 6.7 (Peeking Simulation). An A/B test where the null is true (no difference):

- Look once at the end: $P(\text{false positive}) = 0.05$
- Peek 5 times: $P(\text{false positive}) \approx 0.23$
- Peek 10 times: $P(\text{false positive}) \approx 0.40$
- Continuous monitoring: $P(\text{false positive}) \rightarrow 1$

Solutions:

1. **Pre-Plan Sample Size:** Calculate required n , collect data, analyze once
2. **Multiple Testing Correction:** If peeking at k time points, apply Bonferroni: α/k per peek
3. **Sequential Testing Procedures:** Group sequential methods with optimized boundaries (Chapter 8)
4. **Always-Valid P-Values:** Modern methods that remain valid under continuous monitoring

6.4.4 Platform-Wide Experiment Programs

Large technology companies run hundreds of experiments concurrently, creating platform-level multiple testing challenges.

Challenges:

1. **Volume:** Hundreds of tests per week
2. **Independence:** Some experiments affect overlapping metrics
3. **Coordination:** Decentralized teams running experiments
4. **Standards:** Maintaining consistent statistical practices

Solutions:

1. **Automatic Correction:** Platform applies FDR control by default
2. **Metric Registries:** Pre-registered metrics with designated hierarchies
3. **Experiment Review:** Statistical review before launch and analysis
4. **Meta-Analysis:** Periodic review of false discovery rates empirically
5. **Education:** Training programs on multiple testing for all experimenters

Example 6.8 (Google's Approach). Google's experimentation platform implements:

- Automatic FDR control using BH for secondary metrics
- Pre-registered primary metrics tested without correction
- Guardrail metrics with strict FWER control
- Dashboards showing adjusted p-values and q-values
- Annual reviews of platform-wide false discovery rates

6.5 Implementation Strategies

6.5.1 Pre-Planned vs. Exploratory Analysis

The distinction between confirmatory (pre-planned) and exploratory analysis is crucial for appropriate multiple testing control.

Pre-Planned (Confirmatory):

- Hypotheses specified before data collection
- Metrics, subgroups, and comparisons pre-registered
- Standard FWER or FDR control applied
- Results can support decision-making

Exploratory:

- Hypotheses generated from data patterns
- Post-hoc subgroup analyses
- Requires stricter correction or validation
- Results generate hypotheses for future testing

Mixed Strategy:

1. **Primary Analysis:** Pre-specified hypotheses with standard corrections
2. **Exploratory Analysis:** Additional investigations with:
 - Strict corrections (Bonferroni, BY)
 - Clear labeling as exploratory
 - Plan for validation experiments

Listing 6.5: Pre-Planned vs Exploratory Framework

```

1 class ExperimentAnalysis:
2     """
3     Framework for managing pre-planned and exploratory analyses.
4     """
5
6     def __init__(self, alpha: float = 0.05):
7         self.alpha = alpha
8         self.preregistered = {
9             'primary_metrics': [],
10            'secondary_metrics': [],
11            'guardrail_metrics': [],
12            'subgroups': []
13        }
14        self.exploratory_analyses = []
15
16    def register_primary(self, metric_names: List[str]):
17        """Register primary metrics (no correction needed)."""
18        self.preregistered['primary_metrics'].extend(metric_names)
19
20    def register_secondary(self, metric_names: List[str]):
21        """Register secondary metrics (FDR control)."""
22        self.preregistered['secondary_metrics'].extend(metric_names)
23
24    def register_guardrails(self, metric_names: List[str]):
25        """Register guardrail metrics (FWER control)."""
26        self.preregistered['guardrail_metrics'].extend(metric_names)
27
28    def register_subgroups(self, subgroup_names: List[str]):
29        """Register pre-planned subgroups."""
30        self.preregistered['subgroups'].extend(subgroup_names)
31
32    def analyze_primary(self, pvalues: Dict[str, float]) -> Dict:
33        """
34        Analyze primary metrics without correction.

```



```

35
36 #####Parameters:
37 #####-----
38 #####pvalues_:dict
39 #####Metric_name_to_p-value_mapping
40
41 #####Returns:
42 #####-----
43 #####dict_:Analysis_results
44 #####"""
45     results = {}
46     for metric in self.preregistered['primary_metrics']:
47         if metric in pvalues:
48             results[metric] = {
49                 'pvalue': pvalues[metric],
50                 'significant': pvalues[metric] < self.alpha,
51                 'correction': 'none',
52                 'type': 'primary'
53             }
54     return results
55
56     def analyze_secondary(self, pvalues: Dict[str, float]) -> Dict:
57         """
58         #####Analyze_secondary_metrics_with_BH_correction.
59
60         #####Parameters:
61         #####-----
62         #####pvalues_:dict
63         #####Metric_name_to_p-value_mapping
64
65         #####Returns:
66         #####-----
67         #####dict_:Analysis_results
68         #####"""
69         metric_names = self.preregistered['secondary_metrics']
70         pval_array = np.array([pvalues[m] for m in metric_names if m in
71                                pvalues])
72
73         if len(pval_array) == 0:
74             return {}
75
76         # BH correction
77         from statsmodels.stats.multitest import multipletests
78         reject, pvals_adj, _, _ = multipletests(
79             pval_array, alpha=self.alpha, method='fdr_bh'
80         )
81
82         results = {}
83         for i, metric in enumerate(metric_names):
84             if metric in pvalues:
85                 results[metric] = {
86                     'pvalue': pval_array[i],
87                     'pvalue_adj': pvals_adj[i],
88                     'significant': reject[i],
89                     'correction': 'BH',
90                     'type': 'secondary'
91                 }
92         return results

```

```

92     def analyze_guardrails(self, pvalues: Dict[str, float]) -> Dict:
93         """
94         """
95         """Analyze guardrail metrics with Holm correction.
96
97         Parameters:
98         -----
99         pvalues: dict
100             Metric name to p-value mapping
101
102         Returns:
103         -----
104         dict: Analysis results
105         """
106         metric_names = self.preregistered['guardrail_metrics']
107         pval_array = np.array([pvalues[m] for m in metric_names if m in
108                               pvalues])
109
110         if len(pval_array) == 0:
111             return {}
112
113         # Holm correction
114         from statsmodels.stats.multitest import multipletests
115         reject, pvals_adj, _, _ = multipletests(
116             pval_array, alpha=self.alpha, method='holm'
117         )
118
119         results = {}
120         for i, metric in enumerate(metric_names):
121             if metric in pvalues:
122                 results[metric] = {
123                     'pvalue': pval_array[i],
124                     'pvalue_adj': pvals_adj[i],
125                     'significant': reject[i],
126                     'correction': 'Holm',
127                     'type': 'guardrail'
128                 }
129         return results
130
131     def add_exploratory(
132         self,
133         analysis_name: str,
134         pvalues: Dict[str, float],
135         correction: str = 'bonferroni'
136     ) -> Dict:
137         """
138         """
139         """Add exploratory analysis with strict correction.
140
141         Parameters:
142         -----
143         analysis_name: str
144             Name for this exploratory analysis
145         pvalues: dict
146             Metric/hypothesis to p-value mapping
147         correction: str
148             Correction method ('bonferroni' or 'by')
149
150         Returns:
151         """

```

```

149 #####-----
150 #####dict_: Analysis_results
151 #####"""
152     pval_array = np.array(list(pvalues.values()))
153     metric_names = list(pvalues.keys())
154
155     # Strict correction for exploratory
156     if correction == 'bonferroni':
157         m = len(pval_array)
158         reject = pval_array <= self.alpha / m
159         pvals_adj = np.minimum(pval_array * m, 1.0)
160     elif correction == 'by':
161         from statsmodels.stats.multitest import multipletests
162         reject, pvals_adj, _, _ = multipletests(
163             pval_array, alpha=self.alpha, method='fdr_by'
164         )
165     else:
166         raise ValueError(f"Unknown correction: {correction}")
167
168     results = {}
169     for i, metric in enumerate(metric_names):
170         results[metric] = {
171             'pvalue': pval_array[i],
172             'pvalue_adj': pvals_adj[i],
173             'significant': reject[i],
174             'correction': correction,
175             'type': 'exploratory',
176             'validation_needed': True
177         }
178
179     self.exploratory_analyses.append({
180         'name': analysis_name,
181         'results': results
182     })
183
184     return results
185
186 def generate_report(self, all_pvalues: Dict[str, float]) -> str:
187     """Generate comprehensive analysis report."""
188     report = []
189     report.append("=" * 60)
190     report.append("EXPERIMENT ANALYSIS REPORT")
191     report.append("=" * 60)
192
193     # Primary metrics
194     report.append("\nPRIMARY METRICS (No correction):")
195     primary_results = self.analyze_primary(all_pvalues)
196     for metric, res in primary_results.items():
197         sig_str = "SIGNIFICANT" if res['significant'] else "Not_
            significant"
198         report.append(f"  {metric}: p={res['pvalue']:.4f} [{sig_str}]")
199
200     # Secondary metrics
201     report.append("\nSECONDARY METRICS (BH correction):")
202     secondary_results = self.analyze_secondary(all_pvalues)
203     for metric, res in secondary_results.items():
204         sig_str = "SIGNIFICANT" if res['significant'] else "Not_

```

```

205         significant"
206         report.append(
207             f"_{metric}:_p={res['pvalue']:.4f},_"
208             f"q={res['pvalue_adj']:.4f}_{sig_str}"
209         )
210
211     # Guardrails
212     report.append("\nGUARDRAIL_METRICS_(Holm_correction):")
213     guardrail_results = self.analyze_guardrails(all_pvalues)
214     for metric, res in guardrail_results.items():
215         sig_str = "HARM_DETECTED" if res['significant'] else "No_
216             harm"
217         report.append(
218             f"_{metric}:_p={res['pvalue']:.4f},_"
219             f"p_adj={res['pvalue_adj']:.4f}_{sig_str}"
220         )
221
222     # Exploratory
223     if self.exploratory_analyses:
224         report.append("\nEXPLORATORY_ANALYSES:")
225         for analysis in self.exploratory_analyses:
226             report.append(f"\n_{analysis['name']}:")
227             for metric, res in analysis['results'].items():
228                 sig_str = "SIGNIFICANT*" if res['significant'] else
229                     "Not_significant"
230             report.append(
231                 f"_{metric}:_p={res['pvalue']:.4f},_"
232                 f"p_adj={res['pvalue_adj']:.4f}_{sig_str}"
233             )
234         report.append("\n*_Exploratory_findings_require_
235             validation_in_follow-up_experiments")
236
237     report.append("\n" + "=" * 60)
238     return "\n".join(report)
239
240 # Example usage
241 np.random.seed(999)
242
243 # Initialize framework
244 experiment = ExperimentAnalysis(alpha=0.05)
245
246 # Pre-register metrics
247 experiment.register_primary(['conversion_rate'])
248 experiment.register_secondary(['revenue_per_user', 'session_duration',
249     'bounce_rate'])
250 experiment.register_guardrails(['page_load_time', 'error_rate'])
251
252 # Simulate p-values
253 all_pvalues = {
254     'conversion_rate': 0.012,
255     'revenue_per_user': 0.045,
256     'session_duration': 0.123,
257     'bounce_rate': 0.089,
258     'page_load_time': 0.567,
259     'error_rate': 0.234
260 }

```

```
258 # Add exploratory analysis
259 exploratory_pvals = {
260     'mobile_conversion': 0.023,
261     'desktop_conversion': 0.456,
262     'new_user_conversion': 0.034
263 }
264
265 experiment.add_exploratory(
266     'Subgroup_Analysis_by_Device/User_Type',
267     exploratory_pvals,
268     correction='bonferroni'
269 )
270
271 # Generate report
272 report = experiment.generate_report(all_pvalues)
273 print(report)
```

6.5.2 Hierarchical Testing

Hierarchical testing structures hypotheses into ordered families, testing more important hypotheses first and preserving power for exploratory analysis.

Three-Level Hierarchy:

1. Level 1 - Primary: 1-2 key decision metrics

- Test at $\alpha = 0.05$ without correction
- Must show significance to launch

2. Level 2 - Secondary: 3-7 supporting metrics

- Test only if primary shows significance
- Apply FDR control (BH) at $\alpha = 0.05$
- Strengthen case for launch

3. Level 3 - Exploratory: Unlimited metrics

- Test only if at least one secondary shows significance
- Apply strict correction (Bonferroni or BY)
- Generate hypotheses for future testing

This hierarchical approach controls overall FWER while preserving power for the most important tests.

6.5.3 Primary vs. Secondary Endpoints

Clear designation of primary and secondary endpoints is essential for maintaining statistical integrity.

Primary Endpoints:

- Pre-specified before data collection

- Typically 1-2 metrics maximum
- Directly tied to business objectives
- Tested at nominal α without correction
- Must show significance for positive decision
- Statistical power calculated for these metrics

Secondary Endpoints:

- Pre-specified supporting metrics
- Typically 3-7 metrics
- Provide additional evidence
- Require multiple testing correction
- Non-significance doesn't preclude launch
- May have lower power

Decision Framework:

Primary Result	Secondary Results	Decision
Significant	Positive (some significant)	Strong launch
Significant	Mixed (some negative)	Careful launch
Significant	Negative (harm detected)	Do not launch
Not significant	Any	Do not launch

Example 6.9 (Primary vs Secondary Framework). A streaming service tests a new recommendation algorithm:

Primary Endpoint:

- Monthly viewing hours per user

Power: 80% to detect 5% increase

Secondary Endpoints:

- Content discovery rate
- User satisfaction score
- Subscription retention
- Revenue per user

Results:

- Primary: +6.2% viewing hours, $p = 0.008 \rightarrow$ Significant
- Secondary (BH corrected):

- Discovery rate: +8.1%, $q = 0.012 \rightarrow$ Significant
- Satisfaction: +0.3 points, $q = 0.089 \rightarrow$ Not significant
- Retention: +1.2%, $q = 0.156 \rightarrow$ Not significant
- Revenue: +2.3%, $q = 0.067 \rightarrow$ Not significant

Decision: Launch. Primary endpoint shows significant improvement. Two secondary metrics (discovery and satisfaction) show positive directional effects, though only discovery is significant after correction. No negative effects detected.

6.6 Guidelines for Experiment Programs

6.6.1 Organizational Best Practices

1. Pre-Registration:

- Require pre-registration of hypotheses and metrics
- Use experiment registry or design document
- Distinguish primary, secondary, and exploratory analyses

2. Metric Hierarchy:

- Maintain organization-wide metric taxonomy
- Designate standard primary metrics by product area
- Define standard guardrail metrics

3. Automatic Corrections:

- Experimentation platform applies corrections by default
- Display both raw and adjusted p-values
- Show q-values for FDR-controlled metrics

4. Statistical Review:

- Review experiment designs before launch
- Check power calculations and correction methods
- Review results before major launches

5. Education:

- Training programs on multiple testing
- Documentation of correction methods
- Regular workshops on statistical best practices

6. Monitoring:

- Track false discovery rates empirically
- Monitor ratio of significant to non-significant results
- Investigate anomalies in the experimentation pipeline

6.6.2 Decision-Making Framework

Use this comprehensive framework to choose appropriate multiple testing strategies:

Algorithm 2 Multiple Testing Strategy Selection

Input: Number of tests m , test context, cost of false positives**Output:** Recommended correction method**Step 1:** Determine analysis type**if** Pre-registered confirmatory analysis **then**

Proceed to Step 2

else if Exploratory or post-hoc analysis **then**

Use strict correction (Bonferroni or BY)

Flag results as requiring validation

return**end if****Step 2:** Count comparisons**if** $m = 1$ **then**

No correction needed

return**else if** $m \in [2, 10]$ **then**

Consider FWER control

else if $m > 10$ **then**

Consider FDR control

end if**Step 3:** Assess false positive cost**if** Very high (safety, irreversible) **then**

Use FWER control (Holm)

else if High (major product changes) **then** Use FWER or FDR based on m **else if** Moderate (minor changes, exploration) **then**

Use FDR control (BH)

end if**Step 4:** Check test structure**if** Hierarchical structure (primary/secondary) **then**

Use hierarchical testing or gatekeeping

else if Multiple metrics, single variant **then**

Apply chosen correction across metrics

else if Single metric, multiple variants **then**

Apply chosen correction across variants

end if**Step 5:** Verify assumptions**if** Tests are independent **then**

Can use any method

else if Tests are correlated **then**

Use Holm (FWER) or BH (FDR)

Avoid Šidák

else if Unknown dependence **then**

Use robust methods (Holm, BY)

end if

6.7 Summary

Multiple testing is a pervasive challenge in modern A/B testing that requires careful attention:

- **The Problem:** Each additional test increases the probability of false discoveries. Without correction, false positive rates can reach 40% or higher with just 10 tests.
- **Basic Corrections:** Bonferroni, Holm (FWER control) and Benjamini-Hochberg, Benjamini-Yekutieli (FDR control) provide foundational methods.
- **Advanced Approaches:** Closed testing, gatekeeping, adaptive procedures, and Bayesian methods offer sophisticated solutions for complex scenarios.
- **Practical Applications:** Multiple metrics, subgroup analysis, sequential testing, and platform-wide programs each require tailored strategies.
- **Implementation:** Pre-registration, hierarchical testing, clear primary/secondary distinction, and organizational best practices maintain integrity.
- **Choice of Method:** Depends on number of tests, false positive costs, test structure, independence, and whether analysis is confirmatory or exploratory.

The multiple testing problem is serious, but solvable. By understanding the available methods and applying them thoughtfully within an organizational framework, you can maintain statistical rigor while preserving power to detect real effects and making sound business decisions.

6.8 Further Reading

- **Benjamini, Y., & Hochberg, Y. (1995).** “Controlling the false discovery rate: a practical and powerful approach to multiple testing.” *Journal of the Royal Statistical Society: Series B*, 57(1), 289-300.
- **Bretz, F., Maurer, W., Brannath, W., & Posch, M. (2009).** “A graphical approach to sequentially rejective multiple test procedures.” *Statistics in Medicine*, 28(4), 586-604.
- **Dudoit, S., & van der Laan, M. J. (2008).** *Multiple Testing Procedures with Applications to Genomics*. Springer.
- **Efron, B. (2010).** *Large-Scale Inference: Empirical Bayes Methods for Estimation, Testing, and Prediction*. Cambridge University Press.
- **Dmitrienko, A., Tamhane, A. C., & Bretz, F. (2009).** *Multiple Testing Problems in Pharmaceutical Statistics*. CRC Press.
- **Kohavi, R., Tang, D., & Xu, Y. (2020).** *Trustworthy Online Controlled Experiments*. Cambridge University Press, Chapter 20.

6.9 Exercises

6.9.1 Conceptual Questions

1. Explain why the probability of at least one false positive increases with the number of tests, even when each test individually has only a 5% false positive rate.
2. Why is the Benjamini-Hochberg procedure more powerful than Bonferroni correction? What does BH control that is different from what Bonferroni controls?
3. A colleague argues: “We should never use multiple testing corrections because they reduce our power to detect real effects.” How would you respond?
4. Explain the difference between FWER and FDR. Give examples of situations where each would be more appropriate.
5. Describe closed testing and explain why it can be more powerful than simple Bonferroni correction.
6. What is gatekeeping and when is it appropriate? Give an example from industry.
7. Compare and contrast pre-planned and exploratory analyses. Why do they require different multiple testing approaches?

6.9.2 Calculation Exercises

1. You conduct 8 hypothesis tests at $\alpha = 0.05$. If all null hypotheses are true:
 - (a) What is the probability of at least one false positive without correction?
 - (b) What is the Bonferroni-corrected threshold?
 - (c) What is the Holm-corrected threshold for the 3rd ordered p-value?
2. Given p-values: [0.001, 0.015, 0.023, 0.047, 0.089, 0.134] at $\alpha = 0.05$:
 - (a) Apply Bonferroni correction. Which are significant?
 - (b) Apply Holm’s procedure. Which are significant?
 - (c) Apply Benjamini-Hochberg. Which are significant?
 - (d) Apply Benjamini-Yekutieli with $c(6) = 2.45$. Which are significant?
3. For Storey’s π_0 estimator with $\lambda = 0.5$, you have 100 p-values and 63 exceed 0.5. Estimate π_0 and compute the adaptive BH threshold for the 10th ordered p-value.
4. Design a serial gatekeeping procedure for:
 - Family 1: 1 primary hypothesis
 - Family 2: 3 secondary hypotheses
 - Family 3: 5 exploratory hypotheses

Specify the testing procedure and thresholds for each family.

6.9.3 Programming Exercises

1. Implement a simulation demonstrating the multiple testing problem:
 - (a) Generate data under the null for $m = 1, 5, 10, 20, 50$ tests
 - (b) Run 10,000 simulations
 - (c) Plot empirical FWER vs. m
 - (d) Compare to theoretical FWER
2. Implement and compare power of correction methods:
 - (a) Simulate $m = 50$ tests where 20% have true effects
 - (b) Calculate power for Bonferroni, Holm, BH, and adaptive BH
 - (c) Vary effect sizes and repeat
 - (d) Plot power curves
3. Implement a gatekeeping procedure:
 - (a) Code serial and parallel gatekeeping
 - (b) Test on simulated data with hierarchical structure
 - (c) Compare FWER control to single-step procedures
 - (d) Demonstrate power advantage
4. Create a complete experiment analysis framework:
 - (a) Support pre-registration of metrics
 - (b) Implement hierarchical testing
 - (c) Apply appropriate corrections automatically
 - (d) Generate comprehensive report
 - (e) Include visualization of results

6.9.4 Applied Exercises

1. You're testing a new product feature with:
 - Primary: Conversion rate ($p = 0.012$)
 - Secondary: Revenue ($p = 0.045$), Engagement ($p = 0.089$), Retention ($p = 0.003$)
 - Guardrails: Load time ($p = 0.234$), Errors ($p = 0.456$)

Design the analysis strategy, apply corrections, and make a launch decision.

2. A social media platform runs 500 experiments per quarter. Design a platform-wide multiple testing strategy including:
 - Metric hierarchies
 - Default correction methods

- Review processes
- Monitoring procedures

3. Design a subgroup analysis for an A/B test examining effects across:

- 3 geographic regions
- 2 device types (mobile, desktop)
- 2 user types (new, returning)

Specify correction method and decision criteria.

4. Review a real A/B test from your organization:

- (a) Identify all hypothesis tests performed
- (b) Determine if correction was applied
- (c) Calculate what correction should have been used
- (d) Assess if conclusions would change

6.9.5 Advanced Exercises

1. Prove that Holm's procedure is uniformly more powerful than Bonferroni.
2. Implement local FDR estimation and compare to traditional FDR control.
3. Design a graphical gatekeeping procedure (Bretz et al., 2009) for a complex clinical trial with multiple endpoints and populations.
4. Conduct a comprehensive simulation study comparing FWER and FDR methods:
 - Varying m (5, 20, 100 tests)
 - Varying proportion of true alternatives
 - Varying effect sizes
 - Varying correlation structures (independent, positive, negative)

Create visualizations and recommendations.

5. Implement a Bayesian multiple testing procedure with empirical Bayes estimation of prior parameters from data.

Chapter 7

Bayesian Approaches to A/B Testing

7.1 Learning Objectives

After completing this chapter, readers will be able to:

- Apply Bayes' theorem to update beliefs based on experimental data
- Select appropriate prior distributions for different testing scenarios
- Construct and interpret posterior distributions using conjugate and non-conjugate priors
- Distinguish between credible intervals and confidence intervals
- Implement Bayesian A/B tests using analytical solutions and Monte Carlo methods
- Use PyMC3 and Stan for complex Bayesian models
- Apply MCMC methods when analytical solutions are unavailable
- Make risk-based decisions using expected loss and value of information
- Implement early stopping rules based on posterior probabilities
- Compare Bayesian and frequentist approaches in practical settings
- Understand computational trade-offs and approximation methods

7.2 Introduction: Why Bayesian A/B Testing?

The frequentist framework, dominant in classical statistics and covered in previous chapters, treats parameters as fixed but unknown constants. Statistical inference proceeds by asking: "If the null hypothesis were true and we repeated this experiment infinitely many times, how often would we see data as extreme as what we observed?"

This indirect reasoning—conditioning on hypothetical repetitions of an experiment we'll never actually repeat—can feel unnatural. When a product manager asks "What's the probability that variant B is better than A?", the frequentist framework cannot directly answer this question. P-values tell us something different: the probability of observing our data (or more extreme data) under the null hypothesis.

The Bayesian framework offers a more direct approach. It treats parameters as random variables with probability distributions that represent our uncertainty. We can make statements like "There is an 85% probability that variant B's conversion rate exceeds variant A's" or "The expected revenue loss from choosing the wrong variant is \$12,000." These probability statements about parameters align naturally with how stakeholders think about uncertainty.

This chapter explores Bayesian approaches to A/B testing, from fundamental principles through advanced implementations. We'll see how Bayesian methods:

- Provide intuitive probability statements about parameters
- Naturally incorporate prior knowledge from previous experiments
- Handle sequential testing without inflating error rates
- Enable risk-based decision making through expected loss
- Quantify uncertainty through full posterior distributions

We'll also examine when Bayesian methods are most valuable and when frequentist approaches remain preferable.

7.3 Bayesian Fundamentals

7.3.1 Bayes' Theorem and Its Applications

Bayes' theorem, first formulated by Reverend Thomas Bayes in the 18th century, provides a mathematical framework for updating beliefs in light of new evidence.

Theorem 7.1 (Bayes' Theorem). Let θ denote a parameter (or hypothesis) and D denote observed data. Bayes' theorem states:

$$p(\theta|D) = \frac{p(D|\theta) \cdot p(\theta)}{p(D)} \quad (7.1)$$

This can be rewritten as:

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior} \quad (7.2)$$

where:

- $p(\theta)$ is the **prior distribution**: our beliefs about θ before seeing data
- $p(D|\theta)$ is the **likelihood**: probability of observing data D given parameter θ
- $p(D)$ is the **marginal likelihood** or evidence: $p(D) = \int p(D|\theta)p(\theta)d\theta$
- $p(\theta|D)$ is the **posterior distribution**: our updated beliefs about θ after seeing data

The marginal likelihood $p(D)$ serves as a normalizing constant ensuring the posterior integrates to 1. For parameter estimation, we often work with the unnormalized posterior:

$$p(\theta|D) \propto p(D|\theta) \cdot p(\theta) \quad (7.3)$$

Example 7.1 (Medical Diagnosis). Consider a diagnostic test for a rare disease affecting 0.1% of the population. The test has 99% sensitivity (true positive rate) and 99% specificity (true negative rate). If you test positive, what's the probability you have the disease?

Let $D = 1$ denote a positive test and $\theta = 1$ denote having the disease.

Prior: $P(\theta = 1) = 0.001$, $P(\theta = 0) = 0.999$

Likelihood: $P(D = 1|\theta = 1) = 0.99$, $P(D = 1|\theta = 0) = 0.01$

Posterior:

$$P(\theta = 1|D = 1) = \frac{P(D = 1|\theta = 1) \cdot P(\theta = 1)}{P(D = 1)} \quad (7.4)$$

$$= \frac{0.99 \times 0.001}{0.99 \times 0.001 + 0.01 \times 0.999} \quad (7.5)$$

$$= \frac{0.00099}{0.01098} \approx 0.090 \quad (7.6)$$

Despite a positive test with 99% accuracy, you have only a 9% probability of having the disease! The low prior probability dominates. This illustrates the power of prior information in Bayesian reasoning.

7.3.2 Prior Distributions and Their Selection

The prior distribution $p(\theta)$ encodes our beliefs about parameters before seeing data. Prior selection is both an art and a science, requiring domain knowledge, mathematical convenience, and sensitivity analysis.

Types of Priors

1. Uninformative (Flat) Priors

These priors express minimal knowledge or "letting the data speak." For a parameter $p \in [0, 1]$:

- Uniform prior: $p \sim \text{Beta}(1, 1)$ (all values equally likely)
- Jeffreys prior: $p \sim \text{Beta}(0.5, 0.5)$ (invariant to reparameterization)
- Flat improper prior: $p(\theta) \propto 1$ (not a proper distribution but often yields proper posteriors)

When to use: True lack of prior knowledge, first experiment on a new metric, regulatory requirements for objectivity.

2. Weakly Informative Priors

These priors gently constrain parameters to plausible regions without strongly influencing results. For conversion rates, $\text{Beta}(2, 18)$ centers around 10% but allows substantial uncertainty.

When to use: Some domain knowledge exists but shouldn't dominate. Provides regularization to avoid extreme estimates with limited data.

3. Informative Priors

Strong priors based on historical data or subject matter expertise. For a metric with extensive history showing 10% conversion (SE $\pm 1\%$), use Beta(50, 450).

When to use: Extensive historical data exists, testing rare events requiring stabilization, sequential experiments where previous posteriors become new priors.

4. Empirical Bayes

Estimate prior parameters from data itself, typically from related experiments. If 100 previous A/B tests show average conversion of 8.5% with variance 0.002, fit a Beta distribution matching these moments.

When to use: Multiple similar experiments available, hierarchical modeling, platform-wide experimentation programs.

5. Skeptical Priors

Priors centered on the null hypothesis (no effect). Forces strong evidence to overcome skepticism. For lift, use Normal(0, 0.02) centered at zero effect.

When to use: High cost of false positives, extraordinary claims requiring extraordinary evidence, testing potentially harmful interventions.

Prior Selection Guidelines

Scenario	Recommended Prior	Rationale
First experiment	Beta(1, 1) or Beta(0.5, 0.5)	Minimal assumptions
Typical e-commerce	Beta(5, 95) for 5% conversion	Weak regularization
Established metric	Match historical mean/variance	Incorporate knowledge
Rare events	Informative prior from pooling	Stabilize estimates
Sequential testing	Previous posterior	Natural updating

Table 7.1: Prior selection guidelines for common A/B testing scenarios

Mathematical Properties of Common Priors

Beta Distribution:

The Beta distribution is the conjugate prior for binomial likelihoods (conversion rates):

$$\text{Beta}(\alpha, \beta) : f(p; \alpha, \beta) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1} (1-p)^{\beta-1} \quad (7.7)$$

Properties:

$$\mathbb{E}[p] = \frac{\alpha}{\alpha + \beta} \quad (7.8)$$

$$\text{Mode}[p] = \frac{\alpha - 1}{\alpha + \beta - 2} \quad \text{for } \alpha, \beta > 1 \quad (7.9)$$

$$\text{Var}(p) = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)} \quad (7.10)$$

$$\text{Precision} = \alpha + \beta \quad (\text{inverse variance}) \quad (7.11)$$

Interpretation: $\alpha - 1$ can be viewed as "prior successes" and $\beta - 1$ as "prior failures." Total "prior sample size" is $\alpha + \beta - 2$.

Normal Distribution:

For continuous metrics (revenue, time spent):

$$\mathcal{N}(\mu, \sigma^2) : f(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right) \quad (7.12)$$

Conjugate prior for normal likelihood with known variance.

Gamma Distribution:

For rate parameters (purchases per user):

$$\text{Gamma}(\alpha, \beta) : f(\lambda; \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} \lambda^{\alpha-1} e^{-\beta\lambda} \quad (7.13)$$

Conjugate prior for Poisson likelihood.

7.3.3 Posterior Distributions

The posterior distribution $p(\theta|D)$ represents updated beliefs after observing data. All subsequent inference flows from the posterior.

Point Estimates from Posteriors

Several summaries of the posterior distribution serve as point estimates:

1. **Posterior Mean:**

$$\hat{\theta}_{\text{mean}} = \mathbb{E}[\theta|D] = \int \theta \cdot p(\theta|D) d\theta \quad (7.14)$$

Minimizes expected squared error loss: $\mathbb{E}[(\theta - \hat{\theta})^2|D]$

2. **Posterior Median:**

$$\hat{\theta}_{\text{median}} = \text{Median}(\theta|D) \quad (7.15)$$

Minimizes expected absolute error loss: $\mathbb{E}[|\theta - \hat{\theta}||D]$

3. **Posterior Mode (MAP):**

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} p(\theta|D) \quad (7.16)$$

Maximum a posteriori estimate. With flat priors, equivalent to maximum likelihood estimate.

For symmetric posteriors (e.g., normal), all three coincide. For skewed posteriors (e.g., gamma for rate parameters), they differ.

Credible Intervals vs. Confidence Intervals

This distinction is crucial for interpreting uncertainty quantification.

Definition 7.1 (Credible Interval). A $(1 - \alpha)$ credible interval is an interval $[L, U]$ such that:

$$P(L \leq \theta \leq U | D) = 1 - \alpha \quad (7.17)$$

The parameter θ lies in this interval with probability $1 - \alpha$ given the observed data.

Definition 7.2 (Confidence Interval). A $(1 - \alpha)$ confidence interval is constructed by a procedure that, over repeated sampling, contains the true parameter θ in $1 - \alpha$ of realized intervals.

Key Difference:

- **Credible interval:** Direct probability statement about the parameter: "Given the data, there's a 95% probability the conversion rate is between 4% and 6%."
- **Confidence interval:** Frequency statement about the procedure: "If we repeated this experiment infinitely, 95% of constructed intervals would contain the true conversion rate."

The confidence interval interpretation is correct but indirect. Once we've observed specific data, the confidence interval either contains the true parameter or it doesn't (we just don't know which). The credible interval directly quantifies our uncertainty about the parameter given the data we observed.

Example 7.2 (Credible vs. Confidence Interval Interpretation). Suppose we observe 50 conversions from 1000 users and compute:

- Frequentist 95% CI: $[0.038, 0.067]$
- Bayesian 95% credible interval: $[0.037, 0.066]$

Incorrect confidence interval interpretation: "There's a 95% probability the conversion rate is between 3.8% and 6.7%."

Correct confidence interval interpretation: "The procedure used to construct this interval captures the true conversion rate in 95% of repeated samples."

Credible interval interpretation: "Given our data and prior, there's a 95% probability the conversion rate is between 3.7% and 6.6%."

For decision-making, the credible interval interpretation aligns with what stakeholders actually want to know.

Highest Posterior Density (HPD) Intervals:

For skewed posteriors, equal-tailed credible intervals (cutting 2.5% from each tail) may not contain the most probable parameter values. HPD intervals contain the $(1 - \alpha)$ highest density region:

$$\text{HPD}_{1-\alpha} = \{\theta : p(\theta | D) \geq k\} \quad (7.18)$$

where k is chosen such that $P(\theta \in \text{HPD}_{1-\alpha} | D) = 1 - \alpha$.

HPD intervals are shortest among all credible intervals and contain the most probable parameter values.

7.4 Bayesian A/B Testing Framework

7.4.1 Beta-Binomial Model for Conversion Rates

The Beta-binomial model provides an elegant analytical solution for binary outcomes like conversions, clicks, or signups.

Model Specification

Likelihood:

For n independent Bernoulli trials with probability p :

$$X|p \sim \text{Binomial}(n, p) \quad (7.19)$$

The likelihood function is:

$$p(X = x|p) = \binom{n}{x} p^x (1-p)^{n-x} \quad (7.20)$$

Prior:

We place a Beta prior on the conversion rate:

$$p \sim \text{Beta}(\alpha, \beta) \quad (7.21)$$

Posterior:

The Beta distribution is conjugate to the binomial likelihood, yielding a Beta posterior:

Theorem 7.2 (Beta-Binomial Conjugacy). If $p \sim \text{Beta}(\alpha, \beta)$ and $X|p \sim \text{Binomial}(n, p)$, then:

$$p|X = x \sim \text{Beta}(\alpha + x, \beta + n - x) \quad (7.22)$$

Proof. The posterior is proportional to likelihood times prior:

$$p(p|x) \propto p(x|p) \cdot p(p) \quad (7.23)$$

$$\propto p^x (1-p)^{n-x} \cdot p^{\alpha-1} (1-p)^{\beta-1} \quad (7.24)$$

$$= p^{x+\alpha-1} (1-p)^{n-x+\beta-1} \quad (7.25)$$

This is the kernel of $\text{Beta}(\alpha + x, \beta + n - x)$. □

Intuitive Interpretation:

The posterior parameters have a simple interpretation:

- Start with $\alpha - 1$ "prior successes" and $\beta - 1$ "prior failures"
- Observe x successes and $n - x$ failures
- Posterior has $\alpha + x - 1$ total successes and $\beta + n - x - 1$ total failures

This is Bayesian updating in its simplest form: combining prior pseudo-observations with actual observations.

Example 7.3 (Beta-Binomial Updating). Consider three scenarios with uniform prior $\text{Beta}(1, 1)$ and observed data of 80 conversions from 1000 users:

1. **Uninformative prior:** Beta(1, 1)

Posterior: Beta(81, 921)

Mean: $81/1002 = 0.0808$

95% CI: [0.0649, 0.0992]

2. **Weakly informative prior:** Beta(5, 95) (prior belief around 5%)

Posterior: Beta(85, 1015)

Mean: $85/1100 = 0.0773$

95% CI: [0.0625, 0.0942]

3. **Strongly informative prior:** Beta(50, 950) (strong prior belief of 5%)

Posterior: Beta(130, 1870)

Mean: $130/2000 = 0.0650$

95% CI: [0.0549, 0.0765]

Notice how stronger priors pull the posterior toward the prior mean. With 1000 observations, the data has substantial influence, but priors still matter. With 10,000 observations, even strong priors would be overwhelmed by data.

Listing 7.1: Beta-Binomial Bayesian Inference

```

1 import numpy as np
2 from scipy.stats import beta
3 import matplotlib.pyplot as plt
4
5 class BayesianConversionRate:
6     """
7     Bayesian inference for conversion rates using Beta-Binomial model.
8     """
9
10    def __init__(self, prior_alpha=1, prior_beta=1):
11        """
12        Initialize with prior parameters.
13
14        Parameters:
15        prior_alpha: float
16        prior_beta: float
17        Beta prior parameters. Default (1,1) is uniform.
18        """
19        self.prior_alpha = prior_alpha
20        self.prior_beta = prior_beta
21        self.posterior_alpha = prior_alpha
22        self.posterior_beta = prior_beta
23        self.data_points = []
24
25    def update(self, conversions, trials):
26        """
27        Update posterior with new data.
28
29        Parameters:
30        conversions: int

```

```

32  """Number of conversions observed
33  """
34  """Number of trials (users/impressions)
35  """
36  self.posterior_alpha += conversions
37  self.posterior_beta += trials - conversions
38  self.data_points.append((conversions, trials))
39
40  def posterior_mean(self):
41      """Posterior mean conversion rate."""
42      return self.posterior_alpha / (self.posterior_alpha +
43                                     self.posterior_beta)
44
45  def posterior_mode(self):
46      """Posterior mode (MAP estimate)."""
47      if self.posterior_alpha > 1 and self.posterior_beta > 1:
48          return (self.posterior_alpha - 1) / (self.posterior_alpha +
49                                                  self.posterior_beta -
50                                                  2)
51
52      return None
53
54  def posterior_std(self):
55      """Posterior standard deviation."""
56      a, b = self.posterior_alpha, self.posterior_beta
57      var = (a * b) / ((a + b)**2 * (a + b + 1))
58      return np.sqrt(var)
59
60  def credible_interval(self, alpha=0.05):
61      """
62      Equal-tailed credible interval.
63
64      Parameters:
65      -----
66      alpha: float
67      Significance level (default 0.05 for 95% CI)
68
69      Returns:
70      -----
71      tuple: (lower, upper) bounds
72      """
73      return beta.ppf([alpha/2, 1-alpha/2],
74                      self.posterior_alpha,
75                      self.posterior_beta)
76
77  def hpd_interval(self, alpha=0.05):
78      """
79      Highest posterior density interval.
80
81      Parameters:
82      -----
83      alpha: float
84      Significance level
85
86      Returns:
87      -----
88      tuple: (lower, upper) bounds of HPD interval
89      """
90      from scipy.optimize import minimize_scalar

```

```

89
90     def interval_width(lower_tail):
91         lower = beta.ppf(lower_tail, self.posterior_alpha,
92                           self.posterior_beta)
93         upper = beta.ppf(lower_tail + (1-alpha),
94                           self.posterior_alpha,
95                           self.posterior_beta)
96         return upper - lower
97
98     # Find lower tail probability that minimizes interval width
99     result = minimize_scalar(interval_width, bounds=(0, alpha),
100                             method='bounded')
101     lower_tail = result.x
102
103     lower = beta.ppf(lower_tail, self.posterior_alpha,
104                       self.posterior_beta)
105     upper = beta.ppf(lower_tail + (1-alpha),
106                       self.posterior_alpha,
107                       self.posterior_beta)
108
109     return (lower, upper)
110
111     def probability_greater_than(self, threshold):
112         """
113         Probability that conversion rate exceeds threshold.
114
115         Parameters:
116         -----
117         threshold: float
118         Conversion rate threshold
119
120         Returns:
121         -----
122         float: P(p > threshold | data)
123         """
124         return 1 - beta.cdf(threshold, self.posterior_alpha,
125                             self.posterior_beta)
126
127     def sample(self, n_samples=10000):
128         """
129         Draw samples from posterior distribution.
130
131         Parameters:
132         -----
133         n_samples: int
134         Number of samples to draw
135
136         Returns:
137         -----
138         np.array: Posterior samples
139         """
140         return np.random.beta(self.posterior_alpha, self.posterior_beta
141                               ,
142                               n_samples)
143
144     def plot_posterior(self, ax=None, label=None):
145         """
146         Plot posterior distribution.

```



```

146
147 #####Parameters:
148 #####-----
149 #####ax: matplotlib axis
150 #####Axis to plot on (creates new if None)
151 #####label: str
152 #####Label for the plot
153 #####"""
154     if ax is None:
155         fig, ax = plt.subplots(figsize=(10, 6))
156
157     # Generate x values
158     mean = self.posterior_mean()
159     std = self.posterior_std()
160     x = np.linspace(max(0, mean - 4*std), min(1, mean + 4*std),
161                     1000)
162
163     # Compute PDF
164     y = beta.pdf(x, self.posterior_alpha, self.posterior_beta)
165
166     # Plot
167     ax.plot(x, y, linewidth=2, label=label or 'Posterior')
168
169     # Add mean and credible interval
170     ci = self.credible_interval()
171     ax.axvline(mean, color='red', linestyle='--',
172               label=f'Mean={mean:.4f}')
173     ax.fill_between(x, 0, y, where=(x >= ci[0]) & (x <= ci[1]),
174                   alpha=0.3, label='95% Credible Interval')
175
176     ax.set_xlabel('Conversion Rate')
177     ax.set_ylabel('Density')
178     ax.set_title('Posterior Distribution')
179     ax.legend()
180     ax.grid(alpha=0.3)
181
182     return ax
183
184 # Example usage
185 np.random.seed(42)
186
187 # Create Bayesian model with uniform prior
188 model = BayesianConversionRate(prior_alpha=1, prior_beta=1)
189
190 # Update with observed data
191 model.update(conversions=80, trials=1000)
192
193 print("Posterior Statistics:")
194 print(f"Mean: {model.posterior_mean():.4f}")
195 print(f"Mode: {model.posterior_mode():.4f}")
196 print(f"Std Dev: {model.posterior_std():.4f}")
197 print(f"95% Credible Interval: {model.credible_interval()}")
198 print(f"95% HPD Interval: {model.hpd_interval()}")
199 print(f"P(p > 0.10): {model.probability_greater_than(0.10):.4f}")
200
201 # Plot posterior
202 model.plot_posterior()
203 plt.tight_layout()

```

```

203 plt.savefig('beta_binomial_posterior.pdf')
204 plt.show()

```

7.4.2 Normal-Normal Model for Continuous Metrics

For continuous outcomes like revenue, time on site, or page load time, the normal-normal conjugate model provides analytical solutions.

Known Variance Case

Model:

$$Y_i|\mu \sim \mathcal{N}(\mu, \sigma^2) \quad (\text{likelihood, known } \sigma^2) \quad (7.26)$$

$$\mu \sim \mathcal{N}(\mu_0, \tau_0^2) \quad (\text{prior}) \quad (7.27)$$

Posterior:

Theorem 7.3 (Normal-Normal Conjugacy with Known Variance). Given n observations $\mathbf{y} = (y_1, \dots, y_n)$ with sample mean $\bar{y}$:

$$\mu|\mathbf{y} \sim \mathcal{N}(\mu_n, \tau_n^2) \quad (7.28)$$

where:

$$\tau_n^2 = \left(\frac{1}{\tau_0^2} + \frac{n}{\sigma^2} \right)^{-1} \quad (\text{posterior variance}) \quad (7.29)$$

$$\mu_n = \tau_n^2 \left(\frac{\mu_0}{\tau_0^2} + \frac{n\bar{y}}{\sigma^2} \right) \quad (\text{posterior mean}) \quad (7.30)$$

Alternative form (precision-weighted average):

$$\mu_n = \frac{\frac{1}{\tau_0^2}}{\frac{1}{\tau_0^2} + \frac{n}{\sigma^2}} \mu_0 + \frac{\frac{n}{\sigma^2}}{\frac{1}{\tau_0^2} + \frac{n}{\sigma^2}} \bar{y} \quad (7.31)$$

The posterior mean is a precision-weighted average of prior mean and data mean. As $n \rightarrow \infty$, the data overwhelms the prior and $\mu_n \rightarrow \bar{y}$.

Unknown Variance Case

When variance is unknown, we need a prior for both μ and σ^2 . The normal-inverse-gamma is the conjugate prior:

Model:

$$Y_i|\mu, \sigma^2 \sim \mathcal{N}(\mu, \sigma^2) \quad (7.32)$$

$$\sigma^2 \sim \text{Inverse-Gamma}(\alpha_0, \beta_0) \quad (7.33)$$

$$\mu|\sigma^2 \sim \mathcal{N}(\mu_0, \sigma^2/\kappa_0) \quad (7.34)$$

Posterior:

The posterior is normal-inverse-gamma with updated parameters. The marginal posterior for μ is Student's t:

$$\mu|\mathbf{y} \sim t_{2\alpha_n} \left(\mu_n, \frac{\beta_n}{\alpha_n \kappa_n} \right) \quad (7.35)$$

This accounts for uncertainty in both mean and variance, yielding wider credible intervals than the known-variance case.

Listing 7.2: Normal-Normal Model for Revenue Per User

```

1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4
5 class BayesianNormalModel:
6     """
7     Bayesian inference for normal mean with known variance.
8     """
9
10    def __init__(self, prior_mean=0, prior_var=1000, data_var=100):
11        """
12        Initialize with prior and data variance.
13
14        Parameters:
15        -----
16        prior_mean: float
17        Prior mean
18        prior_var: float
19        Prior variance (tau_0^2)
20        data_var: float
21        Known data variance (sigma^2)
22        """
23        self.prior_mean = prior_mean
24        self.prior_var = prior_var
25        self.data_var = data_var
26        self.posterior_mean = prior_mean
27        self.posterior_var = prior_var
28        self.n_obs = 0
29        self.data_mean = 0
30
31    def update(self, data):
32        """
33        Update posterior with new data.
34
35        Parameters:
36        -----
37        data: array-like
38        Observed data points
39        """
40        data = np.asarray(data)
41        n = len(data)
42        y_bar = np.mean(data)
43
44        # Update posterior parameters (precision-weighted)
45        prior_precision = 1 / self.prior_var
46        data_precision = n / self.data_var
47
48        self.posterior_var = 1 / (prior_precision + data_precision)
49        self.posterior_mean = self.posterior_var * (

```

```

50         prior_precision * self.prior_mean +
51         data_precision * y_bar
52     )
53
54     self.n_obs += n
55     self.data_mean = y_bar
56
57     def credible_interval(self, alpha=0.05):
58         """95% credible interval for mean."""
59         z = stats.norm.ppf(1 - alpha/2)
60         sd = np.sqrt(self.posterior_var)
61         return (self.posterior_mean - z * sd,
62                 self.posterior_mean + z * sd)
63
64     def probability_greater_than(self, threshold):
65         """P(mu > threshold | data)"""
66         return 1 - stats.norm.cdf(threshold, self.posterior_mean,
67                                   np.sqrt(self.posterior_var))
68
69     def plot_posterior(self, ax=None):
70         """Plot posterior distribution."""
71         if ax is None:
72             fig, ax = plt.subplots(figsize=(10, 6))
73
74         sd = np.sqrt(self.posterior_var)
75         x = np.linspace(self.posterior_mean - 4*sd,
76                         self.posterior_mean + 4*sd, 1000)
77         y = stats.norm.pdf(x, self.posterior_mean, sd)
78
79         ax.plot(x, y, linewidth=2, label='Posterior')
80         ax.axvline(self.posterior_mean, color='red', linestyle='--',
81                   label=f'Mean={self.posterior_mean:.2f}')
82
83         ci = self.credible_interval()
84         ax.fill_between(x, 0, y, where=(x >= ci[0]) & (x <= ci[1]),
85                        alpha=0.3, label='95% Credible Interval')
86
87         ax.set_xlabel('Mean Revenue')
88         ax.set_ylabel('Density')
89         ax.set_title('Posterior Distribution of Mean')
90         ax.legend()
91         ax.grid(alpha=0.3)
92
93         return ax
94
95     # Example: Revenue per user
96     # Assume we know revenue has high variance (sigma^2 = 10000)
97     # Weakly informative prior centered at $50 with large uncertainty
98     model = BayesianNormalModel(prior_mean=50, prior_var=1000, data_var
99                                =10000)
100
101     # Observe revenue data
102     np.random.seed(42)
103     revenue_data = np.random.normal(55, 100, size=500) # True mean $55
104
105     model.update(revenue_data)
106
107     print("Posterior Statistics:")

```

```

107 print(f"Prior mean: {model.prior_mean:.2f}")
108 print(f>Data mean: {model.data_mean:.2f}")
109 print(f"Posterior mean: {model.posterior_mean:.2f}")
110 print(f"Posterior SD: {np.sqrt(model.posterior_var):.2f}")
111 print(f"95% Credible Interval: [{ci[0]:.2f}, {ci[1]:.2f}]")
112 print(f"P(mu > $50): {model.probability_greater_than(50):.4f}")
113
114 model.plot_posterior()
115 plt.tight_layout()
116 plt.savefig('normal_normal_posterior.pdf')
117 plt.show()

```

7.4.3 Comparing Two Variants: Bayesian A/B Testing

The ultimate goal is comparing treatments. For variants A and B, we want to know:

1. $P(p_B > p_A | D)$: Probability B is better than A
2. $\mathbb{E}[p_B - p_A | D]$: Expected difference
3. Expected loss from choosing each variant

Probability of Superiority

For conversion rates with Beta posteriors:

$$p_A | D_A \sim \text{Beta}(\alpha_A, \beta_A) \quad (7.36)$$

$$p_B | D_B \sim \text{Beta}(\alpha_B, \beta_B) \quad (7.37)$$

We want:

$$P(p_B > p_A | D) = \int_0^1 \int_0^{p_B} f(p_A | D_A) f(p_B | D_B) dp_A dp_B \quad (7.38)$$

For independent Beta distributions, no closed form exists. Three approaches:

1. Numerical Integration:

For small α, β parameters, numerical integration is feasible:

$$P(p_B > p_A) = \sum_{i=0}^{\alpha_A-1} \frac{B(\alpha_B + i, \beta_A + \beta_B)}{(\beta_A + i) \cdot B(1 + i, \beta_A) \cdot B(\alpha_B, \beta_B)} \quad (7.39)$$

where B is the beta function. This formula comes from expanding the integral.

2. Monte Carlo Simulation:

Simple and general:

1. Sample $p_A^{(1)}, \dots, p_A^{(S)} \sim \text{Beta}(\alpha_A, \beta_A)$
2. Sample $p_B^{(1)}, \dots, p_B^{(S)} \sim \text{Beta}(\alpha_B, \beta_B)$
3. Estimate $P(p_B > p_A) \approx \frac{1}{S} \sum_{s=1}^S \mathbb{I}(p_B^{(s)} > p_A^{(s)})$

With $S = 100,000$ samples, Monte Carlo standard error is approximately $\sqrt{p(1-p)/S} \approx 0.0016$, giving high precision.

3. Normal Approximation:

For large sample sizes, Beta distributions are approximately normal:

$$p_A|D \approx \mathcal{N}(\mu_A, \sigma_A^2) \quad (7.40)$$

$$p_B|D \approx \mathcal{N}(\mu_B, \sigma_B^2) \quad (7.41)$$

Then $p_B - p_A \sim \mathcal{N}(\mu_B - \mu_A, \sigma_A^2 + \sigma_B^2)$ and:

$$P(p_B > p_A) = \Phi\left(\frac{\mu_B - \mu_A}{\sqrt{\sigma_A^2 + \sigma_B^2}}\right) \quad (7.42)$$

where Φ is the standard normal CDF.

Listing 7.3: Bayesian A/B Test Comparison

```

1 # Comprehensive Bayesian A/B test analysis
2 bayesian_ab_test <- function(x_a, n_a, x_b, n_b,
3                             prior_alpha = 1, prior_beta = 1,
4                             n_simulations = 100000) {
5   # Posterior parameters
6   post_alpha_a <- prior_alpha + x_a
7   post_beta_a <- prior_beta + n_a - x_a
8   post_alpha_b <- prior_alpha + x_b
9   post_beta_b <- prior_beta + n_b - x_b
10
11  # Monte Carlo simulation
12  set.seed(42)
13  p_a_samples <- rbeta(n_simulations, post_alpha_a, post_beta_a)
14  p_b_samples <- rbeta(n_simulations, post_alpha_b, post_beta_b)
15
16  # Probability B > A
17  prob_b_better <- mean(p_b_samples > p_a_samples)
18
19  # Absolute difference distribution
20  diff <- p_b_samples - p_a_samples
21  diff_mean <- mean(diff)
22  diff_median <- median(diff)
23  diff_ci <- quantile(diff, c(0.025, 0.975))
24
25  # Relative lift distribution
26  lift <- (p_b_samples / p_a_samples - 1) * 100
27  lift_mean <- mean(lift)
28  lift_median <- median(lift)
29  lift_ci <- quantile(lift, c(0.025, 0.975))
30
31  # Probability of practical significance (e.g., >1% absolute lift)
32  prob_practical_sig <- mean(diff > 0.01)
33
34  # Expected loss (opportunity cost)
35  loss_a <- mean(pmax(0, p_b_samples - p_a_samples))
36  loss_b <- mean(pmax(0, p_a_samples - p_b_samples))
37
38  # Posterior statistics
39  stats_a <- list(
40    mean = post_alpha_a / (post_alpha_a + post_beta_a),

```

```

41     median = qbeta(0.5, post_alpha_a, post_beta_a),
42     ci = qbeta(c(0.025, 0.975), post_alpha_a, post_beta_a)
43 )
44
45 stats_b <- list(
46     mean = post_alpha_b / (post_alpha_b + post_beta_b),
47     median = qbeta(0.5, post_alpha_b, post_beta_b),
48     ci = qbeta(c(0.025, 0.975), post_alpha_b, post_beta_b)
49 )
50
51 # Return comprehensive results
52 list(
53     prob_b_better = prob_b_better,
54     prob_practical_significance = prob_practical_sig,
55     difference = list(mean = diff_mean, median = diff_median, ci = diff
56         _ci),
57     lift = list(mean = lift_mean, median = lift_median, ci = lift_ci),
58     expected_loss = list(loss_a = loss_a, loss_b = loss_b),
59     posterior_a = stats_a,
60     posterior_b = stats_b,
61     samples = list(p_a = p_a_samples, p_b = p_b_samples)
62 )
63
64 # Example: E-commerce checkout flow test
65 # Control (A): 245 conversions from 3,500 users
66 # Treatment (B): 289 conversions from 3,500 users
67
68 result <- bayesian_ab_test(x_a = 245, n_a = 3500,
69     x_b = 289, n_b = 3500,
70     prior_alpha = 1, prior_beta = 1)
71
72 # Print results
73 cat("===Bayesian A/B Test Results===\n\n")
74
75 cat("\nVariant A (Control):\n")
76 cat(sprintf("Posterior mean: %.4f\n", result$posterior_a$mean))
77 cat(sprintf("95% CI: %.4f, %.4f\n",
78     result$posterior_a$ci[1], result$posterior_a$ci[2]))
79
80 cat("\nVariant B (Treatment):\n")
81 cat(sprintf("Posterior mean: %.4f\n", result$posterior_b$mean))
82 cat(sprintf("95% CI: %.4f, %.4f\n",
83     result$posterior_b$ci[1], result$posterior_b$ci[2]))
84
85 cat("\nComparison:\n")
86 cat(sprintf("P(B > A): %.4f\n", result$prob_b_better))
87 cat(sprintf("Expected difference: %.4f\n", result$difference$mean))
88 cat(sprintf("95% CI for difference: %.4f, %.4f\n",
89     result$difference$ci[1], result$difference$ci[2]))
90 cat(sprintf("Expected lift: %.2f%\n", result$lift$mean))
91 cat(sprintf("95% CI for lift: %.2f%, %.2f%\n",
92     result$lift$ci[1], result$lift$ci[2]))
93
94 cat("\nDecision Analysis:\n")
95 cat(sprintf("Expected loss choosing A: %.6f\n",
96     result$expected_loss$loss_a))
97 cat(sprintf("Expected loss choosing B: %.6f\n",

```

```

98     result$expected_loss$loss_b))
99
100 recommendation <- if (result$expected_loss$loss_a <
101     result$expected_loss$loss_b) "A" else "B"
102 cat(sprintf("\nRecommendation: Choose variant %s\n", recommendation))
103
104 # Visualization
105 library(ggplot2)
106 library(tidyr)
107
108 # Plot posterior distributions
109 df_posteriors <- data.frame(
110     Variant_A = result$samples$p_a[1:10000],
111     Variant_B = result$samples$p_b[1:10000]
112 ) %>% pivot_longer(everything(), names_to = "Variant",
113     values_to = "Conversion_Rate")
114
115 ggplot(df_posteriors, aes(x = Conversion_Rate, fill = Variant)) +
116     geom_density(alpha = 0.5) +
117     geom_vline(xintercept = result$posterior_a$mean,
118         color = "red", linetype = "dashed") +
119     geom_vline(xintercept = result$posterior_b$mean,
120         color = "blue", linetype = "dashed") +
121     labs(title = "Posterior Distributions of Conversion Rates",
122         x = "Conversion_Rate", y = "Density") +
123     theme_minimal()
124
125 ggsave("bayesian_ab_posteriors.pdf", width = 10, height = 6)
126
127 # Plot difference distribution
128 df_diff <- data.frame(
129     Difference = result$samples$p_b[1:10000] - result$samples$p_a
130     [1:10000]
131 )
132
133 ggplot(df_diff, aes(x = Difference)) +
134     geom_histogram(aes(y = ..density..), bins = 50,
135         fill = "steelblue", alpha = 0.7) +
136     geom_density(color = "darkblue", size = 1) +
137     geom_vline(xintercept = 0, color = "red",
138         linetype = "dashed", size = 1) +
139     geom_vline(xintercept = result$difference$mean,
140         color = "darkgreen", linetype = "dashed", size = 1) +
141     labs(title = "Posterior Distribution of Difference (B - A)",
142         subtitle = sprintf("P(B > A) = %.2f%%",
143             result$prob_b_better * 100),
144         x = "Difference in Conversion Rate", y = "Density") +
145     theme_minimal()
146 ggsave("bayesian_ab_difference.pdf", width = 10, height = 6)

```

7.4.4 Expected Loss and Risk-Based Decisions

Binary decisions based solely on $P(B > A)$ ignore effect magnitude. Expected loss provides a more nuanced decision framework.

Definition 7.3 (Expected Loss). The expected loss from choosing variant i is the ex-

pected opportunity cost:

$$\mathcal{L}(i) = \mathbb{E}[\max(0, \max_j p_j - p_i) | D] \quad (7.43)$$

For two variants:

$$\mathcal{L}(A) = \mathbb{E}[\max(0, p_B - p_A) | D] \quad (7.44)$$

$$\mathcal{L}(B) = \mathbb{E}[\max(0, p_A - p_B) | D] \quad (7.45)$$

Interpretation: $\mathcal{L}(A)$ is the expected loss (in terms of conversion rate or revenue) if we choose A but B is actually better. It quantifies regret.

Decision Rule: Choose the variant with minimum expected loss. This is optimal under expected loss criterion.

Threshold-Based Decisions:

In practice, we might only switch from A to B if expected loss exceeds a threshold:

$$\text{Switch to B if } \mathcal{L}(A) - \mathcal{L}(B) > \delta \quad (7.46)$$

This builds in conservatism, accounting for implementation costs or risk aversion.

Listing 7.4: Expected Loss and Value of Information

```

1 import numpy as np
2 from scipy.stats import beta
3 import matplotlib.pyplot as plt
4
5 def expected_loss_ab(x_a, n_a, x_b, n_b,
6                     prior_alpha=1, prior_beta=1,
7                     n_simulations=100000,
8                     cost_per_user=1.0):
9     """
10    Compute expected loss and value-based decision metrics.
11
12    Parameters:
13    -----
14    x_a, n_a: int
15    Conversions and trials for variant A
16    x_b, n_b: int
17    Conversions and trials for variant B
18    prior_alpha, prior_beta: float
19    Beta prior parameters
20    n_simulations: int
21    Number of Monte Carlo samples
22    cost_per_user: float
23    Value per percentage point of conversion rate
24
25    Returns:
26    -----
27    dict: Comprehensive decision metrics
28    """
29    # Posterior parameters
30    post_alpha_a = prior_alpha + x_a
31    post_beta_a = prior_beta + n_a - x_a
32    post_alpha_b = prior_alpha + x_b
33    post_beta_b = prior_beta + n_b - x_b
34

```



```

93  Visualize loss distributions for both decisions.
94  """
95  # Posterior parameters
96  post_alpha_a = prior_alpha + x_a
97  post_beta_a = prior_beta + n_a - x_a
98  post_alpha_b = prior_alpha + x_b
99  post_beta_b = prior_beta + n_b - x_b
100
101  # Sample
102  np.random.seed(42)
103  p_a = np.random.beta(post_alpha_a, post_beta_a, n_simulations)
104  p_b = np.random.beta(post_alpha_b, post_beta_b, n_simulations)
105
106  # Loss distributions
107  loss_if_choose_a = np.maximum(0, p_b - p_a)
108  loss_if_choose_b = np.maximum(0, p_a - p_b)
109
110  # Plot
111  fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
112
113  # Loss if choose A
114  ax1.hist(loss_if_choose_a, bins=50, density=True,
115           alpha=0.7, color='red', edgecolor='black')
116  ax1.axvline(np.mean(loss_if_choose_a), color='darkred',
117             linestyle='--', linewidth=2,
118             label=f'Expected Loss = {np.mean(loss_if_choose_a):.5f}')
119  ax1.axvline(np.percentile(loss_if_choose_a, 95), color='orange',
120             linestyle=':', linewidth=2,
121             label=f'95% VaR = {np.percentile(loss_if_choose_a, 95):.5f}')
122  ax1.set_xlabel('Loss (if B is better)')
123  ax1.set_ylabel('Density')
124  ax1.set_title('Loss Distribution if Choose A')
125  ax1.legend()
126  ax1.grid(alpha=0.3)
127
128  # Loss if choose B
129  ax2.hist(loss_if_choose_b, bins=50, density=True,
130           alpha=0.7, color='blue', edgecolor='black')
131  ax2.axvline(np.mean(loss_if_choose_b), color='darkblue',
132             linestyle='--', linewidth=2,
133             label=f'Expected Loss = {np.mean(loss_if_choose_b):.5f}')
134  ax2.axvline(np.percentile(loss_if_choose_b, 95), color='cyan',
135             linestyle=':', linewidth=2,
136             label=f'95% VaR = {np.percentile(loss_if_choose_b, 95):.5f}')
137  ax2.set_xlabel('Loss (if A is better)')
138  ax2.set_ylabel('Density')
139  ax2.set_title('Loss Distribution if Choose B')
140  ax2.legend()
141  ax2.grid(alpha=0.3)
142
143  plt.tight_layout()
144  plt.savefig('expected_loss_distributions.pdf')
145  plt.show()
146

```

```

147 # Example: Close call scenario
148 result = expected_loss_ab(
149     x_a=245, n_a=3500, # Control: 7.0% conversion
150     x_b=260, n_b=3500, # Treatment: 7.4% conversion
151     cost_per_user=1000 # $1000 value per percentage point
152 )
153
154 print("=== Risk-Based Decision Analysis ===\n")
155 print(f"Recommendation: Choose variant {result['recommendation']}")
156 print(f"Confidence: {result['confidence']:.2%}")
157 print(f"Expected regret: ${result['expected_regret']:.2f}\n")
158
159 print("Expected Loss if Choose A:")
160 print(f"Rate: {result['loss_a']['rate']:.6f}")
161 print(f"Value: ${result['loss_a']['value']:.2f}")
162 print(f"Relative: {result['loss_a']['relative']:.2%}")
163 print(f"95% VaR: {result['loss_a']['var_95']:.6f}\n")
164
165 print("Expected Loss if Choose B:")
166 print(f"Rate: {result['loss_b']['rate']:.6f}")
167 print(f"Value: ${result['loss_b']['value']:.2f}")
168 print(f"Relative: {result['loss_b']['relative']:.2%}")
169 print(f"95% VaR: {result['loss_b']['var_95']:.6f}\n")
170
171 print(f"P(B > A): {result['prob_b_better']:.2%}")
172 print(f"EVPI: ${result['evpi']:.2f}")
173 print(f"Loss ratio: {result['loss_ratio']:.4f}")
174
175 # Visualize
176 plot_loss_distributions(x_a=245, n_a=3500, x_b=260, n_b=3500)

```

7.4.5 Conjugate Priors and Analytical Solutions

Conjugate priors enable closed-form posterior distributions, avoiding computational burden. We've seen Beta-binomial and normal-normal examples. Here's a comprehensive table:

Likelihood	Parameter	Conjugate Prior	Posterior
Binomial	p	$\text{Beta}(\alpha, \beta)$	$\text{Beta}(\alpha + x, \beta + n - x)$
Poisson	λ	$\text{Gamma}(\alpha, \beta)$	$\text{Gamma}(\alpha + \sum x_i, \beta + n)$
Normal (known σ^2)	μ	$\text{Normal}(\mu_0, \tau_0^2)$	$\text{Normal}(\mu_n, \tau_n^2)$
Normal (known μ)	σ^2	$\text{Inverse-Gamma}(\alpha, \beta)$	$\text{Inverse-Gamma}(\alpha', \beta')$
Exponential	λ	$\text{Gamma}(\alpha, \beta)$	$\text{Gamma}(\alpha + n, \beta + \sum x_i)$
Geometric	p	$\text{Beta}(\alpha, \beta)$	$\text{Beta}(\alpha + n, \beta + \sum x_i)$

Table 7.2: Common conjugate prior families for A/B testing

Advantages of Conjugate Priors:

- Fast computation: no MCMC needed
- Interpretable: parameters have clear meaning

- Sequential updating: today's posterior is tomorrow's prior
- Mathematical elegance: closed-form solutions

Limitations:

- Restrictive: not all priors have conjugate form
- Complex models: multivariate or hierarchical models often lack conjugacy
- Practical constraints: may want priors not in conjugate family

7.4.6 Non-Conjugate Priors and MCMC

When conjugate priors are unavailable or inappropriate, we need computational methods. Markov Chain Monte Carlo (MCMC) provides a general framework for sampling from posteriors.

Why MCMC?

Bayes' theorem gives us:

$$p(\theta|D) = \frac{p(D|\theta)p(\theta)}{\int p(D|\theta)p(\theta)d\theta} \quad (7.47)$$

The denominator—the marginal likelihood—requires integrating over all possible parameter values. In high dimensions or with complex likelihoods, this integral is intractable.

MCMC bypasses this problem by sampling from the posterior without computing the normalizing constant. We work with:

$$p(\theta|D) \propto p(D|\theta)p(\theta) \quad (7.48)$$

MCMC constructs a Markov chain whose stationary distribution is the posterior. After a "burn-in" period, samples approximate draws from $p(\theta|D)$.

Metropolis-Hastings Algorithm

The Metropolis-Hastings (MH) algorithm is the foundation of MCMC:

Algorithm 3 Metropolis-Hastings Algorithm

```

1: Initialize  $\theta^{(0)}$ 
2: for  $t = 1$  to  $T$  do
3:   Propose  $\theta^* \sim q(\theta^*|\theta^{(t-1)})$  {Proposal distribution}
4:   Compute acceptance ratio:

```

$$\alpha = \min \left(1, \frac{p(\theta^*|D)q(\theta^{(t-1)}|\theta^*)}{p(\theta^{(t-1)}|D)q(\theta^*|\theta^{(t-1)})} \right)$$

```

5:   Draw  $u \sim \text{Uniform}(0, 1)$ 
6:   if  $u < \alpha$  then
7:      $\theta^{(t)} = \theta^*$  {Accept}
8:   else
9:      $\theta^{(t)} = \theta^{(t-1)}$  {Reject}
10:  end if
11: end for

```

Key Points:

- Proposal distribution q suggests new parameter values
- Acceptance ratio ensures detailed balance (stationary distribution is posterior)
- Symmetric proposals (e.g., random walk) simplify acceptance ratio
- Tune proposal variance: too small yields slow mixing, too large yields high rejection

Gibbs Sampling

For multivariate posteriors, Gibbs sampling updates one parameter at a time:

Algorithm 4 Gibbs Sampling

```

1: Initialize  $\theta_1^{(0)}, \dots, \theta_k^{(0)}$ 
2: for  $t = 1$  to  $T$  do
3:   Sample  $\theta_1^{(t)} \sim p(\theta_1|\theta_2^{(t-1)}, \dots, \theta_k^{(t-1)}, D)$ 
4:   Sample  $\theta_2^{(t)} \sim p(\theta_2|\theta_1^{(t)}, \theta_3^{(t-1)}, \dots, \theta_k^{(t-1)}, D)$ 
5:    $\vdots$ 
6:   Sample  $\theta_k^{(t)} \sim p(\theta_k|\theta_1^{(t)}, \dots, \theta_{k-1}^{(t)}, D)$ 
7: end for

```

Gibbs sampling is a special case of MH where proposals are always accepted (when using full conditionals).

Hamiltonian Monte Carlo

Hamiltonian Monte Carlo (HMC) uses gradient information to propose distant moves efficiently. It's the engine behind Stan and PyMC3's NUTS sampler.

HMC introduces auxiliary momentum variables and simulates Hamiltonian dynamics. This reduces random walk behavior, enabling efficient exploration of high-dimensional posteriors.

Advantages:

- Efficient for high-dimensional problems
- Fewer correlated samples
- Faster convergence

Disadvantages:

- Requires gradient computation (automatic differentiation)
- More complex to implement
- Sensitive to step size tuning

7.4.7 Implementation with PyMC3

PyMC3 provides a high-level interface for Bayesian modeling using modern MCMC samplers.

Listing 7.5: PyMC3 Implementation of Bayesian A/B Test

```

1 import pymc3 as pm
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import arviz as az
5
6 # Set random seed
7 np.random.seed(42)
8
9 # Observed data
10 x_a, n_a = 245, 3500 # Control
11 x_b, n_b = 289, 3500 # Treatment
12
13 # Build model
14 with pm.Model() as ab_test_model:
15     # Priors for conversion rates
16     p_a = pm.Beta('p_a', alpha=1, beta=1)
17     p_b = pm.Beta('p_b', alpha=1, beta=1)
18
19     # Likelihoods
20     obs_a = pm.Binomial('obs_a', n=n_a, p=p_a, observed=x_a)
21     obs_b = pm.Binomial('obs_b', n=n_b, p=p_b, observed=x_b)
22
23     # Derived quantities of interest
24     diff = pm.Deterministic('diff', p_b - p_a)
25     relative_lift = pm.Deterministic('relative_lift',
26                                     (p_b / p_a - 1) * 100)
27     effect_size = pm.Deterministic('effect_size',
28                                   (p_b - p_a) / pm.math.sqrt((p_a *
29                                                             (1-p_a) + p_b * (1-p_b)) / 2))
29
30     # Sample from posterior

```

```

31     trace = pm.sample(draws=5000, tune=1000, chains=4,
32                       return_inferencedata=True, random_seed=42)
33
34 # Print summary
35 print(az.summary(trace, var_names=['p_a', 'p_b', 'diff', 'relative_lift',
36                                   '']))
37
38 # Compute probabilities
39 posterior = trace.posterior
40 prob_b_better = (posterior['p_b'] > posterior['p_a']).mean().item()
41 prob_practical_sig = (posterior['diff'] > 0.005).mean().item() # >
42                       0.5% absolute
43
44 print(f"\nP(B<A): {prob_b_better:.4f}")
45 print(f"P(diff<0.5%): {prob_practical_sig:.4f}")
46
47 # Plot posteriors
48 az.plot_posterior(trace, var_names=['p_a', 'p_b', 'diff', 'relative_
49                                   lift'],
50                   hdi_prob=0.95, figsize=(12, 8))
51 plt.tight_layout()
52 plt.savefig('pymc3_posteriors.pdf')
53 plt.show()
54
55 # Trace plots (check convergence)
56 az.plot_trace(trace, var_names=['p_a', 'p_b'],
57               figsize=(12, 6))
58 plt.tight_layout()
59 plt.savefig('pymc3_trace.pdf')
60 plt.show()
61
62 # Diagnostics
63 print("\nDiagnostics:")
64 print(f"R-hat: {az.rhat(trace, var_names=['p_a', 'p_b'])}")
65 print(f"ESS: {az.ess(trace, var_names=['p_a', 'p_b'])}")
66
67 # Pairplot to check correlations
68 az.plot_pair(trace, var_names=['p_a', 'p_b', 'diff'],
69              kind='kde', divergences=True, figsize=(10, 10))
70 plt.savefig('pymc3_pairplot.pdf')
71 plt.show()

```

Model Extensions with PyMC3:

Listing 7.6: Hierarchical Bayesian A/B Test with PyMC3

```

1 # Hierarchical model for testing across multiple segments
2 # (e.g., mobile vs. desktop, different geographic regions)
3
4 import pymc3 as pm
5 import numpy as np
6
7 # Simulated data: A/B test across 5 market segments
8 n_segments = 5
9 np.random.seed(42)
10
11 # Segment sample sizes
12 n_a_seg = np.array([800, 1200, 600, 900, 1000])
13 n_b_seg = np.array([800, 1200, 600, 900, 1000])

```



```

14
15 # Simulated conversions (treatment has ~15% relative lift)
16 p_a_true_seg = np.random.beta(8, 92, size=n_segments)
17 p_b_true_seg = p_a_true_seg * 1.15
18
19 x_a_seg = np.random.binomial(n_a_seg, p_a_true_seg)
20 x_b_seg = np.random.binomial(n_b_seg, p_b_true_seg)
21
22 # Hierarchical model
23 with pm.Model() as hierarchical_model:
24     # Hyperpriors for segment-level variation
25     mu_a = pm.Beta('mu_a', alpha=1, beta=1)
26     kappa_a = pm.Gamma('kappa_a', alpha=2, beta=0.1)
27
28     mu_b = pm.Beta('mu_b', alpha=1, beta=1)
29     kappa_b = pm.Gamma('kappa_b', alpha=2, beta=0.1)
30
31     # Segment-level conversion rates (partially pooled)
32     p_a_seg = pm.Beta('p_a_seg',
33                       alpha=mu_a * kappa_a,
34                       beta=(1 - mu_a) * kappa_a,
35                       shape=n_segments)
36
37     p_b_seg = pm.Beta('p_b_seg',
38                       alpha=mu_b * kappa_b,
39                       beta=(1 - mu_b) * kappa_b,
40                       shape=n_segments)
41
42     # Likelihoods
43     obs_a = pm.Binomial('obs_a', n=n_a_seg, p=p_a_seg, observed=x_a_seg)
44     obs_b = pm.Binomial('obs_b', n=n_b_seg, p=p_b_seg, observed=x_b_seg)
45
46     # Overall effect (population average)
47     overall_diff = pm.Deterministic('overall_diff', mu_b - mu_a)
48     overall_lift = pm.Deterministic('overall_lift',
49                                     (mu_b / mu_a - 1) * 100)
50
51     # Segment-specific effects
52     seg_diff = pm.Deterministic('seg_diff', p_b_seg - p_a_seg)
53
54     # Sample
55     trace_hier = pm.sample(draws=3000, tune=1000, chains=4,
56                            return_inferencedata=True, random_seed=42)
57
58 # Summary
59 print("Hierarchical Model Results:")
60 print(az.summary(trace_hier, var_names=['mu_a', 'mu_b', 'overall_diff',
61                                         'overall_lift']))
62
63 print("\nSegment-Level Effects:")
64 print(az.summary(trace_hier, var_names=['seg_diff']))
65
66 # Plot segment-level heterogeneity
67 az.plot_forest(trace_hier, var_names=['seg_diff'], combined=False,
68               figsize=(10, 8))
69 plt.axvline(0, color='red', linestyle='--', alpha=0.5)

```

```

69 plt.title('Segment-Level Treatment Effects')
70 plt.savefig('pymc3_hierarchical_forest.pdf')
71 plt.show()

```

7.4.8 Implementation with Stan

Stan is another powerful probabilistic programming language using Hamiltonian Monte Carlo.

Listing 7.7: Stan Implementation via rstan

```

1  library(rstan)
2  library(bayesplot)
3
4  # Stan model code
5  stan_model_code <- "
6  data{
7    int<lower=0> n_a; // Sample size A
8    int<lower=0> n_b; // Sample size B
9    int<lower=0> x_a; // Conversions A
10   int<lower=0> x_b; // Conversions B
11   real<lower=0> prior_alpha; // Prior alpha
12   real<lower=0> prior_beta; // Prior beta
13 }
14
15 parameters{
16   real<lower=0, upper=1> p_a; // Conversion rate A
17   real<lower=0, upper=1> p_b; // Conversion rate B
18 }
19
20 model{
21   // Priors
22   p_a ~ beta(prior_alpha, prior_beta);
23   p_b ~ beta(prior_alpha, prior_beta);
24
25   // Likelihoods
26   x_a ~ binomial(n_a, p_a);
27   x_b ~ binomial(n_b, p_b);
28 }
29
30 generated_quantities{
31   real diff;
32   real relative_lift;
33   real log_odds_ratio;
34   int<lower=0, upper=1> b_is_better;
35
36   diff = p_b - p_a;
37   relative_lift = (p_b / p_a - 1) * 100;
38   log_odds_ratio = log((p_b / (1 - p_b)) / (p_a / (1 - p_a)));
39   b_is_better = (p_b > p_a) ? 1 : 0;
40 }
41 "
42
43 # Compile model
44 stan_model <- stan_model(model_code = stan_model_code)
45
46 # Prepare data

```

```

47 stan_data <- list(
48   n_a = 3500,
49   n_b = 3500,
50   x_a = 245,
51   x_b = 289,
52   prior_alpha = 1,
53   prior_beta = 1
54 )
55
56 # Fit model
57 fit <- sampling(stan_model, data = stan_data,
58                chains = 4, iter = 5000, warmup = 1000,
59                seed = 42, cores = 4)
60
61 # Print results
62 print(fit, pars = c('p_a', 'p_b', 'diff', 'relative_lift'))
63
64 # Extract samples
65 samples <- extract(fit)
66
67 # Compute probabilities
68 prob_b_better <- mean(samples$b_is_better)
69 cat(sprintf("\nP(B>A): %.4f\n", prob_b_better))
70
71 # Diagnostics
72 cat("\nR-hat values (should be < 1.01):\n")
73 print(summary(fit)$summary[, 'Rhat'])
74
75 cat("\nEffective sample sizes:\n")
76 print(summary(fit)$summary[, 'n_eff'])
77
78 # Visualizations
79 # Posterior distributions
80 mcmc_areas(fit, pars = c('p_a', 'p_b', 'diff'), prob = 0.95)
81 ggsave('stan_posteriors.pdf', width = 10, height = 6)
82
83 # Trace plots
84 mcmc_trace(fit, pars = c('p_a', 'p_b'))
85 ggsave('stan_trace.pdf', width = 10, height = 6)
86
87 # Pairs plot
88 mcmc_pairs(fit, pars = c('p_a', 'p_b', 'diff'))
89 ggsave('stan_pairs.pdf', width = 10, height = 10)
90
91 # Posterior predictive check
92 # Generate replicated data from posterior
93 y_rep_a <- rbinom(length(samples$p_a), size = 3500, prob = samples$p_a)
94 y_rep_b <- rbinom(length(samples$p_b), size = 3500, prob = samples$p_b)
95
96 # Compare to observed
97 cat("\nPosterior Predictive Check:\n")
98 cat(sprintf("Observed conversions A: %d\n", 245))
99 cat(sprintf("Mean replicated conversions A: %.1f [95% PI: %d, %d]\n",
100            mean(y_rep_a), quantile(y_rep_a, 0.025),
101            quantile(y_rep_a, 0.975)))
102
103 cat(sprintf("\nObserved conversions B: %d\n", 289))
104 cat(sprintf("Mean replicated conversions B: %.1f [95% PI: %d, %d]\n",

```

```

105 mean(y_rep_b), quantile(y_rep_b, 0.025),
106 quantile(y_rep_b, 0.975)))

```

7.5 Practical Advantages of Bayesian A/B Testing

7.5.1 Intuitive Probability Statements

The Bayesian framework directly answers stakeholder questions:

- "What's the probability variant B beats variant A?"
→ $P(p_B > p_A|D) = 0.92$
- "What's the probability the lift exceeds 10%?"
→ $P(\text{lift} > 0.10|D) = 0.15$
- "What's the expected revenue impact?"
→ $\mathbb{E}[\text{revenue difference}|D] = \$45,000$
- "What's the risk if we choose the wrong variant?"
→ Expected loss = \$12,000

Compare to frequentist p-value: "If the null hypothesis were true and we repeated this experiment infinitely, we'd see data this extreme 3% of the time." This doesn't directly address the questions decision-makers care about.

7.5.2 Incorporation of Prior Knowledge

Bayesian methods naturally leverage historical data and domain expertise.

Example 7.4 (Leveraging Historical Data). A company has run 50 previous A/B tests on similar features, showing an average 5% conversion rate with standard deviation 2%. For a new test, they can use this information to construct an informative prior:

Match a $\text{Beta}(\alpha, \beta)$ prior to historical mean and variance:

$$\frac{\alpha}{\alpha + \beta} = 0.05 \quad (7.49)$$

$$\frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)} = 0.02^2 \quad (7.50)$$

Solving yields $\text{Beta}(11.9, 226.1)$ (approximately).

With this prior and new data (50 conversions from 1200 users), the posterior is:

$$\text{Beta}(61.9, 1376.1) \quad (7.51)$$

Posterior mean: $61.9/1438 = 0.043$

The prior stabilizes the estimate, preventing overfitting to small samples while respecting the new data.

Benefits:

- Rare event estimation: Priors stabilize estimates when events are rare

- Sequential experiments: Previous posterior becomes new prior
- Segment-level tests: Share information across segments via hierarchical priors
- Regulatory compliance: Document expert knowledge formally

7.5.3 Early Stopping Decisions

Frequentist hypothesis tests assume fixed sample size determined a priori. Checking results early ("peeking") and stopping when significant inflates Type I error rates.

Bayesian inference doesn't suffer from this problem. You can check results anytime and stop based on posterior probabilities without invalidating inference.

Example 7.5 (Early Stopping Rule). Define a stopping rule:

- Stop if $P(p_B > p_A | D) > 0.95$ (high confidence B is better)
- Stop if $P(p_B > p_A | D) < 0.05$ (high confidence A is better)
- Stop if $|\mathbb{E}[p_B - p_A | D]| < 0.002$ and $n > 5000$ (effect too small to matter)
- Stop if $n > 20,000$ (maximum sample size)

This rule balances statistical evidence, practical significance, and resource constraints. Checking daily doesn't inflate error rates in the Bayesian framework.

Practical Implementation:

Listing 7.8: Bayesian Early Stopping Simulation

```

1 import numpy as np
2 from scipy.stats import beta
3
4 def bayesian_sequential_test(p_a_true, p_b_true,
5                             max_n=20000,
6                             check_interval=500,
7                             prob_threshold=0.95,
8                             min_effect=0.002):
9     """
10     Simulate Bayesian sequential A/B test with early stopping.
11
12     Parameters:
13     -----
14     p_a_true, p_b_true: float
15         True conversion rates
16     max_n: int
17         Maximum sample size per variant
18     check_interval: int
19         How often to check stopping criteria
20     prob_threshold: float
21         Probability threshold for stopping
22     min_effect: float
23         Minimum detectable effect
24
25     Returns:
26     -----
27     dict: Stopping results

```

```

28 """
29 # Initialize
30 x_a, x_b = 0, 0
31 n_a, n_b = 0, 0
32 stopped = False
33 reason = None
34
35 # Prior parameters
36 prior_alpha, prior_beta = 1, 1
37
38 # Simulate data collection
39 for n in range(check_interval, max_n + 1, check_interval):
40     # Generate new data
41     new_x_a = np.random.binomial(check_interval, p_a_true)
42     new_x_b = np.random.binomial(check_interval, p_b_true)
43
44     x_a += new_x_a
45     x_b += new_x_b
46     n_a += check_interval
47     n_b += check_interval
48
49     # Posterior parameters
50     post_alpha_a = prior_alpha + x_a
51     post_beta_a = prior_beta + n_a - x_a
52     post_alpha_b = prior_alpha + x_b
53     post_beta_b = prior_beta + n_b - x_b
54
55     # Sample from posteriors
56     samples_a = np.random.beta(post_alpha_a, post_beta_a, 10000)
57     samples_b = np.random.beta(post_alpha_b, post_beta_b, 10000)
58
59     # Compute decision metrics
60     prob_b_better = np.mean(samples_b > samples_a)
61     expected_diff = np.mean(samples_b - samples_a)
62
63     # Check stopping criteria
64     if prob_b_better > prob_threshold:
65         stopped = True
66         reason = f'High confidence B better (P={prob_b_better:.3f})'
67         break
68     elif prob_b_better < (1 - prob_threshold):
69         stopped = True
70         reason = f'High confidence A better (P={1-prob_b_better:.3f})'
71         break
72     elif n > 5000 and abs(expected_diff) < min_effect:
73         stopped = True
74         reason = f'Effect too small (E[diff]={expected_diff:.5f})'
75         break
76
77 if not stopped:
78     reason = f'Reached maximum sample size (n={max_n})'
79
80 return {
81     'stopped': stopped,
82     'reason': reason,

```

```

83         'n_total': n_a + n_b,
84         'n_per_variant': n_a,
85         'conversions_a': x_a,
86         'conversions_b': x_b,
87         'observed_rate_a': x_a / n_a,
88         'observed_rate_b': x_b / n_b,
89         'prob_b_better': prob_b_better
90     }
91
92 # Simulation: Compare to fixed sample size
93 np.random.seed(42)
94
95 # Scenario 1: Large effect (easy to detect early)
96 print("Scenario 1: Large effect (7% vs. 8.5%)")
97 result1 = bayesian_sequential_test(p_a_true=0.07, p_b_true=0.085)
98 print(f"Stopped: {result1['reason']}")
99 print(f"Total sample size: {result1['n_total']}")
100 print(f"P(B > A): {result1['prob_b_better']:.4f}\n")
101
102 # Scenario 2: Small effect (needs more data)
103 print("Scenario 2: Small effect (7% vs. 7.2%)")
104 result2 = bayesian_sequential_test(p_a_true=0.07, p_b_true=0.072)
105 print(f"Stopped: {result2['reason']}")
106 print(f"Total sample size: {result2['n_total']}")
107 print(f"P(B > A): {result2['prob_b_better']:.4f}\n")
108
109 # Scenario 3: No effect
110 print("Scenario 3: No effect (7% vs. 7%)")
111 result3 = bayesian_sequential_test(p_a_true=0.07, p_b_true=0.07)
112 print(f"Stopped: {result3['reason']}")
113 print(f"Total sample size: {result3['n_total']}")
114 print(f"P(B > A): {result3['prob_b_better']:.4f}\n")
115
116 # Monte Carlo simulation of stopping times
117 print("Monte Carlo simulation of early stopping (1000 trials):")
118 n_sims = 1000
119 stopping_times = []
120
121 for _ in range(n_sims):
122     result = bayesian_sequential_test(p_a_true=0.07, p_b_true=0.085)
123     stopping_times.append(result['n_total'])
124
125 print(f"Mean stopping time: {np.mean(stopping_times):.0f}")
126 print(f"Median stopping time: {np.median(stopping_times):.0f}")
127 print(f"SD stopping time: {np.std(stopping_times):.0f}")
128 print(f"Fixed sample size (80% power): ~8000")
129 print(f"Average reduction: {(1 - np.mean(stopping_times) / 8000) * 100:.1f}%")

```

7.5.4 Risk-Based Decision Making

Expected loss provides a framework for risk-based decisions that accounts for both statistical evidence and business consequences.

Example 7.6 (Cost-Benefit Analysis with Expected Loss). A company tests a new check-out flow:

- Expected lift: 1.2 percentage points
- Expected loss if wrong: 0.15 percentage points
- Annual revenue impact of 1 percentage point: \$500,000
- Implementation cost: \$100,000

Expected value analysis:

$$\text{Expected benefit} = 0.012 \times \$500,000 = \$6,000 \quad (7.52)$$

$$\text{Expected regret} = 0.0015 \times \$500,000 = \$750 \quad (7.53)$$

$$\text{Net expected value} = \$6,000 - \$750 - \$100,000 = -\$94,750 \quad (7.54)$$

Despite statistical significance, the expected value is negative! The implementation cost exceeds expected benefit after accounting for uncertainty.

This framework integrates statistical and business considerations coherently.

Value of Information:

The expected value of perfect information (EVPI) quantifies how much we'd pay to eliminate uncertainty:

$$\text{EVPI} = \text{Expected loss with current information} \quad (7.55)$$

If $\text{EVPI} < \text{cost of additional data collection}$, stop testing and decide now. Otherwise, continue collecting data.

7.5.5 Full Uncertainty Quantification

The entire posterior distribution is available for downstream analyses:

- Propagate uncertainty through decision models
- Compute any statistic of interest (quantiles, probabilities, etc.)
- Visualize full uncertainty, not just point estimates
- Combine with other probabilistic models (e.g., user lifetime value)

7.6 Comparison with Frequentist Methods

7.6.1 Philosophical Differences

7.6.2 Practical Implications

Sample Size:

- Frequentist: Fixed sample size based on power analysis
- Bayesian: Can stop early based on posterior probabilities; no fixed requirement

Interpretation:

Aspect	Frequentist	Bayesian
Parameters	Fixed but unknown constants	Random variables with distributions
Probability	Long-run frequency over repeated experiments	Degree of belief/uncertainty
Inference	Confidence intervals, p-values	Credible intervals, posterior probabilities
Prior information	Not formally incorporated	Explicitly modeled via priors
Interpretation	Indirect (properties of procedures)	Direct (probability of hypotheses)
Sequential testing	Problematic (inflates Type I error)	Natural (no inflation)
Stopping rules	Must be pre-specified	Can be data-dependent
Null hypothesis	Can test point null	No special status for null

Table 7.3: Philosophical differences between frequentist and Bayesian frameworks

- Frequentist: "We reject the null at $\alpha = 0.05$ " (binary decision)
- Bayesian: "There's an 85% probability B is better, with expected lift of 12%" (continuous uncertainty)

Prior Dependence:

- Frequentist: No priors; "objective"
- Bayesian: Results depend on priors, especially with limited data; requires sensitivity analysis

Computational Burden:

- Frequentist: Often closed-form (t-test, z-test); fast
- Bayesian: May require MCMC; can be slow for complex models

7.6.3 When to Use Each Approach

Recommendation: Both frameworks have value. Consider:

- Organizational culture and stakeholder familiarity
- Regulatory requirements
- Availability of prior information
- Need for sequential testing
- Computational resources

Scenario	Frequentist	Bayesian
Fixed sample size, simple test	✓✓	✓
Sequential/adaptive testing		✓✓
Prior information available	✓	✓✓
Regulatory requirement (FDA, etc.)	✓✓	
Simple stakeholder communication	✓✓	✓
Probability statements about parameters		✓✓
Multiple testing	✓	✓
Rare events/small samples		✓✓
Hierarchical structure	✓	✓✓
Cost-benefit decision framework		✓✓

Table 7.4: Recommended approach by scenario (checkmarks indicate suitability)

- Complexity of decision framework

Many organizations adopt hybrid approaches: frequentist for primary confirmatory analysis (regulatory compliance), Bayesian for secondary exploratory analysis and decision-making.

7.6.4 Empirical Comparison Study

Listing 7.9: Simulation Study Comparing Frequentist and Bayesian Approaches

```

1 import numpy as np
2 from scipy import stats
3 from scipy.stats import beta
4 import matplotlib.pyplot as plt
5
6 def frequentist_test(x_a, n_a, x_b, n_b, alpha=0.05):
7     """Two-proportion z-test."""
8     p_a = x_a / n_a
9     p_b = x_b / n_b
10    p_pool = (x_a + x_b) / (n_a + n_b)
11
12    se = np.sqrt(p_pool * (1 - p_pool) * (1/n_a + 1/n_b))
13    z = (p_b - p_a) / se
14    p_value = 2 * (1 - stats.norm.cdf(abs(z)))
15
16    reject_null = p_value < alpha
17    ci = (p_b - p_a) + np.array([-1, 1]) * stats.norm.ppf(1 - alpha/2)
18        * se
19
20    return {
21        'reject_null': reject_null,
22        'p_value': p_value,
23        'ci': ci,
24        'effect': p_b - p_a
25    }
26
27 def bayesian_test(x_a, n_a, x_b, n_b, prior_alpha=1, prior_beta=1):
28     """Bayesian test with Beta-Binomial model."""
29     post_alpha_a = prior_alpha + x_a

```

```

29 post_beta_a = prior_beta + n_a - x_a
30 post_alpha_b = prior_alpha + x_b
31 post_beta_b = prior_beta + n_b - x_b
32
33 # Monte Carlo
34 samples_a = np.random.beta(post_alpha_a, post_beta_a, 100000)
35 samples_b = np.random.beta(post_alpha_b, post_beta_b, 100000)
36
37 prob_b_better = np.mean(samples_b > samples_a)
38 diff = samples_b - samples_a
39 ci = np.percentile(diff, [2.5, 97.5])
40
41 # Decision: "significant" if  $P(B > A) > 0.95$ 
42 decision = prob_b_better > 0.95 or prob_b_better < 0.05
43
44 return {
45     'decision': decision,
46     'prob_b_better': prob_b_better,
47     'ci': ci,
48     'effect': np.mean(diff)
49 }
50
51 # Simulation study
52 np.random.seed(42)
53 n_sims = 1000
54 sample_sizes = [500, 1000, 2000, 5000]
55 true_effects = [0, 0.01, 0.02, 0.03] # True differences
56
57 results = []
58
59 for n_per_variant in sample_sizes:
60     for true_effect in true_effects:
61         p_a_true = 0.05
62         p_b_true = p_a_true + true_effect
63
64         freq_rejections = 0
65         bayes_decisions = 0
66
67         for _ in range(n_sims):
68             # Generate data
69             x_a = np.random.binomial(n_per_variant, p_a_true)
70             x_b = np.random.binomial(n_per_variant, p_b_true)
71
72             # Frequentist test
73             freq_result = frequentist_test(x_a, n_per_variant,
74                                           x_b, n_per_variant)
75             if freq_result['reject_null']:
76                 freq_rejections += 1
77
78             # Bayesian test
79             bayes_result = bayesian_test(x_a, n_per_variant,
80                                         x_b, n_per_variant)
81             if bayes_result['decision']:
82                 bayes_decisions += 1
83
84         freq_rate = freq_rejections / n_sims
85         bayes_rate = bayes_decisions / n_sims
86

```

```

87     results.append({
88         'n': n_per_variant,
89         'true_effect': true_effect,
90         'frequentist_rate': freq_rate,
91         'bayesian_rate': bayes_rate
92     })
93
94     print(f"n={n_per_variant}, effect={true_effect:.3f}: "
95           f"Freq={freq_rate:.3f}, Bayes={bayes_rate:.3f}")
96
97 # Visualize
98 import pandas as pd
99
100 df = pd.DataFrame(results)
101
102 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
103 axes = axes.flatten()
104
105 for i, n in enumerate(sample_sizes):
106     ax = axes[i]
107     df_n = df[df['n'] == n]
108
109     ax.plot(df_n['true_effect'], df_n['frequentist_rate'],
110            'o-', label='Frequentist', linewidth=2)
111     ax.plot(df_n['true_effect'], df_n['bayesian_rate'],
112            's--', label='Bayesian', linewidth=2)
113     ax.axhline(0.05, color='red', linestyle=':',
114               label='Nominal Type I error rate')
115     ax.set_xlabel('True Effect Size')
116     ax.set_ylabel('Rejection/Decision Rate')
117     ax.set_title(f'Sample Size = {n} per variant')
118     ax.legend()
119     ax.grid(alpha=0.3)
120
121 plt.tight_layout()
122 plt.savefig('frequentist_vs_bayesian_comparison.pdf')
123 plt.show()
124
125 print("\nInterpretation:")
126 print("- When true effect = 0: Rejection rate approximates Type I error")
127 print("- When true effect > 0: Rejection rate approximates Power")
128 print("- Bayesian tends to be more conservative with uniform priors")
129 print("- Both approaches converge to high power with large effects / samples")

```

7.6.5 Computational Considerations

Frequentist Computation:

- Closed-form solutions for common tests (t-test, z-test, chi-square)
- Very fast: milliseconds for standard tests
- Scales easily to massive datasets
- No convergence diagnostics needed

Bayesian Computation:

- Conjugate models: analytical, as fast as frequentist
- Monte Carlo: moderate cost, seconds to minutes
- MCMC: can be slow (minutes to hours for complex models)
- Requires convergence diagnostics (R-hat, ESS)
- Parallelizable across chains

Practical Guidelines:

- Simple A/B test with Beta-Binomial: Bayesian analytical (milliseconds)
- Multivariate/hierarchical: Bayesian MCMC (minutes) or frequentist approximations
- Real-time dashboards: Frequentist or Bayesian analytical only
- Offline analysis: Either approach feasible

7.7 Summary

Bayesian A/B testing offers a powerful alternative framework to frequentist methods:

- **Direct probability statements:** "85% probability B is better" vs. indirect p-values
- **Prior incorporation:** Leverage historical data and expert knowledge
- **Sequential testing:** Check anytime without inflating error rates
- **Risk quantification:** Expected loss guides decisions
- **Full uncertainty:** Entire posterior distribution available
- **Flexible:** Analytical conjugate models or MCMC for complex cases

Trade-offs:

- Requires prior specification and sensitivity analysis
- Can be computationally intensive (MCMC)
- Less familiar to many stakeholders
- No Type I/II error rate guarantees (different paradigm)

Recommendation: Learn both frameworks. Choose based on organizational context, stakeholder preferences, regulatory requirements, and testing needs. Hybrid approaches combining frequentist confirmatory tests with Bayesian decision analysis are increasingly common and capture benefits of both paradigms.

7.8 Further Reading

- **Gelman et al. (2013)**, *Bayesian Data Analysis*: Comprehensive graduate-level treatment
- **Kruschke (2014)**, *Doing Bayesian Data Analysis*: Accessible introduction with extensive examples
- **McElreath (2020)**, *Statistical Rethinking*: Modern Bayesian perspective with causal inference focus
- **Stucchio (2015)**, "Bayesian A/B Testing at VWO": Practitioner-focused guide
- **Davidson-Pilon (2015)**, *Bayesian Methods for Hackers*: Programming-first introduction using PyMC
- **Carpenter et al. (2017)**, "Stan: A Probabilistic Programming Language": Reference for Stan

7.9 Exercises

1. Beta-Binomial Derivation

- Derive the posterior distribution for the Beta-Binomial conjugate model from first principles using Bayes' theorem.
- Show that $\text{Beta}(1, 1)$ is uniform on $[0, 1]$ and that $\text{Beta}(0.5, 0.5)$ is Jeffrey's prior.
- Prove that the posterior mean is a weighted average of prior mean and data mean.

2. Prior Sensitivity Analysis

- For observed data of 15 conversions from 200 users, compute posteriors for:
 - Uniform prior: $\text{Beta}(1, 1)$
 - Weakly informative: $\text{Beta}(5, 95)$
 - Informative: $\text{Beta}(20, 380)$
- Plot the three posteriors. At what sample size do they converge?
- Compute 95% credible intervals for each. How much do they differ?

3. Bayesian A/B Test Implementation

- Implement a complete Bayesian A/B test in Python including:
 - Posterior computation
 - Probability of superiority
 - Expected loss
 - Visualizations
- Apply to data: A (245/3500), B (289/3500). Should you launch B?
- Conduct prior sensitivity analysis varying prior strength.

4. Sequential Testing Simulation

- (a) Simulate Bayesian sequential testing with stopping rule: stop if $P(B > A) > 0.95$.
- (b) Compare average stopping time to fixed sample size (80% power, $\alpha = 0.05$).
- (c) Vary true effect size (0%, 1%, 2%, 5%). How does stopping time depend on effect?
- (d) Does early stopping inflate Type I error? Run simulations with null hypothesis true.

5. Expected Loss and Value of Information

- (a) For an A/B test with 100 conversions from 1500 users (A) and 115 from 1500 (B), compute:
 - Expected loss for each decision
 - EVPI (expected value of perfect information)
 - Value of collecting 500 more users per variant
- (b) If each user costs \$0.10 to test and each percentage point of conversion is worth \$10,000 annually, should you collect more data or decide now?

6. PyMC3 Hierarchical Model

- (a) Implement a hierarchical Bayesian A/B test in PyMC3 for testing across 4 market segments.
- (b) Use partial pooling to share information across segments.
- (c) Compare to unpooled (separate tests per segment) and completely pooled (ignoring segments).
- (d) Assess convergence using R-hat and effective sample size.

7. Frequentist vs. Bayesian Comparison

- (a) Run a simulation study comparing frequentist two-proportion z-test to Bayesian Beta-Binomial.
- (b) Vary: sample size (100-10000), true effect (0%-5%), prior (uniform vs. informative).
- (c) Compare: Type I error rate, power, average confidence/credible interval width.
- (d) Under what conditions do they give substantially different conclusions?

8. Non-Conjugate Model with MCMC

- (a) Implement a Bayesian logistic regression for A/B testing with covariates using PyMC3 or Stan.
- (b) Compare to frequentist logistic regression with interaction terms.
- (c) Assess MCMC convergence and effective sample size.
- (d) Compute posterior probabilities for treatment effect in different segments.

Chapter 8

Advanced Experimental Designs

8.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design and analyze factorial experiments testing multiple factors simultaneously
- Implement multi-armed bandit algorithms for dynamic traffic allocation
- Apply contextual bandits when user features are available
- Conduct sequential testing with appropriate statistical controls
- Use SPRT and alpha spending functions for early stopping
- Apply quasi-experimental designs when randomization isn't possible
- Handle network effects and interference in experimental settings
- Design experiments for two-sided marketplaces
- Choose appropriate advanced designs for specific business contexts

8.2 Introduction: Beyond Traditional A/B Testing

Traditional A/B tests—randomly splitting users 50/50 between control and treatment, collecting a predetermined sample size, and analyzing once—serve well for many scenarios. But modern experimentation demands more sophisticated approaches:

- Testing multiple factors simultaneously (factorial designs)
- Adapting allocation based on performance (bandits)
- Stopping experiments early when evidence accumulates (sequential testing)
- Drawing causal inferences without randomization (quasi-experiments)
- Handling interference when users interact (network effects)
- Accounting for platform-specific complexities (marketplaces, ecosystems)

This chapter explores these advanced experimental designs, their theoretical foundations, implementation challenges, and practical applications.

8.3 Factorial Designs

Factorial designs test multiple factors simultaneously, providing efficiency gains and enabling the detection of interaction effects that would be missed in separate experiments.

8.3.1 Full Factorial Designs

A 2^k factorial design tests k binary factors with all 2^k combinations.

Example 8.1 (2×2 Factorial Design). Testing email subject line (A vs. B) and send time (morning vs. evening):

	Morning	Evening
Subject A	Cell 1 (n=250)	Cell 2 (n=250)
Subject B	Cell 3 (n=250)	Cell 4 (n=250)

With 1,000 total users, we can estimate:

- Main effect of subject line: Effect of A vs. B averaged across times
- Main effect of send time: Effect of morning vs. evening averaged across subjects
- Interaction: Does subject line effect depend on time of day?

8.3.2 Statistical Model for Factorial Experiments

For a two-factor design:

$$Y_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \epsilon_{ijk} \quad (8.1)$$

where:

- μ : Grand mean
- α_i : Main effect of factor A (level i)
- β_j : Main effect of factor B (level j)
- $(\alpha\beta)_{ij}$: Interaction effect
- ϵ_{ijk} : Random error, $\epsilon_{ijk} \sim \mathcal{N}(0, \sigma^2)$

Main Effects:

$$\text{Effect of A} = \bar{Y}_{A \text{ high}} - \bar{Y}_{A \text{ low}} \quad (8.2)$$

$$\text{Effect of B} = \bar{Y}_{B \text{ high}} - \bar{Y}_{B \text{ low}} \quad (8.3)$$

Interaction Effect:

$$\text{Interaction} = (\bar{Y}_{11} - \bar{Y}_{10}) - (\bar{Y}_{01} - \bar{Y}_{00}) \quad (8.4)$$

An interaction exists when the effect of one factor depends on the level of another factor.

Theorem 8.1 (Efficiency of Factorial Designs). A 2^k factorial design with n observations per cell provides the same precision for estimating main effects as k separate experiments each with $2^{k-1} \times n$ observations, while also estimating all interaction effects.

Proof. For main effect of factor A in factorial design:

$$\text{Var}(\hat{\alpha}_A) = \frac{4\sigma^2}{2^k n} \quad (8.5)$$

For separate A/B test with sample size N :

$$\text{Var}(\hat{\alpha}_A^{\text{sep}}) = \frac{2\sigma^2}{N} \quad (8.6)$$

Setting variances equal:

$$\frac{4\sigma^2}{2^k n} = \frac{2\sigma^2}{N} \implies N = 2^{k-1} n \quad (8.7)$$

Thus, factorial design achieves same precision with fraction of sample size, while also estimating interactions. $\square$

Listing 8.1: Comprehensive Factorial Design Analysis

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from scipy import stats
6 import statsmodels.api as sm
7 from statsmodels.formula.api import ols
8 from typing import Dict, Tuple
9
10 def simulate_factorial_experiment(
11     n_per_cell: int,
12     main_effect_A: float,
13     main_effect_B: float,
14     interaction: float,
15     baseline: float = 0.10,
16     seed: int = None
17 ) -> pd.DataFrame:
18     """
19     Simulate a 2x2 factorial experiment with binary outcome.
20
21     Parameters:
22     -----
23     n_per_cell: int
24         Sample size per cell
25     main_effect_A: float
26         Main effect of factor A (additive on probability scale)
27     main_effect_B: float
28         Main effect of factor B
29     interaction: float
30         Interaction effect (additional effect when both A and B are
31         present)
32     baseline: float
33         Baseline conversion rate (A=0, B=0)

```

```

33     """seed: int
34     """Random seed
35
36     Returns:
37     """-----
38     """pd.DataFrame: Simulated experiment data
39     """
40     if seed is not None:
41         np.random.seed(seed)
42
43     # Define cell probabilities
44     probabilities = {
45         (0, 0): baseline,
46         (1, 0): baseline + main_effect_A,
47         (0, 1): baseline + main_effect_B,
48         (1, 1): baseline + main_effect_A + main_effect_B + interaction
49     }
50
51     data_list = []
52
53     for factor_A in [0, 1]:
54         for factor_B in [0, 1]:
55             p = probabilities[(factor_A, factor_B)]
56             conversions = np.random.binomial(1, p, n_per_cell)
57
58             cell_data = pd.DataFrame({
59                 'factor_A': factor_A,
60                 'factor_B': factor_B,
61                 'conversion': conversions,
62                 'user_id': range(
63                     (factor_A * 2 + factor_B) * n_per_cell,
64                     (factor_A * 2 + factor_B + 1) * n_per_cell
65                 )
66             })
67             data_list.append(cell_data)
68
69     data = pd.concat(data_list, ignore_index=True)
70
71     # Add categorical labels
72     data['factor_A_label'] = data['factor_A'].map({0: 'A0', 1: 'A1'})
73     data['factor_B_label'] = data['factor_B'].map({0: 'B0', 1: 'B1'})
74
75     return data
76
77 def analyze_factorial_experiment(data: pd.DataFrame) -> Dict:
78     """
79     Comprehensive analysis of 2x2 factorial experiment.
80
81     Parameters:
82     """-----
83     """data: pd.DataFrame
84     """Must contain columns: factor_A, factor_B, conversion
85
86     Returns:
87     """-----
88     """dict: Analysis results including effects, tests, and diagnostics
89     """
90     # Cell means

```

```

91 cell_means = data.groupby(['factor_A', 'factor_B'])['conversion'].
92     agg([
93         'mean', 'count', 'std'
94     ]).reset_index()
95
96 # Extract cell means
97 y00 = cell_means[(cell_means['factor_A']==0) &
98                 (cell_means['factor_B']==0)]['mean'].values[0]
99 y10 = cell_means[(cell_means['factor_A']==1) &
100                 (cell_means['factor_B']==0)]['mean'].values[0]
101 y01 = cell_means[(cell_means['factor_A']==0) &
102                 (cell_means['factor_B']==1)]['mean'].values[0]
103 y11 = cell_means[(cell_means['factor_A']==1) &
104                 (cell_means['factor_B']==1)]['mean'].values[0]
105
106 # Calculate effects
107 main_effect_A = ((y10 + y11) / 2) - ((y00 + y01) / 2)
108 main_effect_B = ((y01 + y11) / 2) - ((y00 + y10) / 2)
109 interaction_effect = (y11 - y10) - (y01 - y00)
110
111 # ANOVA using statsmodels
112 formula = 'conversion ~ C(factor_A) * C(factor_B)'
113 model = ols(formula, data=data).fit()
114 anova_table = sm.stats.anova_lm(model, typ=2)
115
116 # Effect sizes (Cohen's d for binary factors)
117 overall_std = data['conversion'].std()
118 cohens_d_A = main_effect_A / overall_std
119 cohens_d_B = main_effect_B / overall_std
120 cohens_d_interaction = interaction_effect / overall_std
121
122 # Confidence intervals for effects (using bootstrap)
123 n_bootstrap = 1000
124 bootstrap_effects = {
125     'main_A': [],
126     'main_B': [],
127     'interaction': []
128 }
129
130 np.random.seed(42)
131 for _ in range(n_bootstrap):
132     boot_sample = data.sample(n=len(data), replace=True)
133     boot_means = boot_sample.groupby(
134         ['factor_A', 'factor_B']
135     )['conversion'].mean()
136
137     b_y00 = boot_means[(0, 0)]
138     b_y10 = boot_means[(1, 0)]
139     b_y01 = boot_means[(0, 1)]
140     b_y11 = boot_means[(1, 1)]
141
142     bootstrap_effects['main_A'].append(
143         ((b_y10 + b_y11) / 2) - ((b_y00 + b_y01) / 2)
144     )
145     bootstrap_effects['main_B'].append(
146         ((b_y01 + b_y11) / 2) - ((b_y00 + b_y10) / 2)
147     )
148     bootstrap_effects['interaction'].append(

```

```

148         (b_y11 - b_y10) - (b_y01 - b_y00)
149     )
150
151     ci_main_A = np.percentile(bootstrap_effects['main_A'], [2.5, 97.5])
152     ci_main_B = np.percentile(bootstrap_effects['main_B'], [2.5, 97.5])
153     ci_interaction = np.percentile(bootstrap_effects['interaction'],
154                                   [2.5, 97.5])
155
156     return {
157         'cell_means': {
158             'A0_B0': y00,
159             'A1_B0': y10,
160             'A0_B1': y01,
161             'A1_B1': y11
162         },
163         'effects': {
164             'main_effect_A': main_effect_A,
165             'main_effect_B': main_effect_B,
166             'interaction': interaction_effect
167         },
168         'effect_sizes': {
169             'cohens_d_A': cohens_d_A,
170             'cohens_d_B': cohens_d_B,
171             'cohens_d_interaction': cohens_d_interaction
172         },
173         'confidence_intervals': {
174             'main_A_ci': ci_main_A,
175             'main_B_ci': ci_main_B,
176             'interaction_ci': ci_interaction
177         },
178         'anova_table': anova_table,
179         'model': model
180     }
181
182 def visualize_factorial_experiment(data: pd.DataFrame, results: Dict):
183     """Create comprehensive visualizations for factorial experiment."""
184     fig, axes = plt.subplots(2, 2, figsize=(14, 12))
185
186     # 1. Cell means with error bars
187     ax = axes[0, 0]
188     cell_stats = data.groupby(['factor_A_label', 'factor_B_label'])['conversion']
189     ].agg(['mean', 'sem'])
190
191     x_pos = np.arange(4)
192     cell_labels = ['A0_B0', 'A1_B0', 'A0_B1', 'A1_B1']
193     means = [results['cell_means'][label] for label in cell_labels]
194
195     # Calculate SEM for error bars
196     sems = []
197     for a in [0, 1]:
198         for b in [0, 1]:
199             subset = data[(data['factor_A']==a) & (data['factor_B']==b)]
200             sems.append(subset['conversion'].sem())
201
202     ax.bar(x_pos, means, yerr=[1.96*s for s in sems],
203           capsize=5, alpha=0.7, color=['C0', 'C1', 'C0', 'C1'])

```

```

204 ax.set_xticks(x_pos)
205 ax.set_xticklabels(cell_labels)
206 ax.set_ylabel('Conversion_Rate')
207 ax.set_title('Cell_Means_with_95%_CI')
208 ax.grid(axis='y', alpha=0.3)
209
210 # 2. Interaction plot
211 ax = axes[0, 1]
212 for b_val, b_label in [(0, 'B0'), (1, 'B1')]:
213     subset = data[data['factor_B'] == b_val]
214     means_by_A = subset.groupby('factor_A')['conversion'].mean()
215     ax.plot([0, 1], means_by_A.values, marker='o',
216            markersize=10, linewidth=2, label=b_label)
217
218 ax.set_xticks([0, 1])
219 ax.set_xticklabels(['A0', 'A1'])
220 ax.set_xlabel('Factor_A')
221 ax.set_ylabel('Conversion_Rate')
222 ax.set_title('Interaction_Plot')
223 ax.legend(title='Factor_B')
224 ax.grid(alpha=0.3)
225
226 # 3. Main effects visualization
227 ax = axes[1, 0]
228 effects = results['effects']
229 effect_names = ['Main_Effect_A', 'Main_Effect_B', 'Interaction']
230 effect_values = [
231     effects['main_effect_A'],
232     effects['main_effect_B'],
233     effects['interaction']
234 ]
235
236 cis = [
237     results['confidence_intervals']['main_A_ci'],
238     results['confidence_intervals']['main_B_ci'],
239     results['confidence_intervals']['interaction_ci']
240 ]
241
242 y_pos = np.arange(len(effect_names))
243 colors = ['green' if ci[0] > 0 or ci[1] < 0 else 'gray'
244          for ci in cis]
245
246 ax.barh(y_pos, effect_values, color=colors, alpha=0.7)
247
248 for i, (val, ci) in enumerate(zip(effect_values, cis)):
249     ax.plot([ci[0], ci[1]], [i, i], 'k-', linewidth=2)
250     ax.plot([ci[0], ci[0]], [i-0.1, i+0.1], 'k-', linewidth=2)
251     ax.plot([ci[1], ci[1]], [i-0.1, i+0.1], 'k-', linewidth=2)
252
253 ax.axvline(x=0, color='red', linestyle='--', linewidth=1)
254 ax.set_yticks(y_pos)
255 ax.set_yticklabels(effect_names)
256 ax.set_xlabel('Effect_Size_(Absolute)')
257 ax.set_title('Effects_with_95%_Bootstrap_CI')
258 ax.grid(axis='x', alpha=0.3)
259
260 # 4. Heatmap of cell means
261 ax = axes[1, 1]

```

```

262 heatmap_data = np.array([
263     [results['cell_means']['A0_B0'], results['cell_means']['A0_B1',
264         [results['cell_means']['A1_B0'], results['cell_means']['A1_B1'
265         ]])
266
267 im = ax.imshow(heatmap_data, cmap='RdYlGn', aspect='auto')
268 ax.set_xticks([0, 1])
269 ax.set_yticks([0, 1])
270 ax.set_xticklabels(['B0', 'B1'])
271 ax.set_yticklabels(['A0', 'A1'])
272 ax.set_xlabel('Factor_B')
273 ax.set_ylabel('Factor_A')
274 ax.set_title('Heatmap of Conversion Rates')
275
276 # Add text annotations
277 for i in range(2):
278     for j in range(2):
279         text = ax.text(j, i, f'{heatmap_data[i,j]:.3f}',
280             ha="center", va="center", color="black",
281             fontsize=14, fontweight='bold')
282
283 plt.colorbar(im, ax=ax)
284
285 plt.tight_layout()
286 return fig
287
288 # Example: Simulate and analyze factorial experiment
289 np.random.seed(42)
290 data = simulate_factorial_experiment(
291     n_per_cell=500,
292     main_effect_A=0.02,      # Factor A increases conversion by 2pp
293     main_effect_B=0.015,    # Factor B increases conversion by 1.5pp
294     interaction=0.01,       # Additional 1pp when both are combined
295     baseline=0.10
296 )
297
298 results = analyze_factorial_experiment(data)
299
300 print("=== Factorial Experiment Analysis ===\n")
301 print("Cell Means:")
302 for cell, mean in results['cell_means'].items():
303     print(f"{cell}: {mean:.4f}")
304
305 print("\nEstimated Effects:")
306 print(f"Main Effect A: {results['effects']['main_effect_A']:.4f}")
307 print(f"Main Effect B: {results['effects']['main_effect_B']:.4f}")
308 print(f"Interaction: {results['effects']['interaction']:.4f}")
309
310 print("\nEffect Sizes (Cohen's d):")
311 print(f"Main Effect A: {results['effect_sizes']['cohens_d_A']:.3f}")
312 print(f"Main Effect B: {results['effect_sizes']['cohens_d_B']:.3f}")
313 print(f"Interaction: {results['effect_sizes']['cohens_d_interaction']:.3f}")
314
315 print("\n95% Confidence Intervals:")
316 print(f"Main A: [{results['confidence_intervals']['main_A_ci'][0]:.4f

```



```

317         f"{results['confidence_intervals']['main_A_ci'][1]:.4f}]"
318 print(f"Main B: [{results['confidence_intervals']['main_B_ci'][0]:.4f
319         },
319         f"{results['confidence_intervals']['main_B_ci'][1]:.4f}]"
320 print(f"Interaction: [{results['confidence_intervals']['interaction_
321         ci'][0]:.4f},
321         f"{results['confidence_intervals']['interaction_ci'][1]:.4f}]"
322
323 print("\nANOVA Table:")
324 print(results['anova_table'])
325
326 # Create visualizations
327 fig = visualize_factorial_experiment(data, results)
328 plt.savefig('factorial_analysis.pdf', bbox_inches='tight')
329 plt.show()

```

8.3.3 Fractional Factorial Designs

When testing many factors, full factorial designs become impractical. A 2^k design requires 2^k experimental conditions.

Problem: Testing 6 factors requires $2^6 = 64$ cells. With 100 users per cell, need 6,400 users.

Solution: Fractional factorial designs test a carefully chosen subset of conditions.

Example 8.2 (Half-Fraction Design). A 2^{5-1} (read: "two to the five minus one") design tests 5 factors using only $2^4 = 16$ conditions instead of 32.

We sacrifice the ability to estimate certain high-order interactions (which are often negligible) to gain efficiency.

Confounding: In fractional designs, some effects cannot be separately estimated (they are "aliased").

For 2^{5-1} with defining relation $I = ABCDE$:

- Main effect A is aliased with 4-way interaction BCDE
- Two-way interaction AB is aliased with CDE

Resolution: Describes which effects are confounded:

- Resolution III: Main effects clear of each other but aliased with 2-way interactions
- Resolution IV: Main effects clear; 2-way interactions aliased with other 2-way interactions
- Resolution V: Main effects and 2-way interactions clear

Practical Recommendation: Use Resolution IV or V designs when possible. Assume high-order interactions are negligible.

8.4 Multi-Armed Bandits

Multi-armed bandits (MAB) dynamically allocate traffic to variants, balancing exploration (learning about variants) and exploitation (assigning users to better-performing variants).

8.4.1 The Exploration-Exploitation Tradeoff

Traditional A/B test: Fixed allocation (e.g., 50/50) throughout experiment.

Multi-armed bandit: Adaptive allocation that shifts traffic toward better variants as data accumulates.

Regret: Cumulative opportunity cost from suboptimal assignments:

$$R(T) = \sum_{t=1}^T (\mu^* - \mu_{A_t}) \quad (8.8)$$

where:

- T : Total number of users
- μ^* : True mean of best variant
- A_t : Variant assigned to user t
- μ_{A_t} : True mean of assigned variant

Goal: Minimize cumulative regret while maintaining valid inference.

8.4.2 ϵ -Greedy Algorithm

Simplest bandit algorithm: with probability ϵ , explore randomly; otherwise, exploit current best.

Algorithm 5 ϵ -Greedy Bandit

Initialize: $N_i = 0, \hat{\mu}_i = 0$ for all variants $i = 1, \dots, K$

for each user $t = 1, 2, \dots, T$ **do**

 Generate $u \sim \text{Uniform}(0, 1)$

if $u < \epsilon$ **then**

 Select random variant A_t (explore)

else

 Select $A_t = \arg \max_i \hat{\mu}_i$ (exploit)

end if

 Observe reward r_t

 Update: $N_{A_t} \leftarrow N_{A_t} + 1$

 Update: $\hat{\mu}_{A_t} \leftarrow \frac{(N_{A_t}-1)\hat{\mu}_{A_t} + r_t}{N_{A_t}}$

end for

Parameter Selection:

- Fixed $\epsilon \in [0.05, 0.20]$: Simple but continues exploring indefinitely
- Decaying $\epsilon_t = \min(1, \epsilon_0/t)$: Reduces exploration over time

8.4.3 Upper Confidence Bound (UCB) Algorithm

UCB addresses ϵ -greedy's limitation by explicitly balancing exploitation and exploration using confidence intervals.

Algorithm 6 UCB1 Algorithm

Initialize: $N_i = 0, \hat{\mu}_i = 0$ for all variants i

Play each variant once to initialize

for each user $t = K + 1, K + 2, \dots, T$ **do**

 Select variant:

$$A_t = \arg \max_i \left[\hat{\mu}_i + \sqrt{\frac{2 \ln t}{N_i}} \right]$$

 Observe reward r_t

 Update N_{A_t} and $\hat{\mu}_{A_t}$

end for

The UCB term $\sqrt{\frac{2 \ln t}{N_i}}$ represents uncertainty: variants with fewer observations get bonus exploration.

Theorem 8.2 (Auer et al., 2002). UCB1 achieves logarithmic regret:

$$R(T) \leq 8 \sum_{i: \mu_i < \mu^*} \frac{\ln T}{\Delta_i} + \left(1 + \frac{\pi^2}{3}\right) \sum_{i=1}^K \Delta_i \quad (8.9)$$

where $\Delta_i = \mu^* - \mu_i$ is the gap between optimal and variant i .

8.4.4 Thompson Sampling (Bayesian Bandit)

Thompson sampling maintains posterior distributions over variant success rates and samples from them to make allocation decisions.

For Bernoulli rewards (conversion rates):

Algorithm 7 Thompson Sampling for Bernoulli Bandits

Initialize: $\alpha_i = 1, \beta_i = 1$ for all variants i (Beta prior)

for each user $t = 1, 2, \dots, T$ **do**

for each variant $i = 1, \dots, K$ **do**

 Sample $\theta_i \sim \text{Beta}(\alpha_i, \beta_i)$

end for

 Select variant $A_t = \arg \max_i \theta_i$

 Observe reward $r_t \in \{0, 1\}$

 Update: $\alpha_{A_t} \leftarrow \alpha_{A_t} + r_t$

 Update: $\beta_{A_t} \leftarrow \beta_{A_t} + (1 - r_t)$

end for

Intuition: Variants with higher uncertainty (wider posteriors) have higher probability of being sampled, naturally balancing exploration and exploitation.

8.4.5 Contextual Bandits

Contextual bandits extend MAB by incorporating user features to personalize variant selection.

Definition 8.1 (Contextual Bandit). At each time t :

1. Observe context vector $x_t \in \mathbb{R}^d$ (user features)
2. Select arm a_t based on context
3. Observe reward $r_t(a_t)$
4. Update model

Goal: Learn mapping from contexts to optimal arms.

LinUCB Algorithm

LinUCB assumes linear reward models:

$$\mathbb{E}[r_{t,a}|x_t] = x_t^T \theta_a \quad (8.10)$$

where θ_a is a parameter vector for arm a .

Algorithm 8 LinUCB

```

Initialize:  $A_a = I_d$ ,  $b_a = 0$  for all arms  $a$ 
for each user  $t = 1, 2, \dots, T$  do
  Observe context  $x_t$ 
  for each arm  $a$  do
    Compute:  $\hat{\theta}_a = A_a^{-1} b_a$ 
    Compute:  $\text{UCB}_a = x_t^T \hat{\theta}_a + \alpha \sqrt{x_t^T A_a^{-1} x_t}$ 
  end for
  Select  $a_t = \arg \max_a \text{UCB}_a$ 
  Observe reward  $r_t$ 
  Update:  $A_{a_t} \leftarrow A_{a_t} + x_t x_t^T$ 
  Update:  $b_{a_t} \leftarrow b_{a_t} + r_t x_t$ 
end for

```

Key Insight: $\alpha \sqrt{x_t^T A_a^{-1} x_t}$ is the uncertainty in prediction for context x_t under arm a .

Listing 8.2: Contextual Bandit Implementation

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from typing import List, Tuple
4
5 class LinUCB:
6     """
7     Linear Upper Confidence Bound algorithm for contextual bandits.
8     """
9

```

```

10     def __init__(self, n_arms: int, n_features: int, alpha: float =
11         1.0):
12         """
13         Parameters:
14         -----
15         n_arms: int
16         Number of arms (variants)
17         n_features: int
18         Dimension of context vectors
19         alpha: float
20         Exploration parameter
21         """
22         self.n_arms = n_arms
23         self.n_features = n_features
24         self.alpha = alpha
25
26         # Initialize for each arm
27         self.A = [np.identity(n_features) for _ in range(n_arms)]
28         self.b = [np.zeros((n_features, 1)) for _ in range(n_arms)]
29
30         # Track selections
31         self.arm_counts = np.zeros(n_arms)
32         self.total_reward = 0
33
34     def select_arm(self, context: np.ndarray) -> int:
35         """
36         Select arm based on context.
37
38         Parameters:
39         -----
40         context: np.ndarray
41         Context vector (n_features,)
42
43         Returns:
44         -----
45         int: Selected arm index
46         """
47         context = context.reshape(-1, 1)
48         ucb_values = np.zeros(self.n_arms)
49
50         for a in range(self.n_arms):
51             # Compute  $\hat{\theta}_a = A_a^{-1} b_a$ 
52             A_inv = np.linalg.inv(self.A[a])
53             theta_hat = A_inv @ self.b[a]
54
55             # Compute UCB
56             mean_reward = (context.T @ theta_hat).item()
57             uncertainty = self.alpha * np.sqrt(
58                 (context.T @ A_inv @ context).item()
59             )
60             ucb_values[a] = mean_reward + uncertainty
61
62         return np.argmax(ucb_values)
63
64     def update(self, arm: int, context: np.ndarray, reward: float):
65         """
66         Update model after observing reward.

```

```

67  """Parameters:
68  """
69  """arm_: int
70  """Selected arm
71  """context_: np.ndarray
72  """Context vector
73  """reward_: float
74  """Observed reward
75  """
76      context = context.reshape(-1, 1)
77
78      # Update A and b for selected arm
79      self.A[arm] += context @ context.T
80      self.b[arm] += reward * context
81
82      # Track statistics
83      self.arm_counts[arm] += 1
84      self.total_reward += reward
85
86  def simulate_contextual_bandit(
87      n_arms: int,
88      n_features: int,
89      true_theta: List[np.ndarray],
90      n_trials: int,
91      context_distribution: str = 'uniform'
92  ) -> dict:
93      """
94      """Simulate contextual bandit problem.
95
96      """Parameters:
97      """
98      """n_arms_: int
99      """Number of arms
100     """n_features_: int
101     """Dimension of context
102     """true_theta_: list of np.ndarray
103     """True parameter vectors for each arm
104     """n_trials_: int
105     """Number of trials
106     """context_distribution_: str
107     """'uniform' or 'normal'
108
109     """Returns:
110     """
111     """dict_: Simulation results
112     """
113     linucb = LinUCB(n_arms, n_features, alpha=1.0)
114
115     cumulative_reward = np.zeros(n_trials)
116     cumulative_regret = np.zeros(n_trials)
117     arm_selections = np.zeros((n_trials, n_arms))
118
119     for t in range(n_trials):
120         # Generate context
121         if context_distribution == 'uniform':
122             context = np.random.uniform(-1, 1, n_features)
123         else:
124             context = np.random.randn(n_features)

```

```

125
126     # Compute optimal arm and reward
127     expected_rewards = [context @ theta for theta in true_theta]
128     optimal_arm = np.argmax(expected_rewards)
129     optimal_reward = expected_rewards[optimal_arm]
130
131     # Select arm using LinUCB
132     selected_arm = linucb.select_arm(context)
133     arm_selections[t, selected_arm] = 1
134
135     # Observe reward (with noise)
136     true_reward = context @ true_theta[selected_arm]
137     observed_reward = true_reward + np.random.normal(0, 0.1)
138
139     # Update
140     linucb.update(selected_arm, context, observed_reward)
141
142     # Track metrics
143     regret = optimal_reward - true_reward
144     cumulative_reward[t] = linucb.total_reward
145     cumulative_regret[t] = (
146         cumulative_regret[t-1] + regret if t > 0 else regret
147     )
148
149     return {
150         'cumulative_reward': cumulative_reward,
151         'cumulative_regret': cumulative_regret,
152         'arm_selections': arm_selections,
153         'arm_counts': linucb.arm_counts
154     }
155
156 # Example: Content recommendation with user features
157 np.random.seed(42)
158
159 n_arms = 3
160 n_features = 5
161
162 # True parameters (unknown to algorithm)
163 true_theta = [
164     np.array([0.5, 0.3, -0.2, 0.1, 0.4]), # Arm 0
165     np.array([0.3, 0.5, 0.2, -0.1, 0.2]), # Arm 1
166     np.array([0.4, 0.2, 0.3, 0.3, 0.1])   # Arm 2
167 ]
168
169 results = simulate_contextual_bandit(
170     n_arms=n_arms,
171     n_features=n_features,
172     true_theta=true_theta,
173     n_trials=10000
174 )
175
176 print("=== Contextual Bandit Results ===")
177 print(f"\nArm selection counts:")
178 for arm in range(n_arms):
179     print(f"Arm {arm}: {int(results['arm_counts'][arm])}")
180     print(f"({results['arm_counts'][arm]/10000:.1%})")
181
182 print(f"\nFinal cumulative reward: {results['cumulative_reward'][-1]:.1")

```

```

183         f}")
184     print(f"Final cumulative regret: {results['cumulative_regret'][-1]:.1f}")
185
186 # Visualize
187 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
188
189 ax1.plot(results['cumulative_regret'])
190 ax1.set_xlabel('Trial')
191 ax1.set_ylabel('Cumulative Regret')
192 ax1.set_title('LinUCB: Cumulative Regret')
193 ax1.grid(alpha=0.3)
194
195 cumulative_selections = np.cumsum(results['arm_selections'], axis=0)
196 for arm in range(n_arms):
197     ax2.plot(cumulative_selections[:, arm] / np.arange(1, 10001),
198             label=f'Arm {arm}')
199 ax2.set_xlabel('Trial')
200 ax2.set_ylabel('Selection Proportion')
201 ax2.set_title('Arm Selection Over Time')
202 ax2.legend()
203 ax2.grid(alpha=0.3)
204
205 plt.tight_layout()
206 plt.savefig('contextual_bandit.pdf')
207 plt.show()

```

8.4.6 When to Use Bandits vs. Traditional A/B Tests

Criterion	Bandits	Traditional A/B
Optimize during test	✓✓	
Cost of poor experiences	✓✓	
Rapid feedback available	✓✓	
Clean causal inference needed		✓✓
Regulatory requirements		✓✓
Long-term effects matter		✓✓
Many variants (>5)	✓✓	
Permanent implementation decision		✓✓
Continuous optimization	✓✓	

Table 8.1: Bandit vs. traditional A/B test selection guide

8.5 Sequential Testing

Sequential testing allows stopping experiments early when sufficient evidence accumulates, potentially saving time and resources.

8.5.1 The Peeking Problem

Naive peeking: Continuously checking p-values and stopping when $p < 0.05$ inflates Type I error far above 5%.

Example 8.3 (Inflation from Peeking). Simulation with no true effect (H_0 true):

- Fixed sample (n=1000): Type I error = 5.0%
- Check every 100 users, stop if $p < 0.05$: Type I error = 29%
- Check every 50 users: Type I error = 38%

8.5.2 Sequential Probability Ratio Test (SPRT)

Wald's SPRT provides optimal sequential testing for comparing two simple hypotheses.

Definition 8.2 (SPRT). Test $H_0 : \theta = \theta_0$ vs. $H_1 : \theta = \theta_1$ by computing:

$$\Lambda_n = \frac{L(\theta_1; \mathbf{X}_n)}{L(\theta_0; \mathbf{X}_n)} \quad (8.11)$$

Continue sampling until:

- $\Lambda_n \geq B$: Reject H_0 in favor of H_1
- $\Lambda_n \leq A$: Accept H_0
- $A < \Lambda_n < B$: Continue sampling

where boundaries A and B are chosen to control error rates:

$$A \approx \frac{\beta}{1 - \alpha} \quad (8.12)$$

$$B \approx \frac{1 - \beta}{\alpha} \quad (8.13)$$

Theorem 8.3 (Wald, 1947). Among all sequential tests with Type I error $\leq \alpha$ and Type II error $\leq \beta$, SPRT minimizes expected sample size under both H_0 and H_1 .

Listing 8.3: SPRT Implementation for A/B Testing

```

1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4
5 class SPRT:
6     """
7     Sequential Probability Ratio Test for two proportions.
8     """
9
10    def __init__(self, p0: float, p1: float, alpha: float = 0.05,
11                  beta: float = 0.20):
12        """
13        Parameters:
14        -----
15        p0: float

```

```

16 #####Null_hypothesis_conversion_rate
17 #####p1: float
18 #####Alternative_hypothesis_conversion_rate
19 #####alpha: float
20 #####Type_I_error_rate
21 #####beta: float
22 #####Type_II_error_rate
23 #####"""
24     self.p0 = p0
25     self.p1 = p1
26     self.alpha = alpha
27     self.beta = beta
28
29     # Compute boundaries
30     self.A = beta / (1 - alpha)
31     self.B = (1 - beta) / alpha
32
33     # Initialize
34     self.log_lr = 0 # Log likelihood ratio
35     self.n = 0
36     self.successes = 0
37
38     def update(self, conversion: int):
39         """
40         #####Update_with_new_observation.
41
42         #####Parameters:
43         #####-----
44         #####conversion: int
45         #####1 if conversion, 0 otherwise
46
47         #####Returns:
48         #####-----
49         #####str: 'continue', 'reject_h0', or 'accept_h0'
50         #####"""
51         self.n += 1
52         self.successes += conversion
53
54         # Update log likelihood ratio
55         if conversion == 1:
56             self.log_lr += np.log(self.p1 / self.p0)
57         else:
58             self.log_lr += np.log((1 - self.p1) / (1 - self.p0))
59
60         # Check stopping conditions
61         if np.exp(self.log_lr) >= self.B:
62             return 'reject_h0' # Evidence for H1
63         elif np.exp(self.log_lr) <= self.A:
64             return 'accept_h0' # Evidence for H0
65         else:
66             return 'continue'
67
68     def get_statistics(self):
69         """Return current test statistics."""
70         return {
71             'n': self.n,
72             'successes': self.successes,
73             'rate': self.successes / self.n if self.n > 0 else 0,

```

```

74         'log_lr': self.log_lr,
75         'lr': np.exp(self.log_lr)
76     }
77
78 def simulate_sprt(p_true: float, p0: float, p1: float,
79                 max_n: int = 10000, n_simulations: int = 1000):
80     """
81     Simulate SPRT performance.
82
83     Returns:
84     -----
85     dict: Simulation results
86     """
87     stopped_for_h1 = 0
88     stopped_for_h0 = 0
89     sample_sizes = []
90
91     for sim in range(n_simulations):
92         np.random.seed(sim)
93         sprt = SPRT(p0, p1)
94
95         for _ in range(max_n):
96             conversion = np.random.binomial(1, p_true)
97             decision = sprt.update(conversion)
98
99             if decision == 'reject_h0':
100                 stopped_for_h1 += 1
101                 sample_sizes.append(sprt.n)
102                 break
103             elif decision == 'accept_h0':
104                 stopped_for_h0 += 1
105                 sample_sizes.append(sprt.n)
106                 break
107         else:
108             # Reached max_n without decision
109             sample_sizes.append(max_n)
110
111     return {
112         'reject_h0_rate': stopped_for_h1 / n_simulations,
113         'accept_h0_rate': stopped_for_h0 / n_simulations,
114         'no_decision_rate': (n_simulations - stopped_for_h1 -
115                             stopped_for_h0) / n_simulations,
116         'mean_sample_size': np.mean(sample_sizes),
117         'median_sample_size': np.median(sample_sizes),
118         'sample_sizes': sample_sizes
119     }
120
121 # Example: Test if conversion rate differs from 5%
122 print("=== SPRT Simulation ===\n")
123
124 p0 = 0.05 # Null hypothesis
125 p1 = 0.07 # Alternative hypothesis
126
127 # Scenario 1: True rate = p0 (null is true)
128 print("Scenario 1: True rate = 5% (H0 true)")
129 results_h0 = simulate_sprt(p_true=p0, p0=p0, p1=p1, n_simulations=1000)
130 print(f"Reject H0 rate: {results_h0['reject_h0_rate']:.3f} (should be ~0.05)")

```

```

131 print(f"Accept H0 rate: {results_h0['accept_h0_rate']:.3f}")
132 print(f"Mean sample size: {results_h0['mean_sample_size']:.0f}\n")
133
134 # Scenario 2: True rate = p1 (alternative is true)
135 print("Scenario 2: True rate = 7% (H1 true)")
136 results_h1 = simulate_sprt(p_true=p1, p0=p0, p1=p1, n_simulations=1000)
137 print(f"Reject H0 rate: {results_h1['reject_h0_rate']:.3f} (should be ~0.80)")
138 print(f"Accept H0 rate: {results_h1['accept_h0_rate']:.3f}")
139 print(f"Mean sample size: {results_h1['mean_sample_size']:.0f}\n")
140
141 # Compare to fixed sample size
142 fixed_n = 3000 # Hypothetical fixed design
143 print(f"Fixed sample size test: {fixed_n} per variant")
144 print(f"SPRT average savings (H0): {(1 - results_h0['mean_sample_size'] / fixed_n) * 100:.1f}%")
145 print(f"SPRT average savings (H1): {(1 - results_h1['mean_sample_size'] / fixed_n) * 100:.1f}%")
146
147 # Visualize sample size distributions
148 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
149
150 ax1.hist(results_h0['sample_sizes'], bins=50, alpha=0.7, edgecolor='black')
151 ax1.axvline(results_h0['mean_sample_size'], color='red', linestyle='--',
152             linewidth=2, label=f"Mean = {results_h0['mean_sample_size']:.0f}")
153 ax1.axvline(fixed_n, color='blue', linestyle='--',
154             linewidth=2, label=f"Fixed = {fixed_n}")
155 ax1.set_xlabel('Sample Size')
156 ax1.set_ylabel('Frequency')
157 ax1.set_title('SPRT Sample Sizes (H0 True)')
158 ax1.legend()
159 ax1.grid(alpha=0.3)
160
161 ax2.hist(results_h1['sample_sizes'], bins=50, alpha=0.7, edgecolor='black')
162 ax2.axvline(results_h1['mean_sample_size'], color='red', linestyle='--',
163             linewidth=2, label=f"Mean = {results_h1['mean_sample_size']:.0f}")
164 ax2.axvline(fixed_n, color='blue', linestyle='--',
165             linewidth=2, label=f"Fixed = {fixed_n}")
166 ax2.set_xlabel('Sample Size')
167 ax2.set_ylabel('Frequency')
168 ax2.set_title('SPRT Sample Sizes (H1 True)')
169 ax2.legend()
170 ax2.grid(alpha=0.3)
171
172 plt.tight_layout()
173 plt.savefig('sprt_sample_sizes.pdf')
174 plt.show()

```

8.5.3 Group Sequential Designs

Group sequential methods pre-specify interim analyses with adjusted boundaries to control Type I error.

Key Components:

1. **Information fractions:** Times to analyze (e.g., 25%, 50%, 75%, 100%)
2. **Spending function:** How to allocate α across looks
3. **Stopping boundaries:** Thresholds for early stopping

Alpha Spending Functions

Rather than pre-specifying the exact number and timing of analyses, alpha spending functions provide flexibility.

Definition 8.3 (Alpha Spending Function). A function $\alpha^*(t)$ where $t \in [0, 1]$ is the information fraction, satisfying:

- $\alpha^*(0) = 0$
- $\alpha^*(1) = \alpha$
- $\alpha^*(t)$ is non-decreasing

At each analysis with information fraction t_k , spend:

$$\Delta\alpha_k = \alpha^*(t_k) - \alpha^*(t_{k-1}) \quad (8.14)$$

Common Spending Functions:

1. **O'Brien-Fleming:**

$$\alpha^*(t) = 2 - 2\Phi\left(\frac{z_{\alpha/2}}{\sqrt{t}}\right) \quad (8.15)$$

Conservative early, liberal late.

2. **Pocock:**

$$\alpha^*(t) = \alpha \log(1 + (e - 1)t) \quad (8.16)$$

More uniform spending.

3. **Kim-DeMets (Power Family):**

$$\alpha^*(t) = \alpha t^\rho \quad (8.17)$$

where $\rho > 0$ controls shape. $\rho = 1$ is linear, $\rho = 3$ approximates O'Brien-Fleming.

Listing 8.4: Alpha Spending Functions

```

1 import numpy as np
2 from scipy import stats
3 import matplotlib.pyplot as plt
4
5 class AlphaSpending:
6     """
7     Alpha spending functions for group sequential designs.
8     """
9
10    @staticmethod
11    def obrien_fleming(t: float, alpha: float = 0.05) -> float:
12        """O'Brien-Fleming spending function."""
13        if t <= 0:
14            return 0
15        if t >= 1:
16            return alpha
17        z = stats.norm.ppf(1 - alpha/2)
18        return 2 * (1 - stats.norm.cdf(z / np.sqrt(t)))
19
20    @staticmethod
21    def pocock(t: float, alpha: float = 0.05) -> float:
22        """Pocock spending function."""
23        if t <= 0:
24            return 0
25        if t >= 1:
26            return alpha
27        return alpha * np.log(1 + (np.e - 1) * t)
28
29    @staticmethod
30    def kim_demets(t: float, alpha: float = 0.05, rho: float = 3) ->
31    float:
32        """Kim-DeMets power family."""
33        if t <= 0:
34            return 0
35        if t >= 1:
36            return alpha
37        return alpha * (t ** rho)
38
39    @staticmethod
40    def compute_boundary(t: float, alpha_spent: float,
41                        prev_alpha_spent: float = 0,
42                        prev_t: float = 0) -> float:
43        """
44        Compute Z-score boundary for current analysis.
45        Uses Lan-DeMets approximation.
46        """
47        if t <= 0:
48            return np.inf
49
50        delta_alpha = alpha_spent - prev_alpha_spent
51        if delta_alpha <= 0:
52            return np.inf
53
54        # Two-sided boundary
55        z_boundary = stats.norm.ppf(1 - delta_alpha/2)
56

```

```

57     # Adjust for information fraction
58     z_boundary = z_boundary * np.sqrt(1/t)
59
60     return z_boundary
61
62 # Visualize spending functions
63 t_values = np.linspace(0, 1, 1000)
64 alpha = 0.05
65
66 obf_spent = [AlphaSpending.obrien_fleming(t, alpha) for t in t_values]
67 pocock_spent = [AlphaSpending.pocock(t, alpha) for t in t_values]
68 linear_spent = alpha * t_values
69 power3_spent = [AlphaSpending.kim_demets(t, alpha, rho=3) for t in t_
    values]
70
71 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
72
73 # Cumulative alpha spent
74 ax1.plot(t_values, obf_spent, label="O'Brien-Fleming", linewidth=2)
75 ax1.plot(t_values, pocock_spent, label='Pocock', linewidth=2)
76 ax1.plot(t_values, linear_spent, label='Linear', linewidth=2, linestyle
    = '--')
77 ax1.plot(t_values, power3_spent, label='Kim-DeMets ( $\rho=3$ )', linewidth=2)
78 ax1.set_xlabel('Information Fraction (t)', fontsize=12)
79 ax1.set_ylabel('Cumulative  $\alpha$  Spent', fontsize=12)
80 ax1.set_title('Alpha Spending Functions', fontsize=14)
81 ax1.legend(fontsize=11)
82 ax1.grid(alpha=0.3)
83 ax1.set_ylim([0, alpha * 1.1])
84
85 # Incremental alpha spent (derivative)
86 dt = t_values[1] - t_values[0]
87 obf_incr = np.diff(obf_spent) / dt
88 pocock_incr = np.diff(pocock_spent) / dt
89 linear_incr = np.diff(linear_spent) / dt
90 power3_incr = np.diff(power3_spent) / dt
91
92 ax2.plot(t_values[:-1], obf_incr, label="O'Brien-Fleming", linewidth=2)
93 ax2.plot(t_values[:-1], pocock_incr, label='Pocock', linewidth=2)
94 ax2.plot(t_values[:-1], linear_incr, label='Linear', linewidth=2,
    linestyle='--')
95 ax2.plot(t_values[:-1], power3_incr, label='Kim-DeMets ( $\rho=3$ )',
    linewidth=2)
96 ax2.set_xlabel('Information Fraction (t)', fontsize=12)
97 ax2.set_ylabel('Incremental  $\alpha$  Rate', fontsize=12)
98 ax2.set_title('Alpha Spending Rate', fontsize=14)
99 ax2.legend(fontsize=11)
100 ax2.grid(alpha=0.3)
101
102 plt.tight_layout()
103 plt.savefig('alpha_spending_functions.pdf')
104 plt.show()
105
106 # Compute boundaries for specific looks
107 print("=== Alpha Spending Boundaries ===\n")
108 print("For 5 equally-spaced looks:\n")
109
110 info_fracs = [0.2, 0.4, 0.6, 0.8, 1.0]

```

```

111
112 print("O'Brien-Fleming:")
113 prev_alpha = 0
114 prev_t = 0
115 for t in info_fractions:
116     alpha_spent = AlphaSpending.obrien_fleming(t, alpha)
117     delta_alpha = alpha_spent - prev_alpha
118     z_bound = stats.norm.ppf(1 - delta_alpha/2) / np.sqrt(t)
119     print(f"Look {info_fractions.index(t)+1} (t={t:.1f}): "
120           f"Δα={delta_alpha:.4f}, Z={z_bound:.3f}")
121     prev_alpha = alpha_spent
122     prev_t = t
123
124 print("\nPocock:")
125 prev_alpha = 0
126 for t in info_fractions:
127     alpha_spent = AlphaSpending.pocock(t, alpha)
128     delta_alpha = alpha_spent - prev_alpha
129     z_bound = stats.norm.ppf(1 - delta_alpha/2) / np.sqrt(t)
130     print(f"Look {info_fractions.index(t)+1} (t={t:.1f}): "
131           f"Δα={delta_alpha:.4f}, Z={z_bound:.3f}")
132     prev_alpha = alpha_spent

```

8.5.4 Practical Implementation Challenges

1. **Pre-specification:** Must define stopping rules before seeing data
 Violation: "Let's check results and decide if we should stop"
 Solution: Document analysis plan, information fractions, and spending function
2. **Unplanned analyses:** Extra "peeks" between scheduled looks
 Problem: Consumes unaccounted α
 Solution: Use alpha spending functions that accommodate unplanned looks
3. **Multiple metrics:** Each metric requires separate α control
 Solution: Bonferroni correction across metrics, or hierarchical testing
4. **Sample size re-estimation:** Adjusting target sample size mid-experiment
 Problem: Can inflate Type I error
 Solution: Use methods that allow blinded or conditional re-estimation
5. **Delayed outcomes:** Conversions observed days after assignment
 Problem: Information accumulates slowly
 Solution: Account for delay in information fraction calculations

8.6 Quasi-Experimental Designs

When randomization is infeasible, quasi-experimental designs enable causal inference through careful design and statistical adjustment.

8.6.1 When Randomization Fails

Common scenarios:

- Market-level interventions (can't randomize cities)
- Regulatory changes (everyone affected simultaneously)
- Ethical constraints (can't withhold beneficial treatments)
- Technical limitations (single codebase, no variant infrastructure)
- Retrospective analysis (change already implemented)

8.6.2 Difference-in-Differences (DID)

DID exploits variation in treatment timing across groups to estimate causal effects.

Definition 8.4 (DID Estimator). Compare change over time in treated group to change in control group:

$$\hat{\tau}_{\text{DID}} = (\bar{Y}_{\text{treat, post}} - \bar{Y}_{\text{treat, pre}}) - (\bar{Y}_{\text{control, post}} - \bar{Y}_{\text{control, pre}}) \quad (8.18)$$

Key Assumption: Parallel trends—absent treatment, treated and control groups would have followed parallel trends.

Example 8.4 (Geographic Rollout). A food delivery app launches a new feature in Seattle (treatment) but not Portland (control):

	Pre (Week 1)	Post (Week 2)	Change
Seattle (Treated)	100 orders/day	120 orders/day	+20
Portland (Control)	90 orders/day	100 orders/day	+10
DID Estimate			+10

DID estimate: $(120 - 100) - (100 - 90) = 10$ orders/day attributable to feature.

Without DID, naive estimate would be 20 orders/day, confounded by overall market growth.

Regression Form:

$$Y_{it} = \beta_0 + \beta_1 \text{Treated}_i + \beta_2 \text{Post}_t + \beta_3 (\text{Treated}_i \times \text{Post}_t) + \epsilon_{it} \quad (8.19)$$

where β_3 is the DID estimate.

Listing 8.5: Difference-in-Differences Analysis

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import statsmodels.formula.api as smf
5 from scipy import stats
6
7 def generate_did_data(
8     n_treated: int,
```

```

9      n_control: int,
10     n_periods_pre: int,
11     n_periods_post: int,
12     treatment_effect: float,
13     trend: float = 0,
14     noise_std: float = 5
15 ) -> pd.DataFrame:
16     """
17     Simulate DID data.
18
19     Parameters:
20     -----
21     n_treated, n_control: int
22         Number of units in each group
23     n_periods_pre, n_periods_post: int
24         Number of time periods before/after treatment
25     treatment_effect: float
26         True treatment effect
27     trend: float
28         Common time trend
29     noise_std: float
30         Standard deviation of idiosyncratic noise
31
32     Returns:
33     -----
34     pd.DataFrame: Panel data
35     """
36     n_periods = n_periods_pre + n_periods_post
37     data = []
38
39     # Treated units
40     for unit in range(n_treated):
41         baseline = 100 + np.random.normal(0, 10)
42         for t in range(n_periods):
43             post = 1 if t >= n_periods_pre else 0
44             y = (baseline +
45                 trend * t +
46                 treatment_effect * post +
47                 np.random.normal(0, noise_std))
48             data.append({
49                 'unit': f'T{unit}',
50                 'period': t,
51                 'treated': 1,
52                 'post': post,
53                 'y': y
54             })
55
56     # Control units
57     for unit in range(n_control):
58         baseline = 100 + np.random.normal(0, 10)
59         for t in range(n_periods):
60             post = 1 if t >= n_periods_pre else 0
61             y = (baseline +
62                 trend * t +
63                 np.random.normal(0, noise_std))
64             data.append({
65                 'unit': f'C{unit}',
66                 'period': t,

```

```

67         'treated': 0,
68         'post': post,
69         'y': y
70     })
71
72     df = pd.DataFrame(data)
73     df['treated_post'] = df['treated'] * df['post']
74     return df
75
76 def analyze_did(df: pd.DataFrame) -> dict:
77     """
78     Analyze DID data.
79
80     Returns:
81     -----
82     dict: DID results
83     """
84     # Manual DID calculation
85     treated_pre = df[(df['treated']==1) & (df['post']==0)]['y'].mean()
86     treated_post = df[(df['treated']==1) & (df['post']==1)]['y'].mean()
87     control_pre = df[(df['treated']==0) & (df['post']==0)]['y'].mean()
88     control_post = df[(df['treated']==0) & (df['post']==1)]['y'].mean()
89
90     did_manual = (treated_post - treated_pre) - (control_post - control_pre)
91
92     # Regression DID
93     model = smf.ols('y~treated+post+treated_post', data=df).fit()
94
95     # Robust standard errors (clustered by unit)
96     model_robust = smf.ols('y~treated+post+treated_post',
97                             data=df).fit(cov_type='cluster',
98                                             cov_kwds={'groups': df['unit']})
99
100     return {
101         'did_manual': did_manual,
102         'treated_pre': treated_pre,
103         'treated_post': treated_post,
104         'control_pre': control_pre,
105         'control_post': control_post,
106         'model': model,
107         'model_robust': model_robust,
108         'did_estimate': model_robust.params['treated_post'],
109         'did_se': model_robust.bse['treated_post'],
110         'did_pvalue': model_robust.pvalues['treated_post']
111     }
112
113 def plot_did(df: pd.DataFrame, results: dict):
114     """Visualize DID analysis."""
115     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
116
117     # Time series plot
118     treated_means = df[df['treated']==1].groupby('period')['y'].mean()
119     control_means = df[df['treated']==0].groupby('period')['y'].mean()
120
121     ax1.plot(treated_means.index, treated_means.values,
122              'o-', label='Treated', linewidth=2, markersize=8)
123     ax1.plot(control_means.index, control_means.values,

```

```

124         's-', label='Control', linewidth=2, markersize=8)
125
126     # Add vertical line at treatment
127     n_periods_pre = df[df['post']==0]['period'].nunique()
128     ax1.axvline(n_periods_pre - 0.5, color='red', linestyle='--',
129                 label='Treatment_Start', linewidth=2)
130
131     ax1.set_xlabel('Period', fontsize=12)
132     ax1.set_ylabel('Outcome', fontsize=12)
133     ax1.set_title('DID: Time Series', fontsize=14)
134     ax1.legend(fontsize=11)
135     ax1.grid(alpha=0.3)
136
137     # 2x2 table visualization
138     means = [
139         [results['control_pre'], results['control_post']],
140         [results['treated_pre'], results['treated_post']]
141     ]
142
143     ax2.imshow(means, cmap='RdYlGn', aspect='auto')
144     ax2.set_xticks([0, 1])
145     ax2.set_yticks([0, 1])
146     ax2.set_xticklabels(['Pre', 'Post'], fontsize=12)
147     ax2.set_yticklabels(['Control', 'Treated'], fontsize=12)
148
149     # Add values
150     for i in range(2):
151         for j in range(2):
152             ax2.text(j, i, f'{means[i][j]:.1f}',
153                     ha='center', va='center',
154                     fontsize=14, fontweight='bold')
155
156     # Add arrows showing differences
157     control_change = results['control_post'] - results['control_pre']
158     treated_change = results['treated_post'] - results['treated_pre']
159
160     ax2.text(1.3, 0, f' $\Delta$ ={control_change:.1f}', fontsize=11)
161     ax2.text(1.3, 1, f' $\Delta$ ={treated_change:.1f}', fontsize=11, fontweight
162             = 'bold')
163
164     did = results['did_manual']
165     ax2.text(0.5, -0.5, f'DID={did:.1f}', fontsize=14,
166             ha='center', fontweight='bold',
167             bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.5))
168
169     ax2.set_title('DID: 2x2 Table', fontsize=14)
170
171     plt.tight_layout()
172     return fig
173
174 # Example: Feature rollout
175 np.random.seed(42)
176 df = generate_did_data(
177     n_treated=50,
178     n_control=50,
179     n_periods_pre=4,
180     n_periods_post=4,
181     treatment_effect=15, # True effect

```

```

181     trend=2,
182     noise_std=10
183 )
184
185 results = analyze_did(df)
186
187 print("=== Difference-in-Differences Analysis ===\n")
188 print("Means: ")
189 print(f"Control Pre: {results['control_pre']:.2f}")
190 print(f"Control Post: {results['control_post']:.2f}")
191 print(f"Treated Pre: {results['treated_pre']:.2f}")
192 print(f"Treated Post: {results['treated_post']:.2f}\n")
193
194 print("DID Estimates: ")
195 print(f"Manual calculation: {results['did_manual']:.2f}")
196 print(f"Regression estimate: {results['did_estimate']:.2f}")
197 print(f"Standard error (clustered): {results['did_se']:.2f}")
198 print(f"95% CI: [{results['did_estimate'] - 1.96*results['did_se']:.2f}, {"
199         f"{results['did_estimate'] + 1.96*results['did_se']:.2f}"]")
200 print(f"p-value: {results['did_pvalue']:.4f}\n")
201
202 print("Regression Results: ")
203 print(results['model_robust'].summary().tables[1])
204
205 fig = plot_did(df, results)
206 plt.savefig('did_analysis.pdf')
207 plt.show()

```

8.6.3 Regression Discontinuity Design (RDD)

RDD exploits discontinuities in treatment assignment based on a running variable.

Definition 8.5 (Sharp RDD). Treatment assigned based on threshold:

$$D_i = \begin{cases} 1 & \text{if } X_i \geq c \\ 0 & \text{if } X_i < c \end{cases} \quad (8.20)$$

Treatment effect estimated as:

$$\tau = \lim_{x \downarrow c} \mathbb{E}[Y_i | X_i = x] - \lim_{x \uparrow c} \mathbb{E}[Y_i | X_i = x] \quad (8.21)$$

Example 8.5 (Scholarship Program). Students with GPA ≥ 3.5 receive scholarship. Compare outcomes just above and below threshold.

Assumption: Students near threshold (3.49 vs. 3.51 GPA) are similar except for scholarship.

Key Assumption: No manipulation of running variable around cutoff.

8.6.4 Instrumental Variables (IV)

IV exploits exogenous variation to instrument for endogenous treatment.

Definition 8.6 (Instrumental Variable). Variable Z is a valid instrument for treatment D if:

1. **Relevance:** $\text{Cov}(Z, D) \neq 0$
2. **Exclusion:** Z affects outcome Y only through D
3. **Exogeneity:** $\text{Cov}(Z, \epsilon) = 0$ where ϵ is error term

Two-Stage Least Squares (2SLS):

1. First stage: Regress D on Z to get $\hat{D}$
2. Second stage: Regress Y on $\hat{D}$

Example 8.6 (Email Encouragement). Testing effect of mobile app usage on engagement:

- Y : Engagement (outcome)
- D : App usage (endogenous—engaged users self-select into app)
- Z : Randomized email encouraging app download (instrument)

Email affects engagement only through app usage (exclusion), is randomized (exogenous), and increases app adoption (relevance).

2SLS estimates Local Average Treatment Effect (LATE) for compliers—users induced to adopt app by email.

8.7 Platform-Specific Considerations

8.7.1 Network Effects and Interference

When users interact, standard A/B testing assumptions break down. One user's treatment can affect another user's outcome.

Definition 8.7 (SUTVA Violation). The Stable Unit Treatment Value Assumption (SUTVA) requires:

1. No interference: User i 's outcome depends only on their own treatment
2. Consistency: Treatment is well-defined and consistently applied

Network effects violate SUTVA.

Mitigation Strategies:

1. **Cluster Randomization**

Randomize groups of connected users together.

Example: For social network experiment, randomize entire friend groups or geographic regions.

Trade-off: Reduces interference but decreases effective sample size.

2. Ego Network Randomization

Randomize both focal user and their immediate neighbors to same treatment.

3. Temporal Switchbacks

Alternate entire system between control and treatment over time periods.

Must account for temporal correlation and carryover effects.

8.7.2 Two-Sided Marketplaces

Marketplaces with buyers and sellers present unique challenges:

- Supply-demand dynamics
- Cross-side network effects
- Equilibrium effects

Example Challenges:

1. **Spillover:** Increasing driver supply in treatment affects wait times for all users
2. **Equilibrium:** Pricing changes affect both supply and demand
3. **Matching:** Treated buyers compete with control buyers for limited supply

Design Solutions:

1. **Market-level randomization:** Randomize entire markets (cities, regions)
2. **Time-based designs:** Switchback designs alternating treatments over time
3. **Matched market pairs:** Pair similar markets, randomize within pairs

8.7.3 Mobile App Testing

Mobile apps introduce technical constraints:

- **App store review:** Cannot rapidly iterate like web
- **Version fragmentation:** Users on different app versions
- **Update adoption:** Gradual rollout as users update
- **Offline functionality:** Treatment effects while disconnected

Design Approaches:

1. **Server-side flags:** Control features remotely without app updates
2. **Phased rollouts:** Gradual release to percentage of users
3. **A/A tests:** Validate measurement infrastructure before A/B tests

8.7.4 Cross-Platform Experiments

Users interact across web, mobile app, and other platforms.

Challenges:

- Consistent user identification
- Platform-specific effects
- Different usage patterns

Solutions:

- User-level randomization (not session/device)
- Track cross-platform behavior
- Analyze platform-specific and overall effects

8.8 Summary

Advanced experimental designs extend A/B testing capabilities:

- **Factorial designs** efficiently test multiple factors and detect interactions
- **Multi-armed bandits** optimize allocation during experiments, minimizing regret
- **Contextual bandits** personalize variant selection using user features
- **Sequential testing** (SPRT, group sequential) allows early stopping while controlling error rates
- **Alpha spending functions** provide flexibility in interim analysis timing
- **Quasi-experimental designs** (DID, RDD, IV) enable causal inference without randomization
- **Network-aware designs** handle interference when users interact
- **Platform-specific solutions** address marketplaces, mobile apps, and ecosystems

Each design involves trade-offs between efficiency, complexity, and validity. Choose designs based on:

- Business objectives (optimization vs. inference)
- User interaction structure (independent vs. networked)
- Resource constraints (sample size, time, engineering)
- Stakeholder requirements (interpretability, regulatory)
- Technical feasibility (randomization infrastructure)

8.9 Further Reading

- **Box, Hunter & Hunter (2005)**, *Statistics for Experimenters*: Comprehensive treatment of factorial designs
- **Lattimore & Szepesvári (2020)**, *Bandit Algorithms*: Theoretical foundations of multi-armed bandits
- **Jennison & Turnbull (2000)**, *Group Sequential Methods*: Definitive reference on sequential testing
- **Wald (1947)**, *Sequential Analysis*: Original SPRT work
- **Angrist & Pischke (2009)**, *Mostly Harmless Econometrics*: Practical guide to quasi-experimental methods
- **Imbens & Rubin (2015)**, *Causal Inference*: Comprehensive treatment of causal inference
- **Eckles et al. (2016)**, "Design and Analysis of Experiments in Networks": Network interference in experiments

8.10 Exercises

8.10.1 Factorial Designs

1. Design a 2^3 factorial experiment for an e-commerce site testing:

- Product image size (small vs. large)
- Price display format (regular vs. promotional)
- Review placement (top vs. bottom)

Specify sample size per cell for 80% power to detect 1.5 percentage point main effects.

2. Implement a fractional factorial 2^{5-1} design with Resolution V. Which effects can be estimated?
3. Analyze interaction effects: Given cell means for a 2^2 design, compute and interpret the interaction. When would you recommend implementing both factors vs. just one?

8.10.2 Multi-Armed Bandits

1. Compare regret bounds: For 5 arms with gaps $\Delta = [0.05, 0.03, 0.02, 0.01]$, compute theoretical regret bound for UCB1 at $T = 10000$.
2. Implement and compare ϵ -greedy (fixed and decaying), UCB1, and Thompson Sampling on synthetic data. Plot cumulative regret and convergence to optimal arm.
3. Design a contextual bandit for content recommendation: specify features, reward function, and algorithm. How would you handle cold-start problems for new content?

8.10.3 Sequential Testing

1. Implement SPRT for testing $H_0 : p = 0.05$ vs. $H_1 : p = 0.07$ with $\alpha = 0.05, \beta = 0.20$. Simulate under both hypotheses and verify error rates.
2. Design a group sequential test with O'Brien-Fleming boundaries for 4 interim analyses. Compute boundaries and expected sample size under H_0 and H_1 .
3. Compare alpha spending functions: Plot O'Brien-Fleming, Pocock, and Kim-DeMets ($\rho = 2$). Which spends most alpha early? How does this affect stopping probabilities?

8.10.4 Quasi-Experimental Designs

1. A company launches a feature in Texas but not California. Design a difference-in-differences analysis. What are threats to parallel trends assumption? How would you test for pre-treatment parallel trends?
2. Identify potential instruments for estimating effect of customer reviews on sales. Evaluate each against relevance, exclusion, and exogeneity criteria.
3. Design a regression discontinuity study for a loyalty program with eligibility at \$1000 annual spend. Specify bandwidth, functional form, and robustness checks.

8.10.5 Applied Projects

1. **Marketplace Experiment:** Design an experiment for testing dynamic pricing in a two-sided marketplace (e.g., Uber, Airbnb). Address:
 - Randomization unit (user, market, time)
 - Spillover mitigation
 - Equilibrium effects
 - Analysis plan
2. **Mobile App A/B Test:** Design a mobile app experiment with server-side flags. Specify:
 - User identification strategy
 - Version compatibility handling
 - Cross-platform consistency
 - Rollout plan
3. **Social Network Experiment:** Design an experiment testing a viral feature on a social network. Address SUTVA violations through:
 - Cluster randomization (specify clusters)
 - Power analysis accounting for clustering
 - Spillover measurement
 - Alternative designs (switchbacks, etc.)

Chapter 9

Technical Implementation

9.1 Learning Objectives

After completing this chapter, readers will be able to:

- Design and implement feature flag systems for experiment control
- Build scalable random assignment infrastructure
- Architect data collection pipelines for experiment tracking
- Implement real-time monitoring and alerting systems
- Design ETL pipelines for experiment data processing
- Apply data quality validation frameworks
- Use Python, R, and SQL for experiment analysis
- Handle big data experimentation with distributed computing
- Build automated statistical testing frameworks
- Design APIs for experiment management platforms
- Apply object-oriented principles to experiment code
- Ensure reproducibility through version control and documentation

9.2 Introduction: From Theory to Production

Academic treatments of A/B testing focus on statistical theory: hypothesis tests, confidence intervals, power analysis. Production experimentation systems require robust engineering:

- **Scale:** Millions of users, thousands of concurrent experiments
- **Reliability:** 99.9% uptime, no data loss
- **Performance:** Sub-millisecond assignment decisions

- **Flexibility:** Support diverse experiment types and metrics
- **Compliance:** GDPR, CCPA, SOX requirements
- **Observability:** Real-time monitoring and debugging

This chapter bridges the gap between statistical methods and production systems, providing practical implementation patterns for enterprise-scale experimentation platforms.

9.3 Experiment Infrastructure

9.3.1 Feature Flags and Configuration Management

Feature flags decouple deployment from release, enabling safe experimentation in production.

Definition 9.1 (Feature Flag). A conditional statement that controls code path execution based on configuration:

```
if feature_flag('new_checkout_flow', user_id):
    return new_checkout()
else:
    return legacy_checkout()
```

Architecture Requirements:

- **Low latency:** < 10ms evaluation time
- **High availability:** Graceful degradation if flag service down
- **Consistency:** Same user always sees same variant (sticky assignment)
- **Flexibility:** Support percentage rollouts, user targeting, kill switches

Listing 9.1: Feature Flag System Implementation

```
1 import hashlib
2 import json
3 from typing import Dict, Any, Optional, List
4 from enum import Enum
5 from dataclasses import dataclass
6 from datetime import datetime
7 import redis
8
9 class FlagStatus(Enum):
10     """Feature flag status."""
11     ACTIVE = "active"
12     INACTIVE = "inactive"
13     ARCHIVED = "archived"
14
15 @dataclass
16 class FlagConfig:
17     """Configuration for a feature flag."""
18     flag_key: str
```

```

19     status: FlagStatus
20     rollout_percentage: float # 0.0 to 1.0
21     variant_weights: Dict[str, float] # variant -> weight
22     targeting_rules: List[Dict[str, Any]]
23     created_at: datetime
24     updated_at: datetime
25
26     def validate(self):
27         """Validate flag configuration."""
28         # Check weights sum to 1
29         total_weight = sum(self.variant_weights.values())
30         if not (0.99 <= total_weight <= 1.01): # Allow rounding error
31             raise ValueError(f"Variant weights must sum to 1, got {
32                 total_weight}")
33
34         # Check rollout percentage
35         if not (0 <= self.rollout_percentage <= 1):
36             raise ValueError(f"Rollout must be 0-1, got {self.rollout_
37                 percentage}")
38
39 class FeatureFlagSystem:
40     """
41     Production feature flag system with caching and fallback.
42
43     Features:
44     - Consistent hashing for assignment
45     - Local caching with TTL
46     - Graceful degradation
47     - Audit logging
48     """
49
50     def __init__(self, redis_client: Optional[redis.Redis] = None,
51                 default_ttl: int = 60):
52         """
53         Parameters:
54         -----
55         redis_client: redis.Redis
56         Redis client for distributed config storage
57         default_ttl: int
58         Cache TTL in seconds
59         """
60         self.redis_client = redis_client
61         self.default_ttl = default_ttl
62         self.local_cache: Dict[str, FlagConfig] = {}
63         self.cache_timestamps: Dict[str, datetime] = {}
64
65     def evaluate(self, flag_key: str, user_id: str,
66                 context: Optional[Dict[str, Any]] = None) -> str:
67         """
68         Evaluate feature flag for user.
69
70         Parameters:
71         -----
72         flag_key: str
73         Feature flag identifier
74         user_id: str
75         User identifier (for consistent assignment)
76         context: dict

```

```

75 Additional_context_for_targeting(user_attributes, etc.)
76
77 Returns:
78 -----
79 str: Assigned variant (e.g., 'control', 'treatment')
80 """
81     # Get flag configuration
82     config = self._get_config(flag_key)
83
84     if config is None or config.status != FlagStatus.ACTIVE:
85         return 'control' # Default to control if flag not found/
86                             inactive
87
88     # Check if user is in rollout
89     if not self._is_in_rollout(flag_key, user_id,
90                               config.rollout_percentage):
91
92         return 'control'
93
94     # Apply targeting rules
95     if context and not self._matches_targeting(config.targeting_
96                                               rules,
97                                               context):
98
99         return 'control'
100
101     # Assign variant based on consistent hash
102     variant = self._assign_variant(flag_key, user_id,
103                                   config.variant_weights)
104
105     # Log assignment (async in production)
106     self._log_assignment(flag_key, user_id, variant)
107
108     return variant
109
110 def _get_config(self, flag_key: str) -> Optional[FlagConfig]:
111     """Get flag configuration with caching."""
112     # Check local cache
113     if flag_key in self.local_cache:
114         cache_age = (datetime.now() -
115                     self.cache_timestamps[flag_key]).seconds
116         if cache_age < self.default_ttl:
117             return self.local_cache[flag_key]
118
119     # Fetch from Redis
120     if self.redis_client:
121         try:
122             config_json = self.redis_client.get(f"flag:{flag_key}")
123             if config_json:
124                 config_dict = json.loads(config_json)
125                 config = self._deserialize_config(config_dict)
126
127                 # Update cache
128                 self.local_cache[flag_key] = config
129                 self.cache_timestamps[flag_key] = datetime.now()
130
131                 return config
132         except Exception as e:
133             # Log error but continue with cached/default value
134             print(f"Error fetching flag config: {e}")

```

```

131         return None
132
133     def _is_in_rollout(self, flag_key: str, user_id: str,
134                       rollout_percentage: float) -> bool:
135         """
136         Determine if user is in rollout using consistent hashing.
137         Uses hash of (flag_key, user_id) for deterministic assignment.
138         """
139         hash_input = f"{flag_key}:{user_id}"
140         hash_value = int(hashlib.md5(hash_input.encode()).hexdigest(),
141                             16)
142
143         # Map hash to [0, 1]
144         normalized = (hash_value % 10000) / 10000.0
145
146         return normalized < rollout_percentage
147
148     def _assign_variant(self, flag_key: str, user_id: str,
149                        variant_weights: Dict[str, float]) -> str:
150         """
151         Assign variant using weighted consistent hashing.
152         Ensures same user always gets same variant (sticky assignment).
153         """
154         # Create cumulative distribution
155         variants = sorted(variant_weights.keys())
156         cumulative_weights = []
157         cumsum = 0
158         for variant in variants:
159             cumsum += variant_weights[variant]
160             cumulative_weights.append(cumsum)
161
162         # Hash user to [0, 1]
163         hash_input = f"{flag_key}:variant:{user_id}"
164         hash_value = int(hashlib.md5(hash_input.encode()).hexdigest(),
165                             16)
166         random_value = (hash_value % 10000) / 10000.0
167
168         # Find variant
169         for i, threshold in enumerate(cumulative_weights):
170             if random_value < threshold:
171                 return variants[i]
172
173         return variants[-1] # Fallback
174
175     def _matches_targeting(self, targeting_rules: List[Dict[str, Any]],
176                           context: Dict[str, Any]) -> bool:
177         """
178         Check if context matches targeting rules.
179         Targeting rules format:
180         [
181             {
182                 "attribute": "country", "operator": "in", "values": ["US",
183                     "CA"]},
184             {
185                 "attribute": "plan", "operator": "equals", "values": "
premium"}

```

```

185     """
186     """
187     for rule in targeting_rules:
188         attribute = rule['attribute']
189         operator = rule['operator']
190         target_values = rule['values']
191
192         if attribute not in context:
193             return False
194
195         context_value = context[attribute]
196
197         if operator == 'equals':
198             if context_value != target_values:
199                 return False
200         elif operator == 'in':
201             if context_value not in target_values:
202                 return False
203         elif operator == 'not_in':
204             if context_value in target_values:
205                 return False
206         elif operator == 'greater_than':
207             if context_value <= target_values:
208                 return False
209         elif operator == 'less_than':
210             if context_value >= target_values:
211                 return False
212
213     return True
214
215     def _log_assignment(self, flag_key: str, user_id: str, variant: str
216     ):
217         """
218         Log flag assignment for audit trail.
219         In production, send to async queue (Kafka, Kinesis) for
220         processing.
221         """
222         event = {
223             'flag_key': flag_key,
224             'user_id': user_id,
225             'variant': variant,
226             'timestamp': datetime.now().isoformat()
227         }
228
229         # In production: send to message queue
230         # producer.send('flag_assignments', json.dumps(event))
231
232         # For now, just log
233         print(f"Assignment: {event}")
234
235     def _deserialize_config(self, config_dict: Dict) -> FlagConfig:
236         """Deserialize flag config from dict."""
237         return FlagConfig(
238             flag_key=config_dict['flag_key'],
239             status=FlagStatus(config_dict['status']),
240             rollout_percentage=config_dict['rollout_percentage'],
241             variant_weights=config_dict['variant_weights'],

```



```

241         targeting_rules=config_dict.get('targeting_rules', []),
242         created_at=datetime.fromisoformat(config_dict['created_at',
243             ]),
244         updated_at=datetime.fromisoformat(config_dict['updated_at',
245             ])
246     )
247
248     def create_flag(self, config: FlagConfig):
249         """Create or update feature flag."""
250         config.validate()
251
252         if self.redis_client:
253             config_dict = {
254                 'flag_key': config.flag_key,
255                 'status': config.status.value,
256                 'rollout_percentage': config.rollout_percentage,
257                 'variant_weights': config.variant_weights,
258                 'targeting_rules': config.targeting_rules,
259                 'created_at': config.created_at.isoformat(),
260                 'updated_at': config.updated_at.isoformat()
261             }
262
263             self.redis_client.set(
264                 f"flag:{config.flag_key}",
265                 json.dumps(config_dict),
266                 ex=86400 # 24 hour expiry
267             )
268
269 # Example usage
270 flag_system = FeatureFlagSystem()
271
272 # Create experiment flag
273 config = FlagConfig(
274     flag_key='new_checkout_flow',
275     status=FlagStatus.ACTIVE,
276     rollout_percentage=1.0, # 100% of users
277     variant_weights={
278         'control': 0.5,
279         'treatment': 0.5
280     },
281     targeting_rules=[
282         {'attribute': 'country', 'operator': 'in', 'values': ['US', 'CA',
283             '']}
284     ],
285     created_at=datetime.now(),
286     updated_at=datetime.now()
287 )
288
289 flag_system.create_flag(config)
290
291 # Evaluate for user
292 user_context = {'country': 'US', 'plan': 'premium'}
293 variant = flag_system.evaluate('new_checkout_flow', 'user_12345',
294     user_context)
295 print(f"Assigned variant: {variant}")

```

9.3.2 Random Assignment Systems

Random assignment must be:

- **Deterministic:** Same input always produces same output
- **Uniform:** Each variant has exact specified probability
- **Independent:** Assignments across experiments are uncorrelated

Hashing Strategy:

1. Concatenate experiment ID and user ID
2. Hash using cryptographic hash (MD5, SHA-256)
3. Map hash to $[0, 1]$ using modulo
4. Assign variant based on thresholds

Listing 9.2: Production Random Assignment System

```

1 import hashlib
2 from typing import List, Dict
3 from dataclasses import dataclass
4
5 @dataclass
6 class ExperimentConfig:
7     """Experiment configuration."""
8     experiment_id: str
9     variants: List[str]
10    traffic_allocation: float # Fraction of users in experiment
11    variant_weights: Dict[str, float]
12    salt: str = "" # For re-randomization
13
14 class RandomAssignmentSystem:
15     """
16     Production random assignment system.
17
18     Features:
19     - Consistent hashing for deterministic assignment
20     - Support for stratified randomization
21     - Traffic allocation control
22     - Re-randomization via salt
23     """
24
25     def __init__(self):
26         self.experiments: Dict[str, ExperimentConfig] = {}
27
28     def register_experiment(self, config: ExperimentConfig):
29         """Register experiment configuration."""
30         # Validate configuration
31         if sum(config.variant_weights.values()) != 1.0:
32             raise ValueError("Variant weights must sum to 1.0")
33
34         if not (0 <= config.traffic_allocation <= 1):
35             raise ValueError("Traffic allocation must be between 0 and 1")

```

```

36         self.experiments[config.experiment_id] = config
37
38     def assign(self, experiment_id: str, unit_id: str,
39               stratification_key: str = None) -> str:
40         """
41         Assign unit to variant.
42
43         Parameters:
44         experiment_id: str
45         Experiment identifier
46         unit_id: str
47         Unit to assign (user_id, session_id, etc.)
48         stratification_key: str
49         Optional key for stratified randomization
50
51         Returns:
52         str: Assigned variant or 'holdout' if not in traffic
53         """
54         if experiment_id not in self.experiments:
55             raise ValueError(f"Unknown experiment: {experiment_id}")
56
57         config = self.experiments[experiment_id]
58
59         # Check if unit is in traffic allocation
60         if not self._is_in_traffic(experiment_id, unit_id, config):
61             return 'holdout'
62
63         # Assign variant
64         return self._assign_variant(experiment_id, unit_id, config,
65                                    stratification_key)
66
67     def _is_in_traffic(self, experiment_id: str, unit_id: str,
68                       config: ExperimentConfig) -> bool:
69         """Determine if unit is in experiment traffic."""
70         hash_input = f"{experiment_id}:traffic:{unit_id}:{config.salt}"
71         hash_value = self._hash_to_float(hash_input)
72         return hash_value < config.traffic_allocation
73
74     def _assign_variant(self, experiment_id: str, unit_id: str,
75                       config: ExperimentConfig,
76                       stratification_key: str = None) -> str:
77         """Assign variant using consistent hashing."""
78         # Include stratification key if provided
79         strat_suffix = f":{stratification_key}" if stratification_key
80         else ""
81         hash_input = f"{experiment_id}:variant:{unit_id}:{config.salt}{
82             strat_suffix}"
83
84         hash_value = self._hash_to_float(hash_input)
85
86         # Map to variant using cumulative weights
87         cumsum = 0
88         for variant in sorted(config.variants):
89             cumsum += config.variant_weights[variant]
90             if hash_value < cumsum:

```

```

92         return variant
93
94         return config.variants[-1] # Fallback
95
96     @staticmethod
97     def _hash_to_float(input_string: str) -> float:
98         """
99         Hash string to float in [0,1].
100
101         Uses MD5 for speed (not cryptographic security).
102         """
103         hash_bytes = hashlib.md5(input_string.encode()).digest()
104         hash_int = int.from_bytes(hash_bytes[:8], byteorder='big')
105         return (hash_int % 10000) / 10000.0
106
107     def get_assignment_probabilities(self, experiment_id: str,
108                                     unit_id: str) -> Dict[str, float]:
109         """
110         Get probability of each assignment for debugging.
111
112         Useful for validating assignment logic.
113         """
114         if experiment_id not in self.experiments:
115             raise ValueError(f"Unknown experiment: {experiment_id}")
116
117         config = self.experiments[experiment_id]
118
119         # Probability of being in traffic
120         in_traffic_prob = config.traffic_allocation
121
122         # Probability of each variant given in traffic
123         probabilities = {}
124         for variant, weight in config.variant_weights.items():
125             probabilities[variant] = in_traffic_prob * weight
126
127         probabilities['holdout'] = 1 - in_traffic_prob
128
129         return probabilities
130
131 # Example usage
132 assignment_system = RandomAssignmentSystem()
133
134 # Register A/B test
135 config = ExperimentConfig(
136     experiment_id='checkout_redesign_v2',
137     variants=['control', 'treatment'],
138     traffic_allocation=0.8, # 80% in experiment, 20% holdout
139     variant_weights={'control': 0.5, 'treatment': 0.5},
140     salt='v1'
141 )
142
143 assignment_system.register_experiment(config)
144
145 # Assign users
146 for user_id in ['user_1', 'user_2', 'user_3', 'user_4', 'user_5']:
147     variant = assignment_system.assign('checkout_redesign_v2', user_id)
148     probs = assignment_system.get_assignment_probabilities(
149         'checkout_redesign_v2', user_id

```

```

150 )
151 print(f"{user_id}:_{variant}_{Probs:_{probs}}")

```

9.3.3 Data Collection Architecture

Experiment data flows through multiple stages:

1. **Client instrumentation:** Capture events (impressions, clicks, conversions)
2. **Event ingestion:** Stream to data pipeline (Kafka, Kinesis)
3. **Processing:** Enrich, validate, aggregate
4. **Storage:** Write to data warehouse (Snowflake, BigQuery)
5. **Analysis:** Query for statistical tests

Listing 9.3: Event Tracking System

```

1 from dataclasses import dataclass, asdict
2 from typing import Dict, Any, Optional
3 from datetime import datetime
4 import json
5 import uuid
6
7 @dataclass
8 class ExperimentEvent:
9     """
10     Base class for experiment events.
11
12     All events inherit from this and add specific fields.
13     """
14     event_id: str
15     event_type: str
16     timestamp: datetime
17     experiment_id: str
18     variant: str
19     user_id: str
20     session_id: str
21
22     # Context
23     platform: str # web, ios, android
24     version: str # app/site version
25
26     # Additional properties (flexible)
27     properties: Dict[str, Any]
28
29     def to_json(self) -> str:
30         """Serialize to JSON for logging."""
31         data = asdict(self)
32         data['timestamp'] = self.timestamp.isoformat()
33         return json.dumps(data)
34
35     def validate(self):
36         """Validate event structure."""
37         required = ['event_id', 'event_type', 'experiment_id',

```



```

96         platform: str, version: str,
97         conversion_value: float = 0.0,
98         conversion_type: str = "",
99         properties: Dict[str, Any] = None):
100     """Track_conversion_event."""
101     event = ConversionEvent(
102         event_id=str(uuid.uuid4()),
103         event_type='conversion',
104         timestamp=datetime.now(),
105         experiment_id=experiment_id,
106         variant=variant,
107         user_id=user_id,
108         session_id=session_id,
109         platform=platform,
110         version=version,
111         conversion_value=conversion_value,
112         conversion_type=conversion_type,
113         properties=properties or {}
114     )
115
116     event.validate()
117     self._buffer_event(event)
118
119     def _buffer_event(self, event: ExperimentEvent):
120         """Buffer_event_and_flush_if_batch_full."""
121         self.event_buffer.append(event)
122
123         if len(self.event_buffer) >= self.batch_size:
124             self.flush()
125
126     def flush(self):
127         """
128         Flush_buffered_events_to_backend.
129
130         In_production, send to:
131         - Kafka/Kinesis for streaming
132         - Snowplow/Segment for event pipeline
133         - Direct to data warehouse
134         """
135         if not self.event_buffer:
136             return
137
138         # Serialize events
139         events_json = [event.to_json() for event in self.event_buffer]
140
141         # In production: send to message queue
142         # kafka_producer.send('experiment_events', events_json)
143
144         # For now, just log
145         print(f"Flushing {len(events_json)} events")
146
147         # Clear buffer
148         self.event_buffer = []
149
150     # Example usage
151     tracker = EventTracker(batch_size=10)
152
153     # Track exposure

```

```

154 tracker.track_exposure(
155     experiment_id='checkout_v2',
156     variant='treatment',
157     user_id='user_123',
158     session_id='session_456',
159     platform='web',
160     version='2.1.0',
161     properties={'page': 'checkout', 'step': 1}
162 )
163
164 # Track conversion
165 tracker.track_conversion(
166     experiment_id='checkout_v2',
167     variant='treatment',
168     user_id='user_123',
169     session_id='session_456',
170     platform='web',
171     version='2.1.0',
172     conversion_value=49.99,
173     conversion_type='purchase'
174 )
175
176 tracker.flush()

```

9.3.4 Real-Time Monitoring

Production experiments require continuous monitoring for:

- Sample ratio mismatch (SRM)
- Metric anomalies
- Assignment failures
- Performance degradation

Listing 9.4: Real-Time Experiment Monitor

```

1 from dataclasses import dataclass
2 from typing import Dict, List, Optional
3 from datetime import datetime, timedelta
4 import numpy as np
5 from scipy import stats
6
7 @dataclass
8 class MetricSnapshot:
9     """Snapshot of experiment metrics at a point in time."""
10     timestamp: datetime
11     experiment_id: str
12     variant: str
13     sample_size: int
14     metric_values: Dict[str, float]
15
16 @dataclass
17 class Alert:
18     """Alert for experiment issue."""
19     alert_id: str

```



```

20     alert_type: str
21     severity: str # 'info', 'warning', 'critical'
22     experiment_id: str
23     message: str
24     timestamp: datetime
25     metadata: Dict
26
27     class ExperimentMonitor:
28         """
29         Real-time experiment monitoring system.
30
31         Checks for:
32         - Sample ratio mismatch (SRM)
33         - Metric anomalies
34         - Low traffic
35         - High variance
36         """
37
38         def __init__(self, check_interval_seconds: int = 300):
39             self.check_interval = check_interval_seconds
40             self.snapshots: Dict[str, List[MetricSnapshot]] = {}
41             self.alerts: List[Alert] = []
42
43         def record_snapshot(self, snapshot: MetricSnapshot):
44             """Record metrics snapshot."""
45             if snapshot.experiment_id not in self.snapshots:
46                 self.snapshots[snapshot.experiment_id] = []
47
48             self.snapshots[snapshot.experiment_id].append(snapshot)
49
50         def check_experiment(self, experiment_id: str,
51                             expected_variant_weights: Dict[str, float]):
52             """
53             Run all checks for experiment.
54
55             Parameters:
56             -----
57             experiment_id: str
58             Experiment to check
59             expected_variant_weights: dict
60             Expected variant allocation (e.g., {'control': 0.5, 'treatment': 0.5})
61             """
62             if experiment_id not in self.snapshots:
63                 return
64
65             # Get latest snapshots for each variant
66             latest_snapshots = self._get_latest_snapshots(experiment_id)
67
68             # Check SRM
69             self._check_sample_ratio_mismatch(experiment_id, latest_snapshots,
70                                               expected_variant_weights)
71
72             # Check traffic levels
73             self._check_traffic_levels(experiment_id, latest_snapshots)
74
75             # Check metric anomalies

```

```

76         self._check_metric_anomalies(experiment_id, latest_snapshots)
77
78     def _get_latest_snapshots(self, experiment_id: str) -> Dict[str,
MetricSnapshot]:
79         """Get latest snapshot for each variant."""
80         snapshots = self.snapshots[experiment_id]
81
82         # Group by variant and get latest
83         latest = {}
84         for snapshot in snapshots:
85             if snapshot.variant not in latest:
86                 latest[snapshot.variant] = snapshot
87             elif snapshot.timestamp > latest[snapshot.variant].
timestamp:
88                 latest[snapshot.variant] = snapshot
89
90         return latest
91
92     def _check_sample_ratio_mismatch(self, experiment_id: str,
93                                     snapshots: Dict[str, MetricSnapshot
],
94                                     expected_weights: Dict[str, float])
95                                     :
96         """
97         Check for sample ratio mismatch using chi-square test.
98         SRM indicates assignment or tracking issues.
99         """
100         variants = list(snapshots.keys())
101         observed_counts = [snapshots[v].sample_size for v in variants]
102         total = sum(observed_counts)
103
104         expected_counts = [expected_weights[v] * total for v in
variants]
105
106         # Chi-square test
107         chi2, p_value = stats.chisquare(observed_counts, expected_
counts)
108
109         # Alert if p < 0.001 (very strong evidence of mismatch)
110         if p_value < 0.001:
111             alert = Alert(
112                 alert_id=f"srm_{experiment_id}_{datetime.now().
timestamp()}",
113                 alert_type='sample_ratio_mismatch',
114                 severity='critical',
115                 experiment_id=experiment_id,
116                 message=(f"Sample ratio mismatch detected! "
f"p-value={p_value:.6f}, chi2={chi2:.2f}"),
117                 timestamp=datetime.now(),
118                 metadata={
119                     'observed': dict(zip(variants, observed_counts)),
120                     'expected': dict(zip(variants, expected_counts)),
121                     'p_value': p_value,
122                     'chi2': chi2
123                 }
124             )
125         self.alerts.append(alert)
126

```

```

127         print(f"ALERT:_{alert.message}")
128
129     def _check_traffic_levels(self, experiment_id: str,
130                             snapshots: Dict[str, MetricSnapshot],
131                             min_daily_traffic: int = 1000):
132         """Check if experiment has sufficient traffic."""
133         for variant, snapshot in snapshots.items():
134             if snapshot.sample_size < min_daily_traffic:
135                 alert = Alert(
136                     alert_id=f"traffic_{experiment_id}_{variant}_{
137                         datetime.now().timestamp()}",
138                     alert_type='low_traffic',
139                     severity='warning',
140                     experiment_id=experiment_id,
141                     message=(f"Low traffic for variant_{variant}:_{
142                         f'{snapshot.sample_size}_{(expected_{min_
143                             daily_traffic})}"",
144                     timestamp=datetime.now(),
145                     metadata={'variant': variant, 'sample_size':
146                             snapshot.sample_size}
147                 )
148                 self.alerts.append(alert)
149                 print(f"WARNING:_{alert.message}")
150
151     def _check_metric_anomalies(self, experiment_id: str,
152                                 snapshots: Dict[str, MetricSnapshot]):
153         """
154         Check for unusual metric values using historical baselines.
155         Detects data quality issues or genuine effects.
156         """
157         # In production, compare to historical percentiles
158         # For now, just check for extreme values
159
160         for variant, snapshot in snapshots.items():
161             for metric_name, value in snapshot.metric_values.items():
162                 # Example: check conversion rate is plausible
163                 if metric_name == 'conversion_rate':
164                     if value < 0 or value > 1:
165                         alert = Alert(
166                             alert_id=f"anomaly_{experiment_id}_{variant
167                                 }_{metric_name}_{datetime.now().
168                                     timestamp()}",
169                             alert_type='metric_anomaly',
170                             severity='critical',
171                             experiment_id=experiment_id,
172                             message=(f"Invalid_{metric_name}_for_{
173                                 variant}_{value}"",
174                             timestamp=datetime.now(),
175                             metadata={'variant': variant, 'metric':
176                                     metric_name, 'value': value}
177                         )
178                         self.alerts.append(alert)
179                         print(f"ERROR:_{alert.message}")
180
181     def get_recent_alerts(self, hours: int = 24) -> List[Alert]:
182         """Get alerts from last N hours."""
183         cutoff = datetime.now() - timedelta(hours=hours)

```

```

178         return [a for a in self.alerts if a.timestamp > cutoff]
179
180 # Example usage
181 monitor = ExperimentMonitor()
182
183 # Simulate snapshots
184 snapshots = [
185     MetricSnapshot(
186         timestamp=datetime.now(),
187         experiment_id='checkout_v2',
188         variant='control',
189         sample_size=5000,
190         metric_values={'conversion_rate': 0.082, 'revenue_per_user':
191             4.50}
192     ),
193     MetricSnapshot(
194         timestamp=datetime.now(),
195         experiment_id='checkout_v2',
196         variant='treatment',
197         sample_size=5100, # Slight imbalance (should be fine)
198         metric_values={'conversion_rate': 0.095, 'revenue_per_user':
199             5.20}
200     )
201 ]
202
203 for snapshot in snapshots:
204     monitor.record_snapshot(snapshot)
205
206 # Check experiment
207 monitor.check_experiment(
208     'checkout_v2',
209     expected_variant_weights={'control': 0.5, 'treatment': 0.5}
210 )
211
212 # Get recent alerts
213 recent_alerts = monitor.get_recent_alerts(hours=1)
214 print(f"\nRecent alerts: {len(recent_alerts)}")
215 for alert in recent_alerts:
216     print(f"{alert.severity} {alert.alert_type}: {alert.message}")

```

9.4 Data Engineering Considerations

9.4.1 ETL Pipelines for Experiment Data

ETL (Extract, Transform, Load) pipelines process raw event data into analysis-ready tables.

Pipeline Stages:

1. **Extract:** Pull raw events from streaming system (Kafka) or data lake (S3)
2. **Transform:**
 - Join exposures with conversions
 - Aggregate by user/variant

- Compute derived metrics
- Apply attribution windows

3. Load: Write to data warehouse (Snowflake, BigQuery, Redshift)

Listing 9.5: Experiment Data Schema

```

1  -- Exposures table: User-variant assignments
2  CREATE TABLE experiment_exposures (
3      exposure_id VARCHAR(36) PRIMARY KEY,
4      experiment_id VARCHAR(100) NOT NULL,
5      variant VARCHAR(50) NOT NULL,
6      user_id VARCHAR(100) NOT NULL,
7      session_id VARCHAR(100),
8      exposure_timestamp TIMESTAMP NOT NULL,
9      platform VARCHAR(20),
10     version VARCHAR(20),
11     properties JSON,
12
13     -- Metadata
14     ingested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
15
16     -- Indexes
17     INDEX idx_experiment_user (experiment_id, user_id),
18     INDEX idx_experiment_timestamp (experiment_id, exposure_timestamp)
19 );
20
21 -- Conversions table: User actions
22 CREATE TABLE experiment_conversions (
23     conversion_id VARCHAR(36) PRIMARY KEY,
24     experiment_id VARCHAR(100) NOT NULL,
25     variant VARCHAR(50), -- May be NULL if user not in experiment
26     user_id VARCHAR(100) NOT NULL,
27     session_id VARCHAR(100),
28     conversion_timestamp TIMESTAMP NOT NULL,
29     conversion_type VARCHAR(50) NOT NULL,
30     conversion_value DECIMAL(10, 2),
31     platform VARCHAR(20),
32     properties JSON,
33
34     -- Metadata
35     ingested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
36
37     -- Indexes
38     INDEX idx_experiment_user (experiment_id, user_id),
39     INDEX idx_experiment_timestamp (experiment_id, conversion_timestamp)
40 );
41
42 -- Aggregated user-level metrics (materialized for fast querying)
43 CREATE TABLE experiment_user_metrics (
44     experiment_id VARCHAR(100) NOT NULL,
45     variant VARCHAR(50) NOT NULL,
46     user_id VARCHAR(100) NOT NULL,
47
48     -- Exposure info
49     first_exposure_timestamp TIMESTAMP NOT NULL,

```

```

50     total_exposures INT DEFAULT 1,
51
52     -- Conversion metrics (7-day attribution window)
53     converted BOOLEAN DEFAULT FALSE,
54     conversion_count INT DEFAULT 0,
55     total_conversion_value DECIMAL(10, 2) DEFAULT 0,
56
57     -- Engagement metrics
58     sessions INT DEFAULT 0,
59     page_views INT DEFAULT 0,
60     time_spent_seconds INT DEFAULT 0,
61
62     -- Computed at
63     computed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
64
65     PRIMARY KEY (experiment_id, variant, user_id),
66     INDEX idx_experiment_variant (experiment_id, variant)
67 );
68
69 -- Experiment summary statistics (pre-aggregated for dashboards)
70 CREATE TABLE experiment_summary (
71     experiment_id VARCHAR(100) NOT NULL,
72     variant VARCHAR(50) NOT NULL,
73     date DATE NOT NULL,
74
75     -- Sample sizes
76     unique_users INT,
77     total_exposures INT,
78
79     -- Conversion metrics
80     conversions INT,
81     conversion_rate DECIMAL(10, 6),
82     revenue_per_user DECIMAL(10, 2),
83     average_order_value DECIMAL(10, 2),
84
85     -- Engagement metrics
86     avg_sessions_per_user DECIMAL(10, 2),
87     avg_page_views_per_user DECIMAL(10, 2),
88
89     -- Statistical metrics
90     conversion_rate_se DECIMAL(10, 6), -- Standard error
91
92     -- Metadata
93     computed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
94
95     PRIMARY KEY (experiment_id, variant, date)
96 );

```

Listing 9.6: ETL Transformation Queries

```

1  -- Step 1: Compute user-level metrics
2  -- Join exposures with conversions, applying 7-day attribution window
3
4  INSERT INTO experiment_user_metrics
5      (experiment_id, variant, user_id, first_exposure_timestamp,
6       total_exposures, converted, conversion_count, total_conversion_
7       value)
8  WITH exposures AS (

```

```

8      SELECT
9          experiment_id,
10         variant,
11         user_id,
12         MIN(exposure_timestamp) AS first_exposure,
13         COUNT(*) AS exposure_count
14     FROM experiment_exposures
15     WHERE exposure_timestamp >= CURRENT_DATE - INTERVAL '30_days'
16     GROUP BY experiment_id, variant, user_id
17 ),
18 conversions AS (
19     SELECT
20         c.experiment_id,
21         c.user_id,
22         COUNT(*) AS conversion_count,
23         SUM(c.conversion_value) AS total_value
24     FROM experiment_conversions c
25     INNER JOIN exposures e
26         ON c.experiment_id = e.experiment_id
27         AND c.user_id = e.user_id
28     WHERE c.conversion_timestamp BETWEEN e.first_exposure
29           AND e.first_exposure + INTERVAL '7_
30           days'
31     GROUP BY c.experiment_id, c.user_id
32 )
33 SELECT
34     e.experiment_id,
35     e.variant,
36     e.user_id,
37     e.first_exposure AS first_exposure_timestamp,
38     e.exposure_count AS total_exposures,
39     CASE WHEN c.conversion_count > 0 THEN TRUE ELSE FALSE END AS
40     converted,
41     COALESCE(c.conversion_count, 0) AS conversion_count,
42     COALESCE(c.total_value, 0) AS total_conversion_value
43 FROM exposures e
44 LEFT JOIN conversions c
45     ON e.experiment_id = c.experiment_id
46     AND e.user_id = c.user_id
47 ON DUPLICATE KEY UPDATE
48     total_exposures = VALUES(total_exposures),
49     converted = VALUES(converted),
50     conversion_count = VALUES(conversion_count),
51     total_conversion_value = VALUES(total_conversion_value),
52     computed_at = CURRENT_TIMESTAMP;
53
54 -- Step 2: Compute daily summary statistics
55
56 INSERT INTO experiment_summary
57     (experiment_id, variant, date, unique_users, total_exposures,
58      conversions, conversion_rate, revenue_per_user, conversion_rate_se
59      )
60 SELECT
61     experiment_id,
62     variant,
63     DATE(first_exposure_timestamp) AS date,
64     COUNT(DISTINCT user_id) AS unique_users,
65     SUM(total_exposures) AS total_exposures,

```

```

63     SUM(CASE WHEN converted THEN 1 ELSE 0 END) AS conversions,
64     AVG(CASE WHEN converted THEN 1.0 ELSE 0.0 END) AS conversion_rate,
65     AVG(total_conversion_value) AS revenue_per_user,
66     SQRT(AVG(CASE WHEN converted THEN 1.0 ELSE 0.0 END) *
67         (1 - AVG(CASE WHEN converted THEN 1.0 ELSE 0.0 END)) /
68         COUNT(DISTINCT user_id)) AS conversion_rate_se
69 FROM experiment_user_metrics
70 WHERE computed_at >= CURRENT_DATE - INTERVAL '1_day'
71 GROUP BY experiment_id, variant, DATE(first_exposure_timestamp)
72 ON DUPLICATE KEY UPDATE
73     unique_users = VALUES(unique_users),
74     total_exposures = VALUES(total_exposures),
75     conversions = VALUES(conversions),
76     conversion_rate = VALUES(conversion_rate),
77     revenue_per_user = VALUES(revenue_per_user),
78     conversion_rate_se = VALUES(conversion_rate_se),
79     computed_at = CURRENT_TIMESTAMP;
80
81 -- Step 3: Sample Ratio Mismatch (SRM) Check Query
82
83 WITH variant_counts AS (
84     SELECT
85         experiment_id,
86         variant,
87         COUNT(DISTINCT user_id) AS user_count
88     FROM experiment_user_metrics
89     WHERE first_exposure_timestamp >= CURRENT_DATE - INTERVAL '7_days'
90     GROUP BY experiment_id, variant
91 ),
92 totals AS (
93     SELECT
94         experiment_id,
95         SUM(user_count) AS total_users
96     FROM variant_counts
97     GROUP BY experiment_id
98 ),
99 expected_counts AS (
100     SELECT
101         vc.experiment_id,
102         vc.variant,
103         vc.user_count AS observed,
104         t.total_users * 0.5 AS expected -- Assuming 50/50 split
105     FROM variant_counts vc
106     JOIN totals t ON vc.experiment_id = t.experiment_id
107 )
108 SELECT
109     experiment_id,
110     variant,
111     observed,
112     expected,
113     (observed - expected) AS difference,
114     ((observed - expected) * (observed - expected)) / expected AS chi2_
        component
115 FROM expected_counts
116 ORDER BY experiment_id, variant;

```

Listing 9.7: Python ETL Pipeline with Pandas


```

1  import pandas as pd
2  import numpy as np
3  from datetime import datetime, timedelta
4  from typing import Dict, List
5  from sqlalchemy import create_engine
6
7  class ExperimentETL:
8      """
9      ETL pipeline for experiment data processing.
10
11      Processes raw events into analysis-ready metrics.
12      """
13
14      def __init__(self, db_connection_string: str):
15          self.engine = create_engine(db_connection_string)
16
17      def extract_exposures(self, experiment_id: str,
18                          start_date: datetime,
19                          end_date: datetime) -> pd.DataFrame:
20          """Extract exposure events from database."""
21          query = """
22          SELECT
23          exposure_id,
24          experiment_id,
25          variant,
26          user_id,
27          exposure_timestamp,
28          platform,
29          version
30          FROM experiment_exposures
31          WHERE experiment_id = %s
32          AND exposure_timestamp BETWEEN %s AND %s
33          ORDER BY user_id, exposure_timestamp
34          """
35
36          return pd.read_sql(query, self.engine,
37                          params=[experiment_id, start_date, end_date])
38
39      def extract_conversions(self, experiment_id: str,
40                          start_date: datetime,
41                          end_date: datetime) -> pd.DataFrame:
42          """Extract conversion events from database."""
43          query = """
44          SELECT
45          conversion_id,
46          user_id,
47          conversion_timestamp,
48          conversion_type,
49          conversion_value
50          FROM experiment_conversions
51          WHERE experiment_id = %s
52          AND conversion_timestamp BETWEEN %s AND %s
53          ORDER BY user_id, conversion_timestamp
54          """
55
56          return pd.read_sql(query, self.engine,
57                          params=[experiment_id, start_date, end_date])
58

```

```

59     def transform_user_metrics(self, exposures: pd.DataFrame,
60                               conversions: pd.DataFrame,
61                               attribution_window_days: int = 7) -> pd.
                                   DataFrame:
62
63         """
64         Transform raw events into user-level metrics.
65         Applies attribution window for conversions.
66         """
67         # Get first exposure per user
68         first_exposure = exposures.groupby('user_id').agg({
69             'exposure_timestamp': 'min',
70             'variant': 'first',
71             'experiment_id': 'first'
72         }).reset_index()
73         first_exposure.columns = ['user_id', 'first_exposure',
74                                 'variant', 'experiment_id']
75
76         # Count total exposures
77         exposure_counts = exposures.groupby('user_id').size().reset_
78             index()
79         exposure_counts.columns = ['user_id', 'total_exposures']
80
81         # Merge
82         user_metrics = first_exposure.merge(exposure_counts, on='user_
83             id')
84
85         # Apply attribution window to conversions
86         conversions_attributed = []
87         for _, user_row in user_metrics.iterrows():
88             user_id = user_row['user_id']
89             first_exp = user_row['first_exposure']
90
91             # Get conversions within attribution window
92             user_conversions = conversions[
93                 (conversions['user_id'] == user_id) &
94                 (conversions['conversion_timestamp'] >= first_exp) &
95                 (conversions['conversion_timestamp'] <=
96                     first_exp + timedelta(days=attribution_window_days))
97             ]
98
99             if len(user_conversions) > 0:
100                 conversions_attributed.append({
101                     'user_id': user_id,
102                     'converted': True,
103                     'conversion_count': len(user_conversions),
104                     'total_conversion_value': user_conversions['
105                         conversion_value'].sum()
106                 })
107             else:
108                 conversions_attributed.append({
109                     'user_id': user_id,
110                     'converted': False,
111                     'conversion_count': 0,
112                     'total_conversion_value': 0
113                 })
114
115         conv_df = pd.DataFrame(conversions_attributed)

```

```

113
114     # Merge with user metrics
115     user_metrics = user_metrics.merge(conv_df, on='user_id')
116
117     return user_metrics
118
119     def compute_summary_statistics(self, user_metrics: pd.DataFrame) ->
120         pd.DataFrame:
121         """
122         Compute summary statistics by variant.
123         Returns aggregated metrics for statistical testing.
124         """
125         summary = user_metrics.groupby('variant').agg({
126             'user_id': 'count',
127             'total_exposures': 'sum',
128             'converted': 'sum',
129             'conversion_count': 'sum',
130             'total_conversion_value': 'sum'
131         }).reset_index()
132
133         summary.columns = ['variant', 'users', 'exposures', '
134             conversions',
135             'total_conversions', 'total_revenue']
136
137         # Compute rates
138         summary['conversion_rate'] = summary['conversions'] / summary['
139             users']
140         summary['revenue_per_user'] = summary['total_revenue'] /
141             summary['users']
142
143         # Standard errors
144         summary['conversion_rate_se'] = np.sqrt(
145             summary['conversion_rate'] * (1 - summary['conversion_rate']
146             ]) /
147             summary['users']
148
149         summary['revenue_per_user_se'] = user_metrics.groupby('variant'
150             ).agg({
151                 'total_conversion_value': lambda x: x.std() / np.sqrt(len(x
152                 ))
153             }).values
154
155         return summary
156
157     def run_pipeline(self, experiment_id: str,
158                     start_date: datetime,
159                     end_date: datetime) -> Dict[str, pd.DataFrame]:
160         """
161         Run complete ETL pipeline.
162         Returns:
163         dict: Contains 'user_metrics' and 'summary' DataFrames
164         """
165         print(f"Extracting data for {experiment_id}...")

```

```

164         # Extract
165         exposures = self.extract_exposures(experiment_id, start_date,
166                                           end_date)
167         conversions = self.extract_conversions(experiment_id, start_
168                                               date, end_date)
169
170         print(f"Exposures: {len(exposures)}")
171         print(f"Conversions: {len(conversions)}")
172
173         # Transform
174         print("Transforming user-level metrics...")
175         user_metrics = self.transform_user_metrics(exposures,
176                                                    conversions)
177
178         print("Computing summary statistics...")
179         summary = self.compute_summary_statistics(user_metrics)
180
181         return {
182             'user_metrics': user_metrics,
183             'summary': summary
184         }
185
186     # Example usage (with mock connection string)
187     # etl = ExperimentETL('postgresql://user:pass@localhost/experiments')
188
189     # results = etl.run_pipeline(
190     #     experiment_id='checkout_v2',
191     #     start_date=datetime(2024, 1, 1),
192     #     end_date=datetime(2024, 1, 31)
193     # )
194
195     # print("\nSummary Statistics:")
196     # print(results['summary'])

```

9.4.2 Data Quality Checks

Data quality issues can invalidate experiment results. Implement automated checks:

Listing 9.8: Data Quality Validation Framework

```

1  from dataclasses import dataclass
2  from typing import List, Callable
3  import pandas as pd
4
5  @dataclass
6  class QualityCheckResult:
7      """Result of a data quality check."""
8      check_name: str
9      passed: bool
10     message: str
11     severity: str # 'error', 'warning', 'info'
12     details: dict = None
13
14     class DataQualityChecker:
15         """
16         Data quality validation framework for experiments.
17         """

```

```

18 """Checks for common data issues that invalidate experiments.
19 """
20
21 def __init__(self):
22     self.checks: List[Callable] = []
23     self.results: List[QualityCheckResult] = []
24
25 def add_check(self, check_func: Callable):
26     """Add quality check function."""
27     self.checks.append(check_func)
28
29 def run_checks(self, data: pd.DataFrame) -> List[QualityCheckResult]:
30     """Run all registered checks."""
31     self.results = []
32
33     for check in self.checks:
34         result = check(data)
35         self.results.append(result)
36
37         if result.severity == 'error':
38             print(f"ERROR: {result.check_name}: {result.message}")
39         elif result.severity == 'warning':
40             print(f"WARNING: {result.check_name}: {result.message}")
41
42     return self.results
43
44 def get_failures(self) -> List[QualityCheckResult]:
45     """Get failed checks."""
46     return [r for r in self.results if not r.passed]
47
48 # Define quality check functions
49
50 def check_no_nulls(data: pd.DataFrame) -> QualityCheckResult:
51     """Check for NULL values in critical columns."""
52     critical_cols = ['user_id', 'variant', 'experiment_id']
53
54     null_counts = {}
55     for col in critical_cols:
56         if col in data.columns:
57             null_count = data[col].isnull().sum()
58             if null_count > 0:
59                 null_counts[col] = null_count
60
61     if null_counts:
62         return QualityCheckResult(
63             check_name='no_nulls',
64             passed=False,
65             message=f"Found NULL values in critical columns: {null_counts}",
66             severity='error',
67             details=null_counts
68         )
69     else:
70         return QualityCheckResult(
71             check_name='no_nulls',
72             passed=True,

```

```

73         message="No NULL values in critical columns",
74         severity='info'
75     )
76
77 def check_valid_variants(data: pd.DataFrame,
78                          expected_variants: List[str]) ->
79     QualityCheckResult:
80     """Check all variants are expected values."""
81     if 'variant' not in data.columns:
82         return QualityCheckResult(
83             check_name='valid_variants',
84             passed=False,
85             message="Missing 'variant' column",
86             severity='error'
87         )
88
89     actual_variants = set(data['variant'].unique())
90     expected_set = set(expected_variants)
91
92     unexpected = actual_variants - expected_set
93     missing = expected_set - actual_variants
94
95     if unexpected or missing:
96         return QualityCheckResult(
97             check_name='valid_variants',
98             passed=False,
99             message=f"Variant mismatch. Unexpected: {unexpected}, Missing: {missing}",
100             severity='error',
101             details={'unexpected': list(unexpected), 'missing': list(missing)}
102         )
103     else:
104         return QualityCheckResult(
105             check_name='valid_variants',
106             passed=True,
107             message=f"All variants valid: {expected_variants}",
108             severity='info'
109         )
110
111 def check_timestamp_order(data: pd.DataFrame) -> QualityCheckResult:
112     """Check exposures come before conversions for each user."""
113     if 'first_exposure' not in data.columns or 'conversion_timestamp'
114     not in data.columns:
115         return QualityCheckResult(
116             check_name='timestamp_order',
117             passed=True,
118             message="Columns not present for timestamp check",
119             severity='info'
120         )
121
122     # For converted users, check exposure before conversion
123     converted = data[data['converted'] == True]
124
125     if len(converted) == 0:
126         return QualityCheckResult(
127             check_name='timestamp_order',
128             passed=True,

```

```

127         message="No conversions to check",
128         severity='info'
129     )
130
131     invalid = converted[converted['conversion_timestamp'] < converted['
132         first_exposure']]
133
134     if len(invalid) > 0:
135         return QualityCheckResult(
136             check_name='timestamp_order',
137             passed=False,
138             message=f"Found {len(invalid)} conversions before exposure",
139             severity='error',
140             details={'count': len(invalid)}
141         )
142     else:
143         return QualityCheckResult(
144             check_name='timestamp_order',
145             passed=True,
146             message="All conversions after exposures",
147             severity='info'
148         )
149
150 def check_sample_size_balance(data: pd.DataFrame,
151                               tolerance: float = 0.1) ->
152     QualityCheckResult:
153
154     """
155     Check variant sample sizes are balanced.
156     Allows 10% tolerance by default.
157     """
158     variant_counts = data['variant'].value_counts()
159
160     if len(variant_counts) < 2:
161         return QualityCheckResult(
162             check_name='sample_size_balance',
163             passed=False,
164             message=f"Only {len(variant_counts)} variant(s) found",
165             severity='error'
166         )
167
168     expected_size = len(data) / len(variant_counts)
169
170     imbalanced = []
171     for variant, count in variant_counts.items():
172         ratio = count / expected_size
173         if ratio < (1 - tolerance) or ratio > (1 + tolerance):
174             imbalanced.append({
175                 'variant': variant,
176                 'count': count,
177                 'expected': expected_size,
178                 'ratio': ratio
179             })
180
181     if imbalanced:
182         return QualityCheckResult(
183             check_name='sample_size_balance',

```

```

182         passed=False,
183         message=f"Sample_size_imbalance_detected_in_{len(imbalanced
184             )}_variant(s)",
185         severity='warning',
186         details={'imbalanced': imbalanced}
187     )
188 else:
189     return QualityCheckResult(
190         check_name='sample_size_balance',
191         passed=True,
192         message="Sample_sizes_balanced",
193         severity='info'
194     )
195
196 # Example usage
197 checker = DataQualityChecker()
198
199 # Add checks
200 checker.add_check(check_no_nulls)
201 checker.add_check(lambda df: check_valid_variants(df, ['control', '
202     treatment']))
203 checker.add_check(check_timestamp_order)
204 checker.add_check(check_sample_size_balance)
205
206 # Create sample data
207 sample_data = pd.DataFrame({
208     'user_id': [f'user_{i}' for i in range(1000)],
209     'variant': ['control'] * 450 + ['treatment'] * 550, # Imbalanced
210     'experiment_id': ['exp_1'] * 1000,
211     'converted': [False] * 900 + [True] * 100
212 })
213
214 # Run checks
215 results = checker.run_checks(sample_data)
216
217 # Report
218 print(f"\n=== Quality Check Summary ===")
219 print(f"Total checks: {len(results)}")
220 print(f"Passed: {sum(1 for r in results if r.passed)}")
221 print(f"Failed: {sum(1 for r in results if not r.passed)}")
222
223 failures = checker.get_failures()
224 if failures:
225     print(f"\nFailures:")
226     for failure in failures:
227         print(f"{failure.check_name}: {failure.message}")

```

9.4.3 Storage and Retrieval Systems

Storage Strategy:

9.4.4 Privacy and Compliance

Experiment systems must comply with privacy regulations:

- **GDPR:** Right to erasure, data minimization, consent

Data Type	Storage	Retention	Access Pattern
Raw events	S3/Data Lake	2 years	Batch (rare)
Aggregated metrics	Snowflake/BigQuery	Indefinite	Interactive queries
Real-time stats	Redis/DynamoDB	7 days	High-frequency reads
Experiment metadata	PostgreSQL	Indefinite	OLTP

Table 9.1: Data storage strategy by type

- **CCPA:** Do not sell, right to delete
- **HIPAA:** (Healthcare) De-identification, access controls

Listing 9.9: Privacy-Preserving Data Handling

```

1 import hashlib
2 from typing import Optional
3
4 class PrivacyCompliantIDSystem:
5     """
6     Privacy-preserving identifier system.
7
8     Uses hashing to pseudonymize user IDs while maintaining
9     consistency for experiment assignment.
10    """
11
12    def __init__(self, secret_salt: str):
13        """
14        Parameters:
15        secret_salt: str
16        Secret key for hashing (rotate periodically)
17        """
18        self.salt = secret_salt
19
20    def pseudonymize(self, user_id: str) -> str:
21        """
22        Convert user ID to pseudonymous identifier.
23
24        One-way hash prevents reverse lookup while maintaining
25        consistency.
26        """
27        hash_input = f"{self.salt}:{user_id}"
28        return hashlib.sha256(hash_input.encode()).hexdigest()[:16]
29
30    def delete_user_data(self, user_id: str, db_connection):
31        """
32        Delete all experiment data for user (GDPR right to erasure).
33
34        Parameters:
35        user_id: str
36        User ID to delete
37        db_connection: database connection
38        Connection to experiment database
39        """
40

```

```

41     pseudo_id = self.pseudonymize(user_id)
42
43     cursor = db_connection.cursor()
44
45     # Delete from all tables
46     tables = [
47         'experiment_exposures',
48         'experiment_conversions',
49         'experiment_user_metrics'
50     ]
51
52     for table in tables:
53         query = f"DELETE FROM {table} WHERE user_id={pseudo_id}"
54         cursor.execute(query, [pseudo_id])
55
56     db_connection.commit()
57     cursor.close()
58
59     print(f"Deleted all data for user {user_id} (pseudo: {pseudo_id})")
60
61 # Example
62 privacy_system = PrivacyCompliantIDSystem(secret_salt="your-secret-key-2024")
63
64 # Pseudonymize IDs before storing
65 original_id = "user_12345_email@example.com"
66 pseudo_id = privacy_system.pseudonymize(original_id)
67 print(f"Original: {original_id}")
68 print(f"Pseudonymized: {pseudo_id}")
69
70 # On deletion request
71 # privacy_system.delete_user_data(original_id, db_connection)

```

9.5 Statistical Computing

9.5.1 Python Libraries Ecosystem

Production experiment analysis uses specialized libraries:

Listing 9.10: Comprehensive Statistical Analysis with Python

```

1  import numpy as np
2  import pandas as pd
3  from scipy import stats
4  import statsmodels.api as sm
5  from statsmodels.stats.power import zt_ind_solve_power
6  from statsmodels.stats.proportion import proportions_ztest
7  import matplotlib.pyplot as plt
8  import seaborn as sns
9
10 class ExperimentAnalyzer:
11     """
12     Comprehensive experiment analysis using Python scientific stack.
13
14     Libraries:

```

```

15  """-numpy: Numerical computation
16  """-pandas: Data manipulation
17  """-scipy: Statistical tests
18  """-statsmodels: Advanced statistical models
19  """-matplotlib/seaborn: Visualization
20  """
21
22  def __init__(self, data: pd.DataFrame):
23      """
24      Parameters:
25      -----
26      data: pd.DataFrame
27      Experiment data with columns: user_id, variant, converted,
28      revenue, etc.
29      """
30      self.data = data
31      self.results = {}
32
33  def analyze_conversion_rate(self, alpha: float = 0.05) -> dict:
34      """
35      Analyze binary conversion metric.
36      Returns z-test results and confidence intervals.
37      """
38      # Aggregate by variant
39      agg = self.data.groupby('variant').agg({
40          'user_id': 'count',
41          'converted': 'sum'
42      })
43
44      control = agg.loc['control']
45      treatment = agg.loc['treatment']
46
47      # Conversion rates
48      p_control = control['converted'] / control['user_id']
49      p_treatment = treatment['converted'] / treatment['user_id']
50
51      # Two-proportion z-test
52      counts = np.array([treatment['converted'], control['converted']
53                          ])
54      nobs = np.array([treatment['user_id'], control['user_id']])
55
56      z_stat, p_value = proportions_ztest(counts, nobs, alternative='
57      two-sided')
58
59      # Confidence interval for difference
60      p_diff = p_treatment - p_control
61      se_diff = np.sqrt(
62          p_control * (1 - p_control) / control['user_id'] +
63          p_treatment * (1 - p_treatment) / treatment['user_id']
64      )
65
66      ci_lower = p_diff - stats.norm.ppf(1 - alpha/2) * se_diff
67      ci_upper = p_diff + stats.norm.ppf(1 - alpha/2) * se_diff
68
69      results = {
70          'metric': 'conversion_rate',
71          'control_rate': p_control,

```

```

70         'treatment_rate': p_treatment,
71         'absolute_diff': p_diff,
72         'relative_lift': (p_treatment / p_control - 1) * 100,
73         'z_statistic': z_stat,
74         'p_value': p_value,
75         'ci_lower': ci_lower,
76         'ci_upper': ci_upper,
77         'significant': p_value < alpha
78     }
79
80     self.results['conversion_rate'] = results
81     return results
82
83     def analyze_revenue(self, alpha: float = 0.05) -> dict:
84         """
85         Analyze continuous revenue metric using t-test.
86
87         Handles non-normal distributions with Welch's t-test.
88         """
89         control_revenue = self.data[self.data['variant'] == 'control']['revenue']
90         treatment_revenue = self.data[self.data['variant'] == 'treatment']['revenue']
91
92         # Welch's t-test (unequal variances)
93         t_stat, p_value = stats.ttest_ind(treatment_revenue, control_revenue,
94                                           equal_var=False)
95
96         # Descriptive statistics
97         control_mean = control_revenue.mean()
98         treatment_mean = treatment_revenue.mean()
99         diff = treatment_mean - control_mean
100
101         # Confidence interval
102         se_diff = np.sqrt(
103             control_revenue.var() / len(control_revenue) +
104             treatment_revenue.var() / len(treatment_revenue)
105         )
106
107         df = len(control_revenue) + len(treatment_revenue) - 2
108         t_crit = stats.t.ppf(1 - alpha/2, df)
109
110         ci_lower = diff - t_crit * se_diff
111         ci_upper = diff + t_crit * se_diff
112
113         results = {
114             'metric': 'revenue_per_user',
115             'control_mean': control_mean,
116             'treatment_mean': treatment_mean,
117             'absolute_diff': diff,
118             'relative_lift': (treatment_mean / control_mean - 1) * 100,
119             't_statistic': t_stat,
120             'p_value': p_value,
121             'ci_lower': ci_lower,
122             'ci_upper': ci_upper,
123             'significant': p_value < alpha
124         }

```

```

125         self.results['revenue'] = results
126     return results
127
128
129     def bootstrap_confidence_interval(self, metric: str,
130                                     n_bootstrap: int = 10000,
131                                     alpha: float = 0.05) -> dict:
132         """
133         Compute bootstrap confidence interval for any metric.
134
135         Non-parametric method for complex metrics or small samples.
136         """
137         control_data = self.data[self.data['variant'] == 'control']
138         treatment_data = self.data[self.data['variant'] == 'treatment']
139
140         bootstrap_diffs = []
141
142         for _ in range(n_bootstrap):
143             # Resample with replacement
144             control_sample = control_data.sample(n=len(control_data),
145                                                  replace=True)
146             treatment_sample = treatment_data.sample(n=len(treatment_
147                                                         data),
148                                                      replace=True)
149
150             # Compute metric
151             if metric == 'conversion_rate':
152                 control_metric = control_sample['converted'].mean()
153                 treatment_metric = treatment_sample['converted'].mean()
154             elif metric == 'revenue':
155                 control_metric = control_sample['revenue'].mean()
156                 treatment_metric = treatment_sample['revenue'].mean()
157             else:
158                 raise ValueError(f"Unknown metric: {metric}")
159
160             bootstrap_diffs.append(treatment_metric - control_metric)
161
162         # Percentile confidence interval
163         ci_lower = np.percentile(bootstrap_diffs, alpha/2 * 100)
164         ci_upper = np.percentile(bootstrap_diffs, (1 - alpha/2) * 100)
165
166         return {
167             'metric': metric,
168             'bootstrap_mean_diff': np.mean(bootstrap_diffs),
169             'ci_lower': ci_lower,
170             'ci_upper': ci_upper,
171             'bootstrap_samples': n_bootstrap
172         }
173
174     def check_sample_ratio_mismatch(self, expected_ratio: float = 0.5,
175                                     alpha: float = 0.001) -> dict:
176         """
177         Check for Sample Ratio Mismatch using chi-square test.
178
179         SRM indicates assignment or tracking issues.
180         """
181         variant_counts = self.data['variant'].value_counts()

```

```

182     total = variant_counts.sum()
183     observed = variant_counts.values
184     expected = np.array([expected_ratio, 1 - expected_ratio]) *
        total
185
186     chi2_stat, p_value = stats.chisquare(observed, expected)
187
188     return {
189         'chi2_statistic': chi2_stat,
190         'p_value': p_value,
191         'observed_counts': dict(variant_counts),
192         'expected_counts': dict(zip(variant_counts.index, expected)
        ),
193         'srm_detected': p_value < alpha
194     }
195
196     def power_analysis(self, mde: float, alpha: float = 0.05,
197         power: float = 0.8, baseline_rate: float = None)
198         -> dict:
199         """
200         Compute required sample size for desired power.
201
202         Parameters:
203         -----
204         mde: float
205             Minimum detectable effect (absolute for conversion rate)
206         alpha: float
207             Type I error rate
208         power: float
209             Desired power (1 - Type II error)
210         baseline_rate: float
211             Baseline conversion rate
212
213         Returns:
214         -----
215         dict: Required sample size and other parameters
216         """
217         if baseline_rate is None:
218             # Estimate from control group
219             baseline_rate = self.data[self.data['variant'] == 'control'
220             ][['converted']].mean()
221
222         # Convert to effect size (Cohen's h for proportions)
223         p1 = baseline_rate
224         p2 = baseline_rate + mde
225
226         effect_size = 2 * (np.arcsin(np.sqrt(p2)) - np.arcsin(np.sqrt(
227             p1)))
228
229         # Compute required sample size per group
230         n_required = zt_ind_solve_power(
231             effect_size=effect_size,
232             alpha=alpha,
233             power=power,
234             alternative='two-sided'
235         )
236
237     return {

```

```

235         'baseline_rate': baseline_rate,
236         'mde': mde,
237         'effect_size': effect_size,
238         'alpha': alpha,
239         'power': power,
240         'n_required_per_variant': int(np.ceil(n_required)),
241         'n_total_required': int(np.ceil(n_required * 2))
242     }
243
244     def generate_report(self) -> str:
245         """Generate comprehensive analysis report."""
246         report = []
247         report.append("=== Experiment Analysis Report ===\n")
248
249         # Sample size
250         total_users = len(self.data)
251         variant_counts = self.data['variant'].value_counts()
252         report.append(f"Total users: {total_users}")
253         for variant, count in variant_counts.items():
254             report.append(f"{variant}: {count} ({count/total_users:.1%})")
255
256         # Conversion rate analysis
257         if 'conversion_rate' in self.results:
258             report.append("\n--- Conversion Rate ---")
259             r = self.results['conversion_rate']
260             report.append(f"Control: {r['control_rate']:.4f}")
261             report.append(f"Treatment: {r['treatment_rate']:.4f}")
262             report.append(f"Absolute difference: {r['absolute_diff']:.4f}")
263             report.append(f"Relative lift: {r['relative_lift']:.2f}%")
264             report.append(f"95% CI: [{r['ci_lower']:.4f}, {r['ci_upper']:.4f}]")
265             report.append(f"p-value: {r['p_value']:.4f}")
266             report.append(f"Significant: {r['significant']}")
267
268         # Revenue analysis
269         if 'revenue' in self.results:
270             report.append("\n--- Revenue Per User ---")
271             r = self.results['revenue']
272             report.append(f"Control: ${r['control_mean']:.2f}")
273             report.append(f"Treatment: ${r['treatment_mean']:.2f}")
274             report.append(f"Absolute difference: ${r['absolute_diff']:.2f}")
275             report.append(f"Relative lift: {r['relative_lift']:.2f}%")
276             report.append(f"95% CI: [{r['ci_lower']:.2f}, {r['ci_upper']:.2f}]")
277             report.append(f"p-value: {r['p_value']:.4f}")
278             report.append(f"Significant: {r['significant']}")
279
280         return "\n".join(report)
281
282     # Example usage
283     np.random.seed(42)
284
285     # Generate sample data
286     n_users = 10000
287     data = pd.DataFrame({

```

```

288     'user_id': [f'user_{i}' for i in range(n_users)],
289     'variant': ['control'] * 5000 + ['treatment'] * 5000,
290     'converted': (
291         [np.random.binomial(1, 0.10) for _ in range(5000)] + # Control
292         [np.random.binomial(1, 0.12) for _ in range(5000)] #
293         Treatment: 12%
294     ),
295     'revenue': (
296         np.random.gamma(2, 5, 5000).tolist() + # Control
297         np.random.gamma(2, 5.5, 5000).tolist() # Treatment (slightly
298         higher)
299     )
300 })
301
302 # Analyze
303 analyzer = ExperimentAnalyzer(data)
304
305 # Run analyses
306 conv_results = analyzer.analyze_conversion_rate()
307 rev_results = analyzer.analyze_revenue()
308 srm_results = analyzer.check_sample_ratio_mismatch()
309 power_results = analyzer.power_analysis(mde=0.01)
310
311 # Print report
312 print(analyzer.generate_report())
313
314 print("\n--- Sample Ratio Mismatch ---")
315 print(f"SRM detected: {srm_results['srm_detected']}")
316 print(f"p-value: {srm_results['p_value']:.6f}")
317
318 print("\n--- Power Analysis ---")
319 print(f"For MDE of {power_results['mde']:.3f}:")
320 print(f"Required sample size: {power_results['n_required_per_variant']}
321       per variant")

```

9.5.2 R Packages for Experimentation

R provides comprehensive packages for statistical analysis of experiments:

Listing 9.11: R Analysis of Experiments

```

1 # Load required packages
2 library(tidyverse)
3 library(broom)
4 library(pwr)
5
6 # Experiment Analysis in R
7 analyze_experiment <- function(data, metric = "conversion_rate") {
8   # Aggregate by variant
9   summary_stats <- data %>%
10     group_by(variant) %>%
11     summarise(
12       n = n(),
13       mean = mean(!!sym(metric)),
14       sd = sd(!!sym(metric)),
15       se = sd / sqrt(n)

```



```

16   )
17
18   # Two-sample test
19   control_data <- data %>% filter(variant == "control") %>% pull(!sym(
      metric))
20   treatment_data <- data %>% filter(variant == "treatment") %>% pull(!
      sym(metric))
21
22   test_result <- t.test(treatment_data, control_data)
23
24   # Effect size
25   cohens_d <- (mean(treatment_data) - mean(control_data)) /
26               sqrt((var(control_data) + var(treatment_data)) / 2)
27
28   # Return results
29   list(
30     summary = summary_stats,
31     test = tidy(test_result),
32     effect_size = cohens_d
33   )
34 }
35
36 # Power analysis
37 power_analysis_r <- function(effect_size, alpha = 0.05, power = 0.80) {
38   pwr.t.test(d = effect_size, sig.level = alpha, power = power,
39             type = "two.sample", alternative = "two.sided")
40 }
41
42 # Example
43 # results <- analyze_experiment(experiment_data, "conversion_rate")
44 # print(results$summary)
45 # print(results$test)

```

9.5.3 Big Data Considerations with Spark

For experiments with billions of events, use distributed computing:

Listing 9.12: Experiment Analysis with PySpark

```

1  from pyspark.sql import SparkSession
2  from pyspark.sql import functions as F
3  from pyspark.sql.window import Window
4
5  class SparkExperimentAnalyzer:
6      """
7      Distributed experiment analysis using Apache Spark.
8
9      Handles petabyte-scale experiment data.
10     """
11
12     def __init__(self, spark: SparkSession):
13         self.spark = spark
14
15     def load_exposures(self, path: str):
16         """Load exposure events from S3/HDFS."""
17         return self.spark.read.parquet(path)
18

```

```

19     def load_conversions(self, path: str):
20         """Load conversion events from S3/HDFS."""
21         return self.spark.read.parquet(path)
22
23     def compute_user_metrics(self, exposures_df, conversions_df,
24                             attribution_window_days: int = 7):
25         """
26         Compute user-level metrics using distributed joins.
27
28         Processes billions of rows efficiently.
29         """
30         # Get first exposure per user
31         window = Window.partitionBy("experiment_id", "user_id").orderBy(
32             ("exposure_timestamp")
33
34         first_exposures = exposures_df.withColumn(
35             "row_num", F.row_number().over(window)
36         ).filter(
37             F.col("row_num") == 1
38         ).select(
39             "experiment_id",
40             "user_id",
41             "variant",
42             F.col("exposure_timestamp").alias("first_exposure")
43         )
44
45         # Count total exposures
46         exposure_counts = exposures_df.groupBy(
47             "experiment_id", "user_id"
48         ).agg(
49             F.count("*").alias("total_exposures")
50         )
51
52         # Join exposures with counts
53         user_exposures = first_exposures.join(
54             exposure_counts,
55             on=["experiment_id", "user_id"]
56         )
57
58         # Join with conversions (within attribution window)
59         conversions_attributed = conversions_df.join(
60             first_exposures,
61             on=["experiment_id", "user_id"]
62         ).filter(
63             (F.col("conversion_timestamp") >= F.col("first_exposure"))
64             &
65             (F.col("conversion_timestamp") <= F.col("first_exposure") +
66              F.expr(f"INTERVAL_{attribution_window_days} DAYS"))
67         ).groupBy(
68             "experiment_id", "user_id"
69         ).agg(
70             F.count("*").alias("conversion_count"),
71             F.sum("conversion_value").alias("total_conversion_value")
72         )
73
74         # Final user metrics
75         user_metrics = user_exposures.join(
76             conversions_attributed,

```

```

74         on=["experiment_id", "user_id"],
75         how="left"
76     ).fillna({
77         "conversion_count": 0,
78         "total_conversion_value": 0
79     }).withColumn(
80         "converted",
81         F.when(F.col("conversion_count") > 0, True).otherwise(False)
82     )
83
84     return user_metrics
85
86     def compute_summary_statistics(self, user_metrics_df):
87         """
88         Aggregate to variant-level statistics.
89
90         Returns DataFrame ready for analysis.
91         """
92         summary = user_metrics_df.groupBy(
93             "experiment_id", "variant"
94         ).agg(
95             F.count("user_id").alias("users"),
96             F.sum("total_exposures").alias("exposures"),
97             F.sum(F.when(F.col("converted"), 1).otherwise(0)).alias("
                conversions"),
98             F.avg("total_conversion_value").alias("revenue_per_user"),
99             F.stddev("total_conversion_value").alias("revenue_stddev")
100         ).withColumn(
101             "conversion_rate",
102             F.col("conversions") / F.col("users")
103         ).withColumn(
104             "conversion_rate_se",
105             F.sqrt(F.col("conversion_rate") * (1 - F.col("conversion_
                rate"))) / F.col("users"))
106         )
107
108         return summary
109
110     def run_pipeline(self, exposures_path: str, conversions_path: str,
111                     output_path: str):
112         """
113         Run complete analysis pipeline.
114
115         Processes data in distributed fashion across cluster.
116         """
117         print("Loading exposures...")
118         exposures = self.load_exposures(exposures_path)
119
120         print("Loading conversions...")
121         conversions = self.load_conversions(conversions_path)
122
123         print("Computing user metrics...")
124         user_metrics = self.compute_user_metrics(exposures, conversions)
125
126         # Cache for reuse
127         user_metrics.cache()

```

```

128         print("Computing summary statistics...")
129         summary = self.compute_summary_statistics(user_metrics)
130
131         # Write results
132         summary.write.parquet(output_path, mode="overwrite")
133
134         print(f"Results written to {output_path}")
135
136         return summary
137
138
139 # Example usage
140 # spark = SparkSession.builder.appName("ExperimentAnalysis").
141 #     getOrCreate()
142 # analyzer = SparkExperimentAnalyzer(spark)
143 # summary = analyzer.run_pipeline(
144 #     exposures_path="s3://experiments/exposures/",
145 #     conversions_path="s3://experiments/conversions/",
146 #     output_path="s3://experiments/summary/"
147 # )
148
149 # summary.show()

```

9.6 Automation and Scaling

9.6.1 Automated Statistical Tests

Production systems automate routine analyses:

Listing 9.13: Automated Experiment Analysis Pipeline

```

1 from typing import Dict, List
2 from datetime import datetime, timedelta
3 import schedule
4 import logging
5
6 class AutomatedExperimentPipeline:
7     """
8     Automated pipeline for experiment analysis.
9
10    Runs daily:
11    - Fetch data
12    - Quality checks
13    - Statistical tests
14    - Alert on significant results
15    - Update dashboards
16    """
17
18    def __init__(self, db_connection, alert_system):
19        self.db = db_connection
20        self.alert_system = alert_system
21        self.logger = logging.getLogger(__name__)
22
23    def analyze_active_experiments(self):
24        """Analyze all active experiments."""

```

```

25     self.logger.info("Starting automated analysis...")
26
27     # Get list of active experiments
28     active_experiments = self._get_active_experiments()
29
30     self.logger.info(f"Found {len(active_experiments)} active
        experiments")
31
32     for exp_id in active_experiments:
33         try:
34             self._analyze_experiment(exp_id)
35         except Exception as e:
36             self.logger.error(f"Error analyzing {exp_id}: {e}")
37             self.alert_system.send_alert(
38                 severity="error",
39                 message=f"Failed to analyze experiment {exp_id}: {e}"
40             )
41
42     self.logger.info("Automated analysis complete")
43
44     def _get_active_experiments(self) -> List[str]:
45         """Query database for active experiments."""
46         query = """
47         SELECT DISTINCT experiment_id
48         FROM experiment_exposures
49         WHERE exposure_timestamp >= CURRENT_DATE - INTERVAL '7 days'
50         AND experiment_id IN (
51             SELECT experiment_id FROM experiments WHERE status = '
            active'
52         )
53         """
54         result = self.db.execute(query)
55         return [row[0] for row in result]
56
57     def _analyze_experiment(self, experiment_id: str):
58         """Analyze single experiment."""
59         self.logger.info(f"Analyzing {experiment_id}...")
60
61         # Fetch data
62         data = self._fetch_experiment_data(experiment_id)
63
64         # Quality checks
65         quality_issues = self._run_quality_checks(data)
66         if quality_issues:
67             self.alert_system.send_alert(
68                 severity="warning",
69                 message=f"Quality issues in {experiment_id}: {quality_
                    issues}"
70             )
71
72         # Statistical tests
73         results = self._run_statistical_tests(data)
74
75         # Check for significance
76         if results['conversion_rate']['significant']:
77             self.alert_system.send_alert(
78                 severity="info",

```

```

79         message=(f"Significant_result_in_{experiment_id}!"
80                  f"Lift:{results['conversion_rate']['relative_
81                  lift']:.2f}%\n"
82                  f"(p={results['conversion_rate']['p_value']:.4f
83                  })")
84
85     # Update dashboard
86     self._update_dashboard(experiment_id, results)
87
88     def _fetch_experiment_data(self, experiment_id: str) -> pd.
89     DataFrame:
90         """Fetch_experiment_data_from_database."""
91         # Implementation similar to ETL pipeline
92         pass
93
94     def _run_quality_checks(self, data: pd.DataFrame) -> List[str]:
95         """Run_data_quality_checks."""
96         issues = []
97
98         # Check SRM
99         variant_counts = data['variant'].value_counts()
100         if len(variant_counts) == 2:
101             chi2, p_value = stats.chisquare(variant_counts.values)
102             if p_value < 0.001:
103                 issues.append(f"Sample_ratio_mismatch_(p={p_value:.6f})
104                             ")
105
106         # Check minimum sample size
107         min_users = variant_counts.min()
108         if min_users < 100:
109             issues.append(f"Low_sample_size_{min_users}_users")
110
111         return issues
112
113     def _run_statistical_tests(self, data: pd.DataFrame) -> Dict:
114         """Run_statistical_tests."""
115         analyzer = ExperimentAnalyzer(data)
116         return {
117             'conversion_rate': analyzer.analyze_conversion_rate(),
118             'revenue': analyzer.analyze_revenue()
119         }
120
121     def _update_dashboard(self, experiment_id: str, results: Dict):
122         """Update_dashboard_with_latest_results."""
123         # Send to dashboard service (Grafana, Tableau, etc.)
124         pass
125
126     def schedule_daily_run(self, run_time: str = "02:00"):
127         """Schedule_pipeline_to_run_daily."""
128         schedule.every().day.at(run_time).do(self.analyze_active_
129         experiments)
130
131         self.logger.info(f"Scheduled_daily_analysis_at_{run_time}")
132
133         while True:
134             schedule.run_pending()
135             time.sleep(60)

```

```

132
133 # Example usage
134 # pipeline = AutomatedExperimentPipeline(db_connection, alert_system)
135 # pipeline.schedule_daily_run(run_time="02:00")

```

9.6.2 Alert Systems

Listing 9.14: Multi-Channel Alert System

```

1 from enum import Enum
2 from dataclasses import dataclass
3 from typing import Optional
4 import requests
5 import json
6
7 class AlertSeverity(Enum):
8     INFO = "info"
9     WARNING = "warning"
10    ERROR = "error"
11    CRITICAL = "critical"
12
13 @dataclass
14 class Alert:
15     severity: AlertSeverity
16     title: str
17     message: str
18     experiment_id: Optional[str] = None
19     metadata: Optional[dict] = None
20
21 class AlertSystem:
22     """
23     Multi-channel alert system for experiment monitoring.
24
25     Supports:
26     - Slack
27     - PagerDuty
28     - Email
29     - SMS (Twilio)
30     """
31
32     def __init__(self, config: dict):
33         self.slack_webhook = config.get('slack_webhook')
34         self.pagerduty_key = config.get('pagerduty_key')
35         self.email_config = config.get('email')
36
37     def send_alert(self, alert: Alert):
38         """Send alert to appropriate channels based on severity."""
39         if alert.severity == AlertSeverity.INFO:
40             # Send to Slack only
41             self._send_slack(alert)
42
43         elif alert.severity == AlertSeverity.WARNING:
44             # Send to Slack
45             self._send_slack(alert)
46
47         elif alert.severity in [AlertSeverity.ERROR, AlertSeverity.

```

```

48         # Send to all channels
49         self._send_slack(alert)
50         self._send_pagerduty(alert)
51         self._send_email(alert)
52
53     def _send_slack(self, alert: Alert):
54         """Send alert to Slack."""
55         if not self.slack_webhook:
56             return
57
58         # Color based on severity
59         colors = {
60             AlertSeverity.INFO: "#36a64f",
61             AlertSeverity.WARNING: "#ff9900",
62             AlertSeverity.ERROR: "#ff0000",
63             AlertSeverity.CRITICAL: "#990000"
64         }
65
66         payload = {
67             "attachments": [{
68                 "color": colors[alert.severity],
69                 "title": f"[{alert.severity.value.upper()}] {alert.title}",
70                 "text": alert.message,
71                 "fields": [
72                     {
73                         "title": "Experiment ID",
74                         "value": alert.experiment_id or "N/A",
75                         "short": True
76                     }
77                 ],
78                 "ts": int(datetime.now().timestamp())
79             }]
80         }
81
82         if alert.metadata:
83             payload["attachments"][0]["fields"].append({
84                 "title": "Details",
85                 "value": json.dumps(alert.metadata, indent=2),
86                 "short": False
87             })
88
89         try:
90             response = requests.post(
91                 self.slack_webhook,
92                 json=payload,
93                 timeout=5
94             )
95             response.raise_for_status()
96         except Exception as e:
97             print(f"Failed to send Slack alert: {e}")
98
99     def _send_pagerduty(self, alert: Alert):
100         """Send alert to PagerDuty."""
101         if not self.pagerduty_key:
102             return
103
104         payload = {

```



```

105         "routing_key": self.pagerduty_key,
106         "event_action": "trigger",
107         "payload": {
108             "summary": alert.title,
109             "severity": alert.severity.value,
110             "source": "experiment_monitoring",
111             "custom_details": {
112                 "message": alert.message,
113                 "experiment_id": alert.experiment_id,
114                 "metadata": alert.metadata
115             }
116         }
117     }
118
119     try:
120         response = requests.post(
121             "https://events.pagerduty.com/v2/enqueue",
122             json=payload,
123             timeout=5
124         )
125         response.raise_for_status()
126     except Exception as e:
127         print(f"Failed to send PagerDuty alert: {e}")
128
129     def _send_email(self, alert: Alert):
130         """Send email alert."""
131         # Implementation using SMTP or email service API
132         pass
133
134     # Example usage
135     alert_system = AlertSystem({
136         'slack_webhook': 'https://hooks.slack.com/services/YOUR/WEBHOOK/URL',
137         'pagerduty_key': 'your-pagerduty-key'
138     })
139
140     # Send critical alert
141     alert = Alert(
142         severity=AlertSeverity.CRITICAL,
143         title="Sample Ratio Mismatch Detected",
144         message="Experiment checkout_v2 has severe sample ratio mismatch (p < 0.0001)",
145         experiment_id="checkout_v2",
146         metadata={"p_value": 0.00001, "chi2": 45.2}
147     )
148
149     alert_system.send_alert(alert)

```

9.6.3 Experiment Management API

Listing 9.15: REST API for Experiment Management using FastAPI

```

1 from fastapi import FastAPI, HTTPException, Depends
2 from pydantic import BaseModel
3 from typing import List, Optional
4 from datetime import datetime
5 from enum import Enum

```

```

6
7 app = FastAPI(title="Experiment_Management_API")
8
9 class ExperimentStatus(str, Enum):
10     draft = "draft"
11     active = "active"
12     paused = "paused"
13     completed = "completed"
14
15 class ExperimentCreate(BaseModel):
16     name: str
17     description: str
18     variants: List[str]
19     traffic_allocation: float
20     metrics: List[str]
21
22 class ExperimentResponse(BaseModel):
23     id: str
24     name: str
25     description: str
26     status: ExperimentStatus
27     variants: List[str]
28     traffic_allocation: float
29     metrics: List[str]
30     created_at: datetime
31     updated_at: datetime
32
33 class AssignmentRequest(BaseModel):
34     experiment_id: str
35     user_id: str
36     context: Optional[dict] = None
37
38 class AssignmentResponse(BaseModel):
39     experiment_id: str
40     user_id: str
41     variant: str
42     timestamp: datetime
43
44 @app.post("/experiments/", response_model=ExperimentResponse)
45 async def create_experiment(experiment: ExperimentCreate):
46     """Create new experiment."""
47     # Validate
48     if experiment.traffic_allocation < 0 or experiment.traffic_
49         allocation > 1:
50         raise HTTPException(status_code=400, detail="Invalid traffic_
51             allocation")
52
53     # Create in database
54     # exp_id = db.create_experiment(experiment)
55
56     # Return response
57     return ExperimentResponse(
58         id="exp_123", # Generated ID
59         name=experiment.name,
60         description=experiment.description,
61         status=ExperimentStatus.draft,
62         variants=experiment.variants,
63         traffic_allocation=experiment.traffic_allocation,

```

```

62         metrics=experiment.metrics,
63         created_at=datetime.now(),
64         updated_at=datetime.now()
65     )
66
67 @app.get("/experiments/{experiment_id}", response_model=
68     ExperimentResponse)
69 async def get_experiment(experiment_id: str):
70     """Get experiment details."""
71     # Fetch from database
72     # experiment = db.get_experiment(experiment_id)
73
74     # Mock response
75     return ExperimentResponse(
76         id=experiment_id,
77         name="Checkout Redesign",
78         description="Testing new checkout flow",
79         status=ExperimentStatus.active,
80         variants=["control", "treatment"],
81         traffic_allocation=0.8,
82         metrics=["conversion_rate", "revenue"],
83         created_at=datetime(2024, 1, 1),
84         updated_at=datetime(2024, 1, 15)
85     )
86
87 @app.post("/assign/", response_model=AssignmentResponse)
88 async def assign_variant(request: AssignmentRequest):
89     """Assign user to variant."""
90     # Use assignment system
91     # variant = assignment_system.assign(request.experiment_id, request
92     #     .user_id)
93
94     return AssignmentResponse(
95         experiment_id=request.experiment_id,
96         user_id=request.user_id,
97         variant="treatment", # Mock
98         timestamp=datetime.now()
99     )
100
101 @app.get("/experiments/{experiment_id}/results")
102 async def get_results(experiment_id: str):
103     """Get experiment results."""
104     # Run analysis
105     # results = analyzer.analyze_experiment(experiment_id)
106
107     return {
108         "experiment_id": experiment_id,
109         "status": "active",
110         "results": {
111             "conversion_rate": {
112                 "control": 0.102,
113                 "treatment": 0.118,
114                 "lift": 15.7,
115                 "p_value": 0.002,
116                 "significant": True
117             }
118         }
119     }

```

```

118 @app.patch("/experiments/{experiment_id}/status")
119 async def update_status(experiment_id: str, status: ExperimentStatus):
120     """Update experiment status."""
121     # Update in database
122     # db.update_experiment_status(experiment_id, status)
123
124     return {"experiment_id": experiment_id, "status": status}
125
126 # Run with: uvicorn api:app --reload
127

```

9.7 Code Architecture

9.7.1 Object-Oriented Design for Experiments

Listing 9.16: OOP Design Patterns for Experiments

```

1 from abc import ABC, abstractmethod
2 from typing import Dict, Any, List
3 from dataclasses import dataclass
4
5 # Strategy pattern for different metric types
6
7 class Metric(ABC):
8     """Abstract base class for metrics."""
9
10    @abstractmethod
11    def compute(self, data: pd.DataFrame) -> float:
12        """Compute metric value."""
13        pass
14
15    @abstractmethod
16    def variance(self, data: pd.DataFrame) -> float:
17        """Compute metric variance."""
18        pass
19
20    class ConversionRateMetric(Metric):
21        """Binary conversion rate metric."""
22
23        def compute(self, data: pd.DataFrame) -> float:
24            return data['converted'].mean()
25
26        def variance(self, data: pd.DataFrame) -> float:
27            p = self.compute(data)
28            return p * (1 - p) / len(data)
29
30    class RevenuePerUserMetric(Metric):
31        """Continuous revenue metric."""
32
33        def compute(self, data: pd.DataFrame) -> float:
34            return data['revenue'].mean()
35
36        def variance(self, data: pd.DataFrame) -> float:
37            return data['revenue'].var() / len(data)
38
39 # Factory pattern for experiment creation

```

```

40
41 class ExperimentFactory:
42     """Factory for creating different experiment types."""
43
44     @staticmethod
45     def create_ab_test(name: str, variants: List[str],
46                       metrics: List[Metric]) -> 'Experiment':
47         """Create standard A/B test."""
48         return ABTest(name, variants, metrics)
49
50     @staticmethod
51     def create_mab_test(name: str, variants: List[str],
52                        metrics: List[Metric]) -> 'Experiment':
53         """Create multi-armed bandit test."""
54         return MultiarmedBanditTest(name, variants, metrics)
55
56 # Experiment base class
57
58 class Experiment(ABC):
59     """Base class for all experiments."""
60
61     def __init__(self, name: str, variants: List[str], metrics: List[
62 Metric]):
63         self.name = name
64         self.variants = variants
65         self.metrics = metrics
66         self.data = None
67
68     @abstractmethod
69     def assign(self, user_id: str) -> str:
70         """Assign user to variant."""
71         pass
72
73     def load_data(self, data: pd.DataFrame):
74         """Load experiment data."""
75         self.data = data
76
77     def analyze(self) -> Dict[str, Any]:
78         """Analyze experiment."""
79         if self.data is None:
80             raise ValueError("No data loaded")
81
82         results = {}
83         for metric in self.metrics:
84             metric_results = self._analyze_metric(metric)
85             results[metric.__class__.__name__] = metric_results
86
87         return results
88
89     def _analyze_metric(self, metric: Metric) -> Dict:
90         """Analyze single metric."""
91         control_data = self.data[self.data['variant'] == 'control']
92         treatment_data = self.data[self.data['variant'] == 'treatment']
93
94         control_value = metric.compute(control_data)
95         treatment_value = metric.compute(treatment_data)
96
97         # Statistical test

```

```

97     se = np.sqrt(metric.variance(control_data) + metric.variance(
98         treatment_data))
99     z = (treatment_value - control_value) / se
100     p_value = 2 * (1 - stats.norm.cdf(abs(z)))
101
102     return {
103         'control_value': control_value,
104         'treatment_value': treatment_value,
105         'difference': treatment_value - control_value,
106         'z_statistic': z,
107         'p_value': p_value
108     }
109
110 class ABTest(Experiment):
111     """Standard A/B test with fixed allocation."""
112
113     def assign(self, user_id: str) -> str:
114         # Use consistent hashing
115         hash_value = int(hashlib.md5(f"{self.name}:{user_id}".encode())
116             .hexdigest(), 16)
117         return self.variants[hash_value % len(self.variants)]
118
119 class MultiarmedBanditTest(Experiment):
120     """Multi-armed bandit with adaptive allocation."""
121
122     def __init__(self, name: str, variants: List[str], metrics: List[
123         Metric]):
124         super().__init__(name, variants, metrics)
125         self.variant_rewards = {v: [] for v in variants}
126
127     def assign(self, user_id: str) -> str:
128         # Thompson Sampling
129         sampled_means = {}
130         for variant in self.variants:
131             rewards = self.variant_rewards[variant]
132             if len(rewards) == 0:
133                 # Uninformed prior
134                 sampled_means[variant] = np.random.beta(1, 1)
135             else:
136                 # Beta posterior
137                 successes = sum(rewards)
138                 failures = len(rewards) - successes
139                 sampled_means[variant] = np.random.beta(1 + successes,
140                     1 + failures)
141
142         return max(sampled_means, key=sampled_means.get)
143
144     def record_reward(self, variant: str, reward: float):
145         """Record reward for variant."""
146         self.variant_rewards[variant].append(reward)
147
148 # Example usage
149 factory = ExperimentFactory()
150
151 metrics = [
152     ConversionRateMetric(),
153     RevenuePerUserMetric()
154 ]

```

```

151
152 experiment = factory.create_ab_test(
153     name="checkout_redesign",
154     variants=["control", "treatment"],
155     metrics=metrics
156 )
157
158 # Assign users
159 user_variant = experiment.assign("user_123")
160
161 # Load data and analyze
162 # experiment.load_data(data)
163 # results = experiment.analyze()

```

9.7.2 Testing and Validation Frameworks

Listing 9.17: Unit Tests for Experiment System

```

1 import pytest
2 import numpy as np
3 from scipy import stats
4
5 class TestRandomAssignment:
6     """Tests for random assignment system."""
7
8     def test_consistent_assignment(self):
9         """Same user always gets same variant."""
10         system = RandomAssignmentSystem()
11
12         config = ExperimentConfig(
13             experiment_id='test_exp',
14             variants=['control', 'treatment'],
15             traffic_allocation=1.0,
16             variant_weights={'control': 0.5, 'treatment': 0.5}
17         )
18         system.register_experiment(config)
19
20         user_id = 'user_123'
21         variant1 = system.assign('test_exp', user_id)
22         variant2 = system.assign('test_exp', user_id)
23
24         assert variant1 == variant2, "Assignment not consistent"
25
26     def test_uniform_distribution(self):
27         """Assignments follow specified distribution."""
28         system = RandomAssignmentSystem()
29
30         config = ExperimentConfig(
31             experiment_id='test_exp',
32             variants=['control', 'treatment'],
33             traffic_allocation=1.0,
34             variant_weights={'control': 0.5, 'treatment': 0.5}
35         )
36         system.register_experiment(config)
37
38         # Assign many users
39         assignments = []

```

```

40     for i in range(10000):
41         variant = system.assign('test_exp', f'user_{i}')
42         assignments.append(variant)
43
44     # Count assignments
45     counts = pd.Series(assignments).value_counts()
46
47     # Chi-square test for uniformity
48     expected = [5000, 5000]
49     chi2, p_value = stats.chisquare(counts.values, expected)
50
51     assert p_value > 0.01, f"Assignments not uniform (p={p_value})"
52
53     def test_independence(self):
54         """Assignments across experiments are independent."""
55         system = RandomAssignmentSystem()
56
57         # Register two experiments
58         for exp_id in ['exp_1', 'exp_2']:
59             config = ExperimentConfig(
60                 experiment_id=exp_id,
61                 variants=['A', 'B'],
62                 traffic_allocation=1.0,
63                 variant_weights={'A': 0.5, 'B': 0.5}
64             )
65             system.register_experiment(config)
66
67         # Assign users to both experiments
68         assignments = []
69         for i in range(1000):
70             user_id = f'user_{i}'
71             var1 = system.assign('exp_1', user_id)
72             var2 = system.assign('exp_2', user_id)
73             assignments.append((var1, var2))
74
75         # Count joint assignments
76         joint_counts = pd.Series(assignments).value_counts()
77
78         # All combinations should be ~250
79         expected = 250
80         for count in joint_counts.values:
81             assert abs(count - expected) < 50, "Experiments not independent"
82
83     class TestStatisticalAnalysis:
84         """Tests for statistical analysis."""
85
86         def test_type_i_error_control(self):
87             """False positive rate under null hypothesis."""
88             n_simulations = 1000
89             alpha = 0.05
90             false_positives = 0
91
92             for _ in range(n_simulations):
93                 # Generate null data (no effect)
94                 control = np.random.binomial(1, 0.1, 1000)
95                 treatment = np.random.binomial(1, 0.1, 1000)
96

```



```

97         # Test
98         _, p_value = proportions_ztest(
99             [treatment.sum(), control.sum()],
100             [len(treatment), len(control)]
101         )
102
103         if p_value < alpha:
104             false_positives += 1
105
106         fpr = false_positives / n_simulations
107
108         # Should be close to alpha
109         assert abs(fpr - alpha) < 0.02, f"Type I error rate {fpr} != {alpha}"
110
111     def test_power(self):
112         """Power under alternative hypothesis."""
113         n_simulations = 1000
114         alpha = 0.05
115         true_positives = 0
116
117         for _ in range(n_simulations):
118             # Generate data with effect
119             control = np.random.binomial(1, 0.10, 1000)
120             treatment = np.random.binomial(1, 0.12, 1000) # 20%
121                 relative lift
122
123             # Test
124             _, p_value = proportions_ztest(
125                 [treatment.sum(), control.sum()],
126                 [len(treatment), len(control)]
127             )
128
129             if p_value < alpha:
130                 true_positives += 1
131
132             power = true_positives / n_simulations
133
134             # Should have high power for this effect size
135             assert power > 0.70, f"Power {power} too low"
136
137 # Run tests
138 pytest.main([__file__, '-v'])

```

9.7.3 Documentation and Reproducibility

Best Practices:

1. **Experiment Registry:** Central catalog of all experiments with metadata
2. **Analysis Notebooks:** Jupyter notebooks for reproducible analysis
3. **Experiment Configs:** Version-controlled YAML/JSON configs
4. **Documentation:** Auto-generated API docs, runbooks, postmortems

Listing 9.18: Experiment Documentation System

```

1 from dataclasses import dataclass, asdict
2 from datetime import datetime
3 import yaml
4 import json
5
6 @dataclass
7 class ExperimentDocumentation:
8     """Complete experiment documentation."""
9
10    # Metadata
11    experiment_id: str
12    name: str
13    owner: str
14    created_at: datetime
15
16    # Design
17    hypothesis: str
18    success_metrics: List[str]
19    guardrail_metrics: List[str]
20    variants: Dict[str, str] # variant -> description
21
22    # Configuration
23    traffic_allocation: float
24    sample_size_target: int
25    expected_duration_days: int
26
27    # Analysis
28    statistical_test: str
29    alpha: float
30    power: float
31    mde: float # Minimum detectable effect
32
33    # Results
34    status: str
35    results_summary: Optional[Dict] = None
36    decision: Optional[str] = None
37    decision_rationale: Optional[str] = None
38
39    def to_yaml(self, filepath: str):
40        """Export to YAML file."""
41        doc_dict = asdict(self)
42
43        # Convert datetime
44        doc_dict['created_at'] = doc_dict['created_at'].isoformat()
45
46        with open(filepath, 'w') as f:
47            yaml.dump(doc_dict, f, default_flow_style=False)
48
49    @classmethod
50    def from_yaml(cls, filepath: str):
51        """Load from YAML file."""
52        with open(filepath, 'r') as f:
53            data = yaml.safe_load(f)
54
55        data['created_at'] = datetime.fromisoformat(data['created_at'])
56
57        return cls(**data)

```

```

58
59 # Example
60 doc = ExperimentDocumentation(
61     experiment_id='checkout_redesign_v2',
62     name='Checkout_Flow_Redesign',
63     owner='product@company.com',
64     created_at=datetime.now(),
65     hypothesis='New_single-page_checkout_will_increase_conversion_by_
        reducing_friction',
66     success_metrics=['conversion_rate', 'revenue_per_user'],
67     guardrail_metrics=['page_load_time', 'error_rate'],
68     variants={
69         'control': 'Current_multi-step_checkout',
70         'treatment': 'New_single-page_checkout'
71     },
72     traffic_allocation=0.8,
73     sample_size_target=10000,
74     expected_duration_days=14,
75     statistical_test='two-proportion_z-test',
76     alpha=0.05,
77     power=0.80,
78     mde=0.015,
79     status='completed',
80     results_summary={'conversion_rate_lift': 0.018, 'p_value': 0.003},
81     decision='Ship_treatment',
82     decision_rationale='Significant_lift_in_conversion_rate_with_no_
        negative_guardrail_impacts'
83 )
84
85 # doc.to_yaml('experiments/checkout_redesign_v2.yaml')

```

9.8 Summary

This chapter covered the technical infrastructure required for production A/B testing:

- **Experiment Infrastructure:** Feature flags, random assignment, data collection, real-time monitoring
- **Data Engineering:** ETL pipelines, quality checks, storage strategies, privacy compliance
- **Statistical Computing:** Python/R/SQL libraries, big data with Spark
- **Automation:** Automated analysis, alerts, APIs
- **Code Architecture:** OOP design, testing frameworks, documentation

Key principles for production systems:

1. **Reliability:** Graceful degradation, caching, fallbacks
2. **Scalability:** Distributed architecture, efficient data structures
3. **Observability:** Monitoring, logging, alerting
4. **Compliance:** Privacy-preserving design, audit trails
5. **Reproducibility:** Version control, documentation, testing

9.9 Further Reading

- **Kleppmann (2017)**, *Designing Data-Intensive Applications*: System design for data pipelines
- **Kohavi et al. (2020)**, *Trustworthy Online Controlled Experiments*: Industry best practices
- **McKinney (2017)**, *Python for Data Analysis*: Pandas and data manipulation
- **VanderPlas (2016)**, *Python Data Science Handbook*: Scientific Python stack
- **Wickham & Grolemund (2017)**, *R for Data Science*: R programming and tidyverse

9.10 Exercises

1. Feature Flag System:

- Implement a feature flag system with Redis backend
- Add support for gradual rollouts (0% to 100%)
- Implement targeting rules (geography, user attributes)
- Test with load simulation (1000 req/sec)

2. Random Assignment:

- Verify consistent hashing produces uniform distribution (chi-square test)
- Implement stratified randomization by user segment
- Test independence across multiple experiments

3. ETL Pipeline:

- Build complete ETL pipeline from raw events to summary tables
- Implement incremental updates (only process new data)
- Add data quality checks at each stage
- Schedule with Apache Airflow

4. Monitoring System:

- Implement SRM detection with automated alerts
- Create real-time dashboard showing experiment health
- Set up anomaly detection for metric values
- Integrate with PagerDuty/Slack for notifications

5. Statistical Analysis:

- Implement sequential testing with alpha spending
- Add support for ratio metrics (clicks per user)

- Implement variance reduction via CUPED
- Compare Python, R, and SQL implementations for performance

6. **API Design:**

- Design REST API for experiment management (CRUD operations)
- Implement authentication and authorization
- Add rate limiting and caching
- Document with OpenAPI/Swagger

Chapter 10

Data Analysis and Result Interpretation

10.1 Learning Objectives

After completing this chapter, readers will be able to:

- Implement comprehensive data preprocessing pipelines for A/B test data
- Detect and handle outliers using multiple statistical approaches
- Address missing data with appropriate imputation strategies
- Apply data transformation techniques to meet analysis assumptions
- Conduct thorough exploratory data analysis with professional visualizations
- Build automated statistical analysis pipelines with assumption checking
- Calculate and interpret effect sizes and confidence intervals
- Distinguish between statistical and practical significance
- Perform segment analysis to identify heterogeneous treatment effects
- Create executive summaries and technical reports for different audiences
- Implement reproducible analysis workflows using Pandas and NumPy
- Generate interactive visualizations with Plotly and Matplotlib

10.2 Introduction: From Data to Decisions

The quality of experimental conclusions depends not just on experimental design but on the rigor of data analysis and interpretation. Between collecting raw data and making business decisions lies a critical analytical journey requiring statistical expertise, programming proficiency, and business acumen.

This chapter presents a comprehensive framework for analyzing A/B test results, from initial data validation through final reporting. We emphasize:

- **Reproducibility:** Analyses should be scripted, versioned, and automated
- **Robustness:** Results should withstand sensitivity analyses
- **Transparency:** Methods and limitations should be clearly documented
- **Context:** Statistical findings must connect to business impact
- **Communication:** Insights should be accessible to all stakeholders

Modern A/B testing analysis leverages powerful Python libraries:

- **Pandas:** Data manipulation and transformation
- **NumPy:** Numerical computations
- **SciPy:** Statistical tests and distributions
- **Statsmodels:** Advanced statistical modeling
- **Matplotlib/Seaborn:** Static visualizations
- **Plotly:** Interactive dashboards

We'll build practical, production-ready analysis code throughout this chapter.

10.3 Data Preprocessing

Raw experimental data typically requires substantial preprocessing before analysis. This section covers systematic approaches to data cleaning, validation, and transformation.

10.3.1 Data Quality Assessment

Before any analysis, assess data quality systematically.

Listing 10.1: Comprehensive Data Quality Framework

```
1 import pandas as pd
2 import numpy as np
3 from scipy import stats
4 from typing import Dict, List, Tuple
5 import warnings
6
7 class ABTestDataValidator:
8     """
9     Comprehensive data quality validation for A/B tests.
10
11     Checks for:
12     - Sample ratio mismatch
13     - Assignment consistency
14     - Temporal anomalies
15     - Missing data patterns
16     - Duplicate records
17     - Invalid values
18     """
19
```



```

20     def __init__(self, data: pd.DataFrame,
21                   variant_col: str = 'variant',
22                   user_col: str = 'user_id',
23                   timestamp_col: str = 'timestamp',
24                   metric_cols: List[str] = None):
25         """
26         Initialize validator.
27
28         Parameters:
29         -----
30         data: DataFrame
31         Experimental data
32         variant_col: str
33         Column indicating variant assignment
34         user_col: str
35         Column with unique user identifiers
36         timestamp_col: str
37         Column with event timestamps
38         metric_cols: List[str]
39         Columns containing metrics to analyze
40         """
41         self.data = data.copy()
42         self.variant_col = variant_col
43         self.user_col = user_col
44         self.timestamp_col = timestamp_col
45         self.metric_cols = metric_cols or []
46         self.issues = []
47         self.warnings = []
48
49     def run_all_checks(self) -> Dict:
50         """Run all validation checks."""
51         results = {
52             'sample_ratio_mismatch': self.check_sample_ratio_mismatch(),
53             'assignment_consistency': self.check_assignment_consistency(),
54             'temporal_stability': self.check_temporal_stability(),
55             'missing_data': self.check_missing_data(),
56             'duplicates': self.check_duplicates(),
57             'value_ranges': self.check_value_ranges(),
58             'novelty_effects': self.check_novelty_effects()
59         }
60
61         results['summary'] = {
62             'total_issues': len(self.issues),
63             'total_warnings': len(self.warnings),
64             'passed': len(self.issues) == 0,
65             'issues': self.issues,
66             'warnings': self.warnings
67         }
68
69         return results
70
71     def check_sample_ratio_mismatch(self,
72                                     expected_ratios: Dict = None,
73                                     alpha: float = 0.01) -> Dict:
74         """
75         Check for Sample Ratio Mismatch (SRM).

```

```

76
77         Parameters:
78         -----
79         expected_ratios: Dict
80         Expected allocation ratios {variant: proportion}
81         If None, assumes equal allocation
82         alpha: float
83         Significance level for SRM test
84
85         Returns:
86         -----
87         Dict with SRM test results
88         """
89         variant_counts = self.data[self.variant_col].value_counts()
90         n_variants = len(variant_counts)
91         total_n = len(self.data)
92
93         # Default to equal allocation
94         if expected_ratios is None:
95             expected_ratios = {v: 1/n_variants for v in variant_counts.
96                                index}
97
98         # Calculate expected counts
99         expected_counts = np.array([
100             expected_ratios.get(v, 1/n_variants) * total_n
101             for v in variant_counts.index
102         ])
103
104         # Chi-square test
105         chi2_stat, p_value = stats.chisquare(
106             variant_counts.values,
107             expected_counts
108         )
109
110         # Check for SRM
111         has_srm = p_value < alpha
112
113         if has_srm:
114             self.issues.append(
115                 f"Sample Ratio Mismatch detected (p={p_value:.6f})"
116             )
117
118         return {
119             'chi2_statistic': chi2_stat,
120             'p_value': p_value,
121             'has_srm': has_srm,
122             'observed_counts': dict(variant_counts),
123             'expected_counts': dict(zip(variant_counts.index, expected_
124                                         counts)),
125             'percent_deviation': dict(
126                 (v, (variant_counts[v] - expected_counts[i]) /
127                    expected_counts[i] * 100)
128                 for i, v in enumerate(variant_counts.index)
129             )
130         }
131
132     def check_assignment_consistency(self) -> Dict:
133         """

```

```

132     """Check if users have consistent variant assignments.
133
134     Returns:
135     """
136     Dict with consistency check results
137     """
138     # Count unique variants per user
139     variants_per_user = self.data.groupby(
140         self.user_col
141     )[self.variant_col].nunique()
142
143     inconsistent_users = variants_per_user[variants_per_user > 1]
144     n_inconsistent = len(inconsistent_users)
145     pct_inconsistent = n_inconsistent / len(variants_per_user) *
146         100
147
148     if pct_inconsistent > 1:
149         self.issues.append(
150             f"{n_inconsistent} users ({pct_inconsistent:.2f}%) "
151             f"with inconsistent assignments"
152         )
153     elif pct_inconsistent > 0:
154         self.warnings.append(
155             f"{n_inconsistent} users with inconsistent assignments"
156         )
157
158     return {
159         'n_inconsistent_users': n_inconsistent,
160         'pct_inconsistent': pct_inconsistent,
161         'inconsistent_user_ids': list(inconsistent_users.index),
162         'is_consistent': n_inconsistent == 0
163     }
164
165     def check_temporal_stability(self, window: str = 'D') -> Dict:
166         """
167         Check for temporal instability in metric distributions.
168
169         Parameters:
170         """
171         window: str
172         Pandas frequency string for grouping ('D', 'H', etc.)
173
174         Returns:
175         """
176         Dict with temporal stability results
177         """
178         self.data[self.timestamp_col] = pd.to_datetime(
179             self.data[self.timestamp_col]
180         )
181         self.data['period'] = self.data[self.timestamp_col].dt.floor(
182             window)
183
184         temporal_stats = {}
185
186         for metric in self.metric_cols:
187             # Calculate metric by period and variant
188             period_stats = self.data.groupby(
189                 ['period', self.variant_col]

```

```

188         )[metric].agg(['mean', 'std', 'count']).reset_index()
189
190     # Calculate coefficient of variation over time
191     cv_by_variant = {}
192     for variant in period_stats[self.variant_col].unique():
193         variant_data = period_stats[
194             period_stats[self.variant_col] == variant
195         ]
196         mean_of_means = variant_data['mean'].mean()
197         std_of_means = variant_data['mean'].std()
198         cv = std_of_means / mean_of_means if mean_of_means != 0
199             else np.inf
200         cv_by_variant[variant] = cv
201
202     # Flag high variability
203     max_cv = max(cv_by_variant.values())
204     if max_cv > 0.5:
205         self.warnings.append(
206             f"High temporal variability in {metric} (CV={max_cv:.2f})"
207         )
208
209     temporal_stats[metric] = {
210         'cv_by_variant': cv_by_variant,
211         'max_cv': max_cv,
212         'period_data': period_stats
213     }
214
215     return temporal_stats
216
217     def check_missing_data(self, threshold: float = 5.0) -> Dict:
218         """
219         Analyze missing data patterns.
220
221         Parameters:
222         -----
223         threshold: float
224             Percentage threshold for flagging high missingness
225
226         Returns:
227         -----
228         Dict with missing data analysis
229         """
230         missing_pct = (self.data.isnull().sum() / len(self.data) * 100)
231
232         high_missing = missing_pct[missing_pct > threshold]
233
234         if len(high_missing) > 0:
235             self.warnings.append(
236                 f"High missing data in columns: {dict(high_missing)}"
237             )
238
239     # Check if missingness differs by variant
240     missing_by_variant = {}
241     for col in self.data.columns:
242         if self.data[col].isnull().sum() > 0:
243             missing_rates = self.data.groupby(

```

```

244         )[col].apply(lambda x: x.isnull().sum() / len(x) * 100)
245         missing_by_variant[col] = dict(missing_rates)
246
247     return {
248         'missing_percentages': dict(missing_pct),
249         'high_missing_columns': dict(high_missing),
250         'missing_by_variant': missing_by_variant
251     }
252
253     def check_duplicates(self) -> Dict:
254         """Check for duplicate records."""
255         n_total = len(self.data)
256         n_unique = len(self.data.drop_duplicates())
257         n_duplicates = n_total - n_unique
258         pct_duplicates = n_duplicates / n_total * 100
259
260         if pct_duplicates > 0:
261             self.warnings.append(
262                 f"{n_duplicates} duplicate records ({pct_duplicates:.2f}%)"
263             )
264
265         return {
266             'n_duplicates': n_duplicates,
267             'pct_duplicates': pct_duplicates,
268             'has_duplicates': n_duplicates > 0
269         }
270
271     def check_value_ranges(self) -> Dict:
272         """Check for values outside expected ranges."""
273         range_issues = {}
274
275         for metric in self.metric_cols:
276             data_series = self.data[metric].dropna()
277
278             # Basic range statistics
279             min_val = data_series.min()
280             max_val = data_series.max()
281             mean_val = data_series.mean()
282             std_val = data_series.std()
283
284             # Check for negative values where inappropriate
285             if 'revenue' in metric.lower() or 'time' in metric.lower():
286                 n_negative = (data_series < 0).sum()
287                 if n_negative > 0:
288                     self.warnings.append(
289                         f"{n_negative} negative values in {metric}"
290                     )
291                 range_issues[metric] = {'negative_values': n_negative}
292
293             # Check for extreme outliers (> 5 std from mean)
294             z_scores = np.abs(stats.zscore(data_series))
295             n_extreme = (z_scores > 5).sum()
296             if n_extreme > 0:
297                 self.warnings.append(
298                     f"{n_extreme} extreme outliers in {metric} (>5 SD)"
299                 )

```

```

300         if metric in range_issues:
301             range_issues[metric]['extreme_outliers'] = n_
                extreme
302         else:
303             range_issues[metric] = {'extreme_outliers': n_
                extreme}
304
305     return range_issues
306
307     def check_novelty_effects(self, periods: int = 3) -> Dict:
308         """
309         Check for novelty effects (first-period differences).
310
311         Parameters:
312         -----
313         periods: int
314         Number of time periods to examine
315
316         Returns:
317         -----
318         Dict with novelty effect analysis
319         """
320         self.data[self.timestamp_col] = pd.to_datetime(
321             self.data[self.timestamp_col]
322         )
323         self.data['period_rank'] = self.data.groupby(
324             self.variant_col
325         )[self.timestamp_col].rank(method='dense', pct=True)
326
327         # Split into early and late periods
328         early_cutoff = 1.0 / periods
329
330         novelty_results = {}
331         for metric in self.metric_cols:
332             early_data = self.data[self.data['period_rank'] <= early_
                cutoff]
333             late_data = self.data[self.data['period_rank'] > early_
                cutoff]
334
335             early_means = early_data.groupby(self.variant_col)[metric].
                mean()
336             late_means = late_data.groupby(self.variant_col)[metric].
                mean()
337
338             # Compare early vs late
339             relative_change = ((late_means - early_means) / early_means
                * 100)
340
341             novelty_results[metric] = {
342                 'early_means': dict(early_means),
343                 'late_means': dict(late_means),
344                 'relative_change_pct': dict(relative_change)
345             }
346
347         return novelty_results
348
349     def generate_report(self) -> str:
350         """Generate human-readable validation report."""

```

```

351     results = self.run_all_checks()
352
353     report = []
354     report.append("=" * 60)
355     report.append("A/B TEST DATA VALIDATION REPORT")
356     report.append("=" * 60)
357     report.append("")
358
359     # Summary
360     summary = results['summary']
361     status = "PASSED" if summary['passed'] else "FAILED"
362     report.append(f"Overall Status: {status}")
363     report.append(f"Total Issues: {summary['total_issues']}")
364     report.append(f"Total Warnings: {summary['total_warnings']}")
365     report.append("")
366
367     # Issues
368     if summary['issues']:
369         report.append("CRITICAL ISSUES:")
370         for issue in summary['issues']:
371             report.append(f"  {issue}")
372         report.append("")
373
374     # Warnings
375     if summary['warnings']:
376         report.append("WARNINGS:")
377         for warning in summary['warnings']:
378             report.append(f"  {warning}")
379         report.append("")
380
381     # Detailed results
382     report.append("DETAILED RESULTS:")
383     report.append("")
384
385     # SRM
386     srm = results['sample_ratio_mismatch']
387     report.append(f"Sample Ratio Mismatch Test: p-value = {srm['p_value']:.6f}")
388     report.append(f"  Observed counts: {srm['observed_counts']}")
389     report.append("")
390
391     return "\n".join(report)
392
393 # Example usage
394 if __name__ == "__main__":
395     # Load experimental data
396     data = pd.read_csv('experiment_data.csv')
397
398     # Initialize validator
399     validator = ABTestDataValidator(
400         data=data,
401         variant_col='variant',
402         user_col='user_id',
403         timestamp_col='timestamp',
404         metric_cols=['conversion', 'revenue', 'engagement_time']
405     )
406
407     # Run validation

```

```

408     # print(validator.generate_report())
409     pass

```

10.3.2 Outlier Detection and Treatment

Outliers can substantially bias estimates, particularly for mean-based metrics. We need systematic approaches to detect and handle extreme values.

Detection Methods

1. **Statistical methods:** Z-scores, IQR, modified Z-scores
2. **Visual methods:** Box plots, scatter plots, histograms
3. **Domain knowledge:** Business rules and plausibility checks
4. **Robust statistics:** Median absolute deviation (MAD)

Listing 10.2: Comprehensive Outlier Detection

```

1  class OutlierDetector:
2      """
3      Multiple methods for outlier detection in A/B test data.
4      """
5
6      @staticmethod
7      def detect_zscore(data: pd.Series, threshold: float = 3) -> pd.
8          Series:
9          """
10             Detect outliers using Z-score method.
11
12             Parameters:
13             -----
14             data: Series
15             Numerical data
16             threshold: float
17             Z-score threshold (typically 2.5-3)
18
19             Returns:
20             -----
21             Boolean series indicating outliers
22             """
23             z_scores = np.abs(stats.zscore(data.dropna()))
24             return pd.Series(z_scores > threshold, index=data.dropna().
25                             index)
26
27     @staticmethod
28     def detect_iqr(data: pd.Series, k: float = 1.5) -> pd.Series:
29         """
30         Detect outliers using IQR method.
31
32         Parameters:
33         -----
34         data: Series
35         Numerical data
36         k: float

```



```

35 IQR_multiplier(1.5_for_outliers, 3_for_extreme_outliers)
36
37 Returns:
38 -----
39 Boolean series indicating outliers
40 """
41     Q1 = data.quantile(0.25)
42     Q3 = data.quantile(0.75)
43     IQR = Q3 - Q1
44
45     lower_bound = Q1 - k * IQR
46     upper_bound = Q3 + k * IQR
47
48     return (data < lower_bound) | (data > upper_bound)
49
50 @staticmethod
51 def detect_mad(data: pd.Series, threshold: float = 3.5) -> pd.
    Series:
52     """
53 Detect outliers using Median Absolute Deviation (MAD).
54 More robust to outliers than Z-score.
55
56 Parameters:
57 -----
58 data: Series
59 Numerical data
60 threshold: float
61 MAD threshold (typically 3.5)
62
63 Returns:
64 -----
65 Boolean series indicating outliers
66 """
67     median = data.median()
68     mad = np.median(np.abs(data - median))
69
70     # Modified Z-score
71     modified_z_scores = 0.6745 * (data - median) / mad
72
73     return np.abs(modified_z_scores) > threshold
74
75 @staticmethod
76 def detect_isolation_forest(data: pd.DataFrame,
77                             contamination: float = 0.1) -> pd.Series
78     :
79     """
80 Detect outliers using Isolation Forest algorithm.
81
82 Parameters:
83 -----
84 data: DataFrame
85 Numerical features
86 contamination: float
87 Expected proportion of outliers
88
89 Returns:
90 -----
91 Boolean series indicating outliers

```

```

91 """
92     from sklearn.ensemble import IsolationForest
93
94     clf = IsolationForest(contamination=contamination, random_state
95                             =42)
96     predictions = clf.fit_predict(data)
97
98     return pd.Series(predictions == -1, index=data.index)
99
100 @staticmethod
101 def detect_domain_specific(data: pd.Series,
102                             metric_name: str,
103                             business_rules: Dict = None) -> pd.Series
104     :
105     """
106     Apply domain-specific outlier rules.
107
108     Parameters:
109     -----
110     data: Series
111     metric_name: str
112     Name of metric for rule selection
113     business_rules: Dict
114     Custom business rules {metric: {'min': val, 'max': val}}
115
116     Returns:
117     -----
118     Boolean series indicating outliers
119     """
120     if business_rules and metric_name in business_rules:
121         rules = business_rules[metric_name]
122         min_val = rules.get('min', -np.inf)
123         max_val = rules.get('max', np.inf)
124         return (data < min_val) | (data > max_val)
125
126     # Default rules for common metrics
127     if 'revenue' in metric_name.lower():
128         # Revenue should be non-negative and below some maximum
129         max_revenue = data.quantile(0.999) * 2
130         return (data < 0) | (data > max_revenue)
131
132     if 'time' in metric_name.lower():
133         # Time metrics should be non-negative
134         max_time = data.quantile(0.999) * 2
135         return (data < 0) | (data > max_time)
136
137     if 'rate' in metric_name.lower() or 'ratio' in metric_name.
138         lower():
139         # Rates should be in [0, 1] or [0, 100]
140         return (data < 0) | (data > 100)
141
142     return pd.Series(False, index=data.index)
143
144 @staticmethod
145 def consensus_detection(data: pd.Series, methods: List[str] = None,
146                         min_agreement: int = 2) -> pd.Series:
147     """

```

```

146 Combine multiple detection methods with voting.
147
148 Parameters:
149 -----
150 data: Series
151       Numerical data
152 methods: List[str]
153 Methods to use: ['zscore', 'iqr', 'mad']
154 min_agreement: int
155 Minimum number of methods that must agree
156
157 Returns:
158 -----
159 Boolean series indicating consensus outliers
160 """
161     if methods is None:
162         methods = ['zscore', 'iqr', 'mad']
163
164     detector = OutlierDetector()
165     results = []
166
167     if 'zscore' in methods:
168         results.append(detector.detect_zscore(data))
169     if 'iqr' in methods:
170         results.append(detector.detect_iqr(data))
171     if 'mad' in methods:
172         results.append(detector.detect_mad(data))
173
174     # Count agreements
175     agreement_count = sum(results)
176
177     return agreement_count >= min_agreement
178
179 # Example visualization
180 def plot_outlier_analysis(data: pd.Series, metric_name: str):
181     """Create comprehensive outlier visualization."""
182     import matplotlib.pyplot as plt
183     import seaborn as sns
184
185     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
186
187     detector = OutlierDetector()
188
189     # 1. Box plot
190     axes[0, 0].boxplot(data.dropna())
191     axes[0, 0].set_ylabel(metric_name)
192     axes[0, 0].set_title('Box Plot')
193     axes[0, 0].grid(alpha=0.3)
194
195     # 2. Histogram with outlier thresholds
196     axes[0, 1].hist(data.dropna(), bins=50, alpha=0.7, edgecolor='black
197                     ,')
198
199     # Add IQR bounds
200     Q1 = data.quantile(0.25)
201     Q3 = data.quantile(0.75)
202     IQR = Q3 - Q1
203     lower = Q1 - 1.5 * IQR

```

```

203     upper = Q3 + 1.5 * IQR
204
205     axes[0, 1].axvline(lower, color='red', linestyle='--',
206                       label=f'IQR_Lower: {lower:.2f}')
207     axes[0, 1].axvline(upper, color='red', linestyle='--',
208                       label=f'IQR_Upper: {upper:.2f}')
209     axes[0, 1].set_xlabel(metric_name)
210     axes[0, 1].set_ylabel('Frequency')
211     axes[0, 1].set_title('Distribution with IQR Bounds')
212     axes[0, 1].legend()
213     axes[0, 1].grid(alpha=0.3)
214
215     # 3. Q-Q plot
216     stats.probplot(data.dropna(), dist="norm", plot=axes[1, 0])
217     axes[1, 0].set_title('Q-Q Plot (Normal Distribution)')
218     axes[1, 0].grid(alpha=0.3)
219
220     # 4. Detection method comparison
221     methods = {
222         'Z-score': detector.detect_zscore(data),
223         'IQR': detector.detect_iqr(data),
224         'MAD': detector.detect_mad(data)
225     }
226
227     method_counts = {k: v.sum() for k, v in methods.items()}
228
229     axes[1, 1].bar(method_counts.keys(), method_counts.values())
230     axes[1, 1].set_ylabel('Number of Outliers Detected')
231     axes[1, 1].set_title('Outlier Detection Method Comparison')
232     axes[1, 1].grid(alpha=0.3)
233
234     plt.tight_layout()
235     return fig

```

Treatment Strategies

Once outliers are detected, choose appropriate treatment:

1. **Removal:** Delete outlier observations (use cautiously)
2. **Winsorization:** Cap at percentiles (e.g., 1st and 99th)
3. **Transformation:** Log, square root, or other transforms
4. **Robust statistics:** Use median, trimmed mean, or Huber loss
5. **Separate analysis:** Report both with and without outliers

Listing 10.3: Outlier Treatment Methods

```

1 class OutlierTreatment:
2     """Methods for treating outliers in A/B test data."""
3
4     @staticmethod
5     def winsorize(data: pd.Series,
6                  lower_percentile: float = 1,

```

```

7         upper_percentile: float = 99) -> Tuple[pd.Series,
          Dict]:
8
9         """
10        Winorize data by capping at percentiles.
11
12        Parameters:
13        -----
14        data: Series
15        Numerical data
16        lower_percentile, upper_percentile: float
17        Percentile thresholds
18
19        Returns:
20        -----
21        Tuple of (winsorized data, metadata dict)
22        """
23        from scipy.stats.mstats import winsorize as scipy_winsorize
24
25        lower_bound = np.percentile(data.dropna(), lower_percentile)
26        upper_bound = np.percentile(data.dropna(), upper_percentile)
27
28        winsorized = data.clip(lower=lower_bound, upper=upper_bound)
29
30        n_capped_lower = (data < lower_bound).sum()
31        n_capped_upper = (data > upper_bound).sum()
32        n_capped_total = n_capped_lower + n_capped_upper
33
34        metadata = {
35            'lower_bound': lower_bound,
36            'upper_bound': upper_bound,
37            'n_capped_lower': n_capped_lower,
38            'n_capped_upper': n_capped_upper,
39            'n_capped_total': n_capped_total,
40            'pct_capped': n_capped_total / len(data) * 100
41        }
42
43        return winsorized, metadata
44
45    @staticmethod
46    def log_transform(data: pd.Series, offset: float = 1) -> pd.Series:
47        """
48        Apply log transformation.
49
50        Parameters:
51        -----
52        data: Series
53        Numerical data (must be positive)
54        offset: float
55        Value to add before log (handles zeros)
56
57        Returns:
58        -----
59        Log-transformed series
60        """
61        return np.log(data + offset)
62
63    @staticmethod
64    def remove_outliers(data: pd.DataFrame,

```

```

64         metric_col: str,
65         method: str = 'iqr',
66         **kwargs) -> Tuple[pd.DataFrame, Dict]:
67     """
68     Remove outlier observations.
69
70     Parameters:
71     -----
72     data: DataFrame
73     Experimental data
74     metric_col: str
75     Column to check for outliers
76     method: str
77     Detection method: 'zscore', 'iqr', 'mad'
78     **kwargs: additional arguments for detection method
79
80     Returns:
81     -----
82     Tuple of (cleaned DataFrame, metadata dict)
83     """
84     detector = OutlierDetector()
85
86     if method == 'zscore':
87         outliers = detector.detect_zscore(data[metric_col], **
88             kwargs)
89     elif method == 'iqr':
90         outliers = detector.detect_iqr(data[metric_col], **kwargs)
91     elif method == 'mad':
92         outliers = detector.detect_mad(data[metric_col], **kwargs)
93     else:
94         raise ValueError(f"Unknown method: {method}")
95
96     n_outliers = outliers.sum()
97     pct_removed = n_outliers / len(data) * 100
98
99     cleaned_data = data[~outliers]
100
101     metadata = {
102         'method': method,
103         'n_outliers': n_outliers,
104         'pct_removed': pct_removed,
105         'original_n': len(data),
106         'cleaned_n': len(cleaned_data)
107     }
108
109     return cleaned_data, metadata
110
111     @staticmethod
112     def trimmed_mean(data: pd.Series, trim_pct: float = 0.1) -> float:
113         """
114         Calculate trimmed mean (removes extremes from both tails).
115
116         Parameters:
117         -----
118         data: Series
119         Numerical data
120         trim_pct: float
121         Percentage to trim from each tail (0.1 = 10%)

```

```

121
122 """Returns:
123 """
124 """Trimmed mean value
125 """
126 from scipy.stats import trim_mean
127 return trim_mean(data.dropna(), trim_pct)
128
129 @staticmethod
130 def compare_treatments(data: pd.DataFrame,
131                       metric_col: str,
132                       variant_col: str = 'variant') -> pd.DataFrame
133     :
134     """
135 """
136 """Compare effect estimates under different treatments.
137
138 Parameters:
139 """
140 """data: DataFrame
141 """
142 """Experimental data
143 """
144 """metric_col: str
145 """
146 """Metric to analyze
147 """
148 """variant_col: str
149 """
150 """Variant column
151 """
152
153 """Returns:
154 """
155 """DataFrame comparing treatment methods
156 """
157 """
158 treatment = OutlierTreatment()
159
160 results = []
161
162 # Original data
163 for variant in data[variant_col].unique():
164     variant_data = data[data[variant_col] == variant][metric_col]
165     results.append({
166         'treatment': 'original',
167         'variant': variant,
168         'mean': variant_data.mean(),
169         'median': variant_data.median(),
170         'std': variant_data.std(),
171         'n': len(variant_data)
172     })
173
174 # Winsorized
175 data_wins = data.copy()
176 data_wins[f'{metric_col}_wins'], _ = treatment.winsorize(
177     data[metric_col]
178 )
179 for variant in data_wins[variant_col].unique():
180     variant_data = data_wins[
181         data_wins[variant_col] == variant
182     ][f'{metric_col}_wins']
183     results.append({
184         'treatment': 'winsorized',
185         'variant': variant,

```

```

177         'mean': variant_data.mean(),
178         'median': variant_data.median(),
179         'std': variant_data.std(),
180         'n': len(variant_data)
181     })
182
183     # Log-transformed
184     if (data[metric_col] > 0).all():
185         data_log = data.copy()
186         data_log[f'{metric_col}_log'] = treatment.log_transform(
187             data[metric_col]
188         )
189         for variant in data_log[variant_col].unique():
190             variant_data = data_log[
191                 data_log[variant_col] == variant
192             ][f'{metric_col}_log']
193             results.append({
194                 'treatment': 'log_transform',
195                 'variant': variant,
196                 'mean': variant_data.mean(),
197                 'median': variant_data.median(),
198                 'std': variant_data.std(),
199                 'n': len(variant_data)
200             })
201
202     # Outliers removed
203     data_clean, _ = treatment.remove_outliers(
204         data, metric_col, method='iqr'
205     )
206     for variant in data_clean[variant_col].unique():
207         variant_data = data_clean[
208             data_clean[variant_col] == variant
209         ][metric_col]
210         results.append({
211             'treatment': 'outliers_removed',
212             'variant': variant,
213             'mean': variant_data.mean(),
214             'median': variant_data.median(),
215             'std': variant_data.std(),
216             'n': len(variant_data)
217         })
218
219     return pd.DataFrame(results)

```

10.3.3 Missing Data Handling

Missing data is ubiquitous in A/B tests. The approach depends on the missingness mechanism:

- **MCAR (Missing Completely At Random):** Missingness independent of any variable
- **MAR (Missing At Random):** Missingness depends on observed variables
- **MNAR (Missing Not At Random):** Missingness depends on unobserved variables

Listing 10.4: Missing Data Analysis and Imputation

```

1 class MissingDataHandler:
2     """
3     Comprehensive missing data analysis and treatment.
4     """
5
6     @staticmethod
7     def analyze_missingness(data: pd.DataFrame) -> pd.DataFrame:
8         """
9         Comprehensive missing data analysis.
10
11         Returns:
12         DataFrame with missingness statistics per column
13         """
14         missing_stats = []
15
16         for col in data.columns:
17             n_missing = data[col].isnull().sum()
18             pct_missing = n_missing / len(data) * 100
19
20             missing_stats.append({
21                 'column': col,
22                 'n_missing': n_missing,
23                 'pct_missing': pct_missing,
24                 'dtype': str(data[col].dtype)
25             })
26
27         return pd.DataFrame(missing_stats).sort_values(
28             'pct_missing', ascending=False
29         )
30
31     @staticmethod
32     def test_mcar(data: pd.DataFrame,
33                  target_col: str,
34                  predictor_cols: List[str] = None) -> Dict:
35         """
36         Test if data is Missing Completely At Random using Little's
37         test.
38
39         Parameters:
40         data: DataFrame
41         target_col: str
42         predictor_cols: List[str]
43         """
44         if predictor_cols is None:
45             predictor_cols = [c for c in data.columns if c != target_col]
46
47         # Create missingness indicator

```

```

56     data_test = data[predictor_cols + [target_col]].copy()
57     data_test['is_missing'] = data_test[target_col].isnull().astype
        (int)
58
59     # Test if predictors differ between missing and non-missing
60     results = {}
61     for col in predictor_cols:
62         if data_test[col].dtype in ['int64', 'float64']:
63             # T-test for numerical predictors
64             missing_group = data_test[
65                 data_test['is_missing'] == 1
66             ][col].dropna()
67             present_group = data_test[
68                 data_test['is_missing'] == 0
69             ][col].dropna()
70
71             if len(missing_group) > 0 and len(present_group) > 0:
72                 t_stat, p_val = stats.ttest_ind(
73                     missing_group, present_group
74                 )
75                 results[col] = {
76                     'test': 't-test',
77                     'statistic': t_stat,
78                     'p_value': p_val,
79                     'differs_significantly': p_val < 0.05
80                 }
81
82     # Overall assessment
83     n_significant = sum(
84         1 for r in results.values() if r.get('differs_significantly', False)
85     )
86
87     likely_mcar = n_significant == 0
88
89     return {
90         'predictor_tests': results,
91         'n_significant_predictors': n_significant,
92         'likely_mcar': likely_mcar
93     }
94
95     @staticmethod
96     def simple_imputation(data: pd.DataFrame,
97                           col: str,
98                           method: str = 'mean',
99                           variant_col: str = None) -> pd.Series:
100         """
101         Simple imputation methods.
102
103         Parameters:
104         -----
105         data: DataFrame
106             Data with missing values
107         col: str
108             Column to impute
109         method: str
110             'mean', 'median', 'mode', or 'forward_fill'
111         variant_col: str

```

```

112 """If provided, impute within variants
113
114 Returns:
115 -----
116 Series with imputed values
117 """
118 if variant_col:
119     # Impute within each variant
120     result = data[col].copy()
121     for variant in data[variant_col].unique():
122         mask = data[variant_col] == variant
123         variant_data = data.loc[mask, col]
124
125         if method == 'mean':
126             fill_value = variant_data.mean()
127         elif method == 'median':
128             fill_value = variant_data.median()
129         elif method == 'mode':
130             fill_value = variant_data.mode()[0]
131         else:
132             raise ValueError(f"Unknown method: {method}")
133
134         result.loc[mask] = variant_data.fillna(fill_value)
135
136     return result
137 else:
138     # Global imputation
139     if method == 'mean':
140         return data[col].fillna(data[col].mean())
141     elif method == 'median':
142         return data[col].fillna(data[col].median())
143     elif method == 'mode':
144         return data[col].fillna(data[col].mode()[0])
145     elif method == 'forward_fill':
146         return data[col].fillna(method='ffill')
147     else:
148         raise ValueError(f"Unknown method: {method}")
149
150 @staticmethod
151 def multiple_imputation(data: pd.DataFrame,
152                        cols_to_impute: List[str],
153                        n_imputations: int = 5) -> List[pd.DataFrame]:
154     """
155     Multiple imputation using MICE algorithm.
156
157     Parameters:
158     -----
159     data: DataFrame
160         Data with missing values
161     cols_to_impute: List[str]
162         Columns to impute
163     n_imputations: int
164         Number of imputed datasets to create
165
166     Returns:
167     -----
168     List of imputed DataFrames

```

```

169 """
170 from sklearn.experimental import enable_iterative_imputer
171 from sklearn.impute import IterativeImputer
172
173 imputer = IterativeImputer(
174     max_iter=10,
175     random_state=42,
176     sample_posterior=True
177 )
178
179 imputed_datasets = []
180
181 for i in range(n_imputations):
182     data_imputed = data.copy()
183     imputer.set_params(random_state=42 + i)
184
185     # Impute numerical columns
186     numerical_cols = data[cols_to_impute].select_dtypes(
187         include=['float64', 'int64']
188     ).columns
189
190     if len(numerical_cols) > 0:
191         imputed_values = imputer.fit_transform(
192             data[numerical_cols]
193         )
194         data_imputed[numerical_cols] = imputed_values
195
196     imputed_datasets.append(data_imputed)
197
198     return imputed_datasets
199
200 @staticmethod
201 def analyze_with_multiple_imputation(
202     imputed_datasets: List[pd.DataFrame],
203     variant_col: str,
204     metric_col: str
205 ) -> Dict:
206     """
207     Analyze with multiple imputation and pool results (Rubin's
208     rules).
209
210     Parameters:
211     imputed_datasets: List[DataFrame]
212     List of imputed datasets
213     variant_col: str
214     Variant column
215     metric_col: str
216     Metric to analyze
217
218     Returns:
219     Dict with pooled results
220
221     """
222     estimates = []
223     variances = []
224
225     for data_imp in imputed_datasets:

```

```

226         # Calculate treatment effect in this imputed dataset
227         variants = data_imp[variant_col].unique()
228         if len(variants) == 2:
229             control = variants[0]
230             treatment = variants[1]
231
232             control_data = data_imp[
233                 data_imp[variant_col] == control
234             ][metric_col]
235             treatment_data = data_imp[
236                 data_imp[variant_col] == treatment
237             ][metric_col]
238
239             # Difference in means
240             diff = treatment_data.mean() - control_data.mean()
241
242             # Variance of difference
243             var_diff = (
244                 control_data.var() / len(control_data) +
245                 treatment_data.var() / len(treatment_data)
246             )
247
248             estimates.append(diff)
249             variances.append(var_diff)
250
251     # Pool using Rubin's rules
252     m = len(imputed_datasets)
253
254     # Pooled estimate
255     pooled_estimate = np.mean(estimates)
256
257     # Within-imputation variance
258     within_var = np.mean(variances)
259
260     # Between-imputation variance
261     between_var = np.var(estimates, ddof=1)
262
263     # Total variance
264     total_var = within_var + (1 + 1/m) * between_var
265
266     # Standard error
267     se = np.sqrt(total_var)
268
269     # Degrees of freedom (Barnard-Rubin adjustment)
270     df = (m - 1) * (1 + within_var / ((1 + 1/m) * between_var)) **
271         2
272
273     # Confidence interval
274     t_crit = stats.t.ppf(0.975, df)
275     ci_lower = pooled_estimate - t_crit * se
276     ci_upper = pooled_estimate + t_crit * se
277
278     return {
279         'pooled_estimate': pooled_estimate,
280         'standard_error': se,
281         'ci_lower': ci_lower,
282         'ci_upper': ci_upper,
283         'within_variance': within_var,

```

```

283         'between_variance': between_var,
284         'total_variance': total_var,
285         'degrees_of_freedom': df,
286         'individual_estimates': estimates
287     }

```

10.3.4 Data Transformation Techniques

Transformations can improve adherence to statistical assumptions (normality, homoscedasticity).

1. **Log transformation:** For right-skewed data ($y' = \log(y)$)
2. **Square root:** For count data ($y' = \sqrt{y}$)
3. **Box-Cox:** General power transformation
4. **Logit:** For proportions ($y' = \log(p/(1 - p))$)
5. **Standardization:** Zero mean, unit variance ($y' = (y - \mu)/\sigma$)

Listing 10.5: Data Transformation Functions

```

1 class DataTransformer:
2     """Data transformation methods for A/B test analysis."""
3
4     @staticmethod
5     def log_transform(data: pd.Series, offset: float = 1) -> pd.Series:
6         """Log transformation with offset for zeros."""
7         return np.log(data + offset)
8
9     @staticmethod
10    def sqrt_transform(data: pd.Series) -> pd.Series:
11        """Square root transformation for count data."""
12        return np.sqrt(data)
13
14    @staticmethod
15    def boxcox_transform(data: pd.Series) -> Tuple[pd.Series, float]:
16        """
17        Box-Cox transformation.
18
19        Returns:
20        -----
21        Tuple of (transformed data, lambda parameter)
22        """
23        from scipy.stats import boxcox
24
25        # Box-Cox requires positive data
26        if (data <= 0).any():
27            offset = abs(data.min()) + 1
28            data_positive = data + offset
29        else:
30            data_positive = data
31
32        transformed, lambda_param = boxcox(data_positive)
33

```

```

34         return pd.Series(transformed, index=data.index), lambda_param
35
36     @staticmethod
37     def standardize(data: pd.Series) -> pd.Series:
38         """Z-score standardization."""
39         return (data - data.mean()) / data.std()
40
41     @staticmethod
42     def recommend_transformation(data: pd.Series) -> str:
43         """
44         Recommend transformation based on data characteristics.
45
46         Returns:
47         -----
48         str: Recommended transformation
49         """
50         # Check skewness
51         skew = stats.skew(data.dropna())
52
53         # Check if data has zeros
54         has_zeros = (data == 0).any()
55
56         # Check if all positive
57         all_positive = (data > 0).all()
58
59         # Check range
60         data_range = data.max() - data.min()
61         mean_val = data.mean()
62
63         # Decision tree for recommendation
64         if abs(skew) < 0.5:
65             return "none_(data_is_approximately_symmetric)"
66
67         if skew > 1 and all_positive:
68             return "log_(right-skewed,_positive_data)"
69
70         if skew > 1 and has_zeros:
71             return "log_with_offset_(right-skewed_with_zeros)"
72
73         if skew < -1:
74             return "reflect_and_log_(left-skewed)"
75
76         if 0.5 <= abs(skew) <= 1:
77             return "sqrt_(moderately_skewed)"
78
79         return "boxcox_(general_power_transformation)"
80
81     @staticmethod
82     def assess_normality(data: pd.Series) -> Dict:
83         """
84         Assess normality using multiple tests.
85
86         Returns:
87         -----
88         Dict with normality test results
89         """
90         # Shapiro-Wilk test (good for n < 5000)
91         if len(data) < 5000:

```

```

92         shapiro_stat, shapiro_p = stats.shapiro(data.dropna())
93     else:
94         shapiro_stat, shapiro_p = None, None
95
96     # Anderson-Darling test
97     anderson_result = stats.anderson(data.dropna())
98
99     # Kolmogorov-Smirnov test
100    ks_stat, ks_p = stats.kstest(
101        data.dropna(),
102        'norm',
103        args=(data.mean(), data.std())
104    )
105
106    # Skewness and kurtosis
107    skewness = stats.skew(data.dropna())
108    kurt = stats.kurtosis(data.dropna())
109
110    return {
111        'shapiro_statistic': shapiro_stat,
112        'shapiro_p_value': shapiro_p,
113        'is_normal_shapiro': shapiro_p > 0.05 if shapiro_p else
114            None,
115        'ks_statistic': ks_stat,
116        'ks_p_value': ks_p,
117        'is_normal_ks': ks_p > 0.05,
118        'skewness': skewness,
119        'kurtosis': kurt,
120        'anderson_statistic': anderson_result.statistic,
121        'anderson_critical_values': anderson_result.critical_values
122    }
123
124    @staticmethod
125    def compare_transformations(data: pd.DataFrame,
126                               metric_col: str,
127                               transformations: List[str] = None) -> pd
128                               .DataFrame:
129
130        """
131        Compare normality under different transformations.
132
133        Parameters:
134        -----
135        data: DataFrame
136            Data to transform
137        metric_col: str
138            Column to transform
139        transformations: List[str]
140            List of transformations to try
141
142        Returns:
143        -----
144        DataFrame comparing transformation results
145        """
146        if transformations is None:
147            transformations = ['none', 'log', 'sqrt', 'boxcox']
148
149        transformer = DataTransformer()
150        results = []

```



```

148
149     for trans in transformations:
150         if trans == 'none':
151             transformed = data[metric_col]
152         elif trans == 'log':
153             transformed = transformer.log_transform(data[metric_col])
154         elif trans == 'sqrt':
155             transformed = transformer.sqrt_transform(data[metric_col])
156         elif trans == 'boxcox':
157             transformed, _ = transformer.boxcox_transform(data[metric_col])
158         else:
159             continue
160
161     # Assess normality
162     normality = transformer.assess_normality(transformed)
163
164     results.append({
165         'transformation': trans,
166         'skewness': normality['skewness'],
167         'kurtosis': normality['kurtosis'],
168         'shapiro_p': normality['shapiro_p_value'],
169         'ks_p': normality['ks_p_value'],
170         'is_normal': normality['is_normal_ks']
171     })
172
173     return pd.DataFrame(results).sort_values(
174         'shapiro_p' if results[0]['shapiro_p'] else 'ks_p',
175         ascending=False
176     )

```

10.3.5 Quality Assurance Checks

Final QA before analysis:

Listing 10.6: Pre-Analysis Quality Assurance

```

1 def run_pre_analysis_qa(data: pd.DataFrame,
2     variant_col: str = 'variant',
3     metric_cols: List[str] = None,
4     user_col: str = 'user_id',
5     timestamp_col: str = 'timestamp') -> Dict:
6     """
7     Comprehensive pre-analysis quality assurance.
8
9     Returns:
10     -----
11     Dict with QA results and recommendations
12     """
13     qa_results = {
14         'passed': True,
15         'errors': [],
16         'warnings': [],
17         'recommendations': []
18     }

```

```

19
20 # 1. Data validation
21 validator = ABTestDataValidator(
22     data=data,
23     variant_col=variant_col,
24     user_col=user_col,
25     timestamp_col=timestamp_col,
26     metric_cols=metric_cols
27 )
28
29 validation = validator.run_all_checks()
30
31 if not validation['summary']['passed']:
32     qa_results['passed'] = False
33     qa_results['errors'].extend(validation['summary']['issues'])
34
35 qa_results['warnings'].extend(validation['summary']['warnings'])
36
37 # 2. Missing data analysis
38 if metric_cols:
39     missing_handler = MissingDataHandler()
40     missing_analysis = missing_handler.analyze_missingness(
41         data[metric_cols]
42     )
43
44     high_missing = missing_analysis[
45         missing_analysis['pct_missing'] > 5
46     ]
47
48     if len(high_missing) > 0:
49         qa_results['warnings'].append(
50             f"High missing data in {len(high_missing)} metrics"
51         )
52         qa_results['recommendations'].append(
53             "Consider multiple imputation for metrics with >5% missing"
54         )
55
56 # 3. Outlier detection
57 if metric_cols:
58     detector = OutlierDetector()
59     for metric in metric_cols:
60         outliers_iqr = detector.detect_iqr(data[metric])
61         pct_outliers = outliers_iqr.sum() / len(data) * 100
62
63         if pct_outliers > 5:
64             qa_results['warnings'].append(
65                 f"{metric}: {pct_outliers:.1f}% outliers detected"
66             )
67             qa_results['recommendations'].append(
68                 f"Consider winsorization for {metric}"
69             )
70
71 # 4. Normality assessment
72 if metric_cols:
73     transformer = DataTransformer()
74     for metric in metric_cols:
75         normality = transformer.assess_normality(data[metric]).

```

```

dropna())

76
77     if not normality['is_normal_ks']:
78         skew = normality['skewness']
79         qa_results['recommendations'].append(
80             f"{metric}_is_non-normal_(skewness={skew:.2f})."
81             f"Consider_transformation_or_non-parametric_tests."
82         )
83
84 # 5. Sample size check
85 variant_sizes = data[variant_col].value_counts()
86 min_size = variant_sizes.min()
87
88 if min_size < 100:
89     qa_results['warnings'].append(
90         f"Small_sample_size:_minimum_n={min_size}"
91     )
92     qa_results['recommendations'].append(
93         "Small_sample_size_may_limit_statistical_power"
94     )
95
96 return qa_results
97
98 # Generate QA report
99 def generate_qa_report(qa_results: Dict) -> str:
100     """Generate_human-readable_QA_report."""
101     report = []
102     report.append("=" * 60)
103     report.append("PRE-ANALYSIS_QUALITY_ASSURANCE_REPORT")
104     report.append("=" * 60)
105     report.append("")
106
107     # Status
108     status = "PASSED" if qa_results['passed'] else "FAILED"
109     report.append(f"Status:_{status}")
110     report.append("")
111
112     # Errors
113     if qa_results['errors']:
114         report.append("ERRORS_(Must_Fix):")
115         for error in qa_results['errors']:
116             report.append(f"_{error}")
117         report.append("")
118
119     # Warnings
120     if qa_results['warnings']:
121         report.append("WARNINGS:")
122         for warning in qa_results['warnings']:
123             report.append(f"_{warning}")
124         report.append("")
125
126     # Recommendations
127     if qa_results['recommendations']:
128         report.append("RECOMMENDATIONS:")
129         for rec in qa_results['recommendations']:
130             report.append(f"_{rec}")
131         report.append("")
132

```

```
133     return "\n".join(report)
```

10.4 Exploratory Data Analysis

After preprocessing, conduct systematic exploratory analysis to understand data structure, distributions, and relationships.

10.4.1 Descriptive Statistics

Compute comprehensive summary statistics:

Listing 10.7: Descriptive Statistics Framework

```
1 class DescriptiveStats:
2     """Comprehensive descriptive statistics for A/B tests."""
3
4     @staticmethod
5     def summary_statistics(data: pd.DataFrame,
6                           metric_col: str,
7                           variant_col: str = 'variant') -> pd.DataFrame:
8
9         """
10         Calculate comprehensive summary statistics by variant.
11
12         Returns:
13         DataFrame with summary statistics
14         """
15         def compute_stats(series: pd.Series) -> Dict:
16             """Compute statistics for a series."""
17             return {
18                 'n': len(series),
19                 'n_missing': series.isnull().sum(),
20                 'mean': series.mean(),
21                 'median': series.median(),
22                 'std': series.std(),
23                 'min': series.min(),
24                 'q25': series.quantile(0.25),
25                 'q50': series.quantile(0.50),
26                 'q75': series.quantile(0.75),
27                 'max': series.max(),
28                 'iqr': series.quantile(0.75) - series.quantile(0.25),
29                 'skewness': stats.skew(series.dropna()),
30                 'kurtosis': stats.kurtosis(series.dropna()),
31                 'cv': series.std() / series.mean() if series.mean() !=
32                     0 else np.inf
33             }
34
35         # Overall statistics
36         overall = compute_stats(data[metric_col])
37         overall['variant'] = 'overall'
38
39         # By variant
40         variant_stats = []
41         for variant in data[variant_col].unique():
```

```

41         variant_data = data[data[variant_col] == variant][metric_
42             col]
43         stats_dict = compute_stats(variant_data)
44         stats_dict['variant'] = variant
45         variant_stats.append(stats_dict)
46
47     # Combine
48     all_stats = [overall] + variant_stats
49
50     return pd.DataFrame(all_stats).set_index('variant')
51
52 @staticmethod
53 def confidence_intervals(data: pd.DataFrame,
54                         metric_col: str,
55                         variant_col: str = 'variant',
56                         confidence_level: float = 0.95) -> pd.
57                             DataFrame:
58
59     """
60     Calculate confidence intervals for means.
61     Returns:
62     -----
63     DataFrame with confidence intervals
64     """
65     results = []
66
67     alpha = 1 - confidence_level
68
69     for variant in data[variant_col].unique():
70         variant_data = data[data[variant_col] == variant][metric_
71             col].dropna()
72
73         n = len(variant_data)
74         mean = variant_data.mean()
75         sem = stats.sem(variant_data)
76
77         # T-distribution for CI
78         ci = stats.t.interval(
79             confidence_level,
80             df=n-1,
81             loc=mean,
82             scale=sem
83         )
84
85         results.append({
86             'variant': variant,
87             'mean': mean,
88             'sem': sem,
89             'ci_lower': ci[0],
90             'ci_upper': ci[1],
91             'ci_width': ci[1] - ci[0],
92             'n': n
93         })
94
95     return pd.DataFrame(results)
96
97 @staticmethod
98 def proportion_statistics(data: pd.DataFrame,

```

```

96         success_col: str,
97         variant_col: str = 'variant',
98         confidence_level: float = 0.95) -> pd.
          DataFrame:
99
100         """
101         Calculate statistics for binary/proportion metrics.
102
103         Parameters:
104         -----
105         data: DataFrame
106             Experimental data
107         success_col: str
108             Binary column (0/1) indicating success
109         variant_col: str
110             Variant column
111         confidence_level: float
112             CI confidence level
113
114         Returns:
115         -----
116         DataFrame with proportion statistics and CIs
117         """
118         from statsmodels.stats.proportion import proportion_confint
119
120         results = []
121
122         for variant in data[variant_col].unique():
123             variant_data = data[data[variant_col] == variant]
124
125             n = len(variant_data)
126             successes = variant_data[success_col].sum()
127             rate = successes / n
128
129             # Wilson score interval (better for proportions)
130             ci_lower, ci_upper = proportion_confint(
131                 successes,
132                 n,
133                 alpha=1-confidence_level,
134                 method='wilson'
135             )
136
137             # Standard error
138             se = np.sqrt(rate * (1 - rate) / n)
139
140             results.append({
141                 'variant': variant,
142                 'n': n,
143                 'successes': successes,
144                 'rate': rate,
145                 'se': se,
146                 'ci_lower': ci_lower,
147                 'ci_upper': ci_upper,
148                 'ci_width': ci_upper - ci_lower
149             })
150
151         return pd.DataFrame(results)

```

10.4.2 Visualization Techniques

Effective visualizations reveal patterns and communicate findings.

Listing 10.8: Comprehensive EDA Visualizations

```

1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 import plotly.graph_objects as go
4 import plotly.express as px
5 from plotly.subplots import make_subplots
6
7 class EDASVisualizer:
8     """Create comprehensive EDA visualizations."""
9
10    def __init__(self, style: str = 'seaborn-v0_8'):
11        """Initialize visualizer with style."""
12        plt.style.use(style)
13        sns.set_palette("husl")
14
15    def create_dashboard(self,
16                        data: pd.DataFrame,
17                        metric_col: str,
18                        variant_col: str = 'variant',
19                        timestamp_col: str = None) -> plt.Figure:
20        """
21        Create comprehensive EDA dashboard.
22
23        Returns:
24        Matplotlib figure with multiple subplots
25        """
26        fig = plt.figure(figsize=(16, 12))
27        gs = fig.add_gridspec(3, 3, hspace=0.3, wspace=0.3)
28
29        # 1. Distribution comparison (top left)
30        ax1 = fig.add_subplot(gs[0, 0])
31        for variant in data[variant_col].unique():
32            variant_data = data[data[variant_col] == variant][metric_col].dropna()
33            ax1.hist(variant_data, bins=50, alpha=0.6, label=f'{variant}')
34        ax1.set_xlabel(metric_col)
35        ax1.set_ylabel('Frequency')
36        ax1.set_title('Distribution by Variant')
37        ax1.legend()
38        ax1.grid(alpha=0.3)
39
40        # 2. Box plot comparison (top middle)
41        ax2 = fig.add_subplot(gs[0, 1])
42        data.boxplot(column=metric_col, by=variant_col, ax=ax2)
43        ax2.set_xlabel('Variant')
44        ax2.set_ylabel(metric_col)
45        ax2.set_title('Box Plot Comparison')
46        plt.sca(ax2)
47        plt.xticks(rotation=0)
48
49        # 3. Violin plot (top right)
50        ax3 = fig.add_subplot(gs[0, 2])

```

```

52 sns.violinplot(data=data, x=variant_col, y=metric_col, ax=ax3)
53 ax3.set_title('Violin_Plot')
54 ax3.grid(alpha=0.3)
55
56 # 4. Mean with confidence intervals (middle left)
57 ax4 = fig.add_subplot(gs[1, 0])
58 desc_stats = DescriptiveStats()
59 ci_data = desc_stats.confidence_intervals(data, metric_col,
60                                           variant_col)
61
62 x_pos = range(len(ci_data))
63 ax4.errorbar(
64     x_pos,
65     ci_data['mean'],
66     yerr=[
67         ci_data['mean'] - ci_data['ci_lower'],
68         ci_data['ci_upper'] - ci_data['mean']
69     ],
70     fmt='o',
71     markersize=8,
72     capsize=5,
73     capthick=2
74 )
75 ax4.set_xticks(x_pos)
76 ax4.set_xticklabels(ci_data['variant'])
77 ax4.set_ylabel(f'Mean_{metric_col}')
78 ax4.set_title('Means_with_95%_Confidence_Intervals')
79 ax4.grid(alpha=0.3)
80
81 # 5. Q-Q plots (middle middle and right)
82 variants = data[variant_col].unique()
83 for idx, variant in enumerate(variants[:2]):
84     ax = fig.add_subplot(gs[1, idx + 1])
85     variant_data = data[data[variant_col] == variant][metric_col].dropna()
86     stats.probplot(variant_data, dist="norm", plot=ax)
87     ax.set_title(f'Q-Q_Plot:{variant}')
88     ax.grid(alpha=0.3)
89
90 # 6. Time series (bottom row - if timestamp available)
91 if timestamp_col:
92     ax6 = fig.add_subplot(gs[2, :])
93     data[timestamp_col] = pd.to_datetime(data[timestamp_col])
94
95     # Daily aggregation
96     daily_data = data.groupby(
97         [pd.Grouper(key=timestamp_col, freq='D'), variant_col]
98     )[metric_col].agg(['mean', 'sem', 'count']).reset_index()
99
100     for variant in data[variant_col].unique():
101         variant_daily = daily_data[daily_data[variant_col] ==
102                                     variant]
103
104         ax6.plot(
105             variant_daily[timestamp_col],
106             variant_daily['mean'],
107             marker='o',
108             label=f'{variant}',

```



```

107         linewidth=2
108     )
109
110     # Add confidence bands
111     ax6.fill_between(
112         variant_daily[timestamp_col],
113         variant_daily['mean'] - 1.96 * variant_daily['sem',
114         ],
115         variant_daily['mean'] + 1.96 * variant_daily['sem',
116         ],
117         alpha=0.2
118     )
119
120     ax6.set_xlabel('Date')
121     ax6.set_ylabel(f'Daily_Mean_{metric_col}')
122     ax6.set_title('Time_Series_with_95%_Confidence_Bands')
123     ax6.legend()
124     ax6.grid(alpha=0.3)
125     plt.setp(ax6.xaxis.get_majorticklabels(), rotation=45)
126 else:
127     # 7. CDF comparison (bottom left)
128     ax7 = fig.add_subplot(gs[2, 0])
129     for variant in data[variant_col].unique():
130         variant_data = data[
131             data[variant_col] == variant
132         ][metric_col].dropna().sort_values()
133
134         cdf = np.arange(1, len(variant_data) + 1) / len(variant_data)
135         ax7.plot(variant_data, cdf, label=f'{variant}',
136                 linewidth=2)
137
138     ax7.set_xlabel(metric_col)
139     ax7.set_ylabel('Cumulative_Probability')
140     ax7.set_title('Empirical_CDF_Comparison')
141     ax7.legend()
142     ax7.grid(alpha=0.3)
143
144     # 8. Density plot (bottom middle)
145     ax8 = fig.add_subplot(gs[2, 1])
146     for variant in data[variant_col].unique():
147         variant_data = data[data[variant_col] == variant][
148             metric_col].dropna()
149         sns.kdeplot(data=variant_data, label=f'{variant}', ax=
150             ax8, linewidth=2)
151
152     ax8.set_xlabel(metric_col)
153     ax8.set_ylabel('Density')
154     ax8.set_title('Kernel_Density_Estimation')
155     ax8.legend()
156     ax8.grid(alpha=0.3)
157
158     # 9. Summary statistics table (bottom right)
159     ax9 = fig.add_subplot(gs[2, 2])
160     ax9.axis('tight')
161     ax9.axis('off')
162
163     summary = desc_stats.summary_statistics(data, metric_col,
164         variant_col)

```



```

214         row=1, col=1
215     )
216
217     # 2. Box plot
218     for variant in data[variant_col].unique():
219         variant_data = data[data[variant_col] == variant][metric_
220             col].dropna()
221         fig.add_trace(
222             go.Box(
223                 y=variant_data,
224                 name=str(variant),
225                 boxmean='sd',
226             ),
227             row=1, col=2
228         )
229
230     # 3. Time series or CDF
231     if timestamp_col:
232         data[timestamp_col] = pd.to_datetime(data[timestamp_col])
233         daily_data = data.groupby(
234             [pd.Grouper(key=timestamp_col, freq='D'), variant_col]
235         )[metric_col].mean().reset_index()
236
237         for variant in data[variant_col].unique():
238             variant_daily = daily_data[daily_data[variant_col] ==
239                 variant]
240             fig.add_trace(
241                 go.Scatter(
242                     x=variant_daily[timestamp_col],
243                     y=variant_daily[metric_col],
244                     name=str(variant),
245                     mode='lines+markers',
246                 ),
247                 row=2, col=1
248             )
249     else:
250         # CDF
251         for variant in data[variant_col].unique():
252             variant_data = data[
253                 data[variant_col] == variant
254             ][metric_col].dropna().sort_values()
255
256             cdf = np.arange(1, len(variant_data) + 1) / len(variant_
257                 _data)
258             fig.add_trace(
259                 go.Scatter(
260                     x=variant_data,
261                     y=cdf,
262                     name=str(variant),
263                     mode='lines',
264                 ),
265                 row=2, col=1
266             )
267
268     # 4. Mean comparison
269     desc_stats = DescriptiveStats()
270     ci_data = desc_stats.confidence_intervals(data, metric_col,
271         variant_col)

```

```

268     fig.add_trace(
269         go.Bar(
270             x=ci_data['variant'].astype(str),
271             y=ci_data['mean'],
272             error_y=dict(
273                 type='data',
274                 symmetric=False,
275                 array=ci_data['ci_upper'] - ci_data['mean'],
276                 arrayminus=ci_data['mean'] - ci_data['ci_lower']
277             ),
278             name='Mean with 95% CI',
279         ),
280         row=2, col=2
281     )
282 )
283
284 # Update layout
285 fig.update_layout(
286     height=800,
287     showlegend=True,
288     title_text=f"Interactive EDA Dashboard: {metric_col}"
289 )
290
291 fig.update_xaxes(title_text=metric_col, row=1, col=1)
292 fig.update_yaxes(title_text="Frequency", row=1, col=1)
293
294 fig.update_yaxes(title_text=metric_col, row=1, col=2)
295
296 if timestamp_col:
297     fig.update_xaxes(title_text="Date", row=2, col=1)
298     fig.update_yaxes(title_text=f"Mean {metric_col}", row=2,
299                       col=1)
300 else:
301     fig.update_xaxes(title_text=metric_col, row=2, col=1)
302     fig.update_yaxes(title_text="Cumulative Probability", row
303                       =2, col=1)
304
305 fig.update_xaxes(title_text="Variant", row=2, col=2)
306 fig.update_yaxes(title_text=f"Mean {metric_col}", row=2, col=2)
307
308 return fig

```

10.4.3 Distribution Analysis

Understanding the underlying distribution of metrics is critical for selecting appropriate statistical tests and interpreting results.

Listing 10.9: Distribution Analysis Functions

```

1 class DistributionAnalyzer:
2     """Analyze and compare distributions in A/B test data."""
3
4     @staticmethod
5     def compare_distributions(data: pd.DataFrame,
6                             metric_col: str,
7                             variant_col: str = 'variant') -> Dict:
8         """

```

```

9  """Compare distributions between variants using statistical tests.
10
11  """Returns:
12  """-----
13  """Dict with distribution comparison results
14  """
15      from scipy import stats
16
17      variants = data[variant_col].unique()
18      if len(variants) != 2:
19          raise ValueError("This function requires exactly 2 variants")
20
21      control = variants[0]
22      treatment = variants[1]
23
24      control_data = data[data[variant_col] == control][metric_col].dropna()
25      treatment_data = data[data[variant_col] == treatment][metric_col].dropna()
26
27      results = {}
28
29      # Kolmogorov-Smirnov test
30      ks_stat, ks_p = stats.ks_2samp(control_data, treatment_data)
31      results['ks_test'] = {
32          'statistic': ks_stat,
33          'p_value': ks_p,
34          'distributions_differ': ks_p < 0.05
35      }
36
37      # Mann-Whitney U test (for medians)
38      mw_stat, mw_p = stats.mannwhitneyu(
39          control_data, treatment_data, alternative='two-sided'
40      )
41      results['mann_whitney'] = {
42          'statistic': mw_stat,
43          'p_value': mw_p,
44          'medians_differ': mw_p < 0.05
45      }
46
47      # Levene's test (for variance equality)
48      levene_stat, levene_p = stats.levene(control_data, treatment_data)
49      results['levене_test'] = {
50          'statistic': levene_stat,
51          'p_value': levene_p,
52          'variances_differ': levene_p < 0.05,
53          'homoscedastic': levene_p >= 0.05
54      }
55
56      return results
57
58  @staticmethod
59  def plot_distribution_comparison(data: pd.DataFrame,
60                                  metric_col: str,
61                                  variant_col: str = 'variant'):
62      """Create detailed distribution comparison visualization."""

```

```

63 import matplotlib.pyplot as plt
64 import seaborn as sns
65
66 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
67
68 # 1. Overlapping histograms
69 for variant in data[variant_col].unique():
70     variant_data = data[data[variant_col] == variant][metric_
71         col].dropna()
72     axes[0, 0].hist(variant_data, bins=50, alpha=0.6, label=str
73         (variant))
74     axes[0, 0].set_xlabel(metric_col)
75     axes[0, 0].set_ylabel('Frequency')
76     axes[0, 0].set_title('Histogram Comparison')
77     axes[0, 0].legend()
78     axes[0, 0].grid(alpha=0.3)
79
80 # 2. KDE plots
81 for variant in data[variant_col].unique():
82     variant_data = data[data[variant_col] == variant][metric_
83         col].dropna()
84     sns.kdeplot(data=variant_data, label=str(variant),
85         ax=axes[0, 1], linewidth=2)
86     axes[0, 1].set_xlabel(metric_col)
87     axes[0, 1].set_ylabel('Density')
88     axes[0, 1].set_title('Kernel Density Estimation')
89     axes[0, 1].legend()
90     axes[0, 1].grid(alpha=0.3)
91
92 # 3. Empirical CDF
93 for variant in data[variant_col].unique():
94     variant_data = data[
95         data[variant_col] == variant
96     ][metric_col].dropna().sort_values()
97     cdf = np.arange(1, len(variant_data) + 1) / len(variant_
98         data)
99     axes[1, 0].plot(variant_data, cdf, label=str(variant),
100         linewidth=2)
101     axes[1, 0].set_xlabel(metric_col)
102     axes[1, 0].set_ylabel('Cumulative Probability')
103     axes[1, 0].set_title('Empirical CDF')
104     axes[1, 0].legend()
105     axes[1, 0].grid(alpha=0.3)
106
107 # 4. Box and violin plots combined
108 ax_box = axes[1, 1]
109 positions = np.arange(len(data[variant_col].unique()))
110
111 # Violin plots
112 parts = ax_box.violinplot(
113     [data[data[variant_col] == v][metric_col].dropna()
114         for v in data[variant_col].unique()],
115     positions=positions,
116     showmeans=False,
117     showmedians=False
118 )
119
120 # Overlay box plots

```

```

116     ax_box.boxplot(
117         [data[data[variant_col] == v][metric_col].dropna()
118          for v in data[variant_col].unique()],
119         positions=positions,
120         widths=0.15
121     )
122
123     ax_box.set_xticks(positions)
124     ax_box.set_xticklabels(data[variant_col].unique())
125     ax_box.set_ylabel(metric_col)
126     ax_box.set_title('Box+Violin Plot')
127     ax_box.grid(alpha=0.3)
128
129     plt.tight_layout()
130     return fig

```

10.4.4 Trend and Seasonality Detection

Temporal patterns can indicate novelty effects, external events, or seasonal variations.

Listing 10.10: Temporal Pattern Detection

```

1  class TemporalAnalyzer:
2      """Analyze temporal patterns in A/B test data."""
3
4      @staticmethod
5      def detect_trend(data: pd.DataFrame,
6                      metric_col: str,
7                      timestamp_col: str,
8                      variant_col: str = None) -> Dict:
9
10         """
11         Detect linear trend in time series using Mann-Kendall test.
12
13         Returns:
14         Dict with trend detection results
15         """
16         import pymanmkendall as mk
17
18         data = data.copy()
19         data[timestamp_col] = pd.to_datetime(data[timestamp_col])
20         data = data.sort_values(timestamp_col)
21
22         if variant_col:
23             # Test within each variant
24             results = {}
25             for variant in data[variant_col].unique():
26                 variant_data = data[data[variant_col] == variant]
27                 daily_mean = variant_data.groupby(
28                     variant_data[timestamp_col].dt.date
29                 )[metric_col].mean()
30
31                 mk_result = mk.original_test(daily_mean)
32
33                 results[variant] = {
34                     'trend': mk_result.trend,
35                     'p_value': mk_result.p,

```

```

36         'tau': mk_result.Tau,
37         'has_trend': mk_result.trend != 'no_trend'
38     }
39     return results
40 else:
41     # Overall trend
42     daily_mean = data.groupby(
43         data[timestamp_col].dt.date
44     )[metric_col].mean()
45
46     mk_result = mk.original_test(daily_mean)
47
48     return {
49         'trend': mk_result.trend,
50         'p_value': mk_result.p,
51         'tau': mk_result.Tau,
52         'has_trend': mk_result.trend != 'no_trend'
53     }
54
55 @staticmethod
56 def detect_day_of_week_effects(data: pd.DataFrame,
57                               metric_col: str,
58                               timestamp_col: str,
59                               variant_col: str = 'variant') ->
60                               Dict:
61     """
62     Test for day-of-week effects using ANOVA.
63
64     Returns:
65     Dict with day-of-week analysis results
66     """
67     from scipy import stats as sp_stats
68
69     data = data.copy()
70     data[timestamp_col] = pd.to_datetime(data[timestamp_col])
71     data['day_of_week'] = data[timestamp_col].dt.dayofweek
72     data['day_name'] = data[timestamp_col].dt.day_name()
73
74     results = {}
75
76     # Overall effect
77     groups = [data[data['day_of_week'] == d][metric_col].dropna()
78               for d in range(7)]
79     f_stat, p_value = sp_stats.f_oneway(*groups)
80
81     results['overall'] = {
82         'f_statistic': f_stat,
83         'p_value': p_value,
84         'has_dow_effect': p_value < 0.05
85     }
86
87     # By variant
88     variant_results = {}
89     for variant in data[variant_col].unique():
90         variant_data = data[data[variant_col] == variant]
91         groups = [variant_data[variant_data['day_of_week'] == d][

```



```

92         for d in range(7)]
93         f_stat, p_value = sp_stats.f_oneway(*groups)
94
95         variant_results[variant] = {
96             'f_statistic': f_stat,
97             'p_value': p_value,
98             'has_dow_effect': p_value < 0.05
99         }
100
101     results['by_variant'] = variant_results
102
103     # Mean by day of week
104     dow_means = data.groupby(['day_name', variant_col])[metric_col]
105     .mean().unstack()
106     results['means_by_dow'] = dow_means
107
108     return results
109
110 @staticmethod
111 def plot_temporal_patterns(data: pd.DataFrame,
112                           metric_col: str,
113                           timestamp_col: str,
114                           variant_col: str = 'variant'):
115     """Create comprehensive temporal analysis visualizations."""
116     import matplotlib.pyplot as plt
117
118     data = data.copy()
119     data[timestamp_col] = pd.to_datetime(data[timestamp_col])
120     data['date'] = data[timestamp_col].dt.date
121     data['hour'] = data[timestamp_col].dt.hour
122     data['day_of_week'] = data[timestamp_col].dt.dayofweek
123     data['day_name'] = data[timestamp_col].dt.day_name()
124
125     fig, axes = plt.subplots(2, 2, figsize=(16, 10))
126
127     # 1. Daily trend
128     daily_data = data.groupby(['date', variant_col])[metric_col].
129     mean().unstack()
130
131     for variant in daily_data.columns:
132         axes[0, 0].plot(daily_data.index, daily_data[variant],
133                         marker='o', label=str(variant), linewidth=2,
134                         alpha=0.7)
135
136     axes[0, 0].set_xlabel('Date')
137     axes[0, 0].set_ylabel(f'Mean_{metric_col}')
138     axes[0, 0].set_title('Daily Trend')
139     axes[0, 0].legend()
140     axes[0, 0].grid(alpha=0.3)
141     axes[0, 0].tick_params(axis='x', rotation=45)
142
143     # 2. Day of week pattern
144     dow_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday',
145                 'Friday', 'Saturday', 'Sunday']
146     dow_data = data.groupby(['day_name', variant_col])[metric_col].
147     mean().unstack()
148     dow_data = dow_data.reindex(dow_order)

```

```

146     x = np.arange(len(dow_order))
147     width = 0.35
148
149     variants = data[variant_col].unique()
150     for i, variant in enumerate(variants):
151         offset = width * (i - len(variants)/2 + 0.5)
152         axes[0, 1].bar(x + offset, dow_data[variant],
153                        width, label=str(variant), alpha=0.7)
154
155     axes[0, 1].set_xlabel('Day of Week')
156     axes[0, 1].set_ylabel(f'Mean_{metric_col}')
157     axes[0, 1].set_title('Day of Week Pattern')
158     axes[0, 1].set_xticks(x)
159     axes[0, 1].set_xticklabels(dow_order, rotation=45)
160     axes[0, 1].legend()
161     axes[0, 1].grid(alpha=0.3)
162
163     # 3. Hourly pattern (if timestamps include hours)
164     if data['hour'].nunique() > 1:
165         hourly_data = data.groupby(['hour', variant_col])[metric_col].mean().unstack()
166
167         for variant in hourly_data.columns:
168             axes[1, 0].plot(hourly_data.index, hourly_data[variant],
169                             marker='o', label=str(variant), linewidth=2)
170
171             axes[1, 0].set_xlabel('Hour of Day')
172             axes[1, 0].set_ylabel(f'Mean_{metric_col}')
173             axes[1, 0].set_title('Hourly Pattern')
174             axes[1, 0].legend()
175             axes[1, 0].grid(alpha=0.3)
176
177     # 4. Cumulative metric over time
178     cumulative_data = data.sort_values(timestamp_col).groupby(
179         variant_col
180     )[metric_col].cumsum().reset_index()
181
182     for variant in data[variant_col].unique():
183         variant_data = data[data[variant_col] == variant].sort_values(timestamp_col)
184         variant_cumulative = variant_data[metric_col].cumsum()
185
186         axes[1, 1].plot(range(len(variant_cumulative)), variant_cumulative,
187                         label=str(variant), linewidth=2, alpha=0.7)
188
189     axes[1, 1].set_xlabel('Observation Index')
190     axes[1, 1].set_ylabel(f'Cumulative_{metric_col}')
191     axes[1, 1].set_title('Cumulative Metric Over Time')
192     axes[1, 1].legend()
193     axes[1, 1].grid(alpha=0.3)
194
195     plt.tight_layout()
196     return fig

```

10.5 Statistical Analysis Pipeline

After preprocessing and EDA, execute the statistical tests systematically.

10.5.1 Assumption Checking

Before applying parametric tests, verify assumptions.

Listing 10.11: Statistical Assumption Checking

```

1 class AssumptionChecker:
2     """Check statistical assumptions for A/B test analysis."""
3
4     @staticmethod
5     def check_normality(data: pd.DataFrame,
6                         metric_col: str,
7                         variant_col: str = 'variant',
8                         alpha: float = 0.05) -> Dict:
9         """
10         Check normality assumption for each variant.
11
12         Returns:
13         Dict with normality test results
14         """
15         results = {}
16
17         for variant in data[variant_col].unique():
18             variant_data = data[data[variant_col] == variant][metric_col].dropna()
19
20             # Shapiro-Wilk test
21             if len(variant_data) < 5000:
22                 sw_stat, sw_p = stats.shapiro(variant_data)
23             else:
24                 sw_stat, sw_p = None, None
25
26             # Anderson-Darling test
27             ad_result = stats.anderson(variant_data)
28
29             # K-S test
30             ks_stat, ks_p = stats.kstest(
31                 variant_data,
32                 'norm',
33                 args=(variant_data.mean(), variant_data.std())
34             )
35
36             results[variant] = {
37                 'shapiro_wilk': {
38                     'statistic': sw_stat,
39                     'p_value': sw_p,
40                     'is_normal': sw_p > alpha if sw_p else None
41                 },
42                 'kolmogorov_smirnov': {
43                     'statistic': ks_stat,
44                     'p_value': ks_p,
45                     'is_normal': ks_p > alpha
46                 },
47             }

```

```

48         'anderson_darling': {
49             'statistic': ad_result.statistic,
50             'critical_values': ad_result.critical_values.tolist
51             (),
52             'significance_levels': ad_result.significance_level
53             .tolist()
54         },
55         'skewness': stats.skew(variant_data),
56         'kurtosis': stats.kurtosis(variant_data)
57     }
58
59     return results
60
61 @staticmethod
62 def check_variance_equality(data: pd.DataFrame,
63                             metric_col: str,
64                             variant_col: str = 'variant',
65                             alpha: float = 0.05) -> Dict:
66     """
67     Check variance equality assumption (homoscedasticity).
68     Returns:
69     Dict with variance equality test results
70     """
71     variants = data[variant_col].unique()
72     groups = [data[data[variant_col] == v][metric_col].dropna()
73               for v in variants]
74
75     # Levene's test (robust to non-normality)
76     levene_stat, levene_p = stats.levene(*groups)
77
78     # Bartlett's test (assumes normality)
79     bartlett_stat, bartlett_p = stats.bartlett(*groups)
80
81     # Fligner-Killeen test (non-parametric)
82     fligner_stat, fligner_p = stats.fligner(*groups)
83
84     # Variance ratios
85     variances = {v: data[data[variant_col] == v][metric_col].var()
86                  for v in variants}
87
88     return {
89         'levene_test': {
90             'statistic': levene_stat,
91             'p_value': levene_p,
92             'equal_variances': levene_p > alpha
93         },
94         'bartlett_test': {
95             'statistic': bartlett_stat,
96             'p_value': bartlett_p,
97             'equal_variances': bartlett_p > alpha
98         },
99         'fligner_test': {
100             'statistic': fligner_stat,
101             'p_value': fligner_p,
102             'equal_variances': fligner_p > alpha
103         },

```

```

104         'variances_by_variant': variances,
105         'variance_ratio': max(variances.values()) / min(variances.
            values())
106     }
107
108     @staticmethod
109     def check_independence(data: pd.DataFrame,
110                           user_col: str = 'user_id',
111                           variant_col: str = 'variant') -> Dict:
112         """
113         Check independence assumption.
114
115         Returns:
116         -----
117         Dict with independence check results
118         """
119         # Check for users appearing in multiple variants
120         users_per_variant = data.groupby(user_col)[variant_col].nunique
121         ()
122         multi_variant_users = users_per_variant[users_per_variant > 1]
123
124         # Check for temporal autocorrelation (if timestamp available)
125         results = {
126             'multi_variant_users': {
127                 'count': len(multi_variant_users),
128                 'percentage': len(multi_variant_users) / len(users_per_
129                     variant) * 100,
130                 'independent': len(multi_variant_users) == 0
131             }
132         }
133
134         return results
135
136     @staticmethod
137     def generate_assumption_report(data: pd.DataFrame,
138                                   metric_col: str,
139                                   variant_col: str = 'variant',
140                                   user_col: str = 'user_id') -> str:
141         """Generate comprehensive assumption check report."""
142         checker = AssumptionChecker()
143
144         report = []
145         report.append("=" * 60)
146         report.append("STATISTICAL ASSUMPTIONS CHECK")
147         report.append("=" * 60)
148         report.append("")
149
150         # Normality
151         report.append("1. NORMALITY:")
152         normality = checker.check_normality(data, metric_col, variant_
153             col)
154         for variant, results in normality.items():
155             report.append(f"\nVariant: {variant}")
156             if results['shapiro_wilk']['p_value']:
157                 report.append(f"Shapiro-Wilk p-value: {results['
158                     shapiro_wilk']['p_value']:.4f}")
159             report.append(f"K-S p-value: {results['kolmogorov_
160                 smirnov']['p_value']:.4f}")

```

```

156         report.append(f"Skewness:{results['skewness']:.4f}")
157         report.append(f"Kurtosis:{results['kurtosis']:.4f}")
158
159     # Variance equality
160     report.append("\n2. VARIANCE EQUALITY:")
161     var_eq = checker.check_variance_equality(data, metric_col,
162                                              variant_col)
163     report.append(f"Levene's test p-value:{var_eq['levne_test']["p_value"]:.4f}")
164     report.append(f"Equal variances:{var_eq['levne_test']['equal_variances']}")
165     report.append(f"Variance ratio:{var_eq['variance_ratio']:.2f}")
166
167     # Independence
168     report.append("\n3. INDEPENDENCE:")
169     indep = checker.check_independence(data, user_col, variant_col)
170     report.append(f"Multi-variant users:{indep['multi_variant_users']['count']}")
171     report.append(f"Independent:{indep['multi_variant_users']['independent']}")
172
173     report.append("")
174     return "\n".join(report)

```

10.5.2 Test Execution

Execute appropriate statistical tests based on data characteristics.

Listing 10.12: Automated Test Execution

```

1 class ABTestExecutor:
2     """Execute statistical tests for A/B experiments."""
3
4     @staticmethod
5     def t_test(data: pd.DataFrame,
6               metric_col: str,
7               variant_col: str = 'variant',
8               equal_var: bool = None,
9               alpha: float = 0.05) -> Dict:
10
11         """
12         Execute t-test for continuous metrics.
13
14         Parameters:
15         -----
16         data: DataFrame
17             Experimental data
18         metric_col: str
19             Metric to test
20         variant_col: str
21             Variant column
22         equal_var: bool
23             Assume equal variance? If None, will test
24         alpha: float
25             Significance level
26
27         Returns:

```

```

27 #####-----
28 #####Dict with t-test results
29 #####"""
30     variants = data[variant_col].unique()
31     if len(variants) != 2:
32         raise ValueError("T-test requires exactly 2 variants")
33
34     control = variants[0]
35     treatment = variants[1]
36
37     control_data = data[data[variant_col] == control][metric_col].
38         dropna()
39     treatment_data = data[data[variant_col] == treatment][metric_
40         col].dropna()
41
42     # Test for equal variance if not specified
43     if equal_var is None:
44         _, levene_p = stats.levene(control_data, treatment_data)
45         equal_var = levene_p > 0.05
46
47     # Perform t-test
48     t_stat, p_value = stats.ttest_ind(
49         treatment_data,
50         control_data,
51         equal_var=equal_var
52     )
53
54     # Calculate effect size (Cohen's d)
55     pooled_std = np.sqrt(
56         ((len(control_data) - 1) * control_data.var() +
57          (len(treatment_data) - 1) * treatment_data.var()) /
58         (len(control_data) + len(treatment_data) - 2)
59     )
60     cohens_d = (treatment_data.mean() - control_data.mean()) /
61         pooled_std
62
63     # Confidence interval for difference
64     se_diff = pooled_std * np.sqrt(
65         1/len(control_data) + 1/len(treatment_data)
66     )
67     df = len(control_data) + len(treatment_data) - 2
68     t_crit = stats.t.ppf(1 - alpha/2, df)
69
70     diff = treatment_data.mean() - control_data.mean()
71     ci_lower = diff - t_crit * se_diff
72     ci_upper = diff + t_crit * se_diff
73
74     return {
75         'test_type': 't-test',
76         'control_mean': control_data.mean(),
77         'treatment_mean': treatment_data.mean(),
78         'difference': diff,
79         'relative_difference_pct': (diff / control_data.mean() *
80             100)
81         if control_data.mean() != 0 else
82             np.inf,
83         't_statistic': t_stat,
84         'p_value': p_value,

```

```

80         'significant': p_value < alpha,
81         'degrees_of_freedom': df,
82         'equal_variance_assumed': equal_var,
83         'cohens_d': cohens_d,
84         'confidence_interval': {
85             'level': 1 - alpha,
86             'lower': ci_lower,
87             'upper': ci_upper
88         },
89         'sample_sizes': {
90             'control': len(control_data),
91             'treatment': len(treatment_data)
92         }
93     }
94
95     @staticmethod
96     def mann_whitney_test(data: pd.DataFrame,
97                           metric_col: str,
98                           variant_col: str = 'variant',
99                           alpha: float = 0.05) -> Dict:
100         """
101         Execute Mann-Whitney U test (non-parametric).
102
103         Returns:
104         -----
105         Dict with Mann-Whitney test results
106         """
107         variants = data[variant_col].unique()
108         if len(variants) != 2:
109             raise ValueError("Mann-Whitney test requires exactly 2 variants")
110
111         control = variants[0]
112         treatment = variants[1]
113
114         control_data = data[data[variant_col] == control][metric_col].dropna()
115         treatment_data = data[data[variant_col] == treatment][metric_col].dropna()
116
117         # Perform test
118         u_stat, p_value = stats.mannwhitneyu(
119             treatment_data,
120             control_data,
121             alternative='two-sided'
122         )
123
124         # Effect size (rank-biserial correlation)
125         n1 = len(control_data)
126         n2 = len(treatment_data)
127         rank_biserial = 1 - (2*u_stat) / (n1 * n2)
128
129         return {
130             'test_type': 'mann_whitney',
131             'control_median': control_data.median(),
132             'treatment_median': treatment_data.median(),
133             'difference_in_medians': treatment_data.median() - control_data.median(),

```



```

134         'u_statistic': u_stat,
135         'p_value': p_value,
136         'significant': p_value < alpha,
137         'rank_biserial_correlation': rank_biserial,
138         'sample_sizes': {
139             'control': n1,
140             'treatment': n2
141         }
142     }
143
144     @staticmethod
145     def proportion_test(data: pd.DataFrame,
146                        success_col: str,
147                        variant_col: str = 'variant',
148                        alpha: float = 0.05) -> Dict:
149         """
150         Execute proportion_test (z-test for proportions).
151
152         Returns:
153         -----
154         Dict with proportion test results
155         """
156         from statsmodels.stats.proportion import proportions_ztest,
157             proportion_confint
158
159         variants = data[variant_col].unique()
160         if len(variants) != 2:
161             raise ValueError("Proportion test requires exactly 2 variants")
162
163         control = variants[0]
164         treatment = variants[1]
165
166         control_data = data[data[variant_col] == control]
167         treatment_data = data[data[variant_col] == treatment]
168
169         n_control = len(control_data)
170         n_treatment = len(treatment_data)
171
172         successes_control = control_data[success_col].sum()
173         successes_treatment = treatment_data[success_col].sum()
174
175         p_control = successes_control / n_control
176         p_treatment = successes_treatment / n_treatment
177
178         # Z-test
179         counts = np.array([successes_treatment, successes_control])
180         nobs = np.array([n_treatment, n_control])
181
182         z_stat, p_value = proportions_ztest(counts, nobs)
183
184         # Confidence intervals
185         ci_control = proportion_confint(
186             successes_control, n_control, alpha=alpha, method='wilson'
187         )
188         ci_treatment = proportion_confint(
189             successes_treatment, n_treatment, alpha=alpha, method='

```

```

189     )
190
191     # Relative difference (lift)
192     relative_diff = (p_treatment - p_control) / p_control * 100 if
        p_control > 0 else np.inf
193
194     # Absolute difference CI
195     se_diff = np.sqrt(
196         p_control * (1 - p_control) / n_control +
197         p_treatment * (1 - p_treatment) / n_treatment
198     )
199     z_crit = stats.norm.ppf(1 - alpha/2)
200     diff = p_treatment - p_control
201     ci_diff_lower = diff - z_crit * se_diff
202     ci_diff_upper = diff + z_crit * se_diff
203
204     return {
205         'test_type': 'proportion_ztest',
206         'control_rate': p_control,
207         'treatment_rate': p_treatment,
208         'absolute_difference': diff,
209         'relative_difference_pct': relative_diff,
210         'z_statistic': z_stat,
211         'p_value': p_value,
212         'significant': p_value < alpha,
213         'confidence_intervals': {
214             'control': {'lower': ci_control[0], 'upper': ci_control
                [1]},
215             'treatment': {'lower': ci_treatment[0], 'upper': ci_
                treatment[1]},
216             'difference': {'lower': ci_diff_lower, 'upper': ci_diff
                _upper}
217         },
218         'sample_sizes': {
219             'control': n_control,
220             'treatment': n_treatment
221         },
222         'successes': {
223             'control': int(successes_control),
224             'treatment': int(successes_treatment)
225         }
226     }
227
228     @staticmethod
229     def automatic_test_selection(data: pd.DataFrame,
230                                metric_col: str,
231                                variant_col: str = 'variant',
232                                is_binary: bool = None,
233                                alpha: float = 0.05) -> Dict:
234         """
235         Automatically select and execute appropriate test.
236
237         Returns:
238         -----
239         Dict with test results and recommendation
240         """
241         executor = ABTestExecutor()
242         checker = AssumptionChecker()

```

```

243
244     # Determine if binary
245     if is_binary is None:
246         unique_vals = data[metric_col].dropna().unique()
247         is_binary = len(unique_vals) == 2 and set(unique_vals) <=
                {0, 1}
248
249     if is_binary:
250         # Use proportion test
251         result = executor.proportion_test(data, metric_col, variant
                _col, alpha)
252         result['recommendation'] = "Proportion_z-test_(binary_
                outcome)"
253         return result
254
255     # Check normality and variance equality
256     normality = checker.check_normality(data, metric_col, variant_
                col, alpha)
257     var_equality = checker.check_variance_equality(data, metric_col
                , variant_col, alpha)
258
259     # Decision logic
260     all_normal = all(
261         v.get('shapiro_wilk', {}).get('is_normal', False) or
262         v.get('kolmogorov_smirnov', {}).get('is_normal', False)
263         for v in normality.values()
264     )
265
266     equal_var = var_equality['levene_test']['equal_variances']
267
268     if all_normal:
269         # Use t-test
270         result = executor.t_test(data, metric_col, variant_col,
                equal_var, alpha)
271         if equal_var:
272             result['recommendation'] = "Student's_t-test_(equal_
                variances)"
273         else:
274             result['recommendation'] = "Welch's_t-test_(unequal_
                variances)"
275     else:
276         # Use Mann-Whitney
277         result = executor.mann_whitney_test(data, metric_col,
                variant_col, alpha)
278         result['recommendation'] = "Mann-Whitney_U_test_(non-normal
                _distribution)"
279
280     result['assumption_checks'] = {
281         'normality': normality,
282         'variance_equality': var_equality
283     }
284
285     return result

```

10.5.3 Effect Size Calculation

Statistical significance doesn't indicate practical importance. Always calculate and report effect sizes.

Listing 10.13: Effect Size Calculations

```

1 class EffectSizeCalculator:
2     """Calculate various effect size measures for A/B tests."""
3
4     @staticmethod
5     def cohens_d(data: pd.DataFrame,
6                 metric_col: str,
7                 variant_col: str = 'variant',
8                 pooled: bool = True) -> Dict:
9         """
10        Calculate Cohen's d for continuous metrics.
11
12        Parameters:
13        -----
14        data: DataFrame
15        Experimental data
16        metric_col: str
17        Metric to analyze
18        variant_col: str
19        Variant column
20        pooled: bool
21        Use pooled standard deviation
22
23        Returns:
24        -----
25        Dict with Cohen's d and interpretation
26        """
27        variants = data[variant_col].unique()
28        if len(variants) != 2:
29            raise ValueError("Cohen's d requires exactly 2 variants")
30
31        control = variants[0]
32        treatment = variants[1]
33
34        control_data = data[data[variant_col] == control][metric_col].dropna()
35        treatment_data = data[data[variant_col] == treatment][metric_col].dropna()
36
37        mean_diff = treatment_data.mean() - control_data.mean()
38
39        if pooled:
40            # Pooled standard deviation
41            n1, n2 = len(control_data), len(treatment_data)
42            var1, var2 = control_data.var(), treatment_data.var()
43            pooled_std = np.sqrt(((n1 - 1) * var1 + (n2 - 1) * var2) /
44                                (n1 + n2 - 2))
45            d = mean_diff / pooled_std
46        else:
47            # Control group standard deviation
48            d = mean_diff / control_data.std()
49
50        # Interpretation

```

```

50     if abs(d) < 0.2:
51         interpretation = "negligible"
52     elif abs(d) < 0.5:
53         interpretation = "small"
54     elif abs(d) < 0.8:
55         interpretation = "medium"
56     else:
57         interpretation = "large"
58
59     return {
60         'cohens_d': d,
61         'interpretation': interpretation,
62         'mean_difference': mean_diff,
63         'pooled_std': pooled_std if pooled else None
64     }
65
66     @staticmethod
67     def relative_effect(data: pd.DataFrame,
68                        metric_col: str,
69                        variant_col: str = 'variant') -> Dict:
70         """
71         Calculate relative (percentage) effect.
72
73         Returns:
74         -----
75         Dict with relative effect measures
76         """
77         variants = data[variant_col].unique()
78         if len(variants) != 2:
79             raise ValueError("Requires exactly 2 variants")
80
81         control = variants[0]
82         treatment = variants[1]
83
84         control_mean = data[data[variant_col] == control][metric_col].
            mean()
85         treatment_mean = data[data[variant_col] == treatment][metric_
            col].mean()
86
87         absolute_diff = treatment_mean - control_mean
88         relative_diff_pct = (absolute_diff / control_mean * 100) if
            control_mean != 0 else np.inf
89
90         return {
91             'control_mean': control_mean,
92             'treatment_mean': treatment_mean,
93             'absolute_difference': absolute_diff,
94             'relative_difference_pct': relative_diff_pct,
95             'lift_pct': relative_diff_pct
96         }
97
98     @staticmethod
99     def odds_ratio(data: pd.DataFrame,
100                   success_col: str,
101                   variant_col: str = 'variant') -> Dict:
102         """
103         Calculate odds ratio for binary outcomes.
104

```

```

105 """Returns:
106 """
107 """Dict with odds_ratio and confidence_interval
108 """
109 from statsmodels.stats.contingency_tables import Table2x2
110
111 variants = data[variant_col].unique()
112 if len(variants) != 2:
113     raise ValueError("Requires exactly 2 variants")
114
115 control = variants[0]
116 treatment = variants[1]
117
118 # Create contingency table
119 control_data = data[data[variant_col] == control]
120 treatment_data = data[data[variant_col] == treatment]
121
122 # [treatment_success, treatment_failure]
123 # [control_success, control_failure]
124 table = np.array([
125     treatment_data[success_col].sum(),
126     len(treatment_data) - treatment_data[success_col].sum(),
127     control_data[success_col].sum(),
128     len(control_data) - control_data[success_col].sum()
129 ])
130
131 table2x2 = Table2x2(table)
132 or_value = table2x2.oddsratio
133 ci = table2x2.oddsratio_confint()
134
135 # Interpretation
136 if or_value > 1:
137     interpretation = f"Treatment has {or_value:.2f}x odds of success"
138 elif or_value < 1:
139     interpretation = f"Control has {1/or_value:.2f}x odds of success"
140 else:
141     interpretation = "Equal odds"
142
143 return {
144     'odds_ratio': or_value,
145     'confidence_interval': {'lower': ci[0], 'upper': ci[1]},
146     'log_odds_ratio': np.log(or_value),
147     'interpretation': interpretation
148 }
149
150 @staticmethod
151 def risk_ratio(data: pd.DataFrame,
152               success_col: str,
153               variant_col: str = 'variant') -> Dict:
154     """
155     Calculate risk_ratio (relative risk) for binary outcomes.
156
157     Returns:
158     """
159     """Dict with risk_ratio
160     """

```

```

161     variants = data[variant_col].unique()
162     if len(variants) != 2:
163         raise ValueError("Requires exactly 2 variants")
164
165     control = variants[0]
166     treatment = variants[1]
167
168     control_data = data[data[variant_col] == control]
169     treatment_data = data[data[variant_col] == treatment]
170
171     p_control = control_data[success_col].mean()
172     p_treatment = treatment_data[success_col].mean()
173
174     rr = p_treatment / p_control if p_control > 0 else np.inf
175
176     # Confidence interval (log method)
177     se_log_rr = np.sqrt(
178         (1 - p_control) / (control_data[success_col].sum()) +
179         (1 - p_treatment) / (treatment_data[success_col].sum())
180     )
181
182     ci_lower = np.exp(np.log(rr) - 1.96 * se_log_rr)
183     ci_upper = np.exp(np.log(rr) + 1.96 * se_log_rr)
184
185     return {
186         'risk_ratio': rr,
187         'confidence_interval': {'lower': ci_lower, 'upper': ci_upper},
188         'control_risk': p_control,
189         'treatment_risk': p_treatment
190     }

```

10.5.4 Confidence Interval Estimation

Confidence intervals quantify uncertainty and are more informative than p-values alone.

Listing 10.14: Advanced Confidence Interval Methods

```

1 class ConfidenceIntervalEstimator:
2     """Advanced confidence interval estimation methods."""
3
4     @staticmethod
5     def bootstrap_ci(data: pd.DataFrame,
6                     metric_col: str,
7                     variant_col: str = 'variant',
8                     statistic: str = 'mean',
9                     n_bootstrap: int = 10000,
10                    confidence_level: float = 0.95) -> Dict:
11         """
12         Calculate bootstrap confidence intervals.
13
14         Parameters:
15         -----
16         data: DataFrame
17             Experimental data
18         metric_col: str
19             Metric to analyze

```

```

20     variant_col: str
21     Variant column
22     statistic: str
23     Statistic to compute: 'mean', 'median', 'std'
24     n_bootstrap: int
25     Number of bootstrap samples
26     confidence_level: float
27     Confidence level
28
29     Returns:
30     -----
31     Dict with bootstrap CI
32     """
33     from scipy.stats import bootstrap
34
35     variants = data[variant_col].unique()
36     if len(variants) != 2:
37         raise ValueError("Requires exactly 2 variants")
38
39     control = variants[0]
40     treatment = variants[1]
41
42     control_data = data[data[variant_col] == control][metric_col].
43         dropna().values
44     treatment_data = data[data[variant_col] == treatment][metric_
45         col].dropna().values
46
47     # Define statistic function
48     if statistic == 'mean':
49         stat_func = np.mean
50     elif statistic == 'median':
51         stat_func = np.median
52     elif statistic == 'std':
53         stat_func = np.std
54     else:
55         raise ValueError(f"Unknown statistic: {statistic}")
56
57     # Bootstrap for difference
58     def difference_stat(x, y):
59         return stat_func(y) - stat_func(x)
60
61     bootstrap_diffs = []
62     rng = np.random.RandomState(42)
63
64     for _ in range(n_bootstrap):
65         control_sample = rng.choice(control_data, size=len(control_
66             data), replace=True)
67         treatment_sample = rng.choice(treatment_data, size=len(
68             treatment_data), replace=True)
69         bootstrap_diffs.append(stat_func(treatment_sample) - stat_
70             func(control_sample))
71
72     bootstrap_diffs = np.array(bootstrap_diffs)
73
74     # Percentile method
75     alpha = 1 - confidence_level
76     ci_lower = np.percentile(bootstrap_diffs, alpha/2 * 100)
77     ci_upper = np.percentile(bootstrap_diffs, (1 - alpha/2) * 100)

```



```

73
74     # Observed difference
75     obs_diff = stat_func(treatment_data) - stat_func(control_data)
76
77     return {
78         'statistic': statistic,
79         'observed_difference': obs_diff,
80         'confidence_interval': {
81             'method': 'percentile_bootstrap',
82             'level': confidence_level,
83             'lower': ci_lower,
84             'upper': ci_upper
85         },
86         'n_bootstrap': n_bootstrap,
87         'bootstrap_distribution': bootstrap_diffs
88     }
89
90     @staticmethod
91     def delta_method_ci(data: pd.DataFrame,
92                         metric_col: str,
93                         variant_col: str = 'variant',
94                         transformation: str = 'log',
95                         confidence_level: float = 0.95) -> Dict:
96
97         """
98         Calculate CI using delta method for transformed statistics.
99
100        Parameters:
101        -----
102        transformation: str
103        'log', 'logit', or 'sqrt'
104
105        Returns:
106        -----
107        Dict with delta method CI
108        """
109        variants = data[variant_col].unique()
110        if len(variants) != 2:
111            raise ValueError("Requires exactly 2 variants")
112
113        control = variants[0]
114        treatment = variants[1]
115
116        control_data = data[data[variant_col] == control][metric_col].dropna()
117        treatment_data = data[data[variant_col] == treatment][metric_col].dropna()
118
119        # Calculate means and variances
120        mean_c = control_data.mean()
121        mean_t = treatment_data.mean()
122        var_c = control_data.var() / len(control_data)
123        var_t = treatment_data.var() / len(treatment_data)
124
125        alpha = 1 - confidence_level
126        z_crit = stats.norm.ppf(1 - alpha/2)
127
128        if transformation == 'log':
129            # Log transformation

```

```

129     ratio = mean_t / mean_c
130     log_ratio = np.log(ratio)
131
132     # Delta method variance
133     var_log = var_t / (mean_t ** 2) + var_c / (mean_c ** 2)
134     se_log = np.sqrt(var_log)
135
136     ci_log_lower = log_ratio - z_crit * se_log
137     ci_log_upper = log_ratio + z_crit * se_log
138
139     # Back-transform
140     ci_lower = np.exp(ci_log_lower)
141     ci_upper = np.exp(ci_log_upper)
142
143     return {
144         'transformation': 'log',
145         'ratio': ratio,
146         'log_ratio': log_ratio,
147         'confidence_interval': {
148             'level': confidence_level,
149             'lower': ci_lower,
150             'upper': ci_upper
151         }
152     }
153
154 elif transformation == 'logit':
155     # Logit transformation (for proportions)
156     # p must be in (0, 1)
157     def logit(p):
158         return np.log(p / (1 - p))
159
160     def inv_logit(x):
161         return 1 / (1 + np.exp(-x))
162
163     logit_t = logit(mean_t)
164     logit_c = logit(mean_c)
165     diff_logit = logit_t - logit_c
166
167     # Delta method variance
168     var_logit = var_t / (mean_t ** 2 * (1 - mean_t) ** 2) + \
169         var_c / (mean_c ** 2 * (1 - mean_c) ** 2)
170     se_logit = np.sqrt(var_logit)
171
172     ci_logit_lower = diff_logit - z_crit * se_logit
173     ci_logit_upper = diff_logit + z_crit * se_logit
174
175     # Back-transform
176     ci_lower = inv_logit(ci_logit_lower)
177     ci_upper = inv_logit(ci_logit_upper)
178
179     return {
180         'transformation': 'logit',
181         'difference_logit': diff_logit,
182         'confidence_interval': {
183             'level': confidence_level,
184             'lower': ci_lower,
185             'upper': ci_upper
186         }
187     }

```

```

187         }
188
189     else:
190         raise ValueError(f"Unknown transformation: {transformation}")

```

10.6 Result Interpretation

Statistical results must be translated into business insights and actionable recommendations.

10.6.1 Statistical vs. Practical Significance

Statistical significance indicates whether an effect is likely real (not due to chance).

Practical significance indicates whether the effect is large enough to matter for business decisions.

Example 10.1 (Statistical vs. Practical Significance). An A/B test with 1 million users finds that variant B increases conversion rate from 10.0% to 10.05% ($p < 0.001$).

Statistical significance: Yes ($p < 0.001$)

Practical significance: Depends on context

- If incremental implementation cost is \$10,000 and each conversion worth \$50, then:
Additional revenue = $1,000,000 \times 0.0005 \times \$50 = \$25,000$ per million users
- $\text{ROI} = (\$25,000 - \$10,000) / \$10,000 = 150\% \rightarrow$ **Practically significant**

Decision: Ship despite small effect size due to favorable cost-benefit.

Listing 10.15: Business Impact Assessment

```

1 class BusinessImpactAnalyzer:
2     """Assess business impact of A/B test results."""
3
4     @staticmethod
5     def calculate_business_impact(test_results: Dict,
6                                   annual_users: int,
7                                   value_per_conversion: float,
8                                   implementation_cost: float) -> Dict:
9
10         """
11         Calculate business impact metrics.
12
13         Parameters:
14         -----
15         test_results: Dict
16             Results from statistical test (must include difference and CI)
17
18         annual_users: int
19             Expected annual users
20
21         value_per_conversion: float
22             Dollar value per conversion / success
23
24         implementation_cost: float
25             One-time implementation cost

```

```

23 #####Returns:
24 #####-----
25 #####Dict with business impact assessment
26 #####"""
27     # Extract effect size
28     if 'absolute_difference' in test_results:
29         effect = test_results['absolute_difference']
30     elif 'difference' in test_results:
31         effect = test_results['difference']
32     else:
33         raise ValueError("Test results must include effect size")
34
35     ci = test_results.get('confidence_interval', {})
36     ci_lower = ci.get('lower', None)
37     ci_upper = ci.get('upper', None)
38
39     # Annual impact
40     annual_additional_conversions = annual_users * effect
41     annual_revenue_impact = annual_additional_conversions * value_per_conversion
42
43     # ROI calculation
44     roi = (annual_revenue_impact - implementation_cost) /
45           implementation_cost * 100 if implementation_cost > 0 else np.inf
46
47     # Conservative estimate (lower CI bound)
48     if ci_lower is not None:
49         conservative_conversions = annual_users * ci_lower
50         conservative_revenue = conservative_conversions * value_per_conversion
51         conservative_roi = (conservative_revenue - implementation_cost) /
52                           implementation_cost * 100 if implementation_cost > 0
53                           else np.inf
54     else:
55         conservative_revenue = None
56         conservative_roi = None
57
58     # Optimistic estimate (upper CI bound)
59     if ci_upper is not None:
60         optimistic_conversions = annual_users * ci_upper
61         optimistic_revenue = optimistic_conversions * value_per_conversion
62         optimistic_roi = (optimistic_revenue - implementation_cost) /
63                           implementation_cost * 100 if implementation_cost > 0
64                           else np.inf
65     else:
66         optimistic_revenue = None
67         optimistic_roi = None
68
69     # Practical significance assessment
70     min_meaningful_effect = implementation_cost / (annual_users *
71                                                    value_per_conversion)
72     is_practically_significant = abs(effect) > min_meaningful_effect
73
74     return {
75         'effect_size': effect,

```

```

70         'annual_additional_conversions': annual_additional_
71             conversions,
72         'expected_annual_revenue_impact': annual_revenue_impact,
73         'implementation_cost': implementation_cost,
74         'expected_roi_pct': roi,
75         'conservative_estimate': {
76             'annual_revenue_impact': conservative_revenue,
77             'roi_pct': conservative_roi
78         },
79         'optimistic_estimate': {
80             'annual_revenue_impact': optimistic_revenue,
81             'roi_pct': optimistic_roi
82         },
83         'min_meaningful_effect': min_meaningful_effect,
84         'is_practically_significant': is_practically_significant,
85         'is_statistically_significant': test_results.get('
            significant', False)
86     }
87
88     @staticmethod
89     def create_impact_summary(impact_results: Dict) -> str:
90         """Generate human-readable impact summary."""
91         summary = []
92         summary.append("=" * 60)
93         summary.append("BUSINESS IMPACT SUMMARY")
94         summary.append("=" * 60)
95         summary.append("")
96         summary.append(f"Effect Size: {impact_results['effect_size']:.4f}")
97         summary.append(f"Annual Additional Conversions: {impact_results['annual_additional_conversions']:.0f}")
98         summary.append("")
99
100        summary.append(f"Expected Annual Revenue Impact: ${impact_results['expected_annual_revenue_impact']:.2f}")
101        summary.append(f"Implementation Cost: ${impact_results['implementation_cost']:.2f}")
102        summary.append(f"Expected ROI: {impact_results['expected_roi_pct']:.1f}%")
103        summary.append("")
104
105        if impact_results['conservative_estimate']['annual_revenue_impact']:
106            summary.append(f"Conservative Estimate (95% CI lower bound):")
107            summary.append(f"Revenue Impact: ${impact_results['conservative_estimate']['annual_revenue_impact']:.2f}")
108            summary.append(f"ROI: {impact_results['conservative_estimate']['roi_pct']:.1f}%")
109            summary.append("")
110
111        summary.append(f"Statistical Significance: {'Yes' if impact_results['is_statistically_significant'] else 'No'}")
112        summary.append(f"Practical Significance: {'Yes' if impact_results['is_practically_significant'] else 'No'}")
113        summary.append("")
114

```

```

115     # Decision recommendation
116     if impact_results['is_statistically_significant'] and impact_
        results['is_practically_significant']:
117         recommendation = "RECOMMEND: Ship-Both statistically and
            practically significant"
118     elif impact_results['is_statistically_significant'] and not
        impact_results['is_practically_significant']:
119         recommendation = "CONSIDER: Small effect- Evaluate
            strategic value beyond ROI"
120     elif not impact_results['is_statistically_significant'] and
        impact_results['is_practically_significant']:
121         recommendation = "INCONCLUSIVE: Run longer test for more
            power"
122     else:
123         recommendation = "DO NOT SHIP: Not significant"
124
125     summary.append(f"Recommendation: {recommendation}")
126     summary.append("")
127
128     return "\n".join(summary)

```

10.6.2 Segment Analysis

Treatment effects may vary across user segments. Segmented analysis reveals heterogeneous treatment effects (HTEs).

Listing 10.16: Segment Analysis Framework

```

1 class SegmentAnalyzer:
2     """Analyze treatment effects across segments."""
3
4     @staticmethod
5     def segment_analysis(data: pd.DataFrame,
6                          metric_col: str,
7                          segment_col: str,
8                          variant_col: str = 'variant',
9                          test_type: str = 'auto') -> Dict:
10
11         """
12         Analyze treatment effect within each segment.
13
14         Parameters:
15         -----
16         data: DataFrame
17             Experimental data
18         metric_col: str
19             Metric to analyze
20         segment_col: str
21             Column defining segments
22         variant_col: str
23             Variant column
24         test_type: str
25             'auto', 't_test', 'mann_whitney', or 'proportion'
26
27         Returns:
28         -----
29         Dict with segment-specific results
30         """

```

```

30     executor = ABTestExecutor()
31     segment_results = {}
32
33     for segment in data[segment_col].unique():
34         segment_data = data[data[segment_col] == segment]
35
36         # Run appropriate test
37         if test_type == 'auto':
38             result = executor.automatic_test_selection(
39                 segment_data, metric_col, variant_col
40             )
41         elif test_type == 't_test':
42             result = executor.t_test(segment_data, metric_col,
43                                     variant_col)
44         elif test_type == 'mann_whitney':
45             result = executor.mann_whitney_test(segment_data,
46                                                 metric_col, variant_col)
47         elif test_type == 'proportion':
48             result = executor.proportion_test(segment_data, metric_
49                                               col, variant_col)
50         else:
51             raise ValueError(f"Unknown test_type: {test_type}")
52
53         segment_results[segment] = result
54
55     return segment_results
56
57 @staticmethod
58 def test_for_interaction(data: pd.DataFrame,
59                         metric_col: str,
60                         segment_col: str,
61                         variant_col: str = 'variant') -> Dict:
62
63     """
64     Test if treatment effect differs significantly across segments.
65     Uses two-way ANOVA or similar to test for interaction.
66     Returns:
67     Dict with interaction test results
68     """
69     from scipy import stats as sp_stats
70     import statsmodels.formula.api as smf
71
72     # Create interaction term
73     data_test = data.copy()
74     data_test['interaction'] = (
75         data_test[variant_col].astype(str) + '_' +
76         data_test[segment_col].astype(str)
77     )
78
79     # Two-way ANOVA
80     formula = f'{metric_col} ~ C({variant_col}) + C({segment_col}) + C({variant_col}):C({segment_col})'
81
82     try:
83         model = smf.ols(formula, data=data_test).fit()
84         anova_table = sm.stats.anova_lm(model, typ=2)

```

```

84
85     interaction_p = anova_table.loc[
86         f'C({variant_col}):C({segment_col})', 'PR(>F)'
87     ]
88
89     return {
90         'has_interaction': interaction_p < 0.05,
91         'interaction_p_value': interaction_p,
92         'anova_table': anova_table,
93         'interpretation': (
94             "Treatment_effect_varies_significantly_across_
95                 segments"
96             if interaction_p < 0.05
97             else "No_significant_interaction_detected"
98         )
99     except Exception as e:
100         return {
101             'error': str(e),
102             'message': "Could_not_perform_interaction_test"
103         }
104
105 @staticmethod
106 def visualize_segments(data: pd.DataFrame,
107                       metric_col: str,
108                       segment_col: str,
109                       variant_col: str = 'variant'):
110     """Create segment analysis visualization."""
111     import matplotlib.pyplot as plt
112     import seaborn as sns
113
114     segments = data[segment_col].unique()
115     n_segments = len(segments)
116
117     fig, axes = plt.subplots(1, min(n_segments, 4), figsize=(16, 4))
118
119     if n_segments == 1:
120         axes = [axes]
121
122     for idx, segment in enumerate(segments[:4]):
123         segment_data = data[data[segment_col] == segment]
124
125         # Create violin + box plot
126         sns.violinplot(
127             data=segment_data,
128             x=variant_col,
129             y=metric_col,
130             ax=axes[idx],
131             alpha=0.5
132         )
133
134         # Calculate means and plot
135         segment_stats = DescriptiveStats()
136         ci_data = segment_stats.confidence_intervals(
137             segment_data, metric_col, variant_col
138         )
139
140         for i, row in ci_data.iterrows():

```



```

140         axes[idx].plot(
141             [i, i],
142             [row['ci_lower'], row['ci_upper']],
143             'k-',
144             linewidth=2
145         )
146         axes[idx].plot(i, row['mean'], 'ro', markersize=8)
147
148         axes[idx].set_title(f'Segment: {segment}')
149         axes[idx].set_ylabel(metric_col)
150         axes[idx].grid(alpha=0.3)
151
152     plt.tight_layout()
153     return fig
154
155     @staticmethod
156     def create_segment_report(segment_results: Dict) -> pd.DataFrame:
157         """Create summary table of segment results."""
158         rows = []
159
160         for segment, result in segment_results.items():
161             if 'control_mean' in result:
162                 control_val = result['control_mean']
163                 treatment_val = result['treatment_mean']
164             elif 'control_rate' in result:
165                 control_val = result['control_rate']
166                 treatment_val = result['treatment_rate']
167             elif 'control_median' in result:
168                 control_val = result['control_median']
169                 treatment_val = result['treatment_median']
170             else:
171                 continue
172
173             rows.append({
174                 'segment': segment,
175                 'control': control_val,
176                 'treatment': treatment_val,
177                 'difference': result.get('difference', result.get('
178                     absolute_difference', treatment_val - control_val)),
179                 'p_value': result.get('p_value', np.nan),
180                 'significant': result.get('significant', False)
181             })
182
183         return pd.DataFrame(rows)

```

10.6.3 Long-term Effects Consideration

Short-term test results may not reflect long-term impact.

- **Novelty effects:** Users may engage more/less with new features initially
- **Learning effects:** Behavior changes as users become familiar
- **Network effects:** Impact may compound as more users adopt
- **Cannibalization:** Short-term gains may reduce long-term metrics

Strategies for assessing long-term effects:

1. Run experiments for multiple weeks/months
2. Analyze cohorts based on exposure duration
3. Monitor post-experiment metrics
4. Use holdout groups for long-term tracking

10.7 Reporting and Communication

Effective communication ensures stakeholders understand results and can make informed decisions.

10.7.1 Stakeholder Communication Strategies

Different audiences require different levels of detail:

Audience	Focus	Format
Executives	Business impact, ROI	1-page summary
Product managers	Metrics, segments, recommendations	Dashboard + slides
Engineers	Implementation details, edge cases	Technical doc
Data scientists	Methods, assumptions, limitations	Full analysis

10.7.2 Visualization Best Practices

Listing 10.17: Professional Result Visualization

```

1 class ResultVisualizer:
2     """Create professional visualizations for reporting."""
3
4     @staticmethod
5     def create_executive_summary_viz(test_results: Dict,
6                                     metric_name: str):
7         """Create executive summary visualization."""
8         import matplotlib.pyplot as plt
9
10        fig, axes = plt.subplots(1, 2, figsize=(14, 5))
11
12        # 1. Treatment effect with CI
13        if 'control_mean' in test_results:
14            control_val = test_results['control_mean']
15            treatment_val = test_results['treatment_mean']
16            value_type = 'Mean'
17        elif 'control_rate' in test_results:
18            control_val = test_results['control_rate']
19            treatment_val = test_results['treatment_rate']
20            value_type = 'Rate'
21        else:
22            raise ValueError("Unknown result format")
23
24        ci = test_results['confidence_interval']

```

```

25
26     # Bar chart with error bars
27     variants = ['Control', 'Treatment']
28     values = [control_val, treatment_val]
29
30     bars = axes[0].bar(variants, values, color=['#3498db', '#e74c3c'], alpha=0.7)
31
32     # Add CI for treatment
33     diff = test_results.get('difference', test_results.get('absolute_difference'))
34     axes[0].errorbar(
35         [1],
36         [treatment_val],
37         yerr=[[treatment_val - ci['lower']], [ci['upper'] - treatment_val]],
38         fmt='none',
39         color='black',
40         capsize=10,
41         linewidth=2
42     )
43
44     # Annotations
45     for i, (bar, val) in enumerate(zip(bars, values)):
46         height = bar.get_height()
47         axes[0].text(
48             bar.get_x() + bar.get_width()/2.,
49             height,
50             f'{val:.4f}',
51             ha='center',
52             va='bottom',
53             fontsize=12,
54             fontweight='bold'
55         )
56
57     axes[0].set_ylabel(f'{value_type}_{metric_name}', fontsize=12)
58     axes[0].set_title(f'{metric_name} Comparison', fontsize=14, fontweight='bold')
59     axes[0].grid(axis='y', alpha=0.3)
60
61     # 2. Effect size visualization
62     rel_diff = test_results.get('relative_difference_pct', 0)
63
64     color = '#27ae60' if rel_diff > 0 else '#e74c3c'
65
66     axes[1].barh(['Relative\nLift'], [rel_diff], color=color, alpha=0.7)
67     axes[1].axvline(0, color='black', linestyle='--', linewidth=1)
68
69     axes[1].text(
70         rel_diff,
71         0,
72         f'{rel_diff:+.2f}%',
73         ha='left' if rel_diff > 0 else 'right',
74         va='center',
75         fontsize=14,
76         fontweight='bold'
77     )

```

```

78     axes[1].set_xlabel('Percentage Change', fontsize=12)
79     axes[1].set_title('Treatment Effect Size', fontsize=14,
80                       fontweight='bold')
81     axes[1].grid(axis='x', alpha=0.3)
82
83     # Overall title
84     sig_status = "Significant" if test_results.get('significant')
85                       else "Not Significant"
86     p_val = test_results.get('p_value', np.nan)
87
88     fig.suptitle(
89         f'A/B Test Results: {metric_name}\n{sig_status} (p={p_val
90           :.4f})',
91         fontsize=16,
92         fontweight='bold'
93     )
94
95     plt.tight_layout()
96     return fig
97
98 @staticmethod
99 def create_comprehensive_report_viz(data: pd.DataFrame,
100                                   metric_col: str,
101                                   test_results: Dict,
102                                   variant_col: str = 'variant'):
103     """Create comprehensive report with multiple visualizations."""
104     import matplotlib.pyplot as plt
105     import seaborn as sns
106
107     fig = plt.figure(figsize=(16, 12))
108     gs = fig.add_gridspec(3, 2, hspace=0.3, wspace=0.3)
109
110     # 1. Distribution comparison
111     ax1 = fig.add_subplot(gs[0, :])
112     for variant in data[variant_col].unique():
113         variant_data = data[data[variant_col] == variant][metric_col].dropna()
114         sns.kdeplot(data=variant_data, label=str(variant), ax=ax1,
115                     linewidth=2)
116     ax1.set_xlabel(metric_col, fontsize=11)
117     ax1.set_ylabel('Density', fontsize=11)
118     ax1.set_title('Distribution Comparison', fontsize=13,
119                   fontweight='bold')
120     ax1.legend()
121     ax1.grid(alpha=0.3)
122
123     # 2. Effect size with CI
124     ax2 = fig.add_subplot(gs[1, 0])
125
126     diff = test_results.get('difference', test_results.get('
127       absolute_difference', 0))
128     ci = test_results.get('confidence_interval', {})
129
130     ax2.plot([0], [diff], 'o', markersize=12, color='#e74c3c')
131     if ci:
132         ax2.plot(
133             [0, 0],

```

```

129         [ci.get('lower', diff), ci.get('upper', diff)],
130         linewidth=3,
131         color='#3498db'
132     )
133
134     ax2.axhline(0, color='gray', linestyle='--', linewidth=1)
135     ax2.set_xlim(-0.5, 0.5)
136     ax2.set_ylabel(f'Treatment_Effect_on_{metric_col}', fontsize
137                   =11)
138     ax2.set_title('Effect_Size_with_95%_CI', fontsize=13,
139                  fontweight='bold')
140     ax2.set_xticks([])
141     ax2.grid(axis='y', alpha=0.3)
142
143     # 3. Sample sizes
144     ax3 = fig.add_subplot(gs[1, 1])
145
146     sample_sizes = test_results.get('sample_sizes', {})
147     if sample_sizes:
148         variants_list = list(sample_sizes.keys())
149         sizes = list(sample_sizes.values())
150
151         bars = ax3.bar(variants_list, sizes, color=['#3498db', '#
152                    e74c3c'], alpha=0.7)
153
154         for bar, size in zip(bars, sizes):
155             height = bar.get_height()
156             ax3.text(
157                 bar.get_x() + bar.get_width()/2.,
158                 height,
159                 f'n={size:,}',
160                 ha='center',
161                 va='bottom',
162                 fontsize=10
163             )
164
165     ax3.set_ylabel('Sample_Size', fontsize=11)
166     ax3.set_title('Sample_Sizes_by_Variant', fontsize=13,
167                  fontweight='bold')
168     ax3.grid(axis='y', alpha=0.3)
169
170     # 4. Statistical summary table
171     ax4 = fig.add_subplot(gs[2, :])
172     ax4.axis('tight')
173     ax4.axis('off')
174
175     summary_data = [
176         ['Metric', metric_col],
177         ['Test_Type', test_results.get('test_type', 'N/A')],
178         ['P-value', f"{test_results.get('p_value', np.nan):.4f}"],
179         ['Significance_Level', '0.05'],
180         ['Significant', 'Yes' if test_results.get('significant')
181                    else 'No'],
182         ['Effect_Size', f"{diff:.4f}"],
183         ['Relative_Change', f"{test_results.get('relative_
184                    difference_pct', 0):.2f}%"],
185         ['CI_Lower', f"{ci.get('lower', np.nan):.4f}"],
186         ['CI_Upper', f"{ci.get('upper', np.nan):.4f}"]

```

```

181     ]
182
183     table = ax4.table(
184         cellText=summary_data,
185         cellLoc='left',
186         loc='center',
187         colWidths=[0.4, 0.6]
188     )
189     table.auto_set_font_size(False)
190     table.set_fontsize(10)
191     table.scale(1, 2)
192     ax4.set_title('Statistical Summary', fontsize=13, fontweight='
193         bold', pad=20)
194
195     plt.suptitle(
196         f'Comprehensive A/B Test Report: {metric_col}',
197         fontsize=16,
198         fontweight='bold',
199         y=0.995
200     )
201     return fig

```

10.7.3 Executive Summary Template

Listing 10.18: Executive Summary Generation

```

1 class ReportGenerator:
2     """Generate professional A/B test reports."""
3
4     @staticmethod
5     def generate_executive_summary(test_results: Dict,
6                                   metric_name: str,
7                                   experiment_name: str,
8                                   business_impact: Dict = None) -> str
9         :
10
11         """
12         Generate one-page executive summary.
13
14         Parameters:
15         -----
16         test_results: Dict
17             Statistical test results
18         metric_name: str
19             Name of primary metric
20         experiment_name: str
21             Name/description of experiment
22         business_impact: Dict
23             Business impact assessment results
24
25         Returns:
26         -----
27         str: Formatted executive summary
28         """
29         summary = []
30         summary.append("=" * 70)
31         summary.append(f"EXECUTIVE SUMMARY: {experiment_name}")

```

```

30     summary.append("=" * 70)
31     summary.append("")
32
33     # Key findings
34     summary.append("KEY FINDINGS")
35     summary.append("-" * 70)
36
37     sig_status = "SIGNIFICANT" if test_results.get('significant')
38     else "NOT SIGNIFICANT"
39     direction = "increased" if test_results.get('difference', 0) >
40     0 else "decreased"
41
42     rel_change = test_results.get('relative_difference_pct', 0)
43
44     summary.append(
45         f"The treatment {direction} {metric_name} by {abs(rel_
46         change):.2f}%"
47         f"(p={test_results.get('p_value', np.nan):.4f})"
48     )
49     summary.append(f"Result is {sig_status} at 95% confidence level")
50     summary.append("")
51
52     # Business impact
53     if business_impact:
54         summary.append("BUSINESS IMPACT")
55         summary.append("-" * 70)
56
57         revenue = business_impact.get('expected_annual_revenue_
58         impact', 0)
59         roi = business_impact.get('expected_roi_pct', 0)
60
61         summary.append(f"Expected Annual Revenue Impact: ${revenue
62         :,.0f}")
63         summary.append(f"Expected ROI: {roi:.1f}%")
64
65         if business_impact.get('conservative_estimate', {}).get('
66         annual_revenue_impact'):
67             cons_revenue = business_impact['conservative_estimate']
68             ['annual_revenue_impact']
69             summary.append(f"Conservative Estimate (95% CI lower):
70             ${cons_revenue:,.0f}")
71
72         summary.append("")
73
74     # Recommendation
75     summary.append("RECOMMENDATION")
76     summary.append("-" * 70)
77
78     if business_impact:
79         if (business_impact.get('is_statistically_significant') and
80             business_impact.get('is_practically_significant')):
81             recommendation = "SHIP: Both statistically and
82             practically significant"
83         elif (business_impact.get('is_statistically_significant')
84             and
85             not business_impact.get('is_practically_significant')):

```

```

76         recommendation = "CONSIDER: Statistically significant but small business impact"
77     elif (not business_impact.get('is_statistically_significant') and
78           business_impact.get('is_practically_significant')):
79         recommendation = "EXTEND TEST: Promising but needs more data"
80     else:
81         recommendation = "DO NOT SHIP: Not significant"
82 else:
83     recommendation = f"SHIP if {sig_status}" if test_results.get('significant') else "DO NOT SHIP"
84
85 summary.append(recommendation)
86 summary.append("")
87
88 # Technical details
89 summary.append("TECHNICAL DETAILS")
90 summary.append("-" * 70)
91
92 sample_sizes = test_results.get('sample_sizes', {})
93 if sample_sizes:
94     for variant, size in sample_sizes.items():
95         summary.append(f"    {variant}: n={size},")
96
97 summary.append(f"    Test type: {test_results.get('test_type', test_results.get('recommendation', 'N/A'))}")
98 summary.append("")
99
100 return "\n".join(summary)
101
102 @staticmethod
103 def generate_technical_documentation(test_results: Dict,
104                                     data_summary: Dict,
105                                     assumptions: Dict,
106                                     metric_name: str) -> str:
107     """Generate detailed technical documentation."""
108     doc = []
109     doc.append("-" * 70)
110     doc.append("TECHNICAL DOCUMENTATION: A/B TEST ANALYSIS")
111     doc.append("-" * 70)
112     doc.append("")
113
114     # 1. Data Summary
115     doc.append("1. DATA SUMMARY")
116     doc.append("-" * 70)
117     doc.append(f"Primary Metric: {metric_name}")
118     doc.append(f"Total Observations: {data_summary.get('n_total', 'N/A')},")
119     doc.append(f"Date Range: {data_summary.get('date_range', 'N/A')}")
120     doc.append("")
121
122     # 2. Statistical Method
123     doc.append("2. STATISTICAL METHOD")
124     doc.append("-" * 70)
125     doc.append(f"Test Type: {test_results.get('test_type', 'N/A')}")

```



```

126     doc.append(f"Significance_Level:  $\alpha$ =0.05")
127     doc.append(f"Hypothesis: Two-sided")
128     doc.append("")
129
130     # 3. Assumptions
131     doc.append("3. ASSUMPTIONS")
132     doc.append("-" * 70)
133
134     if assumptions:
135         if 'normality' in assumptions:
136             doc.append("Normality:")
137             for variant, result in assumptions['normality'].items():
138                 is_normal = result.get('shapiro_wilk', {}).get('is_normal', 'Unknown')
139                 doc.append(f"{variant}: {'Normal' if is_normal else 'Non-normal'}")
140
141             if 'variance_equality' in assumptions:
142                 equal_var = assumptions['variance_equality'].get('levene_test', {}).get('equal_variances', 'Unknown')
143                 doc.append(f"Equal Variances: {'Yes' if equal_var else 'No'}")
144
145     doc.append("")
146
147     # 4. Results
148     doc.append("4. RESULTS")
149     doc.append("-" * 70)
150
151     p_val = test_results.get('p_value', np.nan)
152     doc.append(f"P-value: {p_val:.6f}")
153
154     if 'control_mean' in test_results:
155         doc.append(f"Control Mean: {test_results['control_mean']:.4f}")
156         doc.append(f"Treatment Mean: {test_results['treatment_mean']:.4f}")
157     elif 'control_rate' in test_results:
158         doc.append(f"Control Rate: {test_results['control_rate']:.4f}")
159         doc.append(f"Treatment Rate: {test_results['treatment_rate']:.4f}")
160
161     diff = test_results.get('difference', test_results.get('absolute_difference', 0))
162     doc.append(f"Absolute Difference: {diff:.4f}")
163     doc.append(f"Relative Difference: {test_results.get('relative_difference_pct', 0):.2f}%")
164
165     ci = test_results.get('confidence_interval', {})
166     if ci:
167         doc.append(f"95% CI: [{ci.get('lower', np.nan):.4f}, {ci.get('upper', np.nan):.4f}]")
168
169     doc.append("")
170
171     # 5. Interpretation

```

```
172     doc.append("5. INTERPRETATION")
173     doc.append("-" * 70)
174
175     if test_results.get('significant'):
176         doc.append("The treatment has a statistically significant effect.")
177     else:
178         doc.append("No statistically significant difference detected.")
179
180     doc.append("")
181
182     return "\n".join(doc)
```

10.8 Summary

This chapter presented a comprehensive framework for analyzing and interpreting A/B test results:

- **Data Preprocessing:** Systematic validation, outlier treatment, missing data handling, and transformations ensure data quality
- **Exploratory Analysis:** Descriptive statistics and visualizations reveal patterns before formal testing
- **Statistical Pipeline:** Automated assumption checking and test selection ensure appropriate methodology
- **Effect Sizes:** Cohen's d, relative effects, and other measures quantify practical significance
- **Business Impact:** ROI calculations connect statistical findings to business decisions
- **Segment Analysis:** Heterogeneous treatment effects reveal differential impacts
- **Communication:** Tailored reporting ensures all stakeholders can act on findings

The Python frameworks provided enable reproducible, production-ready analysis workflows. By combining statistical rigor with business context, we transform experimental data into actionable insights.

10.9 Further Reading

- Deng et al. [2017]: Data quality in online experiments
- Kohavi et al. [2020]: Comprehensive experimentation guide
- Gelman and Hill [2006]: Data analysis using regression and multilevel models
- Harrell [2015]: Regression modeling strategies
- Tukey [1977]: Exploratory data analysis foundations

10.10 Exercises

1. **Data Preprocessing Pipeline:** Implement a complete preprocessing pipeline for A/B test data including:
 - SRM detection
 - Outlier identification using multiple methods
 - Missing data imputation
 - Transformation assessment
2. **EDA Dashboard:** Create an interactive EDA dashboard using Plotly that includes:
 - Distribution comparisons
 - Temporal trends
 - Statistical summaries
 - Assumption diagnostics
3. **Automated Analysis:** Build an automated analysis pipeline that:
 - Selects appropriate statistical tests based on data characteristics
 - Calculates effect sizes and confidence intervals
 - Assesses business impact
 - Generates executive summary
4. **Segment Analysis:** Analyze treatment effects across multiple segments and:
 - Test for interactions
 - Visualize heterogeneous effects
 - Report segment-specific recommendations
5. **Report Generation:** Create templates for:
 - Executive one-pagers
 - Technical documentation
 - Stakeholder presentations

Chapter 11

Case Studies and Real-World Applications

11.1 Learning Objectives

After completing this chapter, readers will be able to:

- Apply A/B testing methodology to diverse business contexts
- Design experiments that address specific business objectives across e-commerce, marketing, product, and media
- Analyze real-world experimental data with common complications
- Interpret results in business terms and make actionable recommendations
- Identify common pitfalls and avoid experimental failures through practical examples
- Build organizational capabilities for scaled experimentation
- Learn from both successful and failed experiments with complete analysis workflows
- Adapt methodologies to industry-specific constraints and opportunities

11.2 Introduction: From Theory to Practice

Theory provides foundation, but practice reveals nuance. This chapter presents comprehensive case studies from four major domains:

1. **E-commerce Optimization:** Product pages, checkout flows, recommendations, and pricing
2. **Digital Marketing:** Email campaigns, ad creative, landing pages, and social engagement
3. **Product Development:** Feature adoption, UI/UX improvements, algorithm performance, mobile optimization
4. **Content and Media:** Content recommendations, video engagement, news feeds, personalization

Each case study includes:

- **Business context:** The problem, stakeholders, and constraints
- **Hypothesis and rationale:** What we expected and why
- **Experimental design:** Complete methodology with power calculations
- **Implementation:** Randomization, instrumentation, and data collection
- **Analysis workflow:** Full Python code for preprocessing, testing, and interpretation
- **Results:** Honest discussion including surprises and complications
- **Business outcome:** Decisions made, impact realized, and monitoring approach
- **Lessons learned:** What succeeded, what failed, and what we'd change

All data has been anonymized and scaled while maintaining statistical properties. The challenges, mistakes, and successes are real.

11.3 Case Study 1: Google Search Results and Page Load Time

11.3.1 Context and Business Problem

In 2009, Google tested whether displaying more search results per page (20 or 30 instead of 10) would improve user engagement. The hypothesis seemed reasonable: fewer clicks to pagination should improve user experience.

11.3.2 Experimental Design

Design: A/B/C test with three variants

- Control: 10 results per page (baseline)
- Variant A: 20 results per page
- Variant B: 30 results per page

Randomization: User-level randomization, 33.3% allocation each

Duration: 3 weeks

Sample size: Millions of users (Google traffic scale)

Metrics:

- Primary: Searches per user session
- Secondary: Page load time, click-through rate, user satisfaction

11.3.3 Results and Analysis

Metric	10 Results	20 Results	30 Results
Page load time (ms)	400	650	900
Load time increase	–	+63%	+125%
Searches per user	5.2	4.9	4.6
Relative change	–	-5.8%	-11.5%
Satisfaction score	7.8/10	7.5/10	7.2/10

Statistical significance: All differences highly significant ($p < 0.001$) due to massive sample size.

Correlation analysis: Strong negative correlation between page load time and searches per user:

$$\text{Elasticity} = \frac{\Delta \text{Searches}}{\Delta \text{Load time}} \approx -1.0 \quad (11.1)$$

A 100ms increase in load time corresponded to approximately 1% decrease in searches.

11.3.4 Deep Dive: Causal Mechanism

Why did more results decrease engagement?

1. **Performance degradation:** Rendering more results increased page weight and computation
2. **Perceived value:** Users valued speed over choice
3. **Cognitive overload:** More options paradoxically made decision harder
4. **User behavior:** Most users (>90%) found results in first 10 regardless

11.3.5 Statistical Analysis Framework

Listing 11.1: Analyzing Load Time vs. Engagement Trade-offs

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy import stats
5 import seaborn as sns
6
7 # Simulated data based on Google's published results
8 np.random.seed(42)
9
10 def simulate_google_experiment(n_users=100000):
11     """Simulate Google search results experiment."""
12
13     # Allocate users to variants
14     variants = np.random.choice(['10_results', '20_results', '30_
15                                 results'],
16                                 size=n_users)
17
18     # Simulate page load times (milliseconds)

```

```

18     load_times = {
19         '10_results': np.random.normal(400, 50),
20         '20_results': np.random.normal(650, 70),
21         '30_results': np.random.normal(900, 90)
22     }
23
24     # Simulate searches per user (Poisson with rate dependent on load
25     # time)
26     # Model: slower load time -> lower engagement
27     base_searches = {
28         '10_results': 5.2,
29         '20_results': 4.9,
30         '30_results': 4.6
31     }
32
33     data = []
34     for i, variant in enumerate(variants):
35         load_time = load_times[variant]
36
37         # Add user-level noise
38         lambda_param = base_searches[variant] * np.random.lognormal(0,
39         0.3)
40         searches = np.random.poisson(lambda_param)
41
42         # Satisfaction (0-10 scale, correlated with searches and load
43         # time)
44         satisfaction = np.clip(
45             7.8 - (load_time - 400) / 200 + np.random.normal(0, 1),
46             0, 10
47         )
48
49         data.append({
50             'user_id': i,
51             'variant': variant,
52             'load_time_ms': load_time,
53             'searches': searches,
54             'satisfaction': satisfaction
55         })
56
57     return pd.DataFrame(data)
58
59 # Generate data
60 data = simulate_google_experiment(n_users=100000)
61
62 # Aggregate statistics by variant
63 summary = data.groupby('variant').agg({
64     'load_time_ms': ['mean', 'std'],
65     'searches': ['mean', 'std'],
66     'satisfaction': ['mean', 'std']
67 }).round(2)
68
69 print("=== Experiment Summary ===")
70 print(summary)
71
72 # Statistical tests
73 variants = ['10_results', '20_results', '30_results']
74
75 print("\n=== Pairwise Comparisons (Two-sample t-tests) ===")

```



```

73 for i in range(len(variants)):
74     for j in range(i+1, len(variants)):
75         v1, v2 = variants[i], variants[j]
76
77         searches_1 = data[data['variant']==v1]['searches']
78         searches_2 = data[data['variant']==v2]['searches']
79
80         t_stat, p_value = stats.ttest_ind(searches_1, searches_2)
81
82         diff = searches_2.mean() - searches_1.mean()
83         rel_diff = diff / searches_1.mean() * 100
84
85         print(f"\n{v1} vs {v2}:")
86         print(f"Mean difference: {diff:.3f} searches ({rel_diff:+.1f}%)")
87         print(f"t-statistic: {t_stat:.2f}")
88         print(f"p-value: {p_value:.2e}")
89
90 # Regression: searches ~ load_time
91 from sklearn.linear_model import LinearRegression
92
93 X = data[['load_time_ms']].values
94 y = data['searches'].values
95
96 model = LinearRegression()
97 model.fit(X, y)
98
99 print("\n=== Regression: Searches ~ Load Time ===")
100 print(f"Slope: {model.coef_[0]:.6f} searches per ms")
101 print(f"Interpretation: Each 100ms increase -> "
102       f"{model.coef_[0]*100:.2f} fewer searches")
103
104 # Visualization
105 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
106
107 # 1. Load time by variant
108 ax = axes[0, 0]
109 data.boxplot(column='load_time_ms', by='variant', ax=ax)
110 ax.set_xlabel('Variant')
111 ax.set_ylabel('Page Load Time (ms)')
112 ax.set_title('Load Time Distribution by Variant')
113 plt.sca(ax)
114 plt.xticks(range(1, 4), ['10_results', '20_results', '30_results'])
115
116 # 2. Searches by variant
117 ax = axes[0, 1]
118 variant_means = data.groupby('variant')['searches'].agg(['mean', 'sem'])
119 variant_order = ['10_results', '20_results', '30_results']
120 means = [variant_means.loc[v, 'mean'] for v in variant_order]
121 sems = [variant_means.loc[v, 'sem'] for v in variant_order]
122
123 x = np.arange(3)
124 ax.bar(x, means, yerr=[1.96*s for s in sems], capsize=5, alpha=0.7)
125 ax.set_xticks(x)
126 ax.set_xticklabels(['10_results', '20_results', '30_results'])
127 ax.set_ylabel('Searches per User')
128 ax.set_title('Mean Searches by Variant (95% CI)')

```

```

129 ax.grid(axis='y', alpha=0.3)
130
131 # 3. Scatter: Load time vs searches
132 ax = axes[1, 0]
133 for variant, color in zip(variant_order, ['C0', 'C1', 'C2']):
134     subset = data[data['variant']==variant].sample(1000)
135     ax.scatter(subset['load_time_ms'], subset['searches'],
136               alpha=0.3, s=20, label=variant.replace('_', ' '), color=
137                 color)
138
139 # Add regression line
140 X_line = np.linspace(300, 1000, 100).reshape(-1, 1)
141 y_line = model.predict(X_line)
142 ax.plot(X_line, y_line, 'r--', linewidth=2, label='Regression_fit')
143
144 ax.set_xlabel('Page_Load_Time_(ms)')
145 ax.set_ylabel('Searches_per_User')
146 ax.set_title('Load_Time_vs_Engagement')
147 ax.legend()
148 ax.grid(alpha=0.3)
149
150 # 4. Satisfaction scores
151 ax = axes[1, 1]
152 data.boxplot(column='satisfaction', by='variant', ax=ax)
153 ax.set_xlabel('Variant')
154 ax.set_ylabel('Satisfaction_Score_(0-10)')
155 ax.set_title('User_Satisfaction_by_Variant')
156 plt.sca(ax)
157 plt.xticks(range(1, 4), ['10_results', '20_results', '30_results'])
158
159 plt.tight_layout()
160 plt.savefig('google_case_study.pdf', bbox_inches='tight')
161 plt.show()
162
163 # Calculate business impact
164 print("\n===Business_Impact_Estimation===")
165 baseline_searches = data[data['variant']=='10_results']['searches'].
166     mean()
167
168 for variant in ['20_results', '30_results']:
169     variant_searches = data[data['variant']==variant]['searches'].mean
170     ()
171     loss = (variant_searches - baseline_searches) / baseline_searches *
172         100
173
174     # At Google scale (billions of searches/day)
175     daily_searches_billions = 8.5 # Approximate 2009 volume
176     annual_loss_millions = daily_searches_billions * 1000 * loss / 100
177         * 365
178
179     print(f"\n{variant}:")
180     print(f"Relative_loss:{loss:.1f}%")
181     print(f"Estimated_annual_search_loss:{annual_loss_millions:.0f}M
182         _searches")
183     print(f"Decision:DO_NOT_SHIP")

```

11.3.6 Lessons Learned

1. **Performance is a feature:** Page load time directly impacts user behavior at all scales, but especially at massive scale
2. **Guardrail metrics:** Always monitor technical performance metrics alongside engagement metrics
3. **User psychology:** More choice isn't always better; cognitive load and perceived value matter
4. **Scale amplification:** At Google's traffic volume, small performance differences have billion-search impacts
5. **Regression analysis:** Quantifying relationship between technical metrics and business outcomes enables forecasting

Actionable framework:

- Measure page load time as standard guardrail
- Set performance budgets (e.g., < 500ms) before testing
- Estimate elasticity of engagement with respect to performance
- Calculate ROI of performance optimization

11.4 Case Study 2: E-commerce Checkout Optimization

11.4.1 Context

An e-commerce company with \$500M annual revenue hypothesized that simplifying checkout from 5 steps to 3 steps would increase conversion.

11.4.2 Experimental Design

Hypothesis: Reducing friction in checkout process will increase completion rate.

Design: A/B test

- Control: 5-step checkout (address, shipping, payment, review, confirm)
- Treatment: 3-step checkout (combined address/shipping, payment/review, confirm)

Randomization: User-level, 50/50 split

Sample size: 50,000 users per variant (calculated for 80% power to detect 5% relative lift)

Duration: 14 days (to capture two full weeks including weekends)

Metrics:

- Primary: Checkout completion rate (among initiated)
- Secondary: Overall conversion rate, revenue per user, abandonment points
- Guardrails: Customer support contacts, return rate

11.4.3 Overall Results

Metric	Control	Treatment	Lift
Checkout initiation	12.0%	12.1%	+0.8%
Checkout completion	68.0%	71.5%	+5.1%**
Overall conversion	8.16%	8.65%	+6.0%**
Revenue per user	\$4.25	\$4.50	+5.9%**
Avg order value	\$52.08	\$52.02	-0.1%
Support contacts	2.1%	2.3%	+9.5%

** Statistically significant at $\alpha = 0.05$

11.4.4 Segmentation Analysis

Critical insight came from examining heterogeneous treatment effects:

By customer type:

Segment	Control	Treatment	Lift	Size
New users	6.5%	7.8%	+20.0%**	60%
Returning users	9.2%	9.0%	-2.2%	40%

By device:

Device	Control	Treatment	Lift	Size
Mobile	5.8%	6.9%	+19.0%**	55%
Desktop	11.2%	11.0%	-1.8%	45%

Listing 11.2: Heterogeneous Treatment Effect Analysis

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy import stats
5 import seaborn as sns
6
7 def simulate_checkout_experiment(n_users=100000):
8     """Simulate e-commerce checkout experiment with segments."""
9
10    np.random.seed(42)
11
12    # User characteristics
13    is_new = np.random.binomial(1, 0.60, n_users) # 60% new users
14    is_mobile = np.random.binomial(1, 0.55, n_users) # 55% mobile
15
16    # Treatment assignment
17    treatment = np.random.binomial(1, 0.5, n_users)
18
19    # Baseline conversion probabilities
20    baseline_conv = np.where(
21        is_new,
22        np.where(is_mobile, 0.058, 0.070), # New users: mobile vs
        desktop
    )

```

```

23     np.where(is_mobile, 0.085, 0.112)    # Returning: mobile vs
        desktop
24 )
25
26 # Treatment effects (heterogeneous)
27 treatment_effect = np.where(
28     is_new,
29     np.where(is_mobile, 0.011, -0.001),  # New users benefit, esp
        mobile
30     np.where(is_mobile, 0.005, -0.002)  # Returning slightly worse
31 )
32
33 # Realized conversion
34 conv_prob = baseline_conv + treatment * treatment_effect
35 converted = np.random.binomial(1, conv_prob)
36
37 # Revenue (conditional on conversion)
38 revenue = converted * np.random.lognormal(np.log(50), 0.6)
39
40 data = pd.DataFrame({
41     'user_id': range(n_users),
42     'treatment': treatment,
43     'is_new': is_new,
44     'is_mobile': is_mobile,
45     'converted': converted,
46     'revenue': revenue
47 })
48
49 # Add labels
50 data['variant'] = data['treatment'].map({0: 'Control', 1: '
    Treatment'})
51 data['user_type'] = data['is_new'].map({1: 'New', 0: 'Returning'})
52 data['device'] = data['is_mobile'].map({1: 'Mobile', 0: 'Desktop'})
53
54 return data
55
56 # Generate data
57 data = simulate_checkout_experiment(n_users=100000)
58
59 # Overall analysis
60 print("=== Overall Results ===")
61 overall = data.groupby('variant').agg({
62     'converted': ['sum', 'mean', 'sem'],
63     'revenue': ['sum', 'mean']
64 })
65
66 control_rate = overall.loc['Control', ('converted', 'mean')]
67 treatment_rate = overall.loc['Treatment', ('converted', 'mean')]
68 lift = (treatment_rate - control_rate) / control_rate * 100
69
70 print(f"Control conversion rate: {control_rate:.4f}")
71 print(f"Treatment conversion rate: {treatment_rate:.4f}")
72 print(f"Relative lift: {lift:+.1f}%")
73
74 # Statistical test
75 control_conv = data[data['treatment']==0]['converted']
76 treatment_conv = data[data['treatment']==1]['converted']
77

```

```

78 z_stat = (treatment_conv.mean() - control_conv.mean()) / np.sqrt(
79     control_conv.var() / len(control_conv) +
80     treatment_conv.var() / len(treatment_conv)
81 )
82 p_value = 2 * (1 - stats.norm.cdf(abs(z_stat)))
83
84 print(f"\nZ-statistic: {z_stat:.3f}")
85 print(f"P-value: {p_value:.4f}")
86
87 # Segmented analysis
88 print("\n== Heterogeneous Treatment Effects ==")
89
90 for segment_var, segment_name in [('user_type', 'User_Type'),
91                                   ('device', 'Device')]:
92     print(f"\nBy {segment_name}:")
93
94     segment_results = data.groupby([segment_var, 'variant']).agg({
95         'converted': ['mean', 'count']
96     }).reset_index()
97
98     # Pivot for easier comparison
99     pivot = segment_results.pivot(
100         index=segment_var,
101         columns='variant',
102         values=('converted', 'mean')
103     )
104
105     for segment in pivot.index:
106         control_rate = pivot.loc[segment, ('Control')]
107         treatment_rate = pivot.loc[segment, ('Treatment')]
108         lift = (treatment_rate - control_rate) / control_rate * 100
109
110         # Test within segment
111         seg_data = data[data[segment_var]==segment]
112         seg_control = seg_data[seg_data['treatment']==0]['converted']
113         seg_treatment = seg_data[seg_data['treatment']==1]['converted']
114
115         _, p = stats.ttest_ind(seg_treatment, seg_control)
116         sig = '**' if p < 0.05 else ''
117
118         print(f"{segment}: {control_rate:.4f} -> {treatment_rate:.4f}
119               "
120               f"({lift:+.1f}%) {sig}")
121
122 # Visualization
123 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
124
125 # 1. Overall conversion rates
126 ax = axes[0, 0]
127 overall_data = data.groupby('variant')['converted'].agg(['mean', 'sem'])
128 x = np.arange(2)
129 ax.bar(x, overall_data['mean'], yerr=1.96*overall_data['sem'],
130       capsize=5, alpha=0.7, color=['C0', 'C1'])
131 ax.set_xticks(x)
132 ax.set_xticklabels(['Control', 'Treatment'])
133 ax.set_ylabel('Conversion Rate')
134 ax.set_title('Overall Conversion Rate (95% CI)')

```

```

134 ax.set_ylim([0, 0.10])
135 for i, (idx, row) in enumerate(overall_data.iterrows()):
136     ax.text(i, row['mean'] + 0.003, f"{row['mean']:.3f}",
137           ha='center', fontweight='bold')
138 ax.grid(axis='y', alpha=0.3)
139
140 # 2. By user type
141 ax = axes[0, 1]
142 user_type_data = data.groupby(['user_type', 'variant'])['converted'].
    mean().unstack()
143 user_type_data.plot(kind='bar', ax=ax, color=['C0', 'C1'], alpha=0.7)
144 ax.set_ylabel('Conversion_Rate')
145 ax.set_title('Conversion_Rate_by_User_Type')
146 ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
147 ax.legend(title='Variant')
148 ax.grid(axis='y', alpha=0.3)
149
150 # 3. By device
151 ax = axes[1, 0]
152 device_data = data.groupby(['device', 'variant'])['converted'].mean().
    unstack()
153 device_data.plot(kind='bar', ax=ax, color=['C0', 'C1'], alpha=0.7)
154 ax.set_ylabel('Conversion_Rate')
155 ax.set_title('Conversion_Rate_by_Device')
156 ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
157 ax.legend(title='Variant')
158 ax.grid(axis='y', alpha=0.3)
159
160 # 4. Treatment effect by segment
161 ax = axes[1, 1]
162
163 segments = []
164 effects = []
165
166 for user_type in ['New', 'Returning']:
167     for device in ['Mobile', 'Desktop']:
168         seg_data = data[(data['user_type']==user_type) &
169                        (data['device']==device)]
170
171         control_mean = seg_data[seg_data['treatment']==0]['converted'].
            mean()
172         treatment_mean = seg_data[seg_data['treatment']==1]['converted'
            ].mean()
173
174         effect = (treatment_mean - control_mean) / control_mean * 100
175
176         label = f"{user_type}\n{device}"
177         segments.append(label)
178         effects.append(effect)
179
180 colors = ['green' if e > 0 else 'red' for e in effects]
181 x_pos = np.arange(len(segments))
182
183 ax.barh(x_pos, effects, color=colors, alpha=0.7)
184 ax.axvline(x=0, color='black', linestyle='-', linewidth=0.8)
185 ax.set_yticks(x_pos)
186 ax.set_yticklabels(segments)
187 ax.set_xlabel('Relative_Lift(%)')

```

```

188 ax.set_title('Treatment_Effect_by_Segment')
189 ax.grid(axis='x', alpha=0.3)
190
191 plt.tight_layout()
192 plt.savefig('checkout_segmentation.pdf', bbox_inches='tight')
193 plt.show()
194
195 # Revenue impact calculation
196 print("\n=== Business_Impact ===")
197 annual_visitors = 10_000_000 # 10M annual visitors
198 current_revenue = 500_000_000 # $500M annually
199
200 baseline_conv = control_rate
201 new_conv = treatment_rate
202 revenue_per_conversion = data[data['converted']==1]['revenue'].mean()
203
204 incremental_conversions = annual_visitors * (new_conv - baseline_conv)
205 incremental_revenue = incremental_conversions * revenue_per_conversion
206
207 print(f"Baseline_conversions: {annual_visitors*baseline_conv:,.0f}")
208 print(f"New_conversions: {annual_visitors*new_conv:,.0f}")
209 print(f"Incremental_conversions: {incremental_conversions:,.0f}")
210 print(f"Avg_revenue_per_conversion: ${revenue_per_conversion:.2f}")
211 print(f"Incremental_annual_revenue: ${incremental_revenue:,.0f}")
212 print(f"Relative_revenue_increase: {(incremental_revenue/current_revenue
    *100:.2f)}%")
213
214 # Recommendation
215 print("\n=== Recommendation ===")
216 print("SHIP TO ALL USERS")
217 print("Rationale:")
218 print("- Statistically significant overall lift")
219 print("- Strong benefit for strategically important segments (new,
    mobile)")
220 print("- Small negative effect on returning/desktop acceptable")
221 print("- Projected $30M+ incremental annual revenue")
222 print("- Support contact increase minimal and manageable")

```

11.4.5 Decision Framework

The team faced a trade-off: overall benefit with segment-specific harm.

Decision criteria:

1. **Overall impact:** +6.0% conversion, highly significant
2. **Strategic alignment:** New users and mobile are growth priorities
3. **Magnitude:** +20% for new/mobile vs. -2% for returning/desktop
4. **Business value:** \$30M+ projected incremental annual revenue
5. **Reversibility:** Can iterate if returning user experience degrades further

Decision: Ship to all users, monitor returning user metrics closely.

11.4.6 Post-Launch Monitoring

After launch, team implemented:

- Weekly cohort analysis of returning users
- Customer feedback analysis (surveys, support tickets)
- Funnel tracking for abandonment points
- Quarterly re-testing to validate continued benefit

11.4.7 Lessons Learned

1. **Segment analysis is critical:** Averages hide important heterogeneity
2. **Strategic weighting:** Not all segments are equal; weight by business priorities
3. **Trade-offs are normal:** Few changes benefit everyone universally
4. **Quantify business impact:** Translate statistical findings to revenue/profit
5. **Monitor post-launch:** Continue learning after shipping

11.5 Case Study 3: Netflix Thumbnail Personalization

11.5.1 Context

Netflix hypothesized that personalizing video thumbnails based on user viewing history and preferences would increase engagement (click-through rate and watch time).

11.5.2 Technical Challenge

Consistency requirement: Users see the same videos multiple times (homepage, search, recommendations). Showing different thumbnails each time would be confusing.

Solution: Deterministic assignment—`hash(user_id, video_id) → thumbnail_variant`
This ensures:

- Same user always sees same thumbnail for given video
- Different users see different thumbnails (personalization)
- Randomization for experimentation

11.5.3 Multi-Stage Experimental Approach

Netflix used an iterative experimental strategy:

Stage 1: Validate thumbnail selection matters

- Test: Random thumbnails vs. editorial picks
- Result: Editorial picks had 30% higher engagement

- Conclusion: Thumbnail selection is important lever

Stage 2: Test personalization hypothesis

- Test: Editorial picks (best average) vs. personalized selection
- Personalization: Select thumbnail based on user's genre preferences and viewing history
- Result: Personalized thumbnails 20% higher engagement than editorial
- Conclusion: Personalization works

Stage 3: Optimize personalization model

- Test: Multiple ML models for personalization
 - Collaborative filtering
 - Content-based filtering
 - Neural network (deep learning)
 - Ensemble
- Result: Neural network approach best (additional 5% lift)
- Conclusion: Ship neural network model

11.5.4 Experimental Design Details (Stage 2)

Randomization: Video-level stratified randomization

- For each video, select N candidate thumbnails (N=3-10)
- Randomly assign users to personalization (treatment) vs. best-on-average (control)
- Within treatment: use model to select best thumbnail for each user

Metrics:

- Primary: Click-through rate (CTR) - play rate when video is visible
- Secondary: Watch time, completion rate, satisfaction rating
- Guardrails: Customer support, cancellations

Duration: 4 weeks (to account for learning effects)

Sample: 10M users, 5M control / 5M treatment

11.5.5 Results

Metric	Control	Treatment	Lift
CTR (overall)	3.2%	3.84%	+20.0%**
Watch time (hrs/user/week)	12.5	13.1	+4.8%**
Completion rate	68%	70%	+2.9%**
Satisfaction (1-5)	4.1	4.2	+2.4%**

** $p < 0.001$

Heterogeneity by video type:

Personalization worked especially well for:

- Ensemble casts (different thumbnails feature different actors)
- Genre-spanning content (show different aspects to different users)
- Foreign language content (different cultural framing)

11.5.6 Implementation Considerations

1. **Infrastructure:** Real-time personalization requires low-latency model serving
2. **Quality control:** Filter inappropriate thumbnails (spoilers, misleading images)
3. **Fallback logic:** When model fails, default to editorial pick
4. **Cold start:** New users see editorial picks until preferences learned
5. **Monitoring:** Track model performance, A/A tests for bias

11.5.7 Lessons Learned

1. **Iterative experimentation:** Build incrementally, validating each assumption
2. **Technical-statistical integration:** Implementation details (hashing, persistence) affect experimental validity
3. **Personalization value:** Context-specific optimization often outperforms one-size-fits-all
4. **Heterogeneous effects:** Personalization benefits vary by content type
5. **Production readiness:** Consider operational constraints before launch

11.6 Case Study 4: Failed Experiment—Subscription Paywall

Not all experiments succeed. Learning from failures is as valuable as from successes.

11.6.1 Context

A digital media platform with freemium model (free articles with monthly limit, premium unlimited access) wanted to increase paid subscriptions.

Hypothesis: More aggressive paywall messaging and earlier paywall triggers would increase conversion to paid subscriptions.

11.6.2 Design

- Control: Paywall after 5 free articles/month, soft messaging
- Treatment: Paywall after 3 free articles/month, aggressive CTA

Primary metric: Subscription conversion rate (

Duration: 3 weeks

Sample: 200,000 users per variant

11.6.3 Results: The Good and The Bad

Metric	Control	Treatment	Abs. Diff	Rel. Lift
<i>Positive:</i>				
Subscription conversion	1.20%	1.38%	+0.18pp	+15.0%**
<i>Negative:</i>				
Bounce rate	35%	49%	+14pp	+40.0%**
Pages per user	8.2	5.1	-3.1	-37.8%**
Return rate (7-day)	42%	31%	-11pp	-26.2%**
<i>Net effect:</i>				
Net new subscribers	1200	1140	-60	-5.0%

** $p < 0.001$

11.6.4 What Went Wrong

1. **Metric misalignment:** Optimized subscription conversion rate (local metric) at expense of overall user base (global metric)
2. **Missing guardrails:** Didn't adequately monitor engagement and retention metrics
3. **Time horizon mismatch:** Immediate conversions vs. long-term user value
4. **User experience:** Aggressive tactics damaged brand perception

Root cause analysis:

The aggressive paywall converted users faster but drove away many more users who would have converted later:

- Fewer users reached the paywall (40% bounce increase)
- Lower return rate meant fewer conversion opportunities
- Net effect: slightly fewer total conversions despite higher rate

11.6.5 Correct Analysis

Listing 11.3: Analyzing Funnel Effects

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 def analyze_paywall_funnel():
6     """Analyze the full funnel, not just conversion rate."""
7
8     # Monthly visitors
9     monthly_visitors = 100000
10
11     # Control funnel
12     control_funnel = {
13         'visitors': monthly_visitors,
14         'bounce_rate': 0.35,
15         'engaged': None, # calculated
16         'reach_paywall_rate': 0.15, # % of engaged users
17         'paywall_conversion': 0.012,
18         'new_subscribers': None # calculated
19     }
20
21     control_funnel['engaged'] = control_funnel['visitors'] * (
22         1 - control_funnel['bounce_rate']
23     )
24     control_funnel['reach_paywall'] = control_funnel['engaged'] * (
25         control_funnel['reach_paywall_rate']
26     )
27     control_funnel['new_subscribers'] = control_funnel['reach_paywall']
28         * (
29             control_funnel['paywall_conversion']
30         )
31
32     # Treatment funnel
33     treatment_funnel = {
34         'visitors': monthly_visitors,
35         'bounce_rate': 0.49,
36         'engaged': None,
37         'reach_paywall_rate': 0.20, # Higher due to earlier trigger
38         'paywall_conversion': 0.0138, # Higher rate
39         'new_subscribers': None
40     }
41
42     treatment_funnel['engaged'] = treatment_funnel['visitors'] * (
43         1 - treatment_funnel['bounce_rate']
44     )
45     treatment_funnel['reach_paywall'] = treatment_funnel['engaged'] * (
46         treatment_funnel['reach_paywall_rate']
47     )
48     treatment_funnel['new_subscribers'] = treatment_funnel['reach_
49         paywall'] * (
50             treatment_funnel['paywall_conversion']
51         )
52
53     # Print analysis
54     print("=== Funnel Analysis ===\n")

```

```

53 print("Control:")
54 print(f"1. Visitors: {control_funnel['visitors']:, .0f}")
55 print(f"2. Engaged (not bounced): {control_funnel['engaged']:, .0f}")
56     f"({(1-control_funnel['bounce_rate'])*100:.1f}%)"
57 print(f"3. Reached paywall: {control_funnel['reach_paywall']:, .0f}")
58     f"({control_funnel['reach_paywall_rate']*100:.1f}%)"
59 print(f"4. NEW SUBSCRIBERS: {control_funnel['new_subscribers']:, .0f}")
60     f"({control_funnel['paywall_conversion']*100:.2f}% conversion)"
61
62
63 print("\nTreatment:")
64 print(f"1. Visitors: {treatment_funnel['visitors']:, .0f}")
65 print(f"2. Engaged (not bounced): {treatment_funnel['engaged']:, .0f}")
66     f"({(1-treatment_funnel['bounce_rate'])*100:.1f}%)"
67 print(f"3. Reached paywall: {treatment_funnel['reach_paywall']:, .0f}")
68     f"({treatment_funnel['reach_paywall_rate']*100:.1f}%)"
69 print(f"4. NEW SUBSCRIBERS: {treatment_funnel['new_subscribers']:, .0f}")
70     f"({treatment_funnel['paywall_conversion']*100:.2f}% conversion)"
71
72 print("\n=== Key Insight ===")
73 subscriber_diff = (treatment_funnel['new_subscribers'] -
74                    control_funnel['new_subscribers'])
75 rel_diff = subscriber_diff / control_funnel['new_subscribers'] *
76            100
77 print(f"Net subscriber change: {subscriber_diff:, .0f} ({rel_diff:+.1f}%)")
78 print("\nProblem: Higher conversion rate, but fewer total subscribers!")
79 print("Reason: Aggressive paywall drove away more users than it converted.")
80
81 # Visualization
82 fig, axes = plt.subplots(1, 2, figsize=(14, 6))
83
84 # Funnel visualization
85 ax = axes[0]
86
87 stages = ['Visitors', 'Engaged', 'Reached\nPaywall', 'Subscribed']
88 control_values = [
89     control_funnel['visitors'],
90     control_funnel['engaged'],
91     control_funnel['reach_paywall'],
92     control_funnel['new_subscribers']
93 ]
94 treatment_values = [
95     treatment_funnel['visitors'],
96     treatment_funnel['engaged'],
97     treatment_funnel['reach_paywall'],
98     treatment_funnel['new_subscribers']

```

```

99     ]
100
101     x = np.arange(len(stages))
102     width = 0.35
103
104     ax.bar(x - width/2, control_values, width, label='Control',
105            alpha=0.8, color='C0')
106     ax.bar(x + width/2, treatment_values, width, label='Treatment',
107            alpha=0.8, color='C1')
108
109     ax.set_ylabel('Users')
110     ax.set_title('Subscription Funnel Comparison')
111     ax.set_xticks(x)
112     ax.set_xticklabels(stages)
113     ax.legend()
114     ax.grid(axis='y', alpha=0.3)
115     ax.set_yscale('log')
116
117     # Highlight the problem
118     for i, (c_val, t_val) in enumerate(zip(control_values, treatment_
119                                         values)):
120         if i == len(stages) - 1: # Final stage
121             if t_val < c_val:
122                 ax.text(i, max(c_val, t_val) * 1.5, 'Problem!',
123                        ha='center', color='red', fontweight='bold',
124                        fontsize=12)
125
126     # Conversion rates vs. absolute numbers
127     ax = axes[1]
128
129     metrics = ['Conversion\nRate', 'Total\nSubscribers']
130     control_rel = [100, 100] # Baseline
131     treatment_rel = [
132         (treatment_funnel['paywall_conversion'] /
133          control_funnel['paywall_conversion']) * 100,
134         (treatment_funnel['new_subscribers'] /
135          control_funnel['new_subscribers']) * 100
136     ]
137
138     x = np.arange(len(metrics))
139     width = 0.35
140
141     ax.bar(x - width/2, control_rel, width, label='Control (baseline)',
142            alpha=0.8, color='C0')
143     bars = ax.bar(x + width/2, treatment_rel, width, label='Treatment',
144                   alpha=0.8, color='C1')
145
146     # Color bars by good/bad
147     bars[0].set_color('green') # Higher rate is green
148     bars[1].set_color('red')   # Fewer subscribers is red
149
150     ax.axhline(y=100, color='black', linestyle='--', alpha=0.5)
151     ax.set_ylabel('Relative to Control (%)')
152     ax.set_title('The Optimization Paradox')
153     ax.set_xticks(x)
154     ax.set_xticklabels(metrics)
155     ax.legend()
156     ax.grid(axis='y', alpha=0.3)

```

```

156
157     # Add value labels
158     for i, (val, bar) in enumerate(zip(treatment_rel, bars)):
159         ax.text(bar.get_x() + bar.get_width()/2, val + 2,
160               f'{val:.0f}%', ha='center', fontweight='bold')
161
162     plt.tight_layout()
163     plt.savefig('paywall_failure_analysis.pdf', bbox_inches='tight')
164     plt.show()
165
166 analyze_paywall_funnel()

```

11.6.6 Lessons Learned

1. **Holistic metrics:** Measure the entire funnel, not isolated steps
2. **Guardrail importance:** Protect critical health metrics (engagement, retention)
3. **Local vs. global optimization:** Improving one metric can harm overall goal
4. **Customer lifetime value:** Consider long-term impact beyond immediate conversion
5. **Document failures:** Failed experiments provide valuable learning

Corrective actions:

- Revised success criteria to include engagement and retention
- Established mandatory guardrail metrics for all experiments
- Implemented funnel analysis as standard practice
- Created decision framework weighting multiple objectives

11.7 Digital Marketing

Digital marketing experiments optimize customer acquisition, engagement, and campaign ROI across channels.

11.7.1 Case Study 5: Email Subject Line A/B Test

Context: A B2B SaaS company with 500K email subscribers noticed declining open rates (18% vs. industry 22-25%). Marketing hypothesized personalized, benefit-focused subject lines would improve engagement.

Design:

- Control: "Weekly Product Update: New Features"
- Treatment: "{Company}: Save 5 Hours/Week with New Automation"
- Randomization: User-level, 50/50

- Primary metric: Open rate
- Secondary: CTR, unsubscribe rate
- Sample: 490K delivered emails

Results:

Metric	Control	Treatment	Lift
Open rate	18.0%	20.5%	+13.9%**
Click rate (of opens)	3.2%	3.9%	+21.9%**
Click rate (of delivered)	0.58%	0.80%	+38.7%**
Unsubscribe rate	0.100%	0.120%	+20.0%

** $p < 0.001$

Business Impact: +\$770K annual value from increased clicks

Decision: Ship with modifications. Fixed personalization token bugs, added fallback for missing company names, ongoing monitoring of unsubscribe trends.

Lessons:

- Implementation quality critical (0.3% received broken tokens)
- Test effects degrade over time (novelty, list fatigue)
- Small guardrail violations compound (unsubscribe monitoring essential)
- Qualitative feedback revealed "creepy" concerns requiring tone adjustments

11.7.2 Case Study 6: Landing Page Redesign

Context: Online education platform with paid search landing page converting at 12%. UX team proposed cleaner design with above-the-fold CTA.

Design: A/B test, session-level randomization, 25K visitors per variant

Results:

- Primary metric (signup rate): 12.3% → 14.8% (+20.3%, $p < 0.001$)
- Trial-to-paid conversion: 8.2% → 8.1% (-1.2%, $p = 0.78$)
- Bounce rate: 42% → 38% (-9.5%, $p = 0.002$)
- Time on page: 85s → 62s (-27%, $p < 0.001$)

Deep Dive Finding: Increased signups but no change in qualified leads. New design attracted more low-intent visitors who signed up but didn't convert to paid.

Decision: Do NOT ship. Despite improving primary metric, overall funnel economics unchanged. Redesigned to optimize for qualified signups, not raw signups.

Lesson: Optimize for business outcome, not intermediate metrics. Funnel analysis essential.

11.8 Product Development

Product experiments test feature adoption, UX improvements, and algorithm performance.

11.8.1 Case Study 7: Feature Onboarding Flow

Context: Productivity app launched collaborative features but only 15% of users tried them. Product team designed interactive tutorial to increase adoption.

Design:

- Control: Existing text-based tour (skipped by 85% of users)
- Treatment: Interactive 3-step tutorial with sample collaboration
- Randomization: New users only, account-level
- Duration: 4 weeks
- Primary: Feature adoption within 7 days

Results:

Feature Adoption (7-day):

Control: 14.8% (n=5,234)

Treatment: 31.2% (n=5,187)

Lift: +110.8% (p<0.001)

Feature Engagement (among adopters, 30-day):

Control: 8.3 uses/user

Treatment: 12.7 uses/user

Lift: +53.0% (p<0.001)

Retention (30-day):

Control: 68%

Treatment: 74%

Lift: +8.8% (p<0.001)

Segment Analysis: Benefit strongest for teams (5+ users): +145% adoption vs. +65% for individual users

Business Impact: Teams using collaborative features have 2.3x higher LTV. Projected \$2.4M incremental annual revenue from increased adoption.

Decision: Ship, with personalization by account size

Lessons:

- Interactive onboarding > passive tutorials
- Segment by use case (teams vs. individuals)
- Long-term impact monitoring essential (30-day retention)
- Feature adoption drives downstream value (retention, LTV)

11.8.2 Case Study 8: Mobile App Performance Optimization

Context: Social app noticed 25% drop-off during signup on mobile (vs. 10% on desktop). Engineering team optimized signup flow to reduce API calls and improve perceived performance.

Technical Changes:

- Reduced network requests from 7 to 3
- Implemented optimistic UI updates
- Added skeleton screens during loading
- Prefetched next step data

Results:

Signup Completion:

Control: 75.2%
Treatment: 83.8%
Lift: +11.4% ($p < 0.001$)

Time to Complete:

Control: 187s (median)
Treatment: 142s (median)
Lift: -24.1% ($p < 0.001$)

30-Day Retention:

Control: 42%
Treatment: 45%
Lift: +7.1% ($p = 0.003$)

Cost-Benefit:

- Engineering cost: 3 weeks (\$60K loaded cost)
- Incremental signups: +8.6 percentage points
- Annual impact: +425K new users $\rightarrow$ +18K retained $\rightarrow$ \$1.8M value (est.)
- ROI: 2,900%

Lesson: Mobile performance directly impacts conversion and retention. "Perceived performance" (skeleton screens, optimistic UI) as important as actual speed.

11.9 Content and Media

Content platforms optimize recommendations, engagement, and personalization.

11.9.1 Case Study 9: Video Recommendation Algorithm

Context: Streaming platform's recommendation engine used collaborative filtering. Data science team developed hybrid model combining collaborative filtering + content features + deep learning.

Design:

- A/A test first (validate randomization)
- A/B test: 5M users per variant, 4 weeks
- Metrics: CTR, watch time, satisfaction, next-day return rate

Results:

Metric	Control (CF)	Treatment (Hybrid)	Lift
Recommendation CTR	5.5%	6.4%	+16.4%**
Watch time (hrs/week)	12.5	13.2	+5.6%**
Completion rate	68%	71%	+4.4%**
Satisfaction (1-5)	4.1	4.2	+2.4%**
Next-day return	62%	64%	+3.2%**

Infrastructure Consideration: New model 3x more computationally expensive

Cost-Benefit Analysis:

Revenue Impact:

Monthly active users: 50M
 Watch time lift: 0.7 hrs/week/user
 Value per hour: \$0.12 (advertising/subscriptions)
 Annual revenue lift: \$218.4M

Costs:

Compute: +\$15M/year
 Migration: \$5M one-time
 Total Year 1: \$20M

ROI: 992% (first year)

Decision: Gradual rollout (25% → 50% → 100%) to validate infrastructure scaling

Post-Launch: After 6 months, engagement lift sustained at +4.8%, infrastructure costs came in 15% under estimate due to optimizations

Lessons:

- A/A testing critical for algorithmic changes
- Cost-benefit must include infrastructure
- Gradual rollout for high-risk changes
- Long-term monitoring for degradation (model drift, novelty effects)

11.9.2 Case Study 10: News Feed Ranking (Partial Failure)

Context: Social network tested "engagement-optimized" feed ranking to increase time on site.

Hypothesis: Showing content likely to generate interactions (comments, shares) will increase engagement.

Results:

Positive:

Time on site: +8.2% ($p < 0.001$)
 Interactions: +12.4% ($p < 0.001$)
 Daily actives: +2.1% ($p < 0.001$)

Negative:

User satisfaction: -5.3% ($p < 0.001$)
 User reports of "low quality": +23% ($p < 0.001$)
 Survey sentiment: -8 points ($p < 0.001$)

Insight: Algorithm optimized for "engagement" but surfaced divisive, clickbait content that users disliked.

Decision: Do NOT ship. Revised to optimize for "meaningful interactions" (comments from close connections, long-form engagement) rather than raw interactions.

Lesson: Metric choice matters profoundly. Optimizing wrong metric can improve numbers while harming product. Must balance engagement with quality, satisfaction, and long-term health.

11.10 Lessons Learned: Meta-Insights Across Cases

Analyzing patterns across all case studies reveals systematic lessons for experimentation programs.

11.10.1 Successes: What Worked

1. **Clear hypothesis and success criteria (Cases 1-9):** Pre-specified metrics and decision thresholds prevented p-hacking and scope creep
2. **Segment analysis revealed key insights (Cases 2, 5, 7):** Overall averages masked important heterogeneity. Mobile vs. desktop, new vs. returning, teams vs. individuals all showed different treatment effects
3. **Guardrail metrics prevented harm (Cases 3, 5, 10):** Monitoring engagement, satisfaction, and retention caught negative side effects before launch
4. **Business impact quantification enabled decision-making (All cases):** Translating statistical significance to revenue/ROI provided clear go/no-go criteria
5. **Gradual rollouts reduced risk (Case 9):** For high-impact or infrastructure-heavy changes, phased launches caught issues early
6. **Long-term monitoring caught degradation (Cases 5, 9):** Test effects often degrade due to novelty, habituation, or changed context

7. **Iterative experimentation builds knowledge (Case 9):** Staged approach (A/A $\rightarrow$ validate assumption $\rightarrow$ optimize model) reduced risk and improved outcomes

11.10.2 Failures: What to Avoid

1. **Optimizing wrong metric (Cases 6, 10):** Improving intermediate metrics (signups, time on site) without checking business outcome (qualified leads, satisfaction) led to poor decisions
2. **Ignoring full funnel (Case 4):** Improving conversion rate at one step while increasing abandonment earlier yielded net negative results
3. **Implementation bugs (Case 5):** 0.3% of users received broken personalization tokens—testing code quality as important as testing hypothesis
4. **Insufficient guardrails (Case 10):** Not monitoring user satisfaction allowed algorithm to optimize engagement at expense of experience
5. **Novelty effects (Cases 1, 5):** Initial test results often overstate long-term impact due to user curiosity or habituation
6. **Infrastructure underestimation (Case 9):** New recommendation model cost 3x more to run—must model operational costs

11.10.3 Organizational Challenges

1. **HiPPO (Highest Paid Person’s Opinion):** Executives overriding data based on intuition
Solution: Establish decision frameworks upfront, require documentation for data overrides, educate leadership on methodology
2. **Analysis paralysis:** Running experiments too long without making decisions
Solution: Set decision deadlines, accept uncertainty, define minimum confidence thresholds
3. **P-hacking:** Analyzing data many ways until finding significance
Solution: Pre-register analysis plans, apply multiple testing corrections, document all analyses
4. **Coordination costs:** Multiple teams running overlapping experiments
Solution: Centralized experiment registry, traffic allocation management, cross-team review process
5. **Skill gaps:** PMs, engineers lacking statistical training
Solution: Training programs, self-service tools with correct defaults, statistical review process

11.10.4 Scaling Experiment Programs

Mature organizations build systematic capabilities:

1. Infrastructure:

- Centralized experimentation platform (randomization, tracking, analysis)
- Metric pipeline with automated computation and variance estimation
- Self-service dashboards with correct statistical methods
- Experiment registry for meta-analysis and knowledge sharing

2. Process:

- Hypothesis template requiring clear success criteria
- Statistical review before launch
- Decision framework (ship/iterate/kill criteria)
- Standardized documentation and results sharing
- Post-launch monitoring plans

3. Culture:

- Leadership championing data-driven decisions
- Training programs for PMs, engineers, analysts
- Celebrating learning (not just "wins")
- Transparent sharing of results and failures
- Institutionalized skepticism and intellectual honesty

11.10.5 Industry-Specific Adaptations

E-commerce:

- Challenge: High-value, low-frequency transactions → high variance
- Solution: Variance reduction (CUPED, stratification), longer tests, revenue-per-user metrics

Social Media:

- Challenge: Network effects and interference between users
- Solution: Cluster randomization (geo, time), network-aware analysis, switchback designs

B2B SaaS:

- Challenge: Long sales cycles, account-level vs. user-level effects
- Solution: Leading indicators, hierarchical models, longer experiments, qualitative research integration

Content Platforms:

- Challenge: Quality vs. engagement trade-offs, creator ecosystem effects
- Solution: Multi-objective optimization, creator metrics as guardrails, long-term cohort tracking

11.11 Common Anti-Patterns and How to Avoid Them

11.11.1 Anti-Pattern 1: P-Hacking

Definition: Manipulating analysis until finding statistical significance.

Manifestations:

- Testing many metrics without correction
- Stopping experiment early when results look good
- Trying different segmentations until finding significance
- Removing "outliers" selectively

Example: A team ran an email subject line test. Primary metric (open rate) showed no effect. They then tested click rate, conversion rate, revenue per user, time to conversion, and found one significant result (time to conversion, $p = 0.04$) and declared success.

Why it's wrong: Testing 5 metrics with $\alpha = 0.05$ gives $1 - (0.95)^5 = 22.6\%$ false positive rate.

Prevention:

- Pre-register hypotheses and analysis plan
- Apply Bonferroni or FDR correction for multiple metrics
- Use sequential testing methods if monitoring continuously
- Document all analyses, including non-significant results

11.11.2 Anti-Pattern 2: HiPPO (Highest Paid Person's Opinion)

Definition: Overriding experimental evidence based on executive intuition or seniority.

Example: An e-commerce A/B test showed that removing a product feature decreased conversion by 8% ($p < 0.001$). VP insisted on shipping anyway because "customers told me they wanted this."

Why it's problematic: Undermines experimentation culture, leads to poor decisions, demoralizes data teams.

Prevention:

- Establish decision-making framework upfront
- Educate leadership on statistical methodology
- Create "escape valves" for overriding data (with documentation)
- Balance quantitative data with qualitative insights systematically
- Foster culture where challenging authority with data is encouraged

11.11.3 Anti-Pattern 3: Analysis Paralysis

Definition: Running experiments indefinitely without making decisions.

Example: Team ran checkout test for 8 weeks (planned duration: 2 weeks). Results were conclusive after 2 weeks, but team kept running "to be sure," delaying feature launch.

Prevention:

- Set clear success criteria before starting
- Establish decision deadlines
- Accept that uncertainty is inherent
- Define minimum confidence levels
- Implement decision frameworks (when to ship, when to iterate, when to kill)

11.11.4 Anti-Pattern 4: Novelty Effect Ignorance

Definition: Ignoring that users may respond differently initially than long-term.

Example: Redesigned navigation initially showed +15% engagement. After 4 weeks, effect degraded to +2

Prevention:

- Run experiments long enough to capture habituation (typically 2-4 weeks minimum)
- Analyze early vs. late periods separately
- Use cohort analysis to distinguish novelty from sustained effects
- For major changes, maintain long-term holdout groups

11.12 Industry-Specific Considerations

11.12.1 E-commerce

Unique challenges:

- High-value, low-frequency transactions (variance)
- Seasonality (holidays, sales events)
- Multi-device journeys (attribution)
- Returning customer effects

Best practices:

- Use pre-experiment covariates (CUPED) to reduce variance
- Stratify by customer type (new/returning) and device
- Run experiments across seasonal periods or control for calendar effects
- Monitor cart abandonment as key guardrail
- Track long-term metrics (LTV, repeat purchase rate)

11.12.2 Social Media

Unique challenges:

- Network effects and interference
- Viral content creates non-independence
- User-generated content variability
- Engagement vs. time spent vs. well-being trade-offs

Best practices:

- Use cluster randomization (geo, time-based)
- Account for network structure in analysis
- Monitor both engagement and well-being metrics
- Use switchback designs for platform-level changes
- Be cautious of metric gaming (e.g., optimizing for clicks vs. value)

11.12.3 SaaS/B2B

Unique challenges:

- Long sales cycles
- Account-level vs. user-level randomization
- Low sample sizes (enterprise customers)
- Contractual obligations limiting experimentation

Best practices:

- Use hierarchical models for account/user nesting
- Focus on leading indicators (trial engagement, feature adoption)
- Stratify by account size/value
- Supplement with qualitative research (customer interviews)
- Run longer experiments to capture full conversion cycle

11.12.4 Healthcare/EdTech

Unique challenges:

- Ethical constraints on randomization
- Long-term outcomes more important than short-term
- Regulatory compliance (HIPAA, FERPA, IRB)
- Equity and access considerations

Best practices:

- Prioritize equipoise (clinical uncertainty) for ethical randomization
- Use stepped-wedge designs (everyone gets treatment eventually)
- Implement robust informed consent processes
- Monitor equity metrics across demographic groups
- Focus on validated outcome measures
- Consider long-term follow-up studies

11.13 Building an Experimentation Culture: Meta-Lessons

Beyond individual experiments, successful organizations build systematic experimentation capabilities:

11.13.1 Infrastructure and Tooling

- **Experimentation platform:** Centralized system for randomization, tracking, analysis
- **Metric pipeline:** Automated computation of key metrics with proper variance estimation
- **Analysis tools:** Self-service dashboards with correct statistical methods
- **Experiment registry:** Database of all experiments for meta-analysis

11.13.2 Process and Governance

- **Hypothesis template:** Structured approach to defining experiments
- **Review process:** Statistical review before launch
- **Decision framework:** Clear criteria for ship/no-ship/iterate
- **Documentation:** Consistent recording of design, results, decisions

11.13.3 Culture and Education

- **Leadership buy-in:** Executives champion data-driven decisions
- **Training programs:** Educate PMs, engineers, analysts on methodology
- **Celebrate learning:** Reward well-designed experiments, not just "wins"
- **Transparent communication:** Share results widely, including failures

11.14 Summary

Real-world case studies provide invaluable lessons:

- **Google:** Technical performance directly impacts user behavior; always monitor guardrails
- **E-commerce:** Segment analysis reveals heterogeneous effects; weight by strategic priorities
- **Netflix:** Iterative experimentation validates assumptions incrementally; personalization adds value
- **Paywall failure:** Local optimization can harm global objectives; measure full funnel
- **Anti-patterns:** Avoid p-hacking, HiPPO, analysis paralysis; establish processes to prevent
- **Industry context:** Adapt methods to specific challenges (network effects, seasonality, regulations)

Learning from both successes and failures builds experimentation expertise. The most mature organizations view experiments as learning opportunities, documenting and sharing insights to improve continuously.

11.15 Further Reading

- Kohavi et al. [2020]: Extensive case studies from Microsoft, Amazon, Google, with detailed analysis
- Christian and Griffiths [2016]: Exploration-exploitation trade-offs in practice
- Company engineering blogs:
 - Netflix TechBlog: experimentation at scale
 - Airbnb Engineering: experiment analysis frameworks
 - Booking.com: 1,000+ experiments per year
 - Spotify Research: sequential testing and bandits

11.16 Exercises

11.16.1 Case Study Analysis

1. **Post-Mortem Template:** Create a structured template for analyzing experiments. Include sections for: hypothesis, design, implementation, results, decision, lessons learned. Apply to a recent experiment from your organization.
2. **Replication:** Reproduce the Google case study analysis using the provided code. Modify parameters to explore sensitivity: What if load time increased by 50ms instead of 500ms? What if engagement elasticity were -0.5 instead of -1.0?
3. **Segment Discovery:** Using the e-commerce checkout code, add a third segment dimension (e.g., product category, geographic region). Analyze three-way interactions. When do you stop slicing data?

11.16.2 Business Impact Calculation

1. For your organization:
 - Define key business metrics (revenue, LTV, retention)
 - Map experiment metrics to business outcomes
 - Create calculator estimating business impact from experimental results
 - Include confidence intervals and break-even analysis
2. **Paywall Re-design:** The failed paywall experiment. Design a better version that balances conversion rate and engagement. Specify success criteria and guardrails.

11.16.3 Anti-Pattern Detection

1. **Audit:** Review 5 recent experiments from your organization. Identify any anti-patterns (p-hacking, missing guardrails, insufficient duration, etc.). Propose corrective actions.
2. **Process Design:** Design a review checklist to prevent common anti-patterns. Include statistical, technical, and business considerations. Specify who reviews, when, and with what authority.

11.16.4 Industry Application

1. Choose your industry and design an experiment addressing industry-specific challenges:
 - E-commerce: Multi-device attribution problem
 - Social media: Network effects mitigation
 - SaaS: Long-sales cycle with leading indicators
 - Healthcare: Ethical randomization with stepped-wedge

Specify design, analysis plan, and decision criteria.

11.16.5 Meta-Analysis Project

1. **Experiment Database:** Create a structured database of past experiments with fields: hypothesis, design, sample size, duration, primary metric, result, decision. Analyze:
 - What proportion of experiments are "successful"?
 - Do success rates differ by experiment type, team, or domain?
 - What factors predict successful experiments?
 - Are sample size and duration adequate on average?

Chapter 12

Ethics and Best Practices

12.1 Learning Objectives

After completing this chapter, readers will be able to:

- Navigate ethical considerations in human subjects experimentation
- Apply principles of informed consent and transparency
- Build organizational experimentation culture
- Establish governance frameworks
- Follow industry best practices

12.2 Ethical Foundations

A/B testing involves human subjects and carries ethical responsibilities.

12.2.1 Core Ethical Principles

1. **Beneficence:** Maximize benefits, minimize harms
2. **Non-maleficence:** Do no harm
3. **Autonomy:** Respect individual agency
4. **Justice:** Ensure fair distribution of benefits and burdens
5. **Transparency:** Be open about practices

12.2.2 The Belmont Report

While focused on medical research, the Belmont Report's principles apply to digital experimentation:

- **Respect for persons:** Treat individuals as autonomous agents
- **Beneficence:** Maximize possible benefits and minimize possible harms
- **Justice:** Fair distribution of research benefits and burdens

12.3 Informed Consent

12.3.1 Consent in Digital Experiments

Unlike medical trials, digital A/B tests typically operate under:

General consent: Users agree to terms of service that may include experimentation

Challenges:

- Users rarely read terms of service
- Consent may not be truly informed
- Ongoing vs. one-time consent
- Right to opt out

12.3.2 When Explicit Consent is Required

- Testing potentially harmful interventions
- Collecting sensitive personal data beyond normal use
- Significantly changing core functionality
- Research published academically
- Regulated industries (healthcare, finance, education)

12.3.3 Balancing Innovation and Consent

Example 12.1 (Acceptable Implicit Consent). Testing button colors, layout variations, recommendation algorithms—changes within normal product optimization scope.

Example 12.2 (Requiring Explicit Consent). Testing emotional manipulation, significantly degraded experiences, collection of biometric data.

12.4 Ethical Guardrails

12.4.1 Minimal Risk Standard

Experiments should pose no more than minimal risk—risks not greater than encountered in daily life or routine tests.

Risk assessment questions:

1. Could this experiment cause physical, psychological, or economic harm?
2. Are vulnerable populations disproportionately affected?
3. Is there potential for manipulation or deception?
4. Could this damage user trust or brand reputation?
5. Are there privacy or data security concerns?

12.4.2 Experimentation Review Board

Large organizations should establish review processes:

Composition:

- Product leaders
- Legal/compliance representatives
- Ethics officer or designated ethicist
- Data scientists
- Privacy/security experts

Responsibilities:

- Review high-risk experiments
- Establish ethical guidelines
- Provide guidance on edge cases
- Monitor incidents and update policies

12.4.3 Automated Guardrails

Implement technical controls:

Listing 12.1: Automated Ethical Guardrails

```

1 class ExperimentEthicsChecker:
2     """Automated ethics and safety checks."""
3
4     def __init__(self):
5         self.high_risk_keywords = [
6             'children', 'minors', 'emotion', 'manipulation',
7             'addiction', 'health', 'safety', 'security'
8         ]
9         self.sensitive_metrics = [
10            'mental_health', 'addiction_score', 'emotional_state'
11        ]
12
13     def review_experiment(self, experiment_config):
14         """
15         Automated ethics review of experiment configuration.
16
17         Returns:
18         -----
19         dict: Review results and recommendations
20         """
21         flags = []
22
23         # Check for high-risk keywords in description
24         description = experiment_config.get('description', '').lower()
25         for keyword in self.high_risk_keywords:
26             if keyword in description:
27                 flags.append({

```

```

28         'severity': 'high',
29         'type': 'high_risk_keyword',
30         'message': f"High-risk_keyword_detected:_{keyword}",
31         'action': 'Requires_human_review'
32     })
33
34     # Check for sensitive metrics
35     metrics = experiment_config.get('metrics', {})
36     all_metrics = (metrics.get('primary', []) +
37                   metrics.get('secondary', []))
38
39     for metric in all_metrics:
40         if metric in self.sensitive_metrics:
41             flags.append({
42                 'severity': 'high',
43                 'type': 'sensitive_metric',
44                 'message': f"Sensitive_metric:_{metric}",
45                 'action': 'Requires_ethics_board_approval'
46             })
47
48     # Check for degraded experiences
49     variants = experiment_config.get('variants', [])
50     for variant in variants:
51         if variant.get('intentionally_degraded', False):
52             flags.append({
53                 'severity': 'medium',
54                 'type': 'degraded_experience',
55                 'message': f"Variant_{variant['name']}_intentionally_degraded",
56                 'action': 'Document_justification'
57             })
58
59     # Check target population
60     targeting = experiment_config.get('targeting', {})
61     if 'age_max' in targeting and targeting['age_max'] < 18:
62         flags.append({
63             'severity': 'critical',
64             'type': 'minors_targeted',
65             'message': 'Experiment_targets_minors',
66             'action': 'Ethics_board_approval_and_parental_consent_required'
67         })
68
69     return {
70         'approved': len([f for f in flags if f['severity'] in
71                        ['high', 'critical']]) == 0,
72         'flags': flags,
73         'recommendation': self._get_recommendation(flags)
74     }
75
76     def _get_recommendation(self, flags):
77         """Generate_recommendation_based_on_flags."""
78         if not flags:
79             return "No_ethical_concerns_detected._Approved_for_launch."
80
81         critical = [f for f in flags if f['severity'] == 'critical']
82         high = [f for f in flags if f['severity'] == 'high']

```

```

83
84     if critical:
85         return "BLOCKED: Critical ethical issues. Ethics board
            review required."
86     elif high:
87         return "CAUTION: High-risk experiment. Manager approval
            required."
88     else:
89         return "PROCEED WITH CAUTION: Document ethical
            considerations."
90
91 # Example usage
92 experiment = {
93     'description': 'Testing notification frequency to increase
            engagement',
94     'metrics': {
95         'primary': 'daily_active_users',
96         'secondary': ['session_duration', 'notification_clicks']
97     },
98     'variants': [
99         {'name': 'control', 'notification_freq': 1},
100        {'name': 'treatment', 'notification_freq': 10}
101    ]
102 }
103
104 checker = ExperimentEthicsChecker()
105 result = checker.review_experiment(experiment)
106
107 print(f"Approved: {result['approved']}")
108 print(f"Recommendation: {result['recommendation']}")
109 for flag in result['flags']:
110     print(f"{flag['message']}")

```

12.5 Privacy and Data Protection

12.5.1 Data Minimization

Collect only data necessary for the experiment:

- Define metrics clearly and collect only those
- Aggregate early when possible
- Delete raw data after analysis
- Anonymize or pseudonymize user identifiers

12.5.2 Regulatory Compliance

GDPR (General Data Protection Regulation):

- Right to access: Users can request their data
- Right to erasure: "Right to be forgotten"

- Data protection by design
- Lawful basis for processing

CCPA (California Consumer Privacy Act):

- Right to know what data is collected
- Right to delete personal information
- Right to opt out of sale
- Non-discrimination for exercising rights

12.5.3 Internal Data Governance

1. **Access controls:** Limit who can view individual-level data
2. **Audit trails:** Log data access and analysis
3. **Retention policies:** Delete data after specified period
4. **Anonymization:** Remove personally identifiable information
5. **Differential privacy:** Add noise to protect individual privacy

12.6 Regulatory Compliance

Operating in regulated environments requires understanding and complying with industry-specific requirements for experimentation.

12.6.1 GDPR and Data Protection Laws

Key GDPR requirements for experiments:

1. **Lawful basis:** Identify legal basis for processing
 - Consent: Explicit agreement to participate
 - Legitimate interest: Balancing test for product improvement
 - Contract: Necessary for service delivery
2. **Data minimization:** Collect only necessary data
3. **Purpose limitation:** Use data only for stated purpose
4. **Storage limitation:** Delete data when no longer needed
5. **Data subject rights:**
 - Right to access experimental data
 - Right to erasure ("right to be forgotten")
 - Right to object to processing

- Right to data portability

6. Data Protection Impact Assessment (DPIA): Required for high-risk processing

Listing 12.2: GDPR Compliance Checker

```

1 from enum import Enum
2 from dataclasses import dataclass
3 from typing import List, Optional
4
5 class LegalBasis(Enum):
6     """GDPR legal basis for processing."""
7     CONSENT = "consent"
8     CONTRACT = "contract"
9     LEGAL_OBLIGATION = "legal_obligation"
10    VITAL_INTERESTS = "vital_interests"
11    PUBLIC_TASK = "public_task"
12    LEGITIMATE_INTEREST = "legitimate_interest"
13
14 class DataCategory(Enum):
15     """Categories of personal data."""
16     STANDARD = "standard" # Name, email, etc.
17     SPECIAL = "special" # Race, health, religion, etc.
18     CRIMINAL = "criminal" # Criminal convictions
19
20 @dataclass
21 class GDPRComplianceAssessment:
22     """Assess GDPR compliance for A/B test."""
23
24     experiment_name: str
25     legal_basis: LegalBasis
26     data_collected: List[str]
27     data_categories: List[DataCategory]
28     retention_period_days: int
29     third_party_sharing: bool
30     automated_decision_making: bool
31     children_targeted: bool
32
33     def assess_dpia_required(self) -> bool:
34         """
35         Determine if Data Protection Impact Assessment is required.
36
37         DPIA required if:
38         - Processing special category data at scale
39         - Systematic monitoring of public areas
40         - Automated decision-making with legal/significant effects
41         - Processing children's data at scale
42         """
43         high_risk_factors = []
44
45         # Special category data
46         if DataCategory.SPECIAL in self.data_categories:
47             high_risk_factors.append("Processing special category data")
48
49         # Automated decisions with significant impact
50         if self.automated_decision_making:

```

```

51         high_risk_factors.append("Automated_decision-making")
52
53     # Children's data
54     if self.children_targeted:
55         high_risk_factors.append("Targeting_children")
56
57     # Large scale processing
58     if self.retention_period_days > 365:
59         high_risk_factors.append("Long-term_data_retention")
60
61     return len(high_risk_factors) >= 2
62
63     def check_compliance(self) -> Dict:
64         """Comprehensive GDPR compliance check."""
65         issues = []
66         warnings = []
67
68         # Check legal basis is appropriate
69         if (DataCategory.SPECIAL in self.data_categories and
70             self.legal_basis not in [LegalBasis.CONSENT,
71                                     LegalBasis.LEGAL_OBLIGATION]):
72             issues.append({
73                 'severity': 'critical',
74                 'issue': 'Invalid_legal_basis_for_special_category_data',
75                 'remedy': 'Obtain_explicit_consent_or_identify_legal_obligation'
76             })
77
78         # Check retention period
79         if self.retention_period_days > 730: # 2 years
80             warnings.append({
81                 'severity': 'medium',
82                 'issue': f'Long_retention_period:{self.retention_period_days}days',
83                 'remedy': 'Document_necessity_for_extended_retention'
84             })
85
86         # Check children's data
87         if self.children_targeted and self.legal_basis != LegalBasis.CONSENT:
88             issues.append({
89                 'severity': 'critical',
90                 'issue': 'Processing_children_data_without_parental_consent',
91                 'remedy': 'Obtain_verifiable_parental_consent'
92             })
93
94         # Check third-party sharing
95         if self.third_party_sharing:
96             warnings.append({
97                 'severity': 'high',
98                 'issue': 'Third-party_data_sharing_detected',
99                 'remedy': 'Ensure_data_processing_agreements_in_place'
100             })
101
102         # DPIA assessment
103         dpia_required = self.assess_dpia_required()

```

```

104         if dpia_required:
105             warnings.append({
106                 'severity': 'high',
107                 'issue': 'Data_Protection_Impact_Assessment_required',
108                 'remedy': 'Conduct_DPIA_before_launching_experiment'
109             })
110
111         return {
112             'compliant': len(issues) == 0,
113             'dpia_required': dpia_required,
114             'critical_issues': issues,
115             'warnings': warnings,
116             'recommendation': self._get_recommendation(issues, warnings
117         )
118
119     def _get_recommendation(self, issues: List, warnings: List) -> str:
120         """Generate compliance recommendation."""
121         if issues:
122             return "BLOCKED: Critical GDPR compliance issues. Legal review required."
123         elif warnings:
124             return f"PROCEED WITH CAUTION: {len(warnings)} compliance warnings. Document mitigations."
125         else:
126             return "COMPLIANT: No GDPR issues detected."
127
128     # Example usage
129     assessment = GDPRComplianceAssessment(
130         experiment_name="Personalized_Health_Recommendations_Test",
131         legal_basis=LegalBasis.LEGITIMATE_INTEREST,
132         data_collected=["user_id", "health_conditions", "medications"],
133         data_categories=[DataCategory.SPECIAL], # Health data
134         retention_period_days=90,
135         third_party_sharing=False,
136         automated_decision_making=True,
137         children_targeted=False
138     )
139
140     result = assessment.check_compliance()
141     print(f"Compliant: {result['compliant']}")
142     print(f"DPIA Required: {result['dpia_required']}")
143     print(f"Recommendation: {result['recommendation']}")

```

12.6.2 HIPAA (Healthcare)

For experiments involving Protected Health Information (PHI):

HIPAA requirements:

- **Authorization:** Obtain patient authorization for research use
- **De-identification:** Remove 18 identifiers or use expert determination
- **Limited data sets:** Remove direct identifiers, use data use agreement
- **Business Associate Agreements:** Required for third-party processors

- **Security Rule:** Technical safeguards for ePHI

Safe harbor de-identification (remove all):

- | | |
|---|---------------------------------------|
| 1. Names | 10. Account numbers |
| 2. Geographic subdivisions smaller than state | 11. Certificate/license numbers |
| 3. Dates (except year) | 12. Vehicle identifiers |
| 4. Phone numbers | 13. Device identifiers/serial numbers |
| 5. Fax numbers | 14. URLs |
| 6. Email addresses | 15. IP addresses |
| 7. Social Security numbers | 16. Biometric identifiers |
| 8. Medical record numbers | 17. Full-face photos |
| 9. Health plan numbers | 18. Any unique identifying number |

12.6.3 COPPA (Children's Privacy)

Testing products used by children under 13 in the US:

COPPA requirements:

- **Parental consent:** Obtain verifiable consent before collecting data
- **Clear privacy notice:** Disclose data collection practices
- **Data minimization:** Collect only necessary information
- **Parental access:** Allow parents to review and delete data
- **Security:** Reasonable safeguards for children's data

Age gate implementation:

- Neutral age screening (don't encourage lying)
- Age verification mechanisms
- Separate consent flows for under-13 users
- Parental email verification

12.6.4 Financial Services Regulations

Experiments in banking, lending, or financial services:

Key regulations:

- **Fair Lending Laws:** Ensure non-discrimination in credit decisions
- **FCRA (Fair Credit Reporting Act):** Accuracy in credit decisions

- **GLBA (Gramm-Leach-Bliley Act):** Privacy of financial information
- **Dodd-Frank Act:** Consumer protection in financial products

Testing lending algorithms:

1. **Disparate impact analysis:** Test for discrimination across protected groups
2. **Adverse action notices:** Inform consumers of credit denials
3. **Model governance:** Document and validate models
4. **Fair lending compliance:** Ensure equal treatment

Listing 12.3: Fair Lending Analysis

```

1 import pandas as pd
2 import numpy as np
3 from scipy import stats
4
5 class FairLendingAnalyzer:
6     """Analyze experiments for fair lending compliance."""
7
8     def __init__(self, data: pd.DataFrame):
9         self.data = data
10        self.protected_classes = ['race', 'gender', 'age_group', 'national_origin']
11
12        def disparate_impact_test(self, outcome_col: str = 'approved',
13                                threshold: float = 0.80) -> Dict:
14            """
15            Test for disparate impact using the 80% rule.
16
17            The 80% rule: Selection rate for protected group should be
18            at least 80% of selection rate for most favored group.
19            """
20            results = {}
21
22            for protected_class in self.protected_classes:
23                if protected_class not in self.data.columns:
24                    continue
25
26                # Calculate approval rates by group
27                approval_rates = (self.data.groupby(protected_class)[
28                                outcome_col]
29                                .mean()
30                                .sort_values(ascending=False))
31
32                reference_rate = approval_rates.iloc[0] # Highest rate
33
34                # Calculate ratios
35                ratios = approval_rates / reference_rate
36
37                # Identify disparate impact
38                failing_groups = ratios[ratios < threshold]
39
40                results[protected_class] = {
41                    'approval_rates': approval_rates.to_dict(),

```

```

41         'reference_group': approval_rates.index[0],
42         'reference_rate': reference_rate,
43         'ratios': ratios.to_dict(),
44         'disparate_impact_detected': len(failing_groups) > 0,
45         'failing_groups': failing_groups.to_dict()
46     }
47
48     return results
49
50     def statistical_significance_test(self, variant_col: str,
51                                     outcome_col: str = 'approved') ->
52                                     Dict:
53         """Test if approval rates differ significantly by variant and
54         protected class."""
55         results = {}
56
57         for protected_class in self.protected_classes:
58             if protected_class not in self.data.columns:
59                 continue
60
61             # Create contingency table
62             contingency = pd.crosstab(
63                 [self.data[variant_col], self.data[protected_class]],
64                 self.data[outcome_col]
65             )
66
67             # Chi-square test
68             chi2, p_value, dof, expected = stats.chi2_contingency(
69                 contingency)
70
71             results[protected_class] = {
72                 'chi2_statistic': chi2,
73                 'p_value': p_value,
74                 'significant': p_value < 0.05,
75                 'contingency_table': contingency.to_dict()
76             }
77
78         return results
79
80     def generate_compliance_report(self, variant_col: str = 'variant')
81     -> str:
82         """Generate fair lending compliance report."""
83         disparate_impact = self.disparate_impact_test()
84         significance = self.statistical_significance_test(variant_col)
85
86         report = ["Fair Lending Compliance Analysis", "="*50, ""]
87
88         # Disparate impact results
89         report.append("Disparate Impact Test (80% Rule):")
90         report.append("-" * 40)
91
92         for protected_class, results in disparate_impact.items():
93             report.append(f"\n{protected_class.upper()}:")
94             report.append(f"    Reference group: {results['reference_
95                 group']}\n"
96                 f"    ({results['reference_rate']:.1%})")
97
98             if results['disparate_impact_detected']:

```

```

94         report.append("\u\u\uWARNING:\uDisparate\uimpact\u Detected!")
95     for group, ratio in results['failing_groups'].items():
96         rate = results['approval_rates'][group]
97         report.append(f"\u\u\u\u{group}:\u{rate:.1%}\u({ratio
          :.1%}\uof\ureference)")
98     else:
99         report.append("\u\u\uPASS:\uNo\udisparate\uimpact\u Detected")
100
101     return "\n".join(report)
102
103 # Example usage
104 # Simulate lending experiment data
105 np.random.seed(42)
106 n = 5000
107
108 data = pd.DataFrame({
109     'variant': np.random.choice(['control', 'treatment'], n),
110     'race': np.random.choice(['white', 'black', 'hispanic', 'asian'], n
111     ),
112     'gender': np.random.choice(['male', 'female'], n),
113     'credit_score': np.random.normal(700, 50, n),
114 })
115
116 # Simulate approval with potential bias
117 data['approved'] = (
118     (data['credit_score'] > 680) &
119     (np.random.random(n) > 0.3) # Base rejection rate
120 ).astype(int)
121
122 analyzer = FairLendingAnalyzer(data)
123 report = analyzer.generate_compliance_report()
124 print(report)

```

12.6.5 Institutional Review Boards (IRBs)

Academic researchers and some industry experiments may require IRB approval:

When IRB review is needed:

- Research involving human subjects
- Intent to publish or present findings
- Federally funded research (in the US)
- Research at regulated institutions
- High-risk experiments

IRB exemptions (US regulations):

1. Research in established educational settings
2. Educational tests, surveys, or observations (with conditions)
3. Benign behavioral interventions with informed consent
4. Secondary research on de-identified data

IRB submission components:

- Protocol description
- Informed consent documents
- Recruitment materials
- Data security plan
- Risk-benefit analysis
- Investigator qualifications

12.6.6 Industry-Specific Compliance Checklist

Industry	Key Regulations	Critical Considerations
Healthcare	HIPAA, FDA	PHI protection, medical device regulations
Financial	FCRA, GLBA, Dodd-Frank	Fair lending, privacy, consumer protection
Education	FERPA, COPPA	Student privacy, parental consent
Telecommunications	TCPA, CPNI	Consent for communications, call data privacy
E-commerce	FTC Act, state laws	Deceptive practices, consumer protection
Gaming	COPPA, gambling laws	Age verification, responsible gaming

12.7 Transparency and Communication**12.7.1 Internal Transparency****Experiment registry:**

- Centralized catalog of all experiments
- Pre-registration of hypotheses
- Documentation of methods and results
- Sharing of negative results

Benefits:

- Prevents duplicate testing
- Enables meta-analysis
- Builds institutional knowledge
- Reduces publication bias

12.7.2 External Transparency

User-facing:

- Clear terms of service
- Privacy policies mentioning experimentation
- Opt-out mechanisms where appropriate
- Transparency reports

Academic:

- Publishing findings in peer-reviewed venues
- Open-sourcing methodologies
- Sharing datasets (when appropriate)
- Contributing to field knowledge

12.8 Experimental Integrity

Maintaining scientific rigor and avoiding statistical malpractice is essential for trustworthy experimentation.

12.8.1 Pre-registration and Documentation

Why pre-registration matters:

- Prevents p-hacking and HARKing (Hypothesizing After Results are Known)
- Creates accountability for experimental design
- Enables distinction between confirmatory and exploratory analysis
- Builds institutional knowledge

What to pre-register:

1. Research question and hypothesis
2. Primary and secondary metrics
3. Planned sample size and duration
4. Statistical analysis plan
5. Subgroup analyses (if planned)
6. Decision criteria for shipping

Listing 12.4: Pre-registration Framework

```

1 from dataclasses import dataclass, field
2 from typing import List, Dict, Optional
3 from datetime import datetime
4 import json
5 import hashlib
6
7 @dataclass
8 class ExperimentPreRegistration:
9     """
10     Pre-registration document for A/B test experiments.
11     Creates immutable record of experimental design before execution.
12     """
13     # Basic information
14     experiment_id: str
15     experiment_name: str
16     owner: str
17     registration_date: datetime = field(default_factory=datetime.now)
18
19     # Research design
20     hypothesis: str
21     rationale: str
22
23     # Metrics
24     primary_metric: str
25     primary_metric_definition: str
26     secondary_metrics: List[str] = field(default_factory=list)
27     guardrail_metrics: List[str] = field(default_factory=list)
28
29     # Statistical design
30     minimum_detectable_effect: float
31     significance_level: float = 0.05
32     statistical_power: float = 0.80
33     planned_sample_size: int
34     planned_duration_days: int
35
36     # Analysis plan
37     statistical_test: str # e.g., "two-sample t-test", "mann-whitney"
38     multiple_testing_correction: str # e.g., "bonferroni", "benjamini-
39     hochberg"
40     planned_subgroups: List[str] = field(default_factory=list)
41
42     # Decision criteria
43     success_criteria: str
44     launch_threshold: str # e.g., "p < 0.05 and lift > 2%"
45
46     # Additional fields
47     exclusion_criteria: Optional[str] = None
48     randomization_unit: str = "user_id"
49     traffic_allocation: Dict[str, float] = field(default_factory=dict)
50
51     def generate_hash(self) -> str:
52         """Generate cryptographic hash of pre-registration for tamper
53         detection."""
54         # Serialize registration to JSON
55         reg_dict = {
56             'experiment_id': self.experiment_id,
57             'hypothesis': self.hypothesis,

```

```

56         'primary_metric': self.primary_metric,
57         'significance_level': self.significance_level,
58         'planned_sample_size': self.planned_sample_size,
59         'statistical_test': self.statistical_test,
60         'success_criteria': self.success_criteria
61     }
62
63     # Create hash
64     reg_json = json.dumps(reg_dict, sort_keys=True)
65     return hashlib.sha256(reg_json.encode()).hexdigest()
66
67     def to_dict(self) -> Dict:
68         """Convert to dictionary for storage."""
69         return {
70             'experiment_id': self.experiment_id,
71             'experiment_name': self.experiment_name,
72             'owner': self.owner,
73             'registration_date': self.registration_date.isoformat(),
74             'hypothesis': self.hypothesis,
75             'rationale': self.rationale,
76             'primary_metric': self.primary_metric,
77             'primary_metric_definition': self.primary_metric_definition
78             ,
79             'secondary_metrics': self.secondary_metrics,
80             'guardrail_metrics': self.guardrail_metrics,
81             'mde': self.minimum_detectable_effect,
82             'alpha': self.significance_level,
83             'power': self.statistical_power,
84             'sample_size': self.planned_sample_size,
85             'duration_days': self.planned_duration_days,
86             'statistical_test': self.statistical_test,
87             'multiple_testing_correction': self.multiple_testing_
88                 correction,
89             'planned_subgroups': self.planned_subgroups,
90             'success_criteria': self.success_criteria,
91             'launch_threshold': self.launch_threshold,
92             'hash': self.generate_hash()
93         }
94
95     # Example usage
96     registration = ExperimentPreRegistration(
97         experiment_id="EXP-2024-042",
98         experiment_name="Checkout Button Color Test",
99         owner="product_team@company.com",
100         hypothesis="Changing checkout button from blue to green will
101             increase conversion rate",
102         rationale="Green signals 'go' and may reduce friction in purchase
103             decision",
104         primary_metric="conversion_rate",
105         primary_metric_definition="Orders / Sessions on checkout page",
106         secondary_metrics=["revenue_per_session", "cart_abandonment_rate"],
107         guardrail_metrics=["page_load_time", "error_rate"],
108         minimum_detectable_effect=0.02, # 2% relative lift
109         planned_sample_size=10000,
110         planned_duration_days=14,
111         statistical_test="two-proportion z-test",
112         multiple_testing_correction="bonferroni",
113         planned_subgroups=["device_type", "user_segment"],

```

```

110     success_criteria="Statistically significant lift in conversion rate
        with no degradation in guardrails",
111     launch_threshold="p < 0.05, lift > 2%, guardrails within 1% of
        baseline",
112     traffic_allocation={"control": 0.5, "treatment": 0.5}
113 )
114
115 # Save with tamper-proof hash
116 registration_doc = registration.to_dict()
117 print(f"Pre-registration hash: {registration_doc['hash']}")

```

12.8.2 Avoiding P-Hacking and Data Dredging

Common p-hacking practices to avoid:

1. **Peeking:** Looking at results before planned sample size and stopping early if significant
2. **Metric switching:** Declaring different metric as primary post-hoc
3. **Subgroup fishing:** Testing many subgroups without correction
4. **Outlier removal:** Removing data points to achieve significance
5. **Optional stopping:** Repeatedly checking for significance
6. **Flexible analysis:** Trying multiple statistical tests until one is significant

Safeguards against p-hacking:

Listing 12.5: P-Hacking Detection and Prevention

```

1 class ExperimentIntegrityMonitor:
2     """Monitor and prevent statistical malpractice in experiments."""
3
4     def __init__(self, pre_registration: ExperimentPreRegistration):
5         self.pre_reg = pre_registration
6         self.analysis_log = []
7
8     def log_analysis(self, analysis_type: str, metric: str,
9                     p_value: float, notes: str):
10        """Log every statistical analysis performed."""
11        self.analysis_log.append({
12            'timestamp': datetime.now(),
13            'analysis_type': analysis_type,
14            'metric': metric,
15            'p_value': p_value,
16            'notes': notes
17        })
18
19    def check_analysis_integrity(self) -> Dict:
20        """
21        Check for signs of p-hacking or fishing.
22
23        Returns:
24        -----
25        dict: Integrity assessment and warnings

```



```

26  """
27  warnings = []
28
29  # Check for multiple looks at data
30  primary_analyses = [a for a in self.analysis_log
31                      if a['metric'] == self.pre_reg.primary_metric
32                      ]
33  if len(primary_analyses) > 1:
34      warnings.append({
35          'severity': 'high',
36          'issue': 'multiple_testing',
37          'message': f"Primary_metric_analyzed_{len(primary_
38                  analyses)}_times."
39                  f"May_indicate_peeking_or_optional_stopping."
40      })
41
42  # Check for fishing across metrics
43  unique_metrics = set(a['metric'] for a in self.analysis_log)
44  expected_metrics = {self.pre_reg.primary_metric} | set(
45      self.pre_reg.secondary_metrics)
46
47  unexpected_metrics = unique_metrics - expected_metrics
48  if unexpected_metrics:
49      warnings.append({
50          'severity': 'high',
51          'issue': 'metric_fishing',
52          'message': f"Metrics_not_in_pre-registration_analyzed:
53                  "
54                  f"{unexpected_metrics}. May indicate fishing.
55                  "
56      })
57
58  # Check for subgroup fishing
59  subgroup_analyses = [a for a in self.analysis_log
60                      if 'subgroup' in a.get('notes', '')]
61  if len(subgroup_analyses) > len(self.pre_reg.planned_subgroups)
62  :
63      warnings.append({
64          'severity': 'medium',
65          'issue': 'subgroup_fishing',
66          'message': f"More_subgroup_analyses_{len(subgroup_
67                  analyses)}_
68                  "
69                  f"than_pre-registered_{len(self.pre_reg.
70                  planned_subgroups)}"
71      })
72
73  # Check for excessive analyses
74  if len(self.analysis_log) > 20:
75      warnings.append({
76          'severity': 'medium',
77          'issue': 'excessive_analysis',
78          'message': f"{len(self.analysis_log)}_analyses_logged.
79                  "
80                  f"Review_for_fishing_behavior."
81      })
82
83  return {
84      'integrity_score': max(0, 100 - len(warnings) * 20),

```

```

76         'warnings': warnings,
77         'total_analyses': len(self.analysis_log),
78         'analysis_log': self.analysis_log
79     }
80
81     def enforce_multiple_testing_correction(self, p_values: List[float
82 ] -> Dict:
83         """Apply multiple testing correction as specified in pre-
84         registration."""
85         from scipy import stats
86
87         method = self.pre_reg.multiple_testing_correction.lower()
88
89         if method == 'bonferroni':
90             alpha_corrected = self.pre_reg.significance_level / len(p_
91 values)
92             reject = [p < alpha_corrected for p in p_values]
93
94         elif method == 'benjamini-hochberg' or method == 'fdr':
95             # Benjamini-Hochberg FDR control
96             reject, p_adjusted, _, _ = stats.multipletests(
97                 p_values, alpha=self.pre_reg.significance_level,
98                 method='fdr_bh'
99             )
100             reject = list(reject)
101
102         elif method == 'holm':
103             reject, p_adjusted, _, _ = stats.multipletests(
104                 p_values, alpha=self.pre_reg.significance_level,
105                 method='holm'
106             )
107             reject = list(reject)
108
109         else:
110             raise ValueError(f"Unknown correction method: {method}")
111
112         return {
113             'method': method,
114             'original_alpha': self.pre_reg.significance_level,
115             'n_tests': len(p_values),
116             'p_values_original': p_values,
117             'reject_null': reject,
118             'significant_count': sum(reject)
119         }
120
121 # Example usage
122 monitor = ExperimentIntegrityMonitor(registration)
123
124 # Log analyses as they happen
125 monitor.log_analysis('t-test', 'conversion_rate', 0.032, 'Primary_
126 metric')
127 monitor.log_analysis('t-test', 'revenue_per_session', 0.089, 'Secondary
128 _metric')
129 monitor.log_analysis('mann-whitney', 'conversion_rate', 0.028,
130                     'Non-parametric alternative') # SUSPICIOUS: re-
131 analyzing
132
133 # Check integrity

```

```

128 integrity = monitor.check_analysis_integrity()
129 print(f"Integrity_score:_{integrity['integrity_score']/100}")
130 for warning in integrity['warnings']:
131     print(f"WARNING:_{warning['message']}")
132
133 # Apply multiple testing correction
134 p_values = [0.032, 0.089, 0.045, 0.012]
135 correction = monitor.enforce_multiple_testing_correction(p_values)
136 print(f"\nMultiple_testing_correction_{(correction['method']):}")
137 for i, (p, reject) in enumerate(zip(p_values, correction['reject_null',
138     ])):
139     print(f"_{i+1}:_p={p:.4f},_significant={reject}")

```

12.8.3 Reproducibility Standards

Reproducibility checklist:

1. **Code version control:** All analysis code in git
2. **Environment documentation:** Package versions, dependencies
3. **Random seed setting:** For reproducible randomization
4. **Data lineage:** Track data sources and transformations
5. **Analysis notebooks:** Literate programming with code + narrative
6. **Automated pipelines:** Reduce manual steps that introduce variation

Listing 12.6: Reproducibility Template

```

1  """
2  Reproducible_A/B_Test_Analysis_Template
3  =====
4
5  Experiment: _EXP-2024-042
6  Analyst: _Jane_Doe
7  Date: _2024-01-15
8  Python_version: _3.11.4
9  """
10
11 # Environment setup
12 import sys
13 import numpy as np
14 import pandas as pd
15 import scipy.stats as stats
16 import matplotlib.pyplot as plt
17
18 # Set random seed for reproducibility
19 RANDOM_SEED = 42
20 np.random.seed(RANDOM_SEED)
21
22 # Log environment info
23 print("Python_version:", sys.version)
24 print("NumPy_version:", np.__version__)
25 print("Pandas_version:", pd.__version__)

```

```

26 print("Random_seed:", RANDOM_SEED)
27
28 # Configuration (should match pre-registration)
29 CONFIG = {
30     'experiment_id': 'EXP-2024-042',
31     'start_date': '2024-01-01',
32     'end_date': '2024-01-14',
33     'alpha': 0.05,
34     'primary_metric': 'conversion_rate',
35     'variants': ['control', 'treatment']
36 }
37
38 def load_data(start_date: str, end_date: str) -> pd.DataFrame:
39     """
40     Load experiment data from data warehouse.
41
42     Returns DataFrame with columns: user_id, variant, converted, revenue
43     """
44     # SQL query or data loading code here
45     # This should be parameterized and logged
46     pass
47
48 def preprocess_data(df: pd.DataFrame) -> pd.DataFrame:
49     """
50     Apply standard preprocessing.
51     All transformations should be documented.
52     """
53     # Remove duplicates
54     initial_rows = len(df)
55     df = df.drop_duplicates(subset='user_id')
56     print(f"Removed {initial_rows - len(df)} duplicate users")
57
58     # Apply exclusion criteria (from pre-registration)
59     # ...
60
61     return df
62
63 def run_primary_analysis(df: pd.DataFrame) -> Dict:
64     """
65     Execute pre-registered primary analysis.
66     """
67     control = df[df['variant'] == 'control']
68     treatment = df[df['variant'] == 'treatment']
69
70     # Conversion rates
71     cr_control = control['converted'].mean()
72     cr_treatment = treatment['converted'].mean()
73
74     # Two-proportion z-test
75     from statsmodels.stats.proportion import proportions_ztest
76
77     counts = np.array([treatment['converted'].sum(),
78                       control['converted'].sum()])
79     nobs = np.array([len(treatment), len(control)])
80
81     z_stat, p_value = proportions_ztest(counts, nobs)
82

```

```

83     # Effect size
84     lift = (cr_treatment - cr_control) / cr_control
85
86     return {
87         'control_cr': cr_control,
88         'treatment_cr': cr_treatment,
89         'absolute_lift': cr_treatment - cr_control,
90         'relative_lift': lift,
91         'z_statistic': z_stat,
92         'p_value': p_value,
93         'significant': p_value < CONFIG['alpha']
94     }
95
96 # Execute analysis pipeline
97 # df = load_data(CONFIG['start_date'], CONFIG['end_date'])
98 # df = preprocess_data(df)
99 # results = run_primary_analysis(df)
100
101 # Save results with timestamp and hash for audit trail
102 # results['analysis_timestamp'] = datetime.now().isoformat()
103 # results['config_hash'] = hash(json.dumps(CONFIG, sort_keys=True))
104 # with open('results.json', 'w') as f:
105 #     json.dump(results, f, indent=2)

```

12.8.4 Conflict of Interest Management

Sources of conflicts:

- Financial incentives tied to positive results
- Career advancement dependent on successful experiments
- Vendor relationships influencing tool selection
- Personal investment in specific outcomes

Mitigations:

1. **Separation of duties:** Different people design, execute, and analyze
2. **Blinded analysis:** Analyst doesn't know which variant is which
3. **Independent review:** Third-party statistician reviews high-stakes experiments
4. **Incentive alignment:** Reward learning, not just wins
5. **Disclosure requirements:** Document potential conflicts

12.9 Building Experimentation Culture

12.9.1 Cultural Principles

1. **Data-informed, not data-driven:** Balance data with judgment
2. **Embrace failure:** Learn from experiments that don't succeed

3. **Democratize experimentation:** Enable all teams to test
4. **Question assumptions:** Challenge conventional wisdom
5. **Iterate rapidly:** Small, frequent experiments over big bets
6. **Share learnings:** Disseminate insights across organization

12.9.2 Organizational Structure

Centralized model:

- Dedicated experimentation team
- Ensures methodological rigor
- Can become bottleneck

Decentralized model:

- Each team runs own experiments
- Faster iteration
- Risk of inconsistent standards

Hybrid model (recommended):

- Central platform and guidelines
- Distributed execution
- Centers of excellence provide support

12.9.3 Training and Education

Invest in capability building:

- Statistical literacy training
- Experimentation best practices workshops
- Case study discussions
- Mentorship programs
- Documentation and runbooks

12.10 Organizational Best Practices

Scaling experimentation requires deliberate organizational design and governance.

12.10.1 Experiment Governance Framework

Governance tiers by risk:

Tier	Risk Level	Examples	Approval Required
1	Minimal	UI tweaks, copy changes	Self-service
2	Low	Feature variations, algorithm parameters	Manager approval
3	Medium	New features, pricing changes	Director + data science review
4	High	Core algorithm changes, degraded experiences	VP + ethics board
5	Critical	Healthcare interventions, children's data	Legal + IRB + executive

Listing 12.7: Experiment Governance Template

```

1 from enum import Enum
2 from dataclasses import dataclass
3 from typing import List, Optional
4
5 class RiskTier(Enum):
6     """Experiment risk classification."""
7     MINIMAL = 1
8     LOW = 2
9     MEDIUM = 3
10    HIGH = 4
11    CRITICAL = 5
12
13 @dataclass
14 class ExperimentGovernance:
15     """Governance framework for experiment approval."""
16
17     experiment_id: str
18     name: str
19     owner: str
20     description: str
21     risk_tier: RiskTier
22     affected_users: int
23     revenue_impact_estimated: Optional[float] = None
24     user_experience_impact: str = "neutral" # "positive", "neutral", "
        negative"
25
26     # Review and approval
27     approvers: List[str] = None
28     approved: bool = False
29     approval_date: Optional[datetime] = None
30
31     def classify_risk_tier(self) -> RiskTier:
32         """
33         Automatically classify experiment risk tier based on
34         characteristics.
35         """

```

```

35     risk_score = 0
36
37     # User impact
38     if self.affected_users > 1_000_000:
39         risk_score += 2
40     elif self.affected_users > 100_000:
41         risk_score += 1
42
43     # Revenue impact
44     if self.revenue_impact_estimated and abs(self.revenue_impact_
45         estimated) > 1_000_000:
46         risk_score += 2
47     elif self.revenue_impact_estimated and abs(self.revenue_impact_
48         estimated) > 100_000:
49         risk_score += 1
50
51     # User experience
52     if self.user_experience_impact == "negative":
53         risk_score += 2
54
55     # Map score to tier
56     if risk_score == 0:
57         return RiskTier.MINIMAL
58     elif risk_score <= 2:
59         return RiskTier.LOW
60     elif risk_score <= 4:
61         return RiskTier.MEDIUM
62     elif risk_score <= 6:
63         return RiskTier.HIGH
64     else:
65         return RiskTier.CRITICAL
66
67 def get_required_approvers(self) -> List[str]:
68     """Get list of required approvers based on risk tier."""
69     approver_map = {
70         RiskTier.MINIMAL: [],
71         RiskTier.LOW: ["manager"],
72         RiskTier.MEDIUM: ["manager", "director", "data_science_lead"],
73         RiskTier.HIGH: ["director", "vp", "ethics_board"],
74         RiskTier.CRITICAL: ["vp", "legal", "ethics_board", "
75             executive"]
76     }
77     return approver_map[self.risk_tier]
78
79 def request_approval(self) -> Dict:
80     """Generate approval request with all necessary information."""
81     return {
82         'experiment_id': self.experiment_id,
83         'name': self.name,
84         'owner': self.owner,
85         'description': self.description,
86         'risk_tier': self.risk_tier.name,
87         'affected_users': self.affected_users,
88         'required_approvers': self.get_required_approvers(),
89         'approval_status': 'pending',
90         'submission_date': datetime.now().isoformat()
91     }

```



```
89
90 # Example usage
91 experiment = ExperimentGovernance(
92     experiment_id="EXP-2024-100",
93     name="New Checkout Flow",
94     owner="product_team@company.com",
95     description="Redesigned checkout with one-click purchase option",
96     risk_tier=RiskTier.MEDIUM,
97     affected_users=500_000,
98     revenue_impact_estimated=2_500_000,
99     user_experience_impact="positive"
100 )
101
102 # Auto-classify risk
103 suggested_tier = experiment.classify_risk_tier()
104 print(f"Suggested risk tier: {suggested_tier.name}")
105
106 # Get approval requirements
107 approval_request = experiment.request_approval()
108 print(f"Required approvers: {approval_request['required_approvers']}")
```

12.10.2 Team Structure and Responsibilities

RACI Matrix for Experimentation:

Activity	Product	Data Science	Engineering	Legal
Hypothesis generation	R/A	C	C	I
Experimental design	C	R/A	C	I
Sample size calculation	I	R/A	I	I
Implementation	C	I	R/A	I
Quality assurance	I	C	R/A	I
Launch approval	A	C	I	C
Data analysis	C	R/A	I	I
Result interpretation	R/A	C	I	I
Decision making	A	C	I	C
Documentation	R	C	I	I

R = Responsible, A = Accountable, C = Consulted, I = Informed

Role definitions:

- **Experiment Owner (Product):** Defines business objectives, success criteria, launch decisions
- **Data Scientist:** Statistical design, sample size, analysis, interpretation support
- **Engineer:** Implementation, instrumentation, monitoring, infrastructure
- **UX Researcher:** Qualitative insights, user research, design input
- **Legal/Compliance:** Risk assessment, regulatory compliance, privacy review

12.10.3 Knowledge Management System

Experiment registry schema:

Listing 12.8: Experiment Registry Template

```

1 from dataclasses import dataclass, field
2 from typing import List, Dict, Optional
3 from datetime import datetime
4
5 @dataclass
6 class ExperimentRecord:
7     """
8     Complete record for experiment registry.
9     Enables knowledge sharing and meta-analysis.
10    """
11    # Identification
12    experiment_id: str
13    name: str
14    owner: str
15    team: str
16    created_date: datetime
17
18    # Business context
19    business_objective: str
20    hypothesis: str
21    success_criteria: str
22    product_area: str
23    tags: List[str] = field(default_factory=list)
24
25    # Technical design
26    variants: List[str] = field(default_factory=list)
27    primary_metric: str
28    secondary_metrics: List[str] = field(default_factory=list)
29    guardrail_metrics: List[str] = field(default_factory=list)
30    randomization_unit: str = "user_id"
31    traffic_allocation: Dict[str, float] = field(default_factory=dict)
32
33    # Execution
34    start_date: Optional[datetime] = None
35    end_date: Optional[datetime] = None
36    status: str = "draft" # draft, running, completed, cancelled
37    sample_size_achieved: Optional[int] = None
38
39    # Results
40    primary_metric_result: Optional[float] = None
41    statistical_significance: Optional[bool] = None
42    p_value: Optional[float] = None
43    confidence_interval: Optional[tuple] = None
44    decision: Optional[str] = None # ship, don't ship, iterate
45    decision_rationale: Optional[str] = None
46
47    # Learnings
48    key_insights: List[str] = field(default_factory=list)
49    surprising_findings: List[str] = field(default_factory=list)
50    challenges_encountered: List[str] = field(default_factory=list)
51    recommendations: List[str] = field(default_factory=list)
52
53    # Links

```

```

54 analysis_notebook_url: Optional[str] = None
55 design_doc_url: Optional[str] = None
56 dashboard_url: Optional[str] = None
57
58 def to_searchable_document(self) -> Dict:
59     """Convert to searchable document for experiment discovery."""
60     return {
61         'id': self.experiment_id,
62         'name': self.name,
63         'description': f"{self.business_objective}|{self.
64             hypothesis}",
65         'owner': self.owner,
66         'team': self.team,
67         'product_area': self.product_area,
68         'tags': self.tags,
69         'status': self.status,
70         'date_range': f"{self.start_date} to {self.end_date}",
71         'decision': self.decision,
72         'metrics_tested': [self.primary_metric] + self.secondary_
73             metrics,
74         'searchable_text': ' '.join([
75             self.name,
76             self.business_objective,
77             self.hypothesis,
78             *self.key_insights,
79             *self.tags
80         ])
81     }
82
83 def generate_summary_report(self) -> str:
84     """Generate human-readable summary."""
85     report = [
86         f"Experiment: {self.name} ({self.experiment_id})",
87         f"Owner: {self.owner} | Team: {self.team}",
88         f"Status: {self.status}",
89         "",
90         "OBJECTIVE:",
91         f"  {self.business_objective}",
92         "",
93         "HYPOTHESIS:",
94         f"  {self.hypothesis}",
95         "",
96         "RESULTS:",
97         f"  Primary Metric: {self.primary_metric}",
98         f"  Result: {self.primary_metric_result}",
99         f"  Significant: {self.statistical_significance} (p={self.p_
100             _value})",
101         f"  Decision: {self.decision}",
102         "",
103         "KEY INSIGHTS:",
104     ]
105
106     for insight in self.key_insights:
107         report.append(f"  {insight}")
108
109     return "\n".join(report)
110
111 # Example usage

```

```

109 experiment = ExperimentRecord(
110     experiment_id="EXP-2024-042",
111     name="Green_Checkout_Button",
112     owner="jane_doe@company.com",
113     team="Checkout_Team",
114     created_date=datetime(2024, 1, 1),
115     business_objective="Increase_checkout_conversion_rate",
116     hypothesis="Green_button_color_will_increase_perceived_urgency_and_
        conversions",
117     success_criteria="2%_relative_lift_in_conversion_rate",
118     product_area="Checkout",
119     tags=["checkout", "ui", "conversion"],
120     variants=["control_blue", "treatment_green"],
121     primary_metric="conversion_rate",
122     secondary_metrics=["revenue_per_session", "cart_abandonment"],
123     status="completed",
124     decision="ship",
125     key_insights=[
126         "Green_button_increased_conversions_by_3.2%_(p=0.012)",
127         "Effect_strongest_among_new_users_(+5.1%)",
128         "No_impact_on_returning_users"
129     ]
130 )
131
132 # Generate searchable record
133 doc = experiment.to_searchable_document()
134 print(experiment.generate_summary_report())

```

12.10.4 Knowledge Sharing Mechanisms

Regular review forums:

1. Weekly experiment reviews:

- Recent launches and results
- Ongoing experiments status
- Upcoming experiments preview
- 30-minute standup format

2. Monthly deep dives:

- Detailed analysis of significant experiments
- Methodology discussions
- Tool and technique sharing
- 1-hour presentation + Q&A

3. Quarterly meta-analysis:

- Patterns across experiments
- Success/failure rate analysis
- Organizational metrics (velocity, quality)

- Strategic planning

Documentation best practices:

- **Pre-experiment:** Design doc with hypothesis, methodology, success criteria
- **During execution:** Living dashboard with real-time metrics
- **Post-experiment:** Results summary with insights and recommendations
- **Retrospectives:** What worked, what didn't, what to improve

12.10.5 Continuous Learning Culture

Celebrating learnings, not just wins:

- **Null result awards:** Recognize valuable negative findings
- **Failure postmortems:** Blameless analysis of experimental failures
- **Insight of the month:** Highlight unexpected or counterintuitive findings
- **Methodology innovations:** Reward improvements to experimental practice

Metrics for experimentation program health:

Metric	Definition	Target
Experiment velocity	Experiments launched per quarter	Increasing
Win rate	Percentage of experiments with positive results	10-30%
Sample ratio mismatch rate	Percentage of experiments with SRM	< 1%
Documentation completeness	Percentage with full documentation	> 95%
Time to decision	Days from launch to decision	< 14 days
Learning share	Percentage of results shared widely	> 80%

12.11 Quality Assurance

Systematic quality control ensures reliable and trustworthy results.

12.11.1 Peer Review Process

Review stages:

1. Design review (pre-launch):

- Hypothesis clarity and testability

- Statistical design appropriateness
- Sample size adequacy
- Metric definitions
- Guardrail metrics
- Ethical considerations

2. Implementation review:

- Randomization correctness
- Instrumentation accuracy
- Configuration validation
- Monitoring setup

3. Analysis review (pre-decision):

- Data quality checks performed
- Statistical methods appropriate
- Multiple testing corrections applied
- Assumptions validated
- Segment analysis justified
- Conclusions supported by data

Listing 12.9: Peer Review Checklist

```

1 from dataclasses import dataclass
2 from typing import List, Dict, Optional
3 from datetime import datetime
4
5 @dataclass
6 class ReviewChecklist:
7     """Structured peer review checklist."""
8
9     experiment_id: str
10    review_stage: str # 'design', 'implementation', 'analysis'
11    reviewer: str
12    review_date: datetime
13
14    # Checklist items (True = pass, False = fail, None = not applicable)
15
16    checks: Dict[str, Optional[bool]]
17    comments: Dict[str, str]
18
19    def is_approved(self) -> bool:
20        """Check if review passes (no False values)."""
21        return all(v is not False for v in self.checks.values())
22
23    def get_blocking_issues(self) -> List[str]:
24        """Get list of failed checks."""
25        return [item for item, passed in self.checks.items() if passed
26                is False]
```

```

25
26 def generate_review_report(self) -> str:
27     """Generate review summary."""
28     status = "APPROVED" if self.is_approved() else "NEEDS_REVISION"
29     blocking = self.get_blocking_issues()
30
31     report = [
32         f"Peer_Review_Report:_{self.experiment_id}",
33         f"Stage:_{self.review_stage}",
34         f"Reviewer:_{selfreviewer}",
35         f>Date:_{self.review_date.strftime('%Y-%m-%d')}",
36         f>Status:_{status}",
37         ""
38     ]
39
40     if blocking:
41         report.append("BLOCKING_ISSUES:")
42         for issue in blocking:
43             report.append(f"_{issue}")
44             if issue in self.comments:
45                 report.append(f"_{self.comments[issue]}")
46         report.append("")
47
48     # Passed items
49     passed = [item for item, v in self.checks.items() if v is True]
50     if passed:
51         report.append("PASSED_CHECKS:")
52         for item in passed:
53             report.append(f"_{item}")
54
55     return "\n".join(report)
56
57 # Example: Design review
58 design_review = ReviewChecklist(
59     experiment_id="EXP-2024-042",
60     review_stage="design",
61     reviewer="senior_data_scientist@company.com",
62     review_date=datetime.now(),
63     checks={
64         'hypothesis_clear': True,
65         'hypothesis_testable': True,
66         'primary_metric_defined': True,
67         'sample_size_calculated': True,
68         'sample_size_adequate': False, # BLOCKING
69         'guardrails_defined': True,
70         'ethical_review_completed': True,
71         'randomization_unit_appropriate': True,
72         'success_criteria_prespecified': True,
73         'multiple_testing_plan': None # Not applicable (single metric)
74     },
75     comments={
76         'sample_size_adequate': "Planned_N=1000, but power analysis shows_N=2500 needed for 80% power. Increase sample size or extend duration."
77     }
78 )
79

```

```

80 print(design_review.generate_review_report())
81 print(f"\nApproved: {design_review.is_approved()}")

```

12.11.2 Statistical Consulting

When to engage statistical expert:

- Novel experimental designs (e.g., switchback tests, multi-armed bandits)
- Complex analysis requirements (e.g., network effects, interference)
- High-stakes decisions (revenue impact > \$1M)
- Unusual data patterns or results
- Methodology questions or disputes

Statistical consulting workflow:

1. **Intake:** Team submits consultation request with context
2. **Review:** Statistician reviews design/analysis
3. **Consultation:** 30-60 minute discussion
4. **Recommendations:** Written guidance provided
5. **Follow-up:** Optional review of final analysis

12.11.3 Automated Validation

Automated quality checks:

Listing 12.10: Automated Validation Framework

```

1 class ExperimentValidator:
2     """Comprehensive automated validation for experiments."""
3
4     def __init__(self, experiment_data: pd.DataFrame,
5                  experiment_config: Dict):
6         self.data = experiment_data
7         self.config = experiment_config
8         self.validation_results = {}
9
10    def validate_all(self) -> Dict:
11        """Run all validation checks."""
12        validations = [
13            ('sample_ratio_mismatch', self.check_sample_ratio_mismatch),
14            ('assignment_consistency', self.check_assignment_
15             consistency),
16            ('data_completeness', self.check_data_completeness),
17            ('metric_sanity', self.check_metric_sanity),
18            ('temporal_stability', self.check_temporal_stability),
19            ('outliers', self.check_outlier_prevalence),
20            ('guardrails', self.check_guardrail_metrics)

```



```

20     ]
21
22     results = {}
23     all_passed = True
24
25     for name, check_func in validations:
26         try:
27             result = check_func()
28             results[name] = result
29             if not result.get('passed', False):
30                 all_passed = False
31         except Exception as e:
32             results[name] = {'passed': False, 'error': str(e)}
33             all_passed = False
34
35     return {
36         'all_passed': all_passed,
37         'checks': results,
38         'summary': self._generate_summary(results)
39     }
40
41     def check_sample_ratio_mismatch(self) -> Dict:
42         """Check if variant sizes match expected allocation."""
43         from scipy import stats
44
45         variant_counts = self.data['variant'].value_counts()
46         total = len(self.data)
47
48         expected_allocation = self.config.get('traffic_allocation', {})
49
50         # Chi-square goodness of fit
51         expected_counts = [total * expected_allocation.get(v, 1/len(
52             variant_counts))
53                             for v in variant_counts.index]
54
55         chi2, p_value = stats.chisquare(variant_counts.values, expected_
56             _counts)
57
58         passed = p_value >= 0.01 # Use stricter threshold for SRM
59
60         return {
61             'passed': passed,
62             'p_value': p_value,
63             'chi2_statistic': chi2,
64             'observed_counts': variant_counts.to_dict(),
65             'expected_counts': dict(zip(variant_counts.index, expected_
66                 counts)),
67             'severity': 'critical' if not passed else 'none',
68             'message': 'Sample_Ratio_Mismatch_detected!' if not passed
69                 else 'Sample_ratios_OK'
70         }
71
72     def check_guardrail_metrics(self) -> Dict:
73         """Ensure guardrail metrics haven't degraded."""
74         guardrails = self.config.get('guardrail_metrics', [])
75         violations = []
76
77         for metric in guardrails:

```

```

74         if metric not in self.data.columns:
75             continue
76
77         control = self.data[self.data['variant'] == 'control'][
78             metric]
79         treatment = self.data[self.data['variant'] == 'treatment'][
80             metric]
81
82         # Check if treatment is worse than control (with margin)
83         degradation_threshold = 0.01 # 1% degradation allowed
84
85         relative_change = (treatment.mean() - control.mean()) /
86             control.mean()
87
88         if relative_change < -degradation_threshold:
89             violations.append({
90                 'metric': metric,
91                 'degradation': relative_change,
92                 'control_mean': control.mean(),
93                 'treatment_mean': treatment.mean()
94             })
95
96         passed = len(violations) == 0
97
98         return {
99             'passed': passed,
100             'violations': violations,
101             'severity': 'high' if not passed else 'none',
102             'message': f"{len(violations)}_guardrail_violations" if not
103                 passed else "All_guardrails_OK"
104         }
105
106     def _generate_summary(self, results: Dict) -> str:
107         """Generate human-readable summary."""
108         critical = [name for name, r in results.items()
109                     if r.get('severity') == 'critical']
110         high = [name for name, r in results.items()
111                 if r.get('severity') == 'high']
112
113         if critical:
114             return f"BLOCKED: Critical issues - {', '.join(critical)}"
115         elif high:
116             return f"WARNING: High-priority issues - {', '.join(high)}"
117         else:
118             return "PASS: All validation checks passed"
119
120 # Example usage
121 # validator = ExperimentValidator(experiment_df, config)
122 # validation_results = validator.validate_all()
123 # print(validation_results['summary'])

```

12.11.4 External Auditing

For high-stakes or sensitive experiments, consider external audits:

External audit scope:

- Independent review of methodology

- Data integrity verification
- Statistical analysis validation
- Compliance assessment
- Recommendations for improvement

When external audits are valuable:

- Regulatory scrutiny (healthcare, finance)
- High-stakes business decisions (> \$10M impact)
- Controversial or high-risk experiments
- Academic publication planned
- Building credibility with stakeholders

12.12 Best Practices Checklist

12.12.1 Design Phase

- ☐ Clear hypothesis stated
- ☐ Primary metric identified
- ☐ Sample size calculated
- ☐ Guardrail metrics defined
- ☐ Ethical review completed
- ☐ Success criteria pre-specified

12.12.2 Implementation Phase

- ☐ Randomization mechanism tested
- ☐ Instrumentation validated
- ☐ SRM monitoring configured
- ☐ Logging working correctly
- ☐ Configuration documented

12.12.3 Analysis Phase

- ☐ Pre-analysis quality checks passed
- ☐ Multiple testing corrections applied
- ☐ Subgroup analysis planned
- ☐ Confidence intervals reported
- ☐ Effect sizes calculated
- ☐ Business impact estimated

12.12.4 Decision Phase

- ☐ Results communicated clearly
- ☐ Statistical and practical significance assessed
- ☐ Stakeholders aligned
- ☐ Implementation plan created
- ☐ Learnings documented

12.13 Common Pitfalls and How to Avoid Them

Pitfall	Consequence	Prevention
No pre-specified hypothesis	P-hacking, false positives	Pre-register experiments
Peeking at results	Inflated Type I error	Sequential testing methods
Ignoring guardrails	Harm to user experience	Monitor health metrics
Small sample sizes	Underpowered tests	Power analysis upfront
Short duration	Missing long-term effects	Run for full business cycles
No documentation	Lost institutional knowledge	Experiment registry

12.14 Summary

Ethical and effective experimentation requires:

- Commitment to ethical principles and user welfare
- Robust governance and review processes
- Privacy protection and regulatory compliance
- Transparency internally and externally
- Strong organizational culture supporting experimentation

- Continuous learning and improvement

A/B testing is powerful but carries responsibility. Practitioners must balance innovation with ethics, rigor with pragmatism, and data with judgment.

12.15 Further Reading

- Kramer et al. [2014]: Controversial study highlighting ethical issues
- Meyer [2014]: Ethics of tech experimentation
- Kohavi et al. [2020]: Building experimentation culture
- IRB guidelines: Institutional Review Board frameworks

12.16 Exercises

1. Create an ethics review framework for your organization
2. Analyze a controversial experiment (e.g., Facebook emotional contagion study)
3. Design an experimentation training program
4. Build an experiment registry system
5. Draft transparency report for external publication

"The goal is to turn data into information, and information into insight."
— Carly Fiorina

Mathematical Proofs

Mathematical Proofs and Derivations

This appendix provides rigorous mathematical foundations for the statistical methods used in A/B testing. We present detailed proofs and derivations with step-by-step explanations.

.1 Sample Size Formulas

Sample size determination is fundamental to experimental design. We derive formulas for various testing scenarios.

.1.1 Sample Size for Two-Sample t-Test

Theorem .1 (Sample Size for Comparing Two Means). To detect a difference $\delta = \mu_2 - \mu_1$ between two population means with common variance σ^2 , significance level α , and power $1 - \beta$, the required sample size per group is:

$$n = \frac{2(z_{\alpha/2} + z_{\beta})^2 \sigma^2}{\delta^2} \quad (1)$$

where $z_{\alpha/2}$ and z_{β} are the critical values from the standard normal distribution.

Proof. Consider testing $H_0 : \mu_1 = \mu_2$ versus $H_1 : \mu_1 \neq \mu_2$ using the test statistic:

$$T = \frac{\bar{X}_2 - \bar{X}_1}{\sqrt{2\sigma^2/n}} \quad (2)$$

Step 1: Distribution under the null hypothesis.

Under H_0 , the difference $\bar{X}_2 - \bar{X}_1$ has mean 0 and variance:

$$\text{Var}(\bar{X}_2 - \bar{X}_1) = \text{Var}(\bar{X}_2) + \text{Var}(\bar{X}_1) = \frac{\sigma^2}{n} + \frac{\sigma^2}{n} = \frac{2\sigma^2}{n} \quad (3)$$

By the Central Limit Theorem, for large n :

$$T \mid H_0 \sim \mathcal{N}(0, 1) \quad (4)$$

Step 2: Rejection region.

For a two-sided test at level α , we reject H_0 if $|T| > z_{\alpha/2}$, which occurs when:

$$|\bar{X}_2 - \bar{X}_1| > z_{\alpha/2} \sqrt{\frac{2\sigma^2}{n}} \quad (5)$$

Step 3: Distribution under the alternative.

Under $H_1 : \mu_2 - \mu_1 = \delta$, the test statistic has mean:

$$\mathbb{E}[T \mid H_1] = \frac{\mathbb{E}[\bar{X}_2 - \bar{X}_1]}{\sqrt{2\sigma^2/n}} \quad (6)$$

$$= \frac{\delta}{\sqrt{2\sigma^2/n}} \quad (7)$$

$$= \frac{\delta\sqrt{n}}{\sqrt{2}\sigma} \quad (8)$$

and variance 1. Therefore:

$$T \mid H_1 \sim \mathcal{N}\left(\frac{\delta\sqrt{n}}{\sqrt{2}\sigma}, 1\right) \quad (9)$$

Step 4: Power calculation.

The power is the probability of rejecting H_0 when H_1 is true. Assuming $\delta > 0$:

$$\text{Power} = P(T > z_{\alpha/2} \mid H_1) \quad (10)$$

$$= P\left(\mathcal{N}\left(\frac{\delta\sqrt{n}}{\sqrt{2}\sigma}, 1\right) > z_{\alpha/2}\right) \quad (11)$$

$$= P\left(\mathcal{N}(0, 1) > z_{\alpha/2} - \frac{\delta\sqrt{n}}{\sqrt{2}\sigma}\right) \quad (12)$$

$$= \Phi\left(\frac{\delta\sqrt{n}}{\sqrt{2}\sigma} - z_{\alpha/2}\right) \quad (13)$$

where Φ is the standard normal CDF.

Step 5: Solve for n.

For power $1 - \beta$, we require:

$$\Phi\left(\frac{\delta\sqrt{n}}{\sqrt{2}\sigma} - z_{\alpha/2}\right) = 1 - \beta \quad (14)$$

Since $\Phi(z_\beta) = 1 - \beta$, we have:

$$\frac{\delta\sqrt{n}}{\sqrt{2}\sigma} - z_{\alpha/2} = z_\beta \quad (15)$$

Solving for n :

$$\frac{\delta\sqrt{n}}{\sqrt{2}\sigma} = z_{\alpha/2} + z_\beta \quad (16)$$

$$\sqrt{n} = \frac{\sqrt{2}\sigma(z_{\alpha/2} + z_\beta)}{\delta} \quad (17)$$

$$n = \frac{2(z_{\alpha/2} + z_\beta)^2\sigma^2}{\delta^2} \quad (18)$$

□

1.1.2 Sample Size for Comparing Two Proportions

Theorem .2 (Sample Size for Proportion Comparison). To detect a difference between two proportions p_1 and p_2 with significance level α and power $1 - \beta$, the required sample size per group is:

$$n = \frac{(z_{\alpha/2} + z_{\beta})^2 [p_1(1 - p_1) + p_2(1 - p_2)]}{(p_2 - p_1)^2} \quad (19)$$

Proof. **Step 1: Test statistic under the alternative.**

Under $H_1 : p_1 \neq p_2$, the difference in sample proportions has:

$$\mathbb{E}[\hat{p}_2 - \hat{p}_1] = p_2 - p_1 \quad (20)$$

$$\text{Var}(\hat{p}_2 - \hat{p}_1) = \frac{p_2(1 - p_2)}{n} + \frac{p_1(1 - p_1)}{n} = \frac{p_1(1 - p_1) + p_2(1 - p_2)}{n} \quad (21)$$

Define the standardized statistic:

$$Z = \frac{(\hat{p}_2 - \hat{p}_1) - (p_2 - p_1)}{\sqrt{[p_1(1 - p_1) + p_2(1 - p_2)]/n}} \sim \mathcal{N}(0, 1) \quad (22)$$

Step 2: Test statistic under the null.

Under $H_0 : p_1 = p_2 = p$ (pooled proportion), the variance is:

$$\text{Var}_0(\hat{p}_2 - \hat{p}_1) = \frac{2p(1 - p)}{n} \quad (23)$$

The null statistic is:

$$Z_0 = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{2p(1 - p)/n}} \sim \mathcal{N}(0, 1) \quad (24)$$

Step 3: Critical value.

Reject H_0 if $|Z_0| > z_{\alpha/2}$, which occurs when:

$$|\hat{p}_2 - \hat{p}_1| > z_{\alpha/2} \sqrt{\frac{2p(1 - p)}{n}} \quad (25)$$

Step 4: Power under the alternative.

Assuming $p_2 > p_1$ and approximating $p \approx \frac{p_1 + p_2}{2}$, the power is:

$$\text{Power} = P \left(\hat{p}_2 - \hat{p}_1 > z_{\alpha/2} \sqrt{\frac{2p(1 - p)}{n}} \mid H_1 \right) \quad (26)$$

$$= P \left(\frac{(\hat{p}_2 - \hat{p}_1) - (p_2 - p_1)}{\sqrt{[p_1(1 - p_1) + p_2(1 - p_2)]/n}} > \frac{z_{\alpha/2} \sqrt{2p(1 - p)/n} - (p_2 - p_1)}{\sqrt{[p_1(1 - p_1) + p_2(1 - p_2)]/n}} \right) \quad (27)$$

For large n , $2p(1 - p) \approx p_1(1 - p_1) + p_2(1 - p_2)$, so:

$$\text{Power} \approx \Phi \left(\frac{(p_2 - p_1)\sqrt{n}}{\sqrt{p_1(1 - p_1) + p_2(1 - p_2)}} - z_{\alpha/2} \right) \quad (28)$$

Step 5: Solve for n.

Setting Power = $1 - \beta$:

$$\frac{(p_2 - p_1)\sqrt{n}}{\sqrt{p_1(1 - p_1) + p_2(1 - p_2)}} - z_{\alpha/2} = z_\beta \quad (29)$$

$$\sqrt{n} = \frac{(z_{\alpha/2} + z_\beta)\sqrt{p_1(1 - p_1) + p_2(1 - p_2)}}{p_2 - p_1} \quad (30)$$

$$n = \frac{(z_{\alpha/2} + z_\beta)^2[p_1(1 - p_1) + p_2(1 - p_2)]}{(p_2 - p_1)^2} \quad (31)$$

□

.1.3 Minimum Detectable Effect

Corollary .1 (Minimum Detectable Effect). Given fixed sample size n , significance level α , and power $1 - \beta$, the minimum detectable effect for comparing means is:

$$\text{MDE} = \delta = \frac{(z_{\alpha/2} + z_\beta)\sigma\sqrt{2}}{\sqrt{n}} \quad (32)$$

Proof. Rearranging the sample size formula:

$$n = \frac{2(z_{\alpha/2} + z_\beta)^2\sigma^2}{\delta^2} \quad (33)$$

$$\delta^2 = \frac{2(z_{\alpha/2} + z_\beta)^2\sigma^2}{n} \quad (34)$$

$$\delta = \frac{(z_{\alpha/2} + z_\beta)\sigma\sqrt{2}}{\sqrt{n}} \quad (35)$$

□

.1.4 Unequal Sample Sizes

Theorem .3 (Sample Size with Allocation Ratio). If the allocation ratio is $r = n_2/n_1$ (i.e., $n_2 = rn_1$), the required sample size for group 1 is:

$$n_1 = \frac{(z_{\alpha/2} + z_\beta)^2\sigma^2(1 + 1/r)}{\delta^2} \quad (36)$$

and $n_2 = rn_1$.

Proof. The variance of the difference with unequal sample sizes is:

$$\text{Var}(\bar{X}_2 - \bar{X}_1) = \frac{\sigma^2}{n_2} + \frac{\sigma^2}{n_1} = \frac{\sigma^2}{rn_1} + \frac{\sigma^2}{n_1} = \frac{\sigma^2(1 + r)}{rn_1} \quad (37)$$

Following the same derivation as before, the test statistic under H_1 is:

$$T = \frac{(\bar{X}_2 - \bar{X}_1) - \delta}{\sqrt{\sigma^2(1 + r)/(rn_1)}} \sim \mathcal{N}(0, 1) \quad (38)$$

The power condition gives:

$$\frac{\delta}{\sqrt{\sigma^2(1+r)/(rn_1)}} - z_{\alpha/2} = z_{\beta} \quad (39)$$

Solving for n_1 :

$$\frac{\delta\sqrt{rn_1}}{\sigma\sqrt{1+r}} = z_{\alpha/2} + z_{\beta} \quad (40)$$

$$n_1 = \frac{(z_{\alpha/2} + z_{\beta})^2 \sigma^2 (1+r)}{r\delta^2} = \frac{(z_{\alpha/2} + z_{\beta})^2 \sigma^2 (1+1/r)}{\delta^2} \quad (41)$$

□

Remark .1. The total sample size $N = n_1 + n_2 = n_1(1+r)$ is minimized when $r = 1$ (equal allocation).

.2 Central Limit Theorem

The Central Limit Theorem is the foundation of large-sample inference in A/B testing.

.2.1 Classical Central Limit Theorem

Theorem .4 (Lindeberg-Lévy CLT). Let $X_1, X_2, \dots$ be independent and identically distributed random variables with $\mathbb{E}[X_i] = \mu$ and $\text{Var}(X_i) = \sigma^2 < \infty$. Define $S_n = \sum_{i=1}^n X_i$ and $\bar{X}_n = S_n/n$. Then:

$$\frac{S_n - n\mu}{\sigma\sqrt{n}} = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1) \quad (42)$$

as $n \rightarrow \infty$, where $\xrightarrow{d}$ denotes convergence in distribution.

Proof Sketch via Characteristic Functions. We prove this using moment generating functions (characteristic functions).

Step 1: Standardize the sum.

Define $Z_i = (X_i - \mu)/\sigma$ so that $\mathbb{E}[Z_i] = 0$ and $\text{Var}(Z_i) = 1$. Then:

$$\frac{S_n - n\mu}{\sigma\sqrt{n}} = \frac{1}{\sqrt{n}} \sum_{i=1}^n Z_i \quad (43)$$

Step 2: Characteristic function.

Let $\varphi_Z(t) = \mathbb{E}[e^{itZ_1}]$ be the characteristic function of Z_1 . The characteristic function of $\frac{1}{\sqrt{n}} \sum_{i=1}^n Z_i$ is:

$$\varphi_n(t) = \mathbb{E} \left[\exp \left(it \frac{1}{\sqrt{n}} \sum_{i=1}^n Z_i \right) \right] = \prod_{i=1}^n \mathbb{E} [e^{itZ_i/\sqrt{n}}] = \left[\varphi_Z \left(\frac{t}{\sqrt{n}} \right) \right]^n \quad (44)$$

Step 3: Taylor expansion.

Since $\mathbb{E}[Z_1] = 0$ and $\text{Var}(Z_1) = 1$, we can expand:

$$\varphi_Z(t) = \mathbb{E}[e^{itZ_1}] \quad (45)$$

$$= \mathbb{E} \left[1 + itZ_1 + \frac{(it)^2 Z_1^2}{2!} + \frac{(it)^3 Z_1^3}{3!} + \dots \right] \quad (46)$$

$$= 1 + it\mathbb{E}[Z_1] + \frac{(it)^2 \mathbb{E}[Z_1^2]}{2} + o(t^2) \quad (47)$$

$$= 1 - \frac{t^2}{2} + o(t^2) \quad (48)$$

Step 4: Substitute and take limit.

$$\varphi_n(t) = \left[\varphi_Z \left(\frac{t}{\sqrt{n}} \right) \right]^n \quad (49)$$

$$= \left[1 - \frac{t^2}{2n} + o \left(\frac{1}{n} \right) \right]^n \quad (50)$$

$$\rightarrow e^{-t^2/2} \quad \text{as } n \rightarrow \infty \quad (51)$$

by the standard limit $\lim_{n \rightarrow \infty} (1 + x/n)^n = e^x$.

Step 5: Recognize the limit.

The function $e^{-t^2/2}$ is the characteristic function of $\mathcal{N}(0, 1)$.

By Lévy's continuity theorem, convergence of characteristic functions implies convergence in distribution:

$$\frac{S_n - n\mu}{\sigma\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1) \quad (52)$$

□

.2.2 Berry-Esseen Theorem

The Berry-Esseen theorem provides a quantitative bound on the rate of convergence in the CLT.

Theorem .5 (Berry-Esseen Bound). Under the conditions of the CLT, if additionally $\mathbb{E}[|X_i|^3] = \rho < \infty$, then:

$$\sup_{x \in \mathbb{R}} \left| P \left(\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \leq x \right) - \Phi(x) \right| \leq \frac{C\rho}{\sigma^3\sqrt{n}} \quad (53)$$

where Φ is the standard normal CDF and C is an absolute constant (best known value: $C \approx 0.4748$).

Remark .2. This shows convergence rate is $O(n^{-1/2})$. For practical purposes, $n \geq 30$ often suffices for near-normality.

.2.3 Delta Method

Theorem .6 (Delta Method). Let $\sqrt{n}(\hat{\theta}_n - \theta) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$ and let $g : \mathbb{R} \rightarrow \mathbb{R}$ be differentiable at θ with $g'(\theta) \neq 0$. Then:

$$\sqrt{n}(g(\hat{\theta}_n) - g(\theta)) \xrightarrow{d} \mathcal{N}(0, [g'(\theta)]^2 \sigma^2) \quad (54)$$

Proof. By first-order Taylor expansion around θ :

$$g(\hat{\theta}_n) = g(\theta) + g'(\theta)(\hat{\theta}_n - \theta) + R_n \quad (55)$$

where the remainder satisfies $R_n = o_p(|\hat{\theta}_n - \theta|)$.

Rearranging:

$$g(\hat{\theta}_n) - g(\theta) = g'(\theta)(\hat{\theta}_n - \theta) + R_n \quad (56)$$

Multiplying by $\sqrt{n}$:

$$\sqrt{n}(g(\hat{\theta}_n) - g(\theta)) = g'(\theta)\sqrt{n}(\hat{\theta}_n - \theta) + \sqrt{n}R_n \quad (57)$$

Since $\hat{\theta}_n \xrightarrow{p} \theta$ (by assumption) and $R_n = o_p(|\hat{\theta}_n - \theta|)$, we have:

$$\sqrt{n}R_n = \sqrt{n} \cdot o_p(|\hat{\theta}_n - \theta|) = o_p(\sqrt{n}|\hat{\theta}_n - \theta|) = o_p(1) \quad (58)$$

Therefore:

$$\sqrt{n}(g(\hat{\theta}_n) - g(\theta)) = g'(\theta)\sqrt{n}(\hat{\theta}_n - \theta) + o_p(1) \quad (59)$$

By assumption, $\sqrt{n}(\hat{\theta}_n - \theta) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$, and by continuous mapping theorem (multiplication by constant):

$$g'(\theta)\sqrt{n}(\hat{\theta}_n - \theta) \xrightarrow{d} \mathcal{N}(0, [g'(\theta)]^2 \sigma^2) \quad (60)$$

By Slutsky's theorem ($o_p(1) \rightarrow 0$), the result follows. $\square$

Example .3 (Application to Relative Lift). For conversion rates $\hat{p}_1, \hat{p}_2$, the relative lift is:

$$g(p_1, p_2) = \frac{p_2 - p_1}{p_1} = \frac{p_2}{p_1} - 1 \quad (61)$$

By multivariate delta method:

$$\text{Var}\left(\frac{\hat{p}_2 - \hat{p}_1}{\hat{p}_1}\right) \approx \frac{1}{p_1^2} \text{Var}(\hat{p}_2) + \frac{p_2^2}{p_1^4} \text{Var}(\hat{p}_1) \quad (62)$$

.3 Power Function Derivations

The power function characterizes the probability of rejecting H_0 as a function of the true parameter value.

.3.1 Power Function for Two-Sample t-Test

Definition .1 (Power Function). For testing $H_0 : \mu_1 = \mu_2$ versus $H_1 : \mu_1 \neq \mu_2$, the power function is:

$$\beta(\delta) = P(\text{Reject } H_0 \mid \mu_2 - \mu_1 = \delta) \quad (63)$$

where δ is the true mean difference.

Theorem .7 (Power Function for t-Test). For the two-sample t-test with equal sample sizes n per group and common variance σ^2 :

$$\beta(\delta) = \Phi\left(\frac{|\delta|\sqrt{n}}{\sigma\sqrt{2}} - z_{\alpha/2}\right) + \Phi\left(-\frac{|\delta|\sqrt{n}}{\sigma\sqrt{2}} - z_{\alpha/2}\right) \quad (64)$$

where Φ is the standard normal CDF.

Proof. The test statistic is:

$$T = \frac{\bar{X}_2 - \bar{X}_1}{\sqrt{2\sigma^2/n}} \quad (65)$$

Reject H_0 if $|T| > z_{\alpha/2}$, i.e., if $T > z_{\alpha/2}$ or $T < -z_{\alpha/2}$.

Under the alternative $\mu_2 - \mu_1 = \delta$:

$$\mathbb{E}[T \mid \delta] = \frac{\delta}{\sqrt{2\sigma^2/n}} = \frac{\delta\sqrt{n}}{\sigma\sqrt{2}} \quad (66)$$

and $\text{Var}(T) = 1$. Therefore:

$$T \mid \delta \sim \mathcal{N}\left(\frac{\delta\sqrt{n}}{\sigma\sqrt{2}}, 1\right) \quad (67)$$

The power is:

$$\beta(\delta) = P(T > z_{\alpha/2} \mid \delta) + P(T < -z_{\alpha/2} \mid \delta) \quad (68)$$

$$= P\left(\mathcal{N}\left(\frac{\delta\sqrt{n}}{\sigma\sqrt{2}}, 1\right) > z_{\alpha/2}\right) + P\left(\mathcal{N}\left(\frac{\delta\sqrt{n}}{\sigma\sqrt{2}}, 1\right) < -z_{\alpha/2}\right) \quad (69)$$

Standardizing:

$$= P\left(\mathcal{N}(0, 1) > z_{\alpha/2} - \frac{\delta\sqrt{n}}{\sigma\sqrt{2}}\right) + P\left(\mathcal{N}(0, 1) < -z_{\alpha/2} - \frac{\delta\sqrt{n}}{\sigma\sqrt{2}}\right) \quad (70)$$

$$= \Phi\left(\frac{\delta\sqrt{n}}{\sigma\sqrt{2}} - z_{\alpha/2}\right) + \Phi\left(-\frac{\delta\sqrt{n}}{\sigma\sqrt{2}} - z_{\alpha/2}\right) \quad (71)$$

For δ large, the second term is negligible, so:

$$\beta(\delta) \approx \Phi\left(\frac{|\delta|\sqrt{n}}{\sigma\sqrt{2}} - z_{\alpha/2}\right) \quad (72)$$

□

.3.2 Power Function for Proportion Test

Theorem .8 (Power for Two-Proportion Test). For testing $H_0 : p_1 = p_2$ versus $H_1 : p_1 \neq p_2$ with sample size n per group:

$$\beta(p_1, p_2) \approx \Phi\left(\frac{(p_2 - p_1)\sqrt{n}}{\sqrt{p_1(1 - p_1) + p_2(1 - p_2)}} - z_{\alpha/2}\right) \quad (73)$$

for $p_2 > p_1$.

Proof. Under H_1 , the test statistic:

$$Z = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{[p_1(1 - p_1) + p_2(1 - p_2)]/n}} \sim \mathcal{N}\left(\frac{(p_2 - p_1)\sqrt{n}}{\sqrt{p_1(1 - p_1) + p_2(1 - p_2)}}, 1\right) \quad (74)$$

Reject if $|Z| > z_{\alpha/2}$ (using pooled variance approximation under H_0). The power is:

$$\beta = P(Z > z_{\alpha/2} \mid H_1) \quad (75)$$

$$= \Phi\left(\frac{(p_2 - p_1)\sqrt{n}}{\sqrt{p_1(1 - p_1) + p_2(1 - p_2)}} - z_{\alpha/2}\right) \quad (76)$$

□

.3.3 Non-Centrality Parameter

Definition .2 (Non-Centrality Parameter). For the two-sample t-test, the non-centrality parameter is:

$$\lambda = \frac{\delta}{\sigma} \sqrt{\frac{n}{2}} \quad (77)$$

Theorem .9 (Power in Terms of Non-Central t). The exact power function using the non-central t-distribution with $\nu = 2(n - 1)$ degrees of freedom and non-centrality parameter λ is:

$$\beta(\delta) = P(t_{\nu,\lambda} > t_{\nu,\alpha/2}) + P(t_{\nu,\lambda} < -t_{\nu,\alpha/2}) \quad (78)$$

where $t_{\nu,\lambda}$ denotes a non-central t random variable.

Remark .3. For large n , the normal approximation we derived earlier is excellent. For small n , the non-central t distribution should be used.

.4 Bayesian Posterior Calculations

Bayesian inference provides an alternative framework for A/B testing, incorporating prior knowledge.

.4.1 Beta-Binomial Model

Theorem .10 (Beta-Binomial Conjugacy). Let $p \sim \text{Beta}(\alpha, \beta)$ be the prior distribution for a proportion. If we observe $X \sim \text{Binomial}(n, p)$ successes in n trials, the posterior distribution is:

$$p \mid X \sim \text{Beta}(\alpha + X, \beta + n - X) \quad (79)$$

Proof. By Bayes' theorem:

$$f(p \mid X) = \frac{f(X \mid p)f(p)}{f(X)} \propto f(X \mid p)f(p) \quad (80)$$

The likelihood function is:

$$f(X \mid p) = \binom{n}{X} p^X (1 - p)^{n-X} \propto p^X (1 - p)^{n-X} \quad (81)$$

The prior density is:

$$f(p) = \frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1} (1 - p)^{\beta-1} \propto p^{\alpha-1} (1 - p)^{\beta-1} \quad (82)$$

The posterior is proportional to:

$$f(p \mid X) \propto p^X (1 - p)^{n-X} \cdot p^{\alpha-1} (1 - p)^{\beta-1} \quad (83)$$

$$= p^{X+\alpha-1} (1 - p)^{n-X+\beta-1} \quad (84)$$

This is the kernel of a $\text{Beta}(\alpha + X, \beta + n - X)$ distribution. The normalizing constant makes it a proper density:

$$f(p \mid X) = \frac{\Gamma(\alpha + \beta + n)}{\Gamma(\alpha + X)\Gamma(\beta + n - X)} p^{\alpha+X-1} (1 - p)^{\beta+n-X-1} \quad (85)$$

□

.4.2 Posterior Predictive Distribution

Theorem .11 (Beta-Binomial Predictive). Given posterior $p \mid X \sim \text{Beta}(\alpha + X, \beta + n - X)$, the predictive probability of success in the next trial is:

$$P(\tilde{X} = 1 \mid X) = \frac{\alpha + X}{\alpha + \beta + n} \quad (86)$$

Proof. The posterior predictive distribution is:

$$P(\tilde{X} = 1 \mid X) = \int_0^1 P(\tilde{X} = 1 \mid p) f(p \mid X) dp \quad (87)$$

$$= \int_0^1 p \cdot f(p \mid X) dp \quad (88)$$

$$= \mathbb{E}[p \mid X] \quad (89)$$

For $p \sim \text{Beta}(\alpha', \beta')$, the mean is $\mathbb{E}[p] = \frac{\alpha'}{\alpha' + \beta'}$. Therefore:

$$P(\tilde{X} = 1 \mid X) = \frac{\alpha + X}{\alpha + X + \beta + n - X} = \frac{\alpha + X}{\alpha + \beta + n} \quad (90)$$

□

.4.3 Probability of Superiority

Theorem .12 (Closed Form for Beta Distributions). If $p_A \sim \text{Beta}(\alpha_A, \beta_A)$ and $p_B \sim \text{Beta}(\alpha_B, \beta_B)$ independently, then:

$$P(p_B > p_A) = \sum_{i=0}^{\alpha_B-1} \frac{B(\alpha_A + i, \beta_A + \beta_B)}{(\beta_B + i) \cdot B(1 + i, \beta_B) \cdot B(\alpha_A, \beta_A)} \quad (91)$$

where $B(\cdot, \cdot)$ is the beta function.

Proof. By definition:

$$P(p_B > p_A) = \int_0^1 \int_0^{p_B} f_A(p_A) f_B(p_B) dp_A dp_B \quad (92)$$

$$= \int_0^1 f_B(p_B) F_A(p_B) dp_B \quad (93)$$

where F_A is the CDF of $\text{Beta}(\alpha_A, \beta_A)$.

The CDF of a Beta distribution can be expressed in terms of the incomplete beta function:

$$F_A(p) = I_p(\alpha_A, \beta_A) = \frac{B_p(\alpha_A, \beta_A)}{B(\alpha_A, \beta_A)} \quad (94)$$

Substituting and using properties of beta integrals leads to the series representation above.

For computational purposes, when α_A, α_B are integers (as in A/B testing with count data), this series is finite and easily computed. □

4.4 Normal-Normal Conjugacy

Theorem .13 (Normal Prior, Normal Likelihood). Suppose $\mu \sim \mathcal{N}(\mu_0, \tau^2)$ is the prior for a mean, and we observe data $\bar{X} \mid \mu \sim \mathcal{N}(\mu, \sigma^2/n)$ with known variance σ^2 . Then the posterior is:

$$\mu \mid \bar{X} \sim \mathcal{N}\left(\frac{\frac{n}{\sigma^2}\bar{X} + \frac{1}{\tau^2}\mu_0}{\frac{n}{\sigma^2} + \frac{1}{\tau^2}}, \left(\frac{n}{\sigma^2} + \frac{1}{\tau^2}\right)^{-1}\right) \quad (95)$$

Proof. The posterior is proportional to likelihood times prior:

$$f(\mu \mid \bar{X}) \propto \exp\left(-\frac{n(\bar{X} - \mu)^2}{2\sigma^2}\right) \cdot \exp\left(-\frac{(\mu - \mu_0)^2}{2\tau^2}\right) \quad (96)$$

$$\propto \exp\left(-\frac{1}{2}\left[\frac{n(\bar{X} - \mu)^2}{\sigma^2} + \frac{(\mu - \mu_0)^2}{\tau^2}\right]\right) \quad (97)$$

Expand the exponent:

$$\frac{n(\bar{X} - \mu)^2}{\sigma^2} + \frac{(\mu - \mu_0)^2}{\tau^2} \quad (98)$$

$$= \frac{n}{\sigma^2}(\bar{X}^2 - 2\bar{X}\mu + \mu^2) + \frac{1}{\tau^2}(\mu^2 - 2\mu_0\mu + \mu_0^2) \quad (99)$$

$$= \left(\frac{n}{\sigma^2} + \frac{1}{\tau^2}\right)\mu^2 - 2\left(\frac{n\bar{X}}{\sigma^2} + \frac{\mu_0}{\tau^2}\right)\mu + \text{const} \quad (100)$$

Let $\lambda = \frac{n}{\sigma^2} + \frac{1}{\tau^2}$ (posterior precision). Complete the square:

$$= \lambda \left[\mu^2 - 2\mu \cdot \frac{1}{\lambda} \left(\frac{n\bar{X}}{\sigma^2} + \frac{\mu_0}{\tau^2} \right) \right] + \text{const} \quad (101)$$

$$= \lambda \left(\mu - \frac{1}{\lambda} \left(\frac{n\bar{X}}{\sigma^2} + \frac{\mu_0}{\tau^2} \right) \right)^2 + \text{const} \quad (102)$$

This is the kernel of $\mathcal{N}(\mu_n, \sigma_n^2)$ where:

$$\mu_n = \frac{1}{\lambda} \left(\frac{n\bar{X}}{\sigma^2} + \frac{\mu_0}{\tau^2} \right) = \frac{\frac{n}{\sigma^2}\bar{X} + \frac{1}{\tau^2}\mu_0}{\frac{n}{\sigma^2} + \frac{1}{\tau^2}} \quad (103)$$

$$\sigma_n^2 = \frac{1}{\lambda} = \left(\frac{n}{\sigma^2} + \frac{1}{\tau^2} \right)^{-1} \quad (104)$$

□

Remark .4 (Interpretation). The posterior mean is a precision-weighted average of the prior mean μ_0 and the data mean $\bar{X}$. As $n \rightarrow \infty$, the posterior converges to $\mathcal{N}(\bar{X}, \sigma^2/n)$, i.e., the data overwhelms the prior.

5 Sequential Testing Boundaries

Sequential testing allows early stopping based on accumulating evidence, while controlling error rates.

.5.1 Sequential Probability Ratio Test (SPRT)

Theorem .14 (Wald's SPRT). For testing $H_0 : \theta = \theta_0$ versus $H_1 : \theta = \theta_1$, the sequential probability ratio test stops at time n when:

$$\Lambda_n = \frac{L(\theta_1; X_1, \dots, X_n)}{L(\theta_0; X_1, \dots, X_n)} \notin (B, A) \quad (105)$$

and rejects H_0 if $\Lambda_n \geq A$, accepts H_0 if $\Lambda_n \leq B$, where:

$$A = \frac{1 - \beta}{\alpha}, \quad B = \frac{\beta}{1 - \alpha} \quad (106)$$

for desired error rates α (Type I) and β (Type II).

Proof Sketch. At stopping time τ , if we reject H_0 :

$$\frac{P(X_1, \dots, X_\tau \mid \theta_1)}{P(X_1, \dots, X_\tau \mid \theta_0)} \geq A \quad (107)$$

This implies:

$$P(X_1, \dots, X_\tau \mid \theta_1) \geq A \cdot P(X_1, \dots, X_\tau \mid \theta_0) \quad (108)$$

Summing over all sequences leading to rejection:

$$P(\text{Reject} \mid \theta_1) \geq A \cdot P(\text{Reject} \mid \theta_0) \quad (109)$$

Therefore:

$$P(\text{Reject} \mid \theta_0) \leq \frac{P(\text{Reject} \mid \theta_1)}{A} = \frac{1 - \beta}{A} \quad (110)$$

Wald showed that choosing $A = (1 - \beta)/\alpha$ achieves approximately $P(\text{Reject} \mid \theta_0) \approx \alpha$. $\square$

.5.2 Group Sequential Boundaries

For group sequential designs with K planned looks, we derive boundaries that maintain overall Type I error rate α .

Theorem .15 (O'Brien-Fleming Boundaries). For K interim analyses at information fractions $t_1, \dots, t_K$ (where $t_K = 1$), the O'Brien-Fleming boundaries are:

$$c_k = \frac{c}{\sqrt{t_k}}, \quad k = 1, \dots, K \quad (111)$$

where c is chosen such that:

$$P\left(\max_{1 \leq k \leq K} \frac{Z_k}{\sqrt{t_k}} > c \mid H_0\right) = \alpha \quad (112)$$

and Z_k is the standardized test statistic at analysis k .

Derivation. Let $Z_1, \dots, Z_K$ be the sequence of standardized test statistics with information times $t_1 < \dots < t_K = 1$.

Under H_0 , by the CLT and properties of independent increments:

$$(Z_1, \dots, Z_K) \text{ follows a multivariate normal with } \text{Cov}(Z_i, Z_j) = \sqrt{t_i/t_j} \text{ for } i \leq j \quad (113)$$

The O'Brien-Fleming approach standardizes by $\sqrt{t_k}$ at each look:

$$W_k = \frac{Z_k}{\sqrt{t_k}} \quad (114)$$

We reject H_0 at the first k where $|W_k| > c$.

The Type I error is:

$$\alpha = P\left(\bigcup_{k=1}^K \{|W_k| > c\}\right) = P\left(\max_{1 \leq k \leq K} |W_k| > c\right) \quad (115)$$

For equal spacing and K looks, approximate values of c are:

- $K = 2$: $c \approx 2.24$ (vs. 1.96 for fixed design)
- $K = 5$: $c \approx 2.04$
- $K \rightarrow \infty$: $c \rightarrow 1.96$

These are computed numerically to achieve exact α (typically $\alpha = 0.05$). □

.5.3 Pocock Boundaries

Theorem .16 (Pocock Boundaries). The Pocock boundaries use a constant critical value at each analysis:

$$c_k = c, \quad k = 1, \dots, K \quad (116)$$

where c satisfies:

$$P\left(\max_{1 \leq k \leq K} Z_k > c \mid H_0\right) = \alpha \quad (117)$$

Remark .5 (Comparison). • O'Brien-Fleming: Early boundaries are very high (conservative early, liberal late). Preserves power if test goes to completion.

- Pocock: Constant boundaries (liberal early, conservative late). Higher early stopping probability.

.5.4 Alpha Spending Functions

Definition .3 (Alpha Spending Function). An alpha spending function $\alpha(t)$ for $t \in [0, 1]$ satisfies:

1. $\alpha(0) = 0$
2. $\alpha(1) = \alpha$ (overall Type I error)
3. $\alpha(t)$ is non-decreasing

At analysis k with information time t_k , the incremental alpha is:

$$\alpha_k = \alpha(t_k) - \alpha(t_{k-1}) \quad (118)$$

Example .4 (Common Spending Functions). **O'Brien-Fleming type:**

$$\alpha(t) = 2 \left[1 - \Phi \left(\frac{z_{\alpha/2}}{\sqrt{t}} \right) \right] \quad (119)$$

Pocock type:

$$\alpha(t) = \alpha \log(1 + (e - 1)t) \quad (120)$$

Power family:

$$\alpha(t) = \alpha t^\rho, \quad \rho > 0 \quad (121)$$

.5.5 Information-Based Monitoring

Theorem .17 (Conditional Error Principle). At interim analysis k with observed test statistic Z_k and information fraction t_k , the boundary c_k is determined by:

$$P \left(\bigcup_{j=k}^K \{Z_j > c_j\} \mid Z_k \right) = \frac{\alpha(t_k) - \alpha(t_{k-1})}{1 - \alpha(t_{k-1})} \quad (122)$$

given that H_0 was not rejected at previous looks.

Derivation Sketch. This follows from the law of total probability and the requirement that:

$$P(\text{Reject at or after look } k \mid H_0) = \alpha(t_k) \quad (123)$$

Conditioning on not having rejected before look k :

$$\alpha(t_k) = P(\text{Reject at look } k \mid H_0) + P(\text{Reject after look } k \mid H_0) \quad (124)$$

$$= P(\text{Reject at look } k \mid H_0, \text{Not rejected before}) \cdot P(\text{Not rejected before}) \quad (125)$$

$$+ P(\text{Reject after look } k \mid H_0, \text{Not rejected at } k) \cdot P(\text{Not rejected at or before } k) \quad (126)$$

Working through the conditional probabilities yields the stated formula for determining c_k . $\square$

.6 Additional Results

.6.1 Variance of Sample Proportion

Theorem .18. For $\hat{p} = X/n$ where $X \sim \text{Binomial}(n, p)$:

$$\text{Var}(\hat{p}) = \frac{p(1-p)}{n} \quad (127)$$

Proof. Since $X \sim \text{Binomial}(n, p)$, we have $\text{Var}(X) = np(1-p)$. Therefore:

$$\text{Var}(\hat{p}) = \text{Var} \left(\frac{X}{n} \right) = \frac{1}{n^2} \text{Var}(X) = \frac{1}{n^2} \cdot np(1-p) = \frac{p(1-p)}{n} \quad (128)$$

$\square$

.6.2 CUPED Variance Reduction

Theorem .19 (CUPED Estimator). For outcome Y and covariate X , the CUPED-adjusted estimator:

$$Y^* = Y - \theta(X - \mathbb{E}[X]) \quad (129)$$

with optimal $\theta = \text{Cov}(Y, X)/\text{Var}(X)$ achieves variance:

$$\text{Var}(Y^*) = \text{Var}(Y)(1 - \rho^2) \quad (130)$$

where $\rho = \text{Cor}(Y, X)$ is the correlation coefficient.

Proof. Expanding the variance:

$$\text{Var}(Y^*) = \text{Var}(Y - \theta(X - \mathbb{E}[X])) \quad (131)$$

$$= \text{Var}(Y) + \theta^2 \text{Var}(X) - 2\theta \text{Cov}(Y, X) \quad (132)$$

Substituting $\theta = \text{Cov}(Y, X)/\text{Var}(X)$:

$$\text{Var}(Y^*) = \text{Var}(Y) + \frac{\text{Cov}(Y, X)^2}{\text{Var}(X)^2} \text{Var}(X) - 2 \frac{\text{Cov}(Y, X)}{\text{Var}(X)} \text{Cov}(Y, X) \quad (133)$$

$$= \text{Var}(Y) + \frac{\text{Cov}(Y, X)^2}{\text{Var}(X)} - \frac{2\text{Cov}(Y, X)^2}{\text{Var}(X)} \quad (134)$$

$$= \text{Var}(Y) - \frac{\text{Cov}(Y, X)^2}{\text{Var}(X)} \quad (135)$$

Since $\rho = \text{Cov}(Y, X)/(\sigma_Y \sigma_X)$, we have $\text{Cov}(Y, X)^2 = \rho^2 \text{Var}(Y) \text{Var}(X)$:

$$\text{Var}(Y^*) = \text{Var}(Y) - \frac{\rho^2 \text{Var}(Y) \text{Var}(X)}{\text{Var}(X)} \quad (136)$$

$$= \text{Var}(Y)(1 - \rho^2) \quad (137)$$

□

.6.3 Bonferroni and Šidák Corrections

Theorem .20 (Bonferroni Inequality). For m hypothesis tests, each conducted at level α/m , the family-wise error rate satisfies:

$$\text{FWER} \leq \alpha \quad (138)$$

Proof. Let R_i be the event that test i rejects a true null hypothesis. By Boole's inequality (union bound):

$$\text{FWER} = P\left(\bigcup_{i=1}^m R_i\right) \quad (139)$$

$$\leq \sum_{i=1}^m P(R_i) \quad (140)$$

$$= \sum_{i=1}^m \frac{\alpha}{m} = \alpha \quad (141)$$

□

Theorem .21 (Šidák Correction). If the m tests are independent, testing each at level $\alpha_s = 1 - (1 - \alpha)^{1/m}$ gives exact FWER = α .

Proof. Under independence:

$$\text{FWER} = P(\text{Reject at least one true } H_0) \quad (142)$$

$$= 1 - P(\text{Accept all true } H_0) \quad (143)$$

$$= 1 - \prod_{i=1}^m P(\text{Accept } H_{0i}) \quad (144)$$

$$= 1 - \prod_{i=1}^m (1 - \alpha_s) \quad (145)$$

$$= 1 - (1 - \alpha_s)^m \quad (146)$$

Setting this equal to α :

$$1 - (1 - \alpha_s)^m = \alpha \quad (147)$$

$$(1 - \alpha_s)^m = 1 - \alpha \quad (148)$$

$$\alpha_s = 1 - (1 - \alpha)^{1/m} \quad (149)$$

□

Remark .6. The Šidák correction is less conservative than Bonferroni when tests are independent, since $(1 - \alpha)^{1/m} > 1 - \alpha/m$ for $m > 1$.

.7 Summary

This appendix has provided rigorous mathematical foundations for:

- **Sample size formulas:** Derived for t-tests and proportion tests, including unequal allocation
- **Central Limit Theorem:** Proof via characteristic functions with Berry-Esseen bound
- **Power functions:** Explicit derivations showing power as function of effect size
- **Bayesian methods:** Beta-binomial and normal-normal conjugacy with posterior calculations
- **Sequential testing:** SPRT, O'Brien-Fleming, and Pocock boundaries with alpha spending functions

These mathematical results underpin the practical A/B testing methods presented throughout this handbook.

Code Examples

Code Examples and Implementation

This appendix provides production-ready code implementations for A/B testing across multiple platforms and languages. All code examples include comprehensive documentation, error handling, and usage examples.

.1 Python Implementation

.1.1 Complete ABTest Framework

Listing 11: ABTest Framework - Core Classes

```
1  """
2  A/B Testing Framework
3  =====
4
5  Complete framework for designing, running, and analyzing A/B tests.
6
7  Author: A/B Testing Handbook
8  License: MIT
9  """
10
11 import numpy as np
12 import pandas as pd
13 from scipy import stats
14 from dataclasses import dataclass, field
15 from typing import Optional, Dict, List, Tuple, Union
16 from datetime import datetime, timedelta
17 import warnings
18
19 #
20 # Data Classes
21 #
22
23 @dataclass
24 class ExperimentConfig:
25     """Configuration for A/B test experiment."""
26
27     experiment_id: str
28     name: str
29     description: str
30     hypothesis: str
31
```

```

32 # Design parameters
33 variants: List[str]
34 primary_metric: str
35 secondary_metrics: List[str] = field(default_factory=list)
36 guardrail_metrics: List[str] = field(default_factory=list)
37
38 # Statistical parameters
39 alpha: float = 0.05
40 power: float = 0.80
41 mde: Optional[float] = None # Minimum detectable effect
42
43 # Sample size
44 sample_size_per_variant: Optional[int] = None
45
46 # Duration
47 start_date: Optional[datetime] = None
48 end_date: Optional[datetime] = None
49
50 # Traffic allocation
51 traffic_allocation: Dict[str, float] = field(default_factory=dict)
52
53 # Metadata
54 owner: str = ""
55 tags: List[str] = field(default_factory=list)
56
57 def __post_init__(self):
58     """Validate configuration."""
59     if len(self.variants) < 2:
60         raise ValueError("At least 2 variants required")
61
62     if self.traffic_allocation:
63         total = sum(self.traffic_allocation.values())
64         if not np.isclose(total, 1.0):
65             raise ValueError(f"Traffic allocation must sum to 1.0, got {total}")
66
67
68 @dataclass
69 class ABTestResult:
70     """Results from A/B test analysis."""
71
72     metric: str
73     control_mean: float
74     treatment_mean: float
75
76     # Effect sizes
77     absolute_difference: float
78     relative_lift: float
79
80     # Statistical inference
81     standard_error: float
82     confidence_interval: Tuple[float, float]
83     p_value: float
84     test_statistic: float
85
86     # Decision support
87     statistically_significant: bool
88     practically_significant: Optional[bool] = None

```

```

89
90     # Sample information
91     sample_size_control: int
92     sample_size_treatment: int
93
94     # Additional metrics
95     cohens_d: Optional[float] = None
96     power_achieved: Optional[float] = None
97
98
99     #
100     =====
101
102 # Statistical Analysis Classes
103 #
104     =====
105
106
107 class SampleSizeCalculator:
108     """Calculate required sample sizes for A/B tests."""
109
110     @staticmethod
111     def proportions(p1: float, p2: float, alpha: float = 0.05,
112                   power: float = 0.80, ratio: float = 1.0) -> Dict:
113         """
114         =====
115         Sample size for comparing two proportions.
116         =====
117         Parameters
118         -----
119         p1: float
120             Control proportion (baseline)
121         p2: float
122             Treatment proportion (alternative)
123         alpha: float
124             Significance level (default 0.05)
125         power: float
126             Statistical power (default 0.80)
127         ratio: float
128             Allocation ratio n2/n1 (default 1.0 for equal allocation)
129
130         Returns
131         -----
132         dict
133             Required sample sizes and related information
134
135         Examples
136         -----
137         >>> calc = SampleSizeCalculator()
138         >>> result = calc.proportions(p1=0.10, p2=0.12, power=0.80)
139         >>> print(f"Need {result['n1']} per group")
140         """
141         z_alpha = stats.norm.ppf(1 - alpha/2)
142         z_beta = stats.norm.ppf(power)
143
144         # Variance under H1
145         var_sum = p1*(1-p1) + p2*(1-p2)/ratio
146
147         # Effect size

```

```

143     delta = abs(p2 - p1)
144
145     # Sample size for group 1 (control)
146     n1 = ((z_alpha + z_beta)**2 * var_sum) / (delta**2)
147     n1 = int(np.ceil(n1))
148
149     # Sample size for group 2 (treatment)
150     n2 = int(np.ceil(n1 * ratio))
151
152     return {
153         'n1': n1,
154         'n2': n2,
155         'n_total': n1 + n2,
156         'n_per_group': n1 if ratio == 1.0 else None,
157         'p1': p1,
158         'p2': p2,
159         'effect_absolute': delta,
160         'effect_relative': delta / p1,
161         'alpha': alpha,
162         'power': power,
163         'ratio': ratio
164     }
165
166     @staticmethod
167     def means(mu1: float, mu2: float, sigma: float,
168             alpha: float = 0.05, power: float = 0.80,
169             ratio: float = 1.0) -> Dict:
170         """
171         Sample size for comparing two means.
172
173         Parameters
174         -----
175         mu1: float
176             Control mean
177         mu2: float
178             Treatment mean
179         sigma: float
180             Common standard deviation
181         alpha: float
182             Significance level
183         power: float
184             Statistical power
185         ratio: float
186             Allocation ratio n2/n1
187
188         Returns
189         -----
190         dict
191             Required sample sizes
192         """
193         z_alpha = stats.norm.ppf(1 - alpha/2)
194         z_beta = stats.norm.ppf(power)
195
196         delta = abs(mu2 - mu1)
197
198         # Sample size for group 1
199         n1 = ((z_alpha + z_beta)**2 * sigma**2 * (1 + 1/ratio)) / (
            delta**2)

```

```

200     n1 = int(np.ceil(n1))
201     n2 = int(np.ceil(n1 * ratio))
202
203     return {
204         'n1': n1,
205         'n2': n2,
206         'n_total': n1 + n2,
207         'n_per_group': n1 if ratio == 1.0 else None,
208         'mu1': mu1,
209         'mu2': mu2,
210         'sigma': sigma,
211         'effect_absolute': delta,
212         'effect_standardized': delta / sigma,
213         'alpha': alpha,
214         'power': power,
215         'ratio': ratio
216     }
217
218
219 class ABTestAnalyzer:
220     """Comprehensive A/B test statistical analysis."""
221
222     def __init__(self, alpha: float = 0.05):
223         """
224         Initialize analyzer.
225
226         Parameters
227         -----
228         alpha: float
229             Significance level for hypothesis tests
230         """
231         self.alpha = alpha
232
233     def analyze_proportions(self,
234                             control_conversions: int,
235                             control_total: int,
236                             treatment_conversions: int,
237                             treatment_total: int,
238                             method: str = 'z_test',
239                             mde: Optional[float] = None) -> ABTestResult:
240         """
241         Analyze A/B test with binary outcome (conversion).
242
243         Parameters
244         -----
245         control_conversions: int
246             Number of conversions in control
247         control_total: int
248             Total users in control
249         treatment_conversions: int
250             Number of conversions in treatment
251         treatment_total: int
252             Total users in treatment
253         method: str
254             Statistical test: 'z_test', 'chi_squared', 'fisher_exact'
255         mde: float, optional
256             Minimum detectable effect for practical significance

```

```

257
258 #####Returns
259 #####-----
260 #####ABTestResult
261 #####Complete_analysis_results
262
263 #####Examples
264 #####-----
265 #####>>> analyzer = ABTestAnalyzer(alpha=0.05)
266 #####>>> result = analyzer.analyze_proportions(
267 #####...control_conversions=100,
268 #####...control_total=1000,
269 #####...treatment_conversions=120,
270 #####...treatment_total=1000
271 #####...)
272 #####>>> print(f"P-value: {result.p_value:.4f}")
273 #####>>> print(f"Lift: {result.relative_lift:.2%}")
274 #####"""
275     # Calculate proportions
276     p1 = control_conversions / control_total
277     p2 = treatment_conversions / treatment_total
278
279     # Absolute and relative effects
280     abs_diff = p2 - p1
281     rel_lift = abs_diff / p1 if p1 > 0 else np.nan
282
283     # Perform statistical test
284     if method == 'z_test':
285         result = self._z_test_proportions(
286             control_conversions, control_total,
287             treatment_conversions, treatment_total
288         )
289     elif method == 'chi_squared':
290         result = self._chi_squared_test(
291             control_conversions, control_total,
292             treatment_conversions, treatment_total
293         )
294     elif method == 'fisher_exact':
295         result = self._fisher_exact_test(
296             control_conversions, control_total,
297             treatment_conversions, treatment_total
298         )
299     else:
300         raise ValueError(f"Unknown method: {method}")
301
302     # Calculate Cohen's h (effect size for proportions)
303     cohens_h = 2 * (np.arcsin(np.sqrt(p2)) - np.arcsin(np.sqrt(p1)))
304
305     # Practical significance
306     practically_sig = None
307     if mde is not None:
308         practically_sig = abs(abs_diff) >= mde
309
310     return ABTestResult(
311         metric="conversion_rate",
312         control_mean=p1,
313         treatment_mean=p2,

```



```

314         absolute_difference=abs_diff,
315         relative_lift=rel_lift,
316         standard_error=result['se'],
317         confidence_interval=result['ci'],
318         p_value=result['p_value'],
319         test_statistic=result['test_stat'],
320         statistically_significant=result['p_value'] < self.alpha,
321         practically_significant=practically_sig,
322         sample_size_control=control_total,
323         sample_size_treatment=treatment_total,
324         cohens_d=cohens_h
325     )
326
327     def _z_test_proportions(self, x1, n1, x2, n2) -> Dict:
328         """Two-proportion z-test."""
329         p1 = x1 / n1
330         p2 = x2 / n2
331
332         # Pooled proportion
333         p_pool = (x1 + x2) / (n1 + n2)
334
335         # Standard error under null
336         se_null = np.sqrt(p_pool * (1 - p_pool) * (1/n1 + 1/n2))
337
338         # Test statistic
339         z_stat = (p2 - p1) / se_null
340         p_value = 2 * (1 - stats.norm.cdf(abs(z_stat)))
341
342         # Confidence interval (unpooled variance)
343         se_unpooled = np.sqrt(p1*(1-p1)/n1 + p2*(1-p2)/n2)
344         z_crit = stats.norm.ppf(1 - self.alpha/2)
345         ci = (
346             (p2 - p1) - z_crit * se_unpooled,
347             (p2 - p1) + z_crit * se_unpooled
348         )
349
350         return {
351             'test_stat': z_stat,
352             'p_value': p_value,
353             'ci': ci,
354             'se': se_unpooled
355         }
356
357     def _chi_squared_test(self, x1, n1, x2, n2) -> Dict:
358         """Chi-squared test for proportions."""
359         table = np.array([[x1, n1 - x1], [x2, n2 - x2]])
360         chi2, p_value, dof, expected = stats.chi2_contingency(table)
361
362         # CI from z-test
363         p1 = x1 / n1
364         p2 = x2 / n2
365         se = np.sqrt(p1*(1-p1)/n1 + p2*(1-p2)/n2)
366         z_crit = stats.norm.ppf(1 - self.alpha/2)
367         ci = ((p2-p1) - z_crit*se, (p2-p1) + z_crit*se)
368
369         return {
370             'test_stat': chi2,
371             'p_value': p_value,

```

```

372         'ci': ci,
373         'se': se
374     }
375
376     def _fisher_exact_test(self, x1, n1, x2, n2) -> Dict:
377         """Fisher's exact test."""
378         table = np.array([[x1, n1 - x1], [x2, n2 - x2]])
379         _, p_value = stats.fisher_exact(table)
380
381         # CI from z-test
382         p1 = x1 / n1
383         p2 = x2 / n2
384         se = np.sqrt(p1*(1-p1)/n1 + p2*(1-p2)/n2)
385         z_crit = stats.norm.ppf(1 - self.alpha/2)
386         ci = ((p2-p1) - z_crit*se, (p2-p1) + z_crit*se)
387
388         return {
389             'test_stat': None,
390             'p_value': p_value,
391             'ci': ci,
392             'se': se
393         }
394
395     def analyze_means(self,
396                       control_data: np.ndarray,
397                       treatment_data: np.ndarray,
398                       equal_variance: bool = False,
399                       mde: Optional[float] = None) -> ABTestResult:
400         """
401         Analyze A/B test with continuous outcome.
402
403         Parameters
404         -----
405         control_data: array-like
406             Control group observations
407         treatment_data: array-like
408             Treatment group observations
409         equal_variance: bool
410             Assume equal variances (Student's t) vs unequal (Welch's t)
411         mde: float, optional
412             Minimum detectable effect
413
414         Returns
415         -----
416         ABTestResult
417             Analysis results
418         """
419         control = np.asarray(control_data)
420         treatment = np.asarray(treatment_data)
421
422         n1 = len(control)
423         n2 = len(treatment)
424         mean1 = np.mean(control)
425         mean2 = np.mean(treatment)
426         var1 = np.var(control, ddof=1)
427         var2 = np.var(treatment, ddof=1)
428         std1 = np.sqrt(var1)
429         std2 = np.sqrt(var2)

```

```

430
431     # T-test
432     t_stat, p_value = stats.ttest_ind(
433         treatment, control, equal_var=equal_variance
434     )
435
436     # Standard error
437     se = np.sqrt(var1/n1 + var2/n2)
438
439     # Degrees of freedom
440     if equal_variance:
441         df = n1 + n2 - 2
442     else:
443         # Welch-Satterthwaite
444         df = (var1/n1 + var2/n2)**2 / (
445             (var1/n1)**2/(n1-1) + (var2/n2)**2/(n2-1)
446         )
447
448     # Confidence interval
449     t_crit = stats.t.ppf(1 - self.alpha/2, df)
450     ci = (
451         (mean2 - mean1) - t_crit * se,
452         (mean2 - mean1) + t_crit * se
453     )
454
455     # Cohen's d
456     if equal_variance:
457         pooled_std = np.sqrt(((n1-1)*var1 + (n2-1)*var2) / (n1+n2
458             -2))
459         cohens_d = (mean2 - mean1) / pooled_std
460     else:
461         cohens_d = (mean2 - mean1) / np.sqrt((var1 + var2) / 2)
462
463     # Practical significance
464     practically_sig = None
465     if mde is not None:
466         practically_sig = abs(mean2 - mean1) >= mde
467
468     return ABTestResult(
469         metric="continuous_metric",
470         control_mean=mean1,
471         treatment_mean=mean2,
472         absolute_difference=mean2 - mean1,
473         relative_lift=(mean2 - mean1) / mean1 if mean1 != 0 else np
474             .nan,
475         standard_error=se,
476         confidence_interval=ci,
477         p_value=p_value,
478         test_statistic=t_stat,
479         statistically_significant=p_value < self.alpha,
480         practically_significant=practically_sig,
481         sample_size_control=n1,
482         sample_size_treatment=n2,
483         cohens_d=cohens_d
484     )
485
486 class DataQualityChecker:

```

```

486     """Pre-analysis_data_quality_checks."""
487
488     @staticmethod
489     def check_sample_ratio_mismatch(data: pd.DataFrame,
490                                     variant_col: str = 'variant',
491                                     expected_allocation: Optional[Dict]
492                                         = None,
493                                     alpha: float = 0.01) -> Dict:
494         """
495         Check for Sample Ratio Mismatch (SRM).
496         Parameters
497         -----
498         data: DataFrame
499             Experiment data
500         variant_col: str
501             Column name for variant assignment
502         expected_allocation: dict, optional
503             Expected traffic allocation {variant: proportion}
504         alpha: float
505             Significance level (stricter than test alpha)
506
507         Returns
508         -----
509         dict
510             SRM check results
511         """
512         observed = data[variant_col].value_counts()
513         total = len(data)
514
515         if expected_allocation is None:
516             # Assume equal allocation
517             n_variants = len(observed)
518             expected = {v: total / n_variants for v in observed.index}
519         else:
520             expected = {v: total * p for v, p in expected_allocation.
521                         items()}
522
523         # Chi-square test
524         obs_counts = np.array([observed[v] for v in expected.keys()])
525         exp_counts = np.array([expected[v] for v in expected.keys()])
526
527         chi2_stat, p_value = stats.chisquare(obs_counts, exp_counts)
528
529         return {
530             'passed': p_value >= alpha,
531             'p_value': p_value,
532             'chi2_statistic': chi2_stat,
533             'observed_counts': observed.to_dict(),
534             'expected_counts': expected,
535             'severity': 'CRITICAL' if p_value < 0.001 else
536                         'HIGH' if p_value < 0.01 else 'OK'
537         }
538
539     @staticmethod
540     def check_novelty_effect(data: pd.DataFrame,
541                             metric_col: str,
542                             variant_col: str,

```

```

542         date_col: str,
543         n_periods: int = 7) -> Dict:
544     """
545     Check for novelty/learning effects over time.
546
547     Parameters
548     -----
549     data: DataFrame
550     Experiment data with timestamps
551     metric_col: str
552     Metric to analyze
553     variant_col: str
554     Variant column
555     date_col: str
556     Date/timestamp column
557     n_periods: int
558     Number of time periods to analyze
559
560     Returns
561     -----
562     dict
563     Temporal stability analysis
564     """
565     data = data.copy()
566     data[date_col] = pd.to_datetime(data[date_col])
567
568     # Split into time periods
569     data['period'] = pd.cut(
570         data[date_col],
571         bins=n_periods,
572         labels=range(n_periods)
573     )
574
575     # Calculate metrics by period and variant
576     temporal_metrics = data.groupby(['period', variant_col])[metric_col].agg([
577         'mean', 'std', 'count'
578     ]).reset_index()
579
580     # Test for trend in treatment effect over time
581     control_data = temporal_metrics[
582         temporal_metrics[variant_col] == 'control'
583     ].set_index('period')['mean']
584
585     treatment_data = temporal_metrics[
586         temporal_metrics[variant_col] == 'treatment'
587     ].set_index('period')['mean']
588
589     effects = treatment_data - control_data
590
591     # Spearman correlation with time
592     periods = np.arange(len(effects))
593     corr, p_value = stats.spearmanr(periods, effects)
594
595     return {
596         'temporal_data': temporal_metrics,
597         'effects_by_period': effects.to_dict(),
598         'trend_correlation': corr,

```

```

599         'trend_p_value': p_value,
600         'stable': abs(corr) < 0.5 or p_value > 0.05
601     }
602
603
604 #
605 # Report Generation
606 #
607
608 class ResultsReporter:
609     """Generate formatted reports from analysis results."""
610
611     @staticmethod
612     def format_result(result: ABTestResult,
613                      metric_name: str = None,
614                      metric_format: str = '.4f') -> str:
615         """
616         Format AB test result as readable report.
617
618         Parameters
619         -----
620         result: ABTestResult
621         Analysis results
622         metric_name: str, optional
623         Display name for metric
624         metric_format: str
625         Number format string
626
627         Returns
628         -----
629         str
630         Formatted report
631         """
632         metric_name = metric_name or result.metric
633
634         report = f"""
635         {'='*70}
636         A/B TEST RESULTS: {metric_name.upper()}
637         {'='*70}
638
639         SAMPLE SIZES
640         -----
641         Control: {result.sample_size_control:,}
642         Treatment: {result.sample_size_treatment:,}
643         Total: {result.sample_size_control+result.sample_size_treatment:,}
644
645         METRIC VALUES
646         -----
647         Control Mean: {result.control_mean:{metric_format}}
648         Treatment Mean: {result.treatment_mean:{metric_format}}
649
650         EFFECT SIZES
651         -----

```

```

652 Absolute_Difference:_{result.absolute_difference:{metric_format}}
653 Relative_Lift:_{result.relative_lift:+.2%}
654 """
655
656     if result.cohens_d is not None:
657         report += f"Cohen's_d:_{result.cohens_d:.4f}\n"
658
659     report += f"""
660 STATISTICAL_INFERENCE
661 -----
662 Standard_Error:_{result.standard_error:{metric_format}}
663 95%_CI:_{result.confidence_interval[0]:{metric_format}},_{
        result.confidence_interval[1]:{metric_format}}]
664 Test_Statistic:_{result.test_statistic:.4f}
665 P-value:_{result.p_value:.6f}
666
667 DECISION
668 -----
669 Statistically_Significant:_{'YES' if result.statistically_significant
        else 'NO'}
670 """
671
672     if result.practically_significant is not None:
673         report += f"Practically_Significant:_{'YES' if result.
        practically_significant else 'NO'}\n"
674
675     report += f"\n{'='*70}\n"
676
677     return report
678
679     @staticmethod
680     def create_summary_table(results: List[ABTestResult],
681                             metric_names: List[str]) -> pd.DataFrame:
682         """
683         Create_summary_table_from_multiple_results.
684
685         Parameters
686         -----
687         results: _list_of_ABTestResult
688         Multiple_test_results
689         metric_names: _list_of_str
690         Names_for_each_metric
691
692         Returns
693         -----
694         DataFrame
695         Summary_table
696         """
697         rows = []
698         for result, name in zip(results, metric_names):
699             rows.append({
700                 'Metric': name,
701                 'Control': f"{result.control_mean:.4f}",
702                 'Treatment': f"{result.treatment_mean:.4f}",
703                 'Difference': f"{result.absolute_difference:.4f}",
704                 'Lift': f"{result.relative_lift:+.2%}",
705                 'CI_Lower': f"{result.confidence_interval[0]:.4f}",
706                 'CI_Upper': f"{result.confidence_interval[1]:.4f}",

```

```

707         'P-value': f"{result.p_value:.4f}",
708         'Significant': 'Yes' if result.statistically_
              significant else 'No',
709     })
710
711     return pd.DataFrame(rows)
712
713
714 #
715 # =====
716 # Complete Example Usage
717 # =====
718
719 if __name__ == "__main__":
720     print("A/B Testing Framework - Example Usage\n")
721
722     # 1. Calculate sample size
723     print("1. SAMPLE SIZE CALCULATION")
724     print("-" * 50)
725     calc = SampleSizeCalculator()
726     ss_result = calc.proportions(p1=0.10, p2=0.12, power=0.80)
727     print(f"Required per group: {ss_result['n_per_group']:,}")
728     print(f"Expected lift: {ss_result['effect_relative']:.2%}\n")
729
730     # 2. Generate simulated data
731     print("2. SIMULATED EXPERIMENT DATA")
732     print("-" * 50)
733     np.random.seed(42)
734     n_control = 5000
735     n_treatment = 5000
736
737     # Simulate conversions
738     control_conversions = np.random.binomial(n_control, 0.10)
739     treatment_conversions = np.random.binomial(n_treatment, 0.12)
740
741     print(f"Control: {control_conversions}/{n_control} = "
742           f"{control_conversions/n_control:.4f}")
743     print(f"Treatment: {treatment_conversions}/{n_treatment} = "
744           f"{treatment_conversions/n_treatment:.4f}\n")
745
746     # 3. Analyze experiment
747     print("3. STATISTICAL ANALYSIS")
748     print("-" * 50)
749     analyzer = ABTestAnalyzer(alpha=0.05)
750     result = analyzer.analyze_proportions(
751         control_conversions=control_conversions,
752         control_total=n_control,
753         treatment_conversions=treatment_conversions,
754         treatment_total=n_treatment,
755         mde=0.01 # 1 percentage point
756     )
757
758     # 4. Generate report
759     reporter = ResultsReporter()

```



```
759     print(reporter.format_result(result, metric_name="Conversion_Rate")
760           )
761
762     # 5. Data quality checks
763     print("4. DATA QUALITY CHECKS")
764     print("-" * 50)
765
766     # Create sample dataframe for quality checks
767     data = pd.DataFrame({
768         'variant': ([ 'control' ] * n_control) + ([ 'treatment' ] * n_
769             treatment),
770         'converted': ([1] * control_conversions +
771             [0] * (n_control - control_conversions) +
772             [1] * treatment_conversions +
773             [0] * (n_treatment - treatment_conversions))
774     })
775
776     checker = DataQualityChecker()
777     srm_check = checker.check_sample_ratio_mismatch(data)
778     print(f"SRM Check: {'PASSED' if srm_check['passed'] else 'FAILED'}")
779
780     print(f"P-value: {srm_check['p_value']:.6f}")
781     print(f"Severity: {srm_check['severity']}")
```

.2 R Implementation

.2.1 Statistical Analysis Functions

Listing 12: R Functions for A/B Test Analysis

```

1 #' A/B Test Analysis Functions in R
2 #'
3 #' Complete suite of functions for analyzing A/B tests
4 #'
5 #' @author A/B Testing Handbook
6 #' @license MIT
7
8 library(tidyverse)
9 library(broom)
10 library(pwr)
11
12 #
13 # =====
14 # Sample Size Calculation
15 # =====
16
17 #' Calculate sample size for proportion test
18 #'
19 #' @param p1 Control proportion
20 #' @param p2 Treatment proportion
21 #' @param alpha Significance level (default 0.05)
22 #' @param power Statistical power (default 0.80)
23 #' @param ratio Allocation ratio n2/n1 (default 1.0)
24 #'
25 #' @return List with sample size requirements
26 #'
27 #' @examples
28 #' calculate_sample_size_proportions(p1 = 0.10, p2 = 0.12)
29 #'
30 #' @export
31 calculate_sample_size_proportions <- function(p1, p2, alpha = 0.05,
32                                               power = 0.80, ratio =
33                                               1.0) {
34
35   # Effect size (Cohen's h)
36   h <- 2 * (asin(sqrt(p2)) - asin(sqrt(p1)))
37
38   # Sample size using pwr package
39   result <- pwr.2p.test(
40     h = h,
41     sig.level = alpha,
42     power = power,
43     alternative = "two.sided"
44   )
45
46   n_per_group <- ceiling(result$n)
47
48   list(
49     n1 = n_per_group,

```

```

48   n2 = ceiling(n_per_group * ratio),
49   n_total = n_per_group * (1 + ratio),
50   n_per_group = n_per_group,
51   p1 = p1,
52   p2 = p2,
53   effect_absolute = p2 - p1,
54   effect_relative = (p2 - p1) / p1,
55   effect_size_h = h,
56   alpha = alpha,
57   power = power
58 )
59 }
60
61
62 #' Calculate sample size for means test
63 #'
64 #' @param mu1 Control mean
65 #' @param mu2 Treatment mean
66 #' @param sigma Common standard deviation
67 #' @param alpha Significance level
68 #' @param power Statistical power
69 #' @param ratio Allocation ratio
70 #'
71 #' @return List with sample size requirements
72 #'
73 #' @export
74 calculate_sample_size_means <- function(mu1, mu2, sigma,
75                                       alpha = 0.05, power = 0.80,
76                                       ratio = 1.0) {
77
78   # Effect size (Cohen's d)
79   d <- abs(mu2 - mu1) / sigma
80
81   # Sample size
82   result <- pwr.t.test(
83     d = d,
84     sig.level = alpha,
85     power = power,
86     type = "two.sample",
87     alternative = "two.sided"
88   )
89
90   n_per_group <- ceiling(result$n)
91
92   list(
93     n1 = n_per_group,
94     n2 = ceiling(n_per_group * ratio),
95     n_total = n_per_group * (1 + ratio),
96     n_per_group = n_per_group,
97     mu1 = mu1,
98     mu2 = mu2,
99     sigma = sigma,
100    effect_absolute = mu2 - mu1,
101    effect_standardized = d,
102    alpha = alpha,
103    power = power
104  )
105 }

```

```

106
107
108 #
109 # Statistical Tests
110 #
111
112 #' Analyze proportions A/B test
113 #'
114 #' @param x1 Conversions in control
115 #' @param n1 Total in control
116 #' @param x2 Conversions in treatment
117 #' @param n2 Total in treatment
118 #' @param alpha Significance level
119 #' @param method Test method: "prop.test", "chisq", "fisher"
120 #'
121 #' @return List with test results
122 #'
123 #' @export
124 analyze_proportions <- function(x1, n1, x2, n2, alpha = 0.05,
125                                method = "prop.test") {
126
127   p1 <- x1 / n1
128   p2 <- x2 / n2
129
130   # Perform test
131   if (method == "prop.test") {
132     test <- prop.test(c(x1, x2), c(n1, n2), correct = FALSE)
133     p_value <- test$p.value
134     ci <- test$conf.int
135     test_stat <- test$statistic
136
137   } else if (method == "chisq") {
138     table <- matrix(c(x1, n1-x1, x2, n2-x2), nrow = 2, byrow = TRUE)
139     test <- chisq.test(table, correct = FALSE)
140     p_value <- test$p.value
141     test_stat <- test$statistic
142
143     # CI from prop.test
144     ci_test <- prop.test(c(x1, x2), c(n1, n2), correct = FALSE)
145     ci <- ci_test$conf.int
146
147   } else if (method == "fisher") {
148     table <- matrix(c(x1, n1-x1, x2, n2-x2), nrow = 2, byrow = TRUE)
149     test <- fisher.test(table)
150     p_value <- test$p.value
151     test_stat <- NA
152
153     # CI from prop.test
154     ci_test <- prop.test(c(x1, x2), c(n1, n2), correct = FALSE)
155     ci <- ci_test$conf.int
156   }
157
158   # Standard error (unpooled)
159   se <- sqrt(p1*(1-p1)/n1 + p2*(1-p2)/n2)

```

```

160
161 # Effect sizes
162 abs_diff <- p2 - p1
163 rel_lift <- abs_diff / p1
164
165 # Cohen's h
166 cohens_h <- 2 * (asin(sqrt(p2)) - asin(sqrt(p1)))
167
168 list(
169   control_mean = p1,
170   treatment_mean = p2,
171   absolute_difference = abs_diff,
172   relative_lift = rel_lift,
173   standard_error = se,
174   confidence_interval = ci,
175   p_value = p_value,
176   test_statistic = test_stat,
177   statistically_significant = p_value < alpha,
178   sample_size_control = n1,
179   sample_size_treatment = n2,
180   cohens_h = cohens_h,
181   method = method
182 )
183 }
184
185
186 #' Analyze continuous metric A/B test
187 #'
188 #' @param control_data Control group values
189 #' @param treatment_data Treatment group values
190 #' @param alpha Significance level
191 #' @param equal_var Assume equal variances (default FALSE)
192 #'
193 #' @return List with test results
194 #'
195 #' @export
196 analyze_means <- function(control_data, treatment_data, alpha = 0.05,
197   equal_var = FALSE) {
198
199   # T-test
200   test <- t.test(treatment_data, control_data, var.equal = equal_var)
201
202   # Summary statistics
203   mean1 <- mean(control_data)
204   mean2 <- mean(treatment_data)
205   sd1 <- sd(control_data)
206   sd2 <- sd(treatment_data)
207   n1 <- length(control_data)
208   n2 <- length(treatment_data)
209
210   # Effect sizes
211   abs_diff <- mean2 - mean1
212   rel_lift <- abs_diff / mean1
213
214   # Cohen's d
215   if (equal_var) {
216     pooled_sd <- sqrt(((n1-1)*sd1^2 + (n2-1)*sd2^2) / (n1+n2-2))
217     cohens_d <- abs_diff / pooled_sd

```

```

218 } else {
219   cohens_d <- abs_diff / sqrt((sd1^2 + sd2^2) / 2)
220 }
221
222 # Standard error
223 se <- sqrt(sd1^2/n1 + sd2^2/n2)
224
225 list(
226   control_mean = mean1,
227   treatment_mean = mean2,
228   absolute_difference = abs_diff,
229   relative_lift = rel_lift,
230   standard_error = se,
231   confidence_interval = test$conf.int,
232   p_value = test$p.value,
233   test_statistic = test$statistic,
234   statistically_significant = test$p.value < alpha,
235   sample_size_control = n1,
236   sample_size_treatment = n2,
237   cohens_d = cohens_d
238 )
239 }
240
241
242 #
243 # =====
244 # Bayesian Analysis
245 # =====
246
247 #' Bayesian A/B test for proportions
248 #'
249 #' @param x1 Conversions in control
250 #' @param n1 Total in control
251 #' @param x2 Conversions in treatment
252 #' @param n2 Total in treatment
253 #' @param prior_alpha Prior alpha parameter (default 1)
254 #' @param prior_beta Prior beta parameter (default 1)
255 #' @param n_sim Number of Monte Carlo simulations (default 100000)
256 #'
257 #' @return List with Bayesian results
258 #'
259 #' @examples
260 #' bayesian_ab_test(x1 = 100, n1 = 1000, x2 = 120, n2 = 1000)
261 #'
262 #' @export
263 bayesian_ab_test <- function(x1, n1, x2, n2,
264                             prior_alpha = 1, prior_beta = 1,
265                             n_sim = 100000) {
266
267   # Posterior parameters
268   post_a1 <- prior_alpha + x1
269   post_b1 <- prior_beta + n1 - x1
270   post_a2 <- prior_alpha + x2
271   post_b2 <- prior_beta + n2 - x2

```



```

326 observed <- table(data[[variant_col]])
327 n_total <- sum(observed)
328
329
330 if (is.null(expected_ratio)) {
331   # Assume equal allocation
332   n_variants <- length(observed)
333   expected <- rep(n_total / n_variants, n_variants)
334   names(expected) <- names(observed)
335 } else {
336   expected <- expected_ratio * n_total
337 }
338
339 # Chi-square test
340 test <- chisq.test(observed, p = expected / n_total)
341
342 list(
343   passed = test$p.value >= alpha,
344   p_value = test$p.value,
345   chi_squared = test$statistic,
346   observed_counts = observed,
347   expected_counts = expected,
348   severity = case_when(
349     test$p.value < 0.001 ~ "CRITICAL",
350     test$p.value < 0.01 ~ "HIGH",
351     TRUE ~ "OK"
352   )
353 )
354 }
355
356
357 #' Check for temporal stability
358 #'
359 #' @param data DataFrame with timestamp column
360 #' @param metric_col Metric column name
361 #' @param variant_col Variant column name
362 #' @param date_col Date/timestamp column name
363 #' @param n_periods Number of time periods to analyze
364 #'
365 #' @return List with temporal analysis
366 #'
367 #' @export
368 check_temporal_stability <- function(data, metric_col, variant_col,
369                                     date_col, n_periods = 7) {
370
371   data <- data %>%
372     mutate(period = cut(!!sym(date_col), breaks = n_periods, labels =
373       FALSE))
374
375   # Calculate metrics by period
376   temporal_stats <- data %>%
377     group_by(period, !!sym(variant_col)) %>%
378     summarise(
379       mean = mean(!!sym(metric_col), na.rm = TRUE),
380       sd = sd(!!sym(metric_col), na.rm = TRUE),
381       n = n(),
382       .groups = 'drop'
383     )

```



```

383
384 # Calculate treatment effects by period
385 effects <- temporal_stats %>%
386   select(period, !!sym(variant_col), mean) %>%
387   pivot_wider(names_from = !!sym(variant_col), values_from = mean)
388   %>%
389   mutate(effect = treatment - control)
390
391 # Test for trend
392 cor_test <- cor.test(effects$period, effects$effect, method = "
393   spearman")
394
395 list(
396   temporal_data = temporal_stats,
397   effects_by_period = effects,
398   trend_correlation = cor_test$estimate,
399   trend_p_value = cor_test$p.value,
400   stable = abs(cor_test$estimate) < 0.5 || cor_test$p.value > 0.05
401 )
402 }
403
404 #
405   =====
406
407 # Reporting
408 #
409   =====
410
411 #' Format results as text report
412 #'
413 #' @param result Results list from analyze_* function
414 #' @param metric_name Name of metric for display
415 #'
416 #' @return Character string with formatted report
417 #'
418 #' @export
419 format_results <- function(result, metric_name = "Metric") {
420
421   report <- sprintf("
422   =====
423
424   A/B TEST RESULTS: %s
425   =====
426
427   SAMPLE SIZES
428   -----
429   Control: %d
430   Treatment: %d
431   Total: %d
432
433   METRIC VALUES
434   -----
435   Control Mean: %.4f
436   Treatment Mean: %.4f

```

```

433 EFFECT_SIZES
434 -----
435 Absolute_Difference: %.4f
436 Relative_Lift: %+.2f%%
437 Cohen's_h/d: %.4f
438
439 STATISTICAL_INFERENCE
440 -----
441 Standard_Error: %.4f
442 95%_CI: [% .4f, % .4f]
443 Test_Statistic: %.4f
444 P-value: %.6f
445
446 DECISION
447 -----
448 Statistically_Significant: %s
449
450 =====
451 ",
452     metric_name,
453     result$sample_size_control,
454     result$sample_size_treatment,
455     result$sample_size_control + result$sample_size_treatment,
456     result$control_mean,
457     result$treatment_mean,
458     result$absolute_difference,
459     result$relative_lift * 100,
460     ifelse(!is.null(result$cohens_h), result$cohens_h, result$cohens_d)
461     ,
462     result$standard_error,
463     result$confidence_interval[1],
464     result$confidence_interval[2],
465     result$test_statistic,
466     result$p_value,
467     ifelse(result$statistically_significant, "YES", "NO")
468 )
469 cat(report)
470 invisible(report)
471 }
472
473 #
474 # =====
475 # Example Usage
476 # =====
477
478 # Uncomment to run examples:
479 #
480 # # Sample size calculation
481 # ss <- calculate_sample_size_proportions(p1 = 0.10, p2 = 0.12, power =
482 #     0.80)
483 # print(ss)
484 #

```

```
484 # # Analyze proportions
485 # result <- analyze_proportions(x1 = 100, n1 = 1000, x2 = 120, n2 =
      1000)
486 # format_results(result, "Conversion Rate")
487 #
488 # # Bayesian analysis
489 # bayes_result <- bayesian_ab_test(x1 = 100, n1 = 1000, x2 = 120, n2 =
      1000)
490 # print(bayes_result)
```

.3 SQL Queries for Data Extraction

.3.1 Experiment Data Extraction

Listing 13: SQL Queries for A/B Test Data

```

1  --
2  -- =====
3  -- A/B Test Data Extraction Queries
4  -- =====
5  -- Query 1: Extract experiment assignment data
6  -- Purpose: Get user-variant assignments with metadata
7  -- =====
8
9  WITH experiment_users AS (
10     SELECT
11         user_id,
12         experiment_id,
13         variant,
14         assignment_timestamp,
15         -- Extract user characteristics at assignment time
16         user_country,
17         user_platform,
18         user_device_type,
19         user_segment,
20         -- Check if user was previously exposed
21         CASE
22             WHEN assignment_timestamp > first_seen_timestamp
23             THEN 'returning'
24             ELSE 'new'
25         END AS user_type
26     FROM experiment_assignments
27     WHERE experiment_id = 'checkout_button_test'
28           AND assignment_timestamp >= '2024-01-01'
29           AND assignment_timestamp < '2024-02-01'
30           -- Exclude internal users
31           AND user_id NOT IN (SELECT user_id FROM internal_users)
32 )
33
34 SELECT
35     eu.*,
36     -- Add randomization check: hash should be uniformly distributed
37     MOD(ABS(FARM_FINGERPRINT(CAST(user_id AS STRING))), 100) AS hash_
38     check
39 FROM experiment_users eu
40 ORDER BY assignment_timestamp;
41
42 -- Query 2: Calculate conversion metrics
43 -- Purpose: Primary and secondary metrics for each user
44 -- =====

```

```

45
46 WITH user_assignments AS (
47     SELECT
48         user_id,
49         experiment_id,
50         variant,
51         assignment_timestamp
52 FROM experiment_assignments
53 WHERE experiment_id = 'checkout_button_test'
54     AND assignment_timestamp >= '2024-01-01'
55     AND assignment_timestamp < '2024-02-01'
56 ),
57
58 user_conversions AS (
59     SELECT
60         ua.user_id,
61         ua.variant,
62         -- Primary metric: Did user convert within 7 days?
63         MAX(
64             CASE
65                 WHEN o.order_timestamp BETWEEN ua.assignment_timestamp
66                     AND ua.assignment_timestamp +
67                     INTERVAL 7 DAY
68                 THEN 1
69                 ELSE 0
70             END
71         ) AS converted,
72         -- Secondary metric: Revenue within 7 days
73         COALESCE(
74             SUM(
75                 CASE
76                     WHEN o.order_timestamp BETWEEN ua.assignment_timestamp
77                     AND ua.assignment_timestamp +
78                     INTERVAL 7 DAY
79                     THEN o.order_value
80                     ELSE 0
81                 END
82             ),
83             0
84         ) AS revenue_7d,
85         -- Secondary metric: Number of orders
86         COUNT(
87             CASE
88                 WHEN o.order_timestamp BETWEEN ua.assignment_timestamp
89                     AND ua.assignment_timestamp +
90                     INTERVAL 7 DAY
91                 THEN o.order_id
92             END
93         ) AS num_orders_7d,
94         -- Secondary metric: Time to first order (in hours)
95         MIN(
96             CASE
97                 WHEN o.order_timestamp > ua.assignment_timestamp

```

```

98         THEN TIMESTAMP_DIFF(o.order_timestamp, ua.assignment_timestamp,
99                                HOUR)
100     ) AS hours_to_first_order
101
102 FROM user_assignments ua
103 LEFT JOIN orders o
104     ON ua.user_id = o.user_id
105 GROUP BY ua.user_id, ua.variant
106 )
107
108 SELECT
109     variant,
110     COUNT(DISTINCT user_id) AS total_users,
111     SUM(converted) AS total_conversions,
112     AVG(converted) AS conversion_rate,
113     AVG(revenue_7d) AS avg_revenue_per_user,
114     AVG(CASE WHEN converted = 1 THEN revenue_7d END) AS avg_revenue_per_
115         converter,
116     AVG(num_orders_7d) AS avg_orders_per_user,
117     APPROX_QUANTILES(hours_to_first_order, 2)[OFFSET(1)] AS median_hours_
118         to_order
119 FROM user_conversions
120 GROUP BY variant;
121
122 -- Query 3: Guardrail metrics
123 -- Purpose: Monitor key health metrics that shouldn't degrade
124
125 =====
126
127 WITH user_assignments AS (
128     SELECT
129         user_id,
130         variant,
131         assignment_timestamp
132     FROM experiment_assignments
133     WHERE experiment_id = 'checkout_button_test'
134         AND assignment_timestamp >= '2024-01-01'
135         AND assignment_timestamp < '2024-02-01'
136 ),
137
138 guardrail_metrics AS (
139     SELECT
140         ua.user_id,
141         ua.variant,
142
143         -- Guardrail 1: Page load time (shouldn't increase)
144         AVG(
145             CASE
146                 WHEN pl.page_name = 'checkout'
147                     AND pl.timestamp BETWEEN ua.assignment_timestamp
148                     AND ua.assignment_timestamp + INTERVAL 7
149                     DAY
150                 THEN pl.load_time_ms
151             END
152         ) AS avg_checkout_load_time_ms,

```

```

150
151 -- Guardrail 2: Error rate (shouldn't increase)
152 COUNTIF(
153     e.error_type IS NOT NULL
154     AND e.timestamp BETWEEN ua.assignment_timestamp
155                          AND ua.assignment_timestamp + INTERVAL 7 DAY
156 ) / GREATEST(
157     COUNT(
158         CASE
159             WHEN pl.timestamp BETWEEN ua.assignment_timestamp
160                          AND ua.assignment_timestamp + INTERVAL
161                          7 DAY
162         THEN 1
163         END
164     ),
165     1
166 ) AS error_rate,
167
168 -- Guardrail 3: Cart abandonment (shouldn't increase)
169 CASE
170     WHEN MAX(
171         CASE
172             WHEN ca.event_type = 'add_to_cart'
173                  AND ca.timestamp BETWEEN ua.assignment_timestamp
174                          AND ua.assignment_timestamp + INTERVAL
175                          7 DAY
176         THEN 1
177         ELSE 0
178         END
179     ) = 1
180     AND MAX(
181         CASE
182             WHEN o.order_id IS NOT NULL
183                  AND o.order_timestamp BETWEEN ua.assignment_timestamp
184                          AND ua.assignment_timestamp +
185                          INTERVAL 7 DAY
186         THEN 1
187         ELSE 0
188         END
189     ) = 0
190     END AS cart_abandoned,
191
192 -- Guardrail 4: Session duration (shouldn't decrease)
193 AVG(
194     CASE
195         WHEN s.start_time BETWEEN ua.assignment_timestamp
196                          AND ua.assignment_timestamp + INTERVAL 7
197                          DAY
198         THEN s.duration_seconds
199     END
200 ) AS avg_session_duration_sec
201
202 FROM user_assignments ua
203 LEFT JOIN page_loads pl ON ua.user_id = pl.user_id
204 LEFT JOIN errors e ON ua.user_id = e.user_id
205 LEFT JOIN cart_actions ca ON ua.user_id = ca.user_id

```

```

204 LEFT JOIN orders o ON ua.user_id = o.user_id
205 LEFT JOIN sessions s ON ua.user_id = s.user_id
206 GROUP BY ua.user_id, ua.variant
207 )
208
209 SELECT
210     variant,
211     COUNT(*) AS n_users,
212     AVG(avg_checkout_load_time_ms) AS mean_load_time_ms,
213     STDDEV(avg_checkout_load_time_ms) AS sd_load_time_ms,
214     AVG(error_rate) AS mean_error_rate,
215     AVG(cart_abandoned) AS cart_abandonment_rate,
216     AVG(avg_session_duration_sec) AS mean_session_duration_sec
217 FROM guardrail_metrics
218 GROUP BY variant;
219
220
221 -- Query 4: Segment analysis
222 -- Purpose: Test for heterogeneous treatment effects
223 --
224 =====
225
226 WITH user_metrics AS (
227     SELECT
228         ua.user_id,
229         ua.variant,
230         ua.user_country,
231         ua.user_platform,
232         ua.user_device_type,
233         ua.user_segment,
234         MAX(
235             CASE
236                 WHEN o.order_timestamp BETWEEN ua.assignment_timestamp
237                     AND ua.assignment_timestamp +
238                     INTERVAL 7 DAY
239                 THEN 1
240                 ELSE 0
241             END
242         ) AS converted
243 FROM experiment_assignments ua
244 LEFT JOIN orders o ON ua.user_id = o.user_id
245 WHERE ua.experiment_id = 'checkout_button_test'
246     AND ua.assignment_timestamp >= '2024-01-01'
247     AND ua.assignment_timestamp < '2024-02-01'
248 GROUP BY 1, 2, 3, 4, 5, 6
249 )
250
251 -- Conversion rate by country
252 SELECT
253     'country' AS segment_type,
254     user_country AS segment_value,
255     variant,
256     COUNT(*) AS n_users,
257     AVG(converted) AS conversion_rate,
258     STDDEV(converted) / SQRT(COUNT(*)) AS standard_error
259 FROM user_metrics
260 GROUP BY user_country, variant

```



```

259
260 UNION ALL
261
262 -- Conversion rate by platform
263 SELECT
264     'platform' AS segment_type,
265     user_platform AS segment_value,
266     variant,
267     COUNT(*) AS n_users,
268     AVG(converted) AS conversion_rate,
269     STDDEV(converted) / SQRT(COUNT(*)) AS standard_error
270 FROM user_metrics
271 GROUP BY user_platform, variant
272
273 UNION ALL
274
275 -- Conversion rate by device type
276 SELECT
277     'device_type' AS segment_type,
278     user_device_type AS segment_value,
279     variant,
280     COUNT(*) AS n_users,
281     AVG(converted) AS conversion_rate,
282     STDDEV(converted) / SQRT(COUNT(*)) AS standard_error
283 FROM user_metrics
284 GROUP BY user_device_type, variant
285
286 ORDER BY segment_type, segment_value, variant;
287
288
289 -- Query 5: Sample Ratio Mismatch (SRM) check
290 -- Purpose: Detect randomization issues
291 --
292 =====
293
294 WITH daily_assignments AS (
295     SELECT
296         DATE(assignment_timestamp) AS assignment_date,
297         variant,
298         COUNT(DISTINCT user_id) AS n_users
299     FROM experiment_assignments
300     WHERE experiment_id = 'checkout_button_test'
301           AND assignment_timestamp >= '2024-01-01'
302           AND assignment_timestamp < '2024-02-01'
303     GROUP BY assignment_date, variant
304 ),
305
306 daily_ratios AS (
307     SELECT
308         assignment_date,
309         MAX(CASE WHEN variant = 'control' THEN n_users END) AS control_
310         users,
311         MAX(CASE WHEN variant = 'treatment' THEN n_users END) AS treatment_
312         users,
313         MAX(CASE WHEN variant = 'control' THEN n_users END) /
314         (MAX(CASE WHEN variant = 'control' THEN n_users END) +

```

```

312         MAX(CASE WHEN variant = 'treatment' THEN n_users END)) AS
           control_proportion
313 FROM daily_assignments
314 GROUP BY assignment_date
315 )
316
317 SELECT
318     assignment_date,
319     control_users,
320     treatment_users,
321     control_users + treatment_users AS total_users,
322     control_proportion,
323     -- Flag days with significant deviation from 50/50 split
324     CASE
325         WHEN ABS(control_proportion - 0.5) > 0.05 THEN 'WARNING'
326         ELSE 'OK'
327     END AS srm_flag
328 FROM daily_ratios
329 ORDER BY assignment_date;
330
331
332 -- Query 6: Temporal stability check
333 -- Purpose: Detect novelty effects or temporal trends
334 --

```

```
=====
```

```

335
336 WITH weekly_metrics AS (
337     SELECT
338         ua.variant,
339         DATE_TRUNC(ua.assignment_timestamp, WEEK) AS week_start,
340         COUNT(DISTINCT ua.user_id) AS n_users,
341         AVG(
342             CASE
343                 WHEN o.order_timestamp BETWEEN ua.assignment_timestamp
344                     AND ua.assignment_timestamp +
345                     INTERVAL 7 DAY
346                 THEN 1
347                 ELSE 0
348             ) AS conversion_rate,
349         AVG(
350             CASE
351                 WHEN o.order_timestamp BETWEEN ua.assignment_timestamp
352                     AND ua.assignment_timestamp +
353                     INTERVAL 7 DAY
354                 THEN o.order_value
355                 ELSE 0
356             ) AS avg_revenue_per_user
357     FROM experiment_assignments ua
358     LEFT JOIN orders o ON ua.user_id = o.user_id
359     WHERE ua.experiment_id = 'checkout_button_test'
360         AND ua.assignment_timestamp >= '2024-01-01'
361         AND ua.assignment_timestamp < '2024-02-01'
362     GROUP BY variant, week_start
363 )
364

```

```

365 SELECT
366     week_start,
367     MAX(CASE WHEN variant = 'control' THEN conversion_rate END) AS
        control_cr,
368     MAX(CASE WHEN variant = 'treatment' THEN conversion_rate END) AS
        treatment_cr,
369     MAX(CASE WHEN variant = 'treatment' THEN conversion_rate END) -
370     MAX(CASE WHEN variant = 'control' THEN conversion_rate END) AS lift
        ,
371     (MAX(CASE WHEN variant = 'treatment' THEN conversion_rate END) -
372     MAX(CASE WHEN variant = 'control' THEN conversion_rate END)) /
373     MAX(CASE WHEN variant = 'control' THEN conversion_rate END) AS
        relative_lift
374 FROM weekly_metrics
375 GROUP BY week_start
376 ORDER BY week_start;

```

```

377
378
379 -- Query 7: User-level data export for analysis
380 -- Purpose: Create analysis-ready dataset
381 --

```

```

=====

```

```

382
383 SELECT
384     ua.user_id,
385     ua.variant,
386     ua.assignment_timestamp,
387     ua.user_country,
388     ua.user_platform,
389     ua.user_device_type,
390
391     -- Primary metric
392     MAX(
393         CASE
394             WHEN o.order_timestamp BETWEEN ua.assignment_timestamp
395                  AND ua.assignment_timestamp + INTERVAL
396                  7 DAY
397             THEN 1
398             ELSE 0
399         END
400     ) AS converted,
401
402     -- Revenue metrics
403     COALESCE(
404         SUM(
405             CASE
406                 WHEN o.order_timestamp BETWEEN ua.assignment_timestamp
407                      AND ua.assignment_timestamp +
408                      INTERVAL 7 DAY
409                 THEN o.order_value
410             END
411         ),
412         0
413     ) AS revenue_7d,
414
415     -- Engagement metrics
416     COUNT(

```

```
415     DISTINCT CASE
416         WHEN pl.timestamp BETWEEN ua.assignment_timestamp
417             AND ua.assignment_timestamp + INTERVAL 7
418                 DAY
419             THEN DATE(pl.timestamp)
420         END
421     ) AS days_active_7d,
422     -- Guardrail metrics
423     AVG(
424         CASE
425             WHEN pl.page_name = 'checkout'
426                 AND pl.timestamp BETWEEN ua.assignment_timestamp
427                     AND ua.assignment_timestamp + INTERVAL 7
428                         DAY
429                 THEN pl.load_time_ms
430             END
431         ) AS avg_checkout_load_time
432 FROM experiment_assignments ua
433 LEFT JOIN orders o
434     ON ua.user_id = o.user_id
435 LEFT JOIN page_loads pl
436     ON ua.user_id = pl.user_id
437
438 WHERE ua.experiment_id = 'checkout_button_test'
439     AND ua.assignment_timestamp >= '2024-01-01'
440     AND ua.assignment_timestamp < '2024-02-01'
441
442 GROUP BY
443     ua.user_id,
444     ua.variant,
445     ua.assignment_timestamp,
446     ua.user_country,
447     ua.user_platform,
448     ua.user_device_type;
```

.4 Configuration Files

.4.1 Experiment Configuration (YAML)

Listing 14: Experiment Configuration File

```

1 #
2 # =====
3 # A/B Test Experiment Configuration
4 # =====
5 experiment:
6   # Basic information
7   id: "checkout_button_color_2024_q1"
8   name: "Checkout_Button_Color_Test"
9   description: "Test_green_vs_blue_checkout_button_to_optimize_conversion"
10  owner: "product_team@company.com"
11  created_date: "2024-01-01"
12
13  # Experiment status
14  status: "running" # draft, running, paused, completed, archived
15
16  # Hypothesis and objectives
17  hypothesis: |
18    Changing the checkout button from blue to green will increase
19    conversion rate by reducing cognitive friction. Green signals "go"
20    and may encourage users to complete their purchase.
21
22  business_objective: "Increase_checkout_conversion_rate"
23  success_criteria: |
24    - Statistically significant lift in conversion rate (p < 0.05)
25    - Minimum 2% relative improvement
26    - No degradation in guardrail metrics
27
28  # Experimental design
29  design:
30    type: "ab_test" # ab_test, multivariate, factorial
31
32    variants:
33      - name: "control"
34        description: "Blue_checkout_button(current)"
35        traffic_allocation: 0.5
36        config:
37          button_color: "#0066CC"
38
39      - name: "treatment"
40        description: "Green_checkout_button"
41        traffic_allocation: 0.5
42        config:
43          button_color: "#00AA00"
44
45  # Randomization
46  randomization:
47    unit: "user_id" # user_id, session_id, device_id

```

```

48     algorithm: "deterministic_hash" # deterministic_hash, random
49     salt: "checkout_button_2024_q1"
50
51 # Targeting
52 targeting:
53     include:
54         - condition: "user_country_IN_('US',_, 'CA',_, 'UK')"
55         - condition: "user_age_>=18"
56         - condition: "NOT_is_internal_user"
57
58     exclude:
59         - condition: "user_id_IN_experiment_exclusion_list"
60
61 # Metrics
62 metrics:
63     primary:
64         - name: "conversion_rate"
65           description: "Proportion_of_users_who_complete_purchase"
66           type: "proportion"
67           numerator_event: "purchase_completed"
68           denominator: "users_in_experiment"
69           attribution_window_days: 7
70
71     secondary:
72         - name: "revenue_per_user"
73           description: "Average_revenue_per_user"
74           type: "continuous"
75           event: "purchase_completed"
76           aggregation: "mean"
77           attribution_window_days: 7
78
79         - name: "orders_per_user"
80           description: "Average_number_of_orders"
81           type: "count"
82           event: "purchase_completed"
83           aggregation: "mean"
84           attribution_window_days: 7
85
86     guardrail:
87         - name: "checkout_load_time"
88           description: "Page_load_time_for_checkout_page"
89           type: "continuous"
90           threshold: "+10%" # Maximum allowed degradation
91           severity: "high"
92
93         - name: "error_rate"
94           description: "Proportion_of_users_encountering_errors"
95           type: "proportion"
96           threshold: "+5%"
97           severity: "critical"
98
99         - name: "cart_abandonment_rate"
100          description: "Proportion_of_users_abandoning_cart"
101          type: "proportion"
102          threshold: "+3%"
103          severity: "high"
104
105 # Statistical parameters

```

```
106 statistics:
107     significance_level: 0.05
108     statistical_power: 0.80
109     minimum_detectable_effect: 0.02 # 2% relative lift
110     test_type: "two_sided" # two_sided, one_sided
111     multiple_testing_correction: "bonferroni"
112
113 # Sample size
114 sample_size:
115     method: "calculated" # calculated, fixed, sequential
116     per_variant: 10000
117     total: 20000
118
119 calculation:
120     baseline_conversion_rate: 0.10
121     expected_conversion_rate: 0.102
122     power: 0.80
123     alpha: 0.05
124
125 # Runtime configuration
126 runtime:
127     start_date: "2024-01-15"
128     end_date: "2024-02-01"
129     expected_duration_days: 14
130
131 # Ramp-up plan
132 ramp_up:
133     enabled: true
134     stages:
135         - date: "2024-01-15"
136           traffic_percentage: 10
137         - date: "2024-01-16"
138           traffic_percentage: 50
139         - date: "2024-01-17"
140           traffic_percentage: 100
141
142 # Monitoring
143 monitoring:
144     check_frequency_hours: 6
145     srm_threshold: 0.01
146     alerts:
147         - type: "sample_ratio_mismatch"
148           threshold: 0.01
149           recipients: ["data_science@company.com"]
150
151         - type: "guardrail_violation"
152           threshold: "any"
153           recipients: ["product_team@company.com", "engineering@company.com"]
154
155         - type: "error_rate_spike"
156           threshold: 0.05
157           recipients: ["engineering@company.com"]
158
159 # Analysis plan
160 analysis:
161     interim_analyses:
162         enabled: true
```

```

163     schedule:
164         - day: 3
165           purpose: "Early_safety_check"
166         - day: 7
167           purpose: "Midpoint_analysis"
168         - day: 14
169           purpose: "Final_analysis"
170
171     sequential_testing:
172         enabled: false
173         method: "obrien_fleming"
174         max_looks: 5
175
176     subgroup_analyses:
177         - dimension: "user_country"
178           values: ["US", "CA", "UK"]
179         - dimension: "device_type"
180           values: ["mobile", "desktop"]
181         - dimension: "user_segment"
182           values: ["new", "returning"]
183
184     # Decision criteria
185     decision:
186         auto_decision: false # Automatically make decision based on
187                               results
188
189     ship_if:
190         - "p_value < 0.05"
191         - "relative_lift > 0.02"
192         - "all_guardrails_pass"
193         - "no_significant_negative_segments"
194
195     iterate_if:
196         - "p_value >= 0.05"
197         - "relative_lift > 0.01"
198
199     kill_if:
200         - "relative_lift < -0.01"
201         - "any_critical_guardrail_fails"
202
203     # Metadata and tags
204     metadata:
205         tags:
206             - "checkout"
207             - "ui"
208             - "conversion_optimization"
209
210     related_experiments:
211         - "checkout_flow_redesign_2023_q4"
212         - "payment_options_test_2023_q3"
213
214     jira_ticket: "PROD-1234"
215     design_doc_url: "https://docs.company.com/exp/checkout_button"
216     dashboard_url: "https://dashboard.company.com/exp/checkout_button_
217                   2024_q1"

```



```

218 # Deployment Configuration
219 #
=====
220
221 deployment:
222   platform: "custom_experimentation_platform"
223
224   feature_flag:
225     enabled: true
226     flag_name: "checkout_button_color_experiment"
227
228   rollback:
229     enabled: true
230     trigger_conditions:
231       - "error_rate > 0.10"
232       - "guardrail_violation_critical"
233     automatic: true
234
235   data_pipeline:
236     events_table: "events.user_events"
237     assignments_table: "experiments.user_assignments"
238     refresh_frequency_minutes: 60

```

4.2 Analysis Pipeline Configuration (JSON)

Listing 15: Analysis Pipeline Configuration

```

1 {
2   "analysis_config": {
3     "experiment_id": "checkout_button_color_2024_q1",
4     "analysis_date": "2024-02-01",
5     "analyst": "data_science@company.com",
6
7     "data_sources": {
8       "assignments": {
9         "type": "bigquery",
10        "project": "company-analytics",
11        "dataset": "experiments",
12        "table": "user_assignments",
13        "filters": {
14          "experiment_id": "checkout_button_color_2024_q1",
15          "assignment_date_gte": "2024-01-15",
16          "assignment_date_lt": "2024-02-01"
17        }
18      },
19
20      "events": {
21        "type": "bigquery",
22        "project": "company-analytics",
23        "dataset": "events",
24        "table": "user_events",

```

```
25     "filters": {
26         "event_date_gte": "2024-01-15",
27         "event_date_lt": "2024-02-08"
28     }
29 }
30 },
31
32 "preprocessing": {
33     "steps": [
34         {
35             "name": "filter_internal_users",
36             "type": "exclusion",
37             "condition": "user_id NOT IN internal_users"
38         },
39         {
40             "name": "filter_bots",
41             "type": "exclusion",
42             "condition": "is_bot = FALSE"
43         },
44         {
45             "name": "check_assignment_consistency",
46             "type": "validation",
47             "condition": "user has single variant assignment"
48         }
49     ],
50
51     "quality_checks": [
52         {
53             "check": "sample_ratio_mismatch",
54             "threshold": 0.01,
55             "action": "warn"
56         },
57         {
58             "check": "missing_data_rate",
59             "threshold": 0.05,
60             "action": "fail"
61         }
62     ]
63 },
64
65 "statistical_tests": {
66     "primary_metric": {
67         "metric": "conversion_rate",
68         "test_type": "two_proportion_z_test",
69         "alpha": 0.05,
70         "alternative": "two_sided",
71         "confidence_level": 0.95
72     },
73
74     "secondary_metrics": [
75         {
```

```
76         "metric": "revenue_per_user",
77         "test_type": "welch_t_test",
78         "alpha": 0.05,
79         "alternative": "two_sided",
80         "confidence_level": 0.95
81     },
82     {
83         "metric": "orders_per_user",
84         "test_type": "mann_whitney_u",
85         "alpha": 0.05,
86         "alternative": "two_sided"
87     }
88 ],
89
90     "multiple_testing_correction": {
91         "method": "bonferroni",
92         "family_alpha": 0.05,
93         "n_tests": 3
94     }
95 },
96
97 "segmentation": {
98     "enabled": true,
99     "dimensions": [
100         {
101             "name": "user_country",
102             "type": "categorical",
103             "min_sample_per_segment": 100
104         },
105         {
106             "name": "device_type",
107             "type": "categorical",
108             "min_sample_per_segment": 100
109         },
110         {
111             "name": "user_segment",
112             "type": "categorical",
113             "values": ["new", "returning"]
114         }
115     ],
116
117     "interaction_tests": {
118         "enabled": true,
119         "method": "regression",
120         "alpha": 0.05
121     }
122 },
123
124 "visualization": {
125     "enabled": true,
126     "types": [
```

```
127     "conversion_funnel",
128     "time_series",
129     "distribution_comparison",
130     "forest_plot_segments"
131 ],
132 "output_format": "png",
133 "dpi": 300
134 },
135
136 "reporting": {
137     "output_formats": ["html", "pdf", "json"],
138     "template": "standard_ab_test_report",
139     "include_sections": [
140         "executive_summary",
141         "experiment_details",
142         "data_quality_checks",
143         "primary_analysis",
144         "secondary_analyses",
145         "segment_analysis",
146         "guardrail_metrics",
147         "recommendations"
148     ],
149
150     "distribution_list": [
151         "product_team@company.com",
152         "data_science@company.com",
153         "stakeholders@company.com"
154     ]
155 }
156 }
157 }
```

.5 API Implementation

.5.1 RESTful API for Experiment Management

Listing 16: Flask API for A/B Testing Platform

```

1  """
2  A/B Testing Platform API
3  =====
4
5  RESTful API for managing and analyzing A/B tests.
6
7  Endpoints:
8  - POST    /api/v1/experiments      Create new experiment
9  - GET     /api/v1/experiments      List all experiments
10 - GET     /api/v1/experiments/{id}  Get experiment details
11 - PUT     /api/v1/experiments/{id}  Update experiment
12 - DELETE  /api/v1/experiments/{id}  Delete experiment
13 - POST    /api/v1/experiments/{id}/assign  Assign user to variant
14 - GET     /api/v1/experiments/{id}/results  Get experiment results
15 - POST    /api/v1/experiments/{id}/analyze  Trigger analysis
16 """
17
18 from flask import Flask, request, jsonify
19 from flask_cors import CORS
20 from dataclasses import dataclass, asdict
21 from typing import Optional, List, Dict
22 from datetime import datetime
23 import hashlib
24 import json
25
26 app = Flask(__name__)
27 CORS(app)
28
29 #
30 # Data Models
31 #
32
33 @dataclass
34 class Experiment:
35     """Experiment model."""
36     id: str
37     name: str
38     description: str
39     status: str # draft, running, paused, completed
40     variants: List[str]
41     traffic_allocation: Dict[str, float]
42     start_date: Optional[str] = None
43     end_date: Optional[str] = None
44     created_at: str = None

```

```

45     updated_at: str = None
46
47     def __post_init__(self):
48         if self.created_at is None:
49             self.created_at = datetime.now().isoformat()
50         if self.updated_at is None:
51             self.updated_at = datetime.now().isoformat()
52
53
54 # In-memory storage (replace with database in production)
55 experiments_db = {}
56 assignments_db = {}
57
58
59 #
60 ↪ =====
61 # Experiment Management Endpoints
62 ↪ =====
63
64 @app.route('/api/v1/experiments', methods=['POST'])
65 def create_experiment():
66     """
67     Create a new experiment.
68
69     Request body:
70     {
71         "name": "Button Color Test",
72         "description": "Test green vs blue button",
73         "variants": ["control", "treatment"],
74         "traffic_allocation": {"control": 0.5, "treatment": 0.5}
75     }
76     """
77     try:
78         data = request.get_json()
79
80         # Validate request
81         required_fields = ['name', 'description', 'variants',
82 ↪ 'traffic_allocation']
83         for field in required_fields:
84             if field not in data:
85                 return jsonify({'error': f'Missing required
86 ↪ field: {field}'}), 400
87
88         # Validate traffic allocation
89         if not abs(sum(data['traffic_allocation'].values()) - 1
90 ↪ .0) < 0.001:
91             return jsonify({'error': 'Traffic allocation must sum
92 ↪ to 1.0'}), 400
93
94         # Generate experiment ID

```

```

90     exp_id = hashlib.md5(
91         f"{data['name']}_{datetime.now().isoformat()}"
92     ).hexdigest()[0:16]
93
94     # Create experiment
95     experiment = Experiment(
96         id=exp_id,
97         name=data['name'],
98         description=data['description'],
99         status='draft',
100         variants=data['variants'],
101         traffic_allocation=data['traffic_allocation'],
102         start_date=data.get('start_date'),
103         end_date=data.get('end_date')
104     )
105
106     # Save to database
107     experiments_db[exp_id] = experiment
108
109     return jsonify({
110         'success': True,
111         'experiment': asdict(experiment)
112     }), 201
113
114     except Exception as e:
115         return jsonify({'error': str(e)}), 500
116
117
118 @app.route('/api/v1/experiments', methods=['GET'])
119 def list_experiments():
120     """
121     List all experiments.
122
123     Query parameters:
124     - status: Filter by status (optional)
125     - limit: Maximum number of results (default 100)
126     - offset: Pagination offset (default 0)
127     """
128     try:
129         status_filter = request.args.get('status')
130         limit = int(request.args.get('limit', 100))
131         offset = int(request.args.get('offset', 0))
132
133         experiments = list(experiments_db.values())
134
135         # Apply filters
136         if status_filter:
137             experiments = [e for e in experiments if e.status ==
138                             ↪ status_filter]
139
140         # Sort by created_at desc

```

```

140     experiments.sort(key=lambda x: x.created_at, reverse=True)
141
142     # Pagination
143     paginated = experiments[offset:offset+limit]
144
145     return jsonify({
146         'success': True,
147         'experiments': [asdict(e) for e in paginated],
148         'total': len(experiments),
149         'limit': limit,
150         'offset': offset
151     }), 200
152
153     except Exception as e:
154         return jsonify({'error': str(e)}), 500
155
156
157 @app.route('/api/v1/experiments/<experiment_id>', methods=['GET'])
158 def get_experiment(experiment_id):
159     """Get experiment details by ID."""
160     try:
161         if experiment_id not in experiments_db:
162             return jsonify({'error': 'Experiment not found'}), 404
163
164         experiment = experiments_db[experiment_id]
165
166         return jsonify({
167             'success': True,
168             'experiment': asdict(experiment)
169         }), 200
170
171     except Exception as e:
172         return jsonify({'error': str(e)}), 500
173
174
175 @app.route('/api/v1/experiments/<experiment_id>', methods=['PUT'])
176 def update_experiment(experiment_id):
177     """
178     Update experiment.
179
180     Request body: Partial experiment data to update
181     """
182     try:
183         if experiment_id not in experiments_db:
184             return jsonify({'error': 'Experiment not found'}), 404
185
186         data = request.get_json()
187         experiment = experiments_db[experiment_id]
188
189         # Update allowed fields

```



```

190     updatable_fields = ['name', 'description', 'status',
191         ↪ 'end_date']
192     for field in updatable_fields:
193         if field in data:
194             setattr(experiment, field, data[field])
195
196     experiment.updated_at = datetime.now().isoformat()
197
198     return jsonify({
199         'success': True,
200         'experiment': asdict(experiment)
201     }), 200
202
203 except Exception as e:
204     return jsonify({'error': str(e)}), 500
205
206 @app.route('/api/v1/experiments/<experiment_id>',
207     ↪ methods=['DELETE'])
208 def delete_experiment(experiment_id):
209     """Delete experiment (soft delete by marking as archived)."""
210     try:
211         if experiment_id not in experiments_db:
212             return jsonify({'error': 'Experiment not found'}), 404
213
214         experiment = experiments_db[experiment_id]
215         experiment.status = 'archived'
216         experiment.updated_at = datetime.now().isoformat()
217
218         return jsonify({
219             'success': True,
220             'message': 'Experiment archived'
221         }), 200
222
223     except Exception as e:
224         return jsonify({'error': str(e)}), 500
225
226 #
227     ↪ =====
228 # Variant Assignment Endpoint
229 #
230     ↪ =====
231
232 @app.route('/api/v1/experiments/<experiment_id>/assign',
233     ↪ methods=['POST'])
234 def assign_variant(experiment_id):
235     """
236     Assign user to variant.
237
238     Request body:

```

```

236     {
237         "user_id": "user_12345",
238         "context": {
239             "country": "US",
240             "platform": "web"
241         }
242     }
243
244     Response:
245     {
246         "variant": "treatment",
247         "experiment_id": "exp_123",
248         "assignment_timestamp": "2024-01-15T10:30:00"
249     }
250     """
251     try:
252         if experiment_id not in experiments_db:
253             return jsonify({'error': 'Experiment not found'}), 404
254
255         experiment = experiments_db[experiment_id]
256
257         if experiment.status != 'running':
258             return jsonify({'error': 'Experiment is not
259                             ↪ running'}), 400
260
261         data = request.get_json()
262
263         if 'user_id' not in data:
264             return jsonify({'error': 'Missing user_id'}), 400
265
266         user_id = data['user_id']
267
268         # Check if user already assigned
269         assignment_key = f"{experiment_id}:{user_id}"
270         if assignment_key in assignments_db:
271             assignment = assignments_db[assignment_key]
272             return jsonify({
273                 'success': True,
274                 'variant': assignment['variant'],
275                 'experiment_id': experiment_id,
276                 'assignment_timestamp': assignment['timestamp'],
277                 'is_new_assignment': False
278             }), 200
279
280         # Assign variant using deterministic hash
281         variant = _assign_variant_deterministic(
282             user_id,
283             experiment.variants,
284             experiment.traffic_allocation,
285             experiment.id

```

```

286
287     # Store assignment
288     assignment = {
289         'user_id': user_id,
290         'experiment_id': experiment_id,
291         'variant': variant,
292         'timestamp': datetime.now().isoformat(),
293         'context': data.get('context', {})
294     }
295     assignments_db[assignment_key] = assignment
296
297     return jsonify({
298         'success': True,
299         'variant': variant,
300         'experiment_id': experiment_id,
301         'assignment_timestamp': assignment['timestamp'],
302         'is_new_assignment': True
303     }), 200
304
305 except Exception as e:
306     return jsonify({'error': str(e)}), 500
307
308
309 def _assign_variant_deterministic(user_id: str, variants:
    ↪ List[str],
310                                   allocation: Dict[str, float],
311                                   salt: str) -> str:
312     """
313     Deterministically assign variant using hash-based
    ↪ randomization.
314
315     Ensures:
316     - Same user always gets same variant
317     - Traffic split matches desired allocation
318     - Cryptographically secure
319     """
320     # Create hash of user_id + salt
321     hash_input = f"{user_id}:{salt}"
322     hash_value =
    ↪ int(hashlib.md5(hash_input.encode()).hexdigest(), 16)
323
324     # Normalize to [0, 1)
325     normalized = (hash_value % 10000) / 10000.0
326
327     # Assign based on allocation
328     cumulative = 0.0
329     for variant in variants:
330         cumulative += allocation[variant]
331         if normalized < cumulative:
332             return variant
333

```

```

334     # Fallback (should not happen if allocation sums to 1.0)
335     return variants[0]
336
337
338 #
339     ↪ =====
340 # Analysis Endpoints
341 #
342     ↪ =====
343
344 @app.route('/api/v1/experiments/<experiment_id>/results',
345     ↪ methods=['GET'])
346 def get_results(experiment_id):
347     """
348     Get experiment results.
349
350     Query parameters:
351     - metric: Specific metric to return (optional)
352     - segment: Segment dimension for subgroup analysis (optional)
353     """
354     try:
355         if experiment_id not in experiments_db:
356             return jsonify({'error': 'Experiment not found'}), 404
357
358         experiment = experiments_db[experiment_id]
359         metric_param = request.args.get('metric')
360         segment_param = request.args.get('segment')
361
362         # Get assignments for this experiment
363         exp_assignments = {
364             k: v for k, v in assignments_db.items()
365             if v['experiment_id'] == experiment_id
366         }
367
368         # Calculate basic statistics
369         variant_counts = {}
370         for assignment in exp_assignments.values():
371             variant = assignment['variant']
372             variant_counts[variant] = variant_counts.get(variant,
373                 ↪ 0) + 1
374
375         results = {
376             'experiment_id': experiment_id,
377             'status': experiment.status,
378             'total_users_assigned': len(exp_assignments),
379             'variant_counts': variant_counts,
380             'metrics': {
381                 # In production, calculate actual metrics from
382                 ↪ event data
383                 'sample_sizes': variant_counts
384             }
385         }

```

```

380     }
381
382     return jsonify({
383         'success': True,
384         'results': results
385     }), 200
386
387 except Exception as e:
388     return jsonify({'error': str(e)}), 500
389
390
391 @app.route('/api/v1/experiments/<experiment_id>/analyze',
392            ↪ methods=['POST'])
393 def trigger_analysis(experiment_id):
394     """
395     Trigger analysis job for experiment.
396
397     Request body:
398     {
399         "analysis_type": "full", // full, interim, segment
400         "metrics": ["conversion_rate", "revenue"],
401         "segments": ["country", "device_type"]
402     }
403
404     Response:
405     {
406         "job_id": "analysis_job_12345",
407         "status": "queued",
408         "estimated_time_seconds": 60
409     }
410     """
411     try:
412         if experiment_id not in experiments_db:
413             return jsonify({'error': 'Experiment not found'}), 404
414
415         data = request.get_json()
416         analysis_type = data.get('analysis_type', 'full')
417
418         # Generate job ID
419         job_id = hashlib.md5(
420             f"{experiment_id}_{datetime.now().isoformat()}"
421             .encode()
422             .hexdigest()[:16]
423         )
424
425         # In production: Queue analysis job to background worker
426         # For now, return mock response
427
428         return jsonify({
429             'success': True,
430             'job_id': job_id,
431             'status': 'queued',
432             'experiment_id': experiment_id,

```

```

430         'analysis_type': analysis_type,
431         'estimated_time_seconds': 60
432     }), 202
433
434     except Exception as e:
435         return jsonify({'error': str(e)}), 500
436
437
438 #
439     ↪ =====
440 # Health Check
441 #
442     ↪ =====
443
444 @app.route('/health', methods=['GET'])
445 def health_check():
446     """Health check endpoint."""
447     return jsonify({
448         'status': 'healthy',
449         'timestamp': datetime.now().isoformat(),
450         'version': '1.0.0'
451     }), 200
452
453 #
454     ↪ =====
455 # Error Handlers
456 #
457     ↪ =====
458
459 @app.errorhandler(404)
460 def not_found(error):
461     return jsonify({'error': 'Not found'}), 404
462
463 @app.errorhandler(500)
464 def internal_error(error):
465     return jsonify({'error': 'Internal server error'}), 500
466
467 #
468     ↪ =====
469 # Run Application
470 #
471     ↪ =====
472
473 if __name__ == '__main__':
474     app.run(debug=True, host='0.0.0.0', port=5000)

```

.5.2 API Client Library (Python)

Listing 17: Python Client for A/B Testing API

```

1  """
2  A/B Testing API Client
3  =====
4
5  Python client library for interacting with A/B testing platform
   ↪ API.
6
7  Usage:
8      from abtest_client import ABTestClient
9
10     client = ABTestClient(base_url="https://api.company.com",
   ↪ api_key="...")
11
12     # Create experiment
13     experiment = client.create_experiment(
14         name="Button Color Test",
15         variants=["control", "treatment"],
16         allocation={"control": 0.5, "treatment": 0.5}
17     )
18
19     # Assign user to variant
20     variant = client.assign_variant(experiment_id,
   ↪ user_id="user_123")
21
22     # Get results
23     results = client.get_results(experiment_id)
24 """
25
26 import requests
27 from typing import Dict, List, Optional
28 from dataclasses import dataclass
29 import json
30
31
32 class ABTestAPIError(Exception):
33     """Exception raised for API errors."""
34     pass
35
36
37 @dataclass
38 class ExperimentConfig:
39     """Configuration for creating an experiment."""
40     name: str
41     description: str
42     variants: List[str]
43     traffic_allocation: Dict[str, float]
44     start_date: Optional[str] = None
45     end_date: Optional[str] = None

```

```

46
47
48 class ABTestClient:
49     """Client for A/B Testing Platform API."""
50
51     def __init__(self, base_url: str, api_key: str, timeout: int
52         ↪ = 30):
53         """
54         Initialize API client.
55
56         Parameters
57         -----
58         base_url : str
59             Base URL for API (e.g., "https://api.company.com")
60         api_key : str
61             API authentication key
62         timeout : int
63             Request timeout in seconds
64
65         self.base_url = base_url.rstrip('/')
66         self.api_key = api_key
67         self.timeout = timeout
68         self.session = requests.Session()
69         self.session.headers.update({
70             'Authorization': f'Bearer {api_key}',
71             'Content-Type': 'application/json'
72         })
73
74     def _request(self, method: str, endpoint: str,
75         params: Optional[Dict] = None,
76         data: Optional[Dict] = None) -> Dict:
77         """
78         Make API request.
79
80         Parameters
81         -----
82         method : str
83             HTTP method (GET, POST, PUT, DELETE)
84         endpoint : str
85             API endpoint path
86         params : dict, optional
87             Query parameters
88         data : dict, optional
89             Request body data
90
91         Returns
92         -----
93         dict
94             Response data
95
96         Raises

```



```

96         -----
97         ABTestAPIError
98             If request fails
99         """
100         url = f"{self.base_url}{endpoint}"
101
102         try:
103             response = self.session.request(
104                 method=method,
105                 url=url,
106                 params=params,
107                 json=data,
108                 timeout=self.timeout
109             )
110
111             response.raise_for_status()
112             return response.json()
113
114         except requests.exceptions.HTTPError as e:
115             error_msg = f"API request failed: {e}"
116             if e.response is not None:
117                 try:
118                     error_data = e.response.json()
119                     error_msg = f"API error:
120                                     ↪ {error_data.get('error', str(e))}"
121                 except:
122                     pass
123             raise ABTestAPIError(error_msg) from e
124
125         except requests.exceptions.RequestException as e:
126             raise ABTestAPIError(f"Request failed: {e}") from e
127
128         #
129         ↪ =====
130
131         # Experiment Management
132         #
133         ↪ =====
134
135         def create_experiment(self,
136                               name: str,
137                               description: str,
138                               variants: List[str],
139                               traffic_allocation: Dict[str, float],
140                               start_date: Optional[str] = None,
141                               end_date: Optional[str] = None) -> Dict:
142
143             """
144             Create a new experiment.
145
146             Parameters
147             -----
148             name : str

```

```

144         Experiment name
145     description : str
146         Experiment description
147     variants : list of str
148         List of variant names
149     traffic_allocation : dict
150         Traffic allocation per variant (must sum to 1.0)
151     start_date : str, optional
152         Start date (ISO format)
153     end_date : str, optional
154         End date (ISO format)
155
156     Returns
157     -----
158     dict
159         Created experiment data
160
161     Examples
162     -----
163     >>> client = ABTestClient(base_url="...", api_key="...")
164     >>> exp = client.create_experiment(
165     ...     name="Button Test",
166     ...     description="Test button colors",
167     ...     variants=["control", "treatment"],
168     ...     traffic_allocation={"control": 0.5, "treatment":
169         ↪ 0.5}
170     ... )
171     >>> print(exp['experiment']['id'])
172     """
173     data = {
174         'name': name,
175         'description': description,
176         'variants': variants,
177         'traffic_allocation': traffic_allocation
178     }
179
180     if start_date:
181         data['start_date'] = start_date
182     if end_date:
183         data['end_date'] = end_date
184
185     response = self._request('POST', '/api/v1/experiments',
186         ↪ data=data)
187     return response['experiment']
188
189     def list_experiments(self,
190         status: Optional[str] = None,
191         limit: int = 100,
192         offset: int = 0) -> List[Dict]:
193         """
194         List experiments.

```

```

193
194     Parameters
195     -----
196     status : str, optional
197         Filter by status
198     limit : int
199         Maximum number of results
200     offset : int
201         Pagination offset
202
203     Returns
204     -----
205     list of dict
206         List of experiments
207     """
208     params = {'limit': limit, 'offset': offset}
209     if status:
210         params['status'] = status
211
212     response = self._request('GET', '/api/v1/experiments',
213                             ↪ params=params)
214     return response['experiments']
215
216 def get_experiment(self, experiment_id: str) -> Dict:
217     """
218     Get experiment details.
219
220     Parameters
221     -----
222     experiment_id : str
223         Experiment ID
224
225     Returns
226     -----
227     dict
228         Experiment data
229     """
230     response = self._request('GET',
231                             ↪ f'/api/v1/experiments/{experiment_id}')
232     return response['experiment']
233
234 def update_experiment(self, experiment_id: str, **kwargs) ->
235     ↪ Dict:
236     """
237     Update experiment.
238
239     Parameters
240     -----
241     experiment_id : str
242         Experiment ID
243     **kwargs

```

```

241         Fields to update (name, description, status, end_date)
242
243     Returns
244     -----
245     dict
246         Updated experiment data
247     """
248     response = self._request(
249         'PUT',
250         f'/api/v1/experiments/{experiment_id}',
251         data=kwargs
252     )
253     return response['experiment']
254
255 def delete_experiment(self, experiment_id: str) -> bool:
256     """
257     Delete (archive) experiment.
258
259     Parameters
260     -----
261     experiment_id : str
262         Experiment ID
263
264     Returns
265     -----
266     bool
267         True if successful
268     """
269     self._request('DELETE',
270                  ↪ f'/api/v1/experiments/{experiment_id}')
271     return True
272
273 # ↪ =====
274 # Variant Assignment
275 # ↪ =====
276
277 def assign_variant(self,
278                   experiment_id: str,
279                   user_id: str,
280                   context: Optional[Dict] = None) -> Dict:
281     """
282     Assign user to variant.
283
284     Parameters
285     -----
286     experiment_id : str
287         Experiment ID
288     user_id : str
289         User identifier

```

```

289         context : dict, optional
290             Additional context (country, platform, etc.)
291
292     Returns
293     -----
294     dict
295         Assignment data including variant
296
297     Examples
298     -----
299     >>> assignment = client.assign_variant(
300     ...     experiment_id="exp_123",
301     ...     user_id="user_456",
302     ...     context={"country": "US", "platform": "web"}
303     ... )
304     >>> print(assignment['variant'])
305     'treatment'
306     """
307     data = {'user_id': user_id}
308     if context:
309         data['context'] = context
310
311     response = self._request(
312         'POST',
313         f'/api/v1/experiments/{experiment_id}/assign',
314         data=data
315     )
316     return response
317
318     #
319     # Analysis
320     #
321     #
322     def get_results(self,
323                     experiment_id: str,
324                     metric: Optional[str] = None,
325                     segment: Optional[str] = None) -> Dict:
326         """
327         Get experiment results.
328
329         Parameters
330         -----
331         experiment_id : str
332             Experiment ID
333         metric : str, optional
334             Specific metric name
335         segment : str, optional
336             Segment dimension
337

```

```

338     Returns
339     -----
340     dict
341         Results data
342     """
343     params = {}
344     if metric:
345         params['metric'] = metric
346     if segment:
347         params['segment'] = segment
348
349     response = self._request(
350         'GET',
351         f'/api/v1/experiments/{experiment_id}/results',
352         params=params
353     )
354     return response['results']
355
356     def trigger_analysis(self,
357                         experiment_id: str,
358                         analysis_type: str = 'full',
359                         metrics: Optional[List[str]] = None,
360                         segments: Optional[List[str]] = None) ->
361                         ↪ Dict:
362
363     """
364     Trigger analysis job.
365
366     Parameters
367     -----
368     experiment_id : str
369         Experiment ID
370     analysis_type : str
371         Type of analysis: 'full', 'interim', 'segment'
372     metrics : list of str, optional
373         Metrics to analyze
374     segments : list of str, optional
375         Segment dimensions
376
377     Returns
378     -----
379     dict
380         Job information
381     """
382     data = {'analysis_type': analysis_type}
383     if metrics:
384         data['metrics'] = metrics
385     if segments:
386         data['segments'] = segments
387
388     response = self._request(
389         'POST',

```

```

388         f'/api/v1/experiments/{experiment_id}/analyze',
389         data=data
390     )
391     return response
392
393
394 #
395 # Example Usage
396 #
397
398 if __name__ == "__main__":
399     # Initialize client
400     client = ABTestClient(
401         base_url="http://localhost:5000",
402         api_key="test_api_key_12345"
403     )
404
405     # Create experiment
406     experiment = client.create_experiment(
407         name="Button Color Test",
408         description="Test green vs blue checkout button",
409         variants=["control", "treatment"],
410         traffic_allocation={"control": 0.5, "treatment": 0.5}
411     )
412
413     print(f"Created experiment: {experiment['id']}")
414
415     # Assign users to variants
416     for user_id in ["user_1", "user_2", "user_3"]:
417         assignment = client.assign_variant(
418             experiment_id=experiment['id'],
419             user_id=user_id,
420             context={"platform": "web"}
421         )
422         print(f"{user_id} -> {assignment['variant']}")
423
424     # Get results
425     results = client.get_results(experiment['id'])
426     print(f"\nResults: {json.dumps(results, indent=2)}")

```

.6 Summary

This appendix provides production-ready code implementations across multiple languages and platforms:

- **Python:** Complete A/B testing framework with sample size calculation, statistical analysis, data quality checks, and reporting

- **R:** Statistical analysis functions for both frequentist and Bayesian approaches
- **SQL:** Comprehensive queries for data extraction, metric calculation, quality checks, and segment analysis
- **Configuration:** YAML and JSON templates for experiment and analysis configuration
- **API:** RESTful API implementation with Flask and corresponding Python client library

All code includes extensive documentation, error handling, and practical examples ready for adaptation to production environments.

Statistical Tables

Statistical Tables and Reference Values

This appendix provides comprehensive statistical tables for A/B testing, including critical values, sample sizes, power calculations, and correction factors. All tables assume two-sided tests unless otherwise noted.

.1 Critical Values for Common Tests

.1.1 Standard Normal Distribution (Z)

Critical values for standard normal distribution used in large-sample tests.

Table 1: Standard Normal Critical Values ($z_{\alpha/2}$)

Confidence Level	α (two-sided)	$\alpha/2$	$z_{\alpha/2}$
80%	0.20	0.10	1.282
85%	0.15	0.075	1.440
90%	0.10	0.05	1.645
95%	0.05	0.025	1.960
96%	0.04	0.02	2.054
98%	0.02	0.01	2.326
99%	0.01	0.005	2.576
99.5%	0.005	0.0025	2.807
99.9%	0.001	0.0005	3.291
99.99%	0.0001	0.00005	3.891

Note: For one-sided tests, use z_{α} instead of $z_{\alpha/2}$ (e.g., for one-sided 95% test, use $z_{0.05} = 1.645$).

.1.2 Student's t-Distribution

Critical values for t-tests with different degrees of freedom.

Note: For A/B tests with equal sample sizes n per group, degrees of freedom = $2(n - 1)$.

.1.3 Chi-Squared Distribution

Critical values for chi-squared tests (e.g., goodness-of-fit, independence).

Usage: For a 2×2 contingency table (A/B test with binary outcome), $df = 1$. Use $\chi^2_{0.05,1} = 3.841$.

Table 2: t-Distribution Critical Values for Two-Sided Tests

df	Significance Level (α)			
	0.10	0.05	0.02	0.01
1	6.314	12.706	31.821	63.657
2	2.920	4.303	6.965	9.925
3	2.353	3.182	4.541	5.841
4	2.132	2.776	3.747	4.604
5	2.015	2.571	3.365	4.032
6	1.943	2.447	3.143	3.707
7	1.895	2.365	2.998	3.499
8	1.860	2.306	2.896	3.355
9	1.833	2.262	2.821	3.250
10	1.812	2.228	2.764	3.169
12	1.782	2.179	2.681	3.055
15	1.753	2.131	2.602	2.947
20	1.725	2.086	2.528	2.845
25	1.708	2.060	2.485	2.787
30	1.697	2.042	2.457	2.750
40	1.684	2.021	2.423	2.704
50	1.676	2.009	2.403	2.678
60	1.671	2.000	2.390	2.660
80	1.664	1.990	2.374	2.639
100	1.660	1.984	2.364	2.626
120	1.658	1.980	2.358	2.617
∞	1.645	1.960	2.326	2.576

.1.4 F-Distribution

Critical values for F-tests at $\alpha = 0.05$ (used in ANOVA, variance ratio tests).

Note: For comparing variances of two groups, $df_1 = n_1 - 1$ and $df_2 = n_2 - 1$.

Table 3: Chi-Squared Distribution Critical Values

df	Significance Level (α)				
	0.10	0.05	0.025	0.01	0.001
1	2.706	3.841	5.024	6.635	10.828
2	4.605	5.991	7.378	9.210	13.816
3	6.251	7.815	9.348	11.345	16.266
4	7.779	9.488	11.143	13.277	18.467
5	9.236	11.070	12.833	15.086	20.515
6	10.645	12.592	14.449	16.812	22.458
7	12.017	14.067	16.013	18.475	24.322
8	13.362	15.507	17.535	20.090	26.125
9	14.684	16.919	19.023	21.666	27.877
10	15.987	18.307	20.483	23.209	29.588
12	18.549	21.026	23.337	26.217	32.909
15	22.307	24.996	27.488	30.578	37.697
20	28.412	31.410	34.170	37.566	45.315
25	34.382	37.652	40.646	44.314	52.620
30	40.256	43.773	46.979	50.892	59.703
40	51.805	55.758	59.342	63.691	73.402
50	63.167	67.505	71.420	76.154	86.661

.2 Sample Size Tables

.2.1 Sample Size for Two-Proportion Test

Required sample size **per variant** for comparing two proportions with $\alpha = 0.05$ (two-sided) and power = 0.80.

Note: Total sample size = $2 \times n$ (both variants combined). Values rounded to nearest integer.

Example: To detect a 10% relative lift from baseline 10% (i.e., $10.0\% \rightarrow 11.0\%$) with 80% power and $\alpha = 0.05$, need 17,761 users per variant (35,522 total).

.2.2 Sample Size for Different Power Levels

Sample size per variant for baseline $p_1 = 0.10$, $\alpha = 0.05$, varying power.

Note: Higher power requires larger sample sizes. Standard power is 80%, but 90% is recommended for important decisions.

.2.3 Sample Size for Continuous Metrics

Sample size per variant for t-test with $\alpha = 0.05$, power = 0.80.

Cohen's d: $d = \frac{\mu_2 - \mu_1}{\sigma}$ where σ is the common standard deviation.

Example: If average session duration is 120 seconds (SD = 30 seconds), a 15-second improvement corresponds to $d = 15/30 = 0.5$, requiring 64 users per variant.

Table 4: F-Distribution Critical Values ($\alpha = 0.05$)

df_2	Numerator df (df_1)						
	1	2	3	4	5	10	20
5	6.61	5.79	5.41	5.19	5.05	4.74	4.56
10	4.96	4.10	3.71	3.48	3.33	2.98	2.77
15	4.54	3.68	3.29	3.06	2.90	2.54	2.33
20	4.35	3.49	3.10	2.87	2.71	2.35	2.12
25	4.24	3.39	2.99	2.76	2.60	2.24	2.01
30	4.17	3.32	2.92	2.69	2.53	2.16	1.93
40	4.08	3.23	2.84	2.61	2.45	2.08	1.84
50	4.03	3.18	2.79	2.56	2.40	2.03	1.78
60	4.00	3.15	2.76	2.53	2.37	1.99	1.75
100	3.94	3.09	2.70	2.46	2.31	1.93	1.68
∞	3.84	3.00	2.60	2.37	2.21	1.83	1.57

Table 5: Sample Size per Variant (Two-Proportion Z-Test, $\alpha = 0.05$, Power = 0.80)

Relative Lift	Baseline Proportion (p_1)						
	0.01	0.05	0.10	0.15	0.20	0.25	0.50
5%	790,063	149,672	72,160	47,272	35,056	27,671	10,852
10%	195,186	36,901	17,761	11,625	8,618	6,800	2,663
15%	86,042	16,251	7,818	5,116	3,793	2,992	1,172
20%	48,095	9,084	4,371	2,860	2,120	1,673	655
25%	30,541	5,768	2,776	1,816	1,346	1,062	416
30%	20,944	3,955	1,904	1,246	923	728	285
40%	11,580	2,187	1,053	689	510	403	158
50%	7,363	1,391	670	438	325	256	100
75%	3,194	604	291	190	141	111	43
100%	1,765	334	161	105	78	61	24

.3 Power Tables

.3.1 Power for Two-Proportion Tests

Statistical power for different sample sizes and effect sizes ($\alpha = 0.05$, baseline $p_1 = 0.10$).

Interpretation: Power is the probability of detecting an effect when it truly exists. 0.80 (80%) is the standard threshold.

.3.2 Power vs. Effect Size

Power for fixed sample size ($n = 5,000$ per variant) with varying baseline and effect.

Key insight: For the same relative lift, power decreases as baseline rate increases (variance increases).

Table 6: Sample Size vs. Power (Baseline 10%, $\alpha = 0.05$)

Relative Lift	Statistical Power					
	0.50	0.60	0.70	0.80	0.90	0.95
5%	41,922	51,019	61,632	72,160	86,870	101,178
10%	10,317	12,551	15,166	17,761	21,381	24,904
15%	4,544	5,530	6,683	7,818	9,415	10,965
20%	2,540	3,091	3,735	4,371	5,262	6,129
25%	1,612	1,963	2,372	2,776	3,342	3,893
30%	1,106	1,347	1,628	1,904	2,293	2,671
50%	389	474	573	670	807	940
100%	93	114	138	161	194	226

Table 7: Sample Size for t-Test (Effect Size = Cohen's d)

Cohen's d (Effect Size)	Sample Size per Variant
0.1 (very small)	1,571
0.2 (small)	394
0.3 (small-medium)	176
0.4 (medium)	100
0.5 (medium)	64
0.6 (medium-large)	45
0.7 (large)	34
0.8 (large)	26
0.9 (large)	21
1.0 (very large)	17
1.5 (very large)	8
2.0 (very large)	5

.4 Minimum Detectable Effect (MDE)

.4.1 MDE for Two-Proportion Tests

Minimum detectable effect (absolute) for $\alpha = 0.05$, power = 0.80.

Note: MDE shown as absolute difference in percentage points (e.g., 2.60% means difference from 1% to 3.60%).

.4.2 MDE in Relative Terms

Same data as above, expressed as relative lift from baseline.

Example: With 5,000 users per variant and 10% baseline, you can detect a 25% relative lift (10% $\rightarrow$ 12.5%) with 80% power.

Table 8: Statistical Power vs. Sample Size (Baseline 10%)

n per Variant	Absolute Effect Size ($p_2 - p_1$)							
	0.005	0.01	0.015	0.02	0.025	0.03	0.04	0.05
500	0.06	0.08	0.10	0.13	0.16	0.20	0.29	0.40
1,000	0.07	0.11	0.15	0.20	0.26	0.33	0.48	0.62
2,500	0.10	0.19	0.30	0.41	0.53	0.64	0.81	0.91
5,000	0.15	0.31	0.49	0.65	0.77	0.86	0.96	0.99
7,500	0.19	0.42	0.63	0.79	0.89	0.95	0.99	1.00
10,000	0.24	0.51	0.72	0.86	0.94	0.98	1.00	1.00
15,000	0.33	0.65	0.85	0.94	0.98	0.99	1.00	1.00
20,000	0.42	0.75	0.91	0.97	0.99	1.00	1.00	1.00
25,000	0.50	0.82	0.95	0.99	1.00	1.00	1.00	1.00
50,000	0.75	0.97	1.00	1.00	1.00	1.00	1.00	1.00

Table 9: Power for Fixed Sample Size ($n = 5,000$ per variant, $\alpha = 0.05$)

Relative Lift	Baseline Proportion (p_1)				
	0.05	0.10	0.15	0.20	0.25
5%	0.15	0.14	0.13	0.12	0.11
10%	0.35	0.31	0.29	0.27	0.25
15%	0.58	0.51	0.47	0.44	0.41
20%	0.76	0.68	0.63	0.59	0.55
25%	0.87	0.80	0.75	0.71	0.67
30%	0.93	0.88	0.84	0.80	0.76
40%	0.99	0.96	0.94	0.92	0.89
50%	1.00	0.99	0.98	0.96	0.95

.5 Multiple Testing Corrections

.5.1 Bonferroni Correction

Adjusted significance levels for family-wise error rate control at $\alpha = 0.05$.

Note: Bonferroni is conservative. For $m = 5$ tests, each test must have $p < 0.01$ to be significant at family-wise $\alpha = 0.05$.

.5.2 Šidák Correction

Less conservative alternative to Bonferroni for independent tests.

Note: Šidák provides slightly more power than Bonferroni when tests are independent. Differences are small in practice.

.5.3 Benjamini-Hochberg FDR Thresholds

Critical values for False Discovery Rate control at $q = 0.05$.

Usage: Sort p-values from smallest to largest. Find largest i where $p_{(i)} \leq \frac{i}{m} \cdot q$. Reject all H_0 for $i' \leq i$.

Table 10: Minimum Detectable Effect (Percentage Points, $\alpha = 0.05$, Power = 0.80)

Baseline	Sample Size per Variant						
	500	1,000	2,500	5,000	10,000	25,000	50,000
1%	2.60%	1.83%	1.16%	0.82%	0.58%	0.37%	0.26%
5%	5.68%	4.01%	2.54%	1.79%	1.27%	0.80%	0.57%
10%	7.78%	5.50%	3.48%	2.46%	1.74%	1.10%	0.78%
15%	9.27%	6.56%	4.15%	2.94%	2.08%	1.32%	0.93%
20%	10.42%	7.37%	4.66%	3.30%	2.33%	1.48%	1.04%
25%	11.28%	7.98%	5.05%	3.57%	2.52%	1.60%	1.13%
30%	11.93%	8.44%	5.34%	3.77%	2.67%	1.69%	1.19%
50%	13.02%	9.20%	5.82%	4.12%	2.91%	1.84%	1.30%

Table 11: Minimum Detectable Effect (Relative %, $\alpha = 0.05$, Power = 0.80)

Baseline	Sample Size per Variant						
	500	1,000	2,500	5,000	10,000	25,000	50,000
1%	260%	183%	116%	82%	58%	37%	26%
5%	114%	80%	51%	36%	25%	16%	11%
10%	78%	55%	35%	25%	17%	11%	8%
15%	62%	44%	28%	20%	14%	9%	6%
20%	52%	37%	23%	16%	12%	7%	5%
25%	45%	32%	20%	14%	10%	6%	5%
30%	40%	28%	18%	13%	9%	6%	4%
50%	26%	18%	12%	8%	6%	4%	3%

.5.4 Sample Size Inflation for Multiple Testing

Factor to multiply sample size by when testing multiple metrics.

Example: Testing 5 metrics with Bonferroni requires $1.69\times$ larger sample than testing one metric.

Table 12: Bonferroni Corrected Significance Levels

Number of Tests (m)	Corrected α (α/m)	Critical Value ($z_{\alpha/(2m)}$)	Required p-value
1	0.0500	1.960	< 0.0500
2	0.0250	2.241	< 0.0250
3	0.0167	2.394	< 0.0167
4	0.0125	2.498	< 0.0125
5	0.0100	2.576	< 0.0100
6	0.0083	2.638	< 0.0083
7	0.0071	2.690	< 0.0071
8	0.0063	2.735	< 0.0063
9	0.0056	2.773	< 0.0056
10	0.0050	2.807	< 0.0050
15	0.0033	2.935	< 0.0033
20	0.0025	3.023	< 0.0025
25	0.0020	3.091	< 0.0020
50	0.0010	3.291	< 0.0010
100	0.0005	3.481	< 0.0005

Table 13: Šidák vs. Bonferroni Corrected Alpha

Number of Tests (m)	Bonferroni α/m	Šidák $1 - (1 - \alpha)^{1/m}$	Difference (Šidák - Bonf)
2	0.02500	0.02532	+0.00032
3	0.01667	0.01695	+0.00028
4	0.01250	0.01274	+0.00024
5	0.01000	0.01020	+0.00020
10	0.00500	0.00512	+0.00012
20	0.00250	0.00256	+0.00006
50	0.00100	0.00103	+0.00003
100	0.00050	0.00051	+0.00001

.6 Sequential Testing Boundaries

.6.1 O'Brien-Fleming Boundaries

Critical values for group sequential design with equal information increments.

Note: O'Brien-Fleming has high early boundaries (conservative early stopping) and boundaries approaching 1.96 at final look.

Example: For 3 equally-spaced looks (33%, 67%, 100% information), critical values are 3.471, 2.127, 1.710.

.6.2 Pocock Boundaries

Constant critical value across all interim analyses.

Note: Pocock uses same critical value at each look but inflates boundary (2.413 for

Table 14: Benjamini-Hochberg Critical Values (FDR = 0.05)

Rank (i)	For m=5 tests	For m=10 tests	For m=20 tests
1 (smallest p)	0.0100	0.0050	0.0025
2	0.0200	0.0100	0.0050
3	0.0300	0.0150	0.0075
4	0.0400	0.0200	0.0100
5	0.0500	0.0250	0.0125
6	-	0.0300	0.0150
7	-	0.0350	0.0175
8	-	0.0400	0.0200
9	-	0.0450	0.0225
10	-	0.0500	0.0250
15	-	-	0.0375
20	-	-	0.0500

Table 15: Sample Size Inflation Factor (to maintain 80% power after correction)

Number of Tests (m)	Bonferroni Inflation	Šidák Inflation
2	1.28	1.27
3	1.46	1.44
4	1.59	1.57
5	1.69	1.67
6	1.78	1.75
8	1.91	1.88
10	2.02	1.99
15	2.23	2.19
20	2.38	2.33

5 looks vs. 1.960 for fixed design).

.6.3 Alpha Spending Function Values

Cumulative alpha spent at each look using O'Brien-Fleming spending function.

Interpretation: Very little alpha is spent early (conservative), with most alpha reserved for final analysis.

Table 16: O'Brien-Fleming Boundaries (Overall $\alpha = 0.05$, two-sided)

Information	Maximum Number of Looks (K)				
Fraction	2	3	4	5	10
0.10	-	-	-	-	4.333
0.20	-	-	-	4.333	3.064
0.25	-	-	4.049	3.872	2.734
0.33	-	3.471	3.301	3.159	2.233
0.50	2.797	2.454	2.337	2.240	1.584
0.67	-	2.127	2.024	1.940	1.371
0.75	-	-	1.985	1.903	1.345
0.80	-	-	-	1.875	1.325
1.00	1.977	1.710	1.651	1.586	1.122

Table 17: Pocock Constant Boundaries (Overall $\alpha = 0.05$, two-sided)

Maximum Number of Looks (K)	Constant Critical Value (all looks)
1	1.960
2	2.178
3	2.289
4	2.361
5	2.413
6	2.453
7	2.485
8	2.512
9	2.535
10	2.555
15	2.621
20	2.664

.7 Distribution Parameters and Properties

.7.1 Common Distributions for A/B Testing

Key parameters for distributions used in experiment analysis.

.7.2 Beta Distribution Quantiles

Commonly used Beta priors for Bayesian A/B testing.

Notes:

- (*) Uniform distribution has no unique mode
- (**) Beta(0.5, 0.5) is U-shaped (Jeffreys prior)
- Larger $\alpha + \beta$ means more concentrated (smaller variance)

Table 18: O'Brien-Fleming Alpha Spending (Total $\alpha = 0.05$)

Information	Maximum Looks (K)				
Fraction	2	3	4	5	10
0.10	-	-	-	-	0.00001
0.20	-	-	-	0.00001	0.00013
0.25	-	-	0.00002	0.00004	0.00031
0.33	-	0.00011	0.00023	0.00041	0.00093
0.50	0.00255	0.00680	0.00985	0.01263	0.02028
0.67	-	0.01758	0.02274	0.02735	0.03817
0.75	-	-	0.02814	0.03336	0.04528
0.80	-	-	-	0.03680	0.04912
1.00	0.05000	0.05000	0.05000	0.05000	0.05000

.7.3 Normal Distribution Probabilities

Probabilities for standard normal $Z \sim \mathcal{N}(0, 1)$.

Usage: For 95% CI, use ± 1.96 standard errors. $P(-1.96 < Z < 1.96) = 0.95$.

.7.4 Variance Formulas for Common Metrics

Table 19: Distribution Parameters and Properties

Distribution	Parameters	Mean	Variance	Use Case
Bernoulli	$p \in [0, 1]$	p	$p(1 - p)$	Single conversion
Binomial	n, p	np	$np(1 - p)$	Count of conversions
Normal	μ, σ^2	μ	σ^2	Continuous metrics
Beta	$\alpha, \beta > 0$	$\frac{\alpha}{\alpha + \beta}$	$\frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}$	Prior/posterior for proportions
Gamma	k, θ	$k\theta$	$k\theta^2$	Time-to-event, revenue
Exponential	$\lambda > 0$	$1/\lambda$	$1/\lambda^2$	Time between events
Poisson	$\lambda > 0$	λ	λ	Count of events

.8 Finite Population Corrections

.8.1 Finite Population Correction Factor

When sampling without replacement from finite population of size N .

Usage: Multiply standard error by FPC when sample is large relative to population ($n/N > 0.05$).

Corrected SE: $SE_{corrected} = SE \times \sqrt{\frac{N-n}{N-1}}$

.8.2 Design Effect for Cluster Randomization

Inflation factor when randomizing by clusters instead of individuals.

Required sample size: $n_{cluster} = n_{individual} \times DEFF$

Example: With cluster size 10 and ICC = 0.05, need 1.45× more participants than individual randomization.

Table 20: Beta Distribution Quantiles

Parameters	Mean	Mode	SD	2.5%	50%	97.5%
Beta(1, 1)	0.500	*	0.289	0.025	0.500	0.975
Beta(2, 2)	0.500	0.500	0.224	0.082	0.500	0.918
Beta(5, 5)	0.500	0.500	0.141	0.186	0.500	0.814
Beta(10, 10)	0.500	0.500	0.100	0.274	0.500	0.726
Beta(1, 9)	0.100	0.000	0.091	0.003	0.071	0.352
Beta(10, 90)	0.100	0.099	0.030	0.050	0.098	0.165
Beta(50, 450)	0.100	0.099	0.013	0.071	0.099	0.133
Beta(100, 900)	0.100	0.100	0.009	0.082	0.100	0.119
Beta(0.5, 0.5)	0.500	**	0.354	0.001	0.500	0.999
Beta(2, 8)	0.200	0.125	0.121	0.033	0.178	0.475

Table 21: Standard Normal Cumulative Probabilities

z	P(Z ≤ z)	z	P(Z ≤ z)	z	P(Z ≤ z)
-3.0	0.0013	0.0	0.5000	2.0	0.9772
-2.5	0.0062	0.5	0.6915	2.5	0.9938
-2.0	0.0228	1.0	0.8413	3.0	0.9987
-1.96	0.0250	1.28	0.9000	3.29	0.9995
-1.64	0.0505	1.64	0.9495	3.5	0.9998
-1.5	0.0668	1.96	0.9750	4.0	1.0000
-1.0	0.1587				
-0.5	0.3085				

.9 Effect Size Benchmarks

.9.1 Cohen’s d (Standardized Mean Difference)

Interpretation guidelines for continuous metrics.

.9.2 Cohen’s h (Standardized Proportion Difference)

Effect size for comparing two proportions.

Interpretation: $h = 0.2$ is small, $h = 0.5$ is medium, $h = 0.8$ is large.

.9.3 Practical Significance Thresholds

Industry-typical minimum meaningful differences.

Note: These are guidelines. Actual thresholds depend on business context, costs, and strategic importance.

Table 22: Variance Formulas for A/B Test Metrics

Metric	Variance Formula	Notes
Proportion $\hat{p}$	$\frac{p(1-p)}{n}$	Binary outcome
Mean $\bar{X}$	$\frac{\sigma^2}{n}$	Continuous outcome
Difference in proportions	$\frac{p_1(1-p_1)}{n_1} + \frac{p_2(1-p_2)}{n_2}$	Two independent samples
Difference in means	$\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}$	Unequal variances
Relative lift	$\frac{1}{p_1^2} \left[\frac{p_2(1-p_2)}{n_2} + \frac{p_2^2 p_1(1-p_1)}{n_1} \right]$	Delta method approximation
Ratio estimator	$\frac{1}{\mu_Y^2} \left[\frac{\sigma_X^2}{n} + \frac{\mu_X^2 \sigma_Y^2}{\mu_Y^2 n} - \frac{2\mu_X \sigma_{XY}}{\mu_Y n} \right]$	Ratio of means

Table 23: Finite Population Correction: $\sqrt{\frac{N-n}{N-1}}$

Population (N)	Sample Size (n)					
	100	500	1,000	5,000	10,000	50,000
1,000	0.949	0.707	-	-	-	-
5,000	0.990	0.975	0.949	-	-	-
10,000	0.995	0.987	0.975	0.707	-	-
50,000	0.999	0.997	0.995	0.949	0.894	-
100,000	1.000	0.999	0.997	0.975	0.949	0.707
500,000	1.000	1.000	0.999	0.990	0.980	0.900
∞	1.000	1.000	1.000	1.000	1.000	1.000

.10 Quick Reference Guide

.10.1 Test Selection Decision Tree

.10.2 Common Pitfalls and Corrections

.10.3 Recommended Parameters by Scenario

.11 Summary

This appendix provides:

- **Critical values:** z, t, χ^2 , F distributions for hypothesis testing
- **Sample size tables:** For proportions and means at various power levels
- **Power tables:** Power as function of sample size and effect size
- **MDE tables:** Minimum detectable effects for given sample sizes

Table 24: Design Effect: $DEFF = 1 + (m - 1)\rho$

Cluster Size (m)	Intraclass Correlation (ρ)					
	0.001	0.01	0.05	0.10	0.20	0.50
2	1.001	1.01	1.05	1.10	1.20	1.50
5	1.004	1.04	1.20	1.40	1.80	3.00
10	1.009	1.09	1.45	1.90	2.80	5.50
20	1.019	1.19	1.95	2.90	4.80	10.50
50	1.049	1.49	3.45	5.90	10.80	25.50
100	1.099	1.99	5.95	10.90	20.80	50.50
200	1.199	2.99	10.95	20.90	40.80	100.50

Table 25: Cohen's d Effect Size Classification

Cohen's d	Classification	n for 80% power	Example (Session Duration)
0.01	Tiny	157,098	Mean 120s, SD 30s: 0.3s difference
0.10	Very small	1,571	Mean 120s, SD 30s: 3s difference
0.20	Small	394	Mean 120s, SD 30s: 6s difference
0.30	Small-medium	176	Mean 120s, SD 30s: 9s difference
0.40	Medium	100	Mean 120s, SD 30s: 12s difference
0.50	Medium	64	Mean 120s, SD 30s: 15s difference
0.60	Medium-large	45	Mean 120s, SD 30s: 18s difference
0.70	Large	34	Mean 120s, SD 30s: 21s difference
0.80	Large	26	Mean 120s, SD 30s: 24s difference
1.00	Very large	17	Mean 120s, SD 30s: 30s difference

- **Multiple testing corrections:** Bonferroni, Šidák, Benjamini-Hochberg
- **Sequential boundaries:** O'Brien-Fleming and Pocock critical values
- **Distribution parameters:** Beta, Normal, and other common distributions
- **Effect size benchmarks:** Cohen's d and h with interpretations
- **Design effects:** Corrections for finite populations and clustering
- **Quick reference:** Test selection and common parameter choices

All tables assume two-sided tests at $\alpha = 0.05$ and power = 0.80 unless otherwise specified.

Table 26: Cohen’s h: $h = 2[\arcsin(\sqrt{p_2}) - \arcsin(\sqrt{p_1})]$

p_1	p_2	Absolute Diff	Relative Lift	Cohen’s h
0.10	0.11	0.01	10%	0.067
0.10	0.12	0.02	20%	0.133
0.10	0.15	0.05	50%	0.327
0.10	0.20	0.10	100%	0.643
0.20	0.22	0.02	10%	0.101
0.20	0.24	0.04	20%	0.200
0.20	0.30	0.10	50%	0.485
0.20	0.40	0.20	100%	0.927
0.50	0.55	0.05	10%	0.200
0.50	0.60	0.10	20%	0.400
0.50	0.75	0.25	50%	1.000

Table 27: Practical Significance Guidelines by Industry

Metric	Typical Baseline	Min Lift	Meaningful
E-commerce conversion	2-5%	5-10% relative	
SaaS signup conversion	5-15%	10-20% relative	
Email click-through	2-5%	10-15% relative	
Landing page conversion	1-10%	15-25% relative	
Add-to-cart rate	10-30%	5-10% relative	
Session duration	2-10 minutes	10-20% relative	
Pages per session	2-5 pages	10-15% relative	
Mobile app retention (D7)	20-40%	5-10% relative	
Revenue per user	\$1-100	5-15% relative	

Table 28: Statistical Test Selection Guide

Scenario	Recommended Test	Requirements/Notes
Binary outcome, large n	Two-proportion z-test	$n_1 p_1, n_1(1 - p_1) \geq 5$
Binary outcome, small n	Fisher's exact test	Exact, no approximation
Continuous, normal	Two-sample t-test	Check normality
Continuous, non-normal	Mann-Whitney U test	Robust to outliers
Continuous, paired	Paired t-test	Same subjects
Multiple groups	ANOVA / Kruskal-Wallis	3+ variants
Count data	Poisson or negative binomial	Events per user
Time-to-event	Log-rank test / Cox model	Censored data

Table 29: Common Issues in A/B Testing

Issue	Detection	Solution
Sample ratio mismatch	χ^2 test, $p < 0.01$	Investigate randomization
Peeking / optional stopping	Multiple looks at data	Sequential design
Multiple testing	Testing > 1 metric	Bonferroni / FDR control
Novelty effect	Temporal instability	Longer duration, cohort analysis
Carryover effect	User exposed to both	Proper washout period
Clustered data	High within-cluster correlation	Cluster-robust SE
Outliers	Extreme values	Winsorize, robust statistics
Non-independence	Network effects	Cluster randomization

Table 30: Standard Parameter Choices

Scenario	Parameter	Standard Value
Significance level	α	0.05 (two-sided)
Statistical power	$1 - \beta$	0.80 (0.90 for critical tests)
SRM detection threshold	α_{SRM}	0.01 (more conservative)
Multiple testing (few)	Correction	Bonferroni
Multiple testing (many)	Correction	Benjamini-Hochberg (FDR = 0.05)
Sequential testing	Boundary	O'Brien-Fleming
Small sample ($n < 30$)	Test	Exact or permutation
Large effect expected	Power	Can use 0.70-0.75

Notation and Symbols

.12 Introduction

This appendix provides a comprehensive reference for the mathematical notation and symbols used throughout this book. Notation is organized by category for easy reference.

.13 General Mathematical Notation

Symbol	Meaning
$\mathbb{N}$	Natural numbers: $\{0, 1, 2, 3, \dots\}$
$\mathbb{Z}$	Integers: $\{\dots, -2, -1, 0, 1, 2, \dots\}$
$\mathbb{R}$	Real numbers
$\mathbb{R}^+$	Positive real numbers
$[a, b]$	Closed interval from a to b (inclusive)
(a, b)	Open interval from a to b (exclusive)
$\lfloor x \rfloor$	Floor function (largest integer $\leq x$)
$\lceil x \rceil$	Ceiling function (smallest integer $\geq x$)
$ x $	Absolute value of x
$\log(x)$	Natural logarithm (base e)
$\ln(x)$	Natural logarithm (alternative notation)
$\exp(x)$	Exponential function, e^x
$x!$	Factorial of x
$\binom{n}{k}$	Binomial coefficient: $\frac{n!}{k!(n-k)!}$
$\sum_{i=1}^n$	Summation from $i = 1$ to n
$\prod_{i=1}^n$	Product from $i = 1$ to n
$\approx$	Approximately equal to
$\propto$	Proportional to
$\sim$	Distributed as
$\stackrel{d}{=}$	Equal in distribution
$\xrightarrow{p}$	Converges in probability
$\xrightarrow{d}$	Converges in distribution
$\mathbb{I}\{\cdot\}$	Indicator function (1 if true, 0 if false)

.14 Probability and Statistics

.14.1 Random Variables and Distributions

Symbol	Meaning
X, Y, Z	Random variables (uppercase)
x, y, z	Realizations or values (lowercase)
$\mathbf{X}$	Random vector
X_i	The i -th random variable or observation
$\bar{X}$	Sample mean
$\tilde{X}$	Sample median
$P(A)$	Probability of event A
$P(A B)$	Conditional probability of A given B
$P(A \cap B)$	Probability of A and B (intersection)
$P(A \cup B)$	Probability of A or B (union)
$f(x)$	Probability density function (PDF)
$f(x \theta)$	PDF parameterized by θ
$F(x)$	Cumulative distribution function (CDF)
$\Phi(x)$	CDF of standard normal distribution
$\phi(x)$	PDF of standard normal distribution
$\mathbb{E}[X]$	Expected value (expectation) of X
μ	Population mean
σ^2	Population variance
σ	Population standard deviation
$\text{Var}(X)$	Variance of X
$\text{Cov}(X, Y)$	Covariance of X and Y
$\rho_{X,Y}$	Correlation coefficient between X and Y
$\text{Corr}(X, Y)$	Correlation between X and Y
s^2	Sample variance
s	Sample standard deviation
SE	Standard error
MSE	Mean squared error
RMSE	Root mean squared error

.14.2 Common Distributions

Symbol	Meaning
$X \sim \mathcal{N}(\mu, \sigma^2)$	X follows normal distribution with mean μ and variance σ^2
$Z \sim \mathcal{N}(0, 1)$	Standard normal distribution
$X \sim \text{Bernoulli}(p)$	Bernoulli distribution with success probability p
$X \sim \text{Binomial}(n, p)$	Binomial distribution: n trials, probability p
$X \sim \text{Poisson}(\lambda)$	Poisson distribution with rate λ
$X \sim \text{Uniform}(a, b)$	Uniform distribution on $[a, b]$

Continued on next page

Table 33 – *Continued from previous page*

Symbol	Meaning
$X \sim \text{Exp}(\lambda)$	Exponential distribution with rate λ
$X \sim t_\nu$	Student's t-distribution with ν degrees of freedom
$X \sim \chi_\nu^2$	Chi-squared distribution with ν degrees of freedom
$X \sim F_{\nu_1, \nu_2}$	F-distribution with ν_1 and ν_2 degrees of freedom
$X \sim \text{Beta}(\alpha, \beta)$	Beta distribution with shape parameters α, β
$X \sim \text{Gamma}(\alpha, \beta)$	Gamma distribution with shape α and rate β

.15 Hypothesis Testing

Symbol	Meaning
H_0	Null hypothesis
H_1 or H_a	Alternative hypothesis
α	Significance level (Type I error rate)
β	Type II error rate
$1 - \beta$	Statistical power
p -value	Probability of observing data as extreme under H_0
z_α	Critical value from standard normal at significance α
$t_{\alpha, \nu}$	Critical value from t-distribution at significance α with ν df
$\chi_{\alpha, \nu}^2$	Critical value from chi-squared at significance α with ν df
CI	Confidence interval
$\text{CI}_{1-\alpha}$	$(1 - \alpha) \times 100\%$ confidence interval
δ	Effect size
Δ	Treatment effect (difference)
θ	Generic parameter
$\hat{\theta}$	Estimator of parameter θ
θ_0	Null hypothesis value of θ
$\text{SE}(\hat{\theta})$	Standard error of estimator $\hat{\theta}$

.16 Experimental Design

.16.1 Treatment and Assignment

Symbol	Meaning
T	Treatment indicator ($T = 1$ treatment, $T = 0$ control)
Z_i	Treatment assignment for unit i
Y_i	Observed outcome for unit i
$Y_i(1)$	Potential outcome under treatment for unit i

Continued on next page

Table 35 – *Continued from previous page*

Symbol	Meaning
$Y_i(0)$	Potential outcome under control for unit i
τ_i	Individual treatment effect: $Y_i(1) - Y_i(0)$
ATE	Average Treatment Effect
τ	Average treatment effect (ATE): $\mathbb{E}[Y(1) - Y(0)]$
ATT	Average Treatment Effect on the Treated
CATE	Conditional Average Treatment Effect
n	Sample size (total)
n_0	Sample size in control group
n_1	Sample size in treatment group
k	Number of treatment groups or variants
π	Allocation ratio or treatment probability

.16.2 Sample Size and Power

Symbol	Meaning
n	Required sample size per group
N	Total sample size
α	Significance level
β	Type II error rate
$1 - \beta$	Power
δ	Effect size (Cohen's d , etc.)
MDE	Minimum Detectable Effect
σ	Population standard deviation
s	Sample standard deviation
σ_{pooled}	Pooled standard deviation
κ	Allocation ratio (n_1/n_0)
$z_{\alpha/2}$	Two-tailed critical value
z_β	Critical value for power

.17 A/B Testing Specific Notation

.17.1 Conversion and Proportions

Symbol	Meaning
p	Proportion or conversion rate
p_0	Control group proportion
p_1	Treatment group proportion
$\hat{p}$	Estimated proportion
$\hat{p}_0$	Estimated control proportion
$\hat{p}_1$	Estimated treatment proportion

Continued on next page

Table 37 – *Continued from previous page*

Symbol	Meaning
$\bar{p}$	Pooled proportion: $\frac{x_0+x_1}{n_0+n_1}$
x	Number of successes (conversions)
CR	Conversion rate
CTR	Click-through rate
CVR	Conversion rate (alternative notation)

.17.2 Metrics and Performance

Symbol	Meaning
RPU	Revenue Per User
ARPU	Average Revenue Per User
LTV	Lifetime Value
AOV	Average Order Value
μ_0	Mean metric value in control
μ_1	Mean metric value in treatment
$\bar{y}_0$	Sample mean in control
$\bar{y}_1$	Sample mean in treatment
Lift	Relative improvement: $\frac{p_1-p_0}{p_0}$ or $\frac{\mu_1-\mu_0}{\mu_0}$
ϵ	Relative lift (alternative notation)

.18 Multiple Testing

Symbol	Meaning
m	Number of hypotheses tested
FWER	Family-Wise Error Rate
FDR	False Discovery Rate
α_{adj}	Adjusted significance level
α^*	Bonferroni-corrected level: α/m
V	Number of false discoveries (Type I errors)
R	Number of rejections
S	Number of true discoveries
Q	False Discovery Proportion: $V/\max(R, 1)$
q	FDR control level
$p_{(1)}, \dots, p_{(m)}$	Ordered p-values

.19 Bayesian Methods

Symbol	Meaning
θ	Parameter of interest
$\pi(\theta)$	Prior distribution
$\pi(\theta \mid \mathcal{D})$	Posterior distribution given data $\mathcal{D}$
$\mathcal{L}(\theta \mid \mathcal{D})$	Likelihood function
$p(\mathcal{D} \mid \theta)$	Likelihood (alternative notation)
α, β	Beta distribution shape parameters
α_0, β_0	Prior parameters for Beta distribution
α_n, β_n	Posterior parameters for Beta distribution
HPD	Highest Posterior Density (credible interval)
$P(\theta > c \mid \mathcal{D})$	Posterior probability
BF	Bayes Factor
B_{10}	Bayes Factor for H_1 vs. H_0
MAP	Maximum A Posteriori estimate

.20 Sequential Testing

Symbol	Meaning
K	Maximum number of looks or interim analyses
t_k	Information time at analysis k
$\alpha(t)$	Alpha spending function
α_k	Alpha spent at look k
c_k	Critical value at look k
OBF	O'Brien-Fleming boundary
Z_k	Test statistic at analysis k
I_k	Information fraction at look k

.21 Variance Reduction

Symbol	Meaning
CUPED	Controlled-experiment Using Pre-Experiment Data
Y_{pre}	Pre-experiment metric value
Y_{post}	Post-experiment metric value
$\tilde{Y}$	CUPED-adjusted outcome
θ	CUPED coefficient
ρ	Correlation between pre and post metrics
VR	Variance Reduction
σ_{adj}^2	Adjusted variance after variance reduction

.22 Network Effects and Interference

Symbol	Meaning
SUTVA	Stable Unit Treatment Value Assumption
$Y_i(\mathbf{Z})$	Potential outcome for unit i under treatment vector $\mathbf{Z}$
$\mathcal{N}_i$	Neighborhood or network of unit i
ICC	Intraclass Correlation Coefficient
ρ_I	Intraclass correlation
DE	Design Effect
m	Cluster size

.23 Causal Inference

Symbol	Meaning
$\text{do}(X = x)$	Intervention operator (do-calculus)
$Y(t)$	Potential outcome under treatment t
ITT	Intention To Treat
LATE	Local Average Treatment Effect
IV	Instrumental Variable
RDD	Regression Discontinuity Design
DID	Difference-in-Differences
$e(X)$	Propensity score: $P(T = 1 \mid X)$

.24 Bandit Algorithms

Symbol	Meaning
K	Number of arms (options/variants)
μ_k	True mean reward of arm k
$\hat{\mu}_k$	Estimated mean reward of arm k
$N_k(t)$	Number of times arm k pulled by time t
R_t	Regret at time t
UCB	Upper Confidence Bound
ϵ	Exploration rate in ϵ -greedy
TS	Thompson Sampling
EXP3	Exponential-weight algorithm for Exploration and Exploitation

.25 Subscripts and Superscripts

.25.1 Common Subscripts

Notation	Meaning
x_i	The i -th observation or value
x_0	Control group value
x_1	Treatment group value
x_k	Value for group/variant k
μ_{MLE}	Maximum likelihood estimate
σ_{pooled}	Pooled standard deviation
α_{adj}	Adjusted significance level
p_{value}	p-value

.25.2 Common Superscripts

Notation	Meaning
$x^{(i)}$	The i -th sample or iteration
$\mathbf{X}^T$	Transpose of matrix $\mathbf{X}$
$\mathbf{X}^{-1}$	Inverse of matrix $\mathbf{X}$
X^2	Square of X or chi-squared statistic
X^*	Optimal value or bootstrap sample

.26 Operators and Functions

Symbol	Meaning
$\mathbb{E}[\cdot]$	Expectation operator
$\text{Var}(\cdot)$	Variance operator
$\text{Cov}(\cdot, \cdot)$	Covariance operator
$\text{Corr}(\cdot, \cdot)$	Correlation operator
$\text{Bias}(\cdot)$	Bias of an estimator
$\mathbb{P}(\cdot)$	Probability measure
$\arg \max$	Argument that maximizes
$\arg \min$	Argument that minimizes
$\sup$	Supremum (least upper bound)
$\inf$	Infimum (greatest lower bound)
$\lim$	Limit
∇	Gradient operator

.27 Usage Notes

.27.1 Notation Conventions

Throughout this book, we follow these conventions:

- **Random variables** are denoted by uppercase letters (X, Y, Z)

- **Realizations/values** are denoted by lowercase letters (x, y, z)
- **Parameters** are denoted by Greek letters (θ, μ, σ)
- **Estimators** are denoted with hats $(\hat{\theta}, \hat{\mu}, \hat{p})$
- **Vectors** are denoted by bold lowercase letters $(\mathbf{x}, \mathbf{y})$
- **Matrices** are denoted by bold uppercase letters $(\mathbf{X}, \mathbf{Y})$
- **Sets** are denoted by calligraphic letters $(\mathcal{D}, \mathcal{N})$
- **Subscript 0** typically denotes control group
- **Subscript 1** typically denotes treatment group
- **Subscript i** indexes individuals or observations
- **Subscript k** indexes groups, variants, or arms

.27.2 Distribution Notation

When we write $X \sim \mathcal{D}(\theta)$, we mean "X is distributed according to distribution $\mathcal{D}$ with parameter(s) θ ."

Common examples:

- $X \sim \mathcal{N}(0, 1)$: X follows a standard normal distribution
- $Y \sim \text{Binomial}(n, p)$: Y follows a binomial distribution
- $Z \sim \text{Beta}(\alpha, \beta)$: Z follows a beta distribution

.27.3 Asymptotic Notation

- $\xrightarrow{p}$: Convergence in probability
- $\xrightarrow{d}$: Convergence in distribution
- $\xrightarrow{a.s.}$: Almost sure convergence
- $O(n)$: Big-O notation (order of magnitude)
- $o(n)$: Little-o notation (grows slower than)

.27.4 Context-Dependent Notation

Some symbols have different meanings in different contexts:

- p can denote: probability, proportion, p-value, or number of parameters (context-dependent)
- n typically denotes sample size but may refer to number of trials in binomial context
- α denotes significance level in hypothesis testing, but shape parameter in Beta/Gamma distributions

- β denotes Type II error rate in hypothesis testing, but shape parameter in Beta distribution or regression coefficient in regression context

Always refer to the surrounding text for clarification when ambiguity exists.

Bibliography

- George E. P. Box, J. Stuart Hunter, and William G. Hunter. *Statistics for Experimenters: Design, Innovation, and Discovery*. Wiley, Hoboken, NJ, 2nd edition, 2005.
- George Casella and Roger L. Berger. *Statistical Inference*. Duxbury Press, Pacific Grove, CA, 2nd edition, 2002.
- Stephane Champely. *pwr: Basic Functions for Power Analysis*, 2020. URL <https://CRAN.R-project.org/package=pwr>. R package version 1.3-0.
- Shein-Chung Chow, Jun Shao, and Hansheng Wang. *Sample Size Calculations in Clinical Research*. Chapman & Hall/CRC, Boca Raton, FL, 2nd edition, 2007.
- Brian Christian and Tom Griffiths. *Algorithms to Live By: The Computer Science of Human Decisions*. Henry Holt and Company, New York, 2016.
- Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988.
- Alex Deng, Ulf Knoblich, and Jiannan Lu. Data-driven metric development for online controlled experiments. *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 77–86, 2017.
- Dean Eckles, Brian Karrer, and Johan Ugander. Design and analysis of experiments in networks: Reducing bias from interference. *Journal of Causal Inference*, 5(1), 2016.
- Ronald A. Fisher. *The Design of Experiments*. Oliver and Boyd, Edinburgh, 1935.
- Andrew Gelman and Jennifer Hill. *Data Analysis Using Regression and Multilevel/Hierarchical Models*. Cambridge University Press, Cambridge, 2006.
- Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. *Bayesian Data Analysis*. CRC Press, Boca Raton, FL, 3rd edition, 2013.
- Phillip I. Good. *Permutation Tests: A Practical Guide to Resampling Methods for Testing Hypotheses*. Springer, New York, 2nd edition, 2013.
- Frank E. Harrell. *Regression Modeling Strategies: With Applications to Linear Models, Logistic and Ordinal Regression, and Survival Analysis*. Springer, Cham, Switzerland, 2nd edition, 2015.
- Richard J. Hayes and Lawrence H. Moulton. *Cluster Randomised Trials*. CRC Press, Boca Raton, FL, 2nd edition, 2017.

- Guido W. Imbens and Donald B. Rubin. *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press, Cambridge, 2015.
- Ron Kohavi, Thomas Crook, and Roger Longbotham. Online experimentation at microsoft. *Data Mining Case Studies*, 11:1–11, 2009.
- Ron Kohavi, Diane Tang, and Ya Xu. *Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing*. Cambridge University Press, Cambridge, 2020.
- Adam D. I. Kramer, Jamie E. Guillory, and Jeffrey T. Hancock. Experimental evidence of massive-scale emotional contagion through social networks. *Proceedings of the National Academy of Sciences*, 111(24):8788–8790, 2014.
- James Lind. *A Treatise of the Scurvy*. A. Millar, London, 1753.
- Robinson Meyer. Everything we know about facebook’s secret mood manipulation experiment. *The Atlantic*, June 2014.
- John A. Rice. *Mathematical Statistics and Data Analysis*. Duxbury Press, Belmont, CA, 3rd edition, 2006.
- John W. Tukey. *Exploratory Data Analysis*. Addison-Wesley, Reading, MA, 1977.
- Larry Wasserman. *All of Statistics: A Concise Course in Statistical Inference*. Springer, New York, 2004.
- Ronald L. Wasserstein and Nicole A. Lazar. The asa statement on p-values: Context, process, and purpose. *The American Statistician*, 70(2):129–133, 2016.